
PyCypher

Release 0.0.19

PyCypher Contributors

Apr 11, 2026

Contents:

1	Zero to Hello World	2
1.1	Prerequisites	2
1.2	Level 1: Your First Query	2
1.3	Level 2: Filtering with WHERE	3
1.4	Level 3: Multiple Entity Types	3
1.5	Level 4: Relationships	4
1.6	Level 5: Aggregation	5
1.7	Runnable Example	5
1.8	Next Steps	5
2	Getting Started	6
2.1	Installation	6
2.2	Quick Start: Run Your First Query	6
2.3	Using ContextBuilder (Recommended)	7
2.4	One-Line Context Builder	8
2.5	Graph Queries: MATCH with Relationships	8
2.6	WHERE Label Predicates	8
2.7	Variable-Length Paths	9
2.8	shortestPath and allShortestPaths	9
2.9	OPTIONAL MATCH — Left-Join Semantics	10
2.10	WITH + UNWIND: List Explosion	10
2.11	Aggregation	11
2.12	WITH * — Pass-Through Projection	11
2.13	UNION — Combining Query Results	12
2.14	Mutating the Graph — CREATE, MERGE, DELETE, REMOVE	12
2.15	FOREACH — Iterate Over a List	13
2.16	Functional Expressions	14
2.17	Scalar Functions	16
2.18	Running Tests	19
2.19	Next Steps	19
3	Tutorials	20
3.1	Cypher for SQL Users	20
3.2	Basic Query Execution	27
3.3	Graph Modeling	30
3.4	Pattern Matching	35

3.5	Query Validation	43
3.6	AST Manipulation	47
3.7	Data ETL Pipeline	52
3.8	Integration Guide	57
3.9	Production Deployment Patterns	64
3.10	ML-Powered Query Optimization	68
3.11	Troubleshooting Queries	73
3.12	Learning Pathway	78
3.13	Prerequisites	79
3.14	Related Resources	79
4	API Reference	81
4.1	PyCypher API	81
4.2	Shared API	358
4.3	Package Overview	378
5	User Guide	380
5.1	Data Model Requirements	380
5.2	Variables in PyCypher	389
5.3	AST Nodes	391
5.4	Query Processing	396
5.5	Error Handling Patterns	411
5.6	Configuration Reference	416
5.7	Performance Tuning	418
5.8	Constants and Magic Values Reference	425
5.9	Overview	429
6	Deployment & Scaling	431
6.1	Security Hardening	431
6.2	Docker Containerisation	438
6.3	Environment Configuration	441
6.4	Scaling & Performance	444
6.5	Monitoring & Operations	447
6.6	Troubleshooting	452
6.7	Architecture Overview	455
7	Developer Guide	457
7.1	Architecture	457
7.2	Architecture Decision Records	463
7.3	Contributing	475
7.4	Testing	480
7.5	Release Process	488
7.6	Security Hardening & Compliance	490
7.7	Threat Model & Trust Boundaries	494
7.8	New Contributors	500
7.9	Overview	501
8	Architecture Decision Records	502
8.1	ADR-001: Use Lark with Earley Algorithm for Cypher Parsing	502
8.2	ADR-002: BindingFrame as Intermediate Representation	503
8.3	ADR-003: ID-Only Column Preservation Strategy	504
8.4	ADR-004: Shadow-Write Mutation Pattern for Atomicity	505
8.5	ADR-005: Backend Engine Protocol for Pluggable DataFrames	506
8.6	ADR-006: Star Class Decomposition into Specialised Modules	508
8.7	ADR-007: Extract Cardinality Estimator from Query Planner	509

9 Overview	511
10 Key Features	512
11 Quick Start	513
12 Examples	514
13 Indices and tables	515
Python Module Index	516
Index	518

PyCypher is a Cypher query engine that executes openCypher queries against ordinary pandas DataFrames. It is built on a Lark-based parser, a Pydantic AST, and a BindingFrame execution layer — no graph database required.

Get a working Cypher query in under 5 minutes. No external files, no databases — just Python and PyCypher.

On this page

- Prerequisites
- Level 1: Your First Query
- Level 2: Filtering with WHERE
- Level 3: Multiple Entity Types
- Level 4: Relationships
- Level 5: Aggregation
- Runnable Example
- Next Steps

1.1 Prerequisites

- Python 3.14+
- PyCypher installed (see [Getting Started](#) for installation)

1.2 Level 1: Your First Query

Three lines of setup, one query, instant results:

```
import pandas as pd
from pycypher import ContextBuilder, Star

# Create in-memory data
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
```

(continues on next page)

(continued from previous page)

```

    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})

# Build context and execute
context = ContextBuilder.from_dict({"Person": people})
star = Star(context=context)

result = star.execute_query(
    "MATCH (p:Person) RETURN p.name AS name, p.age AS age"
)
print(result)

```

Output:

```

name age
Alice 30
Bob   25
Carol 35

```

What just happened:

1. ContextBuilder.from_dict() wrapped your DataFrame as a Person entity table.
2. Star is the query engine — call execute_query() with any Cypher string.
3. The result is a standard pandas.DataFrame.

The `__ID__` column is required — it uniquely identifies each row (like a primary key).

1.3 Level 2: Filtering with WHERE

Add a WHERE clause to filter results:

```

result = star.execute_query(
    "MATCH (p:Person) WHERE p.age >= 30 RETURN p.name AS name, p.age AS age"
)

```

Output:

```

name age
Alice 30
Carol 35

```

Any comparison operator works: `=`, `<>`, `<`, `>`, `<=`, `>=`. Combine with AND, OR, NOT.

1.4 Level 3: Multiple Entity Types

Add more DataFrames to query across different entity types:

```

products = pd.DataFrame({
    "__ID__": [10, 20, 30],
    "title": ["Widget", "Gadget", "Gizmo"],

```

(continues on next page)

(continued from previous page)

```

    "price": [9.99, 49.99, 24.99],
})

context = ContextBuilder.from_dict({
    "Person": people,
    "Product": products,
})
star = Star(context=context)

result = star.execute_query(
    "MATCH (p:Product) WHERE p.price < 30 "
    "RETURN p.title AS product, p.price AS price ORDER BY p.price ASC"
)

```

Output:

```

product  price
Widget   9.99
Gizmo    24.99

```

1.5 Level 4: Relationships

Connect entities with relationships. A relationship DataFrame needs three special columns: `__ID__`, `__SOURCE__` (from-entity ID), and `__TARGET__` (to-entity ID):

```

purchases = pd.DataFrame({
    "__ID__": [100, 101, 102],
    "__SOURCE__": [1, 2, 1], # Person IDs
    "__TARGET__": [10, 20, 30], # Product IDs
    "date": ["2024-01", "2024-02", "2024-03"],
})

context = (
    ContextBuilder()
    .add_entity("Person", people)
    .add_entity("Product", products)
    .add_relationship(
        "BOUGHT", purchases,
        source_col="__SOURCE__", target_col="__TARGET__",
    )
    .build()
)
star = Star(context=context)

result = star.execute_query(
    "MATCH (person:Person)-[:BOUGHT]->(item:Product) "
    "RETURN person.name AS buyer, item.title AS product "
    "ORDER BY buyer, product"
)

```

Output:


```

buyer product
Alice  Gizmo
Alice  Widget
Bob    Gadget

```

The `-[:BOUGHT]->` arrow in the MATCH pattern follows the relationship from source to target.

1.6 Level 5: Aggregation

Use `count()`, `collect()`, `sum()`, `avg()`, and other aggregation functions:

```

result = star.execute_query(
    "MATCH (person:Person)-[:BOUGHT]->(item:Product) "
    "RETURN person.name AS buyer, count(item) AS items_bought, "
    "       collect(item.title) AS products "
    "ORDER BY items_bought DESC"
)

```

Output:

```

buyer  items_bought  products
Alice      2  [Widget, Gizmo]
Bob        1    [Gadget]

```

1.7 Runnable Example

All the code above is available as a single runnable file:

```
uv run python examples/hello_world.py
```

1.8 Next Steps

- [Getting Started](#) — Full getting started guide with all Cypher features
- `examples/quickstart.py` — Extended example with error handling
- `examples/social_network/run_demo.py` — Graph traversal patterns

2.1 Installation

PyCypher requires Python 3.14+ and uses uv for dependency management. It is not yet published on PyPI — install from source:

```
# Install uv (if not already installed)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Clone and install
git clone https://github.com/zacernst/pycypher.git
cd pycypher
uv sync

# Verify
uv run python -c "from pycypher import Star; print('OK')"
```

2.2 Quick Start: Run Your First Query

The fastest path to results — load a pandas DataFrame and execute Cypher:

```
import pandas as pd
from pycypher import Star, Context, EntityTable, ID_COLUMN

# 1. Bring your data as a DataFrame
df = pd.DataFrame({
    "ID": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})

# 2. Wrap it in an EntityTable
table = EntityTable.from_dataframe("Person", df)

# 3. Build a Context and a Star executor
from pycypher.relational_models import EntityMapping, RelationshipMapping
```

(continues on next page)

(continued from previous page)

```

context = Context(
    entity_mapping=EntityMapping(mapping={"Person": table}),
    relationship_mapping=RelationshipMapping(mapping={}),
)
star = Star(context=context)

# 4. Execute Cypher — returns a pandas DataFrame
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 25 RETURN p.name AS name, p.age AS age"
)
print(result)
#   name age
# 0  Alice  30
# 1  Carol  35

```

2.3 Using ContextBuilder (Recommended)

ContextBuilder is the friendliest API for building a Context from files or DataFrames:

```

import pandas as pd
from pycypher import Star
from pycypher.ingestion import ContextBuilder

people = pd.DataFrame({
    "user_id": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})

context = (
    ContextBuilder()
    .add_entity("Person", people, id_col="user_id")
    .build()
)

star = Star(context=context)
result = star.execute_query(
    "MATCH (p:Person) RETURN p.name AS name ORDER BY p.age ASC"
)

```

You can also load directly from files:

```

context = (
    ContextBuilder()
    .add_entity("Person", "data/people.csv")
    .add_entity("Product", "data/products.parquet")
    .add_relationship(
        "BOUGHT",
        "data/purchases.csv",
        source_col="user_id",
        target_col="product_id",
    )
)

```

(continues on next page)

(continued from previous page)

```
)
    .build()
)
```

2.4 One-Line Context Builder

When all your data is already in memory as DataFrames, use `from_dict()` to build a context in a single call:

```
import pandas as pd
from pycypher.ingestion import ContextBuilder
from pycypher import Star

people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})
products = pd.DataFrame({
    "__ID__": [10, 20],
    "title": ["Widget", "Gadget"],
    "price": [9.99, 49.99],
})

# One call — no chaining required
context = ContextBuilder.from_dict({"Person": people, "Product": products})
star = Star(context=context)

result = star.execute_query("MATCH (p:Person) RETURN p.name AS name")
```

2.5 Graph Queries: MATCH with Relationships

```
result = star.execute_query(
    """
    MATCH (buyer:Person)-[:BOUGHT]->(item:Product)
    WHERE buyer.age >= 30
    RETURN buyer.name AS buyer, item.name AS product
    ORDER BY buyer.name
    """
)
```

2.6 WHERE Label Predicates

WHERE `n:Label` tests whether a node variable belongs to a specific entity type at query time. This is useful when MATCH scans multiple entity types with an unlabeled pattern (MATCH (n)) and you want to filter or branch on the actual type:

```
# Filter to only Person nodes from a heterogeneous scan
result = star.execute_query(
```

(continues on next page)

(continued from previous page)

```

    "MATCH (n) WHERE n:Person RETURN n.name AS name"
)

# Negation — exclude a specific type
result = star.execute_query(
    "MATCH (n) WHERE NOT n:Product RETURN n.name AS name"
)

# Combine label predicate with property filter
result = star.execute_query(
    "MATCH (n) WHERE n:Person AND n.age > 30 RETURN n.name AS name"
)

# Redundant but valid — labeled MATCH + WHERE :Label confirms the type
result = star.execute_query(
    "MATCH (p:Person) WHERE p:Person RETURN p.name AS name"
)

```

Label predicates compose naturally with AND / OR / NOT and with all other WHERE predicates. The compound form `n:Label1:Label2` applies AND semantics (the node must have both labels simultaneously) — useful when an entity table carries multiple inherited labels.

2.7 Variable-Length Paths

Match paths of variable hop depth using `*min..max` syntax on relationship patterns:

```

# Any number of KNOWS hops (1 or more)
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*]->(b:Person) RETURN a.name, b.name"
)

# Exactly 2-3 hops
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*2..3]->(b:Person) RETURN a.name, b.name"
)

# Up to 4 hops — find indirect connections
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*1..4]->(b:Person) RETURN a.name, b.name"
)

```

2.8 shortestPath and allShortestPaths

Use `shortestPath()` to find the minimum-hop route between two nodes. `allShortestPaths()` returns every path tied for the minimum length:

```

# Shortest route from Alice to Dave
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})

```

(continues on next page)

(continued from previous page)

```

MATCH p = shortestPath((a)-[:KNOWS*]->(b))
RETURN a.name AS start, b.name AS end, length(p) AS hops
"""
)
# → one row: start='Alice', end='Dave', hops=2

# All shortest paths (same minimum hop count)
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})
    MATCH p = allShortestPaths((a)-[:KNOWS*]->(b))
    RETURN a.name AS start, b.name AS end, length(p) AS hops
    """
)
# → one or more rows, all with hops == minimum

```

If no path exists between the two nodes the result is empty (not an error).

2.9 OPTIONAL MATCH — Left-Join Semantics

OPTIONAL MATCH preserves rows even when the pattern has no match. Unmatched variables are bound to null:

```

result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:BOUGHT]->(item:Product)
    WITH p.name AS person, item.name AS product
    RETURN person, product
    """
)
# People who have not bought anything appear with product = null

```

2.10 WITH + UNWIND: List Explosion

WITH pipelines data between stages. UNWIND explodes a list column into one row per element:

```

# Collect all names and unwind into individual rows
result = star.execute_query(
    """
    MATCH (p:Person)
    WITH collect(p.name) AS names
    UNWIND names AS name
    RETURN name
    """
)

# Explode a list property
result = star.execute_query(
    """

```

(continues on next page)

(continued from previous page)

```

MATCH (p:Person)
WITH p.name AS name, p.tags AS tags
UNWIND tags AS tag
RETURN name, tag
"""
)

```

2.11 Aggregation

```

# Count, group by department
result = star.execute_query(
    "MATCH (p:Person) RETURN p.dept AS dept, count(p) AS n"
)

# Collect values into a list
result = star.execute_query(
    "MATCH (p:Person) RETURN collect(p.name) AS all_names"
)

# Order and paginate
result = star.execute_query(
    "MATCH (p:Person) RETURN p.name, p.age ORDER BY p.age DESC LIMIT 5"
)

# Null placement: NULLS FIRST puts null-valued rows before non-null rows.
# Default (no keyword) is NULLS LAST, matching Neo4j 5.x behaviour.
result = star.execute_query(
    "MATCH (n:Person) RETURN n.name, n.score "
    "ORDER BY n.score ASC NULLS FIRST"
)

```

2.12 WITH * — Pass-Through Projection

WITH * forwards all variables from the preceding MATCH into subsequent clauses without requiring an explicit alias list:

```

# Access any matched variable after WITH * — useful for filtering then projecting
result = star.execute_query(
    "MATCH (p:Person) WITH * WHERE p.age > 28 RETURN p.name AS name, p.age AS age"
)

# Sort and cap rows before explicit projection
result = star.execute_query(
    "MATCH (p:Person) WITH * ORDER BY p.age ASC LIMIT 3 RETURN p.name AS name"
)

```

2.13 UNION — Combining Query Results

UNION merges the results of two or more queries, deduplicating rows. UNION ALL keeps duplicates:

```
# Names from two entity types (deduplication applied)
result = star.execute_query(
    """
    MATCH (p:Person) RETURN p.name AS name
    UNION
    MATCH (c:Company) RETURN c.name AS name
    """
)

# Keep all rows including duplicates
result = star.execute_query(
    """
    MATCH (p:Person) WHERE p.dept = 'Eng' RETURN p.name AS name
    UNION ALL
    MATCH (p:Person) WHERE p.age < 30 RETURN p.name AS name
    """
)
```

2.14 Mutating the Graph — CREATE, MERGE, DELETE, REMOVE

PyCypher supports the full Cypher write path. All mutations are atomic: changes are buffered and only committed when the query succeeds. An exception mid-query rolls back all pending changes.

2.14.1 CREATE — Insert New Nodes and Relationships

```
# Create a single node
star.execute_query(
    """CREATE (p:Person {name: 'Dave', age: 28})"""
)

# Create a relationship between existing nodes
star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'})
    CREATE (a)-[:KNOWS]->(b)
    """
)
```

2.14.2 MERGE — Upsert Semantics

MERGE either matches an existing pattern or creates it. Use ON CREATE SET and ON MATCH SET to apply different updates:

```
# Find or create a Person with this name
star.execute_query(
    """MERGE (p:Person {name: 'Eve'}) SET p.created = true"""
)
```

(continues on next page)

(continued from previous page)

```
# Track first-seen vs update-time
star.execute_query(
    """
    MERGE (p:Person {name: 'Alice'})
    ON CREATE SET p.created_at = timestamp()
    ON MATCH SET p.updated_at = timestamp()
    """
)
```

When the entity type referenced in a MERGE pattern is not registered in the context, MERGE treats it as absent and creates the pattern — this is expected Cypher semantics for bootstrapping a new entity type. If a genuine runtime error occurs during the match phase (e.g. a programming error, not a missing type), the error propagates as a `GraphTypeNotFoundError` so bugs are never silently swallowed.

2.14.3 SET — Update Properties

```
# Set a single property
star.execute_query(
    "MATCH (p:Person {name: 'Alice'}) SET p.score = 99"
)

# Set multiple properties at once
star.execute_query(
    "MATCH (p:Person) SET p.active = true, p.last_seen = timestamp()"
)

# Set relationship properties
star.execute_query(
    "MATCH (a:Person)-[r:KNOWS]->(b:Person) SET r.since = 2020"
)
```

2.14.4 REMOVE — Delete Properties

```
# Remove a property from all matching nodes
star.execute_query(
    "MATCH (p:Person) WHERE p.temp_flag IS NOT NULL REMOVE p.temp_flag"
)
```

2.14.5 DELETE — Remove Nodes

```
# Delete nodes that match the pattern
star.execute_query(
    "MATCH (p:Person {name: 'Dave'}) DELETE p"
)
```

2.15 FOREACH — Iterate Over a List

FOREACH applies a write clause (SET, CREATE, MERGE, DELETE) to each element of a list. It is the standard Cypher idiom for batch mutations:

```
# Mark every person in a list as active
star.execute_query(
    """
    MATCH (p:Person)
    WITH collect(p) AS people
    FOREACH (person IN people | SET person.active = true)
    """
)

# Create a chain of nodes from a list of values
star.execute_query(
    """
    FOREACH (name IN ['Alice', 'Bob', 'Carol'] |
    MERGE (p:Person {name: name})
    )
    """
)
```

2.16 Functional Expressions

PyCypher supports the full suite of Cypher functional expressions for in-query list manipulation and subquery predicates. All are evaluated in a single vectorised pass over the frame — no per-row Python overhead.

2.16.1 List Comprehensions

[variable IN list WHERE condition | map_expression] — filter and/or transform a list inline. The WHERE clause and map expression are optional:

```
# Filter — keep only high scores
result = star.execute_query(
    """MATCH (p:Person) """
    """RETURN [s IN p.scores WHERE s > 50] AS high_scores"""
)

# Transform — double every score
result = star.execute_query(
    """MATCH (p:Person) """
    """RETURN [s IN p.scores | s * 2] AS doubled"""
)

# Filter + transform in one step
result = star.execute_query(
    """MATCH (p:Person) """
    """RETURN [s IN p.scores WHERE s > 50 | s * 2] AS high_doubled"""
)
```

2.16.2 Pattern Comprehensions

[(anchor)-[:REL]->(target) WHERE condition | map_expression] — collect matched graph neighbours into a per-row list:

```
# Names of all people each person knows
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS person, "
    "      [(p)-[:KNOWS]->(f) | f.name] AS friends"
)

# Older friends only
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS person, "
    "      [(p)-[:KNOWS]->(f:Person) WHERE f.age > p.age | f.name] AS older_friends"
)
```

2.16.3 Quantifier Predicates

Test whether a condition holds for some, all, none, or exactly one element:

```
# any() — at least one element satisfies the predicate
result = star.execute_query(
    "MATCH (p:Person) WHERE any(s IN p.scores WHERE s > 90) "
    "RETURN p.name AS name"
)

# all() — every element satisfies the predicate (vacuously true for empty)
result = star.execute_query(
    "MATCH (p:Person) WHERE all(s IN p.scores WHERE s >= 0) "
    "RETURN p.name AS name"
)

# none() — no element satisfies the predicate
result = star.execute_query(
    "MATCH (p:Person) WHERE none(s IN p.scores WHERE s < 0) "
    "RETURN p.name AS name"
)

# single() — exactly one element satisfies the predicate
result = star.execute_query(
    "MATCH (p:Person) WHERE single(s IN p.scores WHERE s > 95) "
    "RETURN p.name AS name"
)
```

2.16.4 EXISTS Subqueries

EXISTS { MATCH ... WHERE ... } tests whether an inner graph pattern produces at least one result. The inner query is batched: it executes once for all outer rows, not once per row:

```
# People who know at least one other person
result = star.execute_query(
    "MATCH (p:Person) WHERE EXISTS { MATCH (p)-[:KNOWS]->(q) } "
    "RETURN p.name AS name"
)
```

(continues on next page)

(continued from previous page)

```

# NOT EXISTS — people who know nobody
result = star.execute_query(
    "MATCH (p:Person) WHERE NOT EXISTS { MATCH (p)-[:KNOWS]->(q) } "
    "RETURN p.name AS name"
)

# Filter the subquery match with WHERE
result = star.execute_query(
    "MATCH (p:Person) "
    "WHERE EXISTS { MATCH (p)-[:KNOWS]->(q:Person) WHERE q.age > 30 } "
    "RETURN p.name AS name"
)

```

2.16.5 REDUCE

`reduce(accumulator = initial, variable IN list | step_expression)` — fold a list into a scalar value. The accumulator carries the running result; `step_expression` receives both the accumulator and the current element:

```

# Sum of a list of scores
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN reduce(total = 0, s IN p.scores | total + s) AS score_sum"
)

# Maximum element
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN reduce(m = -1, s IN p.scores | CASE WHEN s > m THEN s ELSE m END) AS max_score"
    "↪"
)

```

2.17 Scalar Functions

Over 130 scalar functions are available in any expression context. All functions are null-safe: passing null returns null unless documented otherwise.

Math and trigonometric functions (`abs`, `sin`, `log`, `pow`, etc.) are evaluated using numpy C-level array operations — no per-row Python overhead. String predicate functions (`startsWith`, `endsWith`, `contains`) use the pandas `.str` Cython accessor. These two categories are safe to use in `WHERE` clauses on large frames without performance concern.

```

# String manipulation
result = star.execute_query(
    "MATCH (p:Person) RETURN toUpper(p.name) AS upper_name"
)

# Null-safe default via coalesce
result = star.execute_query(
    "MATCH (p:Person) "

```

(continues on next page)

(continued from previous page)

```

    "RETURN coalesce(p.nickname, p.name) AS display_name"
)

# Type predicate — filter rows where a property has the expected type
result = star.execute_query(
    "MATCH (r:Record) WHERE isString(r.code) AND isInteger(r.count) "
    "RETURN r.code, r.count"
)

# Temporal — parse and decompose a duration
result = star.execute_query(
    "RETURN duration('P1Y6M').months AS months"
    # → 6
)

# Rounding with explicit mode (Neo4j 5.x 3-arg form)
# Modes: HALF_UP (default, away from zero), HALF_DOWN, HALF_EVEN (banker's),
#        CEILING, FLOOR, UP (away from zero), DOWN (truncation)
result = star.execute_query(
    "MATCH (p:Person) RETURN round(p.score, 2, 'HALF_EVEN') AS rounded"
)
# round(2.5, 0, 'HALF_EVEN') → 2.0 (nearest even)
# round(2.5, 0, 'HALF_UP')   → 3.0 (away from zero)
# round(2.5, 0, 'CEILING')   → 3.0 (toward +∞)

# Bitwise operations — ETL flag/bitmask processing (Neo4j 5.x)
result = star.execute_query(
    "MATCH (e:Event) WHERE bitAnd(e.flags, 4) > 0 "
    "RETURN e.name AS name, bitOr(e.flags, 1) AS flags_with_bit0"
)
# bitAnd / bitOr / bitXor / bitNot / bitShiftLeft / bitShiftRight
# All accept integer args, return null when any arg is null.

```

Function categories (131 functions registered as of this writing):

Category	Functions
String	toUpper / upper, toLower / lower, trim, ltrim, rtrim, substring, left, right, size, length / len, replace, split, reverse, isEmpty
Extended string	lpad, rpad, repeat, btrim, indexOf, charAt, char, charCodeAt, normalize, toStringOrNull, startsWith, endsWith, contains, byteSize
Type conversion	toString / str, toInteger / int, toFloat / float, toBoolean / bool, toBooleanOrNull, toIntegerOrNull, toFloatOrNull
List type conversion	toStringList, toIntegerList, toFloatList, toBooleanList
Math	abs, ceil, floor, round (1-arg, 2-arg, or 3-arg with mode), sign, sqrt, cbrt, log, exp, log10, log2, pow, hypot, fmod, gcd, lcm
Bitwise	bitAnd, bitOr, bitXor, bitNot, bitShiftLeft, bitShiftRight
Trigonometric	sin, cos, tan, asin, acos, atan, atan2, cot, sinh, cosh, tanh, haversin, degrees, radians
Constants & random	pi, e, rand, infinity, isNaN, isFinite, isInfinite
List	head, last, tail, range, sort, flatten, toList, min, max
Map	keys, values, properties
Temporal (parse)	date, datetime, localdatetime, duration
Temporal (truncate)	date.truncate, datetime.truncate, localdatetime.truncate
Temporal (now)	timestamp / now, localtime, localdate
Type predicates	isString, isInteger, isFloat, isBoolean, isList, isMap
Type introspection	valueType
Graph introspection	id, elementId, labels, type, keys, exists
Hash & encoding	md5, sha1, sha256, encodeBase64, decodeBase64
Temporal arithmetic	date + duration, date - duration, date - date, datetime + duration, datetime - duration, datetime - datetime, duration + duration, duration - duration
Utility	coalesce, nullIf, randomUUID

Discover all registered functions at runtime with `available_functions()`:

```
print(star.available_functions())
# ['abs', 'acos', 'asin', ..., 'valuetype', 'values']
```

2.17.1 Auto-generated Column Names

When a RETURN item has no explicit AS alias, pycypher generates a display name from the expression — matching Neo4j's behaviour:

RETURN expression	Result column name
RETURN p.name	name (property name, unqualified)
RETURN p	p (variable name)
RETURN toUpper(p.name)	toUpper(name)
RETURN abs(p.age)	abs(age)
RETURN p.age + 1	age + 1
RETURN 42	42
RETURN 'hello'	hello
RETURN true	true
RETURN null	null
RETURN p.age > 25	age > 25 (comparison)
RETURN p.name STARTS WITH 'A'	name STARTS WITH A
RETURN p.name IS NULL	name IS NULL
RETURN NOT p.active	NOT active
RETURN p.age > 18 AND p.active	age > 18 AND active
RETURN -p.age	-age (unary negation)
RETURN \$limit	\$limit (query parameter)
RETURN count(*)	count(*)
RETURN xs[0]	xs[0] (index lookup)
RETURN xs[1..3]	xs[1..3] (slicing)
RETURN [x IN xs \ x * 2]	[x IN xs \ x * 2] (list comprehension)
RETURN CASE WHEN ... END	case

Using AS always overrides the auto-generated name:

```
# Two aliasless columns — each gets its own display name
result = star.execute_query(
    "MATCH (p:Person) RETURN toUpper(p.name), p.age + 1"
)
print(result.columns.tolist())
# ['toUpper(name)', 'age + 1']
```

2.18 Running Tests

```
uv run pytest          # full suite
uv run pytest -n 4     # parallel (faster)
uv run pytest -k Person # filter by keyword
```

2.19 Next Steps

- [Query Processing](#) — deep-dive on the execution model
- [PyCypher API](#) — full API reference
- [Tutorials](#) — step-by-step tutorials

Learn PyCypher through structured, hands-on tutorials that progress from first query to production deployment.

3.1 Cypher for SQL Users

A translation guide for developers who know SQL but are new to Cypher. Read this in 10 minutes and you'll understand every PyCypher example.

On this page

- [The 60-Second Mental Model](#)
- [Side-by-Side Examples](#)
 - [SELECT ... FROM ... WHERE](#)
 - [JOIN \(Inner Join via Relationship\)](#)
 - [LEFT JOIN \(OPTIONAL MATCH\)](#)
 - [GROUP BY with Aggregation](#)
 - [Subqueries \(WITH Clause\)](#)
 - [EXISTS \(Subquery Predicate\)](#)
 - [COLLECT \(No SQL Equivalent\)](#)
- [Common SQL Patterns Translated](#)
 - [DISTINCT](#)
 - [COUNT with Filter](#)
 - [LIKE \(String Matching\)](#)
 - [IN List](#)
 - [COALESCE / NULL Handling](#)
 - [CASE Expressions](#)

– INSERT / UPDATE / DELETE

- [When to Use Graph Patterns vs SQL Thinking](#)
- [Key Syntax Differences Cheat Sheet](#)

3.1.1 The 60-Second Mental Model

SQL Concept	Cypher Equivalent	Key Difference
SELECT	RETURN	Same role — choose output columns
FROM table	MATCH (n:Label)	Nodes replace tables
WHERE	WHERE	Same syntax for property filters
JOIN ... ON	()-[:REL]->()	Relationships replace JOINS
LEFT JOIN	OPTIONAL MATCH	Preserves rows with no match
GROUP BY	(implicit)	Non-aggregated RETURN columns group
ORDER BY	ORDER BY	Identical
LIMIT	LIMIT	Identical
INSERT INTO	CREATE	Creates nodes or relationships
UPDATE	SET	Sets properties on matched nodes
DELETE FROM	DELETE	Removes matched nodes
subquery	WITH	Pipes results between query stages

The biggest conceptual shift: in SQL you join tables; in Cypher you traverse relationships.

3.1.2 Side-by-Side Examples

All examples below use this dataset:

```
import pandas as pd
from pycypher import Star
from pycypher.ingestion import ContextBuilder

people = pd.DataFrame({
    "__ID__": [1, 2, 3, 4],
    "name": ["Alice", "Bob", "Carol", "Dave"],
    "age": [30, 25, 35, 28],
    "dept": ["eng", "sales", "eng", "sales"],
})

products = pd.DataFrame({
    "__ID__": [10, 20, 30],
    "title": ["Widget", "Gadget", "Gizmo"],
    "price": [9.99, 49.99, 14.99],
})

purchases = pd.DataFrame({
    "__ID__": [100, 101, 102, 103],
    "__SOURCE__": [1, 2, 1, 3],
    "__TARGET__": [10, 20, 30, 10],
})
```

(continues on next page)

(continued from previous page)

```
context = ContextBuilder.from_dict({
    "Person": people,
    "Product": products,
    "BOUGHT": purchases,
})
star = Star(context=context)
```

SELECT ... FROM ... WHERE

SQL:

```
SELECT name, age FROM Person WHERE age > 28
```

Cypher:

```
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 28 RETURN p.name AS name, p.age AS age"
)
```

```
name age
Alice 30
Carol 35
```

Key difference: In Cypher, `p` is a variable bound to each `Person` node. You access properties with dot notation: `p.age`, `p.name`.

JOIN (Inner Join via Relationship)

SQL:

```
SELECT p.name, pr.title
FROM Person p
JOIN purchases ON purchases.person_id = p.id
JOIN Product pr ON pr.id = purchases.product_id
```

Cypher:

```
result = star.execute_query(
    "MATCH (p:Person)-[:BOUGHT]->(pr:Product) "
    "RETURN p.name AS buyer, pr.title AS product"
)
```

```
buyer product
Alice Widget
Alice Gizmo
Bob Gadget
Carol Widget
```

Key difference: No `ON` clause needed — the `-[:BOUGHT]->` pattern encodes the join condition. The arrow `->` indicates direction.

LEFT JOIN (OPTIONAL MATCH)

SQL:

```
SELECT p.name, pr.title
FROM Person p
LEFT JOIN purchases ON purchases.person_id = p.id
LEFT JOIN Product pr ON pr.id = purchases.product_id
```

Cypher:

```
result = star.execute_query(
    "MATCH (p:Person) "
    "OPTIONAL MATCH (p)-[:BOUGHT]->(pr:Product) "
    "RETURN p.name AS person, pr.title AS product"
)
```

person	product
Alice	Widget
Alice	Gizmo
Bob	Gadget
Carol	Widget
Dave	None

Key difference: OPTIONAL MATCH preserves all Person rows. Dave appears with None because he has no purchases.

GROUP BY with Aggregation

SQL:

```
SELECT dept, COUNT(*) AS headcount
FROM Person
GROUP BY dept
ORDER BY dept
```

Cypher:

```
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.dept AS department, count(p) AS headcount "
    "ORDER BY department"
)
```

department	headcount
eng	2
sales	2

Key difference: No GROUP BY clause. Cypher groups implicitly by all non-aggregated columns in RETURN. count(p) counts nodes, not rows.

Subqueries (WITH Clause)

SQL:

```
SELECT name FROM (
  SELECT name, age FROM Person WHERE age > 25
) sub
WHERE age < 35
```

Cypher:

```
result = star.execute_query(
  "MATCH (p:Person) "
  "WHERE p.age > 25 "
  "WITH p.name AS name, p.age AS age "
  "WHERE age < 35 "
  "RETURN name"
)
```

```
name
Alice
Dave
```

Key difference: WITH pipes results between query stages, like a subquery. Each WITH creates a new scope — only columns listed in WITH are available downstream.

EXISTS (Subquery Predicate)

SQL:

```
SELECT name FROM Person p
WHERE EXISTS (
  SELECT 1 FROM purchases WHERE person_id = p.id
)
```

Cypher:

```
result = star.execute_query(
  "MATCH (p:Person) "
  "WHERE EXISTS { MATCH (p)-[:BOUGHT]->() } "
  "RETURN p.name AS name"
)
```

```
name
Alice
Bob
Carol
```

COLLECT (No SQL Equivalent)

Cypher can aggregate values into lists — something SQL cannot do natively:

```
result = star.execute_query(
  "MATCH (p:Person)-[:BOUGHT]->(pr:Product) "
```

(continues on next page)

(continued from previous page)

```

    "RETURN p.name AS buyer, collect(pr.title) AS products"
)

```

buyer	products
Alice	[Widget, Gizmo]
Bob	[Gadget]
Carol	[Widget]

3.1.3 Common SQL Patterns Translated

DISTINCT

SQL: SELECT DISTINCT dept FROM Person

Cypher:

```
MATCH (p:Person) RETURN DISTINCT p.dept AS dept
```

COUNT with Filter

SQL: SELECT COUNT(*) FROM Person WHERE age > 30

Cypher:

```
MATCH (p:Person) WHERE p.age > 30 RETURN count(p) AS n
```

LIKE (String Matching)

SQL: SELECT name FROM Person WHERE name LIKE 'A%'

Cypher:

```
MATCH (p:Person) WHERE p.name STARTS WITH 'A' RETURN p.name AS name
```

Other string predicates: ENDS WITH, CONTAINS.

IN List

SQL: SELECT name FROM Person WHERE dept IN ('eng', 'sales')

Cypher:

```
MATCH (p:Person) WHERE p.dept IN ['eng', 'sales'] RETURN p.name AS name
```

Note: Cypher uses square brackets [] for lists, not parentheses.

COALESCE / NULL Handling

SQL: SELECT COALESCE(nickname, name) FROM Person

Cypher:

```
MATCH (p:Person) RETURN coalesce(p.nickname, p.name) AS display_name
```

NULL handling: IS NULL, IS NOT NULL work identically to SQL.

CASE Expressions

SQL: `SELECT CASE WHEN age > 30 THEN 'senior' ELSE 'junior' END FROM Person`

Cypher:

```
MATCH (p:Person)
RETURN CASE WHEN p.age > 30 THEN 'senior' ELSE 'junior' END AS level
```

INSERT / UPDATE / DELETE

SQL:

```
INSERT INTO Person (name, age) VALUES ('Eve', 29);
UPDATE Person SET age = 31 WHERE name = 'Alice';
DELETE FROM Person WHERE name = 'Dave';
```

Cypher:

```
CREATE (e:Person {name: 'Eve', age: 29})
MATCH (p:Person {name: 'Alice'}) SET p.age = 31
MATCH (p:Person {name: 'Dave'}) DELETE p
```

3.1.4 When to Use Graph Patterns vs SQL Thinking

Graph patterns work better when:

- You're traversing relationships between entities (social networks, supply chains, dependency graphs)
- Join depth is variable or unknown (`MATCH path = (a)-[:KNOWS*1..5]->(b)`)
- You want to express “find all connected entities” naturally

SQL thinking works fine when:

- You're filtering a single entity type (`MATCH (p:Person) WHERE ...`)
- You're doing simple aggregation (counts, sums, averages)
- You have a fixed, known join structure

PyCypher handles both well — the Cypher syntax is expressive for graph traversal while remaining readable for simple table-like queries.

3.1.5 Key Syntax Differences Cheat Sheet

SQL	Cypher	Note
table.column	variable.property	Dot notation for both
'string'	'string'	Same quoting
NULL	null	Lowercase in Cypher
TRUE / FALSE	true / false	Lowercase in Cypher
(1, 2, 3)	[1, 2, 3]	Square brackets for lists
AS alias	AS alias	Same syntax
AND / OR / NOT	AND / OR / NOT	Same syntax
<> / !=	<>	Cypher uses <> only
BETWEEN a AND b	>= a AND <= b	No BETWEEN keyword
LIKE '%text%'	CONTAINS 'text'	Named predicates instead
OFFSET n	SKIP n	Different keyword
COUNT(*)	count(*)	Lowercase functions

3.2 Basic Query Execution

This tutorial shows how to run Cypher queries against your data using PyCypher.

3.2.1 Learning Objectives

- Load data and build a query context
- Execute MATCH / WHERE / RETURN queries
- Filter, sort, and aggregate results
- Inspect the parsed AST when needed

3.2.2 Setup

```
import pandas as pd
from pycypher.ingestion import ContextBuilder
from pycypher import Star

# Load data from a CSV (or pass a DataFrame directly)
context = (
    ContextBuilder()
    .add_entity("Person", "data/people.csv")
    .build()
)
star = Star(context=context)
```

If your data is already in memory as a DataFrame, the fastest path is from_dict():

```
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})
context = ContextBuilder.from_dict({"Person": people})
star = Star(context=context)
```

3.2.3 Running Queries

Simple MATCH

```
# Returns a pandas DataFrame
result = star.execute_query("MATCH (p:Person) RETURN p.name AS name")
print(result)
#   name
# 0  Alice
# 1   Bob
# 2  Carol
```

Filtering with WHERE

```
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 25 RETURN p.name AS name, p.age AS age"
)
print(result)
#   name  age
# 0  Alice   30
# 1  Carol   35
```

Sorting and Limiting

```
# Top 2 oldest people
result = star.execute_query(
    "MATCH (p:Person) RETURN p.name AS name ORDER BY p.age DESC LIMIT 2"
)
```

Aggregation

```
result = star.execute_query(
    "MATCH (p:Person) RETURN count(p) AS total, avg(p.age) AS avg_age"
)
```

3.2.4 Relationship Patterns

```
context = (
    ContextBuilder()
    .add_entity("Person", "data/people.csv")
    .add_entity("Product", "data/products.csv")
    .add_relationship("BOUGHT", "data/purchases.csv",
                     source_col="user_id", target_col="product_id")
    .build()
)
star = Star(context=context)

result = star.execute_query(
    """
    MATCH (buyer:Person)-[:BOUGHT]->(item:Product)
    WHERE buyer.age >= 30
    RETURN buyer.name AS buyer, item.name AS product
    """
)
```

(continues on next page)

(continued from previous page)

```
ORDER BY buyer.name
"""
)
```

3.2.5 Discovering Available Functions

Use `available_functions()` to see all registered scalar functions at any time:

```
print(star.available_functions())
# ['abs', 'acos', 'asin', ..., 'toupper', 'trim']

# Use them in any expression context
result = star.execute_query(
    "MATCH (p:Person) RETURN toUpper(p.name) AS upper_name"
)
```

3.2.6 Advanced: Inspecting the Parsed AST

For users who need to programmatically inspect or introspect parsed query structure, `from_cypher()` returns a fully-typed Pydantic AST:

```
from pycypher.ast_models import ASTConverter, Match, Return

query = "MATCH (p:Person) WHERE p.age > 30 RETURN p.name AS name"
ast = ASTConverter.from_cypher(query)

# ast is a Query object; clauses is the flat list of parsed clauses
for clause in ast.clauses:
    print(type(clause).__name__, clause)

# Check the first clause is a MATCH
match_clause = ast.clauses[0]
assert isinstance(match_clause, Match)
print("WHERE predicate:", match_clause.where)

# Check the last clause is a RETURN
return_clause = ast.clauses[-1]
assert isinstance(return_clause, Return)
print("RETURN items:", return_clause.items)
```

3.2.7 Error Handling

```
try:
    result = star.execute_query("MATCH (p:Person) RETURN p.salary AS s")
except KeyError as e:
    # Property 'salary' not found on entity type 'Person'
    print(f"Property error: {e}")

try:
    result = star.execute_query("MATCH (x:Unknown) RETURN x.name")
except ValueError as e:
```

(continues on next page)

(continued from previous page)

```
# Entity type 'Unknown' is not registered  
print(f"Entity error: {e}")
```

3.2.8 Next Steps

- Query Processing — execution model deep-dive
- Pattern Matching — advanced graph patterns
- PyCypher API — full API reference

3.3 Graph Modeling

Learn how to design your tabular data as a graph for PyCypher — choosing entity types, modeling relationships, handling properties, and avoiding common pitfalls.

In this tutorial

- Prerequisites
- Why Graph Modeling Matters
- The Core Abstraction
- Designing Entities
 - One Table per Concept
 - Choosing IDs
 - Properties
- Designing Relationships
 - Relationship Direction
 - Relationship Properties
 - Multiple Relationship Types
- Common Modeling Patterns
 - Hub-and-Spoke (Star Schema)
 - Chain (Sequential Events)
 - Bipartite Graph
- Try It Yourself
- Common Pitfalls
- Next Steps

3.3.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Completed [Basic Query Execution](#)
- Familiarity with pandas DataFrames

3.3.2 Why Graph Modeling Matters

PyCypher executes Cypher queries against DataFrames, not a graph database. This means you decide how to carve your tabular data into entities and relationships. Good modeling choices make queries natural and fast; poor choices force awkward workarounds and slow cross-joins.

3.3.3 The Core Abstraction

PyCypher has two primitives:

Entities — things with an identity and properties (nodes in graph terms):

```
import pandas as pd
from pycypher.ingestion import ContextBuilder

people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})
```

Relationships — directed connections between entities:

```
knows = pd.DataFrame({
    "__SOURCE__": [1, 2],
    "__TARGET__": [2, 3],
    "since": [2020, 2021],
})
```

The `__ID__`, `__SOURCE__`, and `__TARGET__` columns are the structural glue. Everything else is a property.

3.3.4 Designing Entities

One Table per Concept

Each real-world concept becomes a separate entity type. Don't merge unrelated data into a single table:

```
# GOOD: separate entity types
context = (
    ContextBuilder()
    .add_entity("Person", people_df)
    .add_entity("Company", companies_df)
    .add_entity("Product", products_df)
    .build()
)

# BAD: one mega-table with a "type" column
# This forces every query to filter by type manually
```

Choosing IDs

The `__ID__` column uniquely identifies each row within its entity type. IDs can be integers or strings — choose whatever your source data provides:

```
# Integer IDs (from a database primary key)
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
})

# String IDs (from a natural key or UUID)
people = pd.DataFrame({
    "__ID__": ["alice-01", "bob-02", "carol-03"],
    "name": ["Alice", "Bob", "Carol"],
})

# Using an existing column as the ID
raw = pd.DataFrame({"user_id": [1, 2, 3], "name": ["Alice", "Bob", "Carol"]})
context = ContextBuilder().add_entity("Person", raw, id_col="user_id").build()
```

Warning

IDs must be unique within each entity type. Duplicate IDs produce unexpected cross-join results during MATCH.

Properties

Every non-ID column becomes a property accessible via `n.property_name` in Cypher:

```
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
    "active": [True, True, False],
    "scores": [[85, 92], [70], [95, 88, 60]], # list properties work too
})
```

Property types map directly from pandas dtypes: `int64`, `float64`, `object` (strings), `bool`, and Python lists.

3.3.5 Designing Relationships

Relationship Direction

Relationships in PyCypher are directed: `__SOURCE__` → `__TARGET__`. Model the direction that reflects the real-world semantics:

```
# Person -[:KNOWS]-> Person (social link)
knows = pd.DataFrame({
    "__SOURCE__": [1, 2], # who initiates
    "__TARGET__": [2, 3], # who is known
})
```

(continues on next page)

(continued from previous page)

```
# Person -[:WORKS_AT]-> Company (employment)
works_at = pd.DataFrame({
    "__SOURCE__": [1, 2], # employee
    "__TARGET__": [10, 20], # company
})
```

In queries, (a)-[:KNOWS]->(b) follows the direction. Use (a)<-[:KNOWS]-(b) or (a)-[:KNOWS]-(b) for reverse or undirected.

Relationship Properties

Add columns beyond __SOURCE__ and __TARGET__ for relationship properties:

```
works_at = pd.DataFrame({
    "__SOURCE__": [1, 2, 3],
    "__TARGET__": [10, 10, 20],
    "role": ["Engineer", "Manager", "Analyst"],
    "start_year": [2018, 2015, 2020],
})
```

Query them by binding the relationship to a variable:

```
result = star.execute_query(
    "MATCH (p:Person)-[r:WORKS_AT]->(c:Company) "
    "RETURN p.name, r.role, c.name"
)
```

Multiple Relationship Types

Register each relationship type separately:

```
context = (
    ContextBuilder()
    .add_entity("Person", people)
    .add_entity("Company", companies)
    .add_entity("Product", products)
    .add_relationship("KNOWS", knows_df,
        source_col="person_a", target_col="person_b")
    .add_relationship("WORKS_AT", works_at_df,
        source_col="employee_id", target_col="company_id")
    .add_relationship("BOUGHT", purchases_df,
        source_col="buyer_id", target_col="product_id")
    .build()
)
```

3.3.6 Common Modeling Patterns

Hub-and-Spoke (Star Schema)

A central entity connected to many satellite entities. Natural for customer-centric or product-centric analytics:

```
Person  WORKS_AT  > Company
```

```
BOUGHT  > Product
```

```
LIVES_IN  > City
```

```
# "Who bought products from their own city?"
result = star.execute_query(
    """
    MATCH (p:Person)-[:BOUGHT]->(prod:Product),
          (p)-[:LIVES_IN]->(c:City),
          (prod)-[:SOLD_IN]->(c)
    RETURN p.name, prod.title, c.name
    """
)
```

Chain (Sequential Events)

Events linked in sequence — log analysis, process workflows:

```
Event1  NEXT  > Event2  NEXT  > Event3
```

```
events = pd.DataFrame({
    "__ID__": ["e1", "e2", "e3", "e4"],
    "action": ["login", "view_page", "add_cart", "checkout"],
    "timestamp": ["2024-01-01 10:00", "2024-01-01 10:05",
                  "2024-01-01 10:12", "2024-01-01 10:15"],
})
next_event = pd.DataFrame({
    "__SOURCE__": ["e1", "e2", "e3"],
    "__TARGET__": ["e2", "e3", "e4"],
})

# Find 3-step funnels
result = star.execute_query(
    """
    MATCH (a:Event)-[:NEXT]->(b:Event)-[:NEXT]->(c:Event)
    RETURN a.action, b.action, c.action
    """
)
```

Bipartite Graph

Two entity types connected only through relationships (no intra-type edges). Common for recommendation systems:

```
User  RATED  > Movie
```

```
User  RATED  > Movie
```

```
# Users who rated the same movie as Alice
result = star.execute_query(
    """
```

(continues on next page)

(continued from previous page)

```

MATCH (alice:User {name: 'Alice'})-[:RATED]->(m:Movie)<-[:RATED]-(other:User)
WHERE other.name <> 'Alice'
RETURN DISTINCT other.name AS similar_user, m.title AS shared_movie
"""
)

```

3.3.7 Try It Yourself

Exercise 1: Model a university dataset with Students, Courses, and Professors. Students enroll in courses (ENROLLED_IN) and professors teach courses (TEACHES).

```

# Your DataFrames here:
students = pd.DataFrame({"__ID__": [...], "name": [...], "year": [...]}
courses = pd.DataFrame({"__ID__": [...], "title": [...], "credits": [...]}
professors = pd.DataFrame({"__ID__": [...], "name": [...], "dept": [...]}
enrolled_in = pd.DataFrame({"__SOURCE__": [...], "__TARGET__": [...]}
teaches = pd.DataFrame({"__SOURCE__": [...], "__TARGET__": [...]}

# Query: "Which professors teach courses that Alice is enrolled in?"

```

Exercise 2: You have a CSV of email messages with sender, recipient, subject, and timestamp columns. How would you model this as entities and relationships?

3.3.8 Common Pitfalls

1. Forgetting “__ID__” — Without a unique ID column, entity rows cannot be joined to relationships. Use `id_col` in `ContextBuilder` to rename an existing column.
2. Mismatched ID types — If `__SOURCE__` contains strings but `__ID__` contains integers, relationships won't match. Ensure consistent types.
3. Denormalized relationships — Putting relationship data as columns on an entity table (e.g. `person.company_name`) prevents graph traversal. Extract it into a separate relationship table.
4. Undirected data forced into one direction — For symmetric relationships (e.g. “is friends with”), either store both directions or use undirected match syntax `(a)-[:FRIENDS]-(b)`.

3.3.9 Next Steps

- [Basic Query Execution](#) — execute queries against your modeled data
- [Pattern Matching](#) — advanced traversal techniques
- [Data ETL Pipeline](#) — load data from files and databases

3.4 Pattern Matching

Advanced techniques for matching graph patterns in Cypher queries.

3.4.1 Learning Objectives

- Match relationships between nodes with typed and untyped patterns
- Use label predicates to filter by entity type at query time
- Traverse variable-length paths and find shortest paths

- Use OPTIONAL MATCH for left-join semantics
- Combine multiple MATCH clauses for cross-product queries
- Collect graph neighbours with pattern comprehensions
- Test list conditions with quantifier predicates and EXISTS subqueries
- Fold lists with REDUCE expressions
- Transform lists inline with list comprehensions
- Pass external values safely with query parameters

3.4.2 Prerequisites

This tutorial assumes you have completed [Basic Query Execution](#) and have a Star instance ready. All examples below use this setup:

```
import pandas as pd
from pycypher import Star
from pycypher.ingestion import ContextBuilder

ctx = ContextBuilder.from_dict({
    "Person": pd.DataFrame({
        "__ID__": ["p1", "p2", "p3", "p4"],
        "name": ["Alice", "Bob", "Carol", "Dave"],
        "age": [30, 25, 35, 28],
        "scores": [[85, 92], [70, 55], [95, 88, 60], [40]],
    }),
    "Product": pd.DataFrame({
        "__ID__": ["x1", "x2"],
        "title": ["Widget", "Gadget"],
        "price": [9.99, 49.99],
    }),
    "KNOWS": pd.DataFrame({
        "__SOURCE__": ["p1", "p2", "p3"],
        "__TARGET__": ["p2", "p3", "p4"],
        "since": [2020, 2021, 2019],
    }),
    "BOUGHT": pd.DataFrame({
        "__SOURCE__": ["p1", "p3"],
        "__TARGET__": ["x1", "x2"],
    }),
})
star = Star(context=ctx)
```

3.4.3 Relationship Patterns

The simplest graph query matches a directed edge between two nodes:

```
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS]->(b:Person) "
    "RETURN a.name AS knower, b.name AS known"
)
# Alice→Bob, Bob→Carol, Carol→Dave
```


The relationship type is specified inside square brackets (`[:KNOWS]`). You can also bind the relationship to a variable to access its properties:

```
result = star.execute_query(
    "MATCH (a:Person)-[r:KNOWS]->(b:Person) "
    "RETURN a.name AS from, b.name AS to, r.since AS since"
)
```

Inline property filters work on both nodes and relationships:

```
# Only KNOWS edges from 2020 or later
result = star.execute_query(
    "MATCH (a:Person)-[r:KNOWS {since: 2020}]->(b:Person) "
    "RETURN a.name AS from, b.name AS to"
)

# Inline node property filter
result = star.execute_query(
    "MATCH (a:Person {name: 'Alice'})-[:KNOWS]->(b:Person) "
    "RETURN b.name AS friend"
)
```

3.4.4 Label Predicates in WHERE

When you use an unlabeled node pattern (`MATCH (n)`), PyCypher scans all registered entity types. `WHERE n:Label` filters to a specific type at query time:

```
# Filter to only Person nodes from a heterogeneous scan
result = star.execute_query(
    "MATCH (n) WHERE n:Person RETURN n.name AS name"
)

# Negation — exclude a specific type
result = star.execute_query(
    "MATCH (n) WHERE NOT n:Product RETURN n.name AS name"
)

# Combine label predicate with property filter
result = star.execute_query(
    "MATCH (n) WHERE n:Person AND n.age > 30 RETURN n.name AS name"
)
```

Label predicates compose with `AND` / `OR` / `NOT` like any other `WHERE` predicate. The compound form `n:Label1:Label2` applies `AND` semantics — the node must have both labels.

3.4.5 Variable-Length Paths

Append `*min..max` to a relationship pattern to match paths of variable hop depth. PyCypher executes these as a BFS traversal:

```
# Any number of KNOWS hops (1 or more, capped internally)
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*]->(b:Person) RETURN a.name, b.name"
)
```

(continues on next page)

(continued from previous page)

```
# Exactly 2-3 hops
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*2..3]->(b:Person) RETURN a.name, b.name"
)

# Up to 4 hops — find indirect connections
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*1..4]->(b:Person) RETURN a.name, b.name"
)
```

 Tip

Always prefer bounded hop limits (`*1..3`) over unbounded (`*`) on large graphs. Unbounded paths explore every reachable edge, and execution time is proportional to the number of reachable edges.

Use `length()` on a named path variable to get the hop count:

```
result = star.execute_query(
    "MATCH p = (a:Person)-[:KNOWS*1..4]->(b:Person) "
    "RETURN a.name, b.name, length(p) AS hops"
)
```

3.4.6 `shortestPath` and `allShortestPaths`

`shortestPath()` finds the minimum-hop route between two nodes. `allShortestPaths()` returns every path tied for the minimum length:

```
# Shortest route from Alice to Dave
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})
    MATCH p = shortestPath((a)-[:KNOWS*]->(b))
    RETURN a.name AS start, b.name AS end, length(p) AS hops
    """
)

# One row: start='Alice', end='Dave', hops depends on graph structure

# All shortest paths (every path at minimum hop count)
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})
    MATCH p = allShortestPaths((a)-[:KNOWS*]->(b))
    RETURN a.name AS start, b.name AS end, length(p) AS hops
    """
)
```

If no path exists between the two nodes, the result is empty (not an error).

3.4.7 OPTIONAL MATCH — Left-Join Semantics

OPTIONAL MATCH preserves rows from the preceding MATCH even when the optional pattern has no match. Unmatched variables are bound to null — just like a SQL LEFT JOIN:

```
# Everyone is returned; product is null for those who bought nothing
result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:BOUGHT]->(item:Product)
    WITH p.name AS person, item.title AS product
    RETURN person, product
    """
)

# Find people who have NOT bought anything
result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:BOUGHT]->(item:Product)
    WITH p.name AS person, item.title AS product
    WHERE product IS NULL
    RETURN person
    """
)

# Null-safe display with coalesce
result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:BOUGHT]->(item:Product)
    WITH p.name AS person, coalesce(item.title, 'nothing') AS product
    RETURN person, product
    """
)
```

3.4.8 Cross-Product MATCH

When two MATCH patterns share no common variables, PyCypher produces the Cartesian product of both scans (equivalent to SQL CROSS JOIN):

```
# All (person, product) pairs
result = star.execute_query(
    """MATCH (p:Person), (pr:Product) """
    """RETURN p.name AS person, pr.title AS product"""
)

# Self cross-join: all ordered (A, B) pairs where A != B
result = star.execute_query(
    """MATCH (p:Person), (q:Person) """
    """WHERE p.name <> q.name """
    """RETURN p.name AS pname, q.name AS qname"""
)
```

3.4.9 Pattern Comprehensions

Pattern comprehensions collect graph neighbours into a per-row list without a separate MATCH clause. The syntax embeds a pattern inside square brackets:

`[(anchor)-[:REL]->(target) WHERE condition | map_expression]`

```
# Names of all people each person knows
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS person, "
    "      [(p)-[:KNOWS]->(f) | f.name] AS friends"
)

# Only older friends (filter with WHERE inside the comprehension)
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS person, "
    "      [(p)-[:KNOWS]->(f:Person) WHERE f.age > p.age | f.name] AS older_friends"
)
```

Pattern comprehensions are evaluated via a single `pd.merge` across all anchor rows — they do not loop per row.

3.4.10 EXISTS Subqueries

`EXISTS { MATCH ... WHERE ... }` tests whether an inner pattern produces at least one result. The subquery executes once for the entire outer frame (batched), not once per row:

```
# People who know at least one other person
result = star.execute_query(
    "MATCH (p:Person) WHERE EXISTS { MATCH (p)-[:KNOWS]->(q) } "
    "RETURN p.name AS name"
)

# NOT EXISTS — people who know nobody
result = star.execute_query(
    "MATCH (p:Person) WHERE NOT EXISTS { MATCH (p)-[:KNOWS]->(q) } "
    "RETURN p.name AS name"
)

# Filter the subquery match with WHERE
result = star.execute_query(
    "MATCH (p:Person) "
    "WHERE EXISTS { MATCH (p)-[:KNOWS]->(q:Person) WHERE q.age > 30 } "
    "RETURN p.name AS name"
)
```

3.4.11 Quantifier Predicates

Test whether a condition holds for some, all, none, or exactly one element of a list:

```
# any() — at least one score above 90
result = star.execute_query(
```

(continues on next page)

(continued from previous page)

```

    "MATCH (p:Person) WHERE any(s IN p.scores WHERE s > 90) "
    "RETURN p.name AS name"
)

# all() — every score is non-negative (vacuously true for empty lists)
result = star.execute_query(
    "MATCH (p:Person) WHERE all(s IN p.scores WHERE s >= 0) "
    "RETURN p.name AS name"
)

# none() — no score below 50
result = star.execute_query(
    "MATCH (p:Person) WHERE none(s IN p.scores WHERE s < 50) "
    "RETURN p.name AS name"
)

# single() — exactly one score above 90
result = star.execute_query(
    "MATCH (p:Person) WHERE single(s IN p.scores WHERE s > 90) "
    "RETURN p.name AS name"
)

```

3.4.12 REDUCE Expressions

`reduce(accumulator = initial, variable IN list | step)` folds a list into a single value. The accumulator carries the running result through each element:

```

# Sum of all scores for each person
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "       reduce(total = 0, s IN p.scores | total + s) AS score_sum"
)
# Alice: 177, Bob: 125, Carol: 243, Dave: 40

# Count elements that meet a condition (scores above 80)
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "       reduce(cnt = 0, s IN p.scores | "
    "           CASE WHEN s > 80 THEN cnt + 1 ELSE cnt END) AS high_scores"
)

# Build a running maximum
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "       reduce(m = -1, s IN p.scores | "
    "           CASE WHEN s > m THEN s ELSE m END) AS max_score"
)

```

REDUCE is the Cypher equivalent of Python's `functools.reduce()` — use it when aggregation functions like `sum()` or `max()` don't cover your use case.

3.4.13 List Comprehensions

List comprehensions transform each element of a list inline:

[variable IN list | map_expression]

```
# Double every score
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "      [s IN p.scores | s * 2] AS doubled"
)

# Add a WHERE filter to keep only high scores
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "      [s IN p.scores WHERE s >= 80 | s] AS high_scores"
)

# Transform to strings
result = star.execute_query(
    "MATCH (p:Person) "
    "RETURN p.name AS name, "
    "      [s IN p.scores | toString(s)] AS score_strings"
)
```

List comprehensions differ from pattern comprehensions (above) — they operate on an existing list property rather than traversing graph edges.

3.4.14 Query Parameters

Use \$name syntax to safely pass external values into queries without string interpolation:

```
# Parameterized lookup — safe from injection
result = star.execute_query(
    "MATCH (p:Person) WHERE p.name = $target RETURN p.age AS age",
    parameters={"target": "Alice"},
)

# Multiple parameters
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age >= $min_age AND p.age <= $max_age "
    "RETURN p.name AS name, p.age AS age",
    parameters={"min_age": 25, "max_age": 32},
)
```

Warning

Never interpolate user input directly into query strings. Always use `parameters={}` — see [Integration Guide](#) for safety patterns.

3.4.15 Performance Tips

1. Push WHERE early. Place selective filters immediately after MATCH to reduce the number of rows before joins and projections.
2. Bound your paths. Use *1..3 instead of * to prevent unbounded BFS traversals on large graphs.
3. Use inline property filters. MATCH (p:Person {name: 'Alice'}) filters during the scan phase, before any joins occur.
4. Prefer pattern comprehensions over separate MATCH + collect() when you need a per-row list of neighbours — they express the intent more concisely and execute in a single merge pass.

3.4.16 Next Steps

- [Basic Query Execution](#) — foundational query execution
- [Query Processing](#) — execution model deep-dive
- [PyCypher API](#) — full API reference

3.5 Query Validation

Learn how to validate openCypher queries before execution using PyCypher’s two-level validation system: syntax validation (grammar parsing) and semantic validation (variable scoping, aggregation rules, etc.).

In this tutorial

- [Prerequisites](#)
- [Overview](#)
- [Syntax Validation](#)
 - [Quick Check](#)
 - [Catching Parse Errors](#)
- [Semantic Validation](#)
 - [Basic Usage](#)
 - [Error Severity Levels](#)
 - [What Gets Validated](#)
 - [Convenience Wrapper](#)
- [AST-Level Validation](#)
- [Runtime Exception Handling](#)
- [Putting It Together](#)
- [Next Steps](#)

3.5.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Familiarity with Cypher query syntax

3.5.2 Overview

PyCypher provides two independent validation layers:

1. Syntax validation — checks whether a query string is grammatically valid Cypher via [validate\(\)](#).
2. Semantic validation — checks whether a syntactically valid query makes logical sense (no undefined variables, correct aggregation usage, etc.) via [SemanticValidator](#).

Both can be used before executing a query, allowing you to surface errors early — especially useful in interactive tools or multi-tenant systems.

3.5.3 Syntax Validation

Quick Check

The [validate\(\)](#) method returns True for valid Cypher and False for syntax errors without raising:

```
from pycypher.grammar_parser import GrammarParser

parser = GrammarParser()

parser.validate("MATCH (n:Person) RETURN n.name") # True
parser.validate("METCH (n) RETRN n")              # False
```

Catching Parse Errors

For richer diagnostics, parse the query directly and catch Lark exceptions:

```
from lark.exceptions import UnexpectedInput

try:
    tree = parser.parse("MATCH (n:Person) RETURN n")
except UnexpectedInput as e:
    print(f"Syntax error at line {e.line}, column {e.column}")
    print(f" {e}")
```

Lark raises specific subclasses — [UnexpectedToken](#), [UnexpectedCharacters](#), [UnexpectedEOF](#) — that you can catch individually for more targeted handling.

3.5.4 Semantic Validation

Basic Usage

The [SemanticValidator](#) walks a parsed Lark tree and reports issues:

```
from pycypher.grammar_parser import GrammarParser
from pycypher.semantic_validator import SemanticValidator

parser = GrammarParser()
validator = SemanticValidator()
```

(continues on next page)

(continued from previous page)

```
tree = parser.parse("MATCH (n:Person) RETURN m")
errors = validator.validate(tree)

for error in errors:
    print(error)
# ERROR: Variable 'm' is used but not defined
```

Error Severity Levels

Each `ValidationError` carries a severity:

```
from pycypher.semantic_validator import ErrorSeverity

for error in errors:
    if error.severity == ErrorSeverity.ERROR:
        print(f"BLOCK: {error.message}")
    elif error.severity == ErrorSeverity.WARNING:
        print(f"WARN: {error.message}")
    elif error.severity == ErrorSeverity.INFO:
        print(f"INFO: {error.message}")
```

- ERROR — the query will fail or produce wrong results
- WARNING — the query will work but may have issues
- INFO — suggestions for improvement

What Gets Validated

The semantic validator checks:

- Undefined variables — using a variable that was never bound in a `MATCH` or `WITH` clause
- Variable scope — variables must be visible in the clause where they are referenced
- Aggregation rules — mixing aggregated and non-aggregated expressions in `RETURN` without grouping
- Function signatures — known function names and arity

Convenience Wrapper

For the simplest use case, import the top-level `validate_query()` helper:

```
from pycypher import validate_query

errors = validate_query("MATCH (n:Person) RETURN m")
for error in errors:
    print(f"{error.severity.value}: {error.message}")
```

3.5.5 AST-Level Validation

Beyond the Lark-tree semantic validator, the typed AST layer (`pycypher.ast_models`) provides a `ValidationResult` framework. This is used internally but is also available for custom checks:

```

from pycypher.ast_models import ValidationResult, ValidationSeverity

result = ValidationResult()
result.add_error("Missing RETURN clause")
result.add_warning("Unused variable 'x'", suggestion="Remove or reference it")

if result.is_valid:
    print("No errors")
else:
    for issue in result.errors:
        print(issue)

```

The ValidationResult object provides filtered access:

- result.errors — error-level issues only
- result.warnings — warning-level issues only
- result.infos — informational issues only
- result.is_valid — True if no errors (warnings are acceptable)
- result.has_errors / result.has_warnings — boolean checks

3.5.6 Runtime Exception Handling

When validation is skipped or insufficient, PyCypher raises specific exceptions at execution time. These are designed to be actionable:

```

from pycypher import (
    Star,
    VariableNotFoundError,
    UnsupportedFunctionError,
    GraphTypeNotFoundError,
)

try:
    result = star.execute_query("MATCH (n:Person) RETURN m.name")
except VariableNotFoundError as e:
    # e.variable_name: the undefined variable
    # e.available_variables: what IS defined
    print(f"Unknown variable '{e.variable_name}'")
    print(f"Available: {e.available_variables}")

try:
    result = star.execute_query("MATCH (n:Person) RETURN noSuchFunc(n)")
except UnsupportedFunctionError as e:
    # e.function_name: what was called
    # e.supported_functions: list of valid alternatives
    print(f"No function '{e.function_name}'")

try:
    result = star.execute_query("MATCH (g:Ghost) RETURN g")
except GraphTypeNotFoundError as e:
    # e.type_name: the unknown entity label
    print(f"Entity type '{e.type_name}' not in context")

```

See the pycypher package docstring for the complete exception hierarchy.

3.5.7 Putting It Together

A robust validation pipeline might look like this:

```
from lark.exceptions import UnexpectedInput
from pycypher.grammar_parser import GrammarParser
from pycypher.semantic_validator import SemanticValidator, ErrorSeverity

def validate_before_run(query: str) -> list[str]:
    """Return a list of human-readable error messages, or [] if valid."""
    parser = GrammarParser()
    issues = []

    # Step 1: syntax check
    try:
        tree = parser.parse(query)
    except UnexpectedInput as e:
        return [f"Syntax error at {e.line}:{e.column}: {e}"]

    # Step 2: semantic check
    validator = SemanticValidator()
    for error in validator.validate(tree):
        if error.severity == ErrorSeverity.ERROR:
            issues.append(error.message)

    return issues
```

3.5.8 Next Steps

- [AST Manipulation](#) — inspect and build AST nodes programmatically
- [Basic Query Execution](#) — execute queries end-to-end
- [Query Processing](#) — detailed pipeline reference

3.6 AST Manipulation

Learn how to create, modify, and traverse AST nodes programmatically using PyCypher’s Pydantic-based model layer.

In this tutorial

- [Prerequisites](#)
- [Overview](#)
- [Parsing a Query to a Typed AST](#)
- [Creating AST Nodes from Scratch](#)
 - [Nodes and Labels](#)
 - [Relationship Patterns](#)

– Expressions

- Traversing the AST
- Pretty-Printing
- Validation
- Common AST Node Types
- Next Steps

3.6.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Familiarity with Cypher query syntax
- Basic knowledge of Pydantic models

3.6.2 Overview

PyCypher represents parsed Cypher queries as a tree of strongly-typed [Pydantic](#) models defined in `pycypher.ast_models`. Every element of the query — nodes, relationships, expressions, clauses — has a corresponding Python class with validated fields.

The typical flow is:

1. Parse a Cypher string with `GrammarParser`
2. Convert the raw Lark tree to typed models with `ASTConverter`
3. Inspect or modify the resulting AST using standard Python attribute access
4. Traverse the tree with `traverse()`

3.6.3 Parsing a Query to a Typed AST

The simplest way to get a typed AST from a Cypher string is the one-liner `from_cypher()`:

```
from pycypher.ast_models import ASTConverter

typed_ast = ASTConverter.from_cypher(
    "MATCH (n:Person) WHERE n.age > 30 RETURN n.name"
)

print(type(typed_ast))
# <class 'pycypher.ast_models.Query'>
```

If you need access to the intermediate Lark parse tree (e.g. for custom transformers or debugging), use the two-step flow instead:

```
from pycypher.grammar_parser import GrammarParser
from pycypher.ast_models import ASTConverter

parser = GrammarParser()
parse_tree = parser.parse("MATCH (n:Person) WHERE n.age > 30 RETURN n.name")
```

(continues on next page)

(continued from previous page)

```

converter = ASTConverter()
typed_ast = converter.convert(
    parser.transformer.transform(parse_tree)
)

```

The returned Query object is the root of a fully validated tree. Every child node carries type information, making IDE auto-complete and static analysis tools work out of the box.

3.6.4 Creating AST Nodes from Scratch

You can construct AST nodes directly without parsing a string. This is useful for code-generation tools, query rewriters, and testing.

Nodes and Labels

```

from pycypher.ast_models import NodePattern, Variable

# A node pattern: (p:Person)
person = NodePattern(
    variable=Variable(name="p"),
    labels=["Person"],
)
print(person.variable.name) # "p"
print(person.labels)       # ["Person"]

# An anonymous node: ()
anon = NodePattern()
print(anon.variable)       # None

```

Relationship Patterns

```

from pycypher.ast_models import (
    RelationshipPattern,
    RelationshipDirection,
    Variable,
)

# -[r:KNOWS]->
rel = RelationshipPattern(
    variable=Variable(name="r"),
    rel_types=["KNOWS"],
    direction=RelationshipDirection.RIGHT,
)
print(rel.direction) # RelationshipDirection.RIGHT

```

Expressions

```

from pycypher.ast_models import (
    Arithmetic,
    Comparison,
    IntegerLiteral,

```

(continues on next page)

(continued from previous page)

```

    PropertyLookup,
    Variable,
)

# n.age > 30
predicate = Comparison(
    operator=">",
    left=PropertyLookup(
        expression=Variable(name="n"),
        property_name="age",
    ),
    right=IntegerLiteral(value=30),
)

# n.score + 10
expr = Arithmetic(
    operator="+",
    left=PropertyLookup(
        expression=Variable(name="n"),
        property_name="score",
    ),
    right=IntegerLiteral(value=10),
)

```

3.6.5 Traversing the AST

Every `ASTNode` subclass exposes a `traverse()` iterator that yields all nodes in the tree in depth-first order:

```

for node in typed_ast.traverse():
    print(f"{node.__class__.__name__}")

```

You can filter by type to find specific constructs:

```

from pycypher.ast_models import NodePattern, PropertyLookup

# Find all node patterns in the query
node_patterns = [
    n for n in typed_ast.traverse()
    if isinstance(n, NodePattern)
]
print(f"Found {len(node_patterns)} node pattern(s)")

# Find all property lookups
lookups = [
    n for n in typed_ast.traverse()
    if isinstance(n, PropertyLookup)
]
for lk in lookups:
    print(f"property: {lk.property_name}")

```

3.6.6 Pretty-Printing

The `pretty()` method produces a human-readable tree representation:

```
print(typed_ast.pretty())
```

This is invaluable for debugging grammar changes and understanding how PyCypher interprets a given Cypher string.

3.6.7 Validation

Because every AST node is a Pydantic model, construction validates field types automatically:

```
from pydantic import ValidationError
from pycypher.ast_models import IntegerLiteral

try:
    bad = IntegerLiteral(value="not_a_number")
except ValidationError as e:
    print(e) # Pydantic reports the type mismatch
```

For semantic validation (undefined variables, aggregation rules, etc.) use [SemanticValidator](#) — see the [Query Validation](#) tutorial for details.

3.6.8 Common AST Node Types

Class	Represents
Query	Top-level query (list of clauses)
Match	MATCH clause with patterns and WHERE
Return	RETURN clause with items and modifiers
With	WITH clause (projection between pipeline stages)
NodePattern	(var:Label {props})
RelationshipPattern	-[var:TYPE]->
PropertyLookup	var.property
Comparison	left op right
Arithmetic	left op right (math)
FunctionInvocation	funcName(args)
Variable	Named variable reference
IntegerLiteral	Integer constant
StringLiteral	String constant

See `pycypher.ast_models` for the complete list.

3.6.9 Next Steps

- [Query Validation](#) — validate queries before execution
- [Basic Query Execution](#) — execute queries end-to-end
- [PyCypher API](#) — full API reference

3.7 Data ETL Pipeline

Build end-to-end ETL pipelines that load data from files or databases, transform it with Cypher queries, and write results to output sinks — using PyCypher’s ingestion layer and the nmetl CLI.

In this tutorial

- [Prerequisites](#)
- [Overview](#)
- [Programmatic Pipeline](#)
 - [Loading Data with ContextBuilder](#)
 - [Quick Setup from a Dict](#)
 - [Supported Data Sources](#)
- [YAML Configuration](#)
 - [Config File Structure](#)
 - [Environment Variables](#)
- [The nmetl CLI](#)
 - [Error Handling](#)
- [Writing Output](#)
- [Security](#)
- [Complete Example](#)
- [Next Steps](#)

3.7.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Sample CSV or Parquet files to work with
- Basic understanding of Cypher queries

3.7.2 Overview

PyCypher ships with a lightweight ETL framework built around three concepts:

1. Data sources — load entities and relationships from files, DataFrames, or SQL databases
2. Cypher queries — transform, filter, and aggregate the loaded graph data
3. Output sinks — write query results to files or external systems

You can drive pipelines programmatically via [ContextBuilder](#) or declaratively via a YAML config file and the nmetl CLI.

3.7.3 Programmatic Pipeline

Loading Data with ContextBuilder

The ContextBuilder provides a fluent API for assembling a query context:

```
from pycypher.ingestion.context_builder import ContextBuilder
from pycypher.star import Star

context = (
    ContextBuilder()
    .add_entity("Person", "people.csv", id_col="person_id")
    .add_entity("Company", "companies.csv", id_col="company_id")
    .add_relationship(
        "WORKS_AT",
        "employment.csv",
        source_col="person_id",
        target_col="company_id",
    )
    .build()
)

star = Star(context=context)
result = star.execute_query(
    "MATCH (p:Person)-[:WORKS_AT]->(c:Company) "
    "RETURN p.name AS employee, c.name AS company"
)
```

The source argument accepts:

- File paths: CSV, Parquet, or JSON (local or remote via s3://, https://)
- pandas DataFrames: for data already in memory
- PyArrow Tables: for zero-copy Arrow integration

Quick Setup from a Dict

For the simplest case — DataFrames keyed by entity type:

```
import pandas as pd
from pycypher.ingestion.context_builder import ContextBuilder

context = ContextBuilder.from_dict({
    "Person": pd.DataFrame({
        "__ID__": [1, 2, 3],
        "name": ["Alice", "Bob", "Carol"],
        "age": [30, 25, 35],
    }),
    "Company": pd.DataFrame({
        "__ID__": [10, 20],
        "name": ["Acme", "Globex"],
    }),
})
```

Supported Data Sources

The `data_source_from_uri()` factory resolves the correct reader based on URI scheme and file extension:

URI pattern	Reader
/path/file.csv, file:///...	CSV via DuckDB
/path/file.parquet	Parquet via DuckDB
/path/file.json	JSON via DuckDB
s3://, gs://, https://	Remote file via DuckDB
postgresql://, mysql://, etc.	SQL query

SQL sources require a query parameter:

```
context = (
    ContextBuilder()
    .add_entity(
        "Person",
        "postgresql://localhost/mydb",
        query="SELECT id, name, age FROM people",
        id_col="id",
    )
    .build()
)
```

3.7.4 YAML Configuration

For repeatable pipelines, define the entire ETL in a YAML file and run it with the `nmetl CLI`.

Config File Structure

```
# pipeline.yaml
entities:
- type: Person
  uri: data/people.csv
  id_col: person_id

- type: Company
  uri: data/companies.parquet

relationships:
- type: WORKS_AT
  uri: data/employment.csv
  source_col: person_id
  target_col: company_id

queries:
- name: employee_report
  cypher: |
    MATCH (p:Person)-[:WORKS_AT]->(c:Company)
    WHERE p.age >= 30
    RETURN p.name AS employee, c.name AS company, p.age AS age
    ORDER BY p.age DESC
```

(continues on next page)

(continued from previous page)

```

output:
  uri: output/senior_employees.csv

- name: company_headcount
  cypher: |
    MATCH (p:Person)-[:WORKS_AT]->(c:Company)
    RETURN c.name AS company, count(p) AS headcount
    ORDER BY headcount DESC
  output:
    uri: output/headcount.parquet

```

Environment Variables

YAML values support `${VAR_NAME}` placeholders that are substituted from the process environment:

```

entities:
- type: Person
  uri: ${DATA_DIR}/people.csv

```

3.7.5 The nmetl CLI

The nmetl command-line tool runs pipelines from YAML configs:

```

# Run a pipeline
uv run nmetl run pipeline.yaml

# Validate config without running
uv run nmetl validate pipeline.yaml

# List all queries in the config
uv run nmetl list-queries pipeline.yaml

# Dry-run: show what would be executed
uv run nmetl run pipeline.yaml --dry-run

```

Error Handling

The CLI translates common errors into user-friendly messages:

- File not found — reports the missing path
- Permission denied — reports the access error
- Parse errors — shows the line and column of the syntax issue
- Validation errors — lists all config problems before attempting to run

3.7.6 Writing Output

Query results can be written to CSV, Parquet, or JSON files via `write_dataframe_to_uri()`:

```

from pycypher.ingestion import write_dataframe_to_uri

```

(continues on next page)

(continued from previous page)

```
write_dataframe_to_uri(result_dataframe, "output/results.csv")
write_dataframe_to_uri(result_dataframe, "output/results.parquet")
```

The output format is inferred from the file extension (.csv, .parquet, .json). Parent directories are created automatically.

3.7.7 Security

The ingestion layer includes built-in security checks (`pycypher.ingestion.security`):

- Path traversal prevention — file paths are sanitized to block `../` escape attempts
- SQL injection protection — SQL queries passed to data sources are validated against injection patterns
- URI scheme validation — only recognised schemes are accepted

3.7.8 Complete Example

```
import pandas as pd
from pycypher.ingestion.context_builder import ContextBuilder
from pycypher.star import Star

# 1. Load data
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "dept": ["Eng", "Sales", "Eng"],
})
knows = pd.DataFrame({
    "__SOURCE__": [1, 2],
    "__TARGET__": [2, 3],
})

context = (
    ContextBuilder()
    .add_entity("Person", people)
    .add_relationship("KNOWS", knows)
    .build()
)

# 2. Query
star = Star(context=context)
result = star.execute_query(
    """
    MATCH (p:Person)-[:KNOWS]->(f:Person)
    RETURN p.name AS person, f.name AS friend, p.dept AS dept
    """
)
print(result)

# 3. Aggregate
headcount = star.execute_query(
    """
```

(continues on next page)

(continued from previous page)

```
MATCH (p:Person)
RETURN p.dept AS department, count(p) AS n
ORDER BY n DESC
"""
)
print(headcount)
```

3.7.9 Next Steps

- Basic Query Execution — deeper dive into query execution
- Pattern Matching — advanced graph pattern techniques
- PyCypher API — full API reference for all ingestion modules

3.8 Integration Guide

Embed PyCypher in real-world applications — web services, data pipelines, batch jobs, and interactive notebooks.

In this tutorial

- Prerequisites
- Core Integration Pattern
- Web API Integration
 - Flask Example
 - FastAPI Example
 - Query Parameters for Safety
- Data Pipeline Integration
 - Batch Processing
 - Mutation Pipelines
 - YAML Pipeline with nmetl
- Jupyter Notebook Integration
- Pre-Execution Validation
- Error Handling Strategy
- Context Refresh
- Production Checklist
- Next Steps

3.8.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Completed [Basic Query Execution and Graph Modeling](#)
- Familiarity with your target integration (Flask/FastAPI, Jupyter, etc.)

3.8.2 Core Integration Pattern

Every PyCypher integration follows the same three steps:

1. Build a Context — load data once
2. Create a Star executor — reuse across queries
3. Execute queries — return DataFrames

```
from pycypher import Star
from pycypher.ingestion import ContextBuilder

# Step 1: Build context (once, at startup)
context = ContextBuilder.from_dict({
    "Person": people_df,
    "KNOWS": knows_df,
})

# Step 2: Create executor (reusable)
star = Star(context=context)

# Step 3: Execute queries (as many as needed)
result = star.execute_query("MATCH (p:Person) RETURN p.name AS name")
```

The Star instance caches parsed ASTs and query results internally. Create it once and reuse it.

3.8.3 Web API Integration

Flask Example

```
from flask import Flask, jsonify, request
from pycypher import Star, QueryTimeoutError, VariableNotFoundError
from pycypher.ingestion import ContextBuilder

app = Flask(__name__)

# Build context at startup
context = ContextBuilder.from_dict(load_your_data())
star = Star(context=context)

@app.route("/query", methods=["POST"])
def run_query():
    query = request.json.get("query", "")
    params = request.json.get("parameters", {})

    try:
        result = star.execute_query(
            query,
```

(continues on next page)

(continued from previous page)

```

        parameters=params,
        timeout_seconds=5.0,
    )
    return jsonify({
        "columns": result.columns.tolist(),
        "rows": result.values.tolist(),
        "count": len(result),
    })
except QueryTimeoutError:
    return jsonify({"error": "Query timed out"}), 408
except VariableNotFoundError as e:
    return jsonify({"error": str(e)}), 400
except Exception as e:
    return jsonify({"error": str(e)}), 500

```

FastAPI Example

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from pycypher import Star, QueryTimeoutError
from pycypher.ingestion import ContextBuilder

app = FastAPI()

context = ContextBuilder.from__dict__(load__your__data())
star = Star(context=context)

class QueryRequest(BaseModel):
    query: str
    parameters: dict = {}
    timeout: float = 5.0

@app.post("/query")
def run_query(req: QueryRequest):
    try:
        result = star.execute_query(
            req.query,
            parameters=req.parameters,
            timeout_seconds=req.timeout,
        )
        return {"columns": result.columns.tolist(), "rows": result.to__dict__("records")}
    except QueryTimeoutError:
        raise HTTPException(408, "Query timed out")
    except Exception as e:
        raise HTTPException(400, str(e))

```

Tip

Always set `timeout_seconds` on user-facing endpoints to prevent malicious or accidental resource exhaustion.

Query Parameters for Safety

Never interpolate user input into query strings. Use query parameters:

```
# SAFE: parameterized query
result = star.execute_query(
    "MATCH (p:Person) WHERE p.name = $name RETURN p.age AS age",
    parameters={"name": user_input},
)

# UNSAFE: string interpolation — never do this
# result = star.execute_query(f"... WHERE p.name = '{user_input}' ...")
```

Parameters are bound safely before execution and support all Cypher types.

3.8.4 Data Pipeline Integration

Batch Processing

Process data in stages, refreshing the context between batches:

```
from pycypher import Star
from pycypher.ingestion import ContextBuilder

def process_batch(batch_df):
    """Process a single batch of incoming data."""
    context = (
        ContextBuilder()
        .add_entity("Event", batch_df, id_col="event_id")
        .add_entity("User", users_df)
        .add_relationship("TRIGGERED", triggered_df,
                        source_col="user_id", target_col="event_id")
        .build()
    )
    star = Star(context=context)

    # Aggregate and filter
    result = star.execute_query(
        """
        MATCH (u:User)-[:TRIGGERED]->(e:Event)
        WHERE e.severity = 'critical'
        RETURN u.name AS user, count(e) AS critical_events
        ORDER BY critical_events DESC
        """
    )
    return result

# Process each batch
for batch in read_batches("events/*.parquet"):
    alerts = process_batch(batch)
    if len(alerts) > 0:
        send_alerts(alerts)
```


Mutation Pipelines

Use Cypher write operations to transform data in-place:

```
# Enrich nodes with computed properties
star.execute_query(
    """
    MATCH (p:Person)-[:KNOWS]->(friend:Person)
    WITH p, count(friend) AS friend_count
    SET p.popularity = friend_count
    """
)

# Create derived relationships
star.execute_query(
    """
    MATCH (a:Person)-[:KNOWS]->(b:Person)-[:KNOWS]->(c:Person)
    WHERE NOT EXISTS { MATCH (a)-[:KNOWS]->(c) }
    AND a <> c
    CREATE (a)-[:FRIEND_OF_FRIEND]->(c)
    """
)
```

YAML Pipeline with nmetl

For declarative pipelines, use the nmetl CLI:

```
# pipeline.yaml
sources:
  Person:
    type: csv
    path: data/people.csv
    id_col: user_id
  KNOWS:
    type: csv
    path: data/relationships.csv
    source_col: from_id
    target_col: to_id

queries:
- name: popular_people
  cypher: |
    MATCH (p:Person)-[:KNOWS]->(f:Person)
    RETURN p.name AS person, count(f) AS friends
    ORDER BY friends DESC LIMIT 10
```

```
uv run nmetl run pipeline.yaml
uv run nmetl query pipeline.yaml "MATCH (p:Person) RETURN count(p)"
```

See Data ETL Pipeline for the full YAML pipeline tutorial.

3.8.5 Jupyter Notebook Integration

PyCypher returns pandas DataFrames, making it naturally compatible with Jupyter's display system:

```
# In a notebook cell:
import pandas as pd
from pycypher import Star
from pycypher.ingestion import ContextBuilder

context = ContextBuilder.from_dict({
    "Person": pd.read_csv("people.csv"),
    "KNOWS": pd.read_csv("relationships.csv"),
})
star = Star(context=context)

# Display as a rich HTML table automatically
star.execute_query("MATCH (p:Person) RETURN p.name, p.age ORDER BY p.age")
```

Combine with plotting libraries:

```
import matplotlib.pyplot as plt

result = star.execute_query(
    """
    MATCH (p:Person)
    RETURN p.dept AS department, count(p) AS headcount
    ORDER BY headcount DESC
    """
)
result.plot.bar(x="department", y="headcount", title="Headcount by Department")
plt.tight_layout()
plt.show()
```

3.8.6 Pre-Execution Validation

In multi-tenant or interactive systems, validate queries before execution to provide fast user feedback:

```
from pycypher import validate_query
from pycypher.semantic_validator import ErrorSeverity

def safe_execute(star, query, **kwargs):
    """Validate, then execute. Returns (result, errors)."""
    errors = validate_query(query)
    blocking = [e for e in errors if e.severity == ErrorSeverity.ERROR]
    if blocking:
        return None, [e.message for e in blocking]
    return star.execute_query(query, **kwargs), []
```

See [Query Validation](#) for the full validation tutorial.

3.8.7 Error Handling Strategy

PyCypher raises specific exception types. Build your error handling around them:

```
from pycypher import (
    Star,
    VariableNotFoundError,
    UnsupportedFunctionError,
    GraphTypeNotFoundError,
    QueryTimeoutError,
    QueryMemoryBudgetError,
)

def execute_safely(star, query, **kwargs):
    """Execute with structured error handling."""
    try:
        return star.execute_query(query, **kwargs)
    except QueryTimeoutError as e:
        log.warning("Timeout after %.1fs: %s", e.elapsed_seconds, query[:80])
        raise
    except QueryMemoryBudgetError as e:
        log.warning("Memory estimate %d MB exceeds budget", e.estimated_bytes // 1e6)
        raise
    except VariableNotFoundError as e:
        log.info("Undefined variable '%s' (available: %s)",
                e.variable_name, e.available_variables)
        raise
    except GraphTypeNotFoundError as e:
        log.info("Unknown entity type '%s'", e.type_name)
        raise
```

3.8.8 Context Refresh

When your source data changes, rebuild the context. The Star executor itself is stateless (beyond caches):

```
# Periodic refresh
def refresh_context():
    new_context = ContextBuilder.from_dict(load_latest_data())
    return Star(context=new_context)

# Use in a long-running service
star = refresh_context()
schedule.every(10).minutes.do(lambda: star := refresh_context())
```

For append-only data, use mutation queries (CREATE, MERGE) to add data without rebuilding the full context.

3.8.9 Production Checklist

Before deploying PyCypher in production:

- [] Set PYCYPHER_QUERY_TIMEOUT_S to prevent runaway queries
- [] Set PYCYPHER_MAX_CROSS_JOIN_ROWS to limit Cartesian explosions
- [] Use query parameters (never string interpolation)

(continues on next page)

(continued from previous page)

```
[ ] Validate untrusted queries before execution
[ ] Handle all exception types in your error handler
[ ] Size the result cache for your workload
[ ] Monitor query metrics via shared.metrics.QUERY_METRICS
[ ] Bound variable-length paths in user-facing queries
```

See [Performance Tuning](#) for detailed tuning guidance and [Deployment & Scaling](#) for container deployment.

3.8.10 Next Steps

- [Performance Tuning](#) — optimize for production workloads
- [Data ETL Pipeline](#) — YAML-driven batch pipelines
- [Query Validation](#) — pre-execution validation
- [PyCypher API](#) — full API reference

3.9 Production Deployment Patterns

Configure PyCypher for production workloads — backend selection, rate limiting, audit logging, and performance tuning for real-world systems.

In this tutorial

- [Prerequisites](#)
- [Backend Selection](#)
- [Rate Limiting](#)
- [Audit Logging](#)
- [Timeouts and Memory Budgets](#)
- [Result Caching](#)
- [Complete Production Checklist](#)
- [Next Steps](#)

3.9.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- Completed [Basic Query Execution and Integration Guide](#)
- Familiarity with environment variable configuration

3.9.2 Backend Selection

PyCypher supports pluggable DataFrame backends. Choose a backend based on your workload characteristics:

Backend	Best for	Trade-offs
pandas (default)	Small-medium datasets, prototyping	Lowest startup cost, widest compatibility
duckdb	Large datasets, analytical queries	2-10x faster joins/aggregations, higher memory efficiency
polars	CPU-bound transformations	Fast columnar operations, requires polars dependency

```

from pycypher import Star
from pycypher.ingestion import ContextBuilder

# Auto-select best available backend
context = ContextBuilder.from_dict({"Person": people_df}).build(backend="auto")

# Explicitly choose DuckDB for analytical workloads
context = ContextBuilder.from_dict({"Person": people_df}).build(backend="duckdb")

star = Star(context=context)
print(f"Using backend: {context.backend_name}")

```

3.9.3 Rate Limiting

Protect shared deployments from query abuse with the built-in rate limiter. Rate limiting is off by default and activated via environment variables:

```

# Allow 50 queries/second with bursts up to 100
export PYCYPHER_RATE_LIMIT_QPS=50
export PYCYPHER_RATE_LIMIT_BURST=100

```

For programmatic control, create a custom limiter:

```

from pycypher.rate_limiter import QueryRateLimiter
from pycypher import RateLimitError

# Per-user rate limiting in a web application
user_limiters = {}

def get_user_limiter(user_id: str) -> QueryRateLimiter:
    if user_id not in user_limiters:
        user_limiters[user_id] = QueryRateLimiter(qps=10.0, burst=20)
    return user_limiters[user_id]

def handle_query(user_id: str, query: str):
    try:
        get_user_limiter(user_id).acquire()
        return star.execute_query(query)
    except RateLimitError:
        raise HTTPError(429, "Too many queries — try again shortly")

```

3.9.4 Audit Logging

Enable structured query audit logging for compliance, debugging, and performance monitoring.

Quick start — enable via environment variable:

```
export PYCYPHER_AUDIT_LOG=1
```

Each query emits a JSON record to the pycypher.audit logger:

```
{
  "query_id": "a1b2c3d4",
  "timestamp": "2026-03-30T12:00:00Z",
  "query": "MATCH (p:Person) WHERE p.age > 25 RETURN p.name",
  "status": "ok",
  "elapsed_ms": 12.3,
  "rows": 42,
  "parameter_keys": []
}
```

Production configuration — route audit logs to a file:

```
import logging

from pycypher.audit import enable_audit_log

# Enable audit logging
enable_audit_log()

# Route to a dedicated file
audit_logger = logging.getLogger("pycypher.audit")
handler = logging.FileHandler("/var/log/pycypher/audit.jsonl")
audit_logger.addHandler(handler)
audit_logger.setLevel(logging.INFO)
```

Security note: parameter values and result data are never logged. Query text is truncated to prevent unbounded log growth.

3.9.5 Timeouts and Memory Budgets

Always set timeouts and memory budgets in production to prevent runaway queries from consuming all resources:

```
from pycypher import Star, QueryTimeoutError, QueryMemoryBudgetError

star = Star(context=context)

try:
    result = star.execute_query(
        user_query,
        timeout_seconds=10.0,
        memory_budget_bytes=500 * 1024 * 1024, # 500 MB
    )
except QueryTimeoutError as e:
    log.warning(f"Query timed out after {e.elapsed_seconds:.1f}s")
```

(continues on next page)

(continued from previous page)

```
except QueryMemoryBudgetError as e:
    log.warning(f"Estimated {e.estimated_bytes / 1e6:.0f}MB exceeds budget")
```

Or set defaults via environment variables:

```
export PYCYPHER_QUERY_TIMEOUT_S=30
export PYCYPHER_MAX_CROSS_JOIN_ROWS=1_000_000
```

3.9.6 Result Caching

PyCypher caches query results automatically. Tune cache size and TTL for your workload:

```
# Large cache with 5-minute TTL for analytical dashboards
star = Star(
    context=context,
    result_cache_max_mb=500,
    result_cache_ttl_seconds=300.0,
)

# Disable caching for mutation-heavy workloads
star = Star(context=context, result_cache_max_mb=0)
```

Monitor cache effectiveness:

```
from pycypher import get_cache_stats

stats = get_cache_stats(star=star)
print(f"Hit rate: {stats['result_cache_hit_rate']:.0%}")
print(f"Size: {stats['result_cache_size_mb']:.1f} MB")
```

3.9.7 Complete Production Checklist

```
# Resource protection
export PYCYPHER_QUERY_TIMEOUT_S=30
export PYCYPHER_MAX_CROSS_JOIN_ROWS=1_000_000
export PYCYPHER_MAX_UNBOUNDED_PATH_HOPS=10

# Caching
export PYCYPHER_RESULT_CACHE_MAX_MB=200
export PYCYPHER_RESULT_CACHE_TTL_S=60

# Monitoring
export PYCYPHER_SLOW_QUERY_MS=500
export PYCYPHER_AUDIT_LOG=1

# Multi-tenant rate limiting
export PYCYPHER_RATE_LIMIT_QPS=50
export PYCYPHER_RATE_LIMIT_BURST=100
```

3.9.8 Next Steps

- [Performance Tuning](#) — detailed performance tuning reference
- [Error Handling Patterns](#) — structured error handling patterns
- [Deployment & Scaling](#) — deployment infrastructure (Docker, scaling)

3.10 ML-Powered Query Optimization

Learn how PyCypher’s online learning system adaptively improves query plans over time without heavyweight ML dependencies.

On this page

- [Learning Objectives](#)
- [Prerequisites](#)
- [Architecture Overview](#)
- [Getting Started](#)
- [Query Fingerprinting](#)
- [Adaptive Plan Caching](#)
 - [Measuring Cache Effectiveness](#)
 - [Cache Configuration](#)
 - [Mutation Invalidation](#)
- [Predicate Selectivity Learning](#)
 - [Using Correction Factors](#)
- [Join Strategy Learning](#)
 - [Size Buckets](#)
 - [Strategy Statistics](#)
- [Diagnostics](#)
 - [Resetting State](#)
- [Configuration Constants](#)
- [Try It Yourself](#)
- [Next Steps](#)

3.10.1 Learning Objectives

After this tutorial you will be able to:

- Use `QueryLearningStore` to access all learning components
- Understand how query fingerprinting enables plan reuse
- Measure cache hit rates and selectivity convergence
- Use correction factors to improve cardinality estimates

- Configure cache TTL and capacity for your workload

3.10.2 Prerequisites

- Completed [Basic Query Execution](#) tutorial
- Basic familiarity with query planning concepts

3.10.3 Architecture Overview

The learning system has five components coordinated by a single facade:

QueryFingerprinter	structural query similarity detection
PredicateSelectivityTracker	learns actual selectivity per predicate
JoinPerformanceTracker	records strategy performance per size bucket
AdaptivePlanCache	caches plans for structurally similar queries
QueryLearningStore	unified facade coordinating all components

All components use exponential moving averages (EMA) and bounded rolling windows rather than heavy-weight ML frameworks. Thread-safe via fine-grained locks.

3.10.4 Getting Started

```
from pycypher.query_learning import QueryLearningStore

# Create a store (or use the module-level singleton)
store = QueryLearningStore()

# Or access the global singleton
from pycypher.query_learning import get_learning_store
store = get_learning_store()
```

3.10.5 Query Fingerprinting

The fingerprinter extracts a query's structural skeleton, stripping literal values so that structurally identical queries share the same fingerprint:

```
from pycypher.ast_converter import ASTConverter
from pycypher.query_learning import QueryLearningStore

store = QueryLearningStore()

q1 = ASTConverter.from_cypher("MATCH (p:Person) WHERE p.age > 30 RETURN p.name")
q2 = ASTConverter.from_cypher("MATCH (p:Person) WHERE p.age > 50 RETURN p.name")

fp1 = store.fingerprint(q1)
fp2 = store.fingerprint(q2)

assert fp1.digest == fp2.digest # Same structure, different literals
```

Each fingerprint contains:

- digest — SHA-256 hex digest (first 16 chars) of the structural representation
- clause_signature — Human-readable clause sequence (e.g. "Match -> Return")

- `entity_types` — Sorted tuple of entity types (e.g. ("Person",))
- `relationship_types` — Sorted tuple of relationship types

3.10.6 Adaptive Plan Caching

Once a query is analyzed, cache the result for structurally similar queries:

```
from pycypher.ast_converter import ASTConverter
from pycypher.query_learning import QueryLearningStore
from pycypher.query_planner import AnalysisResult

store = QueryLearningStore()
query = ASTConverter.from_cypher("MATCH (p:Person) RETURN p.name")
fp = store.fingerprint(query)

# First execution: cache miss
cached = store.get_cached_plan(fp)
assert cached is None

# Cache the analysis result
analysis = AnalysisResult(clause_cardinalities=[100])
store.cache_plan(fp, analysis)

# Subsequent queries with same structure: cache hit
cached = store.get_cached_plan(fp)
assert cached is not None
```

Measuring Cache Effectiveness

```
stats = store.plan_cache.stats
print(f"Entries: {stats['entries']}")
print(f"Hits: {stats['hits']}")
print(f"Misses: {stats['misses']}")
print(f"Hit rate: {stats['hit_rate']:.1%}")
```

Cache Configuration

The cache uses LRU eviction with configurable capacity and TTL:

```
from pycypher.query_learning import AdaptivePlanCache

# Custom cache: 128 entries, 60-second TTL
cache = AdaptivePlanCache(max_entries=128, ttl_seconds=60.0)
```

Default settings:

- `max_entries=256` — Maximum cached plans before LRU eviction
- `ttl_seconds=300.0` — Plans expire after 5 minutes

LRU eviction — When the cache is full, the least-recently-used entry is evicted. Accessing an entry (via `get`) updates its recency.

TTL expiry — Entries older than `ttl_seconds` are not returned (treated as cache misses) and are removed on access.

Mutation Invalidation

After data mutations (CREATE, SET, DELETE), cached plans may be stale:

```
# After executing a mutation
store.invalidate_on_mutation()

# Plan cache is cleared; selectivity and join data are preserved
```

3.10.7 Predicate Selectivity Learning

The selectivity tracker learns the actual filtering ratio of predicates using exponential moving averages, giving more weight to recent observations:

```
from pycypher.query_learning import PredicateSelectivityTracker

tracker = PredicateSelectivityTracker()

# Record observations after each query execution
for actual in [0.12, 0.18, 0.14, 0.16, 0.13, 0.15]:
    tracker.record("Person", "age", ">", estimated=0.33, actual=actual)

# After >= 3 observations, learned value is available
learned = tracker.get_learned_selectivity("Person", "age", ">")
print(f"Learned selectivity: {learned:.3f}") # ~0.14, near actual mean
```

Using Correction Factors

Correction factors let you adjust heuristic estimates toward learned values:

```
# Record enough observations
for _ in range(5):
    tracker.record("Person", "age", ">", estimated=0.33, actual=0.12)

# Get multiplicative correction: learned / heuristic
factor = tracker.correction_factor("Person", "age", ">", heuristic=0.33)
# factor ~ 0.12 / 0.33 ~ 0.36

# Apply: improved_estimate = heuristic * factor
improved = 0.33 * factor # ~0.12, much closer to reality
```

Key behaviors:

- Returns 1.0 when insufficient data (< 3 observations)
- Clamped to [0.01, 100.0] to prevent extreme corrections
- Operators are normalized (case-insensitive, whitespace-stripped)

3.10.8 Join Strategy Learning

The join tracker records execution time per strategy and size bucket, enabling the planner to select the historically fastest approach:

```

from pycypher.query_learning import QueryLearningStore

store = QueryLearningStore()

# Record: hash join is slower for this size pair
for _ in range(5):
    store.record_join_performance(
        strategy="hash", left_rows=5000, right_rows=3000,
        actual_output_rows=2500, elapsed_ms=30.0,
    )

# Record: merge join is faster
for _ in range(5):
    store.record_join_performance(
        strategy="merge", left_rows=5000, right_rows=3000,
        actual_output_rows=2500, elapsed_ms=10.0,
    )

best = store.get_best_join_strategy(5000, 3000)
assert best == "merge"

```

Size Buckets

Row counts are bucketed for strategy lookup:

Bucket	Row Range	Example
tiny	<= 100	In-memory lookups
small	<= 10,000	Small tables
medium	<= 1,000,000	Mid-size datasets
large	> 1,000,000	Warehouse-scale

Strategy Statistics

Get detailed per-strategy metrics for a size bucket:

```

stats = store.join_tracker.strategy_stats(5000, 3000)
for strategy, metrics in stats.items():
    print(f"{strategy}: avg={metrics['avg_ms']:.1f}ms, "
          f"count={int(metrics['count'])}, "
          f"accuracy={metrics['output_accuracy']:.2f}")

```

3.10.9 Diagnostics

Get a snapshot of all learning component state:

```

diag = store.diagnostics()
print(diag)
# {
#   'plan_cache': {'entries': 12, 'max_entries': 256, 'hits': 45,
#                  'misses': 12, 'hit_rate': 0.789},
#   'selectivity_patterns': 8,

```

(continues on next page)

(continued from previous page)

```
# 'join_buckets_tracked': 3,
# }
```

Resetting State

Clear all learning data (useful for testing or after schema changes):

```
store.clear()
```

3.10.10 Configuration Constants

The learning system uses these tuning constants (defined at module level):

3.10.11 Try It Yourself

1. Create a `QueryLearningStore` and fingerprint several queries with different literal values but the same structure. Verify they share a fingerprint.
2. Record 10 selectivity observations with varying actual values and watch the EMA converge. Plot the convergence if you have `matplotlib`.
3. Record join performance for two strategies at different size buckets. Verify `best_strategy` returns the faster one.
4. Set up a cache with `max_entries=2` and observe LRU eviction behavior.

3.10.12 Next Steps

- [Performance Tuning](#) — Environment variables and runtime tuning
- [Production Deployment Patterns](#) — Backend selection for production workloads
- [PyCypher API](#) — Full API reference for `query_learning` module

3.11 Troubleshooting Queries

Diagnose and fix common query issues — parse errors, missing data, performance problems, and unexpected results.

In this guide

- [Prerequisites](#)
- [Diagnostic Tools](#)
 - [Enable Debug Logging](#)
 - [Validate Before Executing](#)
 - [Check Available Functions](#)
 - [Inspect the Parsed AST](#)
 - [Query Metrics](#)
- [Parse Errors](#)

- “Syntax error” or UnexpectedInput
- Missing or Empty Results
 - Query returns empty DataFrame
 - Property returns NULL unexpectedly
 - Unexpected extra rows
- Performance Issues
 - Query is slow
 - Memory usage is high
- Exception Reference
- Try It Yourself
- Next Steps

3.11.1 Prerequisites

- PyCypher installed (see [Getting Started](#))
- A working Star instance (see [Basic Query Execution](#))

3.11.2 Diagnostic Tools

Enable Debug Logging

The single most useful diagnostic step:

```
import logging
logging.getLogger("pycypher").setLevel(logging.DEBUG)
```

This prints parse timing, clause-by-clause execution, row counts, and memory usage for every query.

Validate Before Executing

Catch errors early without running the full query:

```
from pycypher import validate_query

errors = validate_query("MATCH (n:Person) RETURN m")
for e in errors:
    print(f"{e.severity.value}: {e.message}")
# ERROR: Variable 'm' is used but not defined
```

Check Available Functions

```
print(star.available_functions())
# ['abs', 'acos', 'asin', ..., 'valuetype', 'values']
```

Inspect the Parsed AST

```
from pycypher.ast_models import ASTConverter

ast = ASTConverter.from__cypher("MATCH (p:Person) RETURN p.name")
for clause in ast.clauses:
    print(type(clause).__name__, clause)
```

Query Metrics

```
from shared.metrics import QUERY_METRICS

stats = QUERY_METRICS.snapshot()
print(f"Queries: {stats.total_queries}, Errors: {stats.error_rate:.1%}")
print(f"p50: {stats.timing_p50_ms:.0f}ms, p99: {stats.timing_p99_ms:.0f}ms")
```

3.11.3 Parse Errors

“Syntax error” or UnexpectedInput

Symptom: `lark.exceptions.UnexpectedInput` when parsing.

Common causes:

1. Missing closing parenthesis:

```
MATCH (p:Person WHERE p.age > 30 RETURN p.name
#           ^ missing closing )
```

2. Missing relationship direction arrow:

```
MATCH (a)-[:KNOWS]-(b)  # undirected — OK
MATCH (a)-[:KNOWS](b)   # missing arrow — ERROR
```

3. Using SQL keywords instead of Cypher:

```
# SQL style (wrong)
SELECT p.name FROM Person p WHERE p.age > 30

# Cypher style (correct)
MATCH (p:Person) WHERE p.age > 30 RETURN p.name
```

4. Unquoted strings in WHERE:

```
WHERE p.name = Alice  # ERROR — unquoted
WHERE p.name = 'Alice' # correct
```

Fix: Use `GrammarParser.validate()` for a quick syntax check:

```
from pycypher.grammar_parser import GrammarParser
print(GrammarParser().validate("MATCH (p:Person) RETURN p")) # True/False
```

3.11.4 Missing or Empty Results

Query returns empty DataFrame

Checklist:

1. Entity type registered? Check your ContextBuilder call:

```
# If your data uses "User" but your query says "Person":
MATCH (p:Person) ... # returns nothing

# Fix: use the same label
MATCH (u:User) ...
```

2. IDs match between entities and relationships?

```
# Entity IDs are integers
people = pd.DataFrame({"__ID__": [1, 2, 3], ...})

# But relationship sources are strings — no matches!
knows = pd.DataFrame({"__SOURCE__": ["1", "2"], "__TARGET__": ["2", "3"]})

# Fix: ensure consistent types
knows["__SOURCE__"] = knows["__SOURCE__"].astype(int)
knows["__TARGET__"] = knows["__TARGET__"].astype(int)
```

3. Relationship direction wrong?

```
# Data: Alice -> Bob (source=Alice, target=Bob)
# Query: who does Bob know?
MATCH (b:Person {name: 'Bob'})-[:KNOWS]->(other)
# Returns nothing because Bob is the TARGET, not SOURCE

# Fix: reverse the direction
MATCH (other)-[:KNOWS]->(b:Person {name: 'Bob'})
```

4. WHERE filter too restrictive? Remove the WHERE clause temporarily to see if the MATCH alone produces results.

Property returns NULL unexpectedly

Cause: Property name mismatch (case-sensitive).

```
# DataFrame has "Name" but query uses "name"
MATCH (p:Person) RETURN p.name # NULL — wrong case
MATCH (p:Person) RETURN p.Name # correct
```

Fix: Check your DataFrame column names:

```
print(people_df.columns.tolist())
# ['__ID__', 'Name', 'age'] ← note capital N
```


Unexpected extra rows

Cause: Usually a cross-join from unconnected MATCH patterns:

```
# This produces |Person| × |Product| rows (Cartesian product)
MATCH (p:Person), (pr:Product) RETURN p.name, pr.title

# Fix: connect them with a relationship
MATCH (p:Person)-[:BOUGHT]->(pr:Product) RETURN p.name, pr.title
```

3.11.5 Performance Issues

Query is slow

Step 1: Enable debug logging to identify the slow clause.

Step 2: Check for these common causes:

Pattern	Fix
Unbounded path <code>-[*]-></code>	Add bounds: <code>-[*1..3]-></code>
Cross-join MATCH (a), (b)	Use relationship: (a)-[:REL]->(b)
Wide RETURN with many properties	Select only needed properties
Missing WHERE on large scan	Add early filter: WHERE p.active

Step 3: Use the query profiler for detailed timing:

```
from pycypher.query_profiler import QueryProfiler

profiler = QueryProfiler(star)
report = profiler.profile("MATCH (p:Person)-[:KNOWS*]->(q) RETURN p.name, q.name")
print(report)
print(f"Hotspot: {report.hotspot}")
print(f"Suggestions: {report.recommendations}")
```

Step 4: Set a timeout to prevent runaway queries:

```
result = star.execute_query(query, timeout_seconds=5.0)
```

See [Performance Tuning](#) for comprehensive tuning guidance.

Memory usage is high

Cause: Large intermediate results from cross-joins or unbounded paths.

Fix:

```
# Set a memory budget
result = star.execute_query(query, memory_budget_bytes=500 * 1024 * 1024)

# Or limit cross-join size globally
import os
os.environ["PYCYPHER_MAX_CROSS_JOIN_ROWS"] = "1000000"
```

3.11.6 Exception Reference

Exception	Meaning and fix
VariableNotFoundError	Variable used but never bound in MATCH/WITH. Check spelling and scope.
UnsupportedFunctionError	Function name not recognized. Use <code>star.available_functions()</code> to see valid names.
GraphTypeNotFoundError	Entity label not registered in the context. Check your <code>ContextBuilder</code> calls.
QueryTimeoutError	Query exceeded its time budget. Simplify the query or increase the timeout.
QueryMemoryBudgetError	Estimated memory exceeds the budget. Add filters or increase the budget.
UnexpectedInput (Lark)	Cypher syntax error. Check parentheses, quotes, and keyword spelling.

3.11.7 Try It Yourself

Exercise: The following query returns no results. Find and fix the bug:

```
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
})
knows = pd.DataFrame({
    "__SOURCE__": ["1", "2"],
    "__TARGET__": ["2", "3"],
})
context = ContextBuilder.from_dict({"Person": people, "KNOWS": knows})
star = Star(context=context)

result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS]->(b:Person) RETURN a.name, b.name"
)
print(result) # Empty!
```

3.11.8 Next Steps

- [Query Validation](#) — pre-execution validation
- [Performance Tuning](#) — production tuning
- [Troubleshooting](#) — Docker and infrastructure issues

3.12 Learning Pathway

The tutorials are organized into three tiers. Start at your level and work forward:

Beginner — Your first graph queries

0. [Cypher for SQL Users](#) — Know SQL but not Cypher? Side-by-side translation guide covering `SELECT`→`RETURN`, `JOIN`→relationships, `GROUP BY`, subqueries, and more. Read in 10 minutes.
1. [Basic Query Execution](#) — Load data, execute `MATCH`/`WHERE`/`RETURN`, sort and aggregate results.

2. [Graph Modeling](#) — Design your tabular data as entities and relationships. Modeling patterns, ID strategies, common pitfalls.

Intermediate — Real-world graph analysis

3. [Pattern Matching](#) — Variable-length paths, `shortestPath`, `OPTIONAL MATCH`, pattern comprehensions, quantifier predicates, `EXISTS` subqueries.
4. [Query Validation](#) — Pre-execution syntax and semantic validation. Error severity levels, custom validation pipelines.
5. [AST Manipulation](#) — Programmatic AST inspection, traversal, and construction using Pydantic models.

Advanced — Production systems

6. [Data ETL Pipeline](#) — `ContextBuilder`, YAML pipelines, the `nmetl` CLI, data sources, and output sinks.
7. [Integration Guide](#) — Embed PyCypher in web APIs, batch jobs, and notebooks. Query parameters, error handling, context refresh patterns.
8. [Production Deployment Patterns](#) — Backend selection, rate limiting, audit logging, timeouts, and caching for production workloads.
9. [ML-Powered Query Optimization](#) — ML-powered adaptive query optimization with fingerprinting, selectivity learning, and join strategy selection.
10. [Troubleshooting Queries](#) — Diagnose parse errors, missing results, performance problems, and unexpected output.

Tip

Each tutorial builds on earlier ones. The beginner tutorials establish the vocabulary and patterns that intermediate and advanced tutorials assume.

3.13 Prerequisites

- Python 3.14 or higher (free-threaded build recommended)
- PyCypher packages installed (see [Getting Started](#))
- Basic familiarity with pandas DataFrames

Each tutorial includes:

- Clear learning objectives
- Complete, runnable code examples
- “Try it yourself” exercises
- Common pitfalls and solutions

3.14 Related Resources

- [Getting Started](#) — Installation and quick-start examples
- [User Guide](#) — In-depth reference guides
- [Performance Tuning](#) — Production optimization

- [PyCypher API](#) — Full API reference

3.14.1 Example Scripts

The `examples/` directory contains runnable scripts that complement these tutorials. Run any script with `uv run python examples/<script>.py`:

- `quickstart.py` — Minimal end-to-end query (pairs with [Basic Query Execution](#))
- `advanced_grammar_examples.py` — Complex Cypher patterns including variable-length paths, comprehensions, and REDUCE (pairs with [Pattern Matching](#))
- `functions_in_where.py` — Using scalar functions inside WHERE predicates
- `scalar_functions_in_with.py` — Transforming data with functions in WITH clauses
- `ast_conversion_example.py` — Programmatic AST inspection and traversal (pairs with [AST Manipulation](#))
- `multi_query_composition.py` — Multi-query ETL pipelines with cross-query dependencies (pairs with [Data ETL Pipeline](#))
- `backend_selection.py` — Choosing between pandas, DuckDB, and Polars backends (pairs with [Production Deployment Patterns](#))
- `production_patterns.py` — Rate limiting, audit logging, timeouts, and caching for production workloads (pairs with [Production Deployment Patterns](#))

Complete API reference for all PyCypher packages. Each module's public classes, functions, and constants are documented with type signatures, parameter descriptions, return values, and usage examples extracted from source docstrings.

4.1 PyCypher API

The PyCypher package provides comprehensive openCypher query parsing, AST processing, and BindingFrame-based query execution against in-memory DataFrames.

4.1.1 Quick Start

```
import pandas as pd
from pycypher import ContextBuilder, Star

# Build a graph context from DataFrames
people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})
context = ContextBuilder().add_entity("Person", people).build()

# Execute a Cypher query
star = Star(context=context)
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 28 RETURN p.name, p.age ORDER BY p.age"
)
print(result)
```

```
# Pre-execution validation
from pycypher import validate_query

errors = validate_query("MATCH (n:Person) RETURN m")
for error in errors:
    print(f"{error.severity.value}: {error.message}")
```

```
# Backend selection (auto, pandas, duckdb, polars)
context = ContextBuilder().add_entity("Person", people).build(backend="auto")
print(context.backend_name) # "pandas", "duckdb", or "polars"
```

4.1.2 Core Modules

AST Models

Pydantic-based AST node definitions for the openCypher grammar. Every parsed query is represented as a tree of these immutable model classes. Pydantic models for openCypher AST nodes.

This package provides strongly-typed Pydantic models for the Abstract Syntax Tree generated by the grammar parser. It includes utilities for converting dict-based ASTs to typed models, traversing, modifying, and pretty-printing the AST.

Important Notes:

- All variable references are represented as Variable instances (not strings)
- This includes variables in patterns (NodePattern, RelationshipPattern), comprehensions (ListComprehension, PatternComprehension), quantifiers, and all other binding contexts
- The ASTConverter automatically wraps string variable names in Variable instances during conversion

Usage:

```
from pycypher.grammar_parser import GrammarParser
from pycypher.ast_models import ASTConverter

parser = GrammarParser()
parse_tree = parser.parse("MATCH (n:Person) RETURN n")
raw_ast = parser.transformer.transform(parse_tree)

converter = ASTConverter()
typed_ast = converter.convert(raw_ast)

# Traverse the AST
for node in typed_ast.traverse():
    print(f"Node type: {node.__class__.__name__}")

# Pretty print
print(typed_ast.pretty())

# Access variable names
from pycypher.ast_models import NodePattern, Variable
node = NodePattern(variable=Variable(name="n"), labels=["Person"])
print(node.variable.name) # "n"
```

The module is split into four domain submodules for maintainability:

- pycypher.ast_models.core — Base classes, enums, validation framework
- pycypher.ast_models.clauses — Query structure, clauses, and patterns
- pycypher.ast_models.expressions — All expression types
- pycypher.ast_models.validation — Validation functions and utilities

class pycypher.ast_models.ASTNode

Bases: [BaseModel](#), [ABC](#)

Base class for all AST nodes.

clone()

Create a deep copy of this node.

cypher_validate()

Validate this AST node and return any issues found.

Runs a suite of validators to check for: - Undefined variable references - Unused variables - Missing labels (performance issues) - Unreachable conditions - Type mismatches - And other common query anti-patterns

Returns

ValidationResult containing all issues found

Return type

[ValidationResult](#)

Example

```
>>> result = query.cypher_validate()
>>> if not result.is_valid:
...     print(result)
```

find_all(predicate)

Find all nodes matching a predicate or type.

Parameters

predicate ([type](#) | Callable[[[ASTNode](#)], bool]) – Either a type to match or a callable predicate function

Returns

List of matching nodes

Return type

[list](#)[[ASTNode](#)]

find_first(predicate)

Find the first node matching a predicate or type.

Parameters

predicate ([type](#) | Callable[[[ASTNode](#)], bool]) – Either a type to match or a callable predicate function

Returns

First matching node or None

Return type

[ASTNode](#) | None

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [\[ConfigDict\]\[pydantic.config.ConfigDict\]](#).

`pretty(indent=0)`

Pretty print the AST node.

Parameters

`indent` (`int`) – Current indentation level

Returns

Formatted string representation

Return type

`str`

`to_dict()`

Convert back to dictionary representation.

`traverse(depth=0)`

Traverse the AST depth-first, yielding all nodes.

Parameters

`depth` (`int`) – Current depth in the tree (used internally).

Yields

`ASTNode` – Each node in the tree.

Raises

`SecurityError` – If traversal exceeds `MAX_QUERY_NESTING_DEPTH`.

`validate_ast(strict=False)`

Validate this AST node and its children.

Parameters

`strict` (`bool`) – If True, treat warnings as errors.

Returns

`ValidationResult` containing verification details.

Return type

`ValidationResult`

`class pycypher.ast_models.Algebraizable`

Bases: `ASTNode`

AST node that can be translated into a relational algebra operator.

Subclasses (`PatternPath`, `NodePattern`, `RelationshipPattern`, `PatternIntersection`) implement a `to_relation()` method consumed by the STAR translator to build the query execution plan.

`model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.ast_models.JoinType(*values)`

Bases: `StrEnum`

Enumeration of supported SQL join types.

`INNER`

Inner join - returns only matching rows from both tables.

LEFT

Left outer join - returns all rows from left table.

RIGHT

Right outer join - returns all rows from right table.

FULL

Full outer join - returns all rows from both tables.

INNER = 'INNER'

LEFT = 'LEFT'

RIGHT = 'RIGHT'

FULL = 'FULL'

class pycypher.ast_models.RelationshipDirection(*values)

Bases: [StrEnum](#)

Enumeration of relationship directions in graph patterns.

LEFT

Left-directed relationship (e.g., <-[REL]-).

RIGHT

Right-directed relationship (e.g., -[:REL]->).

UNDIRECTED

Undirected relationship (e.g., -[:REL]-).

LEFT = '<-'

RIGHT = '->'

UNDIRECTED = '-'

class pycypher.ast_models.ValidationIssue(severity=None, message=None, node=None, code=None, *
(Keyword-only parameters separator (PEP 3102)),
node_type=None, suggestion=None)

Bases: [BaseModel](#)

A single validation issue found in the AST.

__init__(severity=None, message=None, node=None, code=None, **kwargs)

Initialize ValidationIssue with positional or keyword arguments.

__repr__()

String representation for debugging.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to
[ConfigDict][pydantic.config.ConfigDict].

severity: [ValidationSeverity](#)

message: [str](#)

node_type: [Any](#) | [None](#)

suggestion: [Any](#) | [None](#)

node: [Any](#) | [None](#)

code: [str](#) | [None](#)

class pycypher.ast_models.ValidationResult(*, issues=<factory>)

Bases: [BaseModel](#)

Result of validating an AST.

`__bool__()`

True if validation passed (no errors).

`add_error(message, node_type=None, suggestion=None, node=None, code=None)`

Add an error-level validation issue.

Convenience wrapper around `add_issue()` that sets severity to [ValidationSeverity.ERROR](#). Errors indicate problems that make the query invalid and must be resolved before execution.

Parameters

- message ([str](#)) – Human-readable description of the validation error.
- node_type ([str](#) | [None](#)) – Optional AST node type name where the error occurred.
- suggestion ([str](#) | [None](#)) – Optional suggested fix or corrective action.
- node ([Any](#) | [None](#)) – Optional reference to the AST node that triggered the error.
- code ([str](#) | [None](#)) – Optional machine-readable issue identifier for programmatic handling.

`add_info(message, node_type=None, suggestion=None, node=None, code=None)`

Add an informational validation issue.

Convenience wrapper around `add_issue()` that sets severity to [ValidationSeverity.INFO](#). Informational issues are non-actionable observations about query structure, useful for debugging or optimization hints.

Parameters

- message ([str](#)) – Human-readable description of the observation.
- node_type ([str](#) | [None](#)) – Optional AST node type name relevant to the observation.
- suggestion ([str](#) | [None](#)) – Optional suggested improvement or optimization.
- node ([Any](#) | [None](#)) – Optional reference to the AST node related to the observation.
- code ([str](#) | [None](#)) – Optional machine-readable issue identifier for programmatic handling.

`add_issue(severity, message, node_type=None, suggestion=None, node=None, code=None)`

Add a validation issue with the given severity.

General-purpose method for recording validation issues. For convenience, use `add_error`, `add_warning`, or `add_info` to add issues at a predetermined severity level.

Parameters

- `severity` ([ValidationSeverity](#)) – The severity level of the issue.
- `message` (`str`) – Human-readable description of the validation problem.
- `node_type` (`str` | `None`) – Optional AST node type name where the issue occurred.
- `suggestion` (`str` | `None`) – Optional suggested fix or corrective action.
- `node` (`Any` | `None`) – Optional reference to the AST node that triggered the issue.
- `code` (`str` | `None`) – Optional machine-readable issue identifier for programmatic handling.

`add_warning(message, node_type=None, suggestion=None, node=None, code=None)`

Add a warning-level validation issue.

Convenience wrapper around `add_issue()` that sets severity to [ValidationSeverity.WARNING](#). Warnings indicate potential problems that do not prevent execution but may produce unexpected results.

Parameters

- `message` (`str`) – Human-readable description of the validation warning.
- `node_type` (`str` | `None`) – Optional AST node type name where the warning occurred.
- `suggestion` (`str` | `None`) – Optional suggested fix or corrective action.
- `node` (`Any` | `None`) – Optional reference to the AST node that triggered the warning.
- `code` (`str` | `None`) – Optional machine-readable issue identifier for programmatic handling.

`property errors: list[ValidationIssue]`

Get only error-level issues.

`property has_errors: bool`

Check if there are any error-level issues.

`property has_warnings: bool`

Check if there are any warning-level issues.

`property infos: list[ValidationIssue]`

Get only info-level issues.

`property is_valid: bool`

Check if AST is valid (no errors).

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`property warnings: list[ValidationIssue]`

Get only warning-level issues.

issues: `list[ValidationIssue]`

class `pycypher.ast_models.ValidationSeverity(*values)`

Bases: `StrEnum`

Severity levels for validation issues.

`ERROR = 'error'`

`WARNING = 'warning'`

`INFO = 'info'`

`pycypher.ast_models.random_hash()`

Generate a cryptographically secure random hash string for column naming.

Creates a unique identifier using cryptographically secure random bytes and SHA256 hashing. Used to generate collision-resistant column names during algebraic operations.

Returns

A 64-character hexadecimal hash string (SHA256 digest).

Return type

`str`

Security:

- Uses `secrets.token_bytes()` for cryptographically secure randomness
- Uses SHA256 instead of broken MD5 hash function
- 32 bytes of entropy provide 2^{256} possible values

class `pycypher.ast_models.AddAllPropertiesItem(*, variable, property=None, expression=None, labels=<factory>, properties)`

Bases: `SetItem`

SET n += {map} item.

`model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

variable: `Variable`

properties: `Expression`

class `pycypher.ast_models.Call(*, procedure_name=None, arguments=<factory>, yield_items=<factory>, where=None)`

Bases: `Clause`

CALL clause – invokes a registered procedure.

Calls the procedure identified by `procedure_name` with arguments, and optionally binds selected output columns via `yield_items`. A where filter may further restrict the yielded rows.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
procedure_name: str | None
```

```
arguments: list[Expression]
```

```
yield_items: list[YieldItem]
```

```
where: Expression | None
```

```
class pycypher.ast_models.Clause
```

```
Bases: ASTNode, ABC
```

Abstract base for Cypher query clauses.

A clause is a top-level statement component (MATCH, RETURN, WITH, CREATE, etc.) that reads or modifies the binding table during query execution.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
class pycypher.ast_models.Create(*, pattern=None)
```

```
Bases: Clause
```

CREATE clause – inserts new nodes and relationships into the graph.

Creates the entities described by pattern. If a variable in the pattern is already bound, the existing entity is reused; otherwise a new entity is created with any specified labels and properties.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
pattern: Pattern | None
```

```
class pycypher.ast_models.Delete(*, detach=False, expressions=<factory>)
```

```
Bases: Clause
```

DELETE clause – removes nodes or relationships from the graph.

Deletes the entities referenced by expressions. When detach is True (DETACH DELETE), any relationships connected to deleted nodes are automatically removed first.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
detach: bool
```

```
expressions: list[Expression]
```

```
class pycypher.ast_models.Foreach(*, variable=None, list_expression=None, clauses=<factory>)
```

Bases: [Clause](#)

FOREACH clause – iterative list mutation.

Iterates over every element of `list_expression`, binds it to `variable`, and executes the inner clauses (SET, CREATE, MERGE, DELETE, REMOVE) for each element. This is a side-effect-only construct; `variable` does not escape into the outer query scope.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

variable: [str](#) | [None](#)

list_expression: [Expression](#) | [None](#)

clauses: [list](#)[[Clause](#)]

```
class pycypher.ast_models.Match(*, optional=False, pattern=None, where=None)
```

Bases: [Clause](#)

MATCH clause – binds graph patterns to the binding table.

Searches the graph for subgraphs matching pattern and adds one row per match. When `optional` is True (OPTIONAL MATCH), rows with no match are retained with NULL-filled columns. An optional where expression filters matches before they enter the binding table.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

optional: [bool](#)

pattern: [Pattern](#) | [None](#)

where: [Expression](#) | [None](#)

```
class pycypher.ast_models.Merge(*, pattern=None, on_create=None, on_match=None)
```

Bases: [Clause](#)

MERGE clause – creates a pattern only if it does not already exist.

Performs an upsert: if pattern matches existing data, the match is bound; otherwise the pattern is created. Optional `on_create` and `on_match` SET actions run conditionally depending on whether a new entity was created.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

pattern: [Pattern](#) | [None](#)

on_create: [list](#)[[SetItem](#)] | [None](#)

`on_match: list[SetItem] | None`

class `pycypher.ast_models.NodePattern(*, variable=None, labels=<factory>, properties={})`

Bases: `Algebraizable`

Node pattern in MATCH/CREATE clauses.

Represents a node pattern like `(n:Person {name: 'Alice'})` in Cypher queries.

`variable`

Variable instance representing the node's binding name (e.g., `Variable(name="n")`)

Type

`Variable | None`

`labels`

List of label names applied to this node

Type

`list[str]`

`properties`

Property map as dict (optional)

Type

`dict[str, Any]`

Example

```
>>> node = NodePattern(
...     variable=Variable(name="person"),
...     labels=["Person"],
...     properties={"name": "Alice"}
... )
>>> print(node.variable.name) # "person"
```

`model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`variable: Variable | None`

`labels: list[str]`

`properties: dict[str, Any]`

class `pycypher.ast_models.OrderByItem(*, expression=None, ascending=True, nulls_placement=None)`

Bases: `ASTNode`

ORDER BY item.

`nulls_placement` controls where null values appear in the sorted output: "first" puts nulls before non-nulls; "last" (or None) puts them after – matching Neo4j's default NULLS LAST behaviour.

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

expression: Expression | None

ascending: bool

nulls_placement: str | None

class pycypher.ast_models.PathLength(*, min=None, max=None, unbounded=False)
    Bases: ASTNode
    Variable-length path specification.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

    min: int | None

    max: int | None

    unbounded: bool

class pycypher.ast_models.Pattern(*, paths=<factory>)
    Bases: ASTNode
    Pattern containing path components.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

    paths: list[PatternPath]

class pycypher.ast_models.PatternIntersection(*, pattern_list)
    Bases: Algebraizable
    Intersection of multiple Patterns, implicit Join on shared variables.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

    pattern_list: list[Algebraizable]

class pycypher.ast_models.PatternPath(*, variable=None, elements=<factory>,
                                       shortest_path_mode='none')
    Bases: Algebraizable
    A single path in a pattern.

    Represents a complete path pattern that may have an optional binding variable. For example: p =
    (a)-[r]->(b)

```


variable

Variable instance for path binding (e.g., `Variable(name="p")`), optional

Type

`Variable` | `None`

elements

List of `NodePattern` and `RelationshipPattern` elements forming the path

Type

`list[NodePattern | RelationshipPattern]`

Example

```
>>> path = PatternPath(
...     variable=Variable(name="path"),
...     elements=[
...         NodePattern(variable=Variable(name="a")),
...         RelationshipPattern(variable=Variable(name="r")),
...         NodePattern(variable=Variable(name="b"))
...     ]
... )
```

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

variable: `Variable` | `None`

elements: `list[NodePattern | RelationshipPattern]`

`shortest_path_mode`: `str`

"none" for normal paths, "one" for `shortestPath`, "all" for `allShortestPaths`.

class `pycypher.ast_models.Query(*, clauses=<factory>)`

Bases: `ASTNode`

Root node of a parsed Cypher query.

Contains the ordered list of clauses that make up the query pipeline (e.g., `MATCH -> WHERE -> WITH -> RETURN`). Clauses are executed sequentially, with each clause consuming and producing a binding table.

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

clauses: `list[Clause]`

class `pycypher.ast_models.RelationshipPattern(*, variable=None, labels=<factory>, properties=None, direction, length=None, where=None)`

Bases: `Algebraizable`

Relationship pattern in `MATCH/CREATE` clauses.

Represents a relationship pattern like `-[r:KNOWS]->` in Cypher queries.

variable

Optional Variable instance representing the relationship's binding name (e.g., `Variable(name="r")`)

Type

`Variable` | `None`

labels

List of relationship type names

Type

`list[str]`

properties

Property map as dict (optional)

Type

`dict[str, Any]` | `None`

direction

Relationship direction token from `RelationshipDirection` ("`<-`", "`>-`", or "`-`")

Type

`RelationshipDirection`

length

`PathLength` specification for variable-length relationships (optional)

Type

`PathLength` | `None`

where

WHERE condition for relationship patterns (optional)

Type

`Expression` | `None`

Example

```
>>> rel = RelationshipPattern(
...     variable=Variable(name="knows"),
...     labels=["KNOWS"],
...     direction=RelationshipDirection.RIGHT
... )
>>> print(rel.variable.name) # "knows"
```

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

variable: `Variable` | `None`

labels: `list[str]`

properties: `dict[str, Any]` | `None`

direction: `RelationshipDirection`

length: [PathLength](#) | [None](#)

where: [Expression](#) | [None](#)

class pycypher.ast_models.Remove(*, items=<factory>)

Bases: [Clause](#)

REMOVE clause – removes properties or labels from existing entities.

Contains a list of [RemoveItem](#) entries specifying which properties or labels to strip from the referenced variables.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

items: list[[RemoveItem](#)]

class pycypher.ast_models.RemoveItem(*, variable=None, property=None, labels=<factory>)

Bases: [ASTNode](#)

Item in REMOVE clause.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

variable: [Variable](#) | [None](#)

property: [str](#) | [None](#)

labels: list[[str](#)]

class pycypher.ast_models.Return(*, distinct=False, items=<factory>, order_by=None, skip=None, limit=None)

Bases: [Clause](#)

RETURN clause – specifies the query output.

Projects expressions from the binding table into the result set, analogous to SQL SELECT. Supports DISTINCT deduplication, ORDER BY sorting, and SKIP/LIMIT pagination.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

distinct: [bool](#)

items: list[[ReturnItem](#)]

order_by: list[[OrderByItem](#)] | [None](#)

skip: [Any](#) | [None](#)

Integer literal or an [Expression](#) AST node (e.g. Parameter).

limit: Any | None

Integer literal or an [Expression](#) AST node (e.g. [Parameter](#)).

class pycypher.ast_models.ReturnAll

Bases: [ASTNode](#)

RETURN * or WITH *.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

class pycypher.ast_models.ReturnItem(*, expression=None, alias=None)

Bases: [ASTNode](#)

Item in RETURN or WITH clause.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

expression: [Expression](#) | None

alias: str | None

class pycypher.ast_models.Set(*, items=<factory>)

Bases: [Clause](#)

SET clause – modifies properties or labels on existing entities.

Contains a list of [SetItem](#) subclasses that specify individual property assignments, label additions, or bulk property replacements. All mutations within a single SET are applied atomically per row.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

items: list[[SetItem](#)]

class pycypher.ast_models.SetAllPropertiesItem(*, variable, property=None, expression=None, labels=<factory>, properties)

Bases: [SetItem](#)

SET n = {map} item.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

variable: [Variable](#)

properties: [Expression](#)

```
class pycypher.ast_models.SetItem(*, variable=None, property=None, expression=None,
                                labels=<factory>)
```

Bases: [ASTNode](#)

Base class for items in SET clause.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
variable: Variable | None
```

```
property: str | None
```

```
expression: Expression | None
```

```
labels: list[str]
```

```
class pycypher.ast_models.SetLabelsItem(*, variable, property=None, expression=None, labels)
```

Bases: [SetItem](#)

SET n:Label item.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
variable: Variable
```

```
labels: list[str]
```

```
class pycypher.ast_models.SetPropertyItem(*, variable, property, expression=None, labels=<factory>,
                                          value)
```

Bases: [SetItem](#)

SET n.property = value item.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
variable: Variable
```

```
property: str
```

```
value: Expression
```

```
class pycypher.ast_models.UnionQuery(*, statements=<factory>, all_flags=<factory>)
```

Bases: [ASTNode](#)

Two or more queries joined by UNION [ALL].

statements is the ordered list of component [Query](#) objects. all_flags is a parallel list of booleans – all_flags[i] is True when the join between statements[i] and statements[i+1] uses UNION ALL (no deduplication).

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

statements: list[Query]

all_flags: list[bool]

```

```
class pycypher.ast_models.Unwind(*, expression=None, alias=None)
```

Bases: [Clause](#)

UNWIND clause – explodes a list into one row per element.

Evaluates expression (which must yield a list) and produces one output row per element, binding each element to alias.

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

expression: Expression | None

alias: str | None

```

```
class pycypher.ast_models.With(*, distinct=False, items=<factory>, where=None, order_by=None,
                               skip=None, limit=None)
```

Bases: [Clause](#)

WITH clause – intermediate projection and scope boundary.

Acts as a pipeline stage between clauses, projecting selected expressions into the downstream binding table. Supports the same modifiers as RETURN (DISTINCT, ORDER BY, SKIP, LIMIT) plus an optional WHERE filter. Variables not projected by WITH are not visible to subsequent clauses.

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

distinct: bool

items: list[ReturnItem]

where: Expression | None

order_by: list[OrderByItem] | None

skip: Any | None
    Integer literal or an Expression AST node (e.g. Parameter).

limit: Any | None
    Integer literal or an Expression AST node (e.g. Parameter).

```

```
class pycypher.ast_models.YieldItem(*, variable=None, alias=None)
```

Bases: [ASTNode](#)

YIELD item in CALL clause.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

variable: [Variable](#) | [None](#)

alias: [str](#) | [None](#)

```
class pycypher.ast_models.AllShortestPaths(*, pattern=None)
```

Bases: [Expression](#)

ALLSHORTESTPATHS function.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

pattern: [Pattern](#) | [None](#)

```
class pycypher.ast_models.And(*, operator='AND', left=None, right=None, operands=<factory>)
```

Bases: [BinaryExpression](#)

Logical AND – true when all operands are true (Kleene three-valued logic).

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

operands: [list](#)[[Expression](#)]

operator: [str](#)

```
class pycypher.ast_models.Arithmetic(*, operator, left=None, right=None)
```

Bases: [BinaryExpression](#)

Arithmetic expression (+, -, *, /, %, ^).

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

```
class pycypher.ast_models.BinaryExpression(*, operator, left=None, right=None)
```

Bases: [Expression](#), [ABC](#)

Abstract base for expressions with a left operand, operator, and right operand.

Logical operators (Or, And, Xor) additionally use an operands list to represent chains of the same operator without deep nesting.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
operator: str
```

```
left: Expression | None
```

```
right: Expression | None
```

```
class pycypher.ast_models.BooleanLiteral(*, value)
```

```
Bases: Literal
```

```
Boolean literal.
```

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
value: bool
```

```
class pycypher.ast_models.CaseExpression(*, expression=None, when_clauses=<factory>,
                                         else_expr=None)
```

```
Bases: Expression
```

```
CASE expression – conditional value selection.
```

When expression is set (simple form), each WHEN condition is compared to it for equality. When expression is None (searched form), each WHEN condition is evaluated as a boolean. The first matching WHEN returns its result; else_expr provides a fallback (NULL if absent).

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
expression: Expression | None
```

```
when_clauses: list[WhenClause]
```

```
else_expr: Expression | None
```

```
class pycypher.ast_models.Comparison(*, operator, left=None, right=None)
```

```
Bases: BinaryExpression
```

```
Comparison expression (=, <>, <, >, <=, >=).
```

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
class pycypher.ast_models.CountStar
```

```
Bases: Expression
```

```
COUNT(*) function.
```



```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
class pycypher.ast_models.Exists(*, content=None)
```

Bases: [Expression](#)

EXISTS subquery – tests whether a pattern or subquery produces any rows.

Returns True if content (a [Pattern](#) or [Query](#)) matches at least one result, False otherwise. Evaluated using batch semantics in the binding evaluator.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
content: Pattern | Query | None
```

```
class pycypher.ast_models.Expression
```

Bases: [ASTNode](#), [ABC](#)

Abstract base for all Cypher expression nodes.

Expressions produce values when evaluated against a binding table row. Subclasses cover literals, variables, operators, property access, function calls, and advanced constructs (comprehensions, quantifiers, CASE, REDUCE).

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
class pycypher.ast_models.FloatLiteral(*, value)
```

Bases: [Literal](#)

Float literal.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
```

```
value: float
```

```
class pycypher.ast_models.FunctionInvocation(*, name, arguments=None, distinct=False)
```

Bases: [Expression](#)

Scalar or aggregation function call (e.g., toUpper(n.name), count(*)).

The name field may be a plain string or a namespace dict for qualified calls like date.truncate. Use the [function_name](#) property to get the normalised string form for registry lookup.

property function `_name`: `str`

Return the normalised function name as a plain string.

`name` may be either a bare string (e.g. `"toUpper"`) or a namespace dict produced by the grammar for qualified calls such as `date.truncate` (represented as `{"namespace": "date", "name": "truncate"}`). This property returns the qualified name `"date.truncate"` so the scalar-function registry can look it up directly.

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`name`: `str | dict[str, str]`

`arguments`: `dict[str, Any] | list[ASTNode | None] | None`

`distinct`: `bool`

class `pycypher.ast_models.IndexLookup(*, expression=None, index=None)`

Bases: `Expression`

Array/string indexing (e.g., `list[0]`).

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`expression`: `Expression | None`

`index`: `Expression | None`

class `pycypher.ast_models.IntegerLiteral(*, value)`

Bases: `Literal`

Integer literal.

`model_config`: `ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`value`: `int`

class `pycypher.ast_models.LabelPredicate(*, operand=None, labels=[])`

Bases: `Expression`

Label predicate check: `n:Person` or `n:Person:Employee`.

Returns `True` when the node bound to `operand` has all the specified labels, `False` otherwise. In the current single-label entity model, `n:A:B` (two different labels) always returns `False`.

`operand`

The expression being tested (typically a `Variable`).

Type

`Expression | None`

labels

One or more label names that must all match.

Type
list[str]

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

operand: Expression | None

labels: list[str]

```
class pycypher.ast_models.ListComprehension(*, variable=None, list_expr=None, where=None,
                                             map_expr=None)
```

Bases: Expression

List comprehension expression: [x IN list WHERE pred | expr].

Represents Cypher list comprehension syntax for transforming lists.

variable

Variable instance for the iteration variable (e.g., Variable(name="x"))

Type
Variable | None

list_expr

Expression that evaluates to a list

Type
Expression | None

where

Optional filter predicate

Type
Expression | None

map_expr

Optional transformation expression

Type
Expression | None

Example

```
>>> # [x IN [1,2,3] WHERE x > 1 | x * 2]
>>> comp = ListComprehension(
...     variable=Variable(name="x"),
...     list_expr=ListLiteral(value=[1,2,3]),
...     where=Comparison(operator=">", left=Variable(name="x"), right=IntegerLiteral(value=1)),
...     map_expr=Arithmetic(operator="*", left=Variable(name="x"),
... ↪right=IntegerLiteral(value=2))
... )
```

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].
variable: Variable | None

list_expr: Expression | None

where: Expression | None

map_expr: Expression | None

class pycypher.ast_models.ListLiteral(*, value=<factory>, elements=<factory>)
    Bases: Literal
    List literal.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].
    value: list[Any]
    elements: list[Expression]

class pycypher.ast_models.Literal(*, value)
    Bases: Expression, ABC
    Base class for literal values.

    evaluate()
        Evaluate the literal to its value.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].
    value: Any

class pycypher.ast_models.MapElement(*, property=None, expression=None, all_properties=False)
    Bases: ASTNode
    Element in map projection.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].
    property: str | None
    expression: Expression | None
    all_properties: bool

```

```
class pycypher.ast_models.MapLiteral(*, value=<factory>, entries=<factory>)
```

Bases: [Literal](#)

Map literal.

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

```
value: dict[str, Any]
```

```
entries: dict[str, Expression]
```

```
class pycypher.ast_models.MapProjection(*, variable=None, elements=<factory>, include_all=False)
```

Bases: [Expression](#)

Map projection expression: node{.prop, computed: expr}.

Projects a map/object with selected or computed properties.

variable

Variable instance for the source object (e.g., `Variable(name="node")`)

Type

[Variable](#) | None

elements

List of MapElement items defining projections

Type

`list[MapElement]`

include_all

Whether to include all properties (*. syntax)

Type

`bool`

Example

```
>>> # person{.name, .age, adult: person.age >= 18}
>>> proj = MapProjection(
...     variable=Variable(name="person"),
...     elements=[
...         MapElement(property="name"),
...         MapElement(property="age"),
...         MapElement(property="adult", expression=Comparison(...))
...     ]
... )
```

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

```
variable: Variable | None
```

elements: list[MapElement]

include_all: bool

class pycypher.ast_models.Not(*, operand=None)

Bases: Expression

Logical NOT – negates its operand (Kleene three-valued logic: NOT NULL = NULL).

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

operand: Expression | None

class pycypher.ast_models.NullCheck(*, operator, operand=None)

Bases: Expression

IS NULL or IS NOT NULL check.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

operator: str

operand: Expression | None

class pycypher.ast_models.NullLiteral(*, value=None)

Bases: Literal

NULL literal.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

value: None

class pycypher.ast_models.Or(*, operator='OR', left=None, right=None, operands=<factory>)

Bases: BinaryExpression

Logical OR – true when any operand is true (Kleene three-valued logic).

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

operands: list[Expression]

operator: str

```
class pycypher.ast_models.Parameter(*, name)
```

Bases: [Expression](#)

Query parameter (\$param).

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

name: [str](#)

```
class pycypher.ast_models.PatternComprehension(*, variable=None, pattern=None, where=None,
                                              map_expr=None)
```

Bases: [Expression](#)

Pattern comprehension expression: [path = pattern WHERE pred | expr].

Represents Cypher pattern comprehension for matching and transforming graph patterns.

variable

Variable instance for the path binding (e.g., `Variable(name="path")`)

Type

[Variable](#) | None

pattern

Graph pattern to match

Type

[Pattern](#) | None

where

Optional filter predicate

Type

[Expression](#) | None

map_expr

Optional transformation expression

Type

[Expression](#) | None

Example

```
>>> # [p = (a)-[:KNOWS]->(b) WHERE b.age > 30 | b.name]
>>> comp = PatternComprehension(
...     variable=Variable(name="p"),
...     pattern=Pattern(...),
...     where=Comparison(...),
...     map_expr=PropertyLookup(...)
... )
```

```
model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

variable: [Variable](#) | [None](#)

pattern: [Pattern](#) | [None](#)

where: [Expression](#) | [None](#)

map_expr: [Expression](#) | [None](#)

class pycypher.ast_models.PropertyLookup(*, expression=None, property=None, variable=None)

Bases: [Expression](#)

Property access (e.g., n.name).

Deprecated since version The: variable field is deprecated. Use expression instead.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

model_post_init(__PropertyLookup__context)

Emit deprecation warning when legacy variable field is used.

expression: [Expression](#) | [None](#)

property: [str](#) | [None](#)

variable: [Variable](#) | [None](#)

class pycypher.ast_models.Quantifier(*, quantifier, variable=None, list_expr=None, where=None)

Bases: [Expression](#)

Quantifier expression: ALL, ANY, NONE, SINGLE.

Tests whether a predicate holds for elements in a list.

quantifier

Type of quantifier (“ALL”, “ANY”, “NONE”, “SINGLE”)

Type

[str](#)

variable

Variable instance for the iteration variable (e.g., Variable(name=”x”))

Type

[Variable](#) | [None](#)

list_expr

Expression that evaluates to a list

Type

[Expression](#) | [None](#)

where

Predicate expression to test

Type

[Expression](#) | [None](#)

Examples

```

>>> # ALL(x IN [1,2,3] WHERE x > 0)
>>> all_expr = Quantifier(
...     quantifier="ALL",
...     variable=Variable(name="x"),
...     list_expr=ListLiteral(value=[1,2,3]),
...     where=Comparison(operator=">", left=Variable(name="x"), right=IntegerLiteral(value=0))
... )
>>>
>>> # ANY(x IN list WHERE x = value)
>>> any_expr = Quantifier(
...     quantifier="ANY",
...     variable=Variable(name="x"),
...     list_expr=Variable(name="list"),
...     where=Comparison(operator="=", left=Variable(name="x"), right=Variable(name="value"))
... )

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

quantifier: str

variable: Variable | None

list_expr: Expression | None

where: Expression | None

class pycypher.ast_models.Reduce(*, accumulator=None, initial=None, variable=None, list_expr=None, map_expr=None)

Bases: Expression

REDUCE expression for list aggregation.

Iterates over a list, accumulating values using a custom expression.

accumulator

Variable instance for the accumulator (e.g., Variable(name="sum"))

Type

Variable | None

initial

Initial value expression for the accumulator

Type

Expression | None

variable

Variable instance for the iteration variable (e.g., Variable(name="x"))

Type

Variable | None

`list_expr`
 Expression that evaluates to a list
 Type
 Expression | None

`map_expr`
 Expression to compute new accumulator value
 Type
 Expression | None

Example

```
>>> # REDUCE(sum = 0, x IN [1,2,3] | sum + x)
>>> reduce_expr = Reduce(
...     accumulator=Variable(name="sum"),
...     initial=IntegerLiteral(value=0),
...     variable=Variable(name="x"),
...     list_expr=ListLiteral(value=[1,2,3]),
...     map_expr=Arithmetic(operator="+", left=Variable(name="sum"), right=Variable(name="x
...     ))
... )
```

`model_config`: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
 Configuration for the model, should be a dictionary conforming to
 [ConfigDict][pydantic.config.ConfigDict].

`accumulator`: Variable | None

`initial`: Expression | None

`variable`: Variable | None

`list_expr`: Expression | None

`map_expr`: Expression | None

class pycypher.ast_models.ShortestPath(*, pattern=None)

Bases: Expression

SHORTESTPATH function.

`model_config`: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
 Configuration for the model, should be a dictionary conforming to
 [ConfigDict][pydantic.config.ConfigDict].

`pattern`: Pattern | None

class pycypher.ast_models.Slicing(*, expression=None, start=None, end=None)

Bases: Expression

Array/string slicing (e.g., list[1..3]).

```

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

expression: Expression | None

start: Expression | None

end: Expression | None

class pycypher.ast_models.StringLiteral(*, value)
    Bases: Literal
    String literal.

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

    value: str

class pycypher.ast_models.StringPredicate(*, operator, left=None, right=None)
    Bases: BinaryExpression
    String predicate (STARTS WITH, ENDS WITH, CONTAINS, =~, IN).

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

class pycypher.ast_models.Unary(*, operator, operand=None)
    Bases: Expression
    Unary expression (+, -).

    model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

    operator: str

    operand: Expression | None

class pycypher.ast_models.Variable(*, name)
    Bases: Expression
    Variable reference in Cypher queries.

    Represents a variable name used in expressions, patterns, and bindings. Variables are used throughout
    the AST to reference nodes, relationships, paths, and other values.

```

name

The variable name (e.g., 'n', 'person', 'rel')

Type

str

Note

As of the latest refactoring, all variable references in the AST (including in patterns, comprehensions, and other binding contexts) are represented as `Variable` instances, not plain strings. This ensures consistency throughout the AST structure.

Examples

```
>>> # In patterns
>>> node = NodePattern(variable=Variable(name="n"), labels=["Person"])
>>>
>>> # In expressions
>>> return_item = ReturnItem(expression=Variable(name="n"))
>>>
>>> # In comprehensions
>>> comp = ListComprehension(variable=Variable(name="x"), ...)
```

__hash__()

Hash based on variable name.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

name: str

class pycypher.ast_models.WhenClause(*, condition=None, result=None)

Bases: `ASTNode`

WHEN clause in CASE expression.

model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

condition: `Expression` | `None`

result: `Expression` | `None`

class pycypher.ast_models.Xor(*, operator='XOR', left=None, right=None, operands=<factory>)

Bases: `BinaryExpression`

Logical XOR – true when exactly one operand is true (Kleene three-valued logic).

`model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`
 Configuration for the model, should be a dictionary conforming to
`[ConfigDict][pydantic.config.ConfigDict]`.

`operands: list[Expression]`

`operator: str`

`pycypher.ast_models.convert_ast(raw_ast)`

Convert a dictionary-based AST to typed Pydantic models.

Parameters

`raw_ast (Any)` – Dictionary or other value from grammar parser

Returns

Typed `ASTNode` or `None`

Return type

`ASTNode` | `None`

Example

```
>>> from pycypher.grammar_parser import GrammarParser
>>> parser = GrammarParser()
>>> raw_ast = parser.parse_to_ast("MATCH (n) RETURN n")
>>> typed_ast = convert_ast(raw_ast)
>>> print(typed_ast.pretty())
```

`pycypher.ast_models.extract_referenced_variables(node)`

Extract all variable names referenced in an AST expression.

Uses the AST's built-in `find_all(Variable)` visitor to collect variable names. This traversal is complete (handles all AST node types) and depth-limited (raises `SecurityError` if nesting exceeds the configured maximum).

For `PropertyLookup` nodes (e.g. `p.name`), extracts the base variable name (`p`).

This is the canonical variable extraction function – all call sites that need variable names from expression ASTs should use this function rather than implementing ad-hoc traversal.

Parameters

`node (ASTNode)` – An AST node (typically a `WHERE` predicate or expression).

Returns

Set of variable name strings.

Return type

`set[str]`

`pycypher.ast_models.find_nodes(node, node_type)`

Find all nodes of a specific type.

Parameters

- `node (ASTNode)` – Root AST node
- `node_type (type)` – Type to search for

Returns

List of matching nodes

Return type
list[ASTNode]

Example

```
>>> matches = find_nodes(typed_ast, Match)
>>> print(f"Found {len(matches)} MATCH clauses")
```

pycypher.ast_models.print_ast(node, indent=0)

Pretty print an AST.

Parameters

- node (ASTNode) – AST node to print
- indent (int) – Initial indentation level

Example

```
>>> print_ast(typed_ast)
```

pycypher.ast_models.traverse_ast(node)

Traverse an AST depth-first.

Parameters

node (ASTNode) – Root AST node

Yields

Each node in the tree

Example

```
>>> for ast_node in traverse_ast(typed_ast):
...     print(ast_node.__class__.__name__)
```

pycypher.ast_models.validate_ast(node, strict=False)

Validate an AST for common issues and anti-patterns.

This function performs comprehensive validation including: - Undefined variable detection - Unreachable code detection - Performance anti-patterns - Type consistency checks

Parameters

- node (ASTNode) – Root AST node to validate
- strict (bool) – If True, treat warnings as errors

Returns

ValidationResult containing all issues found

Return type

ValidationResult

Example

```
>>> result = validate_ast(typed_ast)
>>> if not result.is_valid:
...     print(result)
```

(continues on next page)

(continued from previous page)

```
...     for error in result.errors:
...         print(f"Error: {error.message}")
```

Grammar Parser

Lark-based Cypher parser with LRU caching of parsed grammars and AST trees. openCypher Grammar Parser using Lark.

This module implements a parser for the openCypher query language based on the grammar specification in `grammar.bnf`. It uses the Lark parsing library to parse Cypher queries into abstract syntax trees (ASTs).

The parser uses a CompositeTransformer architecture with specialized transformers (LiteralTransformer, ExpressionTransformer, PatternTransformer, StatementTransformer) that follow the Single Responsibility Principle, replacing the previous monolithic CypherASTTransformer design.

This is an alternative implementation to `cypher_parser.py`, providing a more comprehensive grammar coverage directly based on the BNF specification.

Example

```
>>> from pycypher.grammar_parser import GrammarParser
>>> parser = GrammarParser()
>>> query = "MATCH (n:Person {name: 'Alice'}) RETURN n"
>>> tree = parser.parse(query)
>>> print(tree.pretty())
```

```
class pycypher.grammar_parser.GrammarParser(debug=False)
```

Bases: `object`

Parser for openCypher queries based on the BNF grammar specification.

This class provides a high-level interface for parsing Cypher queries into abstract syntax trees (ASTs) using the Lark parsing library.

parser

The Lark parser instance.

Type

`lark.lark.Lark`

transformer

The AST transformer instance.

Type

`pycypher.grammar_transformers.CompositeTransformer`

Example

```
>>> parser = GrammarParser()
>>> query = "MATCH (n:Person) RETURN n.name"
>>> tree = parser.parse(query)
>>> ast = parser.parse_to_ast(query)
```

```

__init__(debug=False)
    Initialize the grammar parser.

    Parameters
        debug (bool) – If True, enable debug mode for more verbose parsing errors.

parser: Lark

transformer: CompositeTransformer

parse(query)
    Parse a Cypher query into a parse tree.

    Parameters
        query (str) – The Cypher query string to parse.

    Returns
        The Lark parse tree.

    Return type
        Tree

    Raises
        • CypherSyntaxError – If the query has syntax errors.
        • SecurityError – If the query exceeds the size limit.

parse_to_ast(query)
    Parse a Cypher query into an AST.

    Results are cached by query string so that repeated parsing of the same query (common in ETL
    pipelines) returns instantly.

    Parameters
        query (str) – The Cypher query string to parse.

    Returns
        The abstract syntax tree as a dictionary.

    Return type
        Dict

    Raises
        CypherSyntaxError – If the query has syntax errors.

property cache_stats: dict[str, Any]
    Return cache hit/miss statistics for this parser instance.

    Returns
        ast_hits, ast_misses, ast_size, ast_max_size, ast_evictions, ast_hit_rate,
        lark_hits, lark_misses.

    Return type
        Dict with keys

parse_file(filepath)
    Parse a Cypher query from a file.

    Parameters
        filepath (str | Path) – Path to the file containing the Cypher query.

```


Returns

The Lark parse tree.

Return type

Tree

Raises

- `FileNotFoundError` – If filepath does not exist.
- `IsADirectoryError` – If filepath is a directory.
- `ValueError` – If the resolved path is not a regular file.

`parse_file_to_ast(filepath)`

Parse a Cypher query from a file into an AST.

Parameters

`filepath` (`str` | `Path`) – Path to the file containing the Cypher query.

Returns

The abstract syntax tree as a dictionary.

Return type

Dict

`validate(query)`

Validate a Cypher query without returning the parse tree.

Returns True when query is syntactically valid Cypher. Returns False only for genuine Lark parse errors (`UnexpectedInput` and its subclasses: `UnexpectedToken`, `UnexpectedCharacters`, `UnexpectedEOF`).

Any other exception (e.g. a `VisitError` wrapping an internal transformer bug, or a plain `AttributeError`) is not caught here and propagates to the caller. This ensures that internal bugs surface immediately rather than being silently masked as “invalid query” responses.

Parameters

`query` (`str`) – The Cypher query string to validate.

Returns

True if the query is syntactically valid, False if it contains a genuine parse error.

Return type

`bool`

Raises

- `lark.exceptions.VisitError` – If the grammar transformer raises an internal error while processing the parse tree.
- `Exception` – Any other non-parse exception raised during parsing.

`pycypher.grammar_parser.get_default_parser(*, debug=False)`

Return a cached GrammarParser instance.

The Lark grammar compilation step inside `GrammarParser.__init__` is expensive (~0.175 s per call). This factory builds the parser exactly once per unique debug flag value and returns the same object on every subsequent call, eliminating per-query re-instantiation overhead.

Parameters

`debug` (`bool`) – Passed through to `GrammarParser.__init__`. Separate instances are cached for `debug=True` and `debug=False`.

Returns

A fully-initialised, reusable parser instance.

Return type

[GrammarParser](#)

Grammar Transformers

Visitor classes that convert Lark parse trees into PyCypher AST models. Specialized AST transformers for openCypher grammar parsing.

This module contains focused transformer classes that replace the monolithic CypherASTTransformer with specialized, single-responsibility transformers.

Each transformer inherits from a grammar-rule mixin that provides the production implementations of all transformer methods for its domain:

- **LiteralTransformer**: Handles literal values (strings, numbers, booleans, lists, maps)
- **ExpressionTransformer**: Handles expressions (arithmetic, logical, comparisons, predicates)
- **FunctionTransformer**: Handles function calls, CASE, comprehensions, reduce, quantifiers
- **PatternTransformer**: Handles graph patterns (nodes, relationships, paths)
- **StatementTransformer**: Handles Cypher statements and clauses (MATCH, RETURN, etc.)

```
class pycypher.grammar_transformers.LiteralTransformer(visit_tokens=True)
```

Bases: [LiteralRulesMixin](#), Transformer

Transforms Lark parse-tree nodes for literal values into Python objects.

Handles number literals (signed/unsigned, hex, octal, float, inf, NaN), string literals (with escape-sequence processing), boolean literals, null, lists, maps, parameters, and variable names.

All methods inherited from [LiteralRulesMixin](#).

```
class pycypher.grammar_transformers.ExpressionTransformer(visit_tokens=True)
```

Bases: [ExpressionRulesMixin](#), Transformer

Transforms Lark parse-tree nodes for expressions into AST dicts.

Handles arithmetic operators (+, -, *, /, %, ^), unary operators, comparison expressions, boolean logic (AND, OR, NOT, XOR), string predicates, null predicates, property lookups, index lookups, slicing, EXISTS, and inline pattern predicates.

All methods inherited from [ExpressionRulesMixin](#).

```
class pycypher.grammar_transformers.FunctionTransformer(visit_tokens=True)
```

Bases: [FunctionRulesMixin](#), Transformer

Transforms Lark parse-tree nodes for function calls and comprehensions.

Handles function invocation, CASE expressions, list comprehensions, pattern comprehensions, REDUCE, quantifier expressions (ALL, ANY, NONE, SINGLE), and map projections.

All methods inherited from [FunctionRulesMixin](#).

```
class pycypher.grammar_transformers.PatternTransformer(visit_tokens=True)
```

Bases: [PatternRulesMixin](#), [Transformer](#)

Transforms Lark parse-tree nodes for graph patterns into AST dicts.

Handles node patterns, relationship patterns, path patterns, label expressions, inline property maps, path lengths, and shortest path.

All methods inherited from [PatternRulesMixin](#).

```
class pycypher.grammar_transformers.StatementTransformer(visit_tokens=True)
```

Bases: [ClauseRulesMixin](#), [Transformer](#)

Transforms Lark parse-tree nodes for Cypher statements and clauses.

Handles the top-level query structure (MATCH, RETURN, WITH, WHERE, ORDER BY, SKIP, LIMIT, UNION, CREATE, MERGE, DELETE, SET, REMOVE, FOREACH, UNWIND, CALL) and produces the typed dict AST consumed by [ASTConverter](#).

All methods inherited from [ClauseRulesMixin](#).

```
class pycypher.grammar_transformers.CompositeTransformer
```

Bases: [Transformer](#)

Composite transformer that delegates to specialized transformers.

This class maintains the same interface as the original monolithic [CypherASTTransformer](#) but delegates method calls to focused, specialized transformer instances. Each delegate inherits all production methods from its corresponding grammar-rule mixin, so no fallback is needed.

Method resolution is cached on the instance after first lookup to avoid repeated iteration through delegates. This also eliminates per-call logging that caused `Rich OSError: bad file descriptor` during multi-threaded AST cache warmup, since `Rich's Console` is not thread-safe.

```
__init__()
```

Initialize composite transformer with specialized delegates.

```
__getattr__(name)
```

Delegate method calls to appropriate specialized transformer.

Resolved methods are cached on the instance (via `__dict__`) so that `__getattr__` is only invoked once per method name. This eliminates repeated delegate iteration and all per-call logging, preventing `Rich OSError` during concurrent access.

```
transform(tree)
```

Transform a parse tree to AST using specialized transformers.

Grammar Rule Mixins

Modular mixin classes that compose the grammar transformer. Each mixin handles a specific grammar rule group (expressions, clauses, patterns, etc.). Mixin classes for CypherASTTransformer rule groups.

Splits the monolithic 178-method CypherASTTransformer into focused, composable mixin classes. Each mixin groups methods for a related set of grammar rules. CypherASTTransformer inherits from all mixins, preserving Lark’s method-name-based visitor pattern via Python MRO.

Architecture

LiteralRulesMixin	— number, string, boolean, null, list, map, parameter
ExpressionRulesMixin	— operators, boolean/comparison/arithmetic/string/null expressions
FunctionRulesMixin	— function invocation, CASE, list/pattern comprehension, reduce, quantifiers, ↪
↪ map projection	
PatternRulesMixin	— node/relationship patterns, labels, properties, path lengths
ClauseRulesMixin	— MATCH, RETURN, WITH, SET, DELETE, CREATE, MERGE, UNION, ↪
↪ ORDER BY, etc.	
CypherASTTransformer(ClauseRulesMixin, PatternRulesMixin, FunctionRulesMixin, ↪	
↪ ExpressionRulesMixin, LiteralRulesMixin, Transformer)	
grammar string + GrammarParser (grammar_parser.py)	

```
class pycypher.grammar__rule__mixins.ClauseRulesMixin
```

Bases: `object`

Grammar rule methods for Cypher clauses and statement structure.

Handles: - Top-level query structure (cypher_query, statement_list, query_statement) - UNION operations - Read clauses (MATCH, OPTIONAL MATCH, UNWIND) - Write clauses (CREATE, MERGE, DELETE, SET, REMOVE, FOREACH) - CALL/YIELD for procedure invocation - RETURN and WITH clauses (items, aliases, DISTINCT) - WHERE clause - ORDER BY, SKIP, LIMIT modifiers

```
add_all_properties_item(args)
```

Transform property map merge (e.g., SET n += {age: 30}).

Parameters

args (`list[Any]`) – [variable_name, expression]

Returns

Dict with type “AddAllProperties” for additive property merge.

Return type

`dict[str, Any]`

```
call_clause(args)
```

Transform a CALL clause within a larger query.

Parameters

args (`list[Any]`) – [procedure_reference, explicit_args, yield_clause]

Returns

Dict with type “CallClause” containing procedure info, args, and yield.

Return type

`dict[str, Any]`

```
call_statement(args)
```

Transform a standalone CALL statement for procedure invocation.

Parameters

args (`list[Any]`) – [procedure_reference, optional explicit_args, optional yield_clause]

Returns

Dict with type “CallStatement” containing procedure info, args, and yield.

Return type

`dict[str, Any]`

`create_clause(args)`

Transform a CREATE clause for creating new graph elements.

Parameters

args (`list[Any]`) – Pattern to create.

Returns

Dict with type “CreateClause” containing the creation pattern.

Return type

`dict[str, Any]`

`cypher_query(args)`

Transform the root query node.

Parameters

args (`list[Any]`) – List of statement nodes from the statement_list rule.

Returns

Dict with type “Query” or “UnionQuery” containing all statements.

Return type

`dict[str, Any]`

`delete_clause(args)`

Transform a DELETE clause for removing graph elements.

Parameters

args (`list[Any]`) – [optional “DETACH” keyword, delete_items]

Returns

Dict with type “DeleteClause” containing detach flag and items to delete.

Return type

`dict[str, Any]`

`delete_items(args)`

Transform comma-separated list of expressions to delete.

Parameters

args (`list[Any]`) – Expression nodes identifying items to delete.

Returns

Dict containing list of items.

Return type

`dict[str, list[Any]]`

`detach_keyword(args)`

Transform detach_keyword rule.

`distinct_keyword(args)`

Transform `distinct_keyword` rule.

`explicit_args(args)`

Transform explicit procedure arguments list.

Parameters

`args` (`list[Any]`) – Expression nodes for each argument.

Returns

List of argument expressions.

Return type

`list[Any]`

`field_name(args)`

Extract field name identifier.

Parameters

`args` (`list[Any]`) – IDENTIFIER token.

Returns

Field name string with backticks removed.

Return type

`str`

`foreach_clause(args)`

Transform a FOREACH clause for iterative list mutation.

Parameters

`args` (`list[Any]`) – [`variable_name` (`str`), `list_expression`, `update_clause`, ...]

Returns

Dict with type “ForeachClause” containing variable, `list_expression`, and clauses (list of update clause dicts).

Return type

`dict[str, Any]`

`limit_clause(args)`

Transform a LIMIT clause for result set size restriction.

Parameters

`args` (`list[Any]`) – Expression evaluating to maximum number of rows to return.

Returns

Dict with type “LimitClause” containing limit count expression.

Return type

`dict[str, Any]`

`match_clause(args)`

Transform a MATCH clause for pattern matching.

Parameters

`args` (`list[Any]`) – Mix of OPTIONAL keyword, pattern, and optional `where_clause`.

Returns

Dict with type “MatchClause” containing optional flag, pattern, and where.

Return type
`dict[str, Any]`

`merge_action(args)`
 Transform ON MATCH or ON CREATE action within MERGE.

Parameters
`args (list[Any])` – [“MATCH” or “CREATE” keyword, `set_clause`]

Returns
 Dict with type “MergeAction” specifying trigger type and SET operation.

Return type
`dict[str, Any]`

`merge_action_create(_args)`
 Return normalized CREATE keyword for merge action.

`merge_action_match(_args)`
 Return normalized MATCH keyword for merge action.

`merge_action_type(args)`
 Normalize merge action type tokens to keyword strings.

`merge_clause(args)`
 Transform a MERGE clause for create-or-match operations.

Parameters
`args (list[Any])` – [pattern, optional `merge_action` nodes]

Returns
 Dict with type “MergeClause” containing pattern and conditional actions.

Return type
`dict[str, Any]`

`nulls_placement(args)`
 Transform a NULLS FIRST or NULLS LAST clause.

Parameters
`args (list[Any])` – [NULLS_FIRST_KEYWORD | NULLS_LAST_KEYWORD token]

Returns
 Dict with placement field (“first” or “last”).

Return type
`dict[str, Any]`

`order_clause(args)`
 Transform an ORDER BY clause for result sorting.

Parameters
`args (list[Any])` – `order_items` wrapper containing list of sort specifications.

Returns
 Dict with type “OrderClause” containing list of order items.

Return type
`dict[str, Any]`

`order_direction(args)`
 Normalize ORDER BY direction keywords.

Parameters
`args (list[Any])` – Direction keyword token (optional).

Returns
 “asc” or “desc”.

Return type
 Normalized direction string

`order_item(args)`
 Transform a single ORDER BY item with optional direction and nulls placement.

Parameters
`args (list[Any])` – [expression, optional direction string, optional nulls__placement dict]

Returns
 Dict with expression, direction, and nulls__placement fields.

Return type
`dict[str, Any]`

`order_items(args)`
 Transform comma-separated list of ORDER BY items.

Parameters
`args (list[Any])` – Individual `order_item` nodes.

Returns
 Dict containing list of order specifications.

Return type
`dict[str, list[Any]]`

`procedure_reference(args)`
 Extract procedure name reference.

Parameters
`args (list[Any])` – Function name (possibly namespaced).

Returns
 Procedure name string or dict.

Return type
`Any | None`

`query_statement(args)`
 Transform a read-only query statement (MATCH...RETURN).

Parameters
`args (list[Any])` – Mix of read clause nodes and an optional `ReturnStatement`.

Returns
 Dict with type “QueryStatement” containing separated clauses and return.

Return type
`dict[str, Any]`

`read_clause(args)`

Pass through read clauses without modification.

Parameters

`args` (`list[Any]`) – Single read clause node (MATCH/UNWIND/WITH/CALL).

Returns

The read clause node unchanged.

Return type

`Any` | `None`

`remove_clause(args)`

Transform a REMOVE clause for deleting properties or labels.

Parameters

`args` (`list[Any]`) – `remove_items` wrapper containing list of remove operations.

Returns

Dict with type “RemoveClause” containing list of remove operations.

Return type

`dict[str, Any]`

`remove_item(args)`

Pass through individual REMOVE operation.

Parameters

`args` (`list[Any]`) – Single remove operation (property or labels).

Returns

The remove operation node unchanged.

Return type

`Any` | `None`

`remove_items(args)`

Transform comma-separated list of REMOVE operations.

Parameters

`args` (`list[Any]`) – Individual `remove_item` nodes.

Returns

Dict containing list of remove operations.

Return type

`dict[str, list[Any]]`

`remove_labels_item(args)`

Transform label removal (e.g., REMOVE n:Person).

Parameters

`args` (`list[Any]`) – `[variable_name, node_labels]`

Returns

Dict with type “RemoveLabels” containing variable and labels to remove.

Return type

`dict[str, Any]`

`remove_property_item(args)`

Transform property removal (e.g., `REMOVE n.age`).

Parameters

`args (list[Any])` – `[variable_name, property_lookup]`

Returns

Dict with type “RemoveProperty” containing variable and property name.

Return type

`dict[str, Any]`

`return_alias(args)`

Extract return item alias identifier.

Parameters

`args (list[Any])` – IDENTIFIER token.

Returns

Alias string with backticks removed.

Return type

`str`

`return_body(args)`

Extract the body of a RETURN clause (items or *).

Parameters

`args (list[Any])` – Either “*” string or `return_items` list.

Returns

Either “*” string or list of return items.

Return type

`str | list[Any]`

`return_clause(args)`

Transform a RETURN clause for query output specification.

Parameters

`args (list[Any])` – Mix of DISTINCT keyword, return body/items/*, and optional clauses.

Returns

Dict with type “ReturnStatement” containing all output specifications.

Return type

`dict[str, Any]`

`return_item(args)`

Transform a single return item with optional alias.

Parameters

`args (list[Any])` – `[expression]` or `[expression, alias]`

Returns

Dict with type “ReturnItem” containing expression and optional alias.

Return type

`dict[str, Any]`

`return_items(args)`

Transform comma-separated list of return items.

Parameters

`args` (`list[Any]`) – Individual `return_item` nodes.

Returns

List of return item dictionaries.

Return type

`list[Any]`

`set_all_properties_item(args)`

Transform property map replacement (e.g., `SET n = {name: 'Alice'}`).

Parameters

`args` (`list[Any]`) – `[variable_name, expression]`

Returns

Dict with type “SetAllProperties” for complete property replacement.

Return type

`dict[str, Any]`

`set_clause(args)`

Transform a SET clause for updating graph properties.

Parameters

`args` (`list[Any]`) – `set_items` wrapper containing list of set operations.

Returns

Dict with type “SetClause” containing list of set operations.

Return type

`dict[str, Any]`

`set_item(args)`

Pass through individual SET operation.

Parameters

`args` (`list[Any]`) – Single set operation (property/labels/all properties/add properties).

Returns

The set operation node unchanged.

Return type

`Any | None`

`set_items(args)`

Transform comma-separated list of SET operations.

Parameters

`args` (`list[Any]`) – Individual `set_item` nodes.

Returns

Dict containing list of set operations.

Return type

`dict[str, list[Any]]`

`set_labels_item(args)`

Transform label assignment (e.g., `SET n:Person:Employee`).

Parameters

`args (list[Any])` – [variable_name, node_labels]

Returns

Dict with type “SetLabels” containing variable and label list.

Return type

`dict[str, Any]`

`set_property_item(args)`

Transform a single property assignment (e.g., `SET n.age = 30`).

Parameters

`args (list[Any])` – [variable_name, property_lookup, expression]

Returns

Dict with type “SetProperty” containing variable, property, and value.

Return type

`dict[str, Any]`

`skip_clause(args)`

Transform a SKIP clause for result pagination.

Parameters

`args (list[Any])` – Expression evaluating to number of rows to skip.

Returns

Dict with type “SkipClause” containing skip count expression.

Return type

`dict[str, Any]`

`statement(args)`

Pass through statement nodes.

Parameters

`args (list[Any])` – Single statement node.

Returns

The statement node unchanged.

Return type

`Any | None`

`statement_list(args)`

Transform a list of statements, preserving UNION connectors.

Parameters

`args (list[Any])` – Alternating statement nodes and UnionOp dicts.

Returns

A plain list (single statement) or a UnionStatementList dict.

Return type

`Any`

`union_all_marker(args)`

Return True to signal that this UNION has the ALL qualifier.

`union_op(args)`

Transform a UNION [ALL] connector between two query statements.

Parameters

`args (list[Any])` – [True] when the operator is UNION ALL; [] when it is plain UNION.

Returns

Dict with type='UnionOp' and all flag.

Return type

`dict[str, Any]`

`unwind_clause(args)`

Transform an UNWIND clause for list expansion.

Parameters

`args (list[Any])` – [expression (the list), variable_name (for each element)]

Returns

Dict with type “UnwindClause” containing source expression and variable.

Return type

`dict[str, Any]`

`update_clause(args)`

Pass through update clauses without modification.

Parameters

`args (list[Any])` – Single update clause node (CREATE/MERGE/DELETE/SET/REMOVE).

Returns

The update clause node unchanged.

Return type

`Any | None`

`update_statement(args)`

Transform an update statement (CREATE/MERGE/DELETE/SET/REMOVE).

Parameters

`args (list[Any])` – Mix of prefix clauses, update clauses, and optional return.

Returns

Dict with type “UpdateStatement” containing ordered clauses list plus legacy prefix, updates, and return keys.

Return type

`dict[str, Any]`

`where_clause(args)`

Transform a WHERE clause for filtering.

Parameters

`args (list[Any])` – Single boolean expression for the filter condition.

Returns

Dict with type “WhereClause” containing the filter expression.

Return type
`dict[str, Any]`

`with_clause(args)`

Transform a WITH clause for query chaining and variable passing.

Parameters
`args (list[Any])` – Mix of DISTINCT keyword, return body, and optional clauses.

Returns
 Dict with type “WithClause” containing all components for variable passing.

Return type
`dict[str, Any]`

`yield_clause(args)`

Transform a YIELD clause that selects procedure output fields.

Parameters
`args (list[Any])` – [yield_items or “*”, optional where_clause]

Returns
 Dict with type “YieldClause” containing items and optional where filter.

Return type
`dict[str, Any]`

`yield_item(args)`

Transform a single yielded field with optional alias.

Parameters
`args (list[Any])` – [field_name] or [field_name, alias]

Returns
 Dict with field name and optional alias.

Return type
`dict[str, Any]`

`yield_items(args)`

Transform list of yielded fields.

Parameters
`args (list[Any])` – Individual yield_item nodes.

Returns
 List of yield item dictionaries.

Return type
`list[Any]`

`class pycypher.grammar_rule_mixins.ExpressionRulesMixin`

Bases: `object`

Grammar rule methods for expressions, operators, and predicates.

Handles: - Operator keywords (add_op, mult_op, pow_op, unary_op) - Boolean expressions (or, xor, and, not) - Comparison expressions - Arithmetic expressions (add, mult, power, unary) - String predicate expressions (STARTS WITH, ENDS WITH, CONTAINS, IN, =~) - Null predicate expressions (IS NULL, IS NOT NULL) - Label predicate expressions - Postfix expressions (property lookup, index, slicing) - Special expressions (count(*), EXISTS, inline pattern predicates)

Note: Uses Token from lark for string_predicate_op processing.

`add_expression(args)`

Transform addition and subtraction arithmetic expressions.

Addition and subtraction have equal precedence and associate left-to-right. This method builds a left-associative tree of Arithmetic nodes.

Parameters

`args (list[Any])` – Alternating operands and operators [left, op1, right1, op2, right2, ...].

Returns

Single operand unchanged, or nested dict nodes with type “Arithmetic” containing operator (“+” or “-“), left operand, and right operand.

Return type

`Any | dict[str, Any]`

`add_op(args)`

Extract operator from `add_op` rule.

`and_expression(args)`

Transform AND boolean expression with short-circuit evaluation.

AND has higher precedence than OR/XOR. Multiple ANDs are collected into a single node for easier optimization and execution planning.

Parameters

`args (list[Any])` – One or more NOT expression operands.

Returns

Single operand, or dict with type “And” containing all operands.

Return type

`Any | dict[str, Any]`

`comparison_expression(args)`

Transform comparison expression with operators like `=`, `<>`, `<`, `>`, `<=`, `>=`.

Comparison expressions allow comparing values for equality or ordering. Multiple comparisons can be chained (e.g., `a < b < c`), which is transformed into nested comparison nodes. This structure is necessary for type checking and ensuring all operands are comparable.

Single operands pass through to avoid unnecessary wrapping. This is necessary for efficient AST traversal and avoiding deep nesting for simple expressions.

Parameters

`args (list[Any])` – Alternating operands and operators [left, op1, right1, op2, right2, ...].

Returns

Single operand unchanged, or nested dict nodes with type “Comparison” containing operator, left operand, and right operand for each comparison.

Return type

`Any | dict[str, Any]`

`contains_op(_args)`

Return the normalized CONTAINS operator string.

`count_star(args)`

Transform COUNT(*) aggregate function into a special AST node.

Parameters

`args (list[Any])` – Not used (COUNT(*) has no arguments, * is syntactic).

Returns

Dict with type “CountStar” indicating a count-all operation.

Return type

`dict[str, str]`

`ends_with_op(_args)`

Return the normalized ENDS WITH operator string.

`exists_content(args)`

Extract the content of an EXISTS subquery.

EXISTS content can be either: 1. A simple pattern with optional WHERE clause (implicit match)
2. A full query with MATCH/UNWIND/WITH clauses and optional RETURN

Parameters

`args (list[Any])` – Parsed subquery content (pattern or query clauses).

Returns

The subquery content unchanged, or a wrapped ExistsSubquery dict when the content is a full subquery with multiple clauses.

Return type

`Any | None`

`exists_expression(args)`

Transform EXISTS { ... } subquery expression.

EXISTS evaluates to true if the subquery returns any results, false otherwise.

Parameters

`args (list[Any])` – Single exists_content node containing the subquery specification.

Returns

Dict with type “Exists” containing the subquery content.

Return type

`dict[str, Any]`

`in_op(_args)`

Return the normalized IN operator string.

`index_lookup(args)`

Transform index lookup syntax ([index]) into intermediate node.

Parameters

`args (list[Any])` – Index expression that evaluates to the position/key.

Returns

Dict with type “IndexLookup” and the index expression.

Return type

`dict[str, Any]`

`inline_pattern_predicate(args)`

Transform an inline pattern predicate into an EXISTS expression.

`(a)-[:R]->(b)` in a WHERE clause is shorthand for EXISTS { `(a)-[:R]->(b)` }. This transformer converts the parsed alternating node/relationship args into the same Exists dict that `exists_expression` produces when given a simple pattern body.

Earley with ambiguity="explicit" can produce `_ambig` Tree nodes when an anonymous node `()` matches multiple grammar paths. We resolve each such node by transforming the first valid alternative so that the result is always a plain dict.

Parameters

`args` (`list[Any]`) – Alternating `node_pattern` and `relationship_pattern` dicts (or `_ambig` Trees for anonymous nodes). At least three elements (`node`, `rel`, `node`) are guaranteed by the grammar `+`.

Returns

Dict with type="Exists" whose content is a Pattern wrapping a single PathPattern `-> PatternElement`.

Return type

`dict[str, Any]`

`is_not_null(args)`

Return the IS NOT NULL operator constant.

Parameters

`args` (`list[Any]`) – Not used (rule has no child nodes).

Returns

String constant "IS NOT NULL".

Return type

`str`

`is_null(args)`

Return the IS NULL operator constant.

Parameters

`args` (`list[Any]`) – Not used (rule has no child nodes).

Returns

String constant "IS NULL".

Return type

`str`

`label_predicate_expression(args)`

Transform label predicate expression (`n:Label` or `n:Label1:Label2`).

A label predicate checks whether the node bound to a variable has a specific label. `n:Person` is true when `n` is a Person node. Multiple colon-separated labels (`n:Person:Employee`) mean the node must have ALL listed labels (AND semantics).

When no labels follow the expression, this rule is transparent (passes through unchanged).

Parameters

`args` (`list[Any]`) – `[expression]` or `[expression, label_name1, label_name2, ...]`.

Returns

Expression unchanged if no labels, or dict with type "LabelPredicate" containing operand and labels.

Return type

`Any` | `dict[str, Any]`

`mult_expression(args)`

Transform multiplication, division, and modulo arithmetic expressions.

Multiplication, division, and modulo have equal precedence, higher than addition/subtraction, and associate left-to-right.

Parameters

`args` (`list[Any]`) – Alternating operands and operators [left, op1, right1, op2, right2, ...].

Returns

Single operand unchanged, or nested dict nodes with type “Arithmetic” containing operator (“*”, “/”, or “%”), left operand, and right operand.

Return type

`Any` | `dict[str, Any]`

`mult_op(args)`

Extract operator from `mult_op` rule.

`not_expression(args)`

Transform NOT expression with proper boolean negation handling.

NOT is a unary boolean operator that negates its operand. Multiple NOTs can be chained (e.g., NOT NOT x), so we count them and apply modulo 2 logic: odd count = negate, even count = no-op (double negation cancels).

The NOT_KEYWORD terminal has higher priority than IDENTIFIER in the grammar, ensuring “NOT” is parsed as a keyword rather than a variable name. These terminals are passed as Lark Token objects, so we filter them separately from the expression being negated.

This careful handling is necessary because: 1. NOT can be ambiguous with variable names in some contexts 2. Terminal priorities must be explicit to avoid parse errors 3. Multiple negations need semantic simplification

Parameters

`args` (`list[Any]`) – Mix of NOT_KEYWORD Token objects and the comparison expression.

Returns

The expression unchanged (even NOTs), or wrapped in Not node (odd NOTs).

Return type

`Any` | `dict[str, Any]`

`not_in_op(_args)`

Return the normalized NOT IN operator string.

`null_check_op(args)`

Handle IS NULL / IS NOT NULL operator token (consumed by parent rule).

`null_predicate_expression(args)`

Transform IS NULL or IS NOT NULL predicate expressions.

Null predicates check whether an expression evaluates to NULL, which is necessary because NULL cannot be compared with = or <> in SQL/Cypher semantics (NULL = NULL is false, not true). IS NULL and IS NOT NULL are the only correct ways to test for null values.

This method wraps the expression in a NullCheck node only when a null operator is present. Otherwise, it passes through the expression unchanged to avoid unnecessary wrapper nodes in the AST.

Parameters

`args (list[Any])` – [expression] or [expression, null_check_operator].

Returns

Expression unchanged if no null check, or dict with type “NullCheck” containing the operator (“IS NULL” or “IS NOT NULL”) and operand.

Return type

`Any | dict[str, Any]`

`or_expression(args)`

Transform OR boolean expression with short-circuit evaluation.

OR has lowest precedence among boolean operators. Multiple OR operations are collected into a single node for easier optimization. Single operands pass through to avoid unnecessary wrapping.

Parameters

`args (list[Any])` – One or more XOR expression operands.

Returns

Single operand, or dict with type “Or” containing all operands.

Return type

`Any | dict[str, Any]`

`postfix_expression(args)`

Transform postfix expressions (property access, indexing, slicing).

Postfix operators apply after an expression and associate left-to-right, building a chain of access operations.

Parameters

`args (list[Any])` – [atom_expression, postfix_op1, postfix_op2, ...].

Returns

Single atom unchanged, or nested access nodes (PropertyAccess, IndexAccess, Slice) forming a left-to-right chain of operations.

Return type

`Any | dict[str, Any]`

`postfix_op(args)`

Pass through postfix operator nodes without modification.

Parameters

`args (list[Any])` – Single postfix operation node (PropertyLookup, IndexLookup, or Slicing).

Returns

The postfix operation unchanged.

Return type
`Any` | `None`

`pow_op(args)`

Extract operator from `pow_op` rule.

`power_expression(args)`

Transform exponentiation (power) arithmetic expressions using `^` operator.

Exponentiation has the highest precedence among arithmetic operators and associates left-to-right.

Parameters

`args` (`list[Any]`) – Alternating operands and operators [`left`, `op1`, `right1`, `op2`, `right2`, ...].

Returns

Single operand unchanged, or nested dict nodes with type “Arithmetic” containing operator “`^`”, left operand (base), and right operand (exponent).

Return type

`Any` | `dict[str, Any]`

`property_lookup(args)`

Transform property lookup syntax (`.property_name`) into intermediate node.

Parameters

`args` (`list[Any]`) – Property name string from `property_name` rule.

Returns

Dict with type “PropertyLookup” and the property name.

Return type

`dict[str, Any]`

`regex_match_op(_args)`

Return the normalized regular expression match operator.

`slice_end(args)`

Return the end expression of a slice, or `None` if absent.

`slice_start(args)`

Return the start expression of a slice, or `None` if absent.

`slicing(args)`

Transform list slicing syntax (`[from..to]`) into intermediate node.

Parameters

`args` (`list[Any]`) – [`from_expr`, `to_expr`] where either can be `None` for open ranges.

Returns

Dict with type “Slicing” containing from and to range expressions (may be `None`).

Return type

`dict[str, Any]`

`starts_with_op(_args)`

Return the normalized STARTS WITH operator string.

`string_predicate_expression(args)`

Transform string predicate expressions (STARTS WITH, ENDS WITH, CONTAINS, =~, IN).

String predicates provide specialized string matching operations that are more efficient and expressive than using regular expressions for common patterns. The IN operator tests set membership. These operations are necessary for text search, filtering, and pattern matching in graph queries.

Multiple string predicates can be chained, creating nested nodes for complex string filtering logic. Single operands pass through to avoid unnecessary AST depth.

Parameters

`args` (`list[Any]`) – Alternating operands and operators [left, op1, right1, op2, right2, ...].

Returns

Single operand unchanged, or nested dict nodes with type “StringPredicate” containing operator, left operand, and right operand for each predicate.

Return type

`Any` | `dict[str, Any]`

`string_predicate_op(args)`

Extract and normalize string predicate operator keywords.

String predicate operators can be multi-word keywords (STARTS WITH, ENDS WITH) or single tokens (CONTAINS, IN, =~). This method joins multi-word operators with spaces and uppercases them for consistent AST representation.

Normalization to uppercase is necessary because Cypher is case-insensitive for keywords, and consistent casing enables reliable pattern matching and operator dispatch during query execution.

Parameters

`args` (`list[Any]`) – One or more keyword tokens forming the operator.

Returns

Space-separated, uppercased operator string (e.g., “STARTS WITH”).

Return type

`str`

`unary_expression(args)`

Transform unary plus (+) and minus (-) expressions.

Parameters

`args` (`list[Any]`) – [operator, operand] for unary expressions, or [operand] without operator.

Returns

Operand unchanged if no operator, or dict with type “Unary” containing operator (“+” or “-”) and the operand expression.

Return type

`Any` | `dict[str, Any]`

`unary_op(args)`

Extract operator from unary_op rule.

`xor_expression(args)`

Transform XOR (exclusive OR) boolean expression.

XOR returns true only if operands differ. Multiple XORs are collected for easier analysis. This is less common than AND/OR but necessary for complete boolean logic support.

Parameters

`args (list[Any])` – One or more AND expression operands.

Returns

Single operand, or dict with type “Xor” containing all operands.

Return type

[Any](#) | `dict[str, Any]`

`class pycypher.grammar__rule__mixins.FunctionRulesMixin`

Bases: `object`

Grammar rule methods for function invocations and complex expressions.

Handles: - Function invocation (built-in, user-defined, namespaced) - Function arguments and DISTINCT modifier - CASE expressions (simple and searched forms) - List comprehension (variable IN list WHERE cond | projection) - Pattern comprehension (graph pattern matching into lists) - REDUCE expression (fold/reduce over lists) - Quantifier expressions (ALL, ANY, SINGLE, NONE) - Map projection (property selection and transformation)

`case_expression(args)`

Transform CASE expression (simple or searched form).

Pass-through because the grammar has already dispatched to the appropriate specific handler (`simple_case` or `searched_case`).

Parameters

`args (list[Any])` – Single node (`SimpleCase` or `SearchedCase`) from the specific rule.

Returns

The `SimpleCase` or `SearchedCase` node unchanged.

Return type

[Any](#) | `None`

`else_clause(args)`

Transform the ELSE clause in a CASE expression.

Parameters

`args (list[Any])` – Single expression for the default value.

Returns

Dict with type “Else” containing the default value expression.

Return type

`dict[str, Any]`

`function_arg_list(args)`

Transform comma-separated function argument expressions into a list.

Parameters

`args (list[Any])` – Individual expression nodes for each argument.

Returns

List of argument expressions (empty list if no arguments).

Return type

`list[Any]`

`function_args(args)`

Transform function arguments with optional DISTINCT modifier.

Parameters

`args (list[Any])` – Mix of “DISTINCT” keyword string and `function_arg_list`.

Returns

Dict with “distinct” boolean flag and “arguments” list.

Return type

`dict[str, bool | list[Any]]`

`function_invocation(args)`

Transform function invocation (built-in or user-defined functions).

Parameters

`args (list[Any])` – [`function_name`, `function_args`] where `args` may be None for no arguments.

Returns

Dict with type “FunctionInvocation” containing name and arguments.

Return type

`dict[str, Any]`

`function_name(args)`

Transform function name with optional namespace qualification.

Parameters

`args (list[Any])` – [`namespace_name`, `simple_name`] or just [`simple_name`].

Returns

Simple name string, or dict with namespace and name for qualified functions.

Return type

`str | dict[str, str]`

`function_simple_name(args)`

Extract the unqualified function name identifier.

Parameters

`args (list[Any])` – Single identifier token for the function name.

Returns

Function name as a string with backticks removed.

Return type

`str`

`list_comprehension(args)`

Transform list comprehension expression for filtering and mapping lists.

List comprehension syntax: [variable IN list WHERE condition | projection]

Parameters

`args (list[Any])` – [variable, source_list, optional_filter, optional_projection].

Returns

Dict with type “ListComprehension” containing all components.

Return type
`dict[str, Any]`

`list_filter(args)`

Extract the WHERE filter expression from a list comprehension.

Wraps the expression in a sentinel dict so that `list_comprehension()` can distinguish a bare filter from a bare projection when only one of the two optional clauses is present.

Parameters
`args (list[Any])` – Boolean filter expression.

Returns
`{"_lc_filter": expr}` sentinel dict.

Return type
`Any | None`

`list_projection(args)`

Extract the projection expression from a list comprehension.

Wraps the expression in a sentinel dict so that `list_comprehension()` can distinguish a bare projection from a bare filter when only one of the two optional clauses is present.

Parameters
`args (list[Any])` – Projection expression to transform each element.

Returns
`{"_lc_projection": expr}` sentinel dict.

Return type
`Any | None`

`list_variable(args)`

Extract the iteration variable name from a list comprehension.

Parameters
`args (list[Any])` – Variable name string from `variable_name` rule.

Returns
 Variable name unchanged.

Return type
`Any | None`

`map_element(args)`

Transform a single map projection element.

Parameters
`args (list[Any])` – Either `[selector_string]` or `[property_name, value_expression]` or other.

Returns
 Dict with appropriate structure for the element type.

Return type
`dict[str, Any] | Any | None`

`map_elements(args)`

Transform comma-separated map projection elements into a list.

Parameters
`args (list[Any])` – Individual `map_element` nodes.

Returns

List of map element specifications (empty list if no elements).

Return type

`list[Any]`

`map_projection(args)`

Transform map projection for selecting/transforming object properties.

Map projection syntax: `variable { property1, .property2, property3: expression, ... }`

Parameters

`args (list[Any])` – `[variable_name, list_of_map_elements]`.

Returns

Dict with type “MapProjection” containing variable and elements list.

Return type

`dict[str, Any]`

`namespace_name(args)`

Transform namespace path into a dot-separated string.

Parameters

`args (list[Any])` – List of identifier tokens forming the namespace path.

Returns

Dot-separated namespace string with backticks removed.

Return type

`str`

`pattern_comp_variable(args)`

Extract the optional path variable from a pattern comprehension.

Parameters

`args (list[Any])` – Variable name string.

Returns

Variable name unchanged.

Return type

`Any | None`

`pattern_comprehension(args)`

Transform pattern comprehension for collecting results from pattern matching.

Pattern comprehension syntax: `[path_var = pattern WHERE condition | projection]`

Parameters

`args (list[Any])` – Mix of optional variable, pattern element, optional where, and projection.

Returns

Dict with type “PatternComprehension” containing all components.

Return type

`dict[str, Any]`

`pattern_filter(args)`

Extract the WHERE filter from a pattern comprehension.

Wraps the expression in a {"type": "PatternFilter", "expression": ...} sentinel so that `pattern_comprehension` can distinguish the WHERE expression from the projection expression when both are dicts.

Parameters

`args (list[Any])` – Boolean filter expression.

Returns

Tagged dict {"type": "PatternFilter", "expression": `expr`} or None if no args.

Return type

`Any` | None

`pattern_projection(args)`

Extract the projection expression from a pattern comprehension.

Parameters

`args (list[Any])` – Projection expression.

Returns

Projection expression unchanged.

Return type

`Any` | None

`quantifier(args)`

Extract and normalize the quantifier keyword (ALL, ANY, SINGLE, NONE).

Parameters

`args (list[Any])` – Quantifier keyword token.

Returns

"ALL", "ANY", "SINGLE", or "NONE".

Return type

Uppercased quantifier string

`quantifier_expression(args)`

Transform quantifier expressions (ALL, ANY, SINGLE, NONE) for predicate testing.

Quantifier syntax: QUANTIFIER(variable IN list WHERE predicate)

Parameters

`args (list[Any])` – [quantifier_keyword, variable, source_list, predicate_expression].

Returns

Dict with type "Quantifier" containing quantifier type, variable, source, and predicate.

Return type

`dict[str, Any]`

`quantifier_variable(args)`

Extract the iteration variable from a quantifier expression.

Parameters

`args (list[Any])` – Variable name string.

Returns

Variable name unchanged.

Return type

`Any` | None

`reduce_accumulator(args)`

Transform the accumulator declaration in a REDUCE expression.

Accumulator syntax: `variable_name = initial_expression`

Parameters

`args (list[Any])` – `[variable_name, initialization_expression]`.

Returns

Dict with variable name and init expression.

Return type

`dict[str, Any]`

`reduce_expression(args)`

Transform REDUCE expression for list aggregation with accumulator.

REDUCE syntax: `REDUCE(accumulator = initial, variable IN list | step_expression)`

Parameters

`args (list[Any])` – `[accumulator_dict, iteration_variable, source_list, step_expression]`.

Returns

Dict with type “Reduce” containing accumulator, variable, source, and step.

Return type

`dict[str, Any]`

`reduce_variable(args)`

Extract the iteration variable from a REDUCE expression.

Parameters

`args (list[Any])` – Variable name string.

Returns

Variable name unchanged.

Return type

`Any | None`

`searched_case(args)`

Transform searched CASE expression that evaluates boolean conditions.

Searched CASE syntax: `CASE WHEN condition1 THEN result1 [WHEN ...] [ELSE default] END`

Parameters

`args (list[Any])` – `[when_clause1, when_clause2, ..., optional_else_clause]`.

Returns

Dict with type “SearchedCase” containing when clauses list and optional else.

Return type

`dict[str, Any]`

`searched_when(args)`

Transform a WHEN clause in a searched CASE expression.

Parameters

`args (list[Any])` – `[condition_expression, result_expression]`.

Returns

Dict with type “SearchedWhen” containing condition and result expressions.

Return type
`dict[str, Any]`

`simple_case(args)`

Transform simple CASE expression that matches an operand against values.

Simple CASE syntax: CASE expression WHEN value1 THEN result1 [WHEN ...] [ELSE default] END

Parameters
 args (`list[Any]`) – [operand_expression, when_clause1, when_clause2, ..., optional_else_clause].

Returns
 Dict with type “SimpleCase” containing operand, when clauses list, and optional else.

Return type
`dict[str, Any]`

`simple_when(args)`

Transform a WHEN clause in a simple CASE expression.

Parameters
 args (`list[Any]`) – [when_operands_list, result_expression].

Returns
 Dict with type “SimpleWhen” containing operands list and result expression.

Return type
`dict[str, Any]`

`when_operands(args)`

Transform comma-separated operands in a simple CASE WHEN clause.

Parameters
 args (`list[Any]`) – Individual expression nodes for each value to test.

Returns
 List of operand expressions.

Return type
`list[Any]`

class `pycypher.grammar_rule_mixins.LiteralRulesMixin`

Bases: `object`

Grammar rule methods for literal values, parameters, and variable names.

Handles: - Number literals (signed, unsigned, hex, octal, float, Inf, NaN) - String literals (escape sequences) - Boolean literals (TRUE / FALSE) - NULL literal - List literals and list elements - Map literals, entries, and individual entries - Parameter references (\$name, \$0) - Variable name normalization (backtick stripping)

`false(args)`

Transform FALSE keyword into Python False.

`list_elements(args)`

Transform comma-separated list elements into Python list.

Parameters
 args (`list[Any]`) – Individual element expression nodes.

Returns
Python list of element expressions.

Return type
`list[Any]`

`list_literal(args)`

Transform list literal syntax [...] into Python list.

Parameters
args (`list[Any]`) – list_elements node containing list of element expressions.

Returns
Python list of element values (empty list if no elements).

Return type
`list[Any]`

`map_entries(args)`

Transform comma-separated map entries into an entries wrapper dict.

Parameters
args (`list[Any]`) – Individual map_entry nodes with key and value.

Returns
Dict with "entries" key containing the collected key-value map.

Return type
`dict[str, dict[str, Any]]`

`map_entry(args)`

Transform a single map entry (key: value) into a key-value dict.

Parameters
args (`list[Any]`) – [property_name, value_expression].

Returns
Dict with "key" and "value" keys.

Return type
`dict[str, Any]`

`map_literal(args)`

Transform map literal syntax {...} into MapLiteral AST dict.

Parameters
args (`list[Any]`) – map_entries node containing dict of key-value pairs.

Returns
Dict with type="MapLiteral" and "value" key.

Return type
`dict[str, Any]`

`null_literal(args)`

Transform NULL keyword into a NullLiteral AST dict.

Returns a typed dict rather than Python None to distinguish explicit null expressions from absent/optional AST fields.

`number_literal(args)`

Transform number literals (integers and floats) into Python values.

Pass-through — actual conversion happens in `signed_number` / `unsigned_number`.

Parameters

`args (list[Any])` – Single numeric value from `signed_number` or `unsigned_number`.

Returns

Python int or float value, or 0 as fallback.

Return type

`int` | `float`

`parameter(args)`

Transform parameter reference (`$param`) into a Parameter AST node.

Parameters

`args (list[Any])` – Parameter name (string identifier or integer index).

Returns

Dict with `type="Parameter"` and the parameter name.

Return type

`dict[str, Any]`

`parameter_name(args)`

Extract and normalize parameter name or index.

Numeric indices are parsed as integers; identifiers have backticks stripped.

Parameters

`args (list[Any])` – Parameter name token (identifier or number).

Returns

Integer for numeric indices, string for named parameters.

Return type

`int` | `str`

`signed_number(args)`

Transform signed number literals into Python int or float values.

Supports integers (-42, +100, -0x2A), floats (-3.14, +2.5e10), special values (-INF, +INFINITY, -NAN), and underscore separators.

Parameters

`args (list[Any])` – Single signed number token.

Returns

Python int, float, or special float value (inf/-inf/nan).

Return type

`int` | `float` | `str`

`string_literal(args)`

Transform string literals into structured AST nodes.

Handles quote removal and escape sequences (`\n`, `\t`, `\r`, `\\`, `\'`, `\"`).

Parameters

`args (list[Any])` – Single string token with quotes.

Returns

Dict with type="StringLiteral" and the processed string value.

Return type

`dict[str, Any]`

`true(args)`

Transform TRUE keyword into Python True.

`unsigned__number(args)`

Transform unsigned number literals into Python int or float values.

Supports decimal, hex (0x), octal (0o), float, special values, and underscore separators.

Parameters

`args (list[Any])` – Single unsigned number token.

Returns

Python int, float, or special float value (inf/nan).

Return type

`int | float | str`

`variable__name(args)`

Extract and normalize variable identifier names.

Strips backticks from escaped identifiers and converts Token to str.

Parameters

`args (list[Any])` – Single identifier token (may have backticks).

Returns

Variable name as a string with backticks removed.

Return type

`str`

class `pycypher.grammar__rule__mixins.PatternRulesMixin`

Bases: `object`

Grammar rule methods for graph pattern matching constructs.

Handles: - Pattern and path pattern construction - Pattern elements and shortest path - Node patterns (labels, properties, inline WHERE) - Relationship patterns (direction, types, variable-length paths) - Label expressions (AND, OR, NOT on labels) - Property maps and property key-value pairs - Path length ranges for variable-length relationships

`full__rel__any(args)`

Transform undirected relationship (—).

Parameters

`args (list[Any])` – Optional `rel__detail` dict with relationship components.

Returns

Dict with type "RelationshipPattern", direction "any", and details.

Return type

`dict[str, Any]`

`full_rel_both(args)`
 Transform bidirectional relationship ($\leftrightarrow$).

Parameters
`args (list[Any])` – Optional `rel_detail` dict with relationship components.

Returns
 Dict with type “RelationshipPattern”, direction “both”, and details.

Return type
`dict[str, Any]`

`full_rel_left(args)`
 Transform left-pointing relationship ($\leftarrow$).

Parameters
`args (list[Any])` – Optional `rel_detail` dict with relationship components.

Returns
 Dict with type “RelationshipPattern”, direction “left”, and details.

Return type
`dict[str, Any]`

`full_rel_right(args)`
 Transform right-pointing relationship ($\rightarrow$).

Parameters
`args (list[Any])` – Optional `rel_detail` dict with relationship components.

Returns
 Dict with type “RelationshipPattern”, direction “right”, and details.

Return type
`dict[str, Any]`

`label_expression(args)`
 Transform a single label expression.

Parameters
`args (list[Any])` – Label term(s) from the expression.

Returns
 Single label term or dict with type “LabelExpression” for complex cases.

Return type
`Any | dict[str, Any]`

`label_factor(args)`
 Pass through label factors (possibly negated with !).

Parameters
`args (list[Any])` – Single `label_primary` node.

Returns
 The label primary unchanged.

Return type
`Any | None`

`label_name(args)`

Extract label name identifier.

Parameters

`args (list[Any])` – IDENTIFIER token possibly with leading `:`.

Returns

and backticks removed.

Return type

Label name string with

`label_primary(args)`

Pass through primary label expressions.

Parameters

`args (list[Any])` – Label name or grouped expression.

Returns

The label value unchanged.

Return type

`Any` | `None`

`label_term(args)`

Transform label OR expressions (e.g., `Person|Employee`).

Parameters

`args (list[Any])` – Label factor nodes separated by `|`.

Returns

Single factor or dict with type “LabelOr” for multiple factors.

Return type

`Any` | `dict[str, Any]`

`node_labels(args)`

Transform node label expressions.

Parameters

`args (list[Any])` – List of `label_expression` nodes.

Returns

Dict with “labels” key containing list of label expressions.

Return type

`dict[str, list[Any]]`

`node_pattern(args)`

Transform a node pattern (node in parentheses).

Parameters

`args (list[Any])` – Optional `node_pattern_filler` dict with node components.

Returns

Dict with type “NodePattern” containing variable, labels, properties, where.

Return type

`dict[str, Any]`

`node__pattern__filler(args)`

Extract components from inside node parentheses.

Parameters

`args` (`list[Any]`) – Mix of variable name string and component dicts (labels/properties/where).

Returns

Dict containing all node components with appropriate keys.

Return type

`dict[str, Any]`

`node__properties(args)`

Pass through node properties or WHERE clause.

Parameters

`args` (`list[Any]`) – Properties dict or WHERE clause.

Returns

The properties/where node unchanged.

Return type

`Any | None`

`node__where(args)`

Transform inline WHERE clause within node pattern.

Parameters

`args` (`list[Any]`) – Expression for the WHERE condition.

Returns

Dict with “where” key containing the condition expression.

Return type

`dict[str, Any]`

`path__length(args)`

Transform variable-length path specification (*).

Parameters

`args` (`list[Any]`) – Optional `path__length__range` specification.

Returns

Dict with normalized `PathLength` node for downstream conversion.

Return type

`dict[str, Any]`

`path__length__range(args)`

Transform path length range specification.

Parameters

`args` (`list[Any]`) – One or two integer arguments for range bounds.

Returns

Dict with “fixed”, “min”/”max”, or “unbounded” keys.

Return type

`int | dict[str, int | None]`

`path_pattern(args)`

Transform a single path pattern with optional variable assignment.

Parameters

`args (list[Any])` – Optional variable name followed by `pattern_element`.

Returns

Dict with type “PathPattern” containing optional variable and element.

Return type

`dict[str, Any]`

`pattern(args)`

Transform a graph pattern (one or more paths).

Parameters

`args (list[Any])` – List of `path_pattern` nodes.

Returns

Dict with type “Pattern” containing list of paths.

Return type

`dict[str, Any]`

`pattern_element(args)`

Transform a pattern element (sequence of nodes and relationships).

Parameters

`args (list[Any])` – Alternating `node_pattern` and `relationship_pattern` nodes.

Returns

Dict with type “PatternElement” containing ordered list of parts.

Return type

`dict[str, Any]`

`properties(args)`

Extract property map from curly braces `{...}`.

Parameters

`args (list[Any])` – `property_list` wrapper containing props dict.

Returns

Dict mapping property names to values.

Return type

`dict[str, Any]`

`property_key_value(args)`

Transform a single property assignment (key: value).

Parameters

`args (list[Any])` – `[property_name, expression]`

Returns

Dict with “key” and “value” for the property assignment.

Return type

`dict[str, Any]`

`property__list(args)`
 Transform comma-separated property key-value pairs.

Parameters
 args (`list[Any]`) – List of `property__key__value` dicts.

Returns
 Dict with “props” key containing merged property map.

Return type
`dict[str, dict[str, Any]]`

`property__name(args)`
 Extract property name identifier.

Parameters
 args (`list[Any]`) – IDENTIFIER token.

Returns
 Property name string with backticks removed.

Return type
`str`

`rel__detail(args)`
 Extract details from inside relationship brackets [...].

Parameters
 args (`list[Any]`) – `rel__filler` dict with relationship components.

Returns
 The filler dict unchanged, or empty dict if no details.

Return type
`dict[str, Any]`

`rel__filler(args)`
 Extract components from inside relationship brackets.

Parameters
 args (`list[Any]`) – Mix of variable name string and component dicts.

Returns
 Dict containing all relationship components.

Return type
`dict[str, Any]`

`rel__properties(args)`
 Transform relationship property constraints.

Parameters
 args (`list[Any]`) – Properties map.

Returns
 Dict with “properties” key containing the property map.

Return type
`dict[str, Any]`

`rel_type(args)`
 Extract relationship type name.

Parameters
 args (`list[Any]`) – IDENTIFIER token.

Returns
 Type name string with backticks removed.

Return type
`str`

`rel_types(args)`
 Transform relationship type constraints.

Parameters
 args (`list[Any]`) – List of relationship type names.

Returns
 Dict with “types” key containing list of type names.

Return type
`dict[str, list[Any]]`

`rel_where(args)`
 Transform inline WHERE clause within relationship pattern.

Parameters
 args (`list[Any]`) – Expression for the WHERE condition.

Returns
 Dict with “where” key containing the condition expression.

Return type
`dict[str, Any]`

`relationship_pattern(args)`
 Pass through relationship patterns.

Parameters
 args (`list[Any]`) – Single relationship node from directional rules.

Returns
 The relationship pattern unchanged.

Return type
`Any | None`

`shortest_path(args)`
 Transform SHORTESTPATH or ALLSHORTESTPATHS function.

Parameters
 args (`list[Any]`) – Mix of function name keyword and pattern parts.

Returns
 Dict with type “ShortestPath” containing all flag and pattern parts.

Return type
`dict[str, Any]`

AST Converter

Converts dictionary-based Lark parse trees into typed Pydantic AST models. Includes the `from_cypher()` classmethod for one-step parse-and-convert. Converts dictionary-based AST to typed Pydantic models.

Extracted from `ast_models.py` to separate the ~60 conversion methods from the 76 Pydantic model definitions, reducing cognitive load and enabling independent testing of the converter logic.

The public entry point is `ASTConverter.from_cypher()` which parses a Cypher query string and returns a typed `ASTNode`.

Usage:

```
from pycypher.ast_converter import ASTConverter

ast = ASTConverter.from_cypher("MATCH (n:Person) RETURN n.name")
```

`class pycypher.ast_converter.ASTConverter`

Bases: `object`

Converts dictionary-based AST to Pydantic models.

classmethod `from_cypher(cypher)`

Parse Cypher query and convert to typed AST.

Delegates to `__parse_cypher_cached()`, which caches the result by query string so the expensive Earley parse is performed at most once per unique query in the lifetime of the process.

Parameters

`cypher (str)` – Cypher query string.

Returns

Typed `ASTNode`. The same object is returned for identical cypher strings (the result is shared — do not mutate it).

Raises

- `ValueError` – If AST conversion fails (grammar/model mismatch).
- `lark.exceptions.UnexpectedInput` – If cypher is syntactically invalid.

Return type

`ASTNode`

`convert(node)`

Convert a dictionary-based AST node to a Pydantic model.

Parameters

`node (Any)` – Dictionary, list, or primitive value from grammar parser

Returns

Typed `ASTNode` or `None`

Return type

`ASTNode` | `None`

AST Rewriter

Programmatic AST transformations: `WITH *` expansion, `RETURN` stripping, and multi-query merging for pipeline composition. AST Rewriting Engine for multi-query composition.

Provides utilities for creating, cloning, merging, and serializing Cypher ASTs. The core operations are:

- `create_with_star()` — creates a `WITH *` clause AST node
- `strip_return()` — removes `RETURN` clauses from a Query AST (immutable)
- `merge_queries()` — combines multiple Query ASTs into one with `WITH *` separators and intermediate `RETURN` stripping
- `to_cypher()` — serializes a Query AST back to a valid Cypher string

All operations are immutable — input ASTs are never mutated.

class `pycypher.ast_rewriter.ASTRewriter`

Bases: `object`

Rewrite, merge, and serialize Cypher ASTs.

All methods are stateless and produce new AST objects without mutating the originals.

`create_with_star()`

Create a `WITH *` clause AST node.

The parser represents `WITH *` as a `With` node with an empty items list.

Returns

A new `With` AST node representing `WITH *`.

Return type

`With`

`strip_return(query)`

Return a copy of query with all `RETURN` clauses removed.

The original AST is not mutated.

Parameters

query (`Query`) – The Query AST to strip.

Returns

A new Query with `RETURN` clauses removed.

Return type

`Query`

`merge_queries(queries)`

Merge multiple Query ASTs into a single Query.

Between each pair of queries a `WITH *` clause is inserted so that variables from earlier queries are visible to later ones. Intermediate `RETURN` clauses are stripped (only the final query's `RETURN` is preserved).

Parameters

queries (`list[Query]`) – Ordered list of Query ASTs to merge.

Returns

A single Query combining all input clauses.

Return type

`Query`

`to_cypher(query)`

Serialize a Query AST back to a valid Cypher string.

Parameters

query (`Query`) – The Query AST to serialize.

Returns
A Cypher query string.

Return type
`str`

4.1.3 Data Model

Relational Models

Entity/relationship data containers, Context, and the relational algebra operator hierarchy. Context is the primary data holder — it stores entity and relationship mappings and manages the backend engine lifecycle.

Data containers and context for PyCypher query execution.

This module defines the data containers (`EntityTable`, `RelationshipTable`) and the `Context` object that together form the runtime environment for Cypher query execution via the `BindingFrame` execution path in `star.py`.

Data containers

`EntityTable`

Holds all entity IDs and attributes for a single entity type. Source data is a `pd.DataFrame` or `pyarrow.Table` with an `__ID__` column plus attribute columns. `to_pandas(context)` returns the `DataFrame` with all columns prefixed by the entity type (e.g. `Person__name`, `Person__ID`).

`RelationshipTable`

Holds all relationship IDs and endpoint references for a single relationship type. Source data must have `__ID__`, `__SOURCE__`, and `__TARGET__` columns. `to_pandas(context)` applies the same type-prefix convention (e.g. `KNOWS__SOURCE`).

Context and atomicity

Context holds the entity and relationship mappings that a Star instance queries against. It also implements query-scoped shadow writes:

1. `begin_query()` — opens a transaction by initialising an empty shadow layer.
2. SET-clause mutations write to `context._shadow[entity_type]` (a copy of the canonical `DataFrame`) rather than directly to `source_obj`.
3. `commit_query()` — promotes shadow `DataFrames` to canonical `source_obj` on each entity table.
4. `rollback_query()` — discards the shadow, leaving `source_obj` unchanged.

This guarantees that a failed query never leaves the context in a partially-mutated state.

Key constants

`ID_COLUMN`

Name of the identity column inside source `DataFrames` ("`__ID__`").

`RELATIONSHIP_SOURCE_COLUMN`

Source-node ID column in relationship `DataFrames` ("`__SOURCE__`").

`RELATIONSHIP_TARGET_COLUMN`

Target-node ID column in relationship `DataFrames` ("`__TARGET__`").

```
class pycypher.relational_models.Context(*, backend=None,
                                         entity_mapping=EntityMapping(mapping={}),
                                         relationship_mapping=RelationshipMapping(mapping={}),
                                         cypher_functions=<factory>)
```


Bases: [BaseModel](#)

Context for translation operations.

The optional backend parameter selects the DataFrame computation engine. Pass "pandas" (default), "duckdb", or "auto" to choose a backend. "auto" inspects the total entity count and selects the most efficient engine automatically. You can also pass a pre-constructed [BackendEngine](#) instance directly.

Examples:

```
# Default — identical to current behaviour
Context(entity_mapping=..., relationship_mapping=...)

# Explicit DuckDB backend for analytical workloads
Context(..., backend="duckdb")

# Auto-select based on data size
Context(..., backend="auto")
```

entity_mapping: [EntityMapping](#)

relationship_mapping: [RelationshipMapping](#)

cypher_functions: dict[str, [RegisteredFunction](#)]

model_post_init(_Context __context)

Resolve the backend engine after Pydantic initialisation.

__init__(*, backend=None, **data)

Initialise the query context with entity/relationship data and a backend engine.

Parameters

- backend ([Any](#)) – Backend engine or hint string ("pandas", "duckdb", "polars", "auto"). Defaults to PandasBackend when None or "pandas".
- **data ([Any](#)) – Pydantic field values — typically entity_mapping, relationship_mapping, and functions.

Note

The backend is stored but not yet used by the execution path. See `pycypher.backend_engine` for integration status.

property backend: [Any](#)

Return the active [BackendEngine](#).

property backend_name: str

Return the name of the active backend engine (e.g. 'pandas', 'duckdb', 'polars').

property index `_manager`: [Any](#)

Return the [GraphIndexManager](#).

Created lazily on first access. Automatically invalidates when the data epoch changes (after mutation commits).

`__repr__()`

Return an informative summary for REPL/notebook display.

Shows backend, entity types with row counts, relationship types with row counts, and custom function count. Example:

```
Context(backend='pandas', entities={'Person': 4, 'Company': 2},
        relationships={'WORKS_AT': 3}, custom_functions=1)
```

`set_deadline(timeout_seconds)`

Arm the per-query timeout clock.

Call this before `begin_query()` so that the deadline is in effect for the entire query transaction.

Parameters

`timeout_seconds` (`float` | `None`) – Wall-clock budget in seconds, or `None` to disable timeout enforcement.

`check_timeout(query_fragment=)`

Raise [QueryTimeoutError](#) if the deadline has passed.

This is intentionally cheap — a single `perf_counter()` comparison — so it can be called at the top of every clause iteration without measurable overhead.

Parameters

`query_fragment` (`str`) – Optional truncated query text for the error message.

Raises

[QueryTimeoutError](#) – If the wall-clock deadline has been exceeded.

`clear_deadline()`

Disarm the per-query timeout (called in finally after execution).

`begin_query()`

Initialise the shadow layers for a new query transaction.

`commit_query()`

Promote shadow DataFrames to the canonical entity and relationship tables.

Handles both updates to existing tables and newly created tables (e.g. from a `CREATE` clause referencing a label not yet in the mapping).

The method ensures shadow state is always cleared, even if an exception occurs mid-commit (e.g. constructing an `EntityTable` for a new label fails). This prevents stale shadow data from leaking into subsequent queries.

`rollback_query()`

Discard all shadow writes — leaves the context unmodified.

`savepoint()`

Snapshot the current shadow state for mid-query recovery.

Returns a lightweight snapshot that can be passed to `restore_savepoint()` to roll back shadow mutations to this point. Useful for MERGE ON CREATE/ON MATCH fallback and other speculative mutation paths.

Returns

A dict with "entities" and "relationships" keys, each mapping type names to copies of the shadow DataFrames that existed at the time of the call.

Return type

`dict[str, dict[str, pd.DataFrame]]`

`restore_savepoint(savepoint)`

Restore shadow state to a previous savepoint.

Replaces the current shadow layers with the snapshots captured by `savepoint()`. Any mutations made after the savepoint was created are discarded.

Parameters

`savepoint (dict[str, dict[str, pd.DataFrame]])` – The snapshot returned by `savepoint()`.

`cypher_function(func)`

Decorator to register a function as a Cypher function in the context.

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.relational_models.EntityMapping(*, mapping={})`

Bases: `BaseModel`

Mapping from entity types to the corresponding Table.

`mapping: dict[EntityType, Any]`

`__getitem__(key)`

Return the table for key, raising `KeyError` if absent.

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.relational_models.EntityTable(*args, source_algebraizable=None, variable_map={}, variable_type_map=<factory>, column_names=[], identifier=<factory>, entity_type, source_obj=None, attribute_map=<factory>, source_obj_attribute_map=<factory>)`

Bases: Relation

Source of truth for all IDs and attributes for a specific entity type.

to_pandas prefixes every column with the entity type, e.g. a source DataFrame with columns ["__ID__", "name", "age"] for entity type "Person" is returned as ["Person__ID__", "Person__name", "Person__age"]. This prevents column-name collisions when multiple entity types are joined together.

entity__type

The entity type label (e.g. "Person").

Type
str

source__obj

The underlying pandas (or PySpark) DataFrame.

Type
Any

attribute__map

Maps attribute names to column names in the source.

Type
dict[str, str]

source__obj__attribute__map

Maps attribute names to the raw column names in the underlying source__obj (always strings).

Type
dict[str, str]

entity__type: EntityType

source__obj: Any

attribute__map: dict[Attribute, ColumnName]

source__obj__attribute__map: dict[Attribute, str]

classmethod from_dataframe(entity__type, df, id_col=None)

Construct an EntityTable from a pandas DataFrame.

This is the recommended factory for the common case where you already have a pandas DataFrame. Column names and attribute maps are inferred automatically, eliminating the need to specify them manually.

Parameters

- entity__type (str) – The entity label (e.g. "Person").
- df (pd.DataFrame) – Source pandas DataFrame. Must contain a column named __ID__ unless id_col is provided.
- id_col (str | None) – If the identity column in df is not named __ID__, pass its current name here — it will be renamed automatically. Raises ValueError if the column is not found.

Returns

A new `EntityTable` ready for use in a `Context`.

Raises

`ValueError` – If `id_col` is specified but not found, or if `__ID__` is absent and `id_col` is not provided.

Return type

`EntityTable`

Examples

Minimal usage when `__ID__` is already present:

```
import pandas as pd
from pycypher import EntityTable, ID_COLUMN

df = pd.DataFrame({"__ID__": [1, 2], "name": ["Alice", "Bob"]})
table = EntityTable.from_dataframe("Person", df)
```

Custom identity column:

```
df = pd.DataFrame({"person_id": [1, 2], "name": ["Alice", "Bob"]})
table = EntityTable.from_dataframe("Person", df, id_col="person_id")
```

classmethod `from_arrow(entity_type, table, id_col=None)`

Construct an `EntityTable` from a PyArrow table.

Parameters

- `entity_type` (`str`) – The entity label (e.g. "Person").
- `table` (`pa.Table`) – Arrow table that already has an `__ID__` column (i.e. has been passed through `normalize_entity_table()`).
- `id_col` (`str | None`) – Ignored — the table is expected to be pre-normalised. Kept for API symmetry.

Returns

A new `EntityTable` with `source_obj` set to `table`.

Return type

`EntityTable`

`to_pandas(context)`

Materialise this entity table as a pandas `DataFrame` with prefixed columns.

Converts the underlying source (Arrow table or `DataFrame`) to pandas and prefixes every column with `entity_type` to prevent naming collisions in downstream joins (e.g. `Person__ID__`, `Person__name`).

Parameters

`context` (`Context`) – Current execution context (unused but required by interface).

Returns

A `pd.DataFrame` with type-prefixed column names.

Return type

`pd.DataFrame`

`to_spark(context)`

Convert `EntityTable` to a PySpark `DataFrame`.

Delegates to `_to_spark_with_prefix()` using `entity_type` as the column prefix. Handles both native PySpark `DataFrames` (column rename only) and non-Spark sources (pandas conversion).

Parameters

`context` (`Context`) – PyCypher Context

Returns

PySpark `DataFrame` with columns prefixed by `{entity_type}__`.

Raises

`ImportError` – If PySpark is not installed.

Return type

`Any`

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.relational_models.RegisteredFunction(*, name, implementation, arity=0)`

Bases: `BaseModel`

Represents a registered Cypher function in the context.

`model_config: ClassVar[ConfigDict] = {'arbitrary_types_allowed': True}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`name: str`

`implementation: Callable`

`arity: int`

`__call__(*args)`

Invoke the wrapped function, validating arity if set.

`class pycypher.relational_models.RelationshipMapping(*, mapping={})`

Bases: `BaseModel`

Mapping from relationship types to the corresponding Table.

`mapping: dict[RelationshipType, Any]`

`__getitem__(key)`

Return the table for key, raising `KeyError` if absent.

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

```
class pycypher.relational_models.RelationshipTable(*args, source_algebraizable=None, variable_map={},
                                                    variable_type_map=<factory>, column_names=[],
                                                    identifier=<factory>, relationship_type,
                                                    source_obj=None, attribute_map=<factory>,
                                                    source_obj_attribute_map=<factory>)
```

Bases: Relation

Source of truth for all IDs and attributes for a specific relationship type.

to_pandas prefixes every column with the relationship type, e.g. a source DataFrame with columns ["__ID__", "__SOURCE__", "__TARGET__"] for relationship type "KNOWS" is returned as ["KNOWS__ID__", "KNOWS__SOURCE__", "KNOWS__TARGET__"].

relationship_type

The relationship type label (e.g. "KNOWS").

Type
str

source_obj

The underlying pandas (or PySpark) DataFrame.

Type
Any

attribute_map

Maps attribute names to column names in the source.

Type
dict[str, str]

source_obj_attribute_map

Maps attribute names to the raw column names in the underlying source_obj (always strings).

Type
dict[str, str]

model_config: ClassVar[ConfigDict] = {}

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

relationship_type: RelationshipType

source_obj: Any

attribute_map: dict[Attribute, ColumnName]

source_obj_attribute_map: dict[Attribute, str]

classmethod from_dataframe(relationship_type, df, *, id_col=None, source_col=None, target_col=None)

Construct a RelationshipTable from a pandas DataFrame.

This is the recommended factory for the common case where you already have a pandas DataFrame describing relationships. Column names and attribute maps are inferred automatically, eliminating the need to specify them manually.

Parameters

- `relationship_type` (`str`) – The relationship label (e.g. "KNOWS").
- `df` (`pd.DataFrame`) – Source pandas DataFrame. Must contain columns named `__ID__`, `__SOURCE__`, and `__TARGET__` unless the corresponding `*_col` parameters are provided.
- `id_col` (`str` | `None`) – If the identity column in `df` is not named `__ID__`, pass its current name here — it will be renamed automatically.
- `source_col` (`str` | `None`) – If the source-node column is not named `__SOURCE__`, pass its current name here.
- `target_col` (`str` | `None`) – If the target-node column is not named `__TARGET__`, pass its current name here.

Returns

A new [RelationshipTable](#) ready for use in a [Context](#).

Raises

[ValueError](#) – If a required column (`__ID__`, `__SOURCE__`, `__TARGET__`) is missing and the corresponding `*_col` parameter was not provided, or if a specified `*_col` is not found in the DataFrame.

Return type

[RelationshipTable](#)

Examples

Minimal usage when canonical columns are already present:

```
import pandas as pd
from pycypher import RelationshipTable

df = pd.DataFrame({
    "__ID__": [1, 2],
    "__SOURCE__": [10, 20],
    "__TARGET__": [30, 40],
    "since": [2020, 2021],
})
table = RelationshipTable.from_dataframe("KNOWS", df)
```

Custom column names:

```
df = pd.DataFrame({
    "rel_id": [1, 2],
    "from_node": [10, 20],
    "to_node": [30, 40],
})
table = RelationshipTable.from_dataframe(
    "KNOWS", df,
    id_col="rel_id",
    source_col="from_node",
    target_col="to_node",
)
```

classmethod `from_arrow(relationship_type, table, source_col=None, target_col=None)`

Construct a [RelationshipTable](#) from a PyArrow table.

Parameters

- `relationship_type` (`str`) – The relationship label (e.g. "KNOWS").
- `table` (`pa.Table`) – Arrow table that already has `__ID__`, `__SOURCE__`, and `__TARGET__` columns (pre-normalised).
- `source_col` (`str` | `None`) – Ignored — table is expected to be pre-normalised.
- `target_col` (`str` | `None`) – Ignored — table is expected to be pre-normalised.

Returns

A new `RelationshipTable` with `source_obj` set to `table`.

Return type

`RelationshipTable`

`to_pandas(context)`

Convert the `RelationshipTable` to a pandas `DataFrame`.

`to_spark(context)`

Convert `RelationshipTable` to a PySpark `DataFrame`.

Delegates to `__to_spark_with_prefix()` using `relationship_type` as the column prefix. Handles both native PySpark `DataFrames` (column rename only) and non-Spark sources (pandas conversion).

Parameters

`context` (`Context`) – PyCypher Context

Returns

PySpark `DataFrame` with columns prefixed by `{relationship_type}__`.

Raises

`ImportError` – If PySpark is not installed.

Return type

`Any`

Constants

Shared constants (column names, sentinel values) used across modules. Shared constants and small utilities used across pycypher modules.

Extracting these from `relational_models` and `binding_evaluator` breaks two circular-import cycles:

1. `relational_models` `backend_engine` (shared `ID_COLUMN` constant)
2. `binding_evaluator` `scalar_function_evaluator` (shared `__normalize_func_args` helper)

`pycypher.constants.ID_COLUMN`: `str = '__ID__'`

Name of the identity column inside source `DataFrames`.

`pycypher.constants.RELATIONSHIP_SOURCE_COLUMN`: `str = '__SOURCE__'`

Source-node ID column in relationship `DataFrames`.

`pycypher.constants.RELATIONSHIP_TARGET_COLUMN`: `str = '__TARGET__'`

Target-node ID column in relationship `DataFrames`.

```
class pycypher.constants.NullMaskResult(result, non_null_mask, non_null_vals)
```

Bases: `object`

Result of `__init_null_result()` — holds the pre-allocated result Series, the boolean mask of non-null positions, and the filtered non-null values.

`result`

Object-dtype Series of length `len(s)` initialised to `None`.

`non_null_mask`

Boolean Series — True where `s` is not null.

`non_null_vals`

Filtered Series of non-null values, or `None` if the input was entirely null.

`__init__(result, non_null_mask, non_null_vals)`

`result`

`non_null_mask`

`non_null_vals`

property `all_null`: `bool`

Return True when every value in the input was null.

4.1.4 Query Execution

Star (Query Orchestrator)

The main entry point for query execution. Star accepts a Context, parses Cypher queries, and returns results as pandas DataFrames. Translation layer from Cypher AST to BindingFrame execution.

The main entry point for users is the Star class, which accepts a Context (containing EntityTable and RelationshipTable objects), parses a Cypher query string, and executes it against the registered DataFrames via the BindingFrame execution path.

Star is a thin facade that coordinates specialized components:

- `ClauseExecutor` — clause dispatch and execution
- `QueryAnalyzer` — pre-execution planning and optimization
- `QueryExplainer` — EXPLAIN-style text plans
- `PatternMatcher` — MATCH clause translation
- `MutationEngine` — CREATE/SET/DELETE execution
- `ProjectionPlanner` — RETURN/WITH evaluation
- `FrameJoiner` — frame merging and joining

```
class pycypher.star.ResultCache(max_size_bytes=104857600, ttl_seconds=0.0,
                                lock_timeout_seconds=None)
```

Bases: `object`

LRU cache for query results with size-bounded eviction and TTL support.

Keys are derived from the normalised query string and parameters. Values are copied DataFrames so mutations to the returned frame do not corrupt the cached entry.

The cache is automatically invalidated when the underlying Context commits a mutation (SET / CREATE / DELETE / MERGE / REMOVE).

Thread-safety:

- Mutating operations (get, put, invalidate, clear) acquire an exclusive write lock.
- Read-only operations (stats) acquire a shared read lock, allowing concurrent stats collection without blocking cache operations.
- All lock acquisitions use adaptive timeouts (scaled by operation complexity) to prevent deadlocks.
- The write-lock owner thread is tracked for deadlock diagnostics.

`__init__(max_size_bytes=104857600, ttl_seconds=0.0, lock_timeout_seconds=None)`

Initialize the result cache.

Parameters

- `max_size_bytes` (`int`) – Maximum total memory for cached DataFrames. 0 disables caching entirely.
- `ttl_seconds` (`float`) – Time-to-live per entry in seconds. 0 means entries never expire by time (only by LRU eviction or mutation-based invalidation).
- `lock_timeout_seconds` (`float` | `None`) – Base timeout for lock acquisition in seconds. `None` uses the class default (5 s). The actual timeout is scaled by a per-operation multiplier (e.g. put gets 2x the base timeout). Set to 0 for non-blocking semantics.

`clear()`

Remove all cached entries immediately.

property enabled: `bool`

Whether caching is active (`max_size_bytes > 0`).

`get(query, parameters)`

Look up a cached result.

Uses an exclusive write lock because cache hits mutate LRU order.

Returns

A copy of the cached DataFrame, or `None` on miss. Returns `None` if the lock cannot be acquired (timeout).

Return type

`DataFrame` | `None`

`invalidate()`

Bump the generation counter, lazily invalidating all entries.

Called when a mutation is committed to the Context. Existing entries are not deleted immediately — they are evicted on the next `get()` that detects the stale generation, or during LRU eviction.

`put(query, parameters, result)`

Store a query result in the cache.

If the lock cannot be acquired within the adaptive timeout, the result is silently not cached (query execution is unaffected).

`stats()`

Return cache statistics.

Uses a shared read lock so multiple threads can read stats concurrently without blocking cache operations.

```
class pycypher.star.Star(context=None, *, result_cache_max_mb=None,
                        result_cache_ttl_seconds=None)
```

Bases: `object`

Main entry point for PyCypher query execution.

Accepts a `Context` (containing `EntityTable` and `RelationshipTable` objects), parses a Cypher query string, and executes it against the registered `DataFrames` via the `BindingFrame` execution path.

Query lifecycle

1. Parse — Cypher string → AST via `GrammarParser`.
2. Plan — AST is analysed for complexity, memory budget, and timeout.
3. Execute — clauses are processed sequentially, building up a `BindingFrame` through MATCH, WHERE, WITH, and RETURN stages. Mutations (CREATE/SET/DELETE) are staged in a shadow layer and committed on success.
4. Return — the final `BindingFrame` is projected to a pandas `DataFrame`.

Example:

```
import pandas as pd
from pycypher import ContextBuilder, Star

people = pd.DataFrame({
    "__ID__": [1, 2, 3],
    "name": ["Alice", "Bob", "Carol"],
    "age": [30, 25, 35],
})
star = Star(ContextBuilder().add_entity("Person", people).build())

result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 28 RETURN p.name ORDER BY p.name"
)
```

Internally, `Star` delegates to specialized engines:

- `PatternMatcher` — MATCH clause translation
- `PathExpander` — variable-length path BFS
- `MutationEngine` — CREATE/SET/DELETE execution

- [ClauseExecutor](#) — clause dispatch and execution loop
- [QueryAnalyzer](#) — pre-execution planning
- [QueryExplainer](#) — EXPLAIN text plans

`__init__(context=None, *, result_cache_max_mb=None, result_cache_ttl_seconds=None)`
Initialize Star with a data context.

`__repr__()`
Return an informative summary for REPL/notebook display.

`available_functions()`
Return a sorted list of registered scalar function names.

`explain_query(query)`
Return a text execution plan without running the query.

`execute_query(query, *, parameters=None, timeout_seconds=None, memory_budget_bytes=None, max_complexity_score=None)`

Execute a complete Cypher query and return results as DataFrame.

Parameters

- `query` (`str` | `Any`) – Cypher query string or Query AST node.
- `parameters` (`dict[str, Any]` | `None`) – Optional dict of named query parameters.
- `timeout_seconds` (`float` | `None`) – Optional wall-clock timeout in seconds.
- `memory_budget_bytes` (`int` | `None`) – Optional peak-memory budget in bytes.
- `max_complexity_score` (`int` | `None`) – Optional ceiling for query complexity score.

Returns

DataFrame with columns matching the RETURN clause aliases.

Raises

- [ValueError](#) – If query structure is invalid.
- [NotImplementedError](#) – For unsupported clause types.
- [QueryTimeoutError](#) – If execution exceeds `timeout_seconds`.
- [QueryMemoryBudgetError](#) – If estimated memory exceeds budget.
- [QueryComplexityError](#) – If complexity score exceeds threshold.

Return type

DataFrame

`async execute_query_async(query, *, parameters=None, timeout_seconds=None, memory_budget_bytes=None, max_complexity_score=None)`

Async wrapper around [execute_query\(\)](#).

```
pycypher.star.get_cache_stats(star=None)
```

Return combined cache statistics from all PyCypher caches.

Clause Executor

Clause-by-clause execution engine for the BindingFrame IR. Handles clause-type dispatch (MATCH, WITH, RETURN, SET, DELETE, CREATE, etc.), UNWIND processing, WHERE filter application, and dead column elimination. Clause-by-clause execution engine for the BindingFrame IR.

Extracted from `pycypher.star` to separate clause dispatch and execution logic from the main orchestration facade. The `ClauseExecutor` handles:

- Clause-type dispatch (MATCH, WITH, RETURN, SET, DELETE, CREATE, etc.)
- MATCH clause handling (regular, OPTIONAL, merge)
- UNWIND clause processing with seed-frame management
- WHERE filter application
- Dead column elimination
- The main clause execution loop

```
class pycypher.clause_executor.ClauseExecutor(context, pattern_matcher, mutations, frame_joiner,
                                             projection_planner, query_analyzer)
```

Bases: `object`

Execute Cypher clauses against the BindingFrame IR.

Encapsulates clause dispatch, MATCH handling, UNWIND processing, WHERE filtering, and the main execution loop.

Parameters

- `context` (`Context`) – The PyCypher Context.
- `pattern_matcher` (`PatternMatcher`) – PatternMatcher for MATCH operations.
- `mutations` (`MutationEngine`) – MutationEngine for CREATE/SET/DELETE operations.
- `frame_joiner` (`FrameJoiner`) – FrameJoiner for frame merging.
- `projection_planner` (`ProjectionPlanner`) – ProjectionPlanner for RETURN/WITH clauses.
- `query_analyzer` (`QueryAnalyzer`) – QueryAnalyzer for pre-execution planning.

```
__init__(context, pattern_matcher, mutations, frame_joiner, projection_planner, query_analyzer)
```

```
static apply_where_filter(where_expr, result_frame, fallback_frame=None)
```

Apply a WHERE predicate to `result_frame`, with optional fallback.

First tries evaluating `where_expr` against `result_frame`. If that raises `ValueError` or `KeyError`, falls back to evaluating against `fallback_frame` and applying the boolean mask positionally.

Parameters

- `where_expr` (`Any`) – AST expression node for the WHERE predicate.
- `result_frame` (`BindingFrame`) – The frame to filter.
- `fallback_frame` (`BindingFrame` | `None`) – Optional pre-projection frame for variable lookup.

Returns

A new filtered `BindingFrame`.

Return type

`BindingFrame`

`unwind_binding_frame`(`clause`, `frame`)

Evaluate an UNWIND clause and return the exploded `BindingFrame`.

Parameters

- `clause` (`Any`) – AST `Unwind` node.
- `frame` (`BindingFrame`) – Current `BindingFrame`.

Returns

A new `BindingFrame` with rows expanded from list elements.

Return type

`BindingFrame`

`process_unwind_clause`(`clause`, `current_frame`)

Execute an UNWIND clause, seeding a synthetic frame if needed.

Parameters

- `clause` (`Any`) – The `Unwind` clause.
- `current_frame` (`Any`) – The preceding `BindingFrame`, or `None`.

Returns

A new `BindingFrame` with the unwound variable bound.

Return type

`Any`

`handle_match_clause`(`clause`, `current_frame`, `limit_hint`)

Execute a MATCH or OPTIONAL MATCH clause.

Parameters

- `clause` (`Any`) – The `Match` clause.
- `current_frame` (`Any`) – The preceding `BindingFrame`, or `None`.
- `limit_hint` (`int` | `None`) – Optional row limit for MATCH pushdown.

Returns

The updated `BindingFrame`, or an empty `pd.DataFrame` when an OPTIONAL MATCH as first clause finds no matches.

Return type

`Any` | `DataFrame`

`static require_bound_frame`(`current_frame`, `clause_name`)

Raise `ValueError` if `current_frame` is `None`.

`static frame_size(frame)`

Return row count string for a `BindingFrame`, or `'(none)'`.

`dispatch_clause(clause, current_frame, limit_hint)`

Dispatch a single clause and return the updated frame.

Returns a `pd.DataFrame` for `RETURN` clauses and early-exit `OPTIONAL MATCH`, or a `BindingFrame` / `None` for all other clauses.

`execute_query_inner(query, initial_frame=None)`

Execute a Cypher query using the `BindingFrame` IR.

Handles all clause types (`MATCH`, `WHERE`, `WITH`, `RETURN`, `SET`, `DELETE`, `CREATE`, `UNWIND`, `MERGE`, `CALL`, `FOREACH`) with dead column elimination.

Parameters

- `query` ([Any](#)) – A parsed `Query` AST node.
- `initial_frame` ([Any](#)) – Optional pre-seeded `BindingFrame`.

Returns

A `pd.DataFrame` with columns matching the `RETURN` aliases.

Return type

`DataFrame`

`execute_query_binding_frame(query)`

Execute with query-scoped shadow write atomicity.

Wraps `execute_query_inner()` in a `begin/commit/rollback` transaction.

`execute_union_query(union_query)`

Execute a `UNION [ALL]` query.

Parameters

`union_query` ([Any](#)) – A `UnionQuery` AST node.

Returns

A `pd.DataFrame` combining all sub-query results.

Return type

`DataFrame`

Pattern Matcher

Handles `MATCH` clause pattern evaluation — node matching, relationship traversal, and variable-length path expansion. Pattern matching engine for translating Cypher `MATCH` patterns to `BindingFrames`.

Extracted from [Star](#) to reduce the `god` object. The `PatternMatcher` handles all pattern-related operations:

1. Node scanning — scan a `NodePattern` into a `BindingFrame`.
2. Pattern path traversal — translate a `PatternPath` into a `BindingFrame` via scans, joins, and BFS expansion.
3. `MATCH` clause translation — combine multiple pattern paths and apply `WHERE` predicates.

Architecture

PatternMatcher

node_pattern_to_binding_frame() — single node scan
 pattern_path_to_binding_frame() — full path traversal
 match_to_binding_frame() — MATCH clause coordinator
 delegates to PathExpander for variable-length paths

```
class pycypher.pattern_matcher.VariableLengthHopParams(frame, rel_ast, scan_rel_types, rel_var,
                                                       prev_var, next_var, direction,
                                                       next_type=None, path_var_name=None,
                                                       row_limit=None, anon_counter=<factory>)
```

Bases: [object](#)

Grouped parameters for PatternMatcher._expand_variable_length_hop().

Bundles the 11 positional arguments into a single object with logical groupings, improving call-site readability and making the method signature self-documenting.

frame: [BindingFrame](#)

rel_ast: [RelationshipPattern](#)

scan_rel_types: [list\[str\]](#)

rel_var: [str](#)

prev_var: [str](#)

next_var: [str](#)

direction: [RelationshipDirection](#)

next_type: [str](#) | [None](#)

path_var_name: [str](#) | [None](#)

row_limit: [int](#) | [None](#)

anon_counter: [list\[int\]](#)

```
__init__(frame, rel_ast, scan_rel_types, rel_var, prev_var, next_var, direction,
          next_type=None, path_var_name=None, row_limit=None, anon_counter=<factory>)
```

```
class pycypher.pattern_matcher.PatternMatcher(context, path_expander, coerce_join_fn,
                                              apply_where_fn, multi_way_join_fn=None)
```

Bases: [object](#)

Translates Cypher MATCH patterns into BindingFrame instances.

Delegates BFS-based operations to [PathExpander](#).

Algorithm overview

For a Cypher pattern like `MATCH (a:Person)-[:KNOWS]->(b:Person):`

1. Node scan — `node_pattern_to_binding_frame()` scans each node pattern into a single-column `BindingFrame` (e.g. column "a" with all Person IDs).
2. Relationship join — for each relationship in the path, the relationship table is joined on source/target IDs to connect adjacent node frames.
3. Multi-path merge — when a `MATCH` has multiple comma-separated patterns, each is translated independently and then joined on shared variable names.
4. Predicate pushdown — `WHERE` predicates referencing only one path are pushed down before the cross-path join to reduce row counts.

For variable-length paths (`[*1..3]`), step 2 delegates to `PathExpander` for BFS expansion.

param context

The execution context with entity/relationship tables.

param path_expander

`PathExpander` instance for variable-length paths.

param coerce_join_fn

Callable for joining two `BindingFrames` (inner or cross join).

param apply_where_fn

Callable for applying a `WHERE` predicate to a `BindingFrame`.

param multi_way_join_fn

Optional callable for n-way joins using `LeapfrogTriejoin`.

`__init__(context, path_expander, coerce_join_fn, apply_where_fn, multi_way_join_fn=None)`

Initialize pattern matcher.

Parameters

- context (`Context`) – Execution context with entity/relationship tables.
- path_expander (`PathExpander`) – `PathExpander` for variable-length path BFS.
- coerce_join_fn (`Callable[... , BindingFrame]`) – Callable (left, right) -> `BindingFrame` for joining two frames (inner or cross join).
- apply_where_fn (`Callable[... , BindingFrame]`) – Callable (frame, predicate) -> `BindingFrame` for applying a `WHERE` predicate to a frame.
- multi_way_join_fn (`Callable[... , BindingFrame] | None`) – Optional callable (frames) -> `BindingFrame` for n-way joins using `LeapfrogTriejoin` when applicable.

`node_pattern_to_binding_frame(node, anon_counter, context_frame=None)`

Scan a single `NodePattern` into a `BindingFrame`.

Assigns a synthetic variable name for anonymous nodes. Applies inline property equality filters.

Parameters

- node (`NodePattern`) – AST `NodePattern` to scan.
- anon_counter (`list[int]`) – Mutable counter for synthetic variable names.
- context_frame (`BindingFrame | None`) – Optional preceding `BindingFrame` for unlabelled node variable resolution.

Returns

A BindingFrame with one column per node.

Return type

[BindingFrame](#)

`pattern_path_to_binding_frame(path, anon_counter, context_frame=None, row_limit=None)`

Translate a PatternPath into a BindingFrame via scans and joins.

Walks the path elements (alternating NodePattern / RelationshipPattern) and builds the frame incrementally.

Parameters

- `path` ([PatternPath](#)) – AST PatternPath.
- `anon_counter` (`list[int]`) – Mutable counter for synthetic names.
- `context_frame` ([BindingFrame](#) | None) – Optional preceding BindingFrame.
- `row_limit` (`int` | None) – If given, forward to variable-length path expansion.

Returns

A BindingFrame for this path.

Return type

[BindingFrame](#)

`match_to_binding_frame(match_clause, context_frame=None, row_limit=None)`

Translate a MATCH clause to a BindingFrame.

Translates each PatternPath separately, then joins them on shared variable names. Applies the optional WHERE predicate.

Predicate pushdown: when the MATCH has multiple pattern paths and a WHERE clause, the predicate is analysed to determine if it only references variables from a single path. If so, the filter is applied to that path's frame before the join, reducing the join's input size. Cross-path predicates are applied after the join as before.

Parameters

- `match_clause` ([Match](#)) – AST Match node.
- `context_frame` ([BindingFrame](#) | None) – Optional preceding BindingFrame.
- `row_limit` (`int` | None) – If given, forward to variable-length path expansion.

Returns

A BindingFrame satisfying the MATCH pattern.

Return type

[BindingFrame](#)

Path Expander

BFS-based path expansion for variable-length relationship patterns (e.g. (a)-[:KNOWS*1..3]->(b)). BFS-based path expansion for variable-length and shortest-path patterns.

Extracted from [Star](#) to reduce the god object. The PathExpander handles all BFS-related operations:

1. Variable-length paths — `[*m..n]` hop-bounded BFS expansion.
2. Shortest path — `shortestPath` and `allShortestPaths` BFS.

The expander operates on [BindingFrame](#) instances and returns new frames with expanded path results.

Architecture

PathExpander

`expand_variable_length_path()` — BFS with hop-bounded frontier
`shortest_path_to_binding_frame()` — BFS + min-hop filter

`class pycypher.path_expander.PathExpander(context)`

Bases: `object`

BFS-based path expansion for variable-length and shortest-path patterns.

Algorithm overview

For a pattern like `(a)-[:KNOWS*1..3]->(b)`:

1. Seed — the starting `BindingFrame` provides initial node IDs in column `a`.
2. Frontier expansion — at each hop, the frontier is joined with the relationship table to discover next-hop nodes. The frontier is deduplicated per `(start, current-tip)` pair to avoid exponential blowup in cyclic graphs.
3. Hop collection — rows at each hop in `[min_hops, max_hops]` are collected into the result.
4. Safety limits — unbounded paths (`[*]`) are capped at `__MAX_UNBOUNDED_PATH_HOPS` (20) hops. The frontier size is capped at `__MAX_FRONTIER_ROWS` (1M rows) and total accumulated results at `__MAX_BFS_TOTAL_ROWS` (5M rows); exceeding either raises `SecurityError`.

Complexity: $O(V + E * \text{max_hops})$ per seed row, where V = nodes and E = edges of the traversed type.

param `context`

The execution context with entity/relationship tables.

`__init__(context)`

Initialize path expander.

Parameters

`context` ([Context](#)) – Execution context with entity/relationship tables used for BFS frontier expansion.

`expand_variable_length_path(start_frame, start_var, rel_type, direction, end_var, end_type, min_hops, max_hops, anon_counter, path_length_col=None, row_limit=None)`

BFS hop-by-hop expansion of a variable-length relationship pattern.

Builds a `BindingFrame` containing one row per reachable `(start, end)` pair at each hop count in `[min_hops, max_hops]`. The frontier is deduplicated at each hop to avoid exponential blowup.

When `row_limit` is provided, expansion stops early once enough rows have been collected.

Parameters

- `start_frame` ([BindingFrame](#)) – `BindingFrame` whose `start_var` column seeds the BFS.
- `start_var` (`str`) – Column in `start_frame` that holds the starting node IDs.
- `rel_type` (`str`) – Relationship type to traverse.

- `direction` ([RelationshipDirection](#)) – Traversal direction.
- `end_var` (`str`) – Output column name for the reached endpoint IDs.
- `end_type` (`str` | `None`) – Entity type string for `end_var` in the type registry.
- `min_hops` (`int`) – Minimum hop count to include in results.
- `max_hops` (`int` | `None`) – Maximum hop count; `None` means unbounded.
- `anon_counter` (`list[int]`) – Mutable counter for synthetic variable names.
- `path_length_col` (`str` | `None`) – If given, adds an integer column recording hop count.
- `row_limit` (`int` | `None`) – If given, stop BFS once this many result rows accumulated.

Returns

A [BindingFrame](#) with columns from `start_frame` plus `end_var` (and optionally `path_length_col`).

Return type

[BindingFrame](#)

`shortest_path_to_binding_frame(path, anon_counter, context_frame=None, node_scanner=None)`

Execute a `shortestPath` / `allShortestPaths` pattern via BFS.

Runs BFS from the start node, then filters to the minimum-hop rows per (start, end) pair.

Parameters

- `path` ([PatternPath](#)) – [PatternPath](#) with `shortest_path_mode` set.
- `anon_counter` (`list[int]`) – Mutable counter for synthetic variable names.
- `context_frame` ([BindingFrame](#) | `None`) – Optional preceding [BindingFrame](#) for variable binding.
- `node_scanner` (`Any`) – Callable that scans a [NodePattern](#) into a [BindingFrame](#). Signature: `(node, anon_counter, context_frame) -> BindingFrame`.

Returns

A [BindingFrame](#) with the shortest-path result rows.

Return type

[BindingFrame](#)

Expression Renderer

Converts AST expression nodes into human-readable column names for RETURN/WITH clause output. Visitor-pattern expression renderer for human-readable display text.

Extracted from `_expr_display_text` to reduce cyclomatic complexity in the Star orchestrator.

The [ExpressionRenderer](#) translates Cypher AST expression nodes into short human-readable strings, used for auto-naming RETURN columns when no explicit AS alias is present.

Architecture

Uses a dispatch dictionary mapping AST node types to focused handler methods, replacing the previous 30+ isinstance chain (CC ~54) with individual methods (CC ~3 each).

```
ExpressionRenderer
render() — public dispatch entry point
__render_variable()
__render_property_lookup()
__render_function_invocation()
__render_arithmetic()
__render_comparison()
...
(one handler per AST node type)
```

```
class pycypher.expression_renderer.ExpressionRenderer
```

Bases: `object`

Render Cypher AST expression nodes as human-readable text.

Thread-safe and stateless — a single instance can be shared across queries.

Usage:

```
renderer = ExpressionRenderer()
text = renderer.render(some_ast_expr) # e.g. "n.name" or "count(n)"
```

```
__init__()
```

Initialize renderer with lazy dispatch table.

The type-to-handler dispatch table is built on the first `render()` call to avoid import-time dependency on `pycypher.ast_models`.

```
render(expression)
```

Return a short human-readable text for expression, or `None`.

Follows the unqualified-property convention: `PropertyLookup(p, 'name') → "name"`.

Parameters

expression (`Expression`) – Any Cypher AST expression node.

Returns

A string display name, or `None` if no representation is available for the expression type.

Return type

`str` | `None`

4.1.5 BindingFrame Execution

BindingFrame

The core execution abstraction — a `BindingFrame` is a named, typed `DataFrame` that tracks variable bindings through pattern matching and clause evaluation. `BindingFrame` — the core IR abstraction for the refactored query engine.

A `BindingFrame` is a `DataFrame` whose columns are named after Cypher variables (plain strings such as "p", "q", "r"). Each row is one possible assignment of entity IDs to those variables. Attributes are never stored in a `BindingFrame`; they are fetched on demand from the `Context` via `get_property()`.

This eliminates three sources of fragility in the legacy pipeline:

1. The opaque 32-hex `HASH_ID` column names produced by `Projection`.
2. The `PREFIXED_ENTITY` column names (`Person__name`) that bleed into operator logic via `_ensure_full_entity_data` and similar hacks.
3. The `variable_map` / `variable_type_map` metadata that must be threaded through every `Relation` subclass to recover which column belongs to which Cypher variable.

With `BindingFrame` the column is the variable — no lookup table required.

Usage:

```
bf = BindingFrame(
    bindings=pd.DataFrame({"p": [1, 2, 3], "q": [4, 5, 6]}),
    type_registry={"p": "Person", "q": "Person"},
    context=ctx,
)
names = bf.get_property("p", "name") # pd.Series of person names
filtered = bf.filter(names == "Alice") # BindingFrame with one row
```

`pycypher.binding_frame.PATH_HOP_COLUMN_PREFIX`: `str = '_path_hop_'`

Prefix for path-hop-count columns produced by variable-length path expansion. `star.py` writes `f"{PATH_HOP_COLUMN_PREFIX}{path_var}"` and `binding_evaluator.py` reads it when evaluating `length(path_var)`.

```
class pycypher.binding_frame.BindingFrame(bindings, type_registry, context,
                                          __property__cache=<factory>)
```

Bases: `object`

A table of variable bindings where every column is a Cypher variable name.

A `BindingFrame` is the core intermediate representation (IR) for the query engine. Each column is named after a Cypher variable ("p", "q", "r"), and each row represents one candidate assignment of entity/relationship IDs to those variables. Attributes are never stored here — they are fetched on-demand from the `Context` via `get_property()`.

Key operations

- `get_property()` — fetch attribute values via ID-keyed join
- `filter()` — apply a boolean mask (WHERE clause)
- `join()` — inner join on shared variable names (multi-MATCH)
- `cross_join()` — Cartesian product (disjoint patterns)
- `mutate()` — write property values back through shadow layer

Example:

```
# After MATCH (p:Person)-[:KNOWS]->(q:Person), the BindingFrame
# might look like:
# bindings = DataFrame({"p": [1, 1, 2], "q": [2, 3, 3]})
```

(continues on next page)

(continued from previous page)

```
# type_registry = {"p": "Person", "q": "Person"}
#
# To evaluate WHERE p.age > 25:
ages = bf.get_property("p", "age") # Series: [30, 30, 25]
filtered = bf.filter(ages > 25)    # keeps rows 0 and 1
```

bindings

DataFrame whose columns are variable names (strings) and whose values are entity or relationship IDs.

Type

`pandas.DataFrame`

type_registry

Maps each variable name to its entity or relationship type string (e.g. {"p": "Person", "r": "KNOWS"}).

Type

`dict[str, str]`

context

The query context holding entity and relationship tables.

Type

`Any`

bindings: `DataFrame`**type_registry:** `dict[str, str]`**context:** `Any`**__len__()**

Number of binding rows.

property var_names: `list[str]`

Ordered list of variable names in this frame.

entity_type(var_name)

Return the entity/relationship type for var_name.

Raises

`KeyError` – If var_name is not in the type registry.

get_property(var_name, prop_name)

Fetch entity attribute values for all rows in this frame.

Looks up the entity IDs in `bindings[var_name]`, then performs an ID-keyed lookup against the entity table stored in the context. The result is a `pd.Series` aligned with `self.bindings` (same index, same length).

Parameters

- `var_name` (`str`) – The Cypher variable name (e.g. "p").
- `prop_name` (`str`) – The entity attribute name (e.g. "name").

Returns

A `pd.Series` of property values, one per binding row.

Raises

- `KeyError` – If `var_name` is not in the type registry.
- `ValueError` – If `var_name` is not a column in bindings.

Return type

`Series`

Note

If `prop_name` does not exist as a column in the entity table, a `Series` of `pd.NA` values is returned (per Cypher null semantics) rather than raising an exception.

`get_properties_batch(var_name, prop_names)`

Fetch multiple properties for `var_name` in a single indexed join.

This is a performance optimization over calling `get_property()` multiple times for the same variable. Instead of `N` separate `Series.map()` calls, the IDs are joined against the entity table once and all requested columns are extracted simultaneously.

Falls back to per-property `get_property()` for multi-type variables or when the entity type cannot be resolved.

Parameters

- `var_name` (`str`) – The Cypher variable name (e.g. "p").
- `prop_names` (`list[str]`) – List of property names to fetch.

Returns

A dict mapping each property name to a `pd.Series` of values.

Return type

`dict[str, Series]`

`filter(mask)`

Return a new `BindingFrame` containing only rows where `mask` is `True`.

Parameters

`mask` (`Series`) – A boolean `pd.Series` aligned with `self.bindings`.

Returns

A new `BindingFrame` with the same `type_registry` and context.

Return type

`BindingFrame`

`join(other, left_col, right_col, *, join_plan=None)`

Inner-join two `BindingFrames` on a pair of columns.

Used to connect a node scan to a relationship scan (or two scans of different entity types that share a structural constraint).

After the join the redundant `right_col` is dropped when it differs from `left_col`, leaving a single unified column for the shared ID.

Parameters

- other ([BindingFrame](#)) – The right-hand BindingFrame.
- left_col ([str](#)) – Column in this frame to join on.
- right_col ([str](#)) – Column in other to join on.

Returns

A new BindingFrame whose type_registry merges both sides.

Raises

[VariableNotFoundError](#) – If either join column is absent from its frame.

Return type

[BindingFrame](#)

left_join(other, left_col, right_col)

Left-join two BindingFrames: all rows from self are preserved.

Rows in self that have no matching row in other receive NaN / None for all columns contributed by other. This implements the semantics of OPTIONAL MATCH.

Parameters

- other ([BindingFrame](#)) – The right-hand BindingFrame.
- left_col ([str](#)) – Column in this frame to join on.
- right_col ([str](#)) – Column in other to join on.

Returns

A new BindingFrame whose type_registry merges both sides.

Raises

[VariableNotFoundError](#) – If either join column is absent from its frame.

Return type

[BindingFrame](#)

cross_join(other)

Cartesian-product two BindingFrames (no join key).

Produces a frame with $\text{len}(\text{self}) \times \text{len}(\text{other})$ rows and every column from both frames. When column names collide, other's columns are suffixed with `_right` and then dropped (same behaviour as `join()`).

Parameters

other ([BindingFrame](#)) – The right-hand BindingFrame.

Returns

A new BindingFrame whose type_registry merges both sides.

Raises

[QueryMemoryBudgetError](#) – If the projected result size exceeds MAX_CROSS_JOIN_ROWS.

Return type

[BindingFrame](#)

rename(old_col, new_col, new_type=None)

Return a new BindingFrame with one column renamed.

Used by the pattern translator to promote structural join-key columns (e.g. `_tgt_r`) to named Cypher variables (e.g. `q`).

Parameters

- `old_col` (`str`) – Existing column name.
- `new_col` (`str`) – New column name.
- `new_type` (`str` | `None`) – If given, registers `new_col` with this type in the `type_registry`. If `None` and `old_col` was registered, the registry entry is moved from `old_col` to `new_col`.

Returns

A new `BindingFrame` with the renamed column.

Raises

`VariableNotFoundError` – If `old_col` is not in the bindings.

Return type

`BindingFrame`

`project(computed)`

Build an output `DataFrame` from pre-evaluated column `Series`.

This is the final step before returning results to the caller. Each key in `computed` becomes a column in the output `DataFrame`.

In Phase 1 the caller is responsible for evaluating expressions into `Series` first (e.g. using `get_property`). Phase 4 will introduce `BindingExpressionEvaluator` which accepts `Expression` objects directly.

Parameters

`computed` (`dict[str, Series]`) – Mapping from output alias to a `pd.Series` aligned with `self.bindings`.

Returns

A plain `pd.DataFrame` with one column per alias.

Return type

`DataFrame`

`mutate(var_name, prop_name, values)`

Write a new or updated property back to the entity table in context.

Aligns values by entity ID so that only the rows present in this `BindingFrame` are updated; rows not present are left unchanged.

This mutates `context.entity_mapping[type].source_obj` in place, which is the same object that `get_property` reads from.

Parameters

- `var_name` (`str`) – The Cypher variable whose entity table is updated.
- `prop_name` (`str`) – The attribute name to set or overwrite.
- `values` (`Series`) – A `pd.Series` of new values aligned with `self.bindings`.

Raises

- `VariableNotFoundError` – If `var_name` is not in the frame.
- `GraphTypeNotFoundError` – If the variable's type is not in any mapping.

`mutate_batch(var_name, properties)`

Write multiple properties back to the entity table in a single pass.

This is a batch optimisation of `mutate()` for the SET `p = {...}` map-literal expansion pattern. Instead of calling `mutate()` once per key (each of which builds a separate dict and scans the ID

column), this method builds a single updates DataFrame with all properties and applies them via a single merge + column assignment.

For k properties and n rows, `mutate()` called k times is $O(n * k)$ with k dict constructions and k ID column scans. `mutate_batch` reduces this to $O(n + k)$ — one merge pass plus per-column assignment from the merged result.

Parameters

- `var_name` (`str`) – The Cypher variable whose entity table is updated.
- `properties` (`dict[str, Series]`) – Mapping from property name to a `pd.Series` of new values, each aligned with `self.bindings`.

Raises

- `VariableNotFoundError` – If `var_name` is not in the frame.
- `GraphTypeNotFoundError` – If the variable's type is not in any mapping.

`__init__`(bindings, type_registry, context, __property_cache=<factory>)

Evaluator Protocol

Protocol interface for expression evaluation that breaks circular imports. Defines the minimal contract sub-evaluators need from the top-level expression evaluator. Protocol interface for expression evaluation — breaks circular imports.

The `ExpressionEvaluatorProtocol` defines the minimal contract that sub-evaluators (collection, exists, etc.) need from the top-level expression evaluator. By depending on this protocol instead of the concrete `BindingExpressionEvaluator`, circular import chains are eliminated.

Usage in sub-evaluators:

```
from pycypher.evaluator_protocol import ExpressionEvaluatorProtocol

def some_method(self, evaluator: ExpressionEvaluatorProtocol) -> ...:
    result = evaluator.evaluate(expr)
```

`class pycypher.evaluator_protocol.ExpressionEvaluatorProtocol(*args, **kwargs)`

Bases: `Protocol`

Minimal interface for recursive expression evaluation.

Any evaluator that provides `evaluate()` and `frame` satisfies this protocol, including `BindingExpressionEvaluator`.

property `frame`: `BindingFrame`

The `BindingFrame` against which expressions are evaluated.

`evaluate(expression)`

Evaluate an AST expression and return a per-row Series.

Parameters

- `expression` (`Expression`) – The Cypher AST expression node to evaluate.

Returns

A `pd.Series` of per-row results.

```

    Return type
        FrameSeries
    __init__(*args, **kwargs)

class pycypher.evaluator__protocol.ExpressionEvaluatorFactory(*args, **kwargs)
    Bases: Protocol

    Factory protocol for constructing evaluators from a BindingFrame.

    Sub-evaluators that need to create new evaluators for sub-frames (e.g. list comprehension, REDUCE)
    depend on this factory instead of importing the concrete class.

    __call__(frame)
        Create an evaluator bound to the given frame.

    Parameters
        frame (BindingFrame) – The BindingFrame to bind the evaluator to.

    Returns
        An ExpressionEvaluatorProtocol instance.

    Return type
        ExpressionEvaluatorProtocol
    __init__(*args, **kwargs)

```

BindingExpression Evaluator

Vectorised expression evaluator that dispatches AST expression nodes to specialised evaluator modules.
 BindingExpressionEvaluator — expression evaluator for the BindingFrame IR.

Evaluates Cypher AST expressions against a [BindingFrame](#), returning a `pd.Series` of per-row results.

Design

1. No “Relation“ dependency — property lookups go through `get__property()` instead of joining a prefixed entity `DataFrame`.
2. Clean return type — returns a single `pd.Series` (not a tuple).
3. “CaseExpression“ support — handled via a vectorised `pd.Series.where()` reduction (both searched CASE WHEN and simple CASE expr WHEN forms).
4. Aggregation-aware — `evaluate_aggregation()` returns a scalar for use in WITH and RETURN clause grouping. Supported aggregation functions: `collect`, `count`, `sum`, `avg`, `min`, `max`, `stdev` (sample, `ddof=1`), `stdevp` (population, `ddof=0`), `percentileCont(expr, p)` (linear interpolation), and `percentileDisc(expr, p)` (lower/discrete interpolation).
5. Null-safe boolean logic — AND, OR, NOT, XOR are delegated to `BooleanExpressionEvaluator` which uses Kleene three-valued logic.
6. Modular architecture — arithmetic operations are delegated to `ArithmeticExpressionEvaluator`, boolean logic to `BooleanExpressionEvaluator`, aggregations to `AggregationExpressionEvaluator`, collections to `CollectionExpressionEvaluator`, and scalar functions to `ScalarFunctionEvaluator`.

```

class pycypher.binding__evaluator.BindingExpressionEvaluator(frame)
    Bases: object

    Evaluates Cypher AST expressions against a BindingFrame.

```

All property lookups delegate to `BindingFrame.get_property()`, which performs an ID-keyed join against the entity table stored in the context. No DataFrame column prefixes are ever used.

Supported expression types:

- PropertyLookup (`p.name`)
- Variable references (`p`) — returns the ID column
- Literal values (`IntegerLiteral`, `FloatLiteral`, `StringLiteral`, `BooleanLiteral`, `NullLiteral`, `ListLiteral`)
- Arithmetic operations (`+`, `-`, `*`, `/`, `%`, `^`)
- Unary operations (`+`, `-`)
- Comparison predicates (`=`, `<>`, `<`, `>`, `<=`, `>=`)
- NullCheck predicates (`IS NULL`, `IS NOT NULL`)
- StringPredicate (`STARTS WITH`, `ENDS WITH`, `CONTAINS`, `=~`, `IN`)
- Boolean logic (`And`, `Or`, `Not`, `Xor`)
- FunctionInvocation (dispatched through `ScalarFunctionRegistry`)
- CaseExpression (simple and searched `CASE/WHEN/ELSE`)
- Graph introspection: `labels(n)`, `type(r)`, `keys(n)`, `properties(n)` (pre-evaluated intercepts that read the type registry before evaluating args)

`frame`

The `BindingFrame` against which expressions are evaluated.

`scalar_registry`

The singleton `ScalarFunctionRegistry`.

`__init__(frame)`

Initialise the evaluator.

Sub-evaluators are constructed lazily on first access via `@property` descriptors. This avoids the overhead of importing and instantiating all 5 sub-evaluator modules for every `BindingExpressionEvaluator` construction — most queries only touch 1–2 of them.

Parameters

`frame` (`BindingFrame`) – The `BindingFrame` providing variable bindings and property-lookup access to the context.

property `arithmetic_evaluator`: `ArithmeticExpressionEvaluator`

Lazy-init arithmetic expression evaluator.

property `boolean_evaluator`: `BooleanExpressionEvaluator`

Lazy-init boolean expression evaluator.

property `aggregation_evaluator`: `AggregationExpressionEvaluator`

Lazy-init aggregation expression evaluator.

property `collection_evaluator`: `CollectionExpressionEvaluator`

Lazy-init collection expression evaluator.

property `comparison_evaluator`: `ComparisonEvaluator`

Lazy-init comparison expression evaluator.

property scalar_function_evaluator: [ScalarFunctionEvaluator](#)

Lazy-init scalar function evaluator.

property string_predicate_evaluator: [StringPredicateEvaluator](#)

Lazy-init string predicate evaluator.

property exists_evaluator: [ExistsEvaluator](#)

Lazy-init exists/pattern-comprehension evaluator.

evaluate(expression)

Evaluate expression and return a per-row result Series.

The returned Series has the same length as self.frame and is indexed 0 ... N-1.

Parameters

expression ([Expression](#)) – Any supported Cypher AST expression node.

Returns

A pd.Series of per-row values.

Raises

- [NotImplementedError](#) – For expression types not yet handled.
- [ValueError](#) – For semantic errors (unknown variable, missing property).

Return type

FrameSeries

evaluate_aggregation(agg_expression)

Evaluate an aggregation expression and return a scalar value.

Architecture Loop 280 - Phase 3: Delegates to AggregationExpressionEvaluator.

Supports collect, count / COUNT(*), sum, avg, min, max. Also handles [Arithmetic](#) nodes whose branches contain aggregations (e.g. count(*) + 1 or sum(p.salary) * 2).

Parameters

agg_expression ([Expression](#)) – A [FunctionInvocation](#), [CountStar](#), or [Arithmetic](#) node.

Returns

A scalar aggregated value.

Raises

[ValueError](#) – For unsupported aggregation functions or missing arguments.

Return type

Any

evaluate_aggregation_grouped(agg_expression, group_df, group_key_aliases)

Vectorised grouped aggregation — one scalar per group.

Evaluates agg_expression's inner argument once over the entire frame, then uses pd.Series.groupby().agg() to produce one aggregated value per group. This replaces the previous pattern of creating a new BindingFrame and [BindingExpressionEvaluator](#) per group, cutting O(N_groups) object allocations and O(N_groups × frame_size) mask operations down to a single vectorised pandas pass.

Parameters

- agg_expression ([Expression](#)) – A [FunctionInvocation](#), [CountStar](#), or [Arithmetic](#) aggregation node.

- `group_df` (`pd.DataFrame`) – DataFrame of evaluated GROUP BY key columns (one row per frame row, columns named by `group_key_aliases`).
- `group_key_aliases` (`list[str]`) – Ordered list of GROUP BY column names in `group_df`.

Returns

A `pd.Series` with one value per unique group (in first-seen order, matching the order produced by `group_df.groupby(..., sort=False)`), or `None` when the expression cannot be vectorised (e.g. Arithmetic wrapping aggregations) — the caller should fall back to per-group evaluation in that case.

Return type

`pd.Series` | `None`

Arithmetic Evaluator

Handles `+`, `-`, `*`, `/`, `%`, `^` with null propagation and type coercion. `ArithmeticExpressionEvaluator` - Specialized evaluator for arithmetic and comparison operations.

This module provides focused evaluation of arithmetic operations extracted from the `BindingExpressionEvaluator` god object. This is Phase 1 of the systematic refactoring to decompose the 2904-line evaluator into specialized, maintainable components.

Architecture Loop 277 - Phase 1 Implementation

```
class pycypher.arithmetic_evaluator.ArithmeticExpressionEvaluator(frame)
```

Bases: `object`

Specialized evaluator for arithmetic and comparison expressions.

This class handles all arithmetic operations (`+`, `-`, `*`, `/`, `%`, `^`), comparison operations (`=`, `<>`, `<`, `>`, `<=`, `>=`), and unary operations (`+x`, `-x`) that were previously part of the `BindingExpressionEvaluator` god object.

The evaluator maintains identical semantics to the original implementation while providing a focused, testable interface for arithmetic operations.

```
__init__(frame)
```

Initialize arithmetic evaluator with binding frame context.

Parameters

`frame` (`BindingFrame`) – `BindingFrame` providing variable and expression evaluation context

```
evaluate_arithmetic(op, left_expr, right_expr, expression_evaluator)
```

Evaluate a binary arithmetic expression.

Handles arithmetic operations (`+`, `-`, `*`, `/`, `%`, `^`) with proper null handling, temporal arithmetic support, and Cypher-compliant semantics.

Parameters

- `op` (`str`) – Arithmetic operator string
- `left_expr` (`Any`) – Left operand expression
- `right_expr` (`Any`) – Right operand expression
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with arithmetic results

Raises

- `UnsupportedOperatorError` – If operator is not supported
- `TypeError` – If operands have incompatible types

Return type

FrameSeries

`evaluate_comparison(op, left_expr, right_expr, expression_evaluator)`

Evaluate a binary comparison expression.

Handles comparison operations (`=`, `<>`, `<`, `>`, `<=`, `>=`) with proper three-valued logic (null handling) consistent with Cypher semantics.

Parameters

- `op` (`str`) – Comparison operator string
- `left_expr` (`Any`) – Left operand expression
- `right_expr` (`Any`) – Right operand expression
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Boolean Series with comparison results

Raises

`UnsupportedOperatorError` – If operator is not supported

Return type

FrameSeries

`evaluate_unary(op, operand_expr, expression_evaluator)`

Evaluate a unary arithmetic operator.

Handles unary operations (`+x`, `-x`) with proper null propagation.

Parameters

- `op` (`str`) – Unary operator string (`+` or `-`)
- `operand_expr` (`Any`) – Expression to apply operator to
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with unary operation results

Raises

`UnsupportedOperatorError` – If operator is not supported

Return type

FrameSeries

Boolean Evaluator

Handles AND, OR, NOT, XOR with Cypher's ternary null logic. `BooleanExpressionEvaluator` - Specialized evaluator for boolean logic operations.

This module provides focused evaluation of boolean operations extracted from the `BindingExpressionEvaluator` god object. This is Phase 2 of the systematic refactoring to decompose the 2904-line evaluator into specialized, maintainable components.

Architecture Loop 277 - Phase 2 Implementation

`pycypher.boolean_evaluator.kleene_and(left, right)`

Kleene three-valued AND: null AND false = false; null AND true = null.

Vectorised via numpy — no Python-level loop. `isna()` catches all null sentinels (`None`, `np.nan`, `pd.NA`), making this more correct than the previous implementation which only checked `None` and `NaN` floats.

`pycypher.boolean_evaluator.kleene_or(left, right)`

Kleene three-valued OR: null OR true = true; null OR false = null.

Vectorised via numpy — no Python-level loop.

`pycypher.boolean_evaluator.kleene_xor(left, right)`

Kleene three-valued XOR: null XOR anything = null.

Vectorised via numpy — no Python-level loop.

`pycypher.boolean_evaluator.kleene_not(s)`

Kleene three-valued NOT: NOT true = false, NOT false = true, NOT null = null.

Vectorised via numpy — no Python-level loop. `isna()` catches all null sentinels (`None`, `np.nan`, `pd.NA`), consistent with the other Kleene functions.

`class pycypher.boolean_evaluator.BooleanExpressionEvaluator(frame)`

Bases: `object`

Specialized evaluator for boolean logic expressions.

This class handles all boolean operations (AND, OR, NOT, XOR) with proper Kleene three-valued logic semantics that were previously part of the `BindingExpressionEvaluator` god object.

The evaluator maintains identical semantics to the original implementation while providing a focused, testable interface for boolean operations.

`__init__(frame)`

Initialize boolean evaluator with binding frame context.

Parameters

`frame` (`BindingFrame`) – `BindingFrame` providing variable and expression evaluation context

`evaluate_and(operands, expression_evaluator)`

Evaluate Cypher AND — Kleene three-valued left-fold.

Parameters

- `operands` (`list[Expression]`) – List of expressions to AND together
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with boolean AND results using Kleene logic

Return type

FrameSeries

`evaluate_or(operands, expression_evaluator)`

Evaluate Cypher OR — Kleene three-valued left-fold.

Parameters

- `operands` (`list[Expression]`) – List of expressions to OR together
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with boolean OR results using Kleene logic

Return type

FrameSeries

`evaluate_not(operand_expr, expression_evaluator)`

Evaluate Cypher NOT expr — three-valued boolean negation.

Three-valued logic: NOT true → false, NOT false → true, NOT null → null. The WHERE clause's `fillna(False)` converts the null to false for filtering, correctly excluding null rows.

Parameters

- `operand_expr` (`Expression`) – Expression to negate
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with boolean NOT results using Kleene logic

Return type

FrameSeries

`evaluate_xor(operands, expression_evaluator)`

Evaluate Cypher XOR — Kleene three-valued left-fold (null XOR x = null).

Parameters

- `operands` (`list[Expression]`) – List of expressions to XOR together
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

Series with boolean XOR results using Kleene logic

Return type
FrameSeries

`evaluate_bool_chain(key, operands, expression_evaluator)`

Evaluate a multi-operand boolean fold using the boolean fold ops table.

All operands are null-coerced to False before the binary operation so that missing values do not propagate through filter predicates.

Parameters

- `key` (`str`) – One of “and”, “or”, or “xor”
- `operands` (`list[Expression]`) – List of Cypher AST expression nodes
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

A boolean Series of per-row results

Raises

`ValueError` – If key is not a supported boolean operation

Return type
FrameSeries

Comparison Evaluator

Handles `=`, `<>`, `<`, `>`, `<=`, `>=`, `IS NULL`, `IS NOT NULL`, unary operators, and CASE expressions. ComparisonEvaluator — comparison, null-check, unary, and CASE expression evaluation.

Extracted from `pycypher.binding_evaluator` to isolate the “dispatch table + null propagation” family of operations into a focused, independently testable module.

Handles:

- Binary comparison predicates (`=`, `<>`, `<`, `>`, `<=`, `>=`)
- Null-check predicates (`IS NULL`, `IS NOT NULL`)
- Unary arithmetic operators (`+`, `-`)
- CASE expressions (searched and simple forms)

`class pycypher.comparison_evaluator.ComparisonEvaluator(frame)`

Bases: `object`

Evaluates comparison, null-check, unary, and CASE expressions.

All methods accept an evaluator parameter — the parent `BindingExpressionEvaluator` — so that sub-expressions can be recursively evaluated without circular imports.

`frame`

The BindingFrame providing variable bindings.

`__init__(frame)`

Initialise with a binding frame.

Parameters

`frame` (`BindingFrame`) – The BindingFrame against which expressions are evaluated.

`evaluate_comparison(op, left_expr, right_expr, evaluator)`

Evaluate a binary comparison expression (=, <>, <, >, etc.).

Dispatches to the `__CMP__OPS` table, which maps operator strings to vectorised pandas comparison operations.

Parameters

- `op` (`str`) – Comparison operator string (e.g. "=", "<>", "<=").
- `left_expr` (`Expression`) – Left-hand operand expression.
- `right_expr` (`Expression`) – Right-hand operand expression.
- `evaluator` (`ExpressionEvaluatorProtocol`) – Parent expression evaluator for recursive evaluation.

Returns

A boolean `pd.Series` of per-row results.

Raises

`UnsupportedOperatorError` – If `op` is not a supported comparison operator.

Return type

`FrameSeries`

`evaluate_null_check(op, operand_expr, evaluator)`

Evaluate an IS NULL or IS NOT NULL predicate.

Parameters

- `op` (`str`) – One of "IS NULL" or "IS NOT NULL".
- `operand_expr` (`Expression`) – The expression whose nullness is tested.
- `evaluator` (`ExpressionEvaluatorProtocol`) – Parent expression evaluator for recursive evaluation.

Returns

A boolean `pd.Series` of per-row results.

Raises

`UnsupportedOperatorError` – If `op` is not a supported null check operator.

Return type

`FrameSeries`

`evaluate_unary(op, operand_expr, evaluator)`

Evaluate a unary arithmetic operator (+ or -).

Unary + is a no-op; unary - negates the series.

Parameters

- `op` (`str`) – Unary operator string — "+" or "-".
- `operand_expr` (`Expression`) – The expression to apply the operator to.
- `evaluator` (`ExpressionEvaluatorProtocol`) – Parent expression evaluator for recursive evaluation.

Returns

A numeric `pd.Series` of per-row results.

Raises

`UnsupportedOperatorError` – If `op` is not a supported unary operator.

Return type
FrameSeries

`evaluate_case(case_expr, when_clauses, else_expr, evaluator)`

Evaluate a CASE expression vectorially using `pd.Series.where`.

Supports both searched CASE (CASE WHEN cond THEN val ...) and simple CASE (CASE expr WHEN match THEN val ...).

The algorithm iterates WHEN clauses in reverse and repeatedly applies:

```
result = then_series.where(condition_series, result)
```

so that the first matching WHEN clause wins.

Parameters

- `case_expr` ([Expression](#) | None) – The simple CASE discriminant expression, or None for a searched CASE.
- `when_clauses` (`list`[[WhenClause](#)]) – Ordered list of [WhenClause](#) nodes.
- `else_expr` ([Expression](#) | None) – Optional ELSE expression; produces None when absent.
- `evaluator` ([ExpressionEvaluatorProtocol](#)) – Parent expression evaluator for recursive evaluation.

Returns

A `pd.Series` of per-row CASE results.

Return type
FrameSeries

String Predicate Evaluator

Handles STARTS WITH, ENDS WITH, CONTAINS, IN, and regex matching with null propagation. `StringPredicateEvaluator` — string and membership predicate evaluation.

Extracted from `pycypher.binding_evaluator` to isolate the string predicate family (STARTS WITH, ENDS WITH, CONTAINS, `=~`, IN, NOT IN) into a focused, independently testable module.

Includes ReDoS protection for the `=~` regex operator.

`class pycypher.string_predicate_evaluator.StringPredicateEvaluator`

Bases: `object`

Evaluates string and membership predicates.

Supported operators:

- "STARTS WITH" — vectorised `str.startswith`
- "ENDS WITH" — vectorised `str.endswith`
- "CONTAINS" — vectorised `str.contains` (literal, not regex)
- "`=~`" — full-match regex via `str.fullmatch` with ReDoS protection
- "IN" — membership test against a list (three-valued logic)
- "NOT IN" — inverse membership test (three-valued logic)

`evaluate_string_predicate(op, left_expr, right_expr, evaluator)`

Evaluate a string or membership predicate.

The right-hand operand is evaluated and its first element is used as the scalar pattern/set for all rows — Cypher string predicates always compare against a constant pattern.

Parameters

- `op` (`str`) – The predicate operator string.
- `left_expr` (`Expression`) – The expression producing the string or value to test.
- `right_expr` (`Expression`) – The expression producing the pattern/list to test against.
- `evaluator` (`ExpressionEvaluatorProtocol`) – Parent expression evaluator for recursive evaluation.

Returns

A boolean `pd.Series` of per-row results.

Raises

- `TypeError` – If `left_expr` evaluates to a non-string, non-null `Series` and `op` is a string accessor operator.
- `UnsupportedOperatorError` – If `op` is not a recognised string predicate operator.

Return type

`FrameSeries`

EXISTS Evaluator

Handles EXISTS { MATCH ... } subquery evaluation and pattern comprehensions with batched execution. `ExistsEvaluator` — EXISTS predicate and pattern comprehension evaluation.

Extracted from `pycypher.binding_evaluator` to isolate the EXISTS and pattern comprehension logic (the two largest inline method families) into a focused, independently testable module.

Handles:

- EXISTS { (a)-[:TYPE]->(b) } — single-hop pattern existence check
- EXISTS { MATCH ... WHERE ... } — full subquery existence check
- [(a)-[:TYPE]->(b) WHERE pred | expr] — pattern comprehensions

`class pycypher.exists_evaluator.ExistsEvaluator(frame)`

Bases: `object`

Evaluates EXISTS predicates and pattern comprehensions.

Follows the established evaluator pattern: takes a `BindingFrame` at init time and receives the parent evaluator as a callback parameter on each public method.

Parameters

`frame` (`BindingFrame`) – The current binding frame to evaluate against.

`__init__(frame)`

Initialise with the current binding frame.

Parameters

`frame` (`BindingFrame`) – The `BindingFrame` to evaluate EXISTS predicates and pattern comprehensions against.

`evaluate_exists(content, evaluator)`

Evaluate an EXISTS { pattern } predicate.

For each row in the current BindingFrame, returns True if the inner pattern has at least one match anchored on the row's bound variables, and False otherwise.

Parameters

- `content` (Any) – The content of the EXISTS clause — either a [Pattern](#) or a [Query](#).
- `evaluator` ([ExpressionEvaluatorProtocol](#)) – The parent BindingExpressionEvaluator for recursive expression evaluation.

Returns

A pd.Series of bool values, one per row.

Return type

FrameSeries

`evaluate_pattern_comprehension(pc, evaluator)`

Evaluate a Cypher pattern comprehension row by row.

Supports single-hop directed patterns:

- `[(src)-[:TYPE]->(tgt) | map_expr]`
- `[(src)-[:TYPE]->(tgt) WHERE pred | map_expr]`
- `[(src)-[:TYPE]->(tgt) | map_expr]` (incoming direction)

Parameters

- `pc` ([PatternComprehension](#)) – The [PatternComprehension](#) AST node.
- `evaluator` ([ExpressionEvaluatorProtocol](#)) – The parent BindingExpressionEvaluator for recursive expression evaluation.

Returns

A pd.Series of lists, one per row in the current BindingFrame.

Raises

[PatternComprehensionError](#) – If the pattern is not a single-hop path.

Return type

FrameSeries

Aggregation Evaluator

Handles `count()`, `sum()`, `avg()`, `collect()`, `stDev()`, etc. with grouped and full-table aggregation. [AggregationExpressionEvaluator](#) - Specialized evaluator for aggregation operations.

This module provides focused evaluation of aggregation operations extracted from the BindingExpressionEvaluator god object. This is Phase 3 of the systematic refactoring to decompose the 2904-line evaluator into specialized, maintainable components.

Architecture Loop 280 - Phase 3 Implementation

```
pycypher.aggregation_evaluator.KNOWN_AGGREGATIONS: frozenset[str] = frozenset({'avg', 'collect', 'count', 'max', 'min', 'percentilecont', 'percentiledisc', 'stdev', 'stdevp', 'sum'})
```

Complete set of known aggregation function names for validation and error messages.

```
class pycypher.aggregation_evaluator.AggregationExpressionEvaluator(frame)
```

Bases: `object`

Specialized evaluator for aggregation expressions.

This class handles all aggregation operations (SUM, AVG, MIN, MAX, COUNT, percentiles, standard deviation) that were previously part of the `BindingExpressionEvaluator` god object.

The evaluator maintains identical semantics to the original implementation while providing a focused, testable interface for aggregation operations.

```
__init__(frame)
```

Initialize aggregation evaluator with binding frame context.

Parameters

`frame` (`BindingFrame`) – `BindingFrame` providing variable and expression evaluation context

```
evaluate_aggregation(agg_expression, expression_evaluator)
```

Evaluate an aggregation expression and return a scalar value.

Supports collect, count / COUNT(*), sum, avg, min, max. Also handles `Arithmetic` nodes whose branches contain aggregations (e.g. `count(*) + 1` or `sum(p.salary) * 2`).

Parameters

- `agg_expression` (`FunctionInvocation` | `CountStar` | `Arithmetic`) – A `FunctionInvocation`, `CountStar`, or `Arithmetic` node.
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

A scalar aggregated value.

Raises

`ValueError` – For unsupported aggregation functions or missing arguments.

Return type

Any

```
evaluate_aggregation_grouped(agg_expression, group_df, group_key_aliases,
                             expression_evaluator)
```

Vectorised grouped aggregation — one scalar per group.

Parameters

- `agg_expression` (`FunctionInvocation` | `CountStar`) – A `FunctionInvocation` or `CountStar` aggregation node.
- `group_df` (`pd.DataFrame`) – `DataFrame` with the grouping structure.
- `group_key_aliases` (`list[str]`) – List of group key column aliases.
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation

Returns

A Series of aggregated values (one per group), or None if the aggregation function is unsupported.

Return type
pd.Series | None

Aggregation Planner

Detects aggregation expressions in RETURN/WITH items and plans grouped vs full-table aggregation dispatch. AggregationPlanner — aggregation detection and grouped evaluation.

Extracted from [pycypher.star](#) to isolate the cohesive aggregation planning family (`_contains_aggregation`, `_aggregate_items`) into a focused, independently testable module.

Handles:

- Recursive aggregation detection in expression trees
- Three aggregation modes: no-agg, full-table, grouped
- Dual-purpose min/max function disambiguation
- Vectorised grouped aggregation with per-group fallback

class `pycypher.aggregation_planner.AggregationPlanner`

Bases: `object`

Detects aggregations in expressions and evaluates projection items.

Stateless — all state is passed via method parameters.

`contains_aggregation(expression)`

Check recursively whether expression contains any aggregate function.

Walks the full expression tree so that aggregations nested inside arithmetic or logical operators are detected correctly — e.g. `count(p.name) + 1` or `sum(p.salary) * 2`.

Parameters

`expression` (`Any`) – AST expression node to inspect.

Returns

True if the expression or any descendant is an aggregation function.

Return type

`bool`

`aggregate_items(items, frame)`

Evaluate projection items (with optional aggregation) against a `BindingFrame`.

Alias inference must be complete on all items before calling this method.

Handles three modes:

- No aggregation — vectorized expression evaluation, one column per item.
- Full-table aggregation — reduces all rows to a single aggregated row.
- Grouped aggregation — groups by non-aggregation keys, computes aggregations per group.

Parameters

- `items` (`list[Any]`) – List of `ReturnItem` / `WithItem` nodes with `.alias` set.
- `frame` (`BindingFrame`) – Input `BindingFrame`.

Returns

A plain `pd.DataFrame`.

Return type
pd.DataFrame

Collection Evaluator

Handles list/map operations: [idx], keys(), range(), list comprehensions, REDUCE, and quantifier predicates (all(), any()). Collection Expression Evaluator for PyCypher.

Architecture Loop 283 - Collection operations extracted from BindingExpressionEvaluator god object into focused, testable, and maintainable CollectionExpressionEvaluator.

This module contains all collection-related expression evaluation functionality: - List comprehensions - Quantifiers (ANY/ALL/NONE) - REDUCE expressions - Pattern comprehensions - Slicing operations - Property lookup - Map literals and projections

The evaluator follows the established delegation pattern from previous god object extractions (AggregationExpressionEvaluator, ArithmeticExpressionEvaluator).

```
class pycypher.collection_evaluator.CollectionExpressionEvaluator(frame)
```

Bases: `object`

Evaluates collection-related expressions.

This class handles complex collection operations that were previously embedded within the BindingExpressionEvaluator god object. By extracting these operations, we achieve better separation of concerns and improved maintainability.

Parameters

frame (`BindingFrame`) – The BindingFrame containing the current evaluation context.

```
__init__(frame)
```

Initialize collection evaluator with binding frame context.

Parameters

frame (`BindingFrame`) – BindingFrame providing variable bindings and context for collection expression evaluation.

```
eval_property_lookup(var_expr, prop_name, expression_evaluator)
```

Evaluate a PropertyLookup expression (expr.prop).

Handles two cases:

- Variable target — delegates to `get_property()` which resolves the property from the entity/relationship tables in the current context.
- Any other expression target — evaluates var_expr to obtain a pd.Series of values. Elements that are dict (e.g. produced by `MapLiteral` or `MapProjection`) have prop_name extracted via dict.get; non-dict elements yield None (null).

Parameters

- var_expr (`Expression`) – The expression to the left of the ..
- prop_name (`str`) – The property key to look up.
- expression_evaluator (`ExpressionEvaluatorProtocol`) – Main expression evaluator for recursive evaluation.

Returns

A pd.Series of extracted property values, one per frame row.

Return type
FrameSeries

`eval_slicing(coll_expr, start_expr, end_expr, expression_evaluator)`

Evaluate `coll[from..to]` slicing.

Both bounds are optional and inclusive/exclusive per Cypher semantics (`list[a..b]` returns elements at indices `a` up to but not including `b`, identical to Python `seq[a:b]`). Either bound being `None` means open-ended.

Works for lists, tuples, and strings. Out-of-bounds indices are handled silently by Python's slice semantics.

Parameters

- `coll_expr` ([Expression](#)) – Expression that evaluates to list or string to slice.
- `start_expr` ([Expression](#) | `None`) – Start index expression (can be `None`).
- `end_expr` ([Expression](#) | `None`) – End index expression (can be `None`).
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of sliced values.

Return type

`FrameSeries`

`eval_list_comprehension(lc, expression_evaluator)`

Evaluate a Cypher list comprehension in a single vectorised pass.

Implements `[var IN list_expr WHERE where_expr | map_expr]`.

All `(row_idx, element)` pairs are exploded into a single flat [BindingFrame](#) so that the `WHERE` predicate and map expression are each evaluated once across all elements rather than once per element. Per-row lists are reconstructed from the surviving `(row_idx, mapped_value)` pairs.

Parameters

- `lc` ([ListComprehension](#)) – The ListComprehension AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of lists, one per row in the current `BindingFrame`.

Return type

`FrameSeries`

`eval_quantifier(q, expression_evaluator)`

Evaluate quantifier expressions (`ANY/ALL/NONE`).

Performance Loop 290: Delegates to vectorized implementation for massive performance improvement ($O(\text{rows} \times \text{elements}) \rightarrow O(1)$ `BindingFrame` allocation).

Parameters

- `q` ([Quantifier](#)) – Quantifier AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of boolean results.

Return type
FrameSeries

`eval_quantifier_vectorized(q, expression_evaluator)`

Vectorized quantifier evaluation using explode-evaluate-group strategy.

Performance Loop 290: Replaces $O(\text{rows} \times \text{elements})$ evaluation pattern with single exploded frame evaluation, reducing $10,000 \rightarrow 1$ BindingFrame allocation.

Strategy: 1. Explode all (row, element) pairs into one flat frame 2. Evaluate WHERE condition once on flattened frame 3. Group results back by original row and apply quantifier logic

Parameters

- `q` ([Quantifier](#)) – Quantifier AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of boolean results.

Return type
FrameSeries

`eval_reduce(r, expression_evaluator)`

Evaluate REDUCE expressions using batch-per-step vectorization.

Performance Loop 291: Replaces $O(\text{rows} \times \text{elements})$ evaluation pattern with batch-per-step approach, reducing BindingFrame allocations from $O(\text{rows} \times \text{max_elements})$ to $O(\text{max_elements})$.

Parameters

- `r` ([Reduce](#)) – Reduce AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of reduced values.

Return type
FrameSeries

`eval_pattern_comprehension(pc, expression_evaluator)`

Evaluate pattern comprehension expressions.

Parameters

- `pc` ([PatternComprehension](#)) – PatternComprehension AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of pattern comprehension results.

Return type
FrameSeries

`eval_map_literal(map_literal, expression_evaluator)`

Evaluate `{key: expr, ...}` map literal.

If the entries (converted expression forms) are populated, each expression is evaluated per-row. Otherwise the raw value dict (primitives only) is returned as-is.

Accepts the actual [MapLiteral](#) AST node directly — entries is a dict[str, Expression] and value is a dict[str, Any].

Parameters

- `map_literal` ([MapLiteral](#)) – The [MapLiteral](#) AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of map objects.

Return type

`FrameSeries`

`eval_map_projection(mp, expression_evaluator)`

Evaluate `n{.prop, key: expr, .*}` map projection.

Returns a `pd.Series` of dicts — one per row — where each dict contains the projected keys and their values.

Three element forms are supported:

- `MapElement(property="name")` (no expression) — copies the named property from the source variable.
- `MapElement(property="k", expression=expr)` — evaluates `expr` and stores the result under key `"k"`.
- `MapElement(all_properties=True)` — includes every non-ID property of the source variable (`.*` wildcard).

Parameters

- `mp` ([MapProjection](#)) – [MapProjection](#) AST node.
- `expression_evaluator` ([ExpressionEvaluatorProtocol](#)) – Main expression evaluator for recursive evaluation.

Returns

A `pd.Series` of projected map objects.

Return type

`FrameSeries`

Scalar Functions

Built-in Cypher scalar function registry (`toString()`, `toInteger()`, `trim()`, `abs()`, `math`, `string`, `temporal`, and `type-predicate` functions). Scalar function registry and implementations for Cypher expressions.

Scalar functions operate on entire pandas Series at once (vectorized across rows), as opposed to aggregation functions which reduce multiple rows to one value. Functions never call Python per-row loops where a vectorised alternative exists.

Vectorisation strategy by category

- Math (`abs`, `ceil`, `floor`, `sign`, `sqrt`, `cbrt`, `log`, `log2`, `log10`, `exp`, `pow`, `hypot`, `fmod`): numpy C-level operations via a `__make_math1_np` / `__make_trig1_np` factory. One numpy array call per Series.
- Trigonometric (`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, `cot`, `haversin`, `degrees`, `radians`): same numpy factory.

- String predicates (startsWith, endsWith, contains): pandas .str Cython accessor; contains always uses regex=False for literal matching.
- String transforms (toUpper, toLower, trim, ltrim, rtrim, etc.): pandas .str accessor.
- Type conversion (toInteger, toFloat): pd.to_numeric + numpy np.fix; no per-row Python.
- All remaining categories use pd.Series.apply() because their logic is inherently element-wise (e.g. list operations, temporal parsing, encoding). round() uses Python's decimal module (no numpy equivalent). It supports an optional third mode argument selecting one of seven Neo4j 5.x rounding modes: HALF_UP (default, ties away from zero), HALF_DOWN (ties toward zero), HALF_EVEN (banker's rounding), CEILING, FLOOR, UP (always away from zero), DOWN (truncation).

This module provides: - ScalarFunctionRegistry: Singleton registry for managing scalar functions - Built-in function implementations:

- String: toUpper, toLower, upper, lower, trim, ltrim, rtrim, substring, size, left, right, replace, split, reverse, length, isEmpty
- Extended string: lpad, rpad, repeat, btrim, indexOf, charAt, char, charCodeAt, normalize, toStringOrNull, startsWith, endsWith, contains, byteSize, join
- Type conversion: toString, toInteger, toFloat, toBoolean, toBooleanOrNull, toIntegerOrNull, toFloatOrNull
- Math: abs, ceil, floor, round, sign, sqrt, cbrt, log, exp, log10, pow, sinh, cosh, tanh, log2, hypot, fmod, gcd, lcm
- Bitwise: bitAnd, bitOr, bitXor, bitNot, bitShiftLeft, bitShiftRight
- Trigonometric: sin, cos, tan, asin, acos, atan, atan2, cot, haversin
- Angle conversion: degrees, radians
- Constants and random: pi, e, rand
- List: head, last, tail, range, sort, flatten, toStringList, toIntegerList, toFloatList, toBooleanList, toList, min, max
- Map: keys, values, properties
- Temporal (parse): date, datetime, localdatetime, duration
- Temporal (truncate): date.truncate, datetime.truncate, localdatetime.truncate
- Temporal (now): timestamp, localtime, localdate
- Type introspection: valueType
- Type predicates: isString, isInteger, isFloat, isBoolean, isList, isMap
- Hash & encoding: md5, sha1, sha256, encodeBase64, decodeBase64
- Utility: coalesce, id, elementId, nullIf, isNaN, isInfinite, isFinite, infinity, randomUUID, exists
- Function aliases: now (timestamp), len (length), str (toString), int (toInteger), float (toFloat), bool (toBoolean)
- Plugin architecture for registering custom functions

Example

```

>>> import pandas as pd
>>> from pycypher.scalar_functions import ScalarFunctionRegistry
>>>
>>> registry = ScalarFunctionRegistry.get_instance()
>>> input_series = pd.Series(['hello', 'world'])
>>> result = registry.execute("toUpper", [input_series])
>>> print(result.tolist())
['HELLO', 'WORLD']

```

```

class pycypher.scalar_functions.FunctionMetadata(name, callable, min_args, max_args, description,
                                                example)

```

Bases: `object`

Metadata about a registered scalar function.

`name`

Function name (display version, preserves case)

Type
`str`

`callable`

Function implementation (accepts Series args, returns Series)

Type
`Callable[..., pd.Series]`

`min_args`

Minimum number of required arguments

Type
`int`

`max_args`

Maximum number of arguments (None = unlimited)

Type
`int | None`

`description`

Human-readable description

Type
`str`

`example`

Example usage string

Type
`str`

name: `str`

callable: `Callable[..., pd.Series]`

min_args: `int`


```

max_args: int | None

description: str

example: str

__init__(name, callable, min_args, max_args, description, example)

```

```
class pycypher.scalar_functions.ScalarFunctionRegistry
```

Bases: `object`

Singleton registry for scalar functions.

This registry manages all scalar functions available in Cypher expressions. Functions are registered at module import time and can be executed by name.

Architecture: - Singleton pattern ensures single source of truth - Function names are case-insensitive for Cypher compatibility - All functions operate on pandas Series (vectorized operations) - Returns Series with same length as inputs

WHERE clause usage: all registered functions are safe for use in WHERE clause predicates (MATCH ... WHERE f(p.prop) = value). They operate row-by-row and are evaluated by ExpressionEvaluator. `__evaluate_scalar_function` which joins against the entity table on-demand to fetch property values.

NOT permitted in WHERE: aggregation functions (count, sum, avg, min, max, collect) are intentionally absent from this registry. The query translator in star.py guards against aggregations in WHERE predicates and raises a `ValueError` with an actionable message before evaluation begins.

Null safety contract: every registered function must return NaN/None for null inputs rather than raising. All built-in functions satisfy this contract. Custom functions registered via `register_function` should follow the same rule.

Example

```

>>> registry = ScalarFunctionRegistry.get_instance()
>>> registry.register_function(
...     name="double",
...     callable=lambda s: s * 2,
...     min_args=1,
...     max_args=1,
...     description="Double the value",
...     example="double(3) → 6",
... )
>>> result = registry.execute("double", [pd.Series([1, 2, 3])])

```

```
__init__()
```

Initialize registry with empty function map.

Note

Use `get_instance()` instead of direct instantiation to ensure singleton.

`classmethod get_instance()`

Get or create singleton instance.

Returns

The singleton `ScalarFunctionRegistry` instance

Return type

`ScalarFunctionRegistry`

`register_function(name, callable, min_args, max_args=None, description="", example="")`

Register a scalar function.

Parameters

- `name` (`str`) – Function name (case-insensitive)
- `callable` (`Callable[... , pd.Series]`) – Function implementation (accepts Series args, returns Series)
- `min_args` (`int`) – Minimum number of required arguments
- `max_args` (`int` | `None`) – Maximum number of arguments (`None` = unlimited)
- `description` (`str`) – Human-readable description
- `example` (`str`) – Example usage string

Example

```
>>> registry.register_function(
...     name="toUpper",
...     callable=lambda s: s.str.upper(),
...     min_args=1,
...     max_args=1,
...     description="Convert string to uppercase",
...     example="toUpper('hello') → 'HELLO'"
... )
```

`has_function(name)`

Check if a function is registered.

Parameters

`name` (`str`) – Function name (case-insensitive)

Returns

True if function is registered, False otherwise

Return type

`bool`

`list_functions()`

Return a sorted list of all registered function names.

Names are lowercased, matching Cypher's case-insensitive semantics.

Returns

Sorted list of registered function names.

Return type

`list[str]`

`execute(name, args, **kwargs)`

Execute a registered scalar function.

Parameters

- `name` (`str`) – Function name
- `args` (`list[Series]`) – List of argument Series
- `**kwargs` (`Any`) – Additional keyword arguments

Returns

Result Series with same length as input

Raises

- `UnsupportedFunctionError` – If function name is not registered.
- `FunctionArgumentError` – If argument count is outside valid range.
- `TypeError` – If the function returns a non-Series result, or raises `TypeError` internally during execution.
- `RuntimeError` – If the function raises an unexpected exception at runtime (i.e. not `ValueError` or `TypeError`).

Return type

`Series`

Scalar Function Evaluator

Dispatches scalar function calls from AST nodes to the registered implementations. Scalar Function Evaluator for Cypher Scalar and Graph Introspection Functions.

This module provides the `ScalarFunctionEvaluator` class, extracted from `BindingExpressionEvaluator` as part of Architecture Loop Phase 5. It handles evaluation of scalar functions including graph introspection functions that require pre-argument evaluation.

Architecture Loop Phase 5 Note: This extraction follows the proven delegation pattern established in previous loops, maintaining 100% backward compatibility while achieving clear separation of concerns.

`class pycypher.scalar_function_evaluator.ScalarFunctionEvaluator(frame)`

Bases: `object`

Evaluates Cypher scalar functions and graph introspection functions.

This class handles the evaluation of scalar function calls, with special handling for graph introspection functions that need to access variable metadata before argument evaluation. Functions handled include:

- Graph Introspection Functions (pre-evaluation):
 - `labels(n)` → returns entity type label list
 - `type(r)` → returns relationship type string
 - `keys(n)` → returns property column names (excluding internal columns)
 - `properties(n)` → returns property dict (with shadow layer support)
 - `startNode(r)` → returns relationship source node ID
 - `endNode(r)` → returns relationship target node ID
- Path Length Function:
 - `length(path_var)` → returns hop count from variable-length path

- Registry Delegation: All other scalar functions are delegated to `ScalarFunctionRegistry` for processing.
- Aggregation Prevention: Prevents aggregation functions from being used in scalar expression contexts.

`__init__(frame)`

Initialize scalar function evaluator with binding frame context.

Parameters

`frame` (`BindingFrame`) – The `BindingFrame` containing variable bindings, type registry, and context for function evaluation.

`evaluate_scalar_function(func_name, func_args, expression_evaluator)`

Main entry point for scalar function evaluation.

Handles the following function families before delegating to the `ScalarFunctionRegistry`:

- “length(path_var)” — returns the hop-count column stored by the variable-length-path expander (`__path_hops__<var>` column).
- “labels(n)” — returns [“Person”, ...] using the type registry; does not touch the entity table.
- “type(r)” — returns the relationship type string per row.
- “keys(n)” — returns the list of user-visible property column names for the entity or relationship type, excluding internal `__ID__`/`__SOURCE__`/`__TARGET__` columns.
- “properties(n)” — returns a dict of all user-visible properties per row, reading from the shadow layer if available.
- “startNode(r)” / “endNode(r)” — returns the source or target node ID of each relationship row.

If the function name matches a known aggregation (count, sum, etc.) it raises `ValueError` to prevent accidental use in scalar contexts (e.g. `WHERE count(*) > 3` is invalid Cypher).

All other functions are passed to the `ScalarFunctionRegistry` for dispatch (built-in scalar functions such as `toInteger`, `toLower`, `substring`, etc., plus any user-registered functions).

Parameters

- `func_name` (`str`) – The Cypher function name (case-insensitive matching is applied internally).
- `func_args` (Any) – Raw argument value from the AST — either a list of `Expression` nodes or a single expression; normalised to a list by `__normalize_func_args()`.
- `expression_evaluator` (`ExpressionEvaluatorProtocol`) – Reference to the parent `BindingExpressionEvaluator` for recursive expression evaluation.

Returns

A `pd.Series` of per-row results.

Raises

`ValueError` – If `func_name` is an aggregation function used in a scalar context, or if the function is unknown to the registry.

Return type

`FrameSeries`

4.1.6 Execution Engine

Configuration

Centralized runtime configuration: query timeouts, cross-join limits, result cache sizes, and BFS hop caps. All settings are driven by environment variables. Centralized configuration for pycypher runtime behaviour.

All environment-variable-driven configuration is read here at import time and exposed as module-level constants. This avoids scattered `os.environ` reads and provides a single place to document, validate, and override runtime settings.

Environment Variables

PYCYPHER_QUERY_TIMEOUT_S

Default wall-clock budget for a single query (seconds). None (default) means no timeout.

PYCYPHER_MAX_CROSS_JOIN_ROWS

Hard ceiling on cross-join result size (rows). Prevents accidental Cartesian explosions. Default: 10_000_000.

PYCYPHER_RESULT_CACHE_MAX_MB

Maximum in-memory size for the query result cache (megabytes). Default: 100.

PYCYPHER_RESULT_CACHE_TTL_S

Time-to-live for cached query results (seconds). 0 (default) means entries never expire (only evicted by size pressure).

PYCYPHER_MAX_UNBOUNDED_PATH_HOPS

Hard cap on BFS hops for unbounded variable-length paths (e.g. `[*]`). Default: 20.

PYCYPHER_AST_CACHE_MAX

Maximum number of parsed ASTs cached per GrammarParser instance. LRU eviction when full. Default: 1024. 0 disables caching.

PYCYPHER_MAX_QUERY_SIZE_BYTES

Hard ceiling on raw query string size (bytes). Rejects queries larger than this before parsing. Default: 1_048_576 (1 MiB).

PYCYPHER_MAX_QUERY_NESTING_DEPTH

Maximum nesting depth for AST traversal (e.g. deeply nested WHERE clauses). Prevents stack exhaustion on adversarial inputs. Default: 200.

PYCYPHER_MAX_COLLECTION_SIZE

Hard ceiling on generated collection sizes — `range()` lists, `repeat()/lpad()/rpad()` output lengths, and UNWIND expansion. Prevents memory exhaustion from adversarial inputs. Default: 1_000_000 (1 M elements/characters).

PYCYPHER_AUDIT_LOG

Enable structured query audit logging. Set to 1, true, or yes to emit one JSON record per query to the pycypher.audit logger. Off by default. Records include query ID, timestamp, elapsed time, row count, and parameter names (never values).

Examples

Set a 30-second query timeout and reduce the cross-join ceiling:

```
export PYCYPHER_QUERY_TIMEOUT_S=30
export PYCYPHER_MAX_CROSS_JOIN_ROWS=100000
```

Disable AST caching (useful for grammar development):

```
export PYCYPHER_AST_CACHE_MAX=0
```

Increase the result cache to 500 MB with a 5-minute TTL:

```
export PYCYPHER_RESULT_CACHE_MAX_MB=500
export PYCYPHER_RESULT_CACHE_TTL_S=300
```

Inspect active configuration at runtime:

```
nmetl config --show-effective
```

`pycypher.config.AST_CACHE_MAX_ENTRIES`: `int` = 1024

Maximum number of parsed ASTs to cache per GrammarParser instance.

When the cache reaches this size, the least-recently-used entry is evicted. Set to 0 to disable AST caching entirely.

`pycypher.config.COMPLEXITY_WARN_THRESHOLD`: `int` | `None` = `None`

Soft threshold for query complexity warnings. Queries scoring above this emit a warning but still execute. `None` (default, env 0) disables warnings. Set lower than `MAX_COMPLEXITY_SCORE` to get early alerts.

`pycypher.config.CROSS_JOIN_WARN_THRESHOLDS`: `tuple[int, ...]` = (100000, 1000000)

Progressive warning thresholds for cross-join cardinality.

`pycypher.config.MAX_COLLECTION_SIZE`: `int` = 1000000

1M.

Type

Hard ceiling on generated collection/string sizes. Default

`pycypher.config.MAX_COMPLEXITY_SCORE`: `int` | `None` = `None`

Hard ceiling on query complexity score. Queries exceeding this are rejected before execution. `None` (default, env 0) disables the gate. Typical production values: 50–200.

`pycypher.config.MAX_CROSS_JOIN_ROWS`: `int` = 1000000

Hard ceiling on cross-join result size (rows).

`pycypher.config.MAX_QUERY_NESTING_DEPTH`: `int` = 200

200.

Type

Maximum AST traversal nesting depth. Default

`pycypher.config.MAX_QUERY_SIZE_BYTES`: `int` = 1048576

1 MiB.

Type

Hard ceiling on query string size (bytes). Default

`pycypher.config.MAX_UNBOUNDED_PATH_HOPS`: `int` = 20

Hard cap on BFS hops for unbounded variable-length paths.

`pycypher.config.QUERY_TIMEOUT_S`: `float` | `None` = `None`

Default query timeout in seconds, or `None` for no limit.

`pycypher.config.RATE_LIMIT_BURST`: `int` = 10

Maximum burst size for rate limiting. Allows short bursts above the sustained QPS rate. Only meaningful when `RATE_LIMIT_QPS` > 0.

`pycypher.config.RATE_LIMIT_QPS`: `float` = 0.0

Maximum sustained queries per second. 0 (default) disables rate limiting entirely. When enabled, queries exceeding this rate receive a `RateLimitError`.

`pycypher.config.RESULT_CACHE_MAX_MB`: `int` = 100

Maximum result cache size in MB.

`pycypher.config.RESULT_CACHE_TTL_S`: `float` | `None` = 0.0

Result cache TTL in seconds (0 = no expiry).

`pycypher.config.apply_preset(name)`

Apply a named configuration preset by updating module globals.

Available presets:

”development” (default)

Safe for learning — no timeouts, generous limits, no rate limiting.

”production”

Defensive limits for user-facing queries — 30 s timeout, 100 K cross-join ceiling, complexity gate at 100, rate limiting.

”high_performance”

Trusted environment — 2 min timeout, 10 M cross-join ceiling, 500 MB cache, no complexity gate, no rate limiting.

Example:

```
from pycypher.config import apply_preset
apply_preset("production")
```

Raises

`ValueError` – If name is not a recognised preset.

`pycypher.config.show_config()`

Return a dict of all current configuration values.

Useful for debugging and logging the active configuration:

```
from pycypher.config import show_config
print(show_config())
```

Backend Engine

Protocol-based abstraction layer for pluggable DataFrame computation backends. Includes PandasBackend, DuckDBBackend, PolarsBackend, health checks, and a circuit breaker for graceful failover. Backend abstraction layer for pluggable DataFrame engines.

Provides a BackendEngine protocol that allows the query engine to work with different DataFrame backends (pandas, Dask, DuckDB, Polars) without coupling to any specific implementation.

The protocol surface is derived from an audit of the actual pandas API calls in the PyCypher execution path (688 import references, 918 API calls across 18 source files). Rather than abstracting all 918 calls, the protocol captures the ~15 primitive operations that BindingFrame, star.py, and MutationEngine depend on.

Architecture

BackendEngine (Protocol)

- PandasBackend — current default, zero-cost wrapper
- DuckDBBackend — analytical queries, SQL-based operations
- PolarsBackend — Arrow-native, lazy-evaluated scaling
- (future) DaskBackend

Concrete implementations live in `pycypher.backends`:

- `pycypher.backends.pandas_backend`
- `pycypher.backends.duckdb_backend`
- `pycypher.backends.polars_backend`

Usage:

```
engine = select_backend(hint="auto", estimated_rows=500_000)
ids = engine.scan_entity(source_obj, "Person")
filtered = engine.filter(ids, mask)
joined = engine.join(left, right, on="__ID__")
result = engine.to_pandas(joined)
```

Integration Status

Note

Partially integrated into the execution path.

The BackendEngine protocol is wired into the core execution path via BindingFrame delegation. When `context.backend` is available, the following operations route through the backend:

- `BindingFrame.join()` — inner joins with strategy hints
- `BindingFrame.left_join()` — left outer joins (OPTIONAL MATCH)
- `BindingFrame.cross_join()` — Cartesian products
- `BindingFrame.filter()` — boolean mask filtering
- `BindingFrame.rename()` — column renaming
- `pattern_matcher` — concat and distinct for multi-type scans and variable-length path expansion

Selecting `backend="duckdb"` at Context construction now routes these operations through the DuckDB engine.

- `mutation_engine` — CREATE (concat), DELETE/DETACH DELETE (filter) route through backend when available

Not yet delegated (future work):

- Property resolution in `BindingFrame.get_property()`
- Aggregation operations in `AggregationEvaluator`


```
class pycypher.backend_engine.BackendEngine(*args, **kwargs)
```

Bases: [Protocol](#)

Protocol for pluggable DataFrame computation backends.

All methods operate on “frames” — the backend-specific DataFrame type. The pandas backend uses `pd.DataFrame`; other backends use their native types and convert to pandas only at materialisation boundaries.

The protocol covers five categories of operation, mapped from the actual usage patterns in the PyCypher execution path:

1. Scan — `scan_entity` loads entity IDs from source data.
2. Transform — `filter`, `join`, `rename`, `concat`, `distinct`, `assign_column`, `drop_columns` reshape frames.
3. Aggregate — `aggregate` performs grouped or full-table aggregation.
4. Order — `sort`, `limit`, `skip` control output ordering.
5. Materialise — `to_pandas`, `row_count`, `is_empty`, `memory_estimate_bytes` convert or inspect frames.

property name: [str](#)

Human-readable backend name (e.g. 'pandas', 'duckdb').

```
scan_entity(source_obj, entity_type)
```

Load all entity IDs from `source_obj` as a single-column frame.

Parameters

- `source_obj` ([Any](#)) – The raw data (`pd.DataFrame`, `pa.Table`, etc.).
- `entity_type` ([str](#)) – The entity type label.

Returns

A frame with at least an `__ID__` column.

Return type

[Any](#)

```
filter(frame, mask)
```

Apply a boolean mask to frame, returning matching rows.

Parameters

- `frame` ([Any](#)) – Backend-specific DataFrame.
- `mask` ([Any](#)) – Boolean array/Series aligned with frame.

Returns

Filtered frame with reset index.

Return type

[Any](#)

```
join(left, right, on, how='inner', strategy='auto')
```

Join two frames on the given column(s).

Parameters

- `left` ([Any](#)) – Left frame.
- `right` ([Any](#)) – Right frame.
- `on` ([str](#) | [list](#)[[str](#)]) – Column name(s) to join on. Empty list for cross joins.

- `how` (`str`) – Join type ('inner', 'left', 'cross').
- `strategy` (`str`) – Join algorithm hint — 'auto', 'broadcast', 'hash', or 'merge'. Back-ends may ignore this if they perform their own optimisation (e.g. DuckDB).

Returns

Joined frame.

Return type

`Any`

`rename(frame, columns)`

Rename columns in frame.

Used by `BindingFrame.rename()` to promote structural join-key columns (e.g. `__tgt_r`) to named Cypher variables (e.g. `q`).

Parameters

- `frame` (`Any`) – Input frame.
- `columns` (`dict[str, str]`) – Mapping from old name to new name.

Returns

Frame with renamed columns.

Return type

`Any`

`concat(frames, *, ignore__index=True)`

Concatenate multiple frames vertically.

Used by `MutationEngine.shadow__create__entity()` and union queries.

Parameters

- `frames` (`list[Any]`) – List of frames to concatenate.
- `ignore__index` (`bool`) – If True, reset the index after concatenation.

Returns

Combined frame.

Return type

`Any`

`distinct(frame)`

Remove duplicate rows from frame.

Used by `WITH DISTINCT` and `RETURN DISTINCT`.

Parameters

`frame` (`Any`) – Input frame.

Returns

Deduplicated frame.

Return type

`Any`

`assign__column(frame, name, values)`

Add or replace a column in frame.

Used by `star.py` during pattern traversal to add new variable columns to the binding frame.

Parameters

- frame ([Any](#)) – Input frame.
- name ([str](#)) – Column name.
- values ([Any](#)) – Column values (Series, list, or scalar).

Returns

Frame with the new/replaced column.

Return type

[Any](#)

`drop_columns(frame, columns)`

Remove columns from frame.

Used for post-join cleanup (removing `_right` suffixed duplicates and structural join keys).

Parameters

- frame ([Any](#)) – Input frame.
- columns ([list\[str\]](#)) – Column names to drop. Missing names are silently ignored.

Returns

Frame without the specified columns.

Return type

[Any](#)

`aggregate(frame, group_cols, agg_specs)`

Grouped aggregation.

Parameters

- frame ([Any](#)) – Input frame.
- group_cols ([list\[str\]](#)) – Columns to group by (empty for full-table agg).
- agg_specs ([dict\[str, tuple\[str, str\]\]](#)) – Mapping from output column name to (source_column, agg_func) tuples.

Returns

Aggregated frame.

Return type

[Any](#)

`sort(frame, by, ascending=None)`

Sort frame by the given columns.

Parameters

- frame ([Any](#)) – Input frame.
- by ([list\[str\]](#)) – Column names to sort on.
- ascending ([list\[bool\]](#) | None) – Per-column sort direction. Defaults to all ascending.

Returns

Sorted frame.

Return type

[Any](#)

`limit(frame, n)`

Return the first `n` rows of frame.

Parameters

- `frame (Any)` – Input frame.
- `n (int)` – Number of rows.

Returns

Truncated frame.

Return type

`Any`

`skip(frame, n)`

Skip the first `n` rows of frame.

Used by the `SKIP` clause in `WITH/RETURN`.

Parameters

- `frame (Any)` – Input frame.
- `n (int)` – Number of rows to skip.

Returns

Frame without the first `n` rows.

Return type

`Any`

`to_pandas(frame)`

Materialise frame as a pandas DataFrame.

This is the universal escape hatch — every backend must support this for interop with existing code that expects pandas.

Parameters

`frame (Any)` – Backend-specific frame.

Returns

A pandas DataFrame.

Return type

`DataFrame`

`row_count(frame)`

Return the number of rows in frame without full materialisation.

Parameters

`frame (Any)` – Backend-specific frame.

Returns

Row count.

Return type

`int`

`is_empty(frame)`

Return True if frame has zero rows.

Equivalent to `row_count(frame) == 0` but may be faster for lazy backends that can short-circuit after checking the first partition.

Parameters
 frame ([Any](#)) – Backend-specific frame.

Returns
 True if the frame has no rows.

Return type
[bool](#)

memory_estimate_bytes(frame)

Estimate memory consumption of frame in bytes.

Used by the query planner for join strategy selection and memory budgeting. Does not need to be exact — order of magnitude is fine.

Parameters
 frame ([Any](#)) – Backend-specific frame.

Returns
 Estimated size in bytes.

Return type
[int](#)

__init__(*args, **kwargs)

class pycypher.backend_engine.CircuitBreaker(failure_threshold=3, recovery_timeout=60.0)

Bases: [object](#)

Per-backend circuit breaker with configurable thresholds.

Tracks consecutive failures for a backend. When failures exceed `failure_threshold`, the circuit opens and the backend is bypassed for `recovery_timeout` seconds. After the timeout, one probe request is allowed; if it succeeds the circuit closes, otherwise it reopens.

Parameters

- `failure_threshold` ([int](#)) – Consecutive failures before opening the circuit.
- `recovery_timeout` ([float](#)) – Seconds to wait before probing a failed backend.

__init__(failure_threshold=3, recovery_timeout=60.0)

Initialise the circuit breaker.

Parameters

- `failure_threshold` ([int](#)) – Number of consecutive failures to trigger circuit open.
- `recovery_timeout` ([float](#)) – Seconds before attempting a probe after open.

property failure_threshold: [int](#)

The configured failure threshold.

property recovery_timeout: [float](#)

The configured recovery timeout in seconds.

record_success(backend_name)

Record a successful operation, resetting the failure counter.

`record_failure(backend_name)`

Record a failure, potentially opening the circuit.

`is_available(backend_name)`

Check if a backend is available (circuit closed or half-open).

`state(backend_name)`

Return the current circuit state for a backend.

Handles the OPEN → HALF_OPEN transition when the recovery timeout has elapsed, consistent with `is_available()`.

Parameters

`backend_name` (`str`) – The backend to query.

Returns

The current `CircuitState`.

Return type

`CircuitState`

`reset(backend_name=None)`

Reset circuit breaker state.

If `backend_name` is given, resets only that backend. Otherwise resets all backends to CLOSED.

Parameters

`backend_name` (`str` | `None`) – Optional backend to reset. None resets all.

`get_state(backend_name)`

Return the current circuit state for a backend.

Deprecated since version Use: `state()` instead, which handles OPEN → HALF_OPEN transitions.

`class pycypher.backend_engine.CircuitState(*values)`

Bases: `Enum`

Three-state circuit breaker model.

CLOSED = 'closed'

OPEN = 'open'

HALF_OPEN = 'half_open'

`class pycypher.backend_engine.DuckDBBackend`

Bases: `object`

DuckDB-based backend for analytical workloads.

Uses DuckDB's in-process OLAP engine for efficient columnar operations. Particularly effective for:

- Large aggregations (GROUP BY with many groups)
- Join optimisation (DuckDB's query planner handles strategy selection)
- Sort/limit composition (DuckDB fuses ORDER BY + LIMIT efficiently)

The `sort()` method returns a `DuckDBLazyFrame` so that a subsequent `limit()` can compose `ORDER BY + LIMIT` into a single DuckDB query. All other operations accept `DuckDBLazyFrame` inputs transparently and return `pd.DataFrame` for full backward compatibility.

`__del__()`

Release the DuckDB connection on garbage collection.

`__enter__()`

Enter the context manager, returning this backend instance.

`__exit__(*_exc)`

Exit the context manager, closing the DuckDB connection.

`__init__()`

Create a new DuckDB backend with an in-memory connection.

The connection is created immediately and held for the lifetime of the backend. Use as a context manager or call `close()` to release the underlying DuckDB resources.

`aggregate(frame, group_cols, agg_specs)`

Aggregation via DuckDB SQL.

`assign_column(frame, name, values)`

Add or replace a column — delegates to pandas.

`close()`

Explicitly close the DuckDB connection.

Safe to call multiple times — subsequent calls are no-ops.

`concat(frames, *, ignore_index=True)`

Concatenate via pandas.

`distinct(frame)`

Remove duplicate rows via DuckDB.

`drop_columns(frame, columns)`

Drop columns, ignoring missing names.

`filter(frame, mask)`

Boolean mask filter — delegates to pandas.

Mask-based filtering cannot be expressed as lazy DuckDB SQL because the mask is computed externally as a numpy array.

`is_empty(frame)`

Check if frame has zero rows.

`join(left, right, on, how='inner', strategy='auto')`

Join via DuckDB SQL for optimal join strategy selection.

The strategy parameter is accepted for protocol compatibility but ignored — DuckDB's query planner selects the optimal join algorithm internally based on table statistics.

`limit(frame, n)`

Limit via DuckDB.

When frame is a DuckDBLazyFrame (e.g. from `sort()`), the LIMIT is composed into the existing DuckDB relation, enabling ORDER BY + LIMIT fusion.

`memory_estimate_bytes(frame)`

Estimate memory usage.

property name: `str`

Return 'duckdb'.

`rename(frame, columns)`

Rename columns — delegates to pandas (no SQL benefit).

`row_count(frame)`

Row count — uses DuckDB COUNT(*) for lazy frames.

`scan_entity(source_obj, entity_type)`

Register source in DuckDB and return ID column.

`skip(frame, n)`

Skip first n rows via DuckDB.

`sort(frame, by, ascending=None)`

Sort via DuckDB — returns lazy for sort+limit fusion.

Returns a DuckDBLazyFrame so that a subsequent `limit()` can compose ORDER BY + LIMIT into a single DuckDB query instead of materialising the full sort result first.

The DuckDBLazyFrame auto-materialises when accessed via pandas methods, so callers that don't chain `limit()` still get correct results transparently.

`to_pandas(frame)`

Materialise — executes the DuckDB query DAG if lazy.

`class pycypher.backend_engine.InstrumentedBackend(inner)`

Bases: `object`

Transparent wrapper that records per-operation timing and counts.

Delegates all BackendEngine operations to an inner backend while recording timing, operation counts, and emitting DEBUG-level log messages for each operation.

`operation_timings`

Dict mapping operation names to lists of elapsed times in milliseconds.

`operation_counts`

Dict mapping operation names to invocation counts.

`__init__(inner)`

Wrap inner with per-operation timing and count instrumentation.

Parameters

`inner` (`BackendEngine`) – The `BackendEngine` to delegate operations to. All method calls are forwarded transparently; timing and counts are recorded in `operation_timings` and `operation_counts`.

property name: `str`

Return the inner backend's name.

`timing_summary()`

Return a summary of per-operation timing statistics.

Returns

Dict mapping operation names to dicts with `count` and `total_ms` keys.

Return type

`dict[str, dict[str, float]]`

`scan_entity(source_obj, entity_type)`

Instrumented `scan_entity` with OTel tracing.

`filter(frame, mask)`

Instrumented filter with OTel tracing.

`join(left, right, on, how='inner', strategy='auto')`

Instrumented join with OTel tracing.

`rename(frame, columns)`

Instrumented rename with OTel tracing.

`concat(frames, *, ignore__index=True)`

Instrumented concat with OTel tracing.

`distinct(frame)`

Instrumented distinct with OTel tracing.

`assign__column(frame, name, values)`

Instrumented assign__column with OTel tracing.

`drop__columns(frame, columns)`

Instrumented drop__columns with OTel tracing.

`aggregate(frame, group__cols, agg__specs)`

Instrumented aggregate with OTel tracing.

`sort(frame, by, ascending=None)`

Instrumented sort with OTel tracing.

`limit(frame, n)`

Instrumented limit with OTel tracing.

`skip(frame, n)`

Instrumented skip with OTel tracing.

`to__pandas(frame)`

Instrumented to__pandas with OTel tracing.

`row__count(frame)`

Delegate row__count without instrumentation (trivial op).

`is__empty(frame)`

Delegate is__empty without instrumentation (trivial op).

`memory_estimate_bytes(frame)`
Delegate `memory_estimate_bytes`.

`class pycypher.backend_engine.PandasBackend`

Bases: `object`

Pandas-based backend — wraps existing pandas operations.

This is the default backend and provides identical behaviour to the current codebase. It serves as:

1. The reference implementation for the BackendEngine protocol.
2. The fallback when other backends are unavailable or inappropriate.
3. A zero-cost abstraction — no overhead compared to raw pandas calls.

`aggregate(frame, group_cols, agg_specs)`
Grouped aggregation via pandas groupby.

`assign_column(frame, name, values)`
Add or replace a column.

`concat(frames, *, ignore_index=True)`
Concatenate via pandas.

`distinct(frame)`
Remove duplicate rows via pandas.

`drop_columns(frame, columns)`
Drop columns, ignoring missing names.

`filter(frame, mask)`
Boolean mask filter.

`is_empty(frame)`
Check if DataFrame has zero rows.

`join(left, right, on, how='inner', strategy='auto')`
Merge via pandas with optional strategy routing.

Strategy effects on PandasBackend:

- 'auto' / 'hash': default `pd.merge` (uses hash internally).
- 'broadcast': swap left/right so the smaller side is the build table — reduces hash table memory for asymmetric joins.
- 'merge': sort both sides on the join key first, then merge. Optimal when inputs are already sorted.

`limit(frame, n)`

Head via pandas.

`memory__estimate__bytes(frame)`

Use pandas `memory__usage` for estimation.

property name: `str`

Return 'pandas'.

`rename(frame, columns)`

Rename columns via pandas.

`row__count(frame)`

`len()` on `DataFrame`.

`scan__entity(source__obj, entity__type)`

Convert source to pandas and return ID column.

`skip(frame, n)`

Skip first `n` rows via pandas.

`sort(frame, by, ascending=None)`

Sort via pandas.

`to__pandas(frame)`

Return an independent copy so the caller can mutate freely.

The copy is $O(1)$ under pandas 3.0+ CoW until the caller actually mutates the data. Required because `inplace=True` operations bypass CoW and mutate the original.

`class pycypher.backend__engine.PolarsBackend`

Bases: `object`

Polars-based backend for single-machine scaling (1GB–50GB).

Uses Polars' Apache Arrow-native columnar format with lazy evaluation and query optimisation. Key advantages over pandas:

- Arrow-native: Zero-copy interop with Arrow, Parquet, IPC.
- Lazy evaluation: Builds a query plan, optimises, then executes.
- Multi-threaded: Automatic parallelism across CPU cores.
- Memory-efficient: Columnar + lazy means only materialised data is held in memory.

The backend accepts pandas DataFrames at API boundaries (to match the protocol) and converts internally. For maximum performance, future work will accept Polars LazyFrames directly from data sources.

`__init__()`

Create a new Polars backend.

Imports the polars library on first use and stores a module reference for internal conversion between pandas and Polars frames.

`aggregate(frame, group_cols, agg_specs)`

Grouped aggregation via Polars.

`assign_column(frame, name, values)`

Add or replace a column via Polars.

`concat(frames, *, ignore_index=True)`

Concatenate via Polars.

`distinct(frame)`

Remove duplicate rows via Polars.

`drop_columns(frame, columns)`

Drop columns, ignoring missing names.

`filter(frame, mask)`

Boolean mask filter via Polars.

`is_empty(frame)`

Check if frame has zero rows.

`join(left, right, on, how='inner', strategy='auto')`

Join via Polars.

The strategy parameter is accepted for protocol compatibility but ignored — Polars' query engine handles join optimisation internally.

`limit(frame, n)`

Return first n rows via Polars.

`memory_estimate_bytes(frame)`
Estimate memory usage.

`property name: str`
Return 'polars'.

`rename(frame, columns)`
Rename columns via Polars.

`row_count(frame)`
Row count.

`scan_entity(source_obj, entity_type)`
Extract ID column from source.

`skip(frame, n)`
Skip first n rows via Polars.

`sort(frame, by, ascending=None)`
Sort via Polars.

`to_pandas(frame)`
Return an independent copy so the caller can mutate freely.

`pycypher.backend_engine.check_backend_health(backend)`
Run a lightweight health probe against backend.

Creates a tiny DataFrame, performs a scan + filter + join cycle, and verifies the result. Returns True if the backend is operational, False on any exception.

Parameters
 `backend` ([BackendEngine](#)) – The backend instance to probe.

Returns
 True if all probe operations succeed with correct results.

Return type
 [bool](#)

`pycypher.backend_engine.get_circuit_breaker()`
Return the module-level circuit breaker instance.

`pycypher.backend_engine.select_backend(*, hint='auto', estimated_rows=0, run_health_check=False, instrument=False)`

Select the optimal backend engine for the workload.

Parameters

- `hint (str)` – Backend preference — 'auto', 'pandas', 'duckdb', 'polars'.
- `estimated_rows (int)` – Estimated total rows to process. Used by 'auto' to decide between backends.
- `run_health_check (bool)` – If True, run a lightweight health probe on the selected backend before returning it. On failure the circuit breaker is tripped and the next backend in the fallback chain is tried.
- `instrument (bool)` – If True, wrap the selected backend with `InstrumentedBackend` for timing and count recording.

Returns

A `BackendEngine` instance.

Raises

- `RuntimeError` – If all backends fail health checks (only possible when `run_health_check` is True).
- `ValueError` – If hint is not a recognised backend name.

Return type

`BackendEngine`

The auto-selection heuristic:

- $< 100K$ rows $\rightarrow$ `PandasBackend` (low overhead, no startup cost)
- $\geq 100K$ rows $\rightarrow$ fallback chain with circuit breaker awareness

```
pycypher.backend_engine.select_backend_for_query(*, current_backend, optimization_hints,
                                                estimated_rows=0, instrument=False)
```

Re-evaluate backend selection using post-optimization query hints.

Called after the query optimizer produces cardinality estimates and join strategy hints. Returns a replacement backend if the hints suggest the current backend is suboptimal, or None if no change is needed.

Parameters

- `current_backend (BackendEngine)` – The backend currently in use.
- `optimization_hints (dict[str, object])` – Merged hints from `OptimizationPlan`.
- `estimated_rows (int)` – Row estimate from the query planner (overrides the Context-level entity count if available).
- `instrument (bool)` – Wrap the replacement with `InstrumentedBackend`.

Returns

A replacement `BackendEngine`, or None if the current backend is already optimal.

Return type

`BackendEngine` | None

```
pycypher.backend_engine.validate_identifier(name)
```

Validate that name is a safe SQL identifier.

Column and table names interpolated into DuckDB SQL must not contain characters that could alter query semantics. This is a defence-in-depth measure on top of DuckDB's "quoted identifier" syntax.

Parameters

`name (str)` – The identifier to validate.

Returns

name unchanged if valid.

Raises

`ValueError` — If name contains characters outside `[A-Za-z0-9_]` or does not start with a letter or underscore.

Return type

`str`

Query Planner

Handles memory estimation, operation reordering, and execution plan optimization. Integrates with the backend engine for adaptive join strategy selection. Intelligent query planner for optimised join and aggregation strategies.

The query planner analyses the structure of a query (join cardinalities, filter selectivity, available indices) and selects optimal execution strategies. Think of it as the DHF stack coordinator in Altered Carbon — routing consciousness data through the most efficient neural pathway based on bandwidth, latency, and sleeve compatibility.

Architecture

<code>QueryPlanner</code>	— low-level strategy selection (join/agg)
<code>QueryPlanAnalyzer</code>	— AST-aware analysis (cardinality, memory, pushdown)
<code>JoinStrategy</code>	(enum: <code>HASH</code> , <code>BROADCAST</code> , <code>MERGE</code> , <code>NESTED_LOOP</code>)
<code>JoinPlan</code>	(dataclass: left, right, strategy, estimated_cost)
<code>QueryPlan</code>	(dataclass: ordered list of <code>JoinPlans</code> + agg strategy)
<code>AnalysisResult</code>	(dataclass: per-clause cardinality, memory, pushdown)

The planner is invoked before execution to produce a `QueryPlan`. The executor (`star.py`) then follows the plan rather than using a fixed join order.

Usage:

```
# Low-level strategy selection
planner = QueryPlanner()
plan = planner.plan_join(left_size=1_000_000, right_size=100, ...)
# plan.strategy == JoinStrategy.BROADCAST

# AST-aware analysis
analyzer = QueryPlanAnalyzer(query_ast, context)
result = analyzer.analyze()
# result.clause_cardinalities, result.estimated_peak_bytes, etc.
```

```
class pycypher.query_planner.AggrStrategy(*values)
```

Bases: `Enum`

Available aggregation algorithms.

`HASH_AGG` = 'hash_agg'

Hash aggregation — build hash table on group keys. Best for moderate cardinality groups.

`SORT_AGG` = 'sort_agg'

Sort-based aggregation — sort then scan. Best when data is already sorted on group keys.

STREAMING_AGG = 'streaming_agg'

Streaming aggregation — process chunks, merge partial results. Best for very large datasets that don't fit in memory.

```
class pycypher.query_planner.JoinPlan(left_name, right_name, join_key, strategy, estimated_rows=0,
                                     estimated_memory_bytes=0, notes="")
```

Bases: `object`

Describes how a single join operation should be executed.

`left_name`

Identifier for the left table/frame.

Type
`str`

`right_name`

Identifier for the right table/frame.

Type
`str`

`join_key`

Column(s) to join on.

Type
`str | list[str]`

`strategy`

The selected join algorithm.

Type
`pycypher.query_planner.JoinStrategy`

`estimated_rows`

Estimated output row count.

Type
`int`

`estimated_memory_bytes`

Estimated peak memory during the join.

Type
`int`

`notes`

Human-readable explanation of the strategy choice.

Type
`str`

`left_name: str`

`right_name: str`

`join_key: str | list[str]`

`strategy: JoinStrategy`

```

estimated_rows: int = 0

estimated_memory_bytes: int = 0

notes: str = ""

__init__(left_name, right_name, join_key, strategy, estimated_rows=0,
         estimated_memory_bytes=0, notes="")

```

```
class pycypher.query_planner.JoinStrategy(*values)
```

Bases: `Enum`

Available join algorithms, ordered by memory overhead.

`HASH` = 'hash'

Hash join — build hash table on the smaller side, probe with the larger. Best for general-purpose equi-joins. $O(N + M)$ time, $O(\min(N, M))$ space.

`BROADCAST` = 'broadcast'

Broadcast join — replicate the small table to every partition. Best when one side is orders of magnitude smaller ($< 10K$ rows). $O(N)$ time, $O(\text{small})$ space.

`MERGE` = 'merge'

Merge join — both sides pre-sorted on the join key. Best when data is already sorted (e.g. from `ORDER BY` or index). $O(N + M)$ time, $O(1)$ space.

`NESTED_LOOP` = 'nested_loop'

Nested loop — brute-force. Only for very small datasets or non-equi joins. $O(N \times M)$ time.

`LEAPFROG` = 'leapfrog'

LeapfrogTriejoin — worst-case optimal multi-way join. Best for 3+ relations sharing a common join variable (e.g. triangle queries). $O(N^{\lceil w/2 \rceil})$ vs $O(N^{w-1})$ for iterated binary joins.

```

class pycypher.query_planner.QueryPlan(joins=<factory>, aggregation=None,
                                       total_estimated_memory_bytes=0, warnings=<factory>)

```

Bases: `object`

Full execution plan for a query.

`joins`

Ordered list of join plans.

Type

`list[pycypher.query_planner.JoinPlan]`

`aggregation`

Aggregation plan (if query has `GROUP BY` / aggregation).

Type

`pycypher.query_planner.AggPlan | None`

`total_estimated_memory_bytes`

Sum of estimated peak memory across all ops.

Type

`int`

warnings

Any optimiser warnings (e.g. potential OOM, cross-join).

Type

`list[str]`

joins: `list[JoinPlan]`

aggregation: `AggPlan | None = None`

total_estimated_memory_bytes: `int = 0`

warnings: `list[str]`

`__init__(joins=<factory>, aggregation=None, total_estimated_memory_bytes=0, warnings=<factory>)`

`class pycypher.query_planner.QueryPlanAnalyzer(query, context, feedback_store=None, learning_store=None)`

Bases: `object`

Walks a Cypher AST and Context to produce cardinality, memory, and join strategy analysis.

This is the Phase 2.1 integration point — called from `Star._execute_query_binding_frame_inner()` before execution begins to produce an `AnalysisResult` that guides strategy selection.

Parameters

- query (`Query`) – Parsed Cypher query AST.
- context (`Context`) – The execution context with entity/relationship tables.

`__init__(query, context, feedback_store=None, learning_store=None)`

Initialize query plan analyzer.

Parameters

- query (`Query`) – Parsed Cypher query AST to analyze.
- context (`Context`) – Execution context with entity/relationship tables for cardinality estimation.
- feedback_store (`CardinalityFeedbackStore | None`) – Optional feedback store for correcting estimates based on historical execution data.
- learning_store (`Any | None`) – Optional `QueryLearningStore` for ML-based selectivity corrections and plan caching.

`estimate_predicate_selectivity(predicate)`

Estimate the selectivity of a WHERE predicate using column statistics when available, falling back to heuristic defaults.

Supported predicate patterns: - `p.age > 30` → range selectivity from column stats - `p.name = 'Alice'` → equality selectivity ($1/\text{NDV}$) - `p.age > 30 AND p.dept = 'Eng'` → product of selectivities - `p.age > 30 OR p.dept = 'Eng'` → capped union - `NOT pred` → $1 - \text{selectivity}$

`entity_row_count(entity_type)`

Return the row count for an entity type, or 0 if unknown.

`relationship_row_count(rel_type)`

Return the row count for a relationship type, or 0 if unknown.

`analyze()`

Walk the AST and produce a full analysis.

When a learning store is configured, checks the adaptive plan cache first. If a cached plan exists for a structurally similar query, it is returned immediately. After computing a fresh plan, it is stored in the cache for future reuse.

`log_cardinality_feedback(analysis, actual_rows)`

Log estimated vs actual cardinality for post-execution analysis.

Parameters

- `analysis` (`AnalysisResult`) – The pre-execution analysis result.
- `actual_rows` (`list[int]`) – Actual row counts per clause (same order as `analysis.clause_cardinalities`).

```
class pycypher.query_planner.QueryPlanner(*, memory_budget_bytes=2147483648,
                                           learning_store=None)
```

Bases: `object`

Analyses query structure and produces optimised execution plans.

The planner uses simple heuristics today and can be extended with cost-based optimisation and statistics collection in future phases.

```
__init__(*, memory_budget_bytes=2147483648, learning_store=None)
```

Initialize query planner.

Parameters

- `memory_budget_bytes` (`int`) – Maximum memory budget for join operations. Defaults to 2 GB.
- `learning_store` (`Any` | `None`) – Optional `QueryLearningStore` for adaptive join strategy selection from historical performance.

```
plan_join(*, left_name, right_name, left_rows, right_rows, join_key, left_sorted=False,
           right_sorted=False, avg_row_bytes=100)
```

Select the optimal join strategy for two frames.

Parameters

- `left_name` (`str`) – Identifier for the left frame.
- `right_name` (`str`) – Identifier for the right frame.
- `left_rows` (`int`) – Number of rows in the left frame.
- `right_rows` (`int`) – Number of rows in the right frame.

- `join_key` (`str` | `list[str]`) – Column(s) to join on.
- `left_sorted` (`bool`) – Whether left is sorted on join key.
- `right_sorted` (`bool`) – Whether right is sorted on join key.
- `avg_row_bytes` (`int`) – Average row size for memory estimation.

Returns

A `JoinPlan` with the selected strategy.

Return type

`JoinPlan`

`plan_aggregation(*, input_rows, group_cardinality, is_sorted=False, avg_row_bytes=100)`

Select the optimal aggregation strategy.

Parameters

- `input_rows` (`int`) – Number of input rows.
- `group_cardinality` (`int`) – Estimated number of distinct groups.
- `is_sorted` (`bool`) – Whether input is sorted on group keys.
- `avg_row_bytes` (`int`) – Average row size for memory estimation.

Returns

An `AggPlan` with the selected strategy.

Return type

`AggPlan`

`plan_cross_join(*, left_name, right_name, left_rows, right_rows, avg_row_bytes=100)`

Plan a cross join with memory safety checks.

Parameters

- `left_name` (`str`) – Identifier for the left frame.
- `right_name` (`str`) – Identifier for the right frame.
- `left_rows` (`int`) – Number of rows in the left frame.
- `right_rows` (`int`) – Number of rows in the right frame.
- `avg_row_bytes` (`int`) – Average row size.

Returns

A `JoinPlan` for the cross join.

Return type

`JoinPlan`

`estimate_memory(plan)`

Summarise memory requirements for a query plan.

Parameters

`plan` (`QueryPlan`) – The query plan to analyse.

Returns

A dictionary with memory estimates and budget comparison.

Return type

`dict[str, Any]`

Query Analyzer

Pre-execution query analysis, planning, and MATCH reordering. Performs lazy computation graph planning, rule-based optimization, cardinality-based MATCH clause reordering, LIMIT pushdown, and memory budget enforcement. Pre-execution query analysis, planning, and MATCH reordering.

Extracted from [pycypher.star](#) to separate query analysis concerns from the main execution orchestration. The `QueryAnalyzer` performs:

- Lazy computation graph planning (memory estimation, structural analysis)
- Rule-based query optimization
- Cardinality-based MATCH clause reordering
- LIMIT pushdown hint extraction
- Memory budget enforcement
- Cardinality feedback recording

```
class pycypher.query_analyzer.QueryAnalyzer(context, cardinality_feedback, frame_joiner, agg_planner)
    Bases: object
```

Pre-execution query analysis and optimization.

Encapsulates all planning logic that runs before clause-by-clause execution: computation graph building, optimizer rules, cardinality estimation, memory budget checks, and MATCH reordering.

Parameters

- `context` (`Context`) – The PyCypher Context for entity/relationship stats.
- `cardinality_feedback` (`CardinalityFeedbackStore`) – Feedback store for correcting heuristic estimates.
- `frame_joiner` (`FrameJoiner`) – FrameJoiner for passing pre-computed join plans.
- `agg_planner` (`Any`) – AggregationPlanner for aggregation detection in LIMIT pushdown.

```
__init__(context, cardinality_feedback, frame_joiner, agg_planner)
```

```
plan_query(query)
```

Build a computation graph from the query AST and run optimisation passes.

Returns execution hints derived from the lazy evaluation optimizer's analysis of the query structure.

Parameters

- `query` (`Any`) – A parsed `Query` AST node.

Returns

`estimated_memory_bytes`, `node_count`, `has_filter`, `has_join`, `optimized_graph`.

Return type

A dict with keys

```
extract_limit_hint(query)
```

Extract a LIMIT value for pushdown if the query pattern is safe.

Returns the integer LIMIT value when the query has the simple pattern MATCH ... RETURN ... LIMIT N (no aggregation, no DISTINCT, no ORDER BY, no SKIP, no WITH). Returns None when pushdown is unsafe.

Parameters

query ([Any](#)) – Parsed [Query](#) AST node.

Returns

Integer limit for pushdown, or None if pushdown is unsafe.

Return type

[int](#) | None

`record_cardinality_feedback(actual_rows)`

Record actual row count into the cardinality feedback store.

Uses the last analysis result to compare estimated vs actual cardinality per entity type, building up correction factors for future queries.

`analyze_and_plan(query)`

Run query planning, memory budget enforcement, and LIMIT pushdown.

Performs all pre-execution analysis:

- Builds a lazy computation graph for memory estimates.
- Runs the rule-based [QueryOptimizer](#).
- Runs [QueryPlanAnalyzer](#) for cardinality and join strategies.
- Enforces the memory budget.
- Extracts a LIMIT pushdown hint when safe.

Parameters

query ([Any](#)) – A parsed [Query](#) AST node.

Returns

An optional row-limit hint for MATCH pushdown, or None.

Raises

[QueryMemoryBudgetError](#) – If an explicit memory budget is exceeded.

Return type

[int](#) | None

`apply_match_reordering(query)`

Reorder consecutive MATCH clauses by estimated cardinality.

Processes MATCH clauses smallest-first to minimize intermediate result sizes and reduce cross-join explosion risk. Only reorders consecutive MATCH runs — never moves a MATCH past a WITH, RETURN, SET, or other clause boundary.

Mutates query.clauses in place.

Cardinality Estimator

Column and table statistics for cardinality estimation. Provides per-column NDV, null fraction, histograms, and self-correcting feedback via actual-vs-estimated ratios. Column and table statistics for cardinality estimation.

Provides reusable statistical primitives that can be consumed by the query planner, query plan analyzer, or any other component that needs selectivity estimates. Extracted from `query_planner.py` to reduce module coupling and enable independent testing.

Key classes:

- `ColumnStatistics` — per-column NDV, null fraction, histograms.
- `TableStatistics` — lazily computed column statistics for a table.
- `CardinalityFeedbackStore` — accumulates actual-vs-estimated ratios for self-correcting estimates.

Usage:

```
stats = TableStatistics(df)
col = stats.column_stats("age")
if col is not None:
    sel = col.equality_selectivity() # 1/NDV adjusted for nulls
```

class `pycypher.cardinality_estimator.CardinalityFeedbackStore`

Bases: `object`

Accumulates actual vs estimated cardinality ratios per entity type.

After each query execution, call `record()` with the entity types involved and the (estimated, actual) row counts. Before a future estimate, call `correction_factor()` to get a multiplicative adjustment derived from historical accuracy.

Thread-safe via a simple lock. History is bounded to the most recent `_MAX_HISTORY` observations per entity type.

`__init__()`

`record(entity_type, estimated, actual)`

Record an (estimated, actual) observation for `entity_type`.

`correction_factor(entity_type)`

Return a multiplicative correction for `entity_type`.

If the estimator consistently overestimates by 2x, this returns ~0.5 so the caller can multiply the heuristic estimate by it. Returns 1.0 when no history is available.

property `entity_types_tracked`: `list[str]`

Return entity types with recorded history.

`clear()`

Drop all recorded history.

```
class pycypher.cardinality_estimator.ColumnStatistics(ndv, null_fraction, min_value=None,
                                                    max_value=None, row_count=0,
                                                    histogram_edges=None, histogram_counts=None)
```


Bases: `object`

Statistics for a single column, used for selectivity estimation.

`ndv`

Number of distinct values (excluding nulls).

Type
`int`

`null_fraction`

Fraction of rows that are null (0.0-1.0).

Type
`float`

`min_value`

Minimum non-null value (numeric columns only).

Type
`float | None`

`max_value`

Maximum non-null value (numeric columns only).

Type
`float | None`

`row_count`

Total rows in the table at time of collection.

Type
`int`

`histogram_edges`

Bin edges for equi-width histogram (numeric only). Length is `num_bins + 1`.

Type
`tuple[float, ...] | None`

`histogram_counts`

Row counts per histogram bin (numeric only). Length is `num_bins`.

Type
`tuple[int, ...] | None`

`ndv: int`

`null_fraction: float`

`min_value: float | None = None`

`max_value: float | None = None`

`row_count: int = 0`

`histogram_edges: tuple[float, ...] | None = None`

histogram_counts: `tuple[int, ...] | None = None`

equality_selectivity()

Selectivity for col = value: 1/NDV, adjusted for nulls.

range_selectivity(low=None, high=None)

Selectivity for range predicates (col > low, col < high).

When a histogram is available, estimates selectivity by summing the fraction of rows in bins that overlap the query range. Falls back to a uniform distribution assumption when no histogram is present.

`__init__(ndv, null_fraction, min_value=None, max_value=None, row_count=0, histogram_edges=None, histogram_counts=None)`

`class pycypher.cardinality_estimator.TableStatistics(source_obj)`

Bases: `object`

Collects and caches column-level statistics for an entity or relationship table.

Statistics are computed lazily on first access and cached. For large tables, a random sample of STATS_SAMPLE_SIZE rows is used.

`__init__(source_obj)`

property row_count: `int`

Return the number of rows in the source table.

column_stats(column)

Return cached statistics for column, computing on first call.

Scan Operators

Scan and filter operators for the BindingFrame execution path. Handles entity/relationship table scanning, predicate pushdown, and dtype coercion. Scan and filter operators for the BindingFrame execution path.

Extracted from `pycypher.binding_frame` to separate scan-level concerns (entity/relationship table scanning, predicate pushdown, dtype coercion) from the core BindingFrame data container.

Classes:

EntityScan — produces a BindingFrame of entity IDs for a given type. RelationshipScan — produces a BindingFrame of relationship IDs. BindingFilter — filters a BindingFrame by a boolean AST predicate.

Helpers:

`_coerce_pushdown_ids` — coerce pushdown IDs to match target column dtype. `_coerce_pushdown_series` — coerce a pushdown Series to match target dtype.

```
class pycypher.scan_operators.EntityScan(entity_type, var_name)
```

Bases: `object`

Produces a `BindingFrame` of all entity IDs for a given entity type.

The resulting frame has a single column named `var_name` containing every `__ID__` value from the entity table. Attributes are not included — they are fetched on demand via `BindingFrame.get_property()`.

`entity_type`

The entity label (e.g. "Person").

Type

`str`

`var_name`

The Cypher variable name to bind the IDs to (e.g. "p").

Type

`str`

`entity_type: str`

`var_name: str`

```
scan(context, property_filters=None)
```

Return a `BindingFrame` containing all IDs for this entity type.

Parameters

- `context` (Any) – The query `Context`.
- `property_filters` (`dict[str, Any] | None`) – Optional dict of {prop_name: value} equality predicates. When provided and a `PropertyValueIndex` is available, the scan returns only IDs matching all predicates instead of the full entity table — O(1) per predicate instead of O(N) post-scan filtering.

Returns

A `BindingFrame` with one column (`var_name`) of entity IDs.

Return type

`BindingFrame`

```
__init__(entity_type, var_name)
```

```
class pycypher.scan_operators.RelationshipScan(rel_type, rel_var, src_col="", tgt_col="")
```

Bases: `object`

Produces a `BindingFrame` of all relationship IDs for a given type.

The resulting frame has three columns:

- `rel_var` — the relationship's own `__ID__`.
- `_src_{rel_var}` — the source-node `__SOURCE__` ID.
- `_tgt_{rel_var}` — the target-node `__TARGET__` ID.

The source and target columns are structural join keys consumed by the pattern translator (Phase 5); they are not user-visible Cypher variables and are therefore absent from the `type_registry`.

`rel_type`

The relationship type label (e.g. "KNOWS").

Type
`str`

`rel_var`

The Cypher variable name for the relationship (e.g. "r"). Use a synthetic name such as "__anon_0" for anonymous relationships.

Type
`str`

`rel_type: str`

`rel_var: str`

`src_col: str = ""`

Cached column name for source-node IDs.

`tgt_col: str = ""`

Cached column name for target-node IDs.

`__post_init__()`

Cache derived column names to avoid repeated f-string creation.

`scan(context, *, source_ids=None, target_ids=None)`

Return a `BindingFrame` containing relationship IDs for this type.

Supports predicate pushdown: when `source_ids` or `target_ids` are provided, only relationships whose `__SOURCE__` / `__TARGET__` column values appear in the given ID set are materialised. This avoids loading the full relationship table when the query pattern already constrains one endpoint.

Parameters

- `context` (Any) – The query [Context](#).
- `source_ids` (`FrameSeries` | `None`) – If provided, only materialise relationships whose `__SOURCE__` is in this set.
- `target_ids` (`FrameSeries` | `None`) – If provided, only materialise relationships whose `__TARGET__` is in this set.

Returns

A `BindingFrame` with columns `rel_var`, `__src_{rel_var}`, and `__tgt_{rel_var}`.

Return type

[BindingFrame](#)

`__init__(rel_type, rel_var, src_col="", tgt_col="")`

`class pycypher.scan_operators.BindingFilter(predicate)`

Bases: `object`

Filters a `BindingFrame` by evaluating a boolean AST expression.

This is the Phase-3 analogue of the legacy `FilterRows` operator. Unlike `FilterRows`, it never touches prefixed column names — it delegates directly to `BindingExpressionEvaluator`.

`predicate`

The Cypher AST boolean expression to evaluate (e.g. a `Comparison`, `And`, `NullCheck`, etc.).

Type

`Expression`

`predicate: Expression`

`apply(frame)`

Return a new `BindingFrame` containing only rows where predicate is `True`.

Parameters

`frame (BindingFrame)` – The input `BindingFrame`.

Returns

A filtered `BindingFrame`.

Return type

`BindingFrame`

`__init__(predicate)`

Query Complexity

Scores a parsed Cypher AST for resource-consumption risk: clause count, join count, variable-length paths, aggregation depth, and cross-product potential. Query complexity scoring for resource protection.

Analyzes a parsed Cypher AST to compute a complexity score based on: - Number of clauses - Number of `MATCH` patterns and joins - Variable-length path expansion (unbounded is expensive) - Aggregation and `DISTINCT` operations - Nesting depth (`FOREACH`, subqueries) - Cross-product potential (multiple unrelated `MATCH` clauses)

The score can be compared against a configurable limit to reject or warn about queries that are likely to consume excessive resources.

Usage:

```
from pycypher.query_complexity import score_query, QueryComplexityError

score = score_query(parsed_query)
# score.total == 15, score.breakdown == {"clauses": 3, "joins": 4, ...}

# Or with a hard limit:
score = score_query(parsed_query, max_score=50)
# Raises QueryComplexityError if score exceeds limit
```

```
pycypher.query_complexity.DEFAULT_WEIGHTS: dict[str, int] = {'aggregation': 3, 'case_expression':
1, 'clause': 1, 'create': 2, 'cross_product_risk': 8, 'delete': 2, 'distinct': 2, 'exists_subquery': 4, 'foreach':
4, 'foreach_nesting': 8, 'list_comprehension': 2, 'match_pattern': 2, 'merge': 3, 'optional_match': 3,
'order_by': 1, 'unbounded_path': 10, 'union': 3, 'variable_length_path': 5}
```

Default complexity weights for each AST feature.

```
class pycypher.query_complexity.ComplexityScore(total=0, breakdown=<factory>, warnings=<factory>)
    Bases: object
    Result of query complexity analysis.
```

```

    total
        The total complexity score.
        Type
            int

    breakdown
        Per-feature contribution to the score.
        Type
            dict[str, int]

    warnings
        Human-readable warnings about risky patterns.
        Type
            list[str]

    total: int = 0
    breakdown: dict[str, int]
    warnings: list[str]

    __init__(total=0, breakdown=<factory>, warnings=<factory>)
```

```
pycypher.query_complexity.score_query(query, *, weights=None, max_score=None)
```

Compute a complexity score for a parsed Cypher query AST.

Parameters

- `query` (`Any`) – A parsed `Query` or `UnionQuery` AST node.
- `weights` (`dict[str, int] | None`) – Optional custom weights (merged with defaults).
- `max_score` (`int | None`) – If set, raises `QueryComplexityError` when exceeded.

Returns

A `ComplexityScore` with total, breakdown, and warnings.

Raises

`QueryComplexityError` – If `max_score` is set and the score exceeds it.

Return type

`ComplexityScore`

Query Optimizer

Rule-based query optimization pipeline: filter pushdown, limit pushdown, join reordering, and predicate simplification. Rule-based query optimizer with pluggable optimization passes.

Provides a framework for applying optimization rules to parsed Cypher ASTs before execution. Each rule inspects the AST, estimates cost savings, and optionally transforms the execution hints. Think of it as the Bene Gesserit breeding programme — each optimization rule is a carefully designed genetic modification applied to the query’s execution DNA.

Architecture

QueryOptimizer

OptimizationRule (abstract)	— single optimization pass
FilterPushdownRule	— move WHERE closer to scans
LimitPushdownRule	— early termination for simple queries
PredicateSimplificationRule	— simplify boolean expressions
JoinReorderingRule	— minimize intermediate result sizes
OptimizationResult	— outcome of a single rule
OptimizationPlan	— aggregated plan with all applied rules
OptimizeStage	— Pipeline stage adapter

Usage:

```
# Standalone
optimizer = QueryOptimizer.default()
plan = optimizer.optimize(query_ast, context)
print(plan.explain())

# As Pipeline stage
pipeline = Pipeline.default()
pipeline.insert_after("validate", OptimizeStage())
```

Extending:

```
class MyRule(OptimizationRule):
    name = "my_rule"
    def analyze(self, ast, context):
        return OptimizationResult(
            rule_name=self.name,
            applied=True,
            description="Applied my optimization",
            estimated_speedup=1.5,
        )

optimizer = QueryOptimizer([MyRule()])
```

```
class pycypher.query_optimizer.OptimizationResult(rule_name, applied, description="",
                                                    estimated_speedup=1.0, hints=<factory>)
```

Bases: `object`

Outcome of applying a single optimization rule.

`rule_name`

Name of the rule that produced this result.

Type

`str`

`applied`

Whether the rule actually applied an optimization.

Type

`bool`

`description`
 Human-readable description of what was optimized.
 Type
 `str`

`estimated_speedup`
 Multiplicative speedup factor (1.0 = no change).
 Type
 `float`

`hints`
 Key-value hints passed to the executor.
 Type
 `dict[str, Any]`

`rule_name: str`
`applied: bool`
`description: str = "`
`estimated_speedup: float = 1.0`
`hints: dict[str, Any]`
`__init__(rule_name, applied, description=", estimated_speedup=1.0, hints=<factory>)`

```
class pycypher.query_optimizer.OptimizationPlan(results=<factory>, total_estimated_speedup=1.0,
                                                elapsed_ms=0.0, hints=<factory>)
```

Bases: `object`
 Aggregated optimization plan from all applied rules.

`results`
 List of individual rule outcomes.
 Type
 `list[pycypher.query_optimizer.OptimizationResult]`

`total_estimated_speedup`
 Combined multiplicative speedup.
 Type
 `float`

`elapsed_ms`
 Time spent optimizing in milliseconds.
 Type
 `float`

`hints`
 Merged hints from all applied rules.


```

    Type
    dict[str, Any]
results: list[OptimizationResult]
total_estimated_speedup: float = 1.0
elapsed_ms: float = 0.0
hints: dict[str, Any]
property applied_rules: list[str]
    Return names of rules that actually applied.
property skipped_rules: list[str]
    Return names of rules that did not apply.
explain()
    Return human-readable optimization plan explanation.
    Returns
    Multi-line string describing applied optimizations.
    Return type
    str
__init__(results=<factory>, total_estimated_speedup=1.0, elapsed_ms=0.0, hints=<factory>)

```

```
class pycypher.query__optimizer.OptimizationRule
```

```
Bases: ABC
```

```
Abstract base for a single optimization rule.
```

```
Subclasses implement analyze() to inspect the AST and return an OptimizationResult describing whether and how the optimization applies.
```

```
name
```

```
Unique identifier for this rule.
```

```
Type
```

```
str
```

```
name: str = 'unnamed__rule'
```

```
abstractmethod analyze(ast, context=None)
```

```
Analyze the AST and return optimization result.
```

```
Parameters
```

- ast ([ASTNode](#)) – Parsed Cypher query AST.
- context ([Context](#) | None) – Optional execution context for cardinality estimation.

```
Returns
```

```
OptimizationResult describing the optimization.
```

```
Return type
```

```
OptimizationResult
```

```
class pycypher.query_optimizer.FilterPushdownRule
```

Bases: [OptimizationRule](#)

Move WHERE filters closer to data source scans.

Detects patterns where a WHERE clause can be applied during pattern matching rather than as a post-filter, reducing intermediate result sizes.

Applies when: - MATCH has a WHERE clause referencing only variables from that MATCH - WHERE predicates are simple comparisons on indexed properties

name: `str` = 'filter_pushdown'

analyze(ast, context=None)

Check for filter pushdown opportunities.

```
class pycypher.query_optimizer.LimitPushdownRule
```

Bases: [OptimizationRule](#)

Push LIMIT into pattern matching for early termination.

Safe only for simple MATCH → RETURN LIMIT N patterns without aggregation, DISTINCT, ORDER BY, SKIP, or WITH clauses.

name: `str` = 'limit_pushdown'

analyze(ast, context=None)

Check for LIMIT pushdown opportunity.

```
class pycypher.query_optimizer.JoinReorderingRule
```

Bases: [OptimizationRule](#)

Suggest optimal join ordering based on cardinality estimates.

For multi-MATCH queries, suggests processing smaller tables first to minimize intermediate result sizes.

name: `str` = 'join_reordering'

analyze(ast, context=None)

Check for join reordering opportunities.

```
class pycypher.query_optimizer.PredicateSimplificationRule
```

Bases: [OptimizationRule](#)

Detect and simplify redundant boolean predicates.

Identifies patterns like: - Double negation: NOT NOT x → x - Tautologies: x AND TRUE → x - Contradictions: x AND FALSE → FALSE

name: `str` = 'predicate_simplification'

analyze(ast, context=None)

Check for simplifiable predicates.

```
class pycypher.query_optimizer.IndexScanRule
```

```
    Bases: OptimizationRule
```

```
    Detect opportunities to use graph-native index scans.
```

```
    Identifies MATCH patterns with relationship traversals that can benefit from adjacency index lookups
    (O(degree) instead of O(E) table scans) and inline property filters that can use property value indexes
    (O(1) instead of O(N) scans).
```

```
    name: str = 'index_scan'
```

```
    analyze(ast, context=None)
```

```
        Check for index scan opportunities.
```

```
class pycypher.query_optimizer.QueryOptimizer(rules=None)
```

```
    Bases: object
```

```
    Applies optimization rules to a query AST and produces a plan.
```

```
        Parameters
```

```
        rules (list[OptimizationRule] | None) – Ordered list of optimization rules. If None, uses
        default() rules.
```

```
    __init__(rules=None)
```

```
    classmethod default()
```

```
        Create an optimizer with all built-in rules.
```

```
    property rule_names: list[str]
```

```
        Return names of registered rules.
```

```
    add_rule(rule)
```

```
        Add a rule to the optimizer.
```

```
        Parameters
```

```
        rule (OptimizationRule) – Rule to add.
```

```
        Returns
```

```
        Self for fluent chaining.
```

```
        Return type
```

```
        QueryOptimizer
```

```
    optimize(ast, context=None)
```

```
        Run all optimization rules and produce a plan.
```

```
        Parameters
```

- ast (ASTNode) – Parsed Cypher query AST.
- context (Context | None) – Optional execution context for cardinality estimation.

```
        Returns
```

```
        OptimizationPlan with results from all rules.
```

```
        Return type
```

```
        OptimizationPlan
```

```
class pycypher.query_optimizer.OptimizeStage(optimizer=None)
```

Bases: [Stage](#)

Pipeline stage that runs the query optimizer.

Insert into a [Pipeline](#) to automatically optimize queries before execution:

```
pipeline = Pipeline.default()
pipeline.insert_after("validate", OptimizeStage())
```

Populates `ctx.metadata["optimization_plan"]` and merges hints.

name: `str` = 'optimize'

`__init__`(optimizer=None)

`execute`(ctx)

Run optimization rules on the parsed AST.

Parameters

`ctx` ([PipelineContext](#)) – Pipeline context with ast populated.

Returns

Context with optimization hints in metadata.

Return type

[PipelineContext](#)

Query Validator

Combined validation: RETURN clause checks, clause ordering, and structural rule enforcement. Combined query validation for multi-query composition.

Validates combined Cypher query ASTs for structural integrity:

- Clause ordering — RETURN must be the final clause if present
- RETURN uniqueness — at most one RETURN clause allowed
- Structural consistency — clauses follow valid Cypher sequencing rules

All validation errors are collected (not fail-fast) so the user receives a complete diagnostic picture.

```
class pycypher.query_validator.ValidationResult(is_valid=True, errors=<factory>)
```

Bases: [object](#)

Result of validating a combined Cypher query AST.

`is_valid`

True if no validation errors were found.

Type

[bool](#)

`errors`

List of human-readable error descriptions.

```

    Type
        list[str]
    is_valid: bool = True
    errors: list[str]
    __str__()
        Human-readable summary of validation result.

    __init__(is_valid=True, errors=<factory>)

```

class pycypher.query_validator.CombinedQueryValidator

Bases: object

Validates combined Cypher query ASTs for structural integrity.

Collects all validation errors rather than failing on the first, providing a complete diagnostic picture.

validate(query)

Validate a combined Query AST.

Parameters

query ([Query](#)) – The [Query](#) to validate.

Returns

A [ValidationResult](#) with is_valid and errors.

Return type

[ValidationResult](#)

Query Formatter

Cypher query formatting and linting with configurable style rules. Cypher query formatter and linter.

Normalizes Cypher query formatting with configurable style rules: uppercase keywords, consistent indentation, clause-per-line layout.

Designed for use in:

- CLI: `nmetl format-query "MATCH ..."`
- Pre-commit hooks: validate query formatting in CI
- Editor integrations: format-on-save via LSP
- Programmatic use: `format_query(text)`

Usage:

```

from pycypher.query_formatter import format_query, lint_query

# Format a query
formatted = format_query("match (n:Person) where n.age > 30 return n.name")
# MATCH (n:Person)
# WHERE n.age > 30
# RETURN n.name

```

(continues on next page)

(continued from previous page)

```
# Lint a query (returns list of issues)
issues = lint_query("match (n) return n")
# [LintIssue(line=1, message="Keyword 'match' should be uppercase: MATCH")]
```

Environment variables:

- PYCYPHER_FORMAT_UPPERCASE — uppercase keywords (default: 1)
- PYCYPHER_FORMAT_INDENT — indentation width (default: 2)

`pycypher.query_formatter.format_query(query, *, uppercase=True, indent=2)`

Format a Cypher query string.

Applies consistent formatting:

- One clause per line (MATCH, WHERE, RETURN, etc.)
- Uppercase keywords
- Normalized whitespace
- Consistent indentation for sub-clauses

Parameters

- `query` (`str`) – The Cypher query string to format.
- `uppercase` (`bool`) – Whether to uppercase keywords.
- `indent` (`int`) – Indentation width for sub-clauses.

Returns

The formatted query string.

Return type

`str`

`class pycypher.query_formatter.LintIssue(line, column, message, severity='warning')`

Bases: `object`

A formatting issue found in a Cypher query.

`line`

1-based line number where the issue was found.

Type

`int`

`column`

1-based column number (0 if not applicable).

Type

`int`

`message`

Human-readable description of the issue.

Type

`str`

```

severity
    "warning" or "error".
    Type
        str

line: int

column: int

message: str

severity: str = 'warning'

__init__(line, column, message, severity='warning')

```

`pycypher.query_formatter.lint_query(query)`

Lint a Cypher query for formatting issues.

Checks:

- Keywords should be uppercase
- No trailing whitespace
- Consistent use of single or double quotes
- Parse errors (if parser is available)

Parameters

`query (str)` – The Cypher query to lint.

Returns

List of `LintIssue` instances.

Return type

`list[LintIssue]`

Query Profiler

Traces individual query execution to identify hot spots, track memory allocation, and generate optimization recommendations. Query profiling and bottleneck analysis tools.

Traces individual query execution to identify hot spots, track memory allocation, and generate optimization recommendations. Built on top of the per-clause timing infrastructure in [shared.metrics](#).

Usage:

```

from pycypher.query_profiler import QueryProfiler
from pycypher.star import Star

profiler = QueryProfiler(Star())
report = profiler.profile("MATCH (p:Person) RETURN p.name")
print(report)
print(report.hotspot)          # Slowest clause type
print(report.recommendations)  # Optimization suggestions

```

```
class pycypher.query_profiler.ProfileReport(query, total_time_ms, parse_time_ms, plan_time_ms,
                                           clause_timings, row_count, hotspot, recommendations,
                                           memory_delta_mb=0.0, backend_timings=<factory>)
```

Bases: `object`

Profiling result for a single query execution.

`query`

The original Cypher query string.

Type
`str`

`total_time_ms`

Total wall-clock time including parse + plan + execute.

Type
`float`

`parse_time_ms`

Time spent parsing the query string.

Type
`float`

`plan_time_ms`

Time spent in the query planner.

Type
`float`

`clause_timings`

Per-clause-type execution times in milliseconds.

Type
`dict[str, float]`

`row_count`

Number of rows in the result set.

Type
`int`

`hotspot`

The clause type that consumed the most time, or None.

Type
`str | None`

`recommendations`

List of optimization suggestions based on the profile.

Type
`list[str]`

`memory_delta_mb`

RSS change during execution (MB).

Type
`float`


```

query: str

total_time_ms: float

parse_time_ms: float

plan_time_ms: float

clause_timings: dict[str, float]

row_count: int

hotspot: str | None

recommendations: list[str]

memory_delta_mb: float = 0.0

backend_timings: dict[str, dict[str, float]]

__str__()
    Return a human-readable profile report.

__init__(query, total_time_ms, parse_time_ms, plan_time_ms, clause_timings, row_count,
         hotspot, recommendations, memory_delta_mb=0.0, backend_timings=<factory>)

```

```

class pycypher.query_profiler.QueryProfiler(star, backend=None, history=<factory>)
    Bases: object

    Profiles query execution and generates bottleneck analysis.

    Wraps a Star instance and instruments each query execution to collect detailed timing data.

    Parameters
        star (Any) – The Star instance to profile queries against.

    star: Any

    backend: Any = None

    history: list[ProfileReport]

    profile(query, *, parameters=None)
        Execute and profile a query.

        Parameters
            • query (str) – Cypher query string.
            • parameters (dict[str, Any] | None) – Optional named query parameters.

        Returns
            A ProfileReport with timing breakdown and recommendations.

        Return type
            ProfileReport

```

`metrics_summary()`

Return a combined diagnostic summary of clause and backend timings.

Aggregates all profile reports in history into a single summary with:

- `query_count`: Number of profiled queries.
- `clause_timings`: Aggregated per-clause total milliseconds.
- `backend_timings`: Latest backend operation timing summary.

Returns

Dict with combined clause-level and operation-level metrics.

Return type

`dict[str, Any]`

`clear_history()`

Clear all stored profile reports.

`__init__(star, backend=None, history=<factory>)`

`pycypher.query_profiler.analyze_workload(history)`

Analyze a collection of profile reports for workload-level patterns.

Examines aggregate query patterns to suggest system-level tuning rather than per-query fixes.

Parameters

`history` (`list[ProfileReport]`) – List of profile reports from a [QueryProfiler](#).

Returns

List of workload-level tuning recommendations.

Return type

`list[str]`

Input Validator

Pre-parse input sanitization: query length limits, ID checks, content safety, and parseability verification. Input validation for multi-query composition.

Validates the list of (`query_id`, `cypher_string`) pairs before they enter the composition pipeline. Checks performed:

- Query ID uniqueness — duplicate IDs are rejected
- Non-empty content — empty or whitespace-only Cypher strings are rejected
- Non-empty query IDs — blank query IDs are rejected
- Parseability — each Cypher string must be parseable by `ASTConverter`

All errors are collected (not fail-fast) for complete diagnostics.

`class pycypher.input_validator.InputValidationResult(is_valid=True, errors=<factory>)`

Bases: `object`

Result of validating multi-query inputs.

`is_valid`
 True if no validation errors were found.

Type
`bool`

`errors`
 List of human-readable error descriptions.

Type
`list[str]`

`is_valid: bool = True`

`errors: list[str]`

`__str__()`
 Human-readable summary of validation result.

`__init__(is_valid=True, errors=<factory>)`

class `pycypher.input_validator.InputValidator`

Bases: `object`

Validates multi-query inputs before composition.

Collects all errors rather than failing on the first, providing a complete diagnostic picture.

`validate(queries)`

Validate a list of (`query_id`, `cypher_string`) pairs.

Parameters

`queries` (`list[tuple[str, str]]`) – The query pairs to validate.

Returns

An `InputValidationResult` with `is_valid` and `errors`.

Return type

`InputValidationResult`

Frame Joiner

BindingFrame join and merge operations: `coerce-join`, `multi-MATCH` frame merging, and `OPTIONAL MATCH` left-join semantics. `FrameJoiner` — frame merging and optional-match join orchestration.

Extracted from `pycypher.star` to isolate the cohesive join orchestration family (optional match, frame merging, `coerce-join`) into a focused, independently testable module.

Handles:

- `OPTIONAL MATCH` left-join semantics with null-column injection on failure
- Multi-MATCH frame merging via keyed or cross joins
- Join strategy selection (shared variable → inner join, else cross join)
- Seed frame creation for clause-first queries

```
class pycypher.frame_joiner.FrameJoiner(context, match_fn, where_fn)
```

Bases: `object`

Orchestrates `BindingFrame` joins for `MATCH` clause processing.

Parameters

- `context` (`Context`) – The query execution context.
- `match_fn` (`Callable[...]`, `BindingFrame`) – Callback to execute a `MATCH` clause against a frame.
- `where_fn` (`Callable[...]`, `BindingFrame`) – Callback to apply a `WHERE` filter to a frame.

```
__init__(context, match_fn, where_fn)
```

Initialise the frame joiner.

Parameters

- `context` (`Context`) – The query execution context providing entity and relationship tables.
- `match_fn` (`Callable[...]`, `BindingFrame`) – Callback to execute a `MATCH` clause against a `BindingFrame`.
- `where_fn` (`Callable[...]`, `BindingFrame`) – Callback to apply a `WHERE` filter to a `BindingFrame`.

```
set_join_plans(plans)
```

Store pre-computed join plans from `QueryPlanAnalyzer`.

Plans are consumed in FIFO order by `coerce_join()`.

```
coerce_join(frame_a, frame_b, *, join_plan=None)
```

Join two `BindingFrame` objects.

Selects the join strategy based on shared variables:

- Shared variable: keyed inner-join on the first shared column name.
- No shared variable: Cartesian cross-join.

Parameters

- `frame_a` (`BindingFrame`) – The left `BindingFrame`.
- `frame_b` (`BindingFrame`) – The right `BindingFrame`.
- `join_plan` (`object` | `None`) – Optional pre-computed `JoinPlan` from `QueryPlanAnalyzer` to avoid redundant planning.

Returns

A merged `BindingFrame`.

Return type

`BindingFrame`

`multi_way_join(frames)`

Join multiple frames using LeapfrogTriejoin when applicable.

When 3+ frames share a common variable, uses the worst-case optimal LeapfrogTriejoin algorithm. Otherwise falls back to iterated pairwise joins.

Parameters

`frames` (`list[BindingFrame]`) – Two or more BindingFrames to join.

Returns

A single merged BindingFrame.

Return type

`BindingFrame`

`merge_frames_for_match(current_frame, match_frame, where_clause=None)`

Merge a new MATCH frame into the existing execution frame.

When a query contains multiple MATCH clauses, each new result frame must be combined with the accumulated frame via either a keyed join (shared variable) or a cross-join (no shared variables).

Filter pushdown: WHERE conjuncts that reference only variables from `match_frame` are applied before the join to reduce intermediate result sizes. Remaining predicates (those referencing variables from both frames) are applied after the join.

Parameters

- `current_frame` (`BindingFrame`) – The accumulated BindingFrame from preceding clauses.
- `match_frame` (`BindingFrame`) – The new frame produced by the current MATCH clause.
- `where_clause` (`ASTNode` | `None`) – Optional WHERE predicate node from the MATCH clause.

Returns

The merged `BindingFrame`.

Return type

`BindingFrame`

`process_optional_match(clause, current_frame)`

Execute an OPTIONAL MATCH clause against an existing frame.

Attempts a regular match and left-joins the result on any shared variable. If the match fails (entity/relationship type absent), delegates to `process_optional_match_failure()` to add null columns.

Parameters

- `clause` (`Match`) – An optional=True Match clause.
- `current_frame` (`BindingFrame`) – The preceding BindingFrame.

Returns

Updated BindingFrame.

Return type

`BindingFrame`

`process_optional_match_failure(clause, current_frame)`

Return a frame with NULL columns for variables the failed OPTIONAL MATCH would have bound.

Parameters

- `clause` ([Match](#)) – The failing Match clause.
- `current_frame` ([BindingFrame](#)) – The BindingFrame immediately before the OPTIONAL MATCH.

Returns

An updated BindingFrame with null columns for every new variable.

Return type

[BindingFrame](#)

`make_seed_frame()`

Return a synthetic single-row BindingFrame for clause-first queries.

Used when a clause appears as the first in a query with no preceding MATCH, UNWIND, or other data source.

Returns

BindingFrame with a single row and no entity bindings.

Return type

[BindingFrame](#)

LeapfrogTriejoin

Worst-case optimal multi-way join algorithm (Veldhuizen 2014). Activated automatically for 3+ frame joins on a shared variable, achieving $O(N^{\{w/2\}})$ complexity vs $O(N^{\{w-1\}})$ for iterated binary joins. LeapfrogTriejoin — worst-case optimal join algorithm for multi-way joins.

Implements the LeapfrogTriejoin algorithm from Veldhuizen 2014 for efficient multi-way intersection joins. This is particularly effective for cyclic query patterns (e.g., triangle queries) where pairwise binary joins produce large intermediate results.

Architecture

The algorithm operates on sorted iterators over relation columns. For a multi-way join on a shared variable, instead of building pairwise hash tables, all relations are intersected simultaneously via a “leapfrog” scan:

1. Sort each relation’s join column.
2. Maintain a cursor per relation, always pointing at the current value.
3. Advance cursors in round-robin order, “leaping” past values that cannot appear in the intersection.

This yields worst-case optimal time complexity $O(N^{\{w/2\}})$ for w-way joins on relations of size N, versus $O(N^{\{w-1\}})$ for iterated binary joins.

Integration

- Called from [FrameJoiner](#) when 3+ frames share a common join variable.
- Falls back to pairwise binary joins when the leapfrog strategy is not applicable (no shared variable across all frames, or only 2 frames).

References

- Veldhuizen, T. L. (2014). “Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm.” ICDT 2014.

class pycypher.leapfrog_triejoin.LeapfrogIterator(values, row_indices)

Bases: `object`

Sorted-array iterator supporting seek (leapfrog) operations.

Wraps a sorted numpy array and provides $O(\log N)$ seek via binary search.

Parameters

- values (`np.ndarray`) – A sorted 1-D numpy array of join keys.
- row_indices (`np.ndarray`) – Corresponding row indices into the source DataFrame.

`__init__`(values, row_indices)

property at_end: `bool`

True when the iterator is exhausted.

property key: `Any`

Current key value. Only valid when not at_end.

`seek`(target)

Advance to the first position $\geq$ target via binary search.

Parameters

target (`Any`) – The value to seek to.

`next`()

Advance to the next position.

`get_rows_for_key`(key)

Return all row indices matching key at the current position.

Scans forward from `_pos` while values equal key, returning the corresponding row indices.

Parameters

key (`Any`) – The key to collect rows for.

Returns

A numpy array of row indices.

Return type

`ndarray`

`pycypher.leapfrog_triejoin.leapfrog_triejoin`(frames, join_var)

Perform a multi-way LeapfrogTriejoin on BindingFrames sharing join_var.

This is the main entry point. It:

1. Sorts each frame’s join column.
2. Runs the leapfrog intersection to find common keys.
3. Assembles the result by cross-producing matching rows per key.

Falls back gracefully for edge cases (empty frames, single frame).

Parameters

- `frames` (`list[BindingFrame]`) – Two or more `BindingFrame`s that all contain column `join_var`.
- `join_var` (`str`) – The shared variable name to join on.

Returns

A new `BindingFrame` with the intersection result.

Raises

`ValueError` – If fewer than 2 frames are provided or `join_var` is missing from any frame.

Return type

`BindingFrame`

`pycypher.leapfrog_triejoin.can_use_leapfrog(frames)`

Check whether `LeapfrogTriejoin` is applicable to the given frames.

Returns (`True`, `join_var`) if all frames share at least one common variable, making them eligible for multi-way leapfrog join.

Parameters

`frames` (`list[BindingFrame]`) – The candidate `BindingFrame`s.

Returns

A tuple of (`applicable`, `join_variable_name`).

Return type

`tuple[bool, str | None]`

Graph Index

Graph-native index structures for accelerating pattern matching: adjacency indexes, property value indexes, and label-partitioned indexes. Graph-native index structures for accelerating pattern matching.

Provides adjacency indexes, property value indexes, and label-partitioned indexes that transform pattern matching from $O(E)$ full table scans to $O(\text{degree})$ neighbor lookups.

Architecture

GraphIndexManager

<code>AdjacencyIndex</code>	— per-relationship-type adjacency lists
<code>outgoing[src_id]</code>	→ list of (<code>rel_id</code> , <code>tgt_id</code>)
<code>incoming[tgt_id]</code>	→ list of (<code>rel_id</code> , <code>src_id</code>)
<code>PropertyValueIndex</code>	— per-(<code>entity_type</code> , <code>property</code>) hash index
<code>value_to_ids[val]</code>	→ set of entity IDs
<code>EntityLabelIndex</code>	— per-label sorted ID arrays for fast membership
<code>ids: np.ndarray</code>	— sorted entity IDs for $O(\log n)$ lookup

Indexes are built lazily on first access and invalidated when the `Context` commits mutations (`commit_query()` increments `_data_epoch`).

Usage:

```
# Indexes are managed through the Context:
ctx = Context(entity_mapping=..., relationship_mapping=...)
manager = ctx.index_manager # lazily created
```

(continues on next page)

(continued from previous page)

```
# Neighbor lookup — O(degree) instead of O(E)
neighbors = manager.get_neighbors("KNOWS", source_id=42, direction="outgoing")

# Property index — O(1) instead of O(N)
ids = manager.lookup_property("Person", "name", "Alice")
```

```
class pycypher.graph_index.AdjacencyIndex(rel_type, outgoing=<factory>, incoming=<factory>,
                                          size=0)
```

Bases: `object`

Adjacency list index for a single relationship type.

Stores both outgoing (source → targets) and incoming (target → sources) adjacency lists for O(degree) neighbor lookups instead of O(E) table scans.

`rel_type`

The relationship type this index covers.

Type
`str`

`outgoing`

Maps source node ID → list of (relationship_id, target_id).

Type
`dict[Any, tuple[tuple[Any, Any], ...]]`

`incoming`

Maps target node ID → list of (relationship_id, source_id).

Type
`dict[Any, tuple[tuple[Any, Any], ...]]`

`size`

Total number of relationships indexed.

Type
`int`

`rel_type: str`

`outgoing: dict[Any, tuple[tuple[Any, Any], ...]]`

`incoming: dict[Any, tuple[tuple[Any, Any], ...]]`

`size: int`

classmethod `build(rel_type, source_df)`

Build adjacency index from a relationship DataFrame.

Parameters

- `rel_type` (`str`) – Relationship type label.
- `source_df` (`DataFrame`) – DataFrame with `__ID__`, `__SOURCE__`, `__TARGET__` columns.

Returns

Populated AdjacencyIndex. All adjacency lists are frozen tuples for thread-safe read access after build.

Return type

`AdjacencyIndex`

`neighbors_outgoing(source_id)`

Return outgoing neighbors: tuple of (rel_id, target_id).

`neighbors_incoming(target_id)`

Return incoming neighbors: tuple of (rel_id, source_id).

`neighbors_outgoing_batch(source_ids)`

Batch lookup outgoing neighbors for multiple source IDs.

Returns

Tuple of (rel_ids, src_ids, tgt_ids) arrays — the subset of relationships where `__SOURCE__` is in source_ids.

Return type

`tuple[ndarray, ndarray, ndarray]`

`neighbors_incoming_batch(target_ids)`

Batch lookup incoming neighbors for multiple target IDs.

Returns

Tuple of (rel_ids, src_ids, tgt_ids) arrays — the subset of relationships where `__TARGET__` is in target_ids.

Return type

`tuple[ndarray, ndarray, ndarray]`

`__init__(rel_type, outgoing=<factory>, incoming=<factory>, size=0)`

```
class pycypher.graph_index.PropertyValueIndex(entity_type, property_name, value_to_ids=<factory>,
                                              size=0)
```

Bases: `object`

Hash index on a single property of an entity type.

Maps property values to sets of entity IDs for O(1) equality lookups.

`entity_type`

Entity type label.

Type

`str`

`property_name`

Property being indexed.

Type

`str`

`value_to_ids`
 Maps property value $\rightarrow$ frozenset of entity IDs.

Type
`dict[Any, frozenset]`

`size`
 Total number of indexed entries.

Type
`int`

`entity_type: str`

`property_name: str`

`value_to_ids: dict[Any, frozenset]`

`size: int`

classmethod `build(entity_type, property_name, source_df)`
 Build property value index from an entity DataFrame.

Parameters

- `entity_type` (`str`) – Entity type label.
- `property_name` (`str`) – Column name to index.
- `source_df` (`DataFrame`) – DataFrame with `__ID__` and property columns.

Returns
 Populated `PropertyValueIndex`.

Return type
`PropertyValueIndex`

`lookup(value)`
 Return entity IDs where property equals value. $O(1)$.

`property_distinct_values: int`
 Number of distinct indexed values.

`__init__(entity_type, property_name, value_to_ids=<factory>, size=0)`

class `pycypher.graph_index.EntityLabelIndex(entity_type, ids=<factory>)`
 Bases: `object`

Sorted array index for entity IDs of a single label.

Provides $O(\log n)$ membership testing via binary search and $O(1)$ count retrieval.

`entity_type`
 Entity type label.

Type
`str`

ids

Sorted numpy array of entity IDs.

Type

`numpy.ndarray`

entity_type: `str`

ids: `ndarray`

classmethod `build(entity_type, source_df)`

Build label index from an entity DataFrame.

Parameters

- `entity_type` (`str`) – Entity type label.
- `source_df` (`DataFrame`) – DataFrame with `__ID__` column.

Returns

Populated `EntityLabelIndex`.

Return type

`EntityLabelIndex`

`contains(entity_id)`

Check if `entity_id` exists in this label. $O(\log n)$.

`count()`

Return total number of entities with this label.

`__init__(entity_type, ids=<factory>)`

class `pycypher.graph_index.VectorizedPropertyStore(entity_type, sorted_ids, property_arrays)`

Bases: `object`

Pre-sorted entity ID array with aligned property columns for $O(n \log n)$ bulk lookups.

Instead of hash-based `pd.Series.map()` ($O(n)$ amortised but with high constant factor due to Python-level hashing) or `pd.DataFrame.reindex()` (creates intermediate DataFrames), this store uses `np.searchsorted` on a pre-sorted ID array to resolve entity IDs to row positions, then fancy-indexes directly into numpy property arrays.

Build cost: $O(N \log N)$ for the sort (one-time, cached). Lookup cost: $O(k \log N)$ for k query IDs against N stored entities.

entity_type: `str`

sorted_ids: `ndarray`

property_arrays: `dict[str, ndarray]`

classmethod build(entity_type, source_df)

Build a vectorized store from an entity DataFrame.

Parameters

- entity_type (`str`) – The entity type label.
- source_df (`DataFrame`) – DataFrame with ID_COLUMN and property columns.

Returns

A new VectorizedPropertyStore.

Return type

`VectorizedPropertyStore`

property size: `int`

Number of entities in this store.

property properties: `list[str]`

List of available property names.

fetch(query_ids, prop_name)

Fetch property values for a batch of entity IDs.

Uses np.searchsorted for $O(k \log N)$ bulk resolution instead of hash-based lookups.

Parameters

- query_ids (`ndarray`) – Array of entity IDs to look up.
- prop_name (`str`) – Property name to fetch.

Returns

Array of property values aligned with query_ids. Missing IDs get None.

Return type

`ndarray`

fetch_multi(query_ids, prop_names)

Fetch multiple properties in a single searchsorted pass.

The binary search is done once and reused for all properties, making this significantly faster than N separate fetch() calls.

Parameters

- query_ids (`ndarray`) – Array of entity IDs to look up.
- prop_names (`list[str]`) – List of property names to fetch.

Returns

Dict mapping property names to value arrays aligned with query_ids.

Return type

`dict[str, ndarray]`

__init__(entity_type, sorted_ids, property_arrays)

class pycypher.graph_index.GraphIndexManager(context)

Bases: `object`

Manages all graph-native indexes for a Context.

Indexes are built lazily on first access and invalidated when the data epoch changes (i.e., after mutation commits).

`context`

The query Context whose data is being indexed.

`__init__(context)`

`get_adjacency_index(rel_type)`

Get or build adjacency index for a relationship type.

Returns None if the relationship type doesn't exist in the context. Thread-safe: uses a lock to prevent concurrent builds.

`get_property_index(entity_type, property_name)`

Get or build property value index.

Returns None if the entity type or property doesn't exist. Thread-safe: uses a lock to prevent concurrent builds.

`get_label_index(entity_type)`

Get or build label index for an entity type.

Returns None if the entity type doesn't exist. Thread-safe: uses a lock to prevent concurrent builds.

`indexed_relationship_scan(rel_type, *, source_ids=None, target_ids=None)`

Use adjacency index for relationship scan with endpoint pushdown.

When `source_ids` or `target_ids` are provided, uses $O(\text{degree})$ adjacency lookup instead of $O(E)$ table scan + `isin()` filter.

Returns

DataFrame with columns (`rel_id`, `source_id`, `target_id`), or None if no index is available (caller should fall back to table scan).

Return type

`DataFrame` | None

`indexed_property_lookup(entity_type, property_name, value)`

Use property index for $O(1)$ equality lookup.

Returns

Frozenset of matching entity IDs, or None if no index available.

Return type

`frozenset` | None

`get_vectorized_store(entity_type)`

Get or build a VectorizedPropertyStore for an entity type.

The store pre-sorts entity IDs and aligns property columns for $O(k \log N)$ bulk property resolution via `np.searchsorted`.

Returns None if the entity type doesn't exist in the context. Thread-safe: uses a lock to prevent concurrent builds.

`stats()`

Return index statistics for diagnostics.

`eager_build_vectorized_stores()`

Pre-build VectorizedPropertyStores for all entity and relationship types.

Call this at query start to ensure the fast $O(k \log N)$ path in `BindingFrame.get_property()` is always available, avoiding fallback to the slower hash-based `pd.Series.map()` path.

Build cost is $O(N \log N)$ per entity type (one-time sort), amortised across all subsequent property lookups in the query.

`invalidate()`

Force invalidation of all indexes.

Projection Planner

Plans RETURN and WITH clause projections: alias inference, qualification, and modifier application (ORDER BY, LIMIT, SKIP, DISTINCT). ProjectionPlanner — RETURN/WITH clause evaluation and projection modifiers.

Extracted from [pycypher.star](#) to isolate the cohesive projection family (alias inference, disambiguation, RETURN evaluation, WITH evaluation, DISTINCT/ORDER BY/SKIP/LIMIT modifiers) into a focused, independently testable module.

Handles:

- Alias inference from expression AST nodes (Variable, PropertyLookup, etc.)
- Alias disambiguation for colliding inferred names
- RETURN clause evaluation (projection + aggregation + modifiers)
- WITH clause evaluation (projection + aggregation + WHERE + modifiers)
- DISTINCT, ORDER BY (ASC/DESC with NULLS FIRST/LAST), SKIP, LIMIT

`class pycypher.projection_planner.ProjectionPlanner(agg_planner, renderer, where_fn)`

Bases: `object`

Evaluates RETURN and WITH clauses, applying projection modifiers.

Stateless aside from injected dependencies — all mutable state is passed via method parameters.

Parameters

- `agg_planner` ([AggregationPlanner](#)) – [AggregationPlanner](#) for aggregation detection and grouped evaluation.
- `renderer` ([ExpressionRenderer](#)) – [ExpressionRenderer](#) for generating human-readable expression text.

- `where_fn` (Callable[..., [BindingFrame](#)]) – Callback to apply a WHERE filter to a frame. Signature: (`where_expr`, `result_frame`, `fallback_frame=None`) -> `BindingFrame`.

`__init__(agg_planner, renderer, where_fn)`

Initialise the projection planner.

Parameters

- `agg_planner` ([AggregationPlanner](#)) – Detects aggregation expressions and drives grouped evaluation in WITH/RETURN clauses.
- `renderer` ([ExpressionRenderer](#)) – Generates human-readable column names when no explicit AS alias is provided.
- `where_fn` (Callable[..., [BindingFrame](#)]) – Callback to apply a WHERE filter to a `BindingFrame`.

`infer_alias(expression)`

Infer a display alias from an expression when none is given.

For `Variable` returns the variable name, for `PropertyLookup` returns the property name (unqualified), and for all other expression types delegates to the expression renderer which generates a human-readable text representation. Returns `None` only for expression types that have no sensible short representation.

Parameters

`expression` ([Any](#)) – AST expression node to inspect.

Returns

A string alias, or `None` if no representation is available.

Return type

`str` | `None`

`qualify_alias(expression)`

Return a fully-qualified 'var.prop' alias for a `PropertyLookup`.

Used by `return_from_frame()` to disambiguate colliding inferred aliases. Returns `None` for any expression type other than `PropertyLookup` (`Variable`, `prop`).

Parameters

`expression` ([Any](#)) – AST expression node.

Returns

A "var.prop" string, or `None`.

Return type

`str` | `None`

`apply_projection_modifiers(df, clause, frame)`

Apply DISTINCT, ORDER BY, SKIP, and LIMIT to a projected `DataFrame`.

Called at the end of both `return_from_frame()` and `with_to_binding_frame()` so that both RETURN and WITH honour the full set of projection modifiers defined in the AST.

Parameters

- `df` (`pd.DataFrame`) – The fully-projected result `DataFrame` (aliases as columns).
- `clause` ([Any](#)) – AST Return or With node carrying the modifiers.
- `frame` ([BindingFrame](#)) – The pre-projection [BindingFrame](#) used to evaluate ORDER BY expressions that reference original entity variables.

Returns

A new `pd.DataFrame` with modifiers applied.

Return type

`pd.DataFrame`

`return_from_frame(return_clause, frame)`

Evaluate a RETURN clause against a `BindingFrame` and return results.

Handles simple expression projection and aggregation (full-table and grouped). Delegates to `AggregationPlanner`. Applies DISTINCT, ORDER BY, SKIP, and LIMIT after projection via `apply_projection_modifiers()`.

Parameters

- `return_clause` (Any) – AST `Return` node.
- `frame` (`BindingFrame`) – Current `BindingFrame`.

Returns

A plain `pd.DataFrame` with columns matching RETURN aliases.

Return type

`pd.DataFrame`

`with_to_binding_frame(with_clause, frame)`

Translate a WITH clause into a new `BindingFrame`.

Supports three modes:

- Simple projection (no aggregations): evaluate each expression and build a new frame where the columns are the aliases.
- Full aggregation (all items are aggregations): reduce all rows to a single aggregated row.
- Grouped aggregation (mixed items): group by non-aggregation keys, then compute aggregations per group.

The resulting frame has `type_registry = {}` since the columns are computed values rather than entity IDs. Variable lookups in subsequent WHERE / RETURN clauses will fall through to plain column access in bindings.

Parameters

- `with_clause` (Any) – AST `With` node.
- `frame` (`BindingFrame`) – Current `BindingFrame`.

Returns

A new `BindingFrame` with alias columns.

Return type

`BindingFrame`

Variable Manager

Variable scoping, conflict detection, and safe renaming for multi-query composition. Variable namespace management for Cypher query composition.

Provides conflict detection, systematic renaming, and binding preservation when composing multiple Cypher queries that may share variable namespaces.

Architecture

VariableManager

`collect_variables()` — gather all defined variable names from AST
`detect_conflicts()` — find overlapping names between two ASTs
`generate_unique_name()` — produce collision-free variable names
`rename_variables()` — deep-copy AST with variable names replaced

The rename operation is immutable — it returns a new AST tree and never mutates the original. This is critical for composability: the caller can rename one branch of a UNION without affecting the other.

`class pycypher.variable_manager.VariableManager`

Bases: `object`

Manage variable namespaces across Cypher query ASTs.

Provides utilities for detecting variable conflicts between queries, generating unique variable names, and rewriting AST trees with renamed variables while preserving structural binding relationships.

`collect_variables(node)`

Collect all variable names defined in the AST subtree.

Traverses the AST and extracts variable names from all Variable nodes found anywhere in the tree.

Parameters

`node` (`ASTNode`) – Root of the AST subtree to scan.

Returns

Set of variable name strings found in the subtree.

Return type

`set[str]`

`detect_conflicts(query_a, query_b)`

Return variable names that appear in both AST subtrees.

Parameters

- `query_a` (`ASTNode`) – First AST subtree.
- `query_b` (`ASTNode`) – Second AST subtree.

Returns

Set of variable names present in both subtrees.

Return type

`set[str]`

`generate_unique_name(base_name, existing, prefix='__v')`

Generate a unique variable name that avoids collisions.

Strategy: `prefix + base_name`, with a numeric suffix appended if the candidate already exists in `existing`.

Parameters

- `base_name` (`str`) – Original variable name to derive from.
- `existing` (`set[str]`) – Set of names that must not be reused.
- `prefix` (`str`) – Prefix prepended to the base name.

Returns

A collision-free variable name string.

Raises

`SecurityError` – If no unique name is found within `_MAX_NAME_GENERATION_ATTEMPTS` iterations.

Return type

`str`

`rename_variables(node, rename_map)`

Return a new AST with variables renamed per the mapping.

Deep-copies the AST tree, replacing every `Variable.name` that appears as a key in `rename_map` with the corresponding value. Variables not in the map are left unchanged.

The original AST is never mutated.

Parameters

- `node` (`ASTNode`) – Root AST node to rewrite.
- `rename_map` (`dict[str, str]`) – Mapping from old variable name → new variable name.

Returns

A new AST tree with renamed variables.

Raises

`SecurityError` – If the AST exceeds the maximum nesting depth.

Return type

`ASTNode`

Pipeline

Stage-based query execution pipeline with parse, validate, plan, and execute stages. Query execution pipeline abstraction.

Formalises the implicit execution stages in [Star](#) into a composable, extensible pipeline architecture. Think of it as the Holtzman engine of the query processor — each stage is a precisely calibrated fold in space-time that transforms the query from raw text to final results.

Architecture

Pipeline

Stage (abstract)	— one step in the execution flow
<code>ParseStage</code>	— string → AST
<code>ValidateStage</code>	— semantic checks + complexity scoring
<code>PlanStage</code>	— query analysis + memory estimation
<code>ExecuteStage</code>	— clause-by-clause evaluation
<code>FormatStage</code>	— projection modifiers + result shaping
<code>PipelineContext</code>	— shared state flowing through stages
<code>PipelineResult</code>	— final output with metadata

Each stage receives a `PipelineContext`, transforms it, and passes it to the next stage. Stages may short-circuit (e.g. cache hit) by setting `context.result` early.

Usage:

```
# Default pipeline with all built-in stages
pipeline = Pipeline.default()

# Custom pipeline with extra validation
pipeline = Pipeline([
    ParseStage(),
    my_custom_lint_stage,
    ValidateStage(),
    PlanStage(),
    ExecuteStage(),
    FormatStage(),
])

result = pipeline.run(query="MATCH (n) RETURN n", star=star)
```

Extending the pipeline:

```
class MyCustomStage(Stage):
    name = "custom_lint"

    def execute(self, ctx: PipelineContext) -> PipelineContext:
        # Inspect ctx.ast, modify ctx.metadata, etc.
        return ctx

pipeline = Pipeline.default()
pipeline.insert_after("parse", MyCustomStage())
```

class pycypher.pipeline.Pipeline(stages=None)

Bases: object

Ordered sequence of [Stage](#) instances.

Runs each stage in order, passing the shared [PipelineContext](#). Stages may short-circuit by setting `ctx.short_circuited = True`.

Parameters

stages (list[[Stage](#)] | None) – Ordered list of stages to execute.

__init__(stages=None)

append(stage)

Append a stage to the end of the pipeline.

Parameters

stage ([Stage](#)) – Stage to add.

Returns

Self for fluent chaining.

Return type

[Pipeline](#)

insert_before(target_name, stage)

Insert a stage before the named target stage.

Parameters

- `target_name` (`str`) – Name of the existing stage.
- `stage` (`Stage`) – Stage to insert.

Returns

Self for fluent chaining.

Raises

`ValueError` – If `target_name` is not found.

Return type

`Pipeline`

`insert_after(target_name, stage)`

Insert a stage after the named target stage.

Parameters

- `target_name` (`str`) – Name of the existing stage.
- `stage` (`Stage`) – Stage to insert.

Returns

Self for fluent chaining.

Raises

`ValueError` – If `target_name` is not found.

Return type

`Pipeline`

`remove(stage_name)`

Remove a stage by name.

Parameters

`stage_name` (`str`) – Name of the stage to remove.

Returns

Self for fluent chaining.

Raises

`ValueError` – If `stage_name` is not found.

Return type

`Pipeline`

property `stage_names`: `list[str]`

Return ordered list of stage names.

`run(*, query, star, parameters=None, timeout_seconds=None, memory_budget_bytes=None, max_complexity_score=None)`

Execute the pipeline and return results.

Parameters

- `query` (`str` | `Any`) – Cypher query string or pre-parsed AST node.
- `star` (`Any`) – `Star` instance for execution.
- `parameters` (`dict[str, Any]` | `None`) – Optional named query parameters.
- `timeout_seconds` (`float` | `None`) – Optional wall-clock timeout.
- `memory_budget_bytes` (`int` | `None`) – Optional memory budget.

- `max_complexity_score` (`int` | `None`) – Optional complexity ceiling.

Returns

`PipelineResult` with `DataFrame` and timing metadata.

Return type

`PipelineResult`

classmethod `default()`

Create a pipeline with the standard built-in stages.

Returns a pipeline with: `parse` → `validate` → `plan` → `execute` → `format`.

```
class pycypher.pipeline.PipelineContext(query_string=None, query_input=None, parameters=<factory>,
                                         timeout_seconds=None, memory_budget_bytes=None,
                                         max_complexity_score=None, ast=None, plan_analysis=None,
                                         result=None, metadata=<factory>, stage_timings=<factory>,
                                         short_circuited=False, short_circuit_reason="", star=None)
```

Bases: `object`

Mutable state that flows through pipeline stages.

Each stage reads from and writes to this context. The context accumulates parsed AST, plan analysis, execution results, timing metrics, and any custom metadata injected by user stages.

`query_string`

Original Cypher query string (if provided).

Type

`str` | `None`

`ast`

Parsed AST node (populated by `ParseStage`).

Type

`ASTNode` | `None`

`parameters`

Named query parameters (`$name` placeholders).

Type

`dict[str, Any]`

`result`

Final `DataFrame` result (populated by `ExecuteStage`).

Type

`pd.DataFrame` | `None`

`metadata`

Arbitrary key-value metadata for custom stages.

Type

`dict[str, Any]`

`stage_timings`

Timing in seconds for each completed stage.

```

        Type
        dict[str, float]
short_circuited
    If True, remaining stages are skipped.
    Type
    bool
short_circuit_reason
    Human-readable reason for short-circuit.
    Type
    str
query_string: str | None = None
query_input: str | Any = None
parameters: dict[str, Any]
timeout_seconds: float | None = None
memory_budget_bytes: int | None = None
max_complexity_score: int | None = None
ast: ASTNode | None = None
plan_analysis: Any | None = None
result: pd.DataFrame | None = None
metadata: dict[str, Any]
stage_timings: dict[str, float]
short_circuited: bool = False
short_circuit_reason: str = ''
star: Any = None

__init__(query_string=None, query_input=None, parameters=<factory>,
          timeout_seconds=None, memory_budget_bytes=None, max_complexity_score=None,
          ast=None, plan_analysis=None, result=None, metadata=<factory>,
          stage_timings=<factory>, short_circuited=False, short_circuit_reason='', star=None)

class pycypher.pipeline.PipelineResult(result, stage_timings, metadata)
    Bases: object
    Output of a pipeline run.

    result
        The DataFrame produced by the pipeline.
        Type
        pd.DataFrame | None

```

```

stage_timings
    Wall-clock time per stage in seconds.
    Type
        dict[str, float]
metadata
    Accumulated metadata from all stages.
    Type
        dict[str, Any]
result: pd.DataFrame | None
stage_timings: dict[str, float]
metadata: dict[str, Any]
__init__(result, stage_timings, metadata)

```

```
class pycypher.pipeline.Stage
```

Bases: `ABC`

Abstract base class for a pipeline stage.

Subclasses must define `name` and implement `execute()`. The name is used for logging, timing, and insertion ordering.

`name`

Unique identifier for this stage (e.g. "parse").

Type
`str`

name: `str = 'unnamed'`

abstractmethod `execute(ctx)`

Transform the pipeline context and return it.

Parameters

`ctx` (`PipelineContext`) – Current pipeline context with accumulated state.

Returns

The (possibly modified) context. Return `ctx` directly unless you need to replace it entirely.

Raises

- Any exception propagates to the caller via –
- `Pipeline.run` –

Return type

`PipelineContext`

```
class pycypher.pipeline.ExecuteStage
```

Bases: `Stage`

Execute the parsed query against the Star engine.

Dispatches to the appropriate Star internal execution method based on the AST type (Query vs UnionQuery). This stage calls the inner execution methods directly rather than delegating back to Star. `execute_query()`, which allows the pipeline to be used from within `execute_query()` without circular delegation.

Populates `ctx.result` with the output DataFrame and stores the parsed AST and mutation flag in `ctx.metadata` for downstream use.

name: `str = 'execute'`

`execute(ctx)`

Execute query and populate result.

class `pycypher.pipeline.ParseStage`

Bases: `Stage`

Parse a Cypher query string into a typed AST.

If the input is already an AST node, this stage is a no-op.

Populates `ctx.ast` with the parsed ASTNode.

name: `str = 'parse'`

`execute(ctx)`

Parse query string to AST.

class `pycypher.pipeline.PlanStage`

Bases: `Stage`

Analyse query structure and estimate resource requirements.

Runs `QueryPlanAnalyzer` to produce cardinality estimates, join strategies, and memory projections. Populates `ctx.plan_analysis` and `ctx.metadata["estimated_peak_bytes"]`.

name: `str = 'plan'`

`execute(ctx)`

Plan query execution.

class `pycypher.pipeline.ValidateStage`

Bases: `Stage`

Run semantic validation and complexity scoring.

Checks query complexity against `ctx.max_complexity_score` if set. Also runs scoring when `COMPLEXITY_WARN_THRESHOLD` is configured, even without a hard limit, so that warnings can be emitted.

Populates `ctx.metadata["complexity_score"]`, `ctx.metadata["complexity_details"]`, and `ctx.metadata["complexity_warnings"]` when scoring runs.

When `max_complexity_score` is provided, `score_query()` will raise `QueryComplexityError` if the score exceeds the limit.

name: `str = 'validate'`

```
execute(ctx)
    Validate parsed AST.
```

Lazy Evaluation

Defers computation until results are needed, enabling efficient processing of large datasets and variable-length paths. Lazy evaluation engine for deferred query execution.

Builds a computation graph of operations that are only executed when results are materialised. This enables:

1. Operation fusion — combine compatible sequential operations (e.g. filter→filter becomes a single filter with combined predicate).
2. Predicate pushdown — move filters as early as possible in the graph.
3. Dead column elimination — drop columns that are never referenced downstream.
4. Memory estimation — estimate peak memory before execution to select optimal backend/strategy.

The computation graph is a DAG where nodes are operations and edges represent data flow. Think of it as the DHF stack's neural mapping — consciousness (data) flows through optimised pathways before being materialised in the target sleeve (output DataFrame).

Architecture

```
LazyFrame (user-facing handle)
  ComputationGraph (internal DAG)
    ScanNode (leaf: entity/relationship scan)
    FilterNode (predicate application)
    JoinNode (two-input merge)
    ProjectNode (column selection)
    AggNode (aggregation)
    SortNode (ordering)
    LimitNode (row truncation)
```

Usage:

```
lf = LazyFrame.from_scan("Person", context)
lf = lf.filter(lambda ids: ids > 100)
lf = lf.join(other_lf, on="id")
result = lf.collect() # triggers execution
```

```
class pycypher.lazy_eval.OpType(*values)
    Bases: Enum

    Types of operations in the computation graph.

    SCAN = 'scan'

    FILTER = 'filter'

    JOIN = 'join'

    PROJECT = 'project'

    AGGREGATE = 'aggregate'
```

`SORT = 'sort'`

`LIMIT = 'limit'`

`UNION = 'union'`

```
class pycypher.lazy_eval.OpNode(op_type, params=<factory>, inputs=<factory>, node_id=0,
                                estimated_rows=0, estimated_memory_bytes=0)
```

Bases: `object`

A single operation in the computation graph.

`op_type`

The type of operation.

Type

`pycypher.lazy_eval.OpType`

`params`

Operation-specific parameters.

Type

`dict[str, Any]`

`inputs`

List of input node IDs (empty for scan nodes).

Type

`list[int]`

`node_id`

Unique identifier for this node.

Type

`int`

`estimated_rows`

Estimated output row count (for planning).

Type

`int`

`estimated_memory_bytes`

Estimated peak memory during execution.

Type

`int`

`op_type: OpType`

`params: dict[str, Any]`

`inputs: list[int]`

`node_id: int = 0`

`estimated_rows: int = 0`

`estimated_memory_bytes: int = 0`

```
__init__(op_type, params=<factory>, inputs=<factory>, node_id=0, estimated_rows=0,
         estimated_memory_bytes=0)
```

```
class pycypher.lazy_eval.ComputationGraph(nodes=<factory>, output_node=0, _next_id=0)
```

Bases: `object`

Directed acyclic graph of operations.

The graph tracks operation dependencies and enables optimisation passes before execution.

nodes

Mapping from node ID to `OpNode`.

Type

`dict[int, pycypher.lazy_eval.OpNode]`

output_node

The ID of the terminal (output) node.

Type

`int`

nodes: `dict[int, OpNode]`

output_node: `int = 0`

add_node(op)

Add an operation node and return its ID.

Parameters

op (`OpNode`) – The operation node to add.

Returns

The assigned node ID.

Return type

`int`

get_inputs(node_id)

Return the input nodes for the given node.

Parameters

node_id (`int`) – The node whose inputs to retrieve.

Returns

List of input `OpNode` instances.

Return type

`list[OpNode]`

topological_order()

Return node IDs in topological (execution) order.

Returns

List of node IDs where each node appears after all its inputs. Empty list if the graph has no nodes.

Return type

`list[int]`

```
__init__(nodes=<factory>, output_node=0, _next_id=0)
```

```
pycypher.lazy_eval.fuse_filters(graph)
```

Fuse consecutive filter nodes into a single filter.

When two filters are chained ($F2 \leftarrow F1 \leftarrow \text{source}$), they can be combined into a single filter with an AND-combined predicate. This reduces DataFrame materialisation from $2\times$ to $1\times$.

Parameters

graph ([ComputationGraph](#)) – The computation graph to optimise.

Returns

A new graph with consecutive filters fused.

Return type

[ComputationGraph](#)

```
pycypher.lazy_eval.push_filters_down(graph)
```

Push filter operations below joins where possible.

If a filter only references columns from one side of a join, it can be applied before the join — reducing the join’s input size.

Parameters

graph ([ComputationGraph](#)) – The computation graph to optimise.

Returns

A new graph with filters pushed down past joins.

Return type

[ComputationGraph](#)

```
pycypher.lazy_eval.estimate_memory(graph, avg_row_bytes=100)
```

Estimate peak memory usage for executing the computation graph.

Walks the graph in topological order and estimates the peak concurrent memory — like calculating the maximum neural bandwidth required for a multi-sleeve consciousness transfer.

Parameters

- graph ([ComputationGraph](#)) – The computation graph to analyse.
- avg_row_bytes ([int](#)) – Average bytes per row for estimation.

Returns

Estimated peak memory in bytes.

Return type

[int](#)

```
pycypher.lazy_eval.build_computation_graph(query)
```

Translate a parsed Cypher Query AST into a [ComputationGraph](#).

This bridges the gap between the AST (produced by the grammar parser) and the lazy evaluation engine. The resulting graph can be optimised with [fuse_filters\(\)](#), [push_filters_down\(\)](#), and analysed with [estimate_memory\(\)](#) before execution.

The translation is structural — it mirrors the clause sequence in the AST rather than simulating full execution semantics. This is sufficient for the optimisation passes to detect filter fusion and pushdown opportunities.

Parameters

query ([Any](#)) – A parsed [Query](#) AST node.

Returns

A [ComputationGraph](#) representing the query’s operations.

Return type

[ComputationGraph](#)

```
pycypher.lazy_eval.compute_live_columns(clauses)
```

Compute the set of live (needed) columns after each clause.

For each clause index *i*, returns the set of variable names that are referenced by any clause at index *i*+1 or later. A `None` entry means “keep all columns” (conservative fallback for mutation clauses where dropping columns could lose data).

The result list has the same length as clauses. Entry *i* is the set of columns that must be retained in the frame produced by clause *i* — any column not in this set can be safely dropped.

Parameters

clauses ([list\[Any\]](#)) – The ordered list of AST clause nodes from a parsed query.

Returns

A list parallel to clauses where each element is either a `set[str]` of live variable names or `None` (keep-all).

Return type

`list[set[str] | None]`

Result Cache

LRU query result cache with size-bounded eviction and TTL support. Uses `OrderedDict` for $O(1)$ LRU operations, readers-writer lock for concurrent access, and generation-based invalidation on mutation commits.

LRU query result cache with size-bounded eviction and TTL support.

Extracted from `star.py` to provide a focused, single-responsibility module for query result caching.

The cache uses an `OrderedDict` for $O(1)$ LRU operations, a readers-writer lock for concurrent stats reads, adaptive timeouts based on operation complexity, deadlock detection with owner-thread tracking, and generation-based invalidation triggered by mutation commits.

Usage:

```
cache = ResultCache(max_size_bytes=100 * 1024 * 1024, ttl_seconds=300)
cache.put("MATCH (n) RETURN n", None, result_df)
cached = cache.get("MATCH (n) RETURN n", None) # returns copy or None
```

```
class pycypher.result_cache.ReadWriteLock
```

Bases: `object`

A readers-writer lock with timeout support and owner tracking.

Multiple threads can hold the read lock concurrently. The write lock is exclusive — no readers or other writers may hold a lock while the write lock is held.

All lock acquisitions use timeout-bounded waits to prevent deadlocks. The current write-lock owner thread is tracked for diagnostics.

`__init__()`

property `write_owner`: `int` | `None`

Thread ident of the current write-lock holder, or `None`.

`acquire_read(timeout)`

Acquire a shared read lock.

Returns `True` if acquired within `timeout` seconds.

`release_read()`

Release a shared read lock.

`acquire_write(timeout)`

Acquire an exclusive write lock.

Returns `True` if acquired within `timeout` seconds. Returns `False` immediately if the current thread already holds the write lock (re-entrant deadlock prevention).

`release_write()`

Release the exclusive write lock.

```
class pycypher.result_cache.ResultCache(max_size_bytes=104857600, ttl_seconds=0.0,
                                         lock_timeout_seconds=None)
```

Bases: `object`

LRU cache for query results with size-bounded eviction and TTL support.

Keys are derived from the normalised query string and parameters. Values are copied DataFrames so mutations to the returned frame do not corrupt the cached entry.

The cache is automatically invalidated when the underlying Context commits a mutation (`SET` / `CREATE` / `DELETE` / `MERGE` / `REMOVE`).

Thread-safety:

- Mutating operations (`get`, `put`, `invalidate`, `clear`) acquire an exclusive write lock.
- Read-only operations (`stats`) acquire a shared read lock, allowing concurrent stats collection without blocking cache operations.
- All lock acquisitions use adaptive timeouts (scaled by operation complexity) to prevent deadlocks.
- The write-lock owner thread is tracked for deadlock diagnostics.

```
__init__(max_size_bytes=104857600, ttl_seconds=0.0, lock_timeout_seconds=None)
```

Initialize the result cache.

Parameters

- `max_size_bytes` (`int`) – Maximum total memory for cached DataFrames. 0 disables caching entirely.

- `ttl_seconds` (`float`) – Time-to-live per entry in seconds. 0 means entries never expire by time (only by LRU eviction or mutation-based invalidation).
- `lock_timeout_seconds` (`float` | `None`) – Base timeout for lock acquisition in seconds. `None` uses the class default (5 s). The actual timeout is scaled by a per-operation multiplier (e.g. `put` gets 2x the base timeout). Set to 0 for non-blocking semantics.

property enabled: `bool`

Whether caching is active (`max_size_bytes > 0`).

`get(query, parameters)`

Look up a cached result.

Uses an exclusive write lock because cache hits mutate LRU order.

Returns

A copy of the cached `DataFrame`, or `None` on miss. Returns `None` if the lock cannot be acquired (timeout).

Return type

`DataFrame` | `None`

`put(query, parameters, result)`

Store a query result in the cache.

If the lock cannot be acquired within the adaptive timeout, the result is silently not cached (query execution is unaffected).

`invalidate()`

Bump the generation counter, lazily invalidating all entries.

Called when a mutation is committed to the Context. Existing entries are not deleted immediately — they are evicted on the next `get()` that detects the stale generation, or during LRU eviction.

`clear()`

Remove all cached entries immediately.

`stats()`

Return cache statistics.

Uses a shared read lock so multiple threads can read stats concurrently without blocking cache operations.

Timeout Handler

Timeout management for query execution. Provides a context manager that arms both cooperative and hard (SIGALRM) timeouts with reliable cleanup. Timeout management for query execution.

Provides a context-manager that arms both cooperative and hard (SIGALRM) timeouts and cleans up reliably in the finally block.

Usage:


```

handler = TimeoutHandler(context, timeout_seconds=5.0, query_str="MATCH ...")
with handler:
    # execute query — cooperative check_timeout() between clauses
    # SIGALRM fires if stuck in C extension beyond deadline
    ...

```

```

class pycypher.timeout_handler.TimeoutHandler(context, *, timeout_seconds, query_str="",
                                              start_time=None)

```

Bases: `object`

Arm and disarm query timeouts with cooperative + SIGALRM protection.

The handler is a context manager. On `__enter__` it:

1. Sets the cooperative deadline on the Context (checked between clauses).
2. Installs a SIGALRM handler on Unix main-thread (catches stuck C extensions).

On `__exit__` it:

1. Cancels the SIGALRM.
2. Restores the previous signal handler.
3. Clears the cooperative deadline on the Context.

Thread-safety: SIGALRM is only armed when running on the main thread. On non-main threads or non-Unix platforms, only the cooperative deadline is used.

Parameters

- `context` (Any) – The `Context` to set/clear the cooperative deadline on.
- `timeout_seconds` (float | None) – Wall-clock timeout in seconds. None means no timeout.
- `query_str` (str) – Query text for inclusion in timeout error messages.
- `start_time` (float | None) – Reference time (`time.perf_counter()`) for elapsed calculation. Defaults to the time of `__enter__`.

```

__init__(context, *, timeout_seconds, query_str="", start_time=None)

```

property `timeout_seconds`: float | None

The configured timeout, or None if no timeout.

```

__enter__()

```

Arm cooperative deadline and SIGALRM.

```

__exit__(exc_type, exc_val, exc_tb)

```

Cancel SIGALRM and clear cooperative deadline.

Mutation Engine

Handles write operations: CREATE, SET, DELETE, REMOVE, MERGE, and FOREACH clauses with shadow-layer transaction semantics. Mutation engine — all write-path operations for Cypher query execution.

Extracted from the Star god-object to provide a focused, single-responsibility module for CREATE, SET, DELETE, DETACH DELETE, MERGE, FOREACH, and REMOVE clause execution.

All mutations are staged in the per-query shadow layer on the [Context](#) and only committed when the enclosing query succeeds.

Usage:

```
engine = MutationEngine(context=ctx)
engine.process_create(clause, current_frame)
engine.set_properties(clause, frame)
engine.process_delete(clause, frame)
```

```
class pycypher.mutation_engine.MutationEngine(context)
```

Bases: [object](#)

Encapsulates all write-path operations for Cypher query execution.

Each method corresponds to a mutation clause type (SET, CREATE, DELETE, MERGE, FOREACH, REMOVE). All mutations go through the shadow-write layer on the Context, ensuring atomicity at the query level.

State transition model

Mutations follow a two-phase pattern:

1. Stage — each mutation method writes to the Context's shadow layer (a copy-on-write overlay). The original DataFrames are never modified in-place during execution.
2. Commit — after all clauses execute successfully, the shadow layer is merged into the live entity/relationship tables. On failure, the shadow layer is discarded (rollback semantics).

This ensures that a query like MATCH (a) SET a.x = 1 SET a.y = a.x + 1 sees consistent state within the same query, and a failing query leaves the context unchanged.

Supported clauses:

- CREATE — [process_create\(\)](#) — insert new nodes/relationships
- SET — [set_properties\(\)](#) — update properties on matched entities
- DELETE / DETACH DELETE — [process_delete\(\)](#) — remove entities
- MERGE — [process_merge\(\)](#) — match-or-create with ON CREATE/MATCH SET
- FOREACH — [process_foreach\(\)](#) — iterate and mutate
- REMOVE — handled via SET with null values

param context

The [Context](#) holding entity and relationship mappings.

[__init__](#)(context)

Initialize mutation engine.

Parameters

context ([Context](#)) – The execution context holding entity and relationship mappings. Mutations are applied through the shadow-write layer.

set_properties(set_clause, frame)

Apply a SET clause against a [BindingFrame](#) (mutates context in place).

Only [SetPropertyItem](#) is supported. Each item evaluates its value expression via [BindingExpressionEvaluator](#) and calls [mutate\(\)](#) to write back to the entity table.

Parameters

- set_clause ([Set](#)) – AST [Set](#) node.
- frame ([BindingFrame](#)) – Current [BindingFrame](#).

next_entity_ids(entity_type, n)

Generate n unique new IDs for entity_type.

next_relationship_ids(rel_type, n)

Generate n unique new IDs for rel_type.

shadow_create_entity(entity_type, new_ids, props)

Append new rows to the entity shadow layer for entity_type.

If the entity type does not yet exist in the shadow, the method seeds it from the live entity table (or an empty [DataFrame](#) if the type is brand-new).

Parameters

- entity_type ([str](#)) – Label of the entity to insert.
- new_ids ([list](#)[[Any](#)]) – List of new integer IDs (one per new row).
- props ([dict](#)[[str](#), [list](#)[[Any](#)]]) – Mapping of property name -> list of values.

shadow_create_relationship(rel_type, new_ids, src_ids, tgt_ids)

Append new rows to the relationship shadow layer for rel_type.

Parameters

- rel_type ([str](#)) – Relationship type label (e.g. "KNOWS").
- new_ids ([list](#)[[Any](#)]) – New relationship IDs.
- src_ids ([list](#)[[Any](#)]) – Source node IDs for each new relationship.
- tgt_ids ([list](#)[[Any](#)]) – Target node IDs for each new relationship.

process_create(clause, current_frame, *, make_seed_frame)

Execute a CREATE clause — insert new entities/relationships.

For each [PatternPath](#) in the clause pattern, node and relationship patterns are evaluated against the current frame. When a preceding MATCH is present, one new entity/relationship row is created per frame row.

Parameters

- clause ([Create](#)) – AST [Create](#) node.
- current_frame ([BindingFrame](#) | [None](#)) – The current binding frame (may be [None](#)).

- `make_seed_frame` (`Callable[[], BindingFrame]`) – Callable that produces a single-row seed frame.

Returns

An updated `BindingFrame`.

Return type

`BindingFrame` | `None`

`process_delete`(`clause`, `frame`)

Execute a DELETE or DETACH DELETE clause.

Parameters

- `clause` (`Delete`) – AST `Delete` node.
- `frame` (`BindingFrame`) – Current `BindingFrame`.

`process_merge`(`clause`, `current_frame`, *, `match_to_binding_frame`, `merge_frames_for_match`, `make_seed_frame`)

Execute a MERGE clause — match existing or create new.

Parameters

- `clause` (`Merge`) – AST `Merge` node.
- `current_frame` (`BindingFrame` | `None`) – The current binding frame.
- `match_to_binding_frame` (`Callable[..., BindingFrame]`) – Callback to `Star._match_to_binding_frame`.
- `merge_frames_for_match` (`Callable[[BindingFrame, BindingFrame], BindingFrame]`) – Callback to `Star._merge_frames_for_match`.
- `make_seed_frame` (`Callable[[], BindingFrame]`) – Callback to `Star._make_seed_frame`.

Returns

An updated `BindingFrame`.

Return type

`BindingFrame` | `None`

`process_foreach`(`clause`, `current_frame`, *, `make_seed_frame`)

Execute a FOREACH clause — iterative mutation over a list.

Parameters

- `clause` (`Foreach`) – AST `Foreach` node.
- `current_frame` (`BindingFrame` | `None`) – The current binding frame.
- `make_seed_frame` (`Callable[[], BindingFrame]`) – Callback to `Star._make_seed_frame`.

Returns

`current_frame` unchanged.

Return type

`BindingFrame` | `None`

`remove_properties`(`remove_clause`, `frame`)

Apply a REMOVE clause against a `BindingFrame`.

Parameters

- `remove_clause` (`Remove`) – AST `Remove` node.
- `frame` (`BindingFrame`) – Current `BindingFrame`.

`process_call`(`clause`, `current_frame`)

Execute a `CALL` clause — invoke a registered procedure and `YIELD` results.

Parameters

- `clause` (`Call`) – AST `Call` node.
- `current_frame` (`BindingFrame` | `None`) – The current binding frame.

Returns

An updated `BindingFrame` with the `YIELD`d columns.

Return type

`BindingFrame` | `None`

4.1.7 Multi-Query Composition

Multi-Query Analyzer

Builds a dependency graph from multiple Cypher queries by analyzing entity/relationship types produced and consumed by each query. Multi-query dependency analysis — extracts produces/consumes from ASTs.

Provides the core dependency analysis components for multi-query composition:

- `QueryNode` — represents a single query with produces/consumes metadata
- `DependencyGraph` — container with topological sort capability
- `QueryDependencyAnalyzer` — extracts entity/relationship types from ASTs

These components are used by `pycypher.multi_query_rewriter` to determine execution order when combining multiple Cypher queries.

```
class pycypher.multi_query_analyzer.QueryNode(query_id, cypher_query, ast, produces=<factory>,
                                              consumes=<factory>, dependencies=<factory>)
```

Bases: `object`

Represents one query in the dependency analysis.

`query_id`

Unique identifier for this query.

Type

`str`

`cypher_query`

Original Cypher query string.

Type

`str`

`ast`

Parsed `Query` AST.

Type

`pycypher.ast_models.clauses.Query`

produces

Entity/relationship types created by this query.

Type

set[str]

consumes

Entity/relationship types matched by this query.

Type

set[str]

dependencies

IDs of other QueryNodes this query depends on.

Type

set[str]

query_id: str

cypher_query: str

ast: Query

produces: set[str]

consumes: set[str]

dependencies: set[str]

__init__(query_id, cypher_query, ast, produces=<factory>, consumes=<factory>, dependencies=<factory>)

class pycypher.multi_query_analyzer.DependencyGraph(nodes)

Bases: object

Dependency graph for multi-query execution planning.

nodes

List of QueryNode instances in the graph.

Type

list[pycypher.multi_query_analyzer.QueryNode]

nodes: list[QueryNode]

topological_sort()

Return nodes in dependency-respecting execution order.

Uses Kahn's algorithm: repeatedly selects nodes whose dependencies have all been processed.

Returns

List of QueryNode in valid execution order.

Raises

ValueError – If a circular dependency is detected.

Return type

list[QueryNode]

```
__init__(nodes)
```

```
class pycypher.multi_query_analyzer.QueryDependencyAnalyzer
```

Bases: `object`

Analyzes query ASTs to build a dependency graph.

Parses each Cypher query, extracts the entity/relationship types it creates (produces) and matches (consumes), then infers inter-query dependencies from the produces/consumes overlap.

```
analyze(queries)
```

Analyze multiple queries to build a dependency graph.

Parameters

`queries` (`list[tuple[str, str]]`) – List of (`query_id`, `cypher_string`) pairs.

Returns

`DependencyGraph` with produces/consumes/dependencies populated.

Raises

- `SecurityError` – If too many queries are submitted or a query ID is invalid.
- `ValueError` – If a query ID is invalid.

Return type

`DependencyGraph`

Multi-Query Executor

Orchestrates execution of multi-query pipelines with dependency-aware ordering and shared context propagation. Multi-query executor — the user-facing API for query composition.

Orchestrates all multi-query components into a single entry point:

1. Input validation — validates query pairs before processing
2. Dependency analysis — determines execution order from AST analysis
3. Query combination — merges queries with `WITH *` and `RETURN` stripping
4. Execution — runs the combined query via `Star.execute_query()`

Provides both full-pipeline execution and individual stage access (`validate`, `analyze`, `combine`) for debugging and testing.

Example:

```
from pycypher.multi_query_executor import MultiQueryExecutor
from pycypher.relational_models import Context
from pycypher.star import Star

ctx = Context()
star = Star(context=ctx)
executor = MultiQueryExecutor()

result = executor.execute_multi_query([
    ("q1", "CREATE (n:Person {name: 'Alice'})"),
    ("q2", "MATCH (n:Person) RETURN n.name"),
], star)
```

```
class pycypher.multi_query_executor.MultiQueryExecutor
```

```
    Bases: object
```

```
    Orchestrates multi-query composition and execution.
```

```
    Integrates input validation, dependency analysis, query combination, and execution into a single API.
    Each stage is also accessible individually for debugging and testing.
```

```
    __init__()
```

```
        Initialise the executor with internal components.
```

```
    validate(queries)
```

```
        Validate inputs without executing.
```

```
        Parameters
```

```
            queries (list[tuple[str, str]]) – List of (query_id, cypher_string) pairs.
```

```
        Returns
```

```
            InputValidationResult.
```

```
        Return type
```

```
            InputValidationResult
```

```
    analyze(queries)
```

```
        Analyze dependencies without executing.
```

```
        Parameters
```

```
            queries (list[tuple[str, str]]) – List of (query_id, cypher_string) pairs.
```

```
        Returns
```

```
            DependencyGraph.
```

```
        Return type
```

```
            DependencyGraph
```

```
    combine(queries)
```

```
        Combine queries into a single Cypher string without executing.
```

```
        Parameters
```

```
            queries (list[tuple[str, str]]) – List of (query_id, cypher_string) pairs.
```

```
        Returns
```

```
            Combined Cypher query string.
```

```
        Return type
```

```
            str
```

```
    execute_multi_query(queries, star)
```

```
        Execute multiple queries as a single combined query.
```

```
        Pipeline: validate → analyze → combine → execute.
```

```
        Parameters
```

- queries ([list](#)[[tuple](#)[[str](#), [str](#)]]) – List of (query_id, cypher_string) pairs.
- star ([Star](#)) – [Star](#) instance for execution.

```
        Returns
```

```
            Result DataFrame from combined query execution.
```


Raises
 [ValueError](#) – If input validation fails.

Return type
 pd.DataFrame

4.1.8 Validation

Semantic Validator

Pre-execution validation: undefined variable detection, aggregation rule checking, type constraint verification. Semantic validation for Cypher queries.

This module provides semantic analysis beyond syntax checking, including: - Variable scope and binding validation - Undefined variable detection - Aggregation rule validation - Function signature validation - Type checking for expressions

Example

```
>>> from pycypher.semantic_validator import SemanticValidator
>>> from pycypher.grammar_parser import GrammarParser
>>>
>>> parser = GrammarParser()
>>> validator = SemanticValidator()
>>>
>>> query = "MATCH (n:Person) RETURN m" # 'm' undefined
>>> tree = parser.parse(query)
>>> errors = validator.validate(tree)
>>> for error in errors:
...     print(f"{error.line}:{error.column} - {error.message}")
```

```
class pycypher.semantic_validator.ErrorSeverity(*values)
```

Bases: [Enum](#)

Severity levels for validation errors.

ERROR = 'error'

WARNING = 'warning'

INFO = 'info'

```
class pycypher.semantic_validator.ValidationError(severity, message, line=None, column=None,
                                                  node_type=None, variable_name=None)
```

Bases: [object](#)

Represents a semantic validation error.

severity: [ErrorSeverity](#)

message: [str](#)

line: [int](#) | [None](#) = [None](#)

column: [int](#) | [None](#) = [None](#)

node_type: str | None = None

variable_name: str | None = None

__str__()

Format error as string.

__init__(severity, message, line=None, column=None, node_type=None, variable_name=None)

class pycypher.semantic_validator.VariableScope(parent=None)

Bases: object

Manages variable scope and bindings in Cypher queries.

__init__(parent=None)

Initialize a variable scope.

Parameters

parent (VariableScope | None) – Parent scope for nested scopes (e.g., subqueries, comprehensions).

define(var_name)

Mark a variable as defined in this scope.

use(var_name)

Mark a variable as used in this scope.

is_defined(var_name)

Check if variable is defined in this scope or parent scopes.

get_undefined_vars()

Get variables used but not defined in accessible scopes.

create_child_scope()

Create a nested child scope.

class pycypher.semantic_validator.SemanticValidator

Bases: object

Validates semantic correctness of Cypher queries.

Performs validation beyond syntax checking, including: - Variable scope and binding - Aggregation rules - Function signatures - Return clause validation

```
__init__()
```

Initialize the semantic validator.

```
validate(tree)
```

Validate a parse tree and return any errors found.

Parameters

tree (Tree) – Lark parse tree from grammar_parser.

Returns

List of ValidationError objects found during validation.

Return type

`list[ValidationError]`

```
pycypher.semantic_validator.validate_query(query_string)
```

Convenience function to parse and validate a query string.

Parameters

query_string (str) – Cypher query string to validate.

Returns

List of validation errors.

Return type

`list[ValidationError]`

Example

```
>>> errors = validate_query("MATCH (n) RETURN m")
>>> for error in errors:
...     print(error)
```

4.1.9 Ingestion Layer

The ingestion layer provides Arrow (via PyArrow) as the canonical in-memory tabular format and DuckDB as the universal ingestion adapter.

Context Builder

Fluent API for constructing Context objects from DataFrames, Parquet files, CSV files, and other tabular sources. Fluent builder that assembles a Context from Arrow-loaded data.

Usage example:

```
from pycypher.ingestion import ContextBuilder

context = (
    ContextBuilder()
    .add_entity("Person", "people.csv", id_col="person_id")
    .add_relationship(
        "KNOWS",
        "knows.csv",
        source_col="from_id",
        target_col="to_id",
    )
)
```

(continues on next page)

(continued from previous page)

```
.build()
)
```

```
class pycypher.ingestion.context_builder.ContextBuilder
```

Bases: `object`

Fluent builder that assembles a Context from heterogeneous sources.

Methods can be chained:

```
ctx = ContextBuilder().add_entity(...).add_relationship(...).build()
```

```
__init__()
```

```
add_entity(entity_type, source, *, id_col=None, query=None)
```

Register an entity type loaded from source.

Parameters

- `entity_type` (`str`) – The entity label (e.g. "Person").
- `source` (`str` | `pd.DataFrame` | `pa.Table`) – Data source — a file path, pandas DataFrame, or Arrow table.
- `id_col` (`str` | `None`) – Column to use as `__ID__`. Defaults to auto-generated sequential integers.
- `query` (`str` | `None`) – Optional SQL query applied after loading (file paths only).

Returns

self for chaining.

Return type

`ContextBuilder`

```
add_relationship(relationship_type, source, *, source_col, target_col, id_col=None, query=None)
```

Register a relationship type loaded from source.

Parameters

- `relationship_type` (`str`) – The relationship label (e.g. "KNOWS").
- `source` (`str` | `pd.DataFrame` | `pa.Table`) – Data source — a file path, pandas DataFrame, or Arrow table.
- `source_col` (`str`) – Column that holds the source node ID.
- `target_col` (`str`) – Column that holds the target node ID.
- `id_col` (`str` | `None`) – Column to use as `__ID__`. Defaults to auto-generated sequential integers.
- `query` (`str` | `None`) – Optional SQL query applied after loading (file paths only).

Returns

self for chaining.

Return type

`ContextBuilder`

classmethod from_dict(entity_frames, *, id_column=None)

Build a `Context` from a dict of DataFrames.

Each key is a node or relationship label; the corresponding value is a pandas `DataFrame`. DataFrames are automatically classified:

- If the `DataFrame` contains both `__SOURCE__` and `__TARGET__` columns it is registered as a relationship table.
- Otherwise it is registered as an entity table.

This allows a single `from_dict()` call to supply both nodes and edges without needing the verbose `add_entity` / `add_relationship` builder chain:

```
ctx = ContextBuilder.from_dict({
    "Person": persons_df,      # entity — no __SOURCE__/__TARGET__
    "KNOWS": knows_df,        # relationship — has both columns
})
```

Parameters

- `entity_frames` (`dict[str, DataFrame]`) – Mapping of label to `DataFrame` (entity or relationship).
- `id_column` (`str` | `None`) – Column to use as the `__ID__` identity key. Defaults to the standard `ID_COLUMN` (`"__ID__"`). Pass the name of an existing column to use it as the identifier.

Returns

A fully populated `Context`.

Raises

`TypeError` – If any value in `entity_frames` is not a `pandas.DataFrame`.

Return type

`Context`

`build()`

Assemble and return the `Context`.

Returns

A fully populated `Context`.

Return type

`Context`

Data Sources

Abstract `DataSource` hierarchy and URI-based factory.

`DataSource` is the primary abstraction for reading tabular data into Arrow tables. Every concrete subclass wraps one kind of external storage and exposes a single `read()` method that returns a `pa.Table`.

Use `data_source_from_uri()` to obtain the correct subclass for a given URI string without writing dispatch code yourself.

Supported URI schemes

URI / value	Resolved class
file:///path/file.csv /path/file.csv (bare path) s3://bucket/file.csv https://host/file.csv	FileDataSource FileDataSource FileDataSource FileDataSource
Same variations with .parquet	FileDataSource
Same variations with .json	FileDataSource
postgresql://..., mysql://... sqlite://..., duckdb://...	SqlDataSource SqlDataSource
pd.DataFrame	DataFrameDataSource
pa.Table	ArrowDataSource

class pycypher.ingestion.data_sources.Format

Bases: [ABC](#)

Abstract base for file-format strategies.

Each subclass knows how to generate the DuckDB SQL fragment that reads one specific file format (CSV, Parquet, JSON, ...).

abstract property name: [str](#)

Short lowercase name identifying the format (e.g. "csv").

abstractmethod view_sql(path)

Return a DuckDB SQL read-function call for path.

Parameters

path ([str](#)) – Filesystem path or cloud URI to the file.

Returns

A SQL fragment such as "read_csv_auto('/data/f.csv')".

Return type

[str](#)

class pycypher.ingestion.data_sources.CsvFormat(delimiter=',', header=True, null_padding=False)

Bases: [Format](#)

Strategy for reading CSV files via DuckDB's read_csv_auto.

delimiter

Field delimiter character. Defaults to ','.

Type

[str](#)

header

Whether the file has a header row. Defaults to True.

Type

[bool](#)

null_padding

Pad missing trailing columns with NULL.

Type

[bool](#)

delimiter: `str` = ','

header: `bool` = True

null_padding: `bool` = False

property name: `str`

Return the format identifier for CSV files.

Returns

The string "csv" identifying this format type.

view_sql(path)

Generate a DuckDB SQL fragment to read a CSV file with custom options.

Produces a `read_csv_auto()` call that incorporates this format's delimiter, header, and null_padding settings. The path is validated to prevent SQL injection attacks.

Parameters

path (`str`) – Filesystem path or URI to the CSV file to read.

Returns

A SQL fragment like `"read_csv_auto('/data/file.csv', delim=',')"` that can be embedded in DuckDB queries.

Raises

`ValueError` – If path or delimiter contains characters unsafe for SQL string literals (single quotes or NUL bytes).

Return type

`str`

`__init__(delimiter=',', header=True, null_padding=False)`

class `pycypher.ingestion.data_sources.ParquetFormat`

Bases: `Format`

Strategy for reading Parquet files via DuckDB's `read_parquet`.

property name: `str`

Return the format identifier for Parquet files.

Returns

The string "parquet" identifying this format type.

view_sql(path)

Generate a DuckDB SQL fragment to read a Parquet file.

Produces a `read_parquet()` call for the given path. Parquet files are self-describing and require no additional format options. The path is validated to prevent SQL injection attacks.

Parameters

path (`str`) – Filesystem path or URI to the Parquet file to read.

Returns

A SQL fragment like `"read_parquet('/data/file.parquet')"` that can be embedded in DuckDB queries.

Raises
 `ValueError` – If path contains characters unsafe for SQL string literals (single quotes or NUL bytes).

Return type
 `str`

`__init__()`

class pycypher.ingestion.data_sources.JsonFormat(records='auto')

Bases: `Format`

Strategy for reading JSON files via DuckDB's `read_json_auto`.

records

 JSON record layout. 'auto' (default) lets DuckDB infer the layout; 'newline_delimited' forces NDJSON parsing.

 Type
 `str`

records: `str` = 'auto'

property name: `str`

 Return the format identifier for JSON files.

 Returns
 The string "json" identifying this format type.

`view_sql(path)`

 Generate a DuckDB SQL fragment to read a JSON file with record layout options.

 Produces a `read_json_auto()` call that incorporates this format's records layout setting. When records is "auto", DuckDB infers the JSON structure. When set to "newline_delimited", forces NDJSON parsing. The path and records values are validated to prevent SQL injection attacks.

 Parameters
 path (`str`) – Filesystem path or URI to the JSON file to read.

 Returns
 A SQL fragment like `"read_json_auto('/data/file.json')"` or `"read_json_auto('/data/file.json', format='newline_delimited')"` that can be embedded in DuckDB queries.

 Raises
 `ValueError` – If path or records contains characters unsafe for SQL string literals (single quotes or NUL bytes).

 Return type
 `str`

`__init__(records='auto')`

```
class pycypher.ingestion.data_sources.DataSource
```

```
    Bases: ABC
```

```
    Abstract base for all tabular data sources.
```

```
    Every concrete subclass implements read() to load its backing store and return the result as a pa.Table. All downstream pipeline components (normalisation, query execution) consume Arrow tables produced by read().
```

```
    abstract property uri: str
```

```
        Canonical identifier for this data source.
```

```
        Returns a URI string for file/SQL sources, or a descriptive placeholder (e.g. "<dataframe>") for in-memory sources.
```

```
    abstractmethod read()
```

```
        Load the source and return its contents as an Arrow table.
```

```
        Returns
```

```
            A pa.Table containing the full (or filtered, when a query was supplied) contents of the source.
```

```
        Return type
```

```
            Table
```

```
class pycypher.ingestion.data_sources.FileDataSource(uri, format, *, query=None)
```

```
    Bases: DataSource
```

```
    File-backed data source that delegates format parsing to a Format.
```

```
    The format object encapsulates the DuckDB read function and its options, so adding new formats (Avro, ORC, Delta ...) requires only a new Format subclass without touching this class.
```

```
    Parameters
```

- `uri` (`str`) – Path or URI to the file. Bare filesystem paths, `file://`, `s3://`, `gs://`, `https://`, and `abfss://` URIs are all accepted; DuckDB resolves them at read time.
- `format` (`Format`) – A `Format` instance that supplies the DuckDB SQL read fragment.
- `query` (`str` | `None`) – Optional DuckDB SQL applied after the file is registered as the view source. When `None`, `SELECT * FROM source` is used.

```
    __init__(uri, format, *, query=None)
```

```
    property uri: str
```

```
        The original URI or file path used to construct this data source.
```

```
    property format: Format
```

```
        The Format strategy used to read this file.
```

```
    property query: str | None
```

```
        Optional SQL override applied when reading this source.
```

```
    read()
```

```
        Read the file via DuckDB and return an Arrow table.
```

```
        Returns
```

```
            pa.Table with the file contents (or query results).
```

Raises

`SecurityError` – If URI or query contains dangerous content.

Return type

Table

```
class pycypher.ingestion.data_sources.SqlDataSource(uri, query)
```

Bases: `DataSource`

Reads from a relational database via a DuckDB connection string.

DuckDB supports connecting to PostgreSQL, MySQL, SQLite, and other DuckDB files via its scanner extensions. The query parameter is required because there is no meaningful default when targeting a database.

```
__init__(uri, query)
```

Initialise a SQL source.

Parameters

- `uri` (`str`) – DuckDB-compatible connection string (e.g. `"postgresql://user:pass@host:5432/mydb"`).
- `query` (`str`) – SQL query to execute against the database. The result set is returned as an Arrow table.

property `uri`: `str`

The database connection URI (e.g. `postgresql://...`, `sqlite://...`).

property `query`: `str`

SQL query executed against the database.

```
read()
```

Connect to the database, execute the query, and return Arrow.

Returns

`pa.Table` with the query result set.

Raises

`SecurityError` – If URI or query contains dangerous content.

Return type

Table

```
class pycypher.ingestion.data_sources.DataFrameDataSource(df)
```

Bases: `DataSource`

Wraps a pandas `DataFrame` as a `DataSource`.

The `DataFrame` is passed through DuckDB to produce a `pa.Table` with DuckDB's type mapping, ensuring consistent Arrow schema handling.

```
__init__(df)
```

Initialise with a pandas `DataFrame`.

Parameters

`df` (`DataFrame`) – The source `DataFrame`.

property uri: `str`

Synthetic URI placeholder for in-memory DataFrame sources.

property dataframe: `DataFrame`

The wrapped pandas DataFrame.

`read()`

Convert the DataFrame to an Arrow table via DuckDB.

Returns

`pa.Table` with the DataFrame contents.

Return type

`Table`

class `pycypher.ingestion.data_sources.ArrowDataSource(table)`

Bases: `DataSource`

Wraps an existing `pa.Table` as a `DataSource`.

`read()` is a direct passthrough — no copy or DuckDB round-trip is performed.

`__init__(table)`

Initialise with an Arrow table.

Parameters

`table (Table)` – The source Arrow table.

property uri: `str`

Synthetic URI placeholder for in-memory Arrow table sources.

property table: `Table`

The wrapped Arrow table.

`read()`

Return the wrapped Arrow table.

Returns

The `pa.Table` passed to the constructor.

Return type

`Table`

`pycypher.ingestion.data_sources.data_source_from_uri(uri, *, query=None)`

Construct the appropriate `DataSource` subclass from uri.

Dispatch rules (evaluated in order):

1. `pa.Table` → `ArrowDataSource`
2. `pd.DataFrame` → `DataFrameDataSource`
3. SQL-scheme string (`postgresql:`, `postgres:`, `mysql:`, `sqlite:`, `duckdb:`) → `SqlDataSource` (query is required for SQL sources)
4. String ending in `.csv` → `FileDataSource` with `CsvFormat`
5. String ending in `.parquet` → `FileDataSource` with `ParquetFormat`
6. String ending in `.json` → `FileDataSource` with `JsonFormat`

The URI may use any scheme that DuckDB supports for steps 4-6: `file://`, `s3://`, `gs://`, `abfss://`, `http://`, `https://`, or a bare filesystem path.

Parameters

- `uri` (`str` | `DataFrame` | `Table`) – Source identifier. Accepts a URI string, a `pd.DataFrame`, or a `pa.Table`.
- `query` (`str` | `None`) – Optional SQL override for file-based sources. Required for SQL-scheme (database) sources.

Returns

A `DataSource` instance of the appropriate subclass.

Raises

- `ValueError` – If `uri` is a SQL-scheme string but `query` is `None`, or if the string's extension is not recognised.
- `TypeError` – If `uri` is not a `str`, `pd.DataFrame`, or `pa.Table`.

Return type

`DataSource`

Examples

```
>>> src = data_source_from_uri("s3://bucket/people.parquet")
>>> src = data_source_from_uri(
...     "postgresql://user:pass@host/db",
...     query="SELECT * FROM persons WHERE active",
... )
>>> src = data_source_from_uri(my_dataframe)
```

Pipeline Config

Pydantic models for the pycypher ETL pipeline YAML configuration file.

The top-level model is `PipelineConfig`, loaded from disk via `load_pipeline_config()`. Every public field that is not required for the system to function is typed `Optional` and defaults to `None` or an empty collection.

Environment variables

String values in the YAML file may contain `${VAR_NAME}` placeholders. `load_pipeline_config()` substitutes them from the process environment before Pydantic validation. Unresolved variables (env var not set) are left as-is; an error is raised only when the value is actually used (e.g. when a source is loaded or an output is written).

Supported URI schemes

The `uri` field on source and output records is validated at config-load time. Accepted forms:

- `file:///path/to/file.{csv,parquet,json}` — local files
- `/path/to/file.{csv,parquet,json}` — bare filesystem paths
- `s3://`, `gs://`, `abfss://`, `http://`, `https://` — cloud / remote files, with a `.csv`, `.parquet`, or `.json` extension
- `postgresql://`, `postgres://`, `mysql://`, `sqlite://`, `duckdb://` — SQL databases (a `query` field is also required)

Any URI with an unrecognised scheme + extension combination, or a SQL-scheme URI without a query, raises a `pydantic.ValidationError` immediately.

```
class pycypher.ingestion.config.ErrorHandlingPolicy(*values)
```

Bases: [StrEnum](#)

Policy applied when a source fails to load or a query fails.

FAIL

Abort the pipeline immediately (default).

WARN

Log a warning and continue.

SKIP

Silently skip the failing item and continue.

FAIL = 'fail'

WARN = 'warn'

SKIP = 'skip'

```
class pycypher.ingestion.config.SkippedSource(source_id, error, policy)
```

Bases: [NamedTuple](#)

Record of a data source that was suppressed by an error policy.

Returned by [PipelineConfig.load_with_sources\(\)](#) so callers can programmatically inspect which sources failed and why, rather than relying solely on log output.

source_id

The id of the source that failed.

Type

[str](#)

error

The exception that caused the failure.

Type

[Exception](#)

policy

The error handling policy that suppressed the failure.

Type

[pycypher.ingestion.config.ErrorHandlingPolicy](#)

source_id: [str](#)

Alias for field number 0

error: [Exception](#)

Alias for field number 1

policy: [ErrorHandlingPolicy](#)

Alias for field number 2

```

class pycypher.ingestion.config.OutputFormat(*values)
    Bases: StrEnum
    Explicit output serialisation format.
    When absent, the format is inferred from the uri file extension.
    CSV
        Comma-separated values.
    PARQUET
        Apache Parquet columnar format.
    JSON
        JSON (newline-delimited records).
    CSV = 'csv'
    PARQUET = 'parquet'
    JSON = 'json'

class pycypher.ingestion.config.ProjectConfig(*, name, description=None)
    Bases: BaseModel
    Human-readable project metadata.

    name
        Short identifier for the pipeline (used in logs and artefacts).
        Type
            str

    description
        Optional longer description of the pipeline's purpose.
        Type
            str | None

    name: str
    description: str | None

    model_config: ClassVar[ConfigDict] = {}
        Configuration for the model, should be a dictionary conforming to
        [ConfigDict][pydantic.config.ConfigDict].

class pycypher.ingestion.config.EntitySourceConfig(*, id, uri, entity_type, id_col=None, query=None,
                                                    schema_hints=None, on_error=None)
    Bases: BaseModel
    Configuration for a single entity (node) data source.

    id
        Unique logical identifier for this source within the config file.
        Type
            str

```

`uri`

URI of the data source. See module docstring for supported schemes.

Type
`str`

`entity_type`

The Cypher entity label assigned to rows from this source (e.g. "Person").

Type
`str`

`id_col`

Column in the source whose values become `__ID__`. When absent, sequential integers are auto-generated.

Type
`str` | `None`

`query`

Optional DuckDB SQL applied after loading the source. The loaded data is available as the virtual table source; when omitted `SELECT * FROM source` is used.

Type
`str` | `None`

`schema_hints`

Optional mapping of column name $\rightarrow$ Arrow/DuckDB type string (e.g. `{"zip_code": "VARCHAR", "age": "INTEGER"}`). Applied before the data enters the pipeline to prevent type inference surprises.

Type
`dict[str, str]` | `None`

`on_error`

What to do if this source cannot be loaded. Defaults to the pipeline-level policy (fail when unspecified).

Type
`pycypher.ingestion.config.ErrorHandlingPolicy` | `None`

`id: str`

`uri: str`

`entity_type: str`

`id_col: str` | `None`

`query: str` | `None`

`schema_hints: dict[str, str]` | `None`

`on_error: ErrorHandlingPolicy` | `None`

`check_uri()`

Validate that uri is syntactically correct and of a supported type.

```
model_config: ClassVar[ConfigDict] = {}
```

Configuration for the model, should be a dictionary conforming to [ConfigDict][pydantic.config.ConfigDict].

```
class pycypher.ingestion.config.RelationshipSourceConfig(*, id, uri, relationship_type, source_col,
                                                         target_col, id_col=None, query=None,
                                                         schema_hints=None, on_error=None)
```

Bases: [BaseModel](#)

Configuration for a single relationship (edge) data source.

source_col and target_col are required — there is no sensible default for relationship endpoints.

id

Unique logical identifier for this source.

Type
str

uri

URI of the data source.

Type
str

relationship_type

The Cypher relationship type label (e.g. "WORKS_FOR").

Type
str

source_col

Column whose values are the source node IDs (mapped to __SOURCE__).

Type
str

target_col

Column whose values are the target node IDs (mapped to __TARGET__).

Type
str

id_col

Column to use as __ID__ for the relationship itself. Auto-generated when absent.

Type
str | None

query

Optional DuckDB SQL applied after loading.

Type
str | None

schema_hints

Optional column-type overrides.

Type
dict[str, str] | None

`on_error`

What to do if this source cannot be loaded.

Type

`pycypher.ingestion.config.ErrorHandlingPolicy` | `None`

`id`: `str`

`uri`: `str`

`relationship_type`: `str`

`source_col`: `str`

`target_col`: `str`

`id_col`: `str` | `None`

`query`: `str` | `None`

`schema_hints`: `dict[str, str]` | `None`

`on_error`: `ErrorHandlingPolicy` | `None`

`check_uri()`

Validate that uri is syntactically correct and of a supported type.

`model_config`: `ClassVar[ConfigDict]` = `{}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.ingestion.config.SourcesConfig(*, entities=<factory>, relationships=<factory>)`

Bases: `BaseModel`

Container for all entity and relationship source definitions.

Source id values must be unique across both the entity and relationship lists — all sources share the same logical namespace.

`entities`

Zero or more entity source configurations.

Type

`list[pycypher.ingestion.config.EntitySourceConfig]`

`relationships`

Zero or more relationship source configurations.

Type

`list[pycypher.ingestion.config.RelationshipSourceConfig]`

`entities`: `list[EntitySourceConfig]`

`relationships`: `list[RelationshipSourceConfig]`

`get_source_by_id(source_id)`

Look up a source by its unique id.

Searches both the entity and relationship source lists. The id namespace is shared across both lists (enforced by `check_unique_source_ids()`), so at most one source can match.

Parameters

`source_id` (`str`) – The id value to look up.

Returns

The matching `EntitySourceConfig` or `RelationshipSourceConfig`, or `None` if no source with that id exists.

Return type

`EntitySourceConfig` | `RelationshipSourceConfig` | `None`

`check_unique_source_ids()`

Enforce that every source id is unique across entities and relationships.

Raises

`ValueError` – If any id appears more than once, listing the duplicates.

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

`class pycypher.ingestion.config.FunctionConfig(*, module=None, names=None, callable=None)`

Bases: `BaseModel`

Register one or more Python callables as Cypher functions.

Two mutually exclusive forms are supported:

Module form — register specific functions (or all public ones) from a Python module:

```
module: "mypackage.string_utils"
names:
- normalize_name
- parse_phone_number
# OR: names: "*" (registers all public callables)
```

Callable form — register a single function by its fully-qualified dotted import path:

```
callable: "mypackage.string_utils.normalize_name"
```

`module`

Dotted import path of the module to register from.

Type

`str` | `None`

`names`

List of function names within module to register, or `"*"` to register all public callables. Required when module is specified.

Type

`list[str]` | `str` | `None`

callable

Fully-qualified dotted path to a single callable. Mutually exclusive with module.

Type

`str` | `None`

module: `str` | `None`

names: `list[str]` | `str` | `None`

callable: `str` | `None`

classmethod `validate__module__path(v)`

Validate module import path against blocked list.

classmethod `validate__callable__path(v)`

Validate callable import path against blocked list.

`check__exclusive__forms()`

Enforce mutual exclusivity of module-form vs. callable-form.

`model_config: ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

class `pycypher.ingestion.config.QueryConfig(*, id, description=None, source=None, inline=None)`

Bases: `BaseModel`

A named Cypher query defined either by file reference or inline text.

Exactly one of source or inline must be provided.

id

Unique identifier for this query within the pipeline.

Type

`str`

description

Optional human-readable description.

Type

`str` | `None`

source

Path to a `.cypher` file containing the query. Relative paths are resolved from the directory containing the config file.

Type

`str` | `None`

`inline`

Literal Cypher query text. Intended as a convenience escape hatch for short, stable queries; prefer source for anything non-trivial.

Type

`str` | `None`

`id`: `str`

`description`: `str` | `None`

`source`: `str` | `None`

`inline`: `str` | `None`

`check_query_source()`

Enforce that exactly one of source or inline is provided.

`model_config`: `ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

class `pycypher.ingestion.config.OutputConfig(*, query_id, uri, format=None)`

Bases: `BaseModel`

Sink configuration for writing a query's result.

`query_id`

The id of the `QueryConfig` whose result is written to this sink.

Type

`str`

`uri`

Destination URI. Format is inferred from the file extension when format is absent.

Type

`str`

`format`

Explicit output format. When None, inferred from the URI extension (e.g. `.parquet` → `parquet`).

Type

`pycypher.ingestion.config.OutputFormat` | `None`

`query_id`: `str`

`uri`: `str`

`format`: `OutputFormat` | `None`

`check_uri()`

Validate that uri is syntactically correct and of a supported type.

```

model_config: ClassVar[ConfigDict] = {}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

pycypher.ingestion.config.SUPPORTED_CONFIG_VERSIONS: frozenset[str] = frozenset({'1.0'})
    Versions that the current code can fully load without any migration.

pycypher.ingestion.config.CURRENT_CONFIG_VERSION: str = '1.0'
    The latest schema version — used as the default for new configs.

class pycypher.ingestion.config.PipelineConfig(*, version='1.0', project=None, sources=<factory>,
                                              functions=<factory>, queries=<factory>,
                                              output=<factory>)

    Bases: BaseModel
    Root configuration model for a pycypher ETL pipeline.

version
    Configuration schema version. Must match a version in SUPPORTED_CONFIG_VERSIONS.
    Defaults to "1.0".
    Type
        str

project
    Optional project metadata (name, description).
    Type
        pycypher.ingestion.config.ProjectConfig | None

sources
    Entity and relationship source definitions.
    Type
        pycypher.ingestion.config.SourcesConfig

functions
    Python callables to register as Cypher functions.
    Type
        list[pycypher.ingestion.config.FunctionConfig]

queries
    Named Cypher queries. Execution order is automatically inferred from the entity/relationship
    types referenced in each query's parsed AST.
    Type
        list[pycypher.ingestion.config.QueryConfig]

output
    Sink configurations mapping query results to output URIs.
    Type
        list[pycypher.ingestion.config.OutputConfig]

version: str

project: ProjectConfig | None

```

sources: SourcesConfig

functions: list[FunctionConfig]

queries: list[QueryConfig]

output: list[OutputConfig]

classmethod check_version(v)

Validate that version is a supported config schema version.

Raises [ValueError](#) for unknown versions with actionable guidance, and emits a [FutureWarning](#) for versions that look newer than the current code supports (e.g. "2.0" when only "1.0" is supported).

get_source_by_id(source_id)

Look up a data source by its unique id.

Convenience passthrough to [SourcesConfig.get_source_by_id\(\)](#).

Parameters

source_id ([str](#)) – The id value to look up.

Returns

The matching source config, or None if not found.

Return type

[EntitySourceConfig](#) | [RelationshipSourceConfig](#) | None

classmethod load_with_sources(path)

Load a config file and eagerly read all data sources into Arrow tables.

This is the single entry point for pipelines that want both a validated config and pre-loaded Arrow tables in one call.

Parameters

path ([str](#) | [Path](#)) – Path to the YAML pipeline configuration file.

Returns

A (config, sources, skipped) tuple where config is a validated [PipelineConfig](#), sources is a dict mapping each source id to its pa.Table, and skipped is a list of [SkippedSource](#) records for any sources suppressed by a WARN or SKIP error policy.

Raises

- [FileNotFoundError](#) – If path does not exist.
- [yaml.YAMLError](#) – If the file contains invalid YAML.
- [pydantic.ValidationError](#) – If the configuration is structurally invalid.

Return type

tuple[[PipelineConfig](#), dict[str, Any], list[[SkippedSource](#)]]

model_config: ClassVar[ConfigDict] = {}

Configuration for the model, should be a dictionary conforming to [ConfigDict][[pydantic.config.ConfigDict](#)].

[pycypher.ingestion.config.load_pipeline_config\(path\)](#)

Load and validate a pipeline configuration from a YAML file.

Processing steps:

1. Read and parse the YAML file.
2. Substitute `${VAR}` placeholders from the process environment.
3. Validate the resulting dict against `PipelineConfig`.

Parameters

`path` (`str` | `Path`) – Path to the YAML configuration file.

Returns

A fully validated `PipelineConfig` instance.

Raises

- `FileNotFoundError` – If path does not exist.
- `yaml.YAMLError` – If the file contains invalid YAML.
- `pydantic.ValidationError` – If the configuration is structurally invalid (missing required fields, constraint violations, etc.).

Return type

`PipelineConfig`

DuckDB Reader

DuckDB-backed readers that return Arrow tables.

Every method opens a fresh in-process DuckDB connection, loads the source, and returns the result as a `pa.Table` via `duckdb_relation.to_arrow_table()`.

`class pycypher.ingestion.duckdb_reader.DuckDBReader`

Bases: `object`

Universal ingestion adapter that normalises any data source to Arrow.

All methods are static. Each method opens a fresh in-process DuckDB connection for the duration of the read and closes it via the context manager protocol on return or exception.

`static from_csv(path, id_col=None, query=None)`

Read a CSV file into an Arrow table.

Parameters

- `path` (`str`) – Path to the CSV file.
- `id_col` (`str` | `None`) – Unused by this method — pass to `normalize_entity_table()` after reading.
- `query` (`str` | `None`) – Optional SQL query to execute against the loaded CSV. When `None`, `SELECT * FROM source` is used.

Returns

Arrow table with the CSV contents.

Raises

`SecurityError` – If path or query contains dangerous content.

Return type

`pa.Table`

```
static from_parquet(path, id_col=None, query=None)
```

Read a Parquet file into an Arrow table.

Parameters

- `path` (`str`) – Path to the Parquet file.
- `id_col` (`str` | `None`) – Unused here; pass to normalise utilities afterward.
- `query` (`str` | `None`) – Optional SQL query against the loaded Parquet.

Returns

Arrow table with the Parquet contents.

Raises

`SecurityError` – If path or query contains dangerous content.

Return type

`pa.Table`

```
static from_json(path, id_col=None, query=None)
```

Read a JSON file into an Arrow table.

Parameters

- `path` (`str`) – Path to the JSON file.
- `id_col` (`str` | `None`) – Unused here; pass to normalise utilities afterward.
- `query` (`str` | `None`) – Optional SQL query against the loaded JSON.

Returns

Arrow table with the JSON contents.

Raises

`SecurityError` – If path or query contains dangerous content.

Return type

`pa.Table`

```
static from_sql(connection_string, query)
```

Execute query against an external database and return Arrow.

Parameters

- `connection_string` (`str`) – DuckDB-compatible connection string.
- `query` (`str`) – SQL query to execute.

Returns

Arrow table with the query results.

Raises

`SecurityError` – If connection string or query contains dangerous content.

Return type

`pa.Table`

Security:

Never embed credentials directly in connection strings or config files. Use environment variables to supply sensitive values:


```
import os
conn = f"postgresql://{os.environ['DB_USER']}:{os.environ['DB_PASSWORD']}@host/db
result = DuckDBReader.from_sql(conn, "SELECT * FROM t")
```

In pipeline YAML configs, reference env vars with `${DB_PASSWORD}` syntax. The `nmctl` security-check command will flag embedded credentials and insecure connection patterns.

Connection strings are validated for safe URI schemes but are not logged to prevent credential leakage. The audit logger records parameter keys only, never values.

```
static from_dataframe(df, id_col=None)
```

Convert a pandas DataFrame to an Arrow table via DuckDB.

Parameters

- `df` (`pd.DataFrame`) – Source pandas DataFrame.
- `id_col` (`str` | `None`) – Unused here; pass to normalise utilities afterward.

Returns

Arrow table with the DataFrame contents.

Return type

`pa.Table`

```
static from_arrow(table)
```

Passthrough / re-normalise an Arrow table via DuckDB.

Useful for applying a custom SQL transformation to an existing Arrow table before ingestion.

Parameters

`table` (`pa.Table`) – Source Arrow table.

Returns

Arrow table (may be a copy after round-tripping through DuckDB).

Return type

`pa.Table`

Arrow Utilities

Utilities for normalising Arrow tables before they enter the pycypher pipeline.

All functions return a `pa.Table` with canonical column names: - `__ID__` — unique row identifier - `__SOURCE__` — source node ID for relationships - `__TARGET__` — target node ID for relationships

```
pycypher.ingestion.arrow_utils.normalize_entity_table(table, id_col=None)
```

Return table with an `__ID__` column as the first column.

Parameters

- `table` (`Table`) – Source Arrow table.
- `id_col` (`str` | `None`) – Column to rename to `__ID__`. If `None` or not present, a sequential integer column is prepended.

Returns

Arrow table whose first column is `__ID__`.

Raises

`ValueError` – If `id_col` is specified but does not exist in table.

Return type
Table

`pycypher.ingestion.arrow_utils.normalize_relationship_table(table, source_col, target_col, id_col=None)`

Return table with `__SOURCE__`, `__TARGET__`, and `__ID__` columns.

Parameters

- `table` (Table) – Source Arrow table.
- `source_col` (str) – Column to rename to `__SOURCE__`.
- `target_col` (str) – Column to rename to `__TARGET__`.
- `id_col` (str | None) – Column to rename to `__ID__`. If None, a sequential integer column is prepended.

Returns

Arrow table with `__ID__`, `__SOURCE__`, and `__TARGET__` columns.

Raises

`ValueError` – If `source_col` or `target_col` are missing from table.

Return type
Table

`pycypher.ingestion.arrow_utils.infer_attribute_map(table)`

Return `{col: col}` for every non-reserved column in table.

Reserved columns (`__ID__`, `__SOURCE__`, `__TARGET__`) are excluded.

Parameters

`table` (Table) – Arrow table with normalised column names.

Returns

Attribute map suitable for passing to `EntityTable` or `RelationshipTable`.

Return type

`dict[str, str]`

Output Writer

Utilities for writing query result DataFrames to output sinks.

`write_dataframe_to_uri()` is the single entry point: it accepts a `pd.DataFrame`, a destination URI, and an optional explicit `OutputFormat`, then writes the file in the appropriate format.

Supported URI forms

- Bare filesystem paths: `/path/to/output.csv`
- `file://` URIs: `file:///path/to/output.csv`

Cloud URIs (`s3://`, `gs://`, `https://`, ...) are not currently supported and raise `NotImplementedError`. Add cloud support by integrating `pyarrow.fs` or `fsspec` as a separate enhancement.

Parent directories are created automatically when they do not exist.

`pycypher.ingestion.output_writer.write_dataframe_to_uri(df, uri, fmt=None)`

Write `df` to `uri` in the appropriate format.

Format is determined by `fmt` when provided; otherwise it is inferred from the file extension of `uri`.

Parameters

- `df` (`pd.DataFrame`) – The `pandas.DataFrame` to serialise.
- `uri` (`str`) – Destination URI. Bare paths and `file://` URIs are accepted. Cloud URIs raise `NotImplementedError`.
- `fmt` (`OutputFormat` | `None`) – Explicit `OutputFormat`. When `None`, the format is inferred from the URI extension.

Raises

- `NotImplementedError` – If `uri` uses a cloud URI scheme (`s3://`, `gs://`, `https://`, ...).
- `ValueError` – If `fmt` is `None` and the URI's extension is not `.csv`, `.parquet`, or `.json`.

Security

Security utilities for input validation and sanitization.

This module provides security functions to prevent SQL injection, path traversal, and other security vulnerabilities in data source operations.

`pycypher.ingestion.security.SECURITY_LOGGER = <Logger pycypher.security (WARNING)>`

Dedicated security event logger — separate from the query audit logger so that security events can be routed to a SIEM or alerting pipeline independently.

`pycypher.ingestion.security.is_security_log_enabled()`

Return whether the security event logger has any active handlers.

`pycypher.ingestion.security.enable_security_log(*, level=30)`

Programmatically enable security event logging to stderr.

Safe to call multiple times — a handler is added only once.

Parameters

`level` (`int`) – Logging level for the security logger (default `WARNING`).

`pycypher.ingestion.security.security_event_log(*, event_type, input_sample=None, source_function=None, detail=None)`

Log a security event for incident response.

Emits a structured JSON record to the `pycypher.security` logger whenever a `SecurityError` is raised. Records include enough context for incident triage without leaking sensitive data.

The logger is off by default and activated by setting the `PYCYPHER_SECURITY_LOG` environment variable to 1, true, or yes, or by calling `enable_security_log()`.

Parameters

- `event_type` (`str`) – Category of the blocked threat (e.g. `"sql_injection"`, `"path_traversal"`, `"ssrf"`).
- `input_sample` (`str` | `None`) – The offending input, truncated for safety. Parameter values should never be passed here — use structural descriptions instead.
- `source_function` (`str` | `None`) – The validation function that caught the threat.
- `detail` (`str` | `None`) – Optional free-text detail (e.g. `matched pattern name`).

`pycypher.ingestion.security.sanitize_file_path(path)`

Sanitize a file path to prevent path traversal attacks.

Parameters

path ([str](#)) – The file path to sanitize

Returns

Sanitized file path

Raises

[SecurityError](#) – If the path contains dangerous elements

Return type

[str](#)

`pycypher.ingestion.security.validate__sql__query(query)`

Validate a SQL query using an allowlist approach.

Only SELECT and WITH ... SELECT queries are permitted. The query is first stripped of comments (which are a common injection vector) and then checked against a strict allowlist of permitted statement prefixes. Multiple statements are rejected.

Parameters

query ([str](#)) – SQL query to validate.

Raises

[SecurityError](#) – If the query is empty, contains comments, multiple statements, or is not an allowed statement type.

`pycypher.ingestion.security.sanitize__sql__identifier(identifier)`

Sanitize SQL identifiers (table names, column names) to prevent injection.

Parameters

identifier ([str](#)) – SQL identifier to sanitize

Returns

Sanitized identifier

Raises

[SecurityError](#) – If the identifier is invalid

Return type

[str](#)

`pycypher.ingestion.security.validate__uri__scheme(uri)`

Validate URI scheme for data sources.

Parameters

uri ([str](#)) – URI to validate

Returns

The validated URI

Raises

[SecurityError](#) – If the URI scheme is dangerous

Return type

[str](#)

`pycypher.ingestion.security.escape__sql__string__literal(value)`

Escape a string value for safe inclusion in SQL as a string literal.

Parameters

value ([str](#)) – String value to escape

Returns

Escaped string safe for SQL inclusion

Return type

`str`

`pycypher.ingestion.security.parameterize_duckdb_query(template, **params)`

Create a parameterized DuckDB query with proper escaping.

Parameters

- `template` (`str`) – SQL template with {param} placeholders
- `**params` (`str`) – Named parameters to substitute

Returns

SQL with safely escaped parameters

Raises

`SecurityError` – If template or parameters are invalid

Return type

`str`

`pycypher.ingestion.security.mask_uri_credentials(uri)`

Mask credentials in a URI for safe display in logs and console output.

Replaces the password component of URIs like `postgresql://user:s3cret@host/db` with `***` to prevent credential leakage in logs, CLI output, and error messages.

Parameters

`uri` (`str`) – The URI string to mask.

Returns

The URI with any password replaced by `***`. If the URI has no credentials or cannot be parsed, returns the original string unchanged.

Return type

`str`

4.1.10 Sinks

Neo4j Sink

Write pycypher query results to a Neo4j graph database using idempotent MERGE semantics. Requires the neo4j driver (included in `uv sync --group dev`):

```
uv pip install neo4j
```

Neo4j sink: write pycypher query results to a Neo4j graph database.

All writes use MERGE semantics, making every call idempotent — safe to re-run as part of a recurring ETL pipeline. Relationship endpoint nodes are located with MATCH, so nodes must be written before relationships.

Quick start:

```
import pandas as pd
from pycypher import ContextBuilder, Star
from pycypher.sinks.neo4j import Neo4jSink, NodeMapping, RelationshipMapping
```

(continues on next page)

(continued from previous page)

```

context = ContextBuilder().add_entity("Person", persons_df).build()
result = Star(context=context).execute_query(
    "MATCH (p:Person) RETURN p.__ID__ AS pid, p.name AS name"
)

with Neo4jSink("bolt://localhost:7687", "neo4j", "pycypher") as sink:
    sink.write_nodes(
        result,
        NodeMapping(
            label="Person",
            id_column="pid",
            property_columns={"name": "name"},
        ),
    )

```

Requires the neo4j package:

```
uv pip install neo4j
```

```
class pycypher.sinks.neo4j.Neo4jSink(uri, user, password, *, database=None, batch_size=500,
                                     encrypted=None)
```

Bases: `object`

Write-only sink that persists pycypher query results to Neo4j.

All writes use MERGE semantics, making every call idempotent. Rows are sent to the database in batches via UNWIND for efficiency.

The sink is a context manager; the underlying driver is closed on exit:

```

with Neo4jSink(uri, user, password) as sink:
    sink.write_nodes(df, node_mapping)
    sink.write_relationships(df, rel_mapping)

```

Parameters

- `uri` (`str`) – Neo4j Bolt URI, e.g. `"bolt://localhost:7687"`.
- `user` (`str`) – Database username.
- `password` (`str`) – Database password.
- `database` (`str` | `None`) – Target database name. `None` uses the server default.
- `batch_size` (`int`) – Number of rows sent per UNWIND transaction. Tune this down if individual rows are very wide.
- `encrypted` (`bool` | `None`) – Explicitly enable (`True`) or disable (`False`) TLS encryption for the driver connection. `None` (default) defers to the Neo4j driver's scheme-based defaults (`bolt://` is unencrypted, `bolt+s://` or `neo4j+s://` are encrypted). Set to `True` for production connections to enforce TLS.

Raises

`ImportError` – If the neo4j driver package is not installed.

```
__init__(uri, user, password, *, database=None, batch_size=500, encrypted=None)
```

`write_nodes(df, mapping)`

Merge rows of `df` as nodes into Neo4j using `mapping`.

Parameters

- `df` ([DataFrame](#)) – DataFrame whose rows represent nodes. Must contain `mapping.id_column` and all columns referenced by `mapping.property_columns.values()`.
- `mapping` ([NodeMapping](#)) – Declares how columns map onto the target label and properties.

Returns

Number of rows successfully written. Rows with a null `id_column` value are skipped and not counted.

Raises

[ValueError](#) – If a required column is absent from `df`.

Return type

[int](#)

`write_relationships(df, mapping)`

Merge rows of `df` as relationships into Neo4j using `mapping`.

Both endpoint nodes are located with `MATCH`, so they must already exist in the database. Write nodes first with `write_nodes()`.

Parameters

- `df` ([DataFrame](#)) – DataFrame whose rows represent relationships. Must contain `mapping.source_id_column`, `mapping.target_id_column`, and all columns referenced by `mapping.property_columns.values()`.
- `mapping` ([RelationshipMapping](#)) – Declares how columns map onto the target relationship type and properties.

Returns

Number of rows successfully written. Rows with null endpoint IDs are skipped.

Raises

[ValueError](#) – If a required column is absent from `df`.

Return type

[int](#)

`close()`

Close the underlying Neo4j driver and release all connections.

`__enter__()`

Return self for use as a context manager.

`__exit__(exc_type, exc_val, exc_tb)`

Close the driver on context-manager exit, even if an error occurred.

```
class pycypher.sinks.neo4j.NodeMapping(*, label, id_column, id_property='id',
                                       property_columns=<factory>)
```

Bases: `BaseModel`

Describes how DataFrame columns map onto a Neo4j node label.

The `id_column` is used as the MERGE key — its value becomes the `id_property` on the node. `property_columns` maps each Neo4j property name to the DataFrame column that supplies its value.

Example:

```
NodeMapping(
    label="Person",
    id_column="pid",
    property_columns={"name": "full_name", "age": "age"},
)
```

label: `str`

id_column: `str`

id_property: `str`

property_columns: `dict[str, str]`

model_config: `ClassVar[ConfigDict] = {}`

Configuration for the model, should be a dictionary conforming to `[ConfigDict][pydantic.config.ConfigDict]`.

```
class pycypher.sinks.neo4j.RelationshipMapping(*, rel_type, source_label, target_label,
                                              source_id_column, target_id_column,
                                              source_id_property='id', target_id_property='id',
                                              property_columns=<factory>)
```

Bases: `BaseModel`

Describes how DataFrame columns map onto a Neo4j relationship type.

Both endpoint nodes are located with MATCH using their respective id columns, so the nodes must already exist in the database.

Example:

```
RelationshipMapping(
    rel_type="KNOWS",
    source_label="Person",
    target_label="Person",
    source_id_column="src_pid",
    target_id_column="tgt_pid",
    property_columns={"since": "since_year"},
)
```

rel_type: `str`

source_label: `str`


```

target_label: str

source_id_column: str

target_id_column: str

source_id_property: str

target_id_property: str

property_columns: dict[str, str]

model_config: ClassVar[ConfigDict] = {}
    Configuration for the model, should be a dictionary conforming to
    [ConfigDict][pydantic.config.ConfigDict].

```

4.1.11 Distributed Execution

Cluster

Coordination protocols, worker registration, query routing, and fault tolerance for enterprise-scale distributed execution. Provides local implementations for testing; network transport is deferred to Phase 3. Distributed execution scaffolding for cluster deployment.

Defines the coordination protocols, worker node registration, query distribution strategies, and fault tolerance patterns needed for enterprise-scale distributed query execution.

This module provides interfaces and local implementations — the actual network transport and distributed scheduling are deferred to Phase 3. The local implementations allow testing the coordination logic without a real cluster.

Architecture

ClusterCoordinator
(registers workers, routes
queries, aggregates metrics)

Worker 1 Worker 2 Worker N
(Star) (Star) (Star)

Each worker wraps a [Star](#) instance. The coordinator selects a worker via a pluggable [QueryRouter](#) strategy (round-robin, least-loaded, hash-based).

Usage:

```

from pycypher.cluster import ClusterCoordinator, LocalWorker

coord = ClusterCoordinator()
coord.register_worker(LocalWorker("w1"))
coord.register_worker(LocalWorker("w2"))

result = coord.execute_query("MATCH (p:Person) RETURN p.name")
health = coord.cluster_health()

```

```

class pycypher.cluster.WorkerStatus(*values)
    Bases: Enum
    Health state of a cluster worker.
    ACTIVE = 'active'
    DRAINING = 'draining'
    UNAVAILABLE = 'unavailable'

class pycypher.cluster.Worker(*args, **kwargs)
    Bases: Protocol
    Protocol that every cluster worker must satisfy.
    A worker wraps a query execution engine and exposes health and metrics information for the coordinator.
    property worker_id: str
        Unique identifier for this worker.
    property status: WorkerStatus
        Current health status.
    execute_query(query, *, parameters=None)
        Execute a Cypher query on this worker.
        Parameters
        • query (str) – Cypher query string.
        • parameters (dict[str, Any] | None) – Optional named parameters.
    Returns
        DataFrame with result rows.
    Raises
        Exception – On query failure.
    Return type
        DataFrame
    health_check()
        Return current health snapshot for this worker.

    __init__(*args, **kwargs)

class pycypher.cluster.WorkerHealth(worker_id, status, queries_executed, errors, avg_latency_ms,
                                     last_heartbeat, active_queries)
    Bases: object
    Point-in-time health snapshot for a single worker.

    worker_id
        Unique worker identifier.
        Type
        str

```

status

Current worker status.

Type

`pycypher.cluster.WorkerStatus`

queries__executed

Total queries handled by this worker.

Type

`int`

errors

Total errors on this worker.

Type

`int`

avg__latency__ms

Rolling average query latency in ms.

Type

`float`

last__heartbeat

Monotonic timestamp of last successful heartbeat.

Type

`float`

active__queries

Number of queries currently executing.

Type

`int`

worker_id: `str`

status: `WorkerStatus`

queries__executed: `int`

errors: `int`

avg__latency__ms: `float`

last__heartbeat: `float`

active__queries: `int`

```
__init__(worker_id, status, queries__executed, errors, avg__latency__ms, last__heartbeat,
         active_queries)
```

```
class pycypher.cluster.ClusterHealth(total__workers, active__workers, unavailable__workers, total__queries,
                                     total__errors, cluster_error_rate, avg__latency__ms, worker__health)
```

Bases: `object`

Aggregate health snapshot for the entire cluster.

`total_workers`
Number of registered workers.
Type
`int`

`active_workers`
Workers in ACTIVE status.
Type
`int`

`unavailable_workers`
Workers in UNAVAILABLE status.
Type
`int`

`total_queries`
Sum of queries across all workers.
Type
`int`

`total_errors`
Sum of errors across all workers.
Type
`int`

`cluster_error_rate`
Aggregate error rate (0.0–1.0).
Type
`float`

`avg_latency_ms`
Weighted average latency across workers.
Type
`float`

`worker_health`
Per-worker health snapshots.
Type
`list[pycypher.cluster.WorkerHealth]`

`total_workers: int`

`active_workers: int`

`unavailable_workers: int`

`total_queries: int`

`total_errors: int`

`cluster_error_rate: float`

`avg_latency_ms: float`

worker_health: list[WorkerHealth]

__init__(total_workers, active_workers, unavailable_workers, total_queries, total_errors, cluster_error_rate, avg_latency_ms, worker_health)

class pycypher.cluster.QueryRouter(*args, **kwargs)

Bases: Protocol

Strategy for selecting which worker handles a query.

select_worker(workers, query)

Choose a worker for the given query.

Parameters

- workers (list[Worker]) – List of active workers to choose from.
- query (str) – The Cypher query string (for hash-based routing).

Returns

The selected worker.

Raises

RuntimeError – If no suitable worker is available.

Return type

Worker

__init__(*args, **kwargs)

class pycypher.cluster.RoundRobinRouter

Bases: object

Distributes queries evenly across workers in order.

Thread-safe via atomic counter increment.

__init__()

Initialise the router with a zero counter and a threading lock.

select_worker(workers, query)

Select next worker in round-robin order.

Parameters

- workers (list[Worker]) – Active workers.
- query (str) – Query string (unused for round-robin).

Returns

The next worker in rotation.

Raises

RuntimeError – If workers list is empty.

Return type

Worker

class pycypher.cluster.LeastLoadedRouter

Bases: [object](#)

Routes queries to the worker with fewest active queries.

Falls back to round-robin when all workers have equal load.

select_worker(workers, query)

Select the least-loaded worker.

Parameters

- workers ([list](#)[[Worker](#)]) – Active workers.
- query ([str](#)) – Query string (unused).

Returns

The worker with lowest active_queries count.

Raises

[RuntimeError](#) – If workers list is empty.

Return type

[Worker](#)

class pycypher.cluster.LocalWorker(worker_id, star=None)

Bases: [object](#)

In-process worker that wraps a Star instance.

Provides the [Worker](#) protocol for local testing and single-node deployment.

Parameters

- worker_id ([str](#)) – Unique identifier for this worker.
- star ([Any](#) | [None](#)) – Optional pre-configured Star instance. If [None](#), a default Star is created.

__init__(worker_id, star=None)

Create a local in-process worker.

Parameters

- worker_id ([str](#)) – Unique identifier for this worker.
- star ([Any](#) | [None](#)) – Optional pre-configured [Star](#) instance. If [None](#), a default Star is created lazily.

property worker_id: [str](#)

Unique identifier for this worker.

property status: [WorkerStatus](#)

Current health status.

execute_query(query, *, parameters=None)

Execute a query on the wrapped Star instance.

Parameters

- query ([str](#)) – Cypher query string.
- parameters ([dict](#)[[str](#), [Any](#)] | [None](#)) – Optional named parameters.

Returns
DataFrame with result rows.

Return type
DataFrame

health_check()

Return current health snapshot.

Returns
WorkerHealth with current counters and latency.

Return type
WorkerHealth

```
class pycypher.cluster.ClusterCoordinator(router=<factory>, heartbeat_timeout_s=30.0,
                                          _workers=<factory>, _lock=<factory>)
```

Bases: object

Central coordinator for distributed query execution.

Manages worker registration, query routing, and aggregate health monitoring. Thread-safe for concurrent query submission.

Parameters

- router ([QueryRouter](#)) – Query routing strategy. Defaults to [RoundRobinRouter](#).
- heartbeat_timeout_s ([float](#)) – Seconds after which a worker with no heartbeat is marked unavailable.

router: [QueryRouter](#)

heartbeat_timeout_s: [float](#) = 30.0

register_worker(worker)

Register a worker with the cluster.

Parameters
worker ([Worker](#)) – A Worker instance to add.

Raises
[ValueError](#) – If a worker with the same ID is already registered.

deregister_worker(worker_id)

Remove a worker from the cluster.

Parameters
worker_id ([str](#)) – ID of the worker to remove.

Raises
[ValueError](#) – If worker_id is not registered.

execute_query(query, *, parameters=None)

Route and execute a query on a selected worker.

Parameters

- query ([str](#)) – Cypher query string.
- parameters ([dict](#)[[str](#), [Any](#)] | None) – Optional named parameters.

Returns
DataFrame with result rows.

Raises

`RuntimeError` – If no active workers are available.

Return type

`DataFrame`

`cluster_health()`

Return aggregate health snapshot for the cluster.

Returns

`ClusterHealth` with per-worker and aggregate metrics.

Return type

`ClusterHealth`

`__init__(router=<factory>, heartbeat_timeout_s=30.0, _workers=<factory>, _lock=<factory>)`

property `worker_count`: `int`

Number of registered workers.

4.1.12 CLI

NMETL CLI

Command-line interface for running Cypher queries against YAML-configured data pipelines. Command-line interface for the nmetl ETL pipeline runner.

nmetl is the command-line entry point for running PyCypher-based ETL pipelines defined in a YAML configuration file. It loads data sources, executes Cypher queries, and writes results to configured output sinks.

Example usage:

```
# Run a pipeline from a config file
nmetl run pipeline.yaml

# Validate a config file without running
nmetl validate pipeline.yaml

# List all queries defined in the config
nmetl list-queries pipeline.yaml

# Dry-run: show what would be executed
nmetl run pipeline.yaml --dry-run
```

`pycypher.nmetl_cli.main()`

Entry point for the nmetl command-line tool.

4.1.13 Security & Observability

Audit Logging

Opt-in structured query audit logging. Emits one JSON record per query execution with `query_id`, `timing`, `status`, and `row counts`. Activated via `PYCYPHER_AUDIT_LOG` environment variable. Opt-in structured query audit logging.

Provides a dedicated `pycypher.audit` logger that emits one JSON record per query execution (success or failure). The audit log is off by default and activated by setting the `PYCYPHER_AUDIT_LOG` environment variable to 1, true, or yes.

When enabled, a `logging.StreamHandler` writing to `stderr` is attached automatically. For production use, configure the `pycypher.audit` logger via `logging.config` or `dictConfig` to route records to a file, syslog, or log aggregator.

Each record contains:

- `query_id` — unique correlation ID (hex string).
- `timestamp` — ISO-8601 UTC timestamp.
- `query` — the query text (truncated to `max_query_length`).
- `status` — "ok" or "error".
- `elapsed_ms` — wall-clock execution time in milliseconds.
- `rows` — number of result rows (success only).
- `error_type` — exception class name (failure only).
- `parameter_keys` — list of parameter names (never values).

Security note: parameter values and full result data are never logged. Query text is truncated to prevent unbounded log growth.

Usage:

```
export PYCYPHER_AUDIT_LOG=1
# Queries are now logged to stderr via pycypher.audit logger.
```

Programmatic activation:

```
from pycypher.audit import enable_audit_log
enable_audit_log()
```

```
pycypher.audit.AUDIT_LOGGER = <Logger pycypher.audit (WARNING)>
```

Dedicated audit logger — separate from the application logger so that users can route audit records independently.

```
pycypher.audit.audit_query_success(*, query_id, query, elapsed_s, rows, parameter_keys=None,
                                   cached=False)
```

Log a successful query execution.

Parameters

- `query_id` (`str`) — Unique correlation ID.
- `query` (`str`) — The Cypher query text.
- `elapsed_s` (`float`) — Wall-clock execution time in seconds.
- `rows` (`int`) — Number of result rows.
- `parameter_keys` (`list[str]` | `None`) — Parameter names (values are never logged).
- `cached` (`bool`) — Whether the result was served from cache.

```
pycypher.audit.audit_query_error(*, query_id, query, elapsed_s, error_type, parameter_keys=None)
```

Log a failed query execution.

Parameters

- `query_id` (`str`) – Unique correlation ID.
- `query` (`str`) – The Cypher query text.
- `elapsed_s` (`float`) – Wall-clock execution time before failure.
- `error_type` (`str`) – Exception class name.
- `parameter_keys` (`list[str] | None`) – Parameter names (values are never logged).

`pycypher.audit.audit_mutation(*, query_id, operation, entity_type, affected_count, elapsed_s, details=None)`

Log a mutation operation (CREATE, SET, DELETE, MERGE, REMOVE).

Emits a structured JSON record for each mutation clause execution, enabling audit trails for all write operations.

Parameters

- `query_id` (`str`) – Unique correlation ID linking to the parent query.
- `operation` (`str`) – Mutation type ("CREATE", "SET", "DELETE", "DETACH_DELETE", "MERGE", "REMOVE").
- `entity_type` (`str`) – The entity or relationship type affected.
- `affected_count` (`int`) – Number of rows/entities affected.
- `elapsed_s` (`float`) – Wall-clock execution time in seconds.
- `details` (`dict[str, Any] | None`) – Optional extra context (e.g. property keys modified, detached relationship count). Values must be JSON-serializable.

Security note: entity data (property values, IDs) is never logged. Only structural metadata (counts, types, property names) is recorded.

`pycypher.audit.enable_audit_log(*, level=20)`

Programmatically enable audit logging to stderr.

Safe to call multiple times — a handler is added only once.

Parameters

- `level` (`int`) – Logging level for the audit logger (default INFO).

`pycypher.audit.is_audit_enabled()`

Return whether the audit logger has any active handlers.

Rate Limiter

Thread-safe token-bucket rate limiter for resource protection in multi-tenant deployments. Configured via `PYCYPHER_RATE_LIMIT_QPS` and `PYCYPHER_RATE_LIMIT_BURST`. Query rate limiting for resource protection in multi-tenant deployments.

Provides a thread-safe token-bucket rate limiter that can be applied per-session or per-caller to prevent query abuse. Integrates with the existing audit logging and configuration infrastructure.

Configuration is via environment variables:

`PYCYPHER_RATE_LIMIT_QPS`

Maximum sustained queries per second (token refill rate). 0 (default) disables rate limiting entirely.

`PYCYPHER_RATE_LIMIT_BURST`

Maximum burst size (bucket capacity). Allows short bursts above the sustained rate. Defaults to 10.

Usage:

```
from pycypher.rate_limiter import get_global_limiter

limiter = get_global_limiter()
limiter.acquire() # raises RateLimitError if over limit
```

Programmatic usage with custom settings:

```
limiter = QueryRateLimiter(qps=5.0, burst=20)
limiter.acquire(caller_id="user-123")
```

```
class pycypher.rate_limiter.QueryRateLimiter(qps=0.0, burst=10, *, per_caller=False)
```

Bases: `object`

Configurable query rate limiter with optional per-caller tracking.

When `qps=0` (the default from environment), the limiter is disabled and `acquire()` is a no-op.

Parameters

- `qps` (`float`) – Sustained queries per second (token refill rate). 0 disables rate limiting.
- `burst` (`int`) – Maximum burst size (token bucket capacity).
- `per_caller` (`bool`) – If True, maintain separate buckets per `caller_id`. If False (default), use a single shared bucket.

```
__init__(qps=0.0, burst=10, *, per_caller=False)
```

property enabled: `bool`

Whether rate limiting is active.

property qps: `float`

Configured queries-per-second rate.

property burst: `int`

Configured burst capacity.

`acquire(*, caller_id=None)`

Acquire permission to execute a query.

Parameters

`caller_id` (`str` | `None`) – Optional caller identifier for per-caller limiting. Ignored when `per_caller=False`.

Raises

`RateLimitError` – If the rate limit is exceeded.

`try_acquire(*, caller_id=None)`

Non-raising variant of `acquire()`.

Returns

True if the query is allowed, False if rate-limited.

Return type

`bool`

```
reset(*, caller_id=None)
```

Reset the bucket(s) to full capacity.

Parameters

`caller_id` (`str` | `None`) – If given and `per_caller` is enabled, reset only that caller's bucket. Otherwise reset the shared bucket.

```
pycypher.rate_limiter.get_global_limiter()
```

Return the process-wide rate limiter (lazily created from config).

The limiter is configured from: - `PYCYPHER_RATE_LIMIT_QPS` (default 0 = disabled) - `PYCYPHER_RATE_LIMIT_BURST` (default 10)

```
pycypher.rate_limiter.reset_global_limiter()
```

Reset the global limiter singleton (primarily for testing).

4.1.14 Utilities

Types

Type aliases and type definitions used across the codebase. Backend-agnostic type aliases for the pycypher query engine.

This module defines type aliases that decouple the evaluator stack from concrete pandas types. Today they resolve to pandas types (zero overhead); tomorrow they can be swapped to protocol-based abstractions that support Dask, Polars, or DuckDB backends.

Usage:

```
from pycypher.cypher_types import FrameSeries, FrameDataFrame

def evaluate(self, expr: Expression) -> FrameSeries:
    ...
```

The aliases live here — not in `backend_engine.py` — to avoid circular imports (evaluators import types; `backend_engine` imports `relational_models` which evaluators also need).

```
pycypher.cypher_types.FrameSeries
```

A 1-D array of values aligned with a `BindingFrame`.

Today: literally `pd.Series`. Future: a protocol that `pd.Series`, `dask Series`, and `Polars Series` satisfy.

```
pycypher.cypher_types.FrameDataFrame
```

A 2-D table of bindings or results.

Today: literally `pd.DataFrame`. Future: a protocol that `pd.DataFrame`, `dask DataFrame`, and `Polars DataFrame` satisfy.

```
pycypher.cypher_types.BackendFrame
```

Opaque handle to a backend-specific `DataFrame`.

Used in the `BackendEngine` protocol and `InstrumentedBackend` wrapper to annotate frame parameters. Each concrete backend returns its own type (`pd.DataFrame` for pandas, `DuckDBLazyFrame` for DuckDB, etc.) but callers of the protocol treat frames as opaque handles — they are only passed back into backend methods or materialised via `to_pandas()`.

Today: Any (since the protocol is heterogeneous across backends). Future: a TypeVar bound to a Frame protocol once all backends conform to a structural frame interface.

`pycypher.cypher_types.BackendMask`

Opaque handle to a backend-specific boolean mask.

Used in `BackendEngine.filter()` to annotate the mask parameter. Today `pd.Series[bool]` for pandas, but other backends may use their own mask representations.

`pycypher.cypher_types.ColumnValues`

Values for a new or replaced column in a backend frame.

Passed to `BackendEngine.assign_column()`. May be a Series, list, scalar, or backend-specific column representation.

`pycypher.cypher_types.SourceObject`

Raw data source passed to `BackendEngine.scan_entity()`.

Typically a `pd.DataFrame`, `pa.Table`, or file path, depending on ingestion context.

Query Learning — ML-Powered Optimization

Adaptive query optimization using online learning with exponential moving averages. No heavyweight ML dependencies. Machine-learning feedback loops for adaptive query optimization.

This module is the public API for the query learning subsystem. It provides a unified facade (`QueryLearningStore`) and re-exports all components from the two implementation modules:

- `pycypher.feedback_collector` — data collection (fingerprinting, selectivity tracking, join performance tracking, plan versioning)
- `pycypher.learning_algorithm` — decision-making (adaptive eviction, plan caching, semantic result caching, distributed sync)

All existing from `pycypher.query_learning` import X statements continue to work unchanged.

Usage:

```
store = QueryLearningStore()

# Before execution: get learned plan adjustments
fingerprint = store.fingerprint(query_ast)
cached_plan = store.get_cached_plan(fingerprint)
selectivity = store.get_learned_selectivity("Person", "age", ">")

# After execution: record feedback
store.record_selectivity("Person", "age", ">", estimated=0.33, actual=0.12)
store.record_join_performance(
    strategy=JoinStrategy.HASH,
    left_rows=10000, right_rows=500,
    actual_rows=450, elapsed_ms=12.3,
)
store.cache_plan(fingerprint, plan)
```

```
class pycypher.query_learning.AdaptiveEvictionPolicy(*, frequency_weight=0.6, recency_weight=0.4,
                                                    decay_half_life_s=60.0)
```

Bases: `object`

Frequency-recency adaptive cache eviction policy.

Combines access frequency and recency into a single eviction score. Higher score = more valuable = less likely to be evicted.

Score = frequency_weight * log(access_count + 1) + recency_weight * recency_score

Where recency_score decays exponentially with time since last access.

```
__init__(*, frequency_weight=0.6, recency_weight=0.4, decay_half_life_s=60.0)
```

```
clear()
```

Reset all tracking data.

```
eviction_score(key)
```

Compute eviction score for a key. Higher = more valuable.

```
record_access(key)
```

Record an access for the given cache key.

```
remove(key)
```

Remove tracking data for an evicted key.

```
select_eviction_candidate(keys)
```

Select the best candidate for eviction (lowest score).

```
class pycypher.query_learning.AdaptivePlanCache(*, max_entries=256, ttl_seconds=300.0,
                                                eviction_policy='lru')
```

Bases: `object`

Caches query analysis results keyed by structural fingerprint.

Structurally similar queries (same clause structure, entity types, predicate shapes) reuse cached plans, avoiding repeated analysis.

```
__init__(*, max_entries=256, ttl_seconds=300.0, eviction_policy='lru')
```

```
clear()
```

Drop all cached plans and reset stats.

```
get(fingerprint)
```

Look up a cached plan by fingerprint. Returns None on miss.

`invalidate()`

Clear the entire plan cache (e.g. after mutations).

`put(fingerprint, analysis)`

Cache an analysis result for a fingerprint.

property stats: `dict[str, Any]`

Return cache statistics.

`class pycypher.query_learning.DistributedLearningSync(store)`

Bases: `object`

Serializes and deserializes learning state for multi-instance sharing.

Allows multiple pycypher instances to share learned query optimization knowledge (selectivity patterns, join performance) without requiring shared memory or a database.

State is versioned to support conflict resolution during merges.

`__init__(store)`

Initialize with a `QueryLearningStore` instance.

Parameters

`store (Any)` – A `QueryLearningStore` (typed as `Any` to avoid circular import).

`export_state()`

Export current learning state as a serializable dict.

`import_state(state)`

Import learning state from another instance.

Returns `True` if import was applied, `False` if state was stale.

`class pycypher.query_learning.JoinPerformanceTracker`

Bases: `object`

Tracks join strategy performance for adaptive strategy selection.

Records execution time and output accuracy per `(size_bucket_left, size_bucket_right, strategy)` triple. Over time, this allows the planner to prefer strategies that performed well historically for similar input sizes.

`__init__()`

`best_strategy(left_rows, right_rows)`

Return the historically best-performing strategy for this size pair.

Returns `None` when insufficient data is available (`< _MIN_OBSERVATIONS` for any strategy in this bucket).

`clear()`

Drop all history.

`record(*, strategy, left_rows, right_rows, actual_output_rows, elapsed_ms)`

Record a join execution observation.

`strategy_stats(left_rows, right_rows)`

Return per-strategy statistics for a size bucket.

Returns a dict of strategy -> {avg_ms, count, output_accuracy}.

`class pycypher.query_learning.PlanVersionTracker`

Bases: `object`

Tracks plan evolution and effectiveness metrics per query fingerprint.

Each time a new plan is generated for a structurally similar query, the version is incremented. Execution metrics are recorded per version to detect regressions and identify the best-performing plan.

`__init__()`

`best_version(fingerprint)`

Return the version number of the best-performing plan.

“Best” = lowest average execution time among versions with data. Returns None if no versions have execution metrics.

`clear()`

Drop all version history.

`detect_regression(fingerprint, version)`

Detect if a plan version is a regression vs the previous best.

Returns True if the given version’s average execution time is worse than the best previous version by >10%.

`get_history(fingerprint)`

Get full version history for a fingerprint.

`get_version_metrics(fingerprint, version)`

Get execution metrics for a specific plan version.

`record_execution(fingerprint, version, *, elapsed_ms, rows_produced)`

Record execution metrics for a specific plan version.

`record_plan(fingerprint, analysis)`

Record a new plan version. Returns the version number.

`class pycypher.query_learning.PredicateSelectivityTracker`

Bases: `object`

Learns actual predicate selectivity from execution feedback.

Tracks selectivity per (entity_type, property, operator) triple using an exponential moving average for fast convergence with recency bias.

`__init__()`

`clear()`

Drop all history.

`correction_factor(entity_type, prop, operator, *, heuristic)`

Return a multiplicative correction to apply to a heuristic estimate.

If learned selectivity is 0.12 and heuristic is 0.33, returns $0.12 / 0.33 \sim 0.36$ so the caller can multiply their estimate.

Returns 1.0 when insufficient data is available.

`estimate_compound_selectivity(entity_type, predicates, combinator='AND')`

Estimate compound selectivity for multiple predicates.

Uses independence assumption (adjusted by learned correlation when available) to combine individual selectivities.

Parameters

- `entity_type` (`str`) – The entity type (e.g. “Person”).
- `predicates` (`list[tuple[str, str]]`) – List of (property, operator) pairs.
- `combinator` (`str`) – “AND” or “OR”.

Return type

Estimated compound selectivity, or None if any predicate lacks data.

`get_correlation_factor(entity_type, pred_a, pred_b)`

Get learned correlation factor between two predicates.

Returns a multiplicative factor: < 1.0 means positive correlation (AND selects fewer than independence), > 1.0 means negative correlation. Returns None if insufficient data.

`get_learned_selectivity(entity_type, prop, operator)`

Return the learned selectivity, or None if insufficient data.

Returns None when fewer than `__MIN_OBSERVATIONS` have been recorded, allowing the caller to fall back to heuristic defaults.

`record(entity_type, prop, operator, *, estimated, actual)`

Record an observed selectivity for a predicate pattern.

`record_compound(entity_type, predicates, *, estimated_compound, actual_compound)`

Record observed compound selectivity to learn correlations.

Compares the actual compound selectivity against what independence would predict, storing the ratio as a correlation factor.

`property tracked_patterns: list[tuple[str, str, str]]`

Return all tracked (entity_type, property, operator) triples.

`class pycypher.query_learning.QueryFingerprint(digest, clause_signature, entity_types, relationship_types)`

Bases: `object`

Structural fingerprint of a Cypher query for similarity detection.

Two queries with the same fingerprint have identical clause structure, entity/relationship types, and predicate shapes — differing only in literal values.

`digest`

SHA-256 hex digest of the structural representation.

Type

`str`

`clause_signature`

Human-readable clause type sequence.

Type

`str`

`entity_types`

Sorted entity types referenced.

Type

`tuple[str, ...]`

`relationship_types`

Sorted relationship types referenced.

Type

`tuple[str, ...]`

`__init__(digest, clause_signature, entity_types, relationship_types)`

digest: str

clause_signature: str

entity_types: tuple[str, ...]

relationship_types: tuple[str, ...]

class pycypher.query_learning.QueryFingerprinter

Bases: object

Produces structural fingerprints for query similarity detection.

Strips literal values and extracts the query's structural skeleton so that MATCH (p:Person) WHERE p.age > 30 and MATCH (p:Person) WHERE p.age > 50 produce the same fingerprint.

fingerprint(query)

 Compute a structural fingerprint for query.

class pycypher.query_learning.QueryLearningStore

Bases: object

Unified facade coordinating all query learning components.

Provides a single entry point for the query planner and executor to: - Fingerprint queries for plan reuse - Get/record learned predicate selectivities - Get/record join strategy performance - Cache and retrieve analysis results

__init__()

fingerprint(query)

 Compute a structural fingerprint for query.

get_cached_plan(fingerprint)

 Look up a cached analysis for this fingerprint.

cache_plan(fingerprint, analysis)

 Cache an analysis result for future reuse.

record_selectivity(entity_type, prop, operator, *, estimated, actual)

 Record observed predicate selectivity.

get_learned_selectivity(entity_type, prop, operator)

 Get learned selectivity, or None if insufficient data.

`record_join_performance(*, strategy, left_rows, right_rows, actual_output_rows, elapsed_ms)`
Record join execution performance.

`get_best_join_strategy(left_rows, right_rows)`
Get historically best strategy for this input size pair.

`estimate_compound_selectivity(entity_type, predicates, combinator='AND')`
Estimate compound selectivity for multiple predicates.

`record_plan_version(fingerprint, analysis)`
Record a new plan version and return its version number.

`record_plan_execution(fingerprint, version, *, elapsed_ms, rows_produced)`
Record execution metrics for a plan version.

`get_plan_metrics(fingerprint, version)`
Get execution metrics for a plan version.

`invalidate_on_mutation()`
Invalidate plan cache after data mutations.

`diagnostics()`
Return a diagnostic snapshot of all learning components.

`clear()`
Reset all learning state.

`class pycypher.query_learning.SemanticResultCache(*, max_entries=256)`

Bases: `object`

Cache that shares results between structurally similar queries.

For parameter-independent queries (aggregations, full scans), results are shared across all queries with the same structural fingerprint. For parameter-dependent queries, only exact query+params matches are returned.

This provides higher hit rates for workloads with repeated query patterns that differ only in literal values.

`__init__(*, max_entries=256)`

`get(query, parameters, fingerprint_digest)`

Look up a cached result. Tries exact match first, then semantic.

`invalidate()`

Clear all cached results.

`put(query, parameters, fingerprint_digest, result, *, is__parameter__independent=False)`

Cache a query result.

property stats: `dict[str, Any]`

Return cache statistics.

`pycypher.query_learning.get_learning_store()`

Return the module-level singleton `QueryLearningStore`.

Exceptions

Complete exception hierarchy. All exceptions are importable from the top-level `pycypher` package. Custom exceptions for the PyCypher package.

This package defines custom exception classes used throughout the PyCypher package for handling specific error conditions during query parsing and execution.

The module is split into three submodules for maintainability:

- `pycypher.exceptions.base` — Infrastructure (docs links, environment detection, sanitization)
- `pycypher.exceptions.parse` — Parse-time errors (syntax, AST conversion)
- `pycypher.exceptions.runtime` — Runtime errors (type, variable, function, resource, security)

class `pycypher.exceptions.DocsLink(url, exception_name)`

Bases: `object`

Structured documentation link attached to an exception.

`__init__(url, exception_name)`

`as_html()`

Format as an HTML anchor for Jupyter/IPython display.

`as_plain()`

Format as a plain-text URL (fallback for basic terminals).

`as_terminal()`

Format as a clickable OSC 8 hyperlink for modern terminals.

url: `str`

exception_name: `str`

`pycypher.exceptions.sanitize_error_message(exc)`

Return a user-safe error string from `exc`.

Strips internal file paths and traceback fragments that could leak implementation details. Masks URI credentials (passwords) to prevent credential exposure in error output. The exception type name is preserved so the user can still identify the category of error.

exception `pycypher.exceptions.ASTConversionError(message, query_fragment="", node_type=")`

Bases: `ValueError`

Exception raised when AST conversion from grammar parse tree fails.

This exception is thrown when the grammar parser successfully parses a query but the conversion to typed AST models fails, typically due to grammar/AST model synchronization issues or unsupported syntax constructs.

query_fragment

The portion of the query that failed conversion.

node_type

The AST node type that could not be converted.

`__init__(message, query_fragment="", node_type=")`

Initialize with error message and optional context.

Parameters

- `message` (`str`) – Human-readable description of the conversion failure.
- `query_fragment` (`str`) – The query text that triggered the error.
- `node_type` (`str`) – The AST node type that failed conversion.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.CypherSyntaxError(query, original)`

Bases: `SyntaxError`

User-friendly wrapper around Lark parse errors.

Wraps low-level Lark `UnexpectedInput` exceptions into a message that shows the offending line, column position, and what the parser expected, without leaking internal terminal names like `__ANON_0`.

Includes keyword typo detection: when the token at the error position is close to a known Cypher keyword, a “Did you mean ...?” suggestion is appended automatically.

query

The original query string.

line

1-based line number of the error.

column

1-based column number of the error.

keyword_suggestion

Suggested keyword if a typo was detected, or "".

__init__(query, original)

Initialize from an original Lark exception.

Parameters

- query ([str](#)) – The Cypher query that failed to parse.
- original ([Exception](#)) – The Lark exception that triggered this error.

__repr__()

Return a repr exposing structured attributes for REPL inspection.

```
exception pycypher.exceptions.GrammarTransformerSyncError(message, missing_node_type="",
                                                            query_fragment="")
```

Bases: [ASTConversionError](#)

Exception raised when grammar transformer and AST models are out of sync.

This is a specialized [ASTConversionError](#) for cases where the grammar parser produces a node type that has no corresponding AST model class.

missing_node_type

The node type that lacks a corresponding AST class.

__init__(message, missing_node_type="", query_fragment="")

Initialize with sync error details.

Parameters

- message ([str](#)) – Description of the synchronization issue.
- missing_node_type ([str](#)) – The node type missing from AST models.
- query_fragment ([str](#)) – The query text that triggered the error.

__repr__()

Return a repr exposing structured attributes for REPL inspection.

```
exception pycypher.exceptions.CacheLockTimeoutError(*, timeout_seconds, operation)
```

Bases: [TimeoutError](#)

Raised when a cache lock cannot be acquired within the timeout.

This prevents deadlocks in the result cache from hanging query execution indefinitely. When raised, the cache operation is skipped and query execution proceeds without caching.

`timeout__seconds`

The lock acquisition timeout that was exceeded.

`operation`

The cache operation that timed out (e.g. “get”, “put”).

`__init__(*, timeout__seconds, operation)`

exception `pycypher.exceptions.CyclicDependencyError(remaining__nodes, message=)`

Bases: `ValueError`

Raised when a circular dependency is detected in a multi-query graph.

This is a `ValueError` subclass so that existing code catching `ValueError` continues to work.

`remaining__nodes`

The set of node IDs involved in the cycle.

`__init__(remaining__nodes, message=)`

Initialise with the nodes involved in the cycle.

Parameters

- `remaining__nodes` (`set[str]`) – Node IDs that could not be topologically sorted.
- `message` (`str`) – Optional detail message; if omitted a default is used.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.FunctionArgumentError(function__name, expected__args, actual__args, argument__description=)`

Bases: `ValueError`

Exception raised when a function is called with wrong number of arguments.

This exception is thrown when a function call has an incorrect number of arguments or the arguments don't match the expected signature.

`function__name`

Name of the function with argument error.

`expected__args`

Number of expected arguments.

`actual__args`

Number of actual arguments provided.

`argument__description`

Description of expected arguments.

`__init__`(function_name, expected_args, actual_args, argument_description=)

Initialize with function argument mismatch details.

Parameters

- function_name (str) – Name of the function with wrong arguments.
- expected_args (int) – Expected number of arguments.
- actual_args (int) – Actual number of arguments provided.
- argument_description (str) – Description of what arguments are expected.

`__repr__`()

Return a repr exposing structured attributes for REPL inspection.

exception pycypher.exceptions.GraphTypeNotFoundError(type_name, message="", available_types=None)

Bases: `ValueError`

Raised when a node label or relationship type is not registered in the context.

This is a `ValueError` subclass so that existing code catching `ValueError` continues to work. New code that only wants to handle the “entity/relationship type absent from context” case (e.g. `MERGE`’s create fallback) can catch this specific subclass without accidentally swallowing unrelated errors.

type_name

The label or relationship type that was not found.

Example:

```
try:
    EntityScan("Ghost", "g").scan(context)
except GraphTypeNotFoundError as exc:
    print(f"Type not registered: {exc.type_name}")
```

`__init__`(type_name, message="", available_types=None)

Initialise with the missing type name and an optional detail message.

Parameters

- type_name (str) – The label or relationship type that was absent.
- message (str) – Optional additional detail; if omitted a default is used.
- available_types (list[str] | None) – Known types in the context, shown as a hint.

`__repr__`()

Return a repr exposing structured attributes for REPL inspection.

exception pycypher.exceptions.IncompatibleOperatorError(operator, left_type, right_type, suggestion=)

Bases: `TypeError`

Exception raised when an operator cannot be applied between given types.

This exception is thrown when attempting to use an operator (like `+`, `-`, etc.) between incompatible types that don’t support the operation.

operator

The operator that couldn't be applied.

left_type

Type of the left operand.

right_type

Type of the right operand.

suggestion

Optional suggestion for fixing the type issue.

`__init__(operator, left_type, right_type, suggestion=)`

Initialize with operator and type details.

Parameters

- operator (`str`) – The operator that failed.
- left_type (`str`) – Type of the left operand.
- right_type (`str`) – Type of the right operand.
- suggestion (`str`) – Optional suggestion for resolving the issue.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.InvalidCastError(message)`

Bases: `ValueError`

Exception raised when a type cast operation fails.

This exception is thrown when attempting to cast a value to an incompatible type during query processing or data conversion.

message

Human-readable error message describing the cast failure.

`__init__(message)`

Initialize the exception with an error message.

Parameters

- message (`str`) – Description of the cast error that occurred.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

`__str__()`

Return the original error message without documentation hints.

This ensures backward compatibility with existing tests and code that expect `str(exception)` to return just the core message. The enhanced message with documentation hints is still accessible via the parent `ValueError`'s `args[0]`.

exception pycypher.exceptions.MissingParameterError(parameter_name, example_usage=)

Bases: [ValueError](#)

Exception raised when a required query parameter is missing.

This exception is thrown when a parameterized query references a parameter that wasn't provided in the parameters dictionary.

parameter_name

Name of the missing parameter.

example_usage

Example of how to provide the parameter.

__init__(parameter_name, example_usage=)

Initialize with missing parameter details.

Parameters

- parameter_name ([str](#)) – Name of the parameter that was missing.
- example_usage ([str](#)) – Example of correct parameter usage.

__repr__()

Return a repr exposing structured attributes for REPL inspection.

exception pycypher.exceptions.PatternComprehensionError(detail)

Bases: [ValueError](#)

Raised when a pattern comprehension has an invalid structure.

This is a [ValueError](#) subclass so that existing code catching ValueError continues to work.

detail

Description of what is wrong with the pattern.

__init__(detail)

Initialise with a description of the structural issue.

Parameters

detail ([str](#)) – Human-readable description of the pattern error.

__repr__()

Return a repr exposing structured attributes for REPL inspection.

exception pycypher.exceptions.QueryComplexityError(score, limit, breakdown=None)

Bases: [ValueError](#)

Raised when a query's complexity score exceeds the configured limit.

score

The computed complexity score.

limit

The configured maximum.

breakdown

Per-feature score breakdown.

`__init__(score, limit, breakdown=None)`

Initialize with score details.

Parameters

- `score` (`int`) – The computed complexity score.
- `limit` (`int`) – The configured maximum.
- `breakdown` (`dict[str, int] | None`) – Per-feature score breakdown.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.QueryMemoryBudgetError(estimated_bytes, budget_bytes, suggestion=)`

Bases: `MemoryError`

Exception raised when a query's estimated memory exceeds the budget.

This exception is thrown during query planning when the estimated memory consumption exceeds the configured budget, preventing OOM crashes.

`estimated_bytes`

Estimated memory consumption in bytes.

`budget_bytes`

The configured memory budget in bytes.

`suggestion`

Actionable suggestion for reducing memory usage.

`__init__(estimated_bytes, budget_bytes, suggestion=)`

Initialize with memory budget details.

Parameters

- `estimated_bytes` (`int`) – Estimated memory consumption.
- `budget_bytes` (`int`) – The configured memory budget.
- `suggestion` (`str`) – Actionable suggestion for reducing memory.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.QueryTimeoutError(timeout_seconds, elapsed_seconds=0.0, query_fragment=)`

Bases: `TimeoutError`

Exception raised when a query exceeds its execution time budget.

This exception is thrown when a query’s wall-clock execution time exceeds the configured timeout, preventing runaway queries from consuming resources indefinitely.

`timeout_seconds`

The timeout that was exceeded.

`elapsed_seconds`

How long the query ran before being terminated.

`query_fragment`

Truncated query text for diagnostics.

`__init__(timeout_seconds, elapsed_seconds=0.0, query_fragment=)`

Initialize with timeout details.

Parameters

- `timeout_seconds` (`float`) – The configured timeout in seconds.
- `elapsed_seconds` (`float`) – Actual elapsed time before cancellation.
- `query_fragment` (`str`) – Truncated query text for diagnostics.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.RateLimitError(*, qps, burst, caller_id=None)`

Bases: `Exception`

Raised when a query exceeds the configured rate limit.

`qps`

The configured queries-per-second limit.

`burst`

The configured burst capacity.

`caller_id`

The caller identifier, if per-caller limiting is active.

`__init__(* , qps, burst, caller_id=None)`

exception `pycypher.exceptions.SecurityError(message=, *, violation_type=)`

Bases: `Exception`

Raised when a security violation is detected.

This includes SQL injection attempts, path traversal attacks, dangerous DuckDB table function calls, and SSRF attempts.

`violation_type`

Category of security violation detected.

```
__init__(message="", *, violation_type="")
```

Initialize with security violation details.

Parameters

- `message` (`str`) – Description of the security violation.
- `violation_type` (`str`) – Category (e.g. “`sql_injection`”, “`path_traversal`”).

```
exception pycypher.exceptions.TemporalArithmeticError(operator, left_type, right_type, example="")
```

Bases: `IncompatibleOperatorError`

Exception raised for temporal arithmetic type errors.

This is a specialized `IncompatibleOperatorError` for temporal operations involving dates, times, and durations.

example

An example of correct temporal arithmetic usage.

```
__init__(operator, left_type, right_type, example="")
```

Initialize with temporal arithmetic error details.

Parameters

- `operator` (`str`) – The temporal operator that failed.
- `left_type` (`str`) – Type of the left temporal operand.
- `right_type` (`str`) – Type of the right temporal operand.
- `example` (`str`) – Example of correct temporal arithmetic.

```
__repr__()
```

Return a repr exposing structured attributes for REPL inspection.

```
exception pycypher.exceptions.UnsupportedFunctionError(function_name, supported_functions,
                                                         category="")
```

Bases: `ValueError`

Exception raised when an unsupported function is called.

This exception is thrown when attempting to call a function that isn’t implemented or available in the current context.

`function_name`

Name of the unsupported function.

`supported_functions`

List of functions that are supported.

`category`

Optional category of function (e.g., “`aggregation`”, “`scalar`”).

```
__init__(function_name, supported_functions, category="")
```

Initialize with function name and supported alternatives.

Parameters

- `function_name` (`str`) – Name of the function that isn't supported.
- `supported_functions` (`list[str]`) – Functions that are available.
- `category` (`str`) – Optional category for the function type.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.UnsupportedOperatorError(operator, supported_operators, category=)`

Bases: `ValueError`

Raised when an operator string is not in the supported dispatch table.

This is a `ValueError` subclass so that existing code catching `ValueError` continues to work. It provides structured access to the operator that was rejected and the set of supported alternatives.

`operator`

The operator string that was not recognised.

`supported_operators`

Sorted list of operators that are recognised.

`category`

Optional label for the operator family (e.g. "comparison").

`__init__(operator, supported_operators, category=)`

Initialise with the unsupported operator and available alternatives.

Parameters

- `operator` (`str`) – The operator string that was not recognised.
- `supported_operators` (`list[str]`) – Operators that are available.
- `category` (`str`) – Optional label for the kind of operator.

`__repr__()`

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.VariableNotFoundError(variable_name, available_variables, hint=)`

Bases: `ValueError`

Exception raised when a query variable is not found in the binding context.

This exception is thrown when attempting to access a variable that hasn't been defined in the current query scope or binding frame.

`variable_name`

The name of the missing variable.

`available_variables`

List of variables that are available in current scope.

```
__init__(variable_name, available_variables, hint="")
```

Initialize with variable name and available alternatives.

Parameters

- `variable_name` (`str`) – The name of the variable that was not found.
- `available_variables` (`list[str]`) – Variables that are currently in scope.
- `hint` (`str`) – Optional hint string from `suggest_close_match` (e.g., “ Did you mean ‘person’?”).

```
__repr__()
```

Return a repr exposing structured attributes for REPL inspection.

```
exception pycypher.exceptions.VariableTypeMismatchError(variable_name, expected_type, actual_type,
                                                         suggestion="")
```

Bases: `ValueError`

Exception raised when a variable has an unexpected type in the binding context.

This exception is thrown when a variable exists but its type doesn’t match what’s expected for the operation being performed.

`variable_name`

The name of the variable with wrong type.

`expected_type`

The type that was expected.

`actual_type`

The actual type of the variable.

`suggestion`

Optional suggestion for fixing the issue.

```
__init__(variable_name, expected_type, actual_type, suggestion="")
```

Initialize with type mismatch details.

Parameters

- `variable_name` (`str`) – Name of the variable with wrong type.
- `expected_type` (`str`) – The type that was expected.
- `actual_type` (`str`) – The actual type found.
- `suggestion` (`str`) – Optional suggestion for resolving the issue.

```
__repr__()
```

Return a repr exposing structured attributes for REPL inspection.

```
exception pycypher.exceptions.WorkerExecutionError(worker_id, query_snippet, elapsed_ms, message="")
```

Bases: `RuntimeError`

Raised when a query fails inside a cluster worker.

Wraps the original exception with worker identity and timing context so that distributed failures can be diagnosed without correlating separate log streams.

`worker_id`

Identifier of the worker that encountered the failure.

`query_snippet`

First 80 characters of the query that failed.

`elapsed_ms`

Wall-clock time in milliseconds before the failure.

Example:

```
try:
    worker.execute_query("MATCH (n) RETURN n")
except WorkerExecutionError as exc:
    print(f"Worker {exc.worker_id} failed after {exc.elapsed_ms:.0f}ms")
```

```
__init__(worker_id, query_snippet, elapsed_ms, message=)
```

```
__repr__()
```

Return a repr exposing structured attributes for REPL inspection.

exception `pycypher.exceptions.WrongCypherTypeError(message)`

Bases: `TypeError`

Exception raised when a Cypher expression has an unexpected type.

This exception is thrown when the CypherParser can parse an expression but the resulting type is not what was expected for the given context.

`message`

Human-readable error message describing the type mismatch.

```
__init__(message)
```

Initialize the exception with an error message.

Parameters

`message (str)` – Description of the type error that occurred.

```
__repr__()
```

Return a repr exposing structured attributes for REPL inspection.

```
__str__()
```

Return the original error message without documentation hints.

This ensures backward compatibility with existing tests and code that expect `str(exception)` to return just the core message. The enhanced message with documentation hints is still accessible via the parent `TypeError`'s `args[0]`.

4.2 Shared API

The Shared package contains common utilities used across all PyCypher packages. Shared utilities and common functionality for PyCypher-NMETL.

This package provides shared utilities, logging configuration, and helper functions used across the PyCypher-NMETL ecosystem.

4.2.1 Submodules

`shared.helpers`

Common utility functions: null checks, string matching, URI handling, base64 encoding/decoding.

`shared.logger`

Centralised logging configuration with Rich console formatting and optional JSON output.

`shared.metrics`

Lightweight in-process query metrics collector for diagnostic access.

`shared.telemetry`

Optional Pyroscope continuous profiling configuration.

`shared.otel`

OpenTelemetry distributed tracing integration (optional dependency).

`shared.exporters`

Pluggable metrics export adapters (Prometheus, StatsD, JSON).

`shared.regression_detector`

Statistical performance regression detection engine using z-score analysis with configurable sensitivity thresholds.

`shared.alerting`

Configurable alerting system with threshold rules, cooldown support, and pluggable notification handlers.

`shared.continuous_monitor`

Continuous metrics monitoring orchestration that ties together the metrics collector, dashboard, alerting, regression detection, and exporters into a single periodic monitoring loop.

`shared.dashboard`

Performance monitoring dashboard with web-based visualization, real-time metric updates, and JSON API.

`shared.deprecation`

Standardised deprecation warnings with `deprecated()` decorator and `emit_deprecation()` helper for graceful API evolution.

`shared.compat`

API surface snapshot/diff tools and Neo4j Cypher compatibility notes for detecting breaking changes and guiding migrations.

`shared.file_intent`

File Intent Registration System for multi-agent coordination. Agents register intent before editing shared files; the system classifies risk levels and warns on conflicts.

4.2.2 Core Modules

Logger

Structured logging configuration shared across all packages. Shared logging configuration module.

This module provides a centralized logger configuration using Rich for enhanced console output formatting. The logger level defaults to WARNING but can be overridden via the PYCYPHER_LOG_LEVEL environment variable (e.g. PYCYPHER_LOG_LEVEL=DEBUG).

Set PYCYPHER_LOG_FORMAT=json for machine-readable JSON-lines output suitable for log aggregation pipelines (ELK, Datadog, Splunk).

Query correlation

Use `set_query_id()` / `get_query_id()` to propagate a query correlation ID through the call stack via `contextvars`. The JSON formatter automatically includes the active `query_id` in every log line, eliminating the need to pass `extra={"query_id": ...}` manually.

```
from shared.logger import set_query_id, reset_query_id

token = set_query_id("abc123")
try:
    LOGGER.info("processing") # JSON output includes "query_id": "abc123"
finally:
    reset_query_id(token)
```

`shared.logger.LOGGING_LEVEL`

Effective logging level string (from env or default).

`shared.logger.LOGGER`

Configured logger instance.

`shared.logger.set_query_id(query_id)`

Set the active query correlation ID for the current context.

Returns a token that can be passed to `reset_query_id()` to restore the previous value.

Parameters

`query_id` (`str`) – The correlation ID to propagate through log output.

Returns

A `contextvars.Token` for restoring the previous value.

Return type

`Token`

`shared.logger.get_query_id()`

Return the active query correlation ID, or None if unset.

`shared.logger.reset_query_id(token)`

Restore the query ID to its previous value.

Parameters

`token` (`Token`) – The token returned by `set_query_id()`.

Metrics

Query execution metrics collection, snapshots, and diagnostic summaries. Lightweight in-process query metrics collector.

Aggregates query execution statistics — counts, timing percentiles, error rates, and slow-query detection — without requiring an external metrics backend. All data lives in-process and is designed for diagnostic access via `QueryMetrics.snapshot()`.

The collector is thread-safe (uses a `threading.Lock`) for compatibility with the free-threaded Python 3.14t build.

Enable/disable via the `PYCYPHER_METRICS_ENABLED` environment variable (default: `enabled`). When disabled, all recording methods are no-ops.

Usage:

```
from shared.metrics import QUERY_METRICS

# Record a completed query
QUERY_METRICS.record_query(
    query_id="abc123",
    elapsed_s=0.042,
    rows=150,
    clauses=["Match", "Return"],
)

# Record an error
QUERY_METRICS.record_error(
    query_id="def456",
    error_type="TypeError",
    elapsed_s=0.003,
)

# Get a point-in-time snapshot
stats = QUERY_METRICS.snapshot()
print(stats)
```

`shared.metrics.QUERY_METRICS`

Module-level singleton collector instance.

`shared.metrics.SLOW_QUERY_THRESHOLD_S`

Queries exceeding this duration trigger a warning log. Configurable via `PYCYPHER_SLOW_QUERY_MS` env var (default 1000 ms).

Type
float

`shared.metrics.get_rss_mb()`

Return current process RSS in megabytes using `resource`.

Uses `resource.getrusage()` (stdlib) for zero external dependencies. Returns 0.0 if unavailable.

```
class shared.metrics.MetricsSnapshot(total_queries, total_errors, slow_queries, error_counts,
                                     clause_counts, timing_p50_ms, timing_p90_ms, timing_p99_ms,
                                     timing_max_ms, timing_min_ms, total_rows_returned, uptime_s,
                                     queries_per_second, rows_per_second, recent_queries_per_second,
                                     memory_delta_p50_mb, memory_delta_p90_mb,
                                     memory_delta_max_mb, clause_timing_p50_ms,
                                     clause_timing_p90_ms, clause_timing_max_ms,
                                     parse_time_p50_ms, parse_time_p90_ms, parse_time_max_ms,
                                     error_rate, recent_error_rate, planner_accuracy_ratio_p50,
                                     planner_mae_mb, plan_time_p50_ms, plan_time_p90_ms,
                                     plan_time_max_ms, result_cache_hits=0, result_cache_misses=0,
                                     result_cache_hit_rate=0.0, result_cache_size_mb=0.0,
                                     result_cache_entries=0, result_cache_evictions=0)
```

Bases: `object`

Immutable point-in-time view of collected metrics.

`total_queries`

Total number of successfully executed queries.

Type
`int`

`total_errors`

Total number of failed queries.

Type
`int`

`slow_queries`

Number of queries that exceeded the slow-query threshold.

Type
`int`

`error_counts`

Breakdown of errors by exception type name.

Type
`dict[str, int]`

`clause_counts`

How many times each clause type was executed.

Type
`dict[str, int]`

`timing_p50_ms`

Median query execution time in milliseconds.

Type
`float`

`timing_p90_ms`

90th percentile execution time in milliseconds.

Type
`float`

timing_p99_ms
99th percentile execution time in milliseconds.

Type
float

timing_max_ms
Maximum observed execution time in milliseconds.

Type
float

timing_min_ms
Minimum observed execution time in milliseconds.

Type
float

total_rows_returned
Cumulative rows returned across all queries.

Type
int

uptime_s
Seconds since the collector was created.

Type
float

total_queries: int

total_errors: int

slow_queries: int

error_counts: dict[str, int]

clause_counts: dict[str, int]

timing_p50_ms: float

timing_p90_ms: float

timing_p99_ms: float

timing_max_ms: float

timing_min_ms: float

total_rows_returned: int

uptime_s: float

queries_per_second: float

rows_per_second: float

recent_queries_per_second: float

memory_delta_p50_mb: float

```

memory_delta_p90_mb: float
memory_delta_max_mb: float
clause_timing_p50_ms: dict[str, float]
clause_timing_p90_ms: dict[str, float]
clause_timing_max_ms: dict[str, float]
parse_time_p50_ms: float
parse_time_p90_ms: float
parse_time_max_ms: float
error_rate: float
recent_error_rate: float
planner_accuracy_ratio_p50: float
planner_mae_mb: float
plan_time_p50_ms: float
plan_time_p90_ms: float
plan_time_max_ms: float
result_cache_hits: int = 0
result_cache_misses: int = 0
result_cache_hit_rate: float = 0.0
result_cache_size_mb: float = 0.0
result_cache_entries: int = 0
result_cache_evictions: int = 0

```

summary()

Return a human-readable diagnostic summary.

Suitable for logging, dashboard display, or REPL inspection.

health_status()

Return automated health classification.

Returns

"healthy" — error rate < 5% and slow query rate < 10%. "degraded" — error rate 5–20% or slow query rate 10–30%. "unhealthy" — error rate > 20% or slow query rate > 30%.

Return type

str

`diagnostic_report()`

Return a detailed diagnostic report with actionable recommendations.

Analyses performance degradation, error patterns, clause hotspots, cache efficiency, and resource pressure. Designed for production debugging — each section includes concrete recommendations when problems are detected.

Returns

A multi-section diagnostic report string.

Return type

`str`

`to_dict()`

Return snapshot as a flat dictionary for JSON serialization.

Useful for programmatic access, log aggregation pipelines, and external monitoring integrations.

Returns

A dictionary with all metric fields as key-value pairs.

Return type

`dict[str, Any]`

```
__init__(total_queries, total_errors, slow_queries, error_counts, clause_counts, timing_p50_ms,
         timing_p90_ms, timing_p99_ms, timing_max_ms, timing_min_ms,
         total_rows_returned, uptime_s, queries_per_second, rows_per_second,
         recent_queries_per_second, memory_delta_p50_mb, memory_delta_p90_mb,
         memory_delta_max_mb, clause_timing_p50_ms, clause_timing_p90_ms,
         clause_timing_max_ms, parse_time_p50_ms, parse_time_p90_ms,
         parse_time_max_ms, error_rate, recent_error_rate, planner_accuracy_ratio_p50,
         planner_mae_mb, plan_time_p50_ms, plan_time_p90_ms, plan_time_max_ms,
         result_cache_hits=0, result_cache_misses=0, result_cache_hit_rate=0.0,
         result_cache_size_mb=0.0, result_cache_entries=0, result_cache_evictions=0)
```

```
class shared.metrics.QueryMetrics(_total_queries=0, _total_errors=0, _slow_queries=0,
                                  _total_rows=0, _error_counts=<factory>,
                                  _clause_counts=<factory>, _timings_ms=<factory>,
                                  _memory_deltas_mb=<factory>, _clause_timings_ms=<factory>,
                                  _parse_times_ms=<factory>, _query_timestamps=<factory>,
                                  _error_timestamps=<factory>,
                                  _planner_accuracy_ratios=<factory>,
                                  _planner_abs_errors_mb=<factory>, _plan_times_ms=<factory>,
                                  _cache_stats=<factory>, _lock=<factory>, _created_at=<factory>)
```

Bases: `object`

Thread-safe in-process query metrics collector.

Records query execution statistics and provides diagnostic snapshots. All mutating methods acquire `_lock` before modifying internal state.

```
record_query(*, query_id, elapsed_s, rows=0, clauses=None, memory_delta_mb=None,
             clause_timings_ms=None, parse_time_ms=None, estimated_memory_mb=None,
             plan_time_ms=None)
```

Record a successfully completed query.

Parameters

- `query_id` (`str`) – Correlation ID for the query.
- `elapsed_s` (`float`) – Wall-clock execution time in seconds.
- `rows` (`int`) – Number of rows in the result set.
- `clauses` (`list[str] | None`) – List of clause type names executed (e.g. `["Match", "Return"]`).
- `memory_delta_mb` (`float | None`) – RSS memory change during query execution (MB).
- `clause_timings_ms` (`dict[str, float] | None`) – Per-clause-type execution times in milliseconds.
- `parse_time_ms` (`float | None`) – Time spent parsing the query string (ms).
- `estimated_memory_mb` (`float | None`) – Planner’s memory estimate (MB) for accuracy tracking.
- `plan_time_ms` (`float | None`) – Time spent in query planning phase (ms).

`record_error(*, query_id, error_type, elapsed_s)`

Record a failed query execution.

Parameters

- `query_id` (`str`) – Correlation ID for the query.
- `error_type` (`str`) – Exception class name (e.g. `"TypeError"`).
- `elapsed_s` (`float`) – Wall-clock time before failure in seconds.

`update_cache_stats(stats)`

Update the latest result cache statistics.

Called after each query execution with the output of `ResultCache.stats()`.

Parameters

`stats` (`dict[str, Any]`) – Dict with keys like `result_cache_hits`, `result_cache_misses`, `result_cache_hit_rate`, etc.

`snapshot()`

Return an immutable point-in-time view of all collected metrics.

Returns

A `MetricsSnapshot` with current aggregated statistics.

Return type

`MetricsSnapshot`

`reset()`

Reset all counters and timings to zero.

Useful for test isolation or periodic metric rotation.

```

__init__(
    _total_queries=0, _total_errors=0, _slow_queries=0, _total_rows=0,
    _error_counts=<factory>, _clause_counts=<factory>, _timings_ms=<factory>,
    _memory_deltas_mb=<factory>, _clause_timings_ms=<factory>,
    _parse_times_ms=<factory>, _query_timestamps=<factory>,
    _error_timestamps=<factory>, _planner_accuracy_ratios=<factory>,
    _planner_abs_errors_mb=<factory>, _plan_times_ms=<factory>,
    _cache_stats=<factory>, _lock=<factory>, _created_at=<factory>)

```

Helpers

Serialisation and utility functions. Shared utility functions used across the PyCypher-NMETL project.

This module provides common helper functions for URI handling, encoding/decoding objects, data type conversions, and user-experience helpers that are used throughout the project.

`shared.helpers.is_null_value(x)`

Return True if x is None or a floating-point NaN.

This is the canonical null-check used throughout the PyCypher evaluator pipeline. Cypher treats both None and NaN as null, so every point that needs to distinguish “has a value” from “is missing” should call this function rather than re-implementing the check inline.

Parameters

x (`object`) – The value to test.

Returns

True when x is None or float('nan').

Return type

`bool`

`shared.helpers.is_null_raw_list(value)`

Return True if value is null, empty, or not a valid iterable collection.

Used by collection evaluators to detect when a row’s list expression produced a missing/non-list value so each caller can handle the degenerate case (skip iteration, treat as empty list, or return the accumulator unchanged).

Handles None, empty pandas arrays/Series, empty Python lists/tuples, and scalar pandas null values (NaN, pd.NA, pd.NaT).

Parameters

value (`object`) – A single Python value from a pd.Series iteration.

Returns

True if the value should be treated as an empty/missing list.

Return type

`bool`

`shared.helpers.suggest_close_match(target, candidates, cutoff=0.6)`

Return a “Did you mean ‘...’?” hint string when a close match is found.

Uses `diffib.get_close_matches()` to find the single best candidate within cutoff similarity. Returns an empty string when the target already appears verbatim in candidates (no hint needed for correct input) or when no candidate is close enough.

Parameters

- target (`str`) – The misspelled or unknown name entered by the user.

- `candidates` (`Iterable[str]`) – Iterable of valid names to compare against.
- `cutoff` (`float`) – Minimum similarity ratio in `[0, 1]`. Defaults to 0.6, matching the project-wide convention used at all three call sites.

Returns

A hint string such as `" Did you mean 'tolower'?"` (with a leading two-space indent for inline appending to error messages), or `""` when no close match exists.

Return type

`str`

Examples

```
>>> suggest_close_match("persn", ["person", "company"])
" Did you mean 'person'?"
>>> suggest_close_match("xyz", ["person", "company"])
""
>>> suggest_close_match("person", ["person"]) # exact match — no hint
""
```

`shared.helpers.ensure_uri(uri_input)`

Ensure input is converted to a ParseResult URI object.

`shared.helpers.decode(encoded_data)`

Decode a base64-encoded JSON object.

Accepts JSON-serializable Python values (dicts, lists, strings, numbers, booleans, None). Raises `ValueError` for any input that is not valid base64-encoded JSON — including pickle payloads, which are rejected to prevent deserialisation of arbitrary code.

Parameters

`encoded_data` (`str`) – Base64-encoded UTF-8 JSON string, as produced by `encode()`.

Returns

The decoded Python value.

Raises

`ValueError` – If the input is not valid base64-encoded JSON.

Return type

`Any`

`shared.helpers.encode(source_object, to_bytes=False)`

Encode a JSON-serializable Python object as a base64 string.

Accepts dicts, lists, strings, numbers, booleans, and None. Raises `ValueError` for types that are not JSON-serializable (e.g. sets, custom class instances, bytes). Using JSON rather than pickle prevents deserialisation of arbitrary code in `decode()`.

Parameters

- `source_object` (`Any`) – A JSON-serializable Python value.
- `to_bytes` (`bool`) – If True, return the base64 string as `bytes` instead of `str`.

Returns

Base64-encoded representation as `str` (default) or `bytes`.

Raises

`ValueError` – If `source_object` is not JSON-serializable or if encoding fails for any other reason.

Return type

`str` | `bytes`

`shared.helpers.ensure_bytes(input_value, **encoding_kwargs)`

Ensure the input value is converted to bytes.

Telemetry

Optional telemetry integration (Pyroscope profiling). Configuration for Pyroscope continuous profiling.

All settings are configurable via environment variables:

- `PYROSCOPE_SERVER` – Pyroscope endpoint (default `http://localhost:4040`)
- `PYROSCOPE_APP_NAME` – application name tag (default `nmetl`)
- `PYROSCOPE_SAMPLE_RATE` – samples per second (default `100`)
- `PYROSCOPE_ENABLED` – set to `0` or `false` to disable (default enabled)

If Pyroscope is not installed or the server is unreachable the module silently degrades — no import-time crash.

4.2.3 Observability

OpenTelemetry

Tracing and span instrumentation for query execution phases. Falls back to no-op stubs when opentelemetry is not installed. OpenTelemetry integration for PyCypher distributed tracing.

Wraps query execution with OpenTelemetry spans for end-to-end distributed tracing. All configuration is via standard `OTEL_*` environment variables.

Zero-config design: if opentelemetry-api is not installed, every public function is a silent no-op — no import errors, no runtime cost.

Enable via:

- `PYCYPHER_OTEL_ENABLED=1` (default: `0`)
- Standard OTEL env vars (`OTEL_SERVICE_NAME`, `OTEL_EXPORTER_*`, etc.)

Usage:

```
from shared.otel import trace_query, get_tracer

# Context-manager style
with trace_query("MATCH (p:Person) RETURN p.name", query_id="abc") as span:
    result = star.execute_query(query)
    span.set_attribute("result.rows", len(result))

# Manual tracer access
tracer = get_tracer()
with tracer.start_as_current_span("custom-operation") as span:
    ...
```

`shared.otel.OTEL_ENABLED`

Whether OpenTelemetry tracing is active.

Type
`bool`

`shared.otel.get_tracer()`

Return the active OpenTelemetry tracer, or a no-op tracer.

Returns
An OpenTelemetry Tracer if enabled and installed, otherwise a `__NullTracer` that silently discards all calls.

Return type
`Any`

`shared.otel.trace_query(query, *, query_id=None, parameters=None)`

Wrap a query execution in an OpenTelemetry span.

Sets standard attributes on the span:

- `db.system` = "pocypher"
- `db.statement` = query text (truncated to 500 chars)
- `db.operation` = first Cypher keyword (MATCH, CREATE, etc.)
- `pocypher.query_id` = correlation ID (if provided)

On exception, the span is marked ERROR and the exception is recorded.

Parameters

- `query` (`str`) – The Cypher query string.
- `query_id` (`str` | `None`) – Optional correlation ID for cross-system tracing.
- `parameters` (`dict[str, Any]` | `None`) – Optional query parameters (keys logged, values omitted for security).

Yields

The active span (real or `__NullSpan`).

Example:

```
with trace_query("MATCH (n) RETURN n", query_id="q-123") as span:
    result = star.execute_query(query)
    span.set_attribute("result.rows", len(result))
```

`shared.otel.trace_phase(phase, *, query_id=None)`

Wrap a query execution phase (parse, plan, execute) in a child span.

Parameters

- `phase` (`str`) – Phase name (e.g. "parse", "plan", "execute").
- `query_id` (`str` | `None`) – Optional correlation ID.

Yields

The active span (real or `__NullSpan`).

`shared.otel.record_metrics_to_span(span, snapshot)`

Attach MetricsSnapshot summary data to an existing span.

Useful for periodic health-check spans or diagnostic traces.

Parameters

- `span` ([Any](#)) – An OpenTelemetry span (or `_NullSpan`).
- `snapshot` ([Any](#)) – A [MetricsSnapshot](#) instance.

Exporters

Metrics export adapters: Prometheus, StatsD, and JSON file exporters with a unified MetricsExporter protocol. Pluggable metrics export adapters for external monitoring systems.

Bridges the in-process [QueryMetrics](#) collector to external observability backends — Prometheus, StatsD/Datadog, and JSON file export. All exporters work with zero external dependencies by default (Prometheus text format and JSON need only `stdlib`).

Configuration is entirely via environment variables:

- `PYCYPHER_METRICS_EXPORT` — comma-separated list of active exporters (e.g. `prometheus, statsd,json`). Default: `none`.
- `PYCYPHER_METRICS_PREFIX` — metric name prefix (default: `pycypher`).
- `PYCYPHER_STATSD_HOST` / `PYCYPHER_STATSD_PORT` — StatsD endpoint.
- `PYCYPHER_METRICS_JSON_PATH` — path for JSON file export.
- `PYCYPHER_METRICS_EXPORT_INTERVAL_S` — push interval (default: `60`).

Usage:

```
from shared.exporters import get_exporters, export_once

# Auto-configured from env vars
exporters = get_exporters()

# Push current metrics to all configured backends
from shared.metrics import QUERY_METRICS
export_once(QUERY_METRICS.snapshot())

# Or get Prometheus text format directly
from shared.exporters import PrometheusExporter
exporter = PrometheusExporter()
print(exporter.render(QUERY_METRICS.snapshot()))
```

`class shared.exporters.MetricsExporter(*args, **kwargs)`

Bases: [Protocol](#)

Protocol for metrics export adapters.

property name: `str`

Human-readable exporter name.

`export(snapshot)`

Push a [MetricsSnapshot](#) to the backend.

Parameters

- `snapshot` ([Any](#)) – A metrics snapshot from `QueryMetrics.snapshot()`.

```
__init__(*args, **kwargs)
```

```
class shared.exporters.PrometheusExporter(prefix='pycypher')
```

Bases: `object`

Export metrics in Prometheus text exposition format.

Generates text/plain; version=0.0.4 output suitable for scraping by Prometheus, VictoriaMetrics, or any OpenMetrics-compatible collector.

Does not require any external dependencies — generates the text format directly. For a full Prometheus client with push-gateway support, see `prometheus_client`.

Parameters

`prefix` (`str`) – Metric name prefix. Default: `pycypher`.

```
__init__(prefix='pycypher')
```

property name: `str`

Exporter name.

```
render(snapshot)
```

Render a `MetricsSnapshot` as Prometheus text exposition format.

Parameters

`snapshot` (`Any`) – A `MetricsSnapshot`.

Returns

Multi-line string in Prometheus text format.

Return type

`str`

```
export(snapshot)
```

Log the Prometheus text output at INFO level.

In production, mount an HTTP endpoint that calls `render()` instead. This method is for development/debugging.

Parameters

`snapshot` (`Any`) – A `MetricsSnapshot`.

```
class shared.exporters.StatsDExporter(host=None, port=None, prefix='pycypher')
```

Bases: `object`

Push metrics to a StatsD-compatible daemon via UDP.

Uses a plain UDP socket (stdlib only) — no external dependencies. Compatible with StatsD, Datadog Agent, Telegraf, and similar collectors.

Parameters

- `host` (`str` | `None`) – StatsD host. Default from `PYCYPHER_STATSD_HOST` or `127.0.0.1`.
- `port` (`int` | `None`) – StatsD port. Default from `PYCYPHER_STATSD_PORT` or `8125`.
- `prefix` (`str`) – Metric name prefix.

```
__init__(host=None, port=None, prefix='picypher')
```

property name: `str`

Exporter name.

```
export(snapshot)
```

Push metrics to StatsD.

Parameters

`snapshot` (*Any*) – A `MetricsSnapshot`.

```
class shared.exporters.JSONFileExporter(path=None)
```

Bases: `object`

Append metrics snapshots to a JSON-lines file.

Each export appends a single JSON line with a timestamp, suitable for ingestion by ELK, Datadog Logs, Loki, or any log aggregation pipeline.

Parameters

`path` (`str` | `None`) – Output file path. Default from `PY-CYPHER_METRICS_JSON_PATH` or `metrics.jsonl`.

```
__init__(path=None)
```

property name: `str`

Exporter name.

```
export(snapshot)
```

Append a JSON-lines entry for the snapshot.

Parameters

`snapshot` (*Any*) – A `MetricsSnapshot`.

```
shared.exporters.get_exporters()
```

Auto-configure exporters from `PYCYPHER_METRICS_EXPORT` env var.

Returns

List of configured `MetricsExporter` instances. Empty list if no exporters are configured.

Return type

`list[MetricsExporter]`

Example:

```
# In .env: PYCYPHER_METRICS_EXPORT=prometheus.json
exporters = get_exporters()
# Returns [PrometheusExporter(), JSONFileExporter()]
```

```
shared.exporters.export_once(snapshot)
```

Push a snapshot to all configured exporters.

Convenience function that calls `get_exporters()` and invokes `export()` on each. Errors in individual exporters are logged but do not propagate.

Parameters

`snapshot` (*Any*) – A `MetricsSnapshot`.

4.2.4 Compatibility

API Surface Compatibility

Snapshot and diff the public API surface for backward-compatibility checking between releases. API surface snapshot and compatibility checking utilities.

Provides tools for detecting breaking changes between PyCypher releases by capturing and comparing public API surfaces. This module supports two primary workflows:

1. Snapshot — capture the current public API to a JSON file for future comparison:

```
from shared.compat import snapshot_api_surface, save_snapshot

surface = snapshot_api_surface("pycypher")
save_snapshot(surface, "api_v0.0.19.json")
```

2. Diff — compare two snapshots to find added, removed, and changed symbols:

```
from shared.compat import load_snapshot, diff_surfaces

old = load_snapshot("api_v0.0.18.json")
new = load_snapshot("api_v0.0.19.json")
report = diff_surfaces(old, new)
print(report.summary())
```

The snapshot captures every name in `__all__` (or `dir()` filtered to public names), along with its kind (function, class, exception, constant) and call signature where available.

Neo4j Cypher compatibility

The `NEO4J_COMPAT_NOTES` dictionary documents known syntax differences between standard Neo4j Cypher and the PyCypher dialect so that users migrating queries can check compatibility up front.

```
shared.compat.NEO4J_COMPAT_NOTES: dict[str, dict[str, Any]] = {'APOC functions': {'notes':
'APOC is Neo4j-specific. PyCypher has its own scalar function library.', 'supported': False, 'workaround':
'Check `nmetl functions` for available PyCypher functions.'}, 'CALL procedure()': {'notes': 'CALL clause
supported for registered procedures.', 'supported': True}, 'CALL { ... } IN TRANSACTIONS': {'notes':
'PyCypher operates in-memory; transaction batching is not applicable.', 'supported': False}, 'CASE
expressions': {'notes': 'Both simple and generic CASE WHEN forms supported.', 'supported': True},
'CREATE': {'notes': 'Supported for node and relationship creation.', 'supported': True}, 'CREATE
CONSTRAINT': {'notes': 'Constraints are enforced at the DataFrame level.', 'supported': False},
'CREATE INDEX': {'notes': 'PyCypher uses DataFrame column access; no index DDL needed.',
'supported': False}, 'DELETE': {'notes': 'Supported for node and relationship deletion.', 'supported':
True}, 'DETACH DELETE': {'notes': 'Not yet supported. Use explicit relationship deletion first.',
'supported': False, 'workaround': 'DELETE relationships manually before deleting nodes.'}, 'EXISTS
subqueries': {'notes': 'Supported in WHERE clauses.', 'supported': True}, 'FOREACH': {'notes':
'Supported for mutation within loops.', 'supported': True}, 'LOAD CSV': {'notes': 'Use ContextBuilder
with pandas DataFrames or DuckDBReader instead.', 'supported': False, 'workaround': 'pd.read_csv() →
ContextBuilder().add_entity(...)'}, 'List comprehensions': {'notes': 'Supported: [x IN list WHERE pred |
expr]', 'supported': True}, 'MERGE': {'notes': 'Supported with ON CREATE SET / ON MATCH SET.',
'supported': True}, 'Map projections': {'notes': 'Supported: node { .prop1, .prop2, key: expr }',
'supported': True}, 'OPTIONAL MATCH': {'notes': 'Left-outer-join semantics, same as Neo4j.',
'supported': True}, 'Parameters ($param)': {'notes': 'Supported via execute_query(params={...}).',
'supported': True}, 'Pattern comprehensions': {'notes': 'Supported: [(a)-->(b) | b.name]', 'supported':
True}, 'Temporal types': {'notes': 'date(), time(), datetime(), duration() supported.', 'supported': True},
'UNION / UNION ALL': {'notes': 'Both variants supported.', 'supported': True}, 'UNWIND': {'notes':
'Supported for list expansion.', 'supported': True}, 'Variable-length paths [*m..n]': {'notes': 'Supported
with safety limits (max 20 unbounded hops, 1M frontier rows).', 'supported': True}, 'WITH': {'notes':
'Projection and aggregation piping supported.', 'supported': True}, 'allShortestPaths': {'notes': 'Returns
all minimum-hop paths between node pairs.', 'supported': True}, 'shortestPath': {'notes': 'BFS-based
implementation with configurable hop limits.', 'supported': True}}
```

Known syntax/feature differences between Neo4j Cypher and PyCypher. Keys are feature names; values are dicts with supported, notes, and optional workaround fields.

```
class shared.compat.SymbolInfo(name, kind, signature=None, module="")
```

Bases: `object`

Metadata about a single public API symbol.

name: `str`

kind: `str`

signature: `str` | `None` = `None`

module: `str` = `"`

to_dict()

Serialise to a JSON-friendly dictionary.

classmethod from_dict(data)

Deserialise from a dictionary.

__init__(name, kind, signature=None, module="")

```
class shared.compat.APISurface(module_name, version, symbols=<factory>)
```

Bases: `object`

Snapshot of a module's public API at a point in time.

`module_name`: `str`

`version`: `str`

`symbols`: `dict[str, SymbolInfo]`

`to_dict()`

Serialise the full surface to JSON-friendly dict.

classmethod `from_dict(data)`

Deserialise from a dictionary.

`__init__(module_name, version, symbols=<factory>)`

```
shared.compat.snapshot_api_surface(module_name)
```

Capture the public API surface of a module.

Imports the module, reads its `__all__` (falling back to public `dir()` entries), and records each symbol's kind and signature.

Parameters

`module_name` (`str`) – Fully-qualified module name (e.g. "pycypher").

Returns

An `APISurface` snapshot.

Return type

`APISurface`

```
shared.compat.save_snapshot(surface, path)
```

Write an API surface snapshot to a JSON file.

Parameters

- `surface` (`APISurface`) – The snapshot to save.
- `path` (`str` | `Path`) – Destination file path.

```
shared.compat.load_snapshot(path)
```

Load an API surface snapshot from a JSON file.

Parameters

`path` (`str` | `Path`) – Path to the JSON snapshot.

Returns

The deserialised `APISurface`.

Return type

`APISurface`

```
class shared.compat.SurfaceDiff(old_version, new_version, added=<factory>, removed=<factory>,
                                changed=<factory>)
```

Bases: `object`

Result of comparing two API surface snapshots.

`old_version`: `str`

`new_version`: `str`

`added`: `list[SymbolInfo]`

`removed`: `list[SymbolInfo]`

`changed`: `list[tuple[SymbolInfo, SymbolInfo]]`

property `has_breaking_changes`: `bool`

Whether any removals or signature changes were detected.

`summary()`

Human-readable summary of the diff.

`__init__`(old_version, new_version, added=<factory>, removed=<factory>, changed=<factory>)


```
shared.compat.diff_surfaces(old, new)
```

Compare two API surface snapshots.

Parameters

- `old` (`APISurface`) – The baseline snapshot.
- `new` (`APISurface`) – The current snapshot.

Returns

A `SurfaceDiff` describing additions, removals, and changes.

Return type

`SurfaceDiff`


```
shared.compat.check_neo4j_compat(feature)
```

Look up compatibility notes for a Neo4j Cypher feature.

Parameters

`feature` (`str`) – Feature name (case-insensitive substring match).

Returns

The compatibility note dict, or None if no match.

Return type

`dict[str, Any] | None`

Deprecation Utilities

Emit structured deprecation warnings with configurable severity and suggested replacements. Lightweight deprecation utilities for PyCypher API evolution.

Provides a `deprecated()` decorator and a `emit_deprecation()` helper that standardise deprecation warnings across the codebase. All warnings include a removal version, a migration hint, and are emitted via the standard `warnings` module so that users can filter them with `warnings.filterwarnings()`.

Usage:

```
from shared.deprecation import deprecated

@deprecated(since="0.0.19", removed_in="0.1.0", alternative="new_func")
def old_func(x: int) -> int:
    return x + 1

# Calling old_func() emits:
# DeprecationWarning: old_func is deprecated since v0.0.19 and will
# be removed in v0.1.0. Use new_func instead.
```

For non-decorator contexts (e.g. `__getattr__` module-level hooks), use `emit_deprecation()` directly:

```
from shared.deprecation import emit_deprecation

def __getattr__(name: str) -> type:
    if name == "OldClass":
        emit_deprecation("OldClass", since="0.0.19",
                        removed_in="0.1.0", alternative="NewClass")
        return NewClass
    raise AttributeError(name)
```

`shared.deprecation.emit_deprecation(name, *, since, removed_in="", alternative="", detail="", stacklevel=2)`

Emit a standardised deprecation warning.

Use this in `__getattr__` hooks or other non-decorator contexts.

Parameters

- `name (str)` – Name of the deprecated symbol.
- `since (str)` – Version where deprecation was introduced.
- `removed_in (str)` – Version where the symbol will be removed.
- `alternative (str)` – Replacement symbol name.
- `detail (str)` – Additional migration guidance.
- `stacklevel (int)` – Stack level for the warning (default 2 = caller's caller).

`shared.deprecation.deprecated(*, since, removed_in="", alternative="", detail="")`

Decorator that marks a function or class as deprecated.

Emits a `DeprecationWarning` on every call with a standardised message including version timeline and migration guidance.

Parameters

- `since (str)` – Version where deprecation was introduced (e.g. "0.0.19").
- `removed_in (str)` – Version where the symbol will be removed (e.g. "0.1.0").

- `alternative (str)` – Name of the replacement (e.g. `"new_func"`).
- `detail (str)` – Additional migration context.

Returns

A decorator that wraps the target with a deprecation warning.

Return type

`Any`

Example:

```
@deprecated(since="0.0.19", removed_in="0.1.0", alternative="ContextBuilder")
class OldBuilder:
    ...
```

4.3 Package Overview

4.3.1 PyCypher

The core package — Cypher query parsing, AST processing, and `BindingFrame` execution.

Entry points:

- `Star` — execute Cypher queries against `DataFrames`
- `ContextBuilder` — build graph contexts from tabular data
- `validate_query()` — pre-execution query validation

Execution pipeline:

- `grammar_parser` / `grammar_transformers` — Cypher → Lark → AST
- `ast_models` / `ast_converter` / `ast_rewriter` — immutable Pydantic AST, conversion, and rewriting
- `pattern_matcher` / `path_expander` — MATCH clause evaluation
- `binding_frame` / `binding_evaluator` — vectorised expression execution
- `pipeline` — stage-based parse → validate → plan → execute pipeline
- `backend_engine` — pluggable backends (Pandas, DuckDB, Polars)
- `query_planner` / `query_optimizer` / `query_profiler` — planning, optimization, and profiling

Evaluators:

- `arithmetic_evaluator` / `boolean_evaluator` / `comparison_evaluator` — operator evaluation
- `collection_evaluator` / `string_predicate_evaluator` / `exists_evaluator` — complex expressions
- `aggregation_evaluator` / `aggregation_planner` — grouped and full-table aggregation
- `scalar_function_evaluator` / `scalar_functions` — 131 built-in scalar functions

Multi-query composition:

- `multi_query_analyzer` / `multi_query_executor` — dependency-aware ETL pipelines

Write path:

- `mutation_engine` — CREATE, SET, DELETE, MERGE, FOREACH
- `sinks.neo4j` — write results to Neo4j

4.3.2 Shared

Common utilities: structured logging, query metrics, serialisation helpers, observability, and compatibility checking.

- logger — `LOGGER` singleton for structured output
- metrics — `QUERY_METRICS` collector and `MetricsSnapshot`
- helpers — serialisation and utility functions
- telemetry — optional Pyroscope profiling integration
- otel — OpenTelemetry tracing (no-op fallback when not installed)
- exporters — Prometheus, StatsD, and JSON metrics export
- compat — API surface snapshot and backward-compatibility diffing
- deprecation — structured deprecation warnings

Comprehensive guides for using PyCypher features.

5.1 Data Model Requirements

This guide explains why PyCypher requires specific columns in your data, how to use your existing column names without renaming, and how to troubleshoot common data issues.

In this guide

- Why Does PyCypher Need `__ID__` Columns?
 - The Short Answer
 - Why Not Just Use Row Position?
- Using Your Existing Column Names
 - The Most Common Pattern
 - From the CLI
 - In YAML Pipeline Config
 - When No ID Column Exists
- Real-World Examples
 - Example 1: E-Commerce (Customer → Order → Product)
 - Example 2: Social Network (User follows User)
 - Example 3: Multi-Entity (Person + Company + Employment)
- Data Preparation Guide
 - Converting Existing Tables
 - Handling Composite Keys
 - ID Generation Strategies
 - Type Coercion

- Troubleshooting Common Data Issues
 - Empty Query Results
 - KeyError: `__ID__`
 - Duplicate ID Warning
 - Missing Relationship Columns
- Quick Reference
- Next Steps

5.1.1 Why Does PyCypher Need `__ID__` Columns?

If you’ve tried loading data and been confused by `__ID__`, `__SOURCE__`, or `__TARGET__` — you’re not alone. Here’s what they are and why they exist.

The Short Answer

PyCypher executes graph queries against tabular data. Graph queries traverse connections between things (MATCH (a)-[:KNOWS]->(b)). For that to work, every row needs a unique identity so relationships can reference it.

- `__ID__` — uniquely identifies each entity (node) within its type
- `__SOURCE__` — which entity a relationship starts from
- `__TARGET__` — which entity a relationship points to

These are the structural glue that turns flat tables into a queryable graph.

Note

You do not need to rename your columns. PyCypher accepts your existing column names via the `id_col`, `source_col`, and `target_col` parameters. The `__ID__`/`__SOURCE__`/`__TARGET__` names are internal — PyCypher renames them for you.

Why Not Just Use Row Position?

You might wonder: “Why can’t PyCypher just use row 0, row 1, row 2?”

1. Relationships need stable references. If you say “Person 42 KNOWS Person 17”, you need those IDs to look up the actual rows. Row position can change if the data is filtered, sorted, or reloaded.
2. Cross-type joins. When a relationship connects two different entity types (e.g., Person-[:WORKS_AT]-> Company), the source and target IDs come from different tables. Row positions would be meaningless across tables.
3. Mutation correctness. SET, CREATE, MERGE, and DELETE operations must track which exact row to update. A stable identity column guarantees this.
4. Performance. During query execution, PyCypher stores only ID columns in intermediate results (BindingFrames). Full row data is fetched on-demand only when needed — typically at RETURN or WHERE. This dramatically reduces memory usage and speeds up multi-hop traversals.

5.1.2 Using Your Existing Column Names

The Most Common Pattern

You already have a `customer_id` column. Don't rename it — tell PyCypher which column to use:

```
import pandas as pd
from pycypher.ingestion import ContextBuilder
from pycypher import Star

# Your existing data — no __ID__ column needed
customers = pd.DataFrame({
    "customer_id": [101, 102, 103],
    "name": ["Alice", "Bob", "Carol"],
    "email": ["alice@example.com", "bob@example.com", "carol@example.com"],
})

orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003, 1004],
    "customer_id": [101, 102, 101, 103],
    "product": ["Widget", "Gadget", "Gizmo", "Widget"],
    "amount": [29.99, 49.99, 19.99, 29.99],
})

context = (
    ContextBuilder()
    .add_entity("Customer", customers, id_col="customer_id")
    .add_entity("Order", orders, id_col="order_id")
    .add_relationship(
        "PLACED",
        orders,
        source_col="customer_id",  # which customer placed it
        target_col="order_id",    # which order was placed
    )
    .build()
)

star = Star(context=context)
result = star.execute_query(
    "MATCH (c:Customer)-[:PLACED]->(o:Order) "
    "RETURN c.name, o.product, o.amount"
)
```

Behind the scenes, PyCypher renames `customer_id` → `__ID__` in the Customer table, `order_id` → `__ID__` in the Order table, and `customer_id` → `__SOURCE__` / `order_id` → `__TARGET__` in the relationship table. Your original column names remain accessible as properties (e.g., `c.customer_id` still works in queries).

From the CLI

The `nmetl` query command accepts the same column mapping:

```
# Entity: Label=path[:id_col]
nmetl query "MATCH (c:Customer) RETURN c.name" \
  --entity "Customer=customers.csv:customer_id"
```

(continues on next page)

(continued from previous page)

```
# Relationship: Type=path:source_col:target_col
nmetl query "MATCH (c:Customer)-[:PLACED]->(o:Order) RETURN c.name, o.product" \
  --entity "Customer=customers.csv:customer_id" \
  --entity "Order=orders.csv:order_id" \
  --rel "PLACED=orders.csv:customer_id:order_id"
```

In YAML Pipeline Config

```
sources:
  entities:
    - id: customers
      uri: data/customers.csv
      entity_type: Customer
      id_col: customer_id      # your column name

    - id: orders
      uri: data/orders.csv
      entity_type: Order
      id_col: order_id

  relationships:
    - id: placed
      uri: data/orders.csv
      relationship_type: PLACED
      source_col: customer_id # your column name
      target_col: order_id
```

When No ID Column Exists

If your data has no natural identifier, omit `id_col` and PyCypher auto-generates sequential integer IDs (0, 1, 2, ...):

```
# No id_col — PyCypher assigns __ID__ = 0, 1, 2, ...
products = pd.DataFrame({
  "name": ["Widget", "Gadget", "Gizmo"],
  "price": [9.99, 49.99, 19.99],
})
context = ContextBuilder().add_entity("Product", products).build()
```

This works for entities that are only queried by properties (not referenced by relationships). If you need to connect them via relationships, auto-generated IDs are positional and fragile — prefer explicit IDs.

5.1.3 Real-World Examples

Example 1: E-Commerce (Customer → Order → Product)

Starting data — three CSVs as they might come from a database export:

customers.csv:

```
customer_id,name,email,city
C001,Alice,alice@co.com,NYC
```

(continues on next page)

(continued from previous page)

```
C002,Bob,bob@co.com,LA
C003,Carol,carol@co.com,NYC
```

products.csv:

```
sku,title,price,category
SKU-100,Widget,29.99,Hardware
SKU-200,Gadget,49.99,Electronics
SKU-300,Gizmo,19.99,Hardware
```

orders.csv:

```
order_id,customer_id,sku,quantity,order_date
ORD-1,C001,SKU-100,2,2024-01-15
ORD-2,C002,SKU-200,1,2024-01-16
ORD-3,C001,SKU-300,3,2024-01-17
ORD-4,C003,SKU-100,1,2024-01-18
```

Loading into PyCypher:

```
context = (
    ContextBuilder()
    .add_entity("Customer", "customers.csv", id_col="customer_id")
    .add_entity("Product", "products.csv", id_col="sku")
    .add_relationship(
        "ORDERED",
        "orders.csv",
        source_col="customer_id",
        target_col="sku",
    )
    .build()
)

star = Star(context=context)

# "What hardware did NYC customers order?"
result = star.execute_query("""
    MATCH (c:Customer)-[:ORDERED]->(p:Product)
    WHERE c.city = 'NYC' AND p.category = 'Hardware'
    RETURN c.name AS customer, p.title AS product
""")
```

Example 2: Social Network (User follows User)

Same entity type on both sides of the relationship:

users.csv:

```
user_id,username,joined
1,alice,2023-01-01
2,bob,2023-02-15
3,carol,2023-03-10
4,dave,2023-04-20
```

follows.csv:

```
follower_id,followee_id,since
1,2,2023-03-01
1,3,2023-04-01
2,3,2023-05-01
3,4,2023-06-01
```

```
context = (
    ContextBuilder()
    .add_entity("User", "users.csv", id_col="user_id")
    .add_relationship(
        "FOLLOWS",
        "follows.csv",
        source_col="follower_id",
        target_col="followee_id",
    )
    .build()
)

star = Star(context=context)

# "Who does Alice follow, and who do they follow?"
result = star.execute_query("""
    MATCH (a:User {username: 'alice'})-[:FOLLOWS]->(b:User)-[:FOLLOWS]->(c:User)
    RETURN a.username AS user, b.username AS follows, c.username AS follows_of_follows
""")
```

Example 3: Multi-Entity (Person + Company + Employment)

```
import pandas as pd

people = pd.DataFrame({
    "emp_id": ["E001", "E002", "E003"],
    "name": ["Alice", "Bob", "Carol"],
    "department": ["Engineering", "Marketing", "Engineering"],
})

companies = pd.DataFrame({
    "company_id": ["ACME", "GLOB"],
    "name": ["Acme Corp", "Globex Inc"],
    "industry": ["Tech", "Finance"],
})

employment = pd.DataFrame({
    "emp_id": ["E001", "E002", "E003"],
    "company_id": ["ACME", "GLOB", "ACME"],
    "role": ["Senior Engineer", "Marketing Lead", "Junior Engineer"],
    "start_year": [2020, 2021, 2023],
})

context = (
    ContextBuilder()
```

(continues on next page)

(continued from previous page)

```

.add_entity("Person", people, id_col="emp_id")
.add_entity("Company", companies, id_col="company_id")
.add_relationship(
    "WORKS_AT",
    employment,
    source_col="emp_id",
    target_col="company_id",
)
.build()
)

star = Star(context=context)

# "Who works at Acme Corp, and what's their role?"
result = star.execute_query("""
    MATCH (p:Person)-[r:WORKS_AT]->(c:Company {name: 'Acme Corp'})
    RETURN p.name AS employee, r.role AS role, r.start_year AS since
""")

```

5.1.4 Data Preparation Guide

Converting Existing Tables

Most real-world data needs minimal preparation. The key question is: which column uniquely identifies each row?

Source data	ID column	PyCypher setup
Database table with primary key	The PK column	<code>id_col="pk_column"</code>
CSV with a unique name/code	The unique column	<code>id_col="name"</code> or <code>id_col="code"</code>
CSV with no unique column	Omit <code>id_col</code>	Auto-generated sequential IDs
Data with composite keys	Create a combined column	See below

Handling Composite Keys

If uniqueness comes from multiple columns (e.g., (year, department)), create a combined ID before loading:

```

df = pd.DataFrame({
    "year": [2023, 2023, 2024],
    "dept": ["Eng", "Sales", "Eng"],
    "budget": [100000, 80000, 120000],
})

# Create a composite ID
df["budget_id"] = df["year"].astype(str) + "_" + df["dept"]

context = ContextBuilder().add_entity("Budget", df, id_col="budget_id").build()

```

ID Generation Strategies

When your data has no natural identifier:

```
import uuid

df = pd.DataFrame({
    "event": ["login", "click", "purchase"],
    "timestamp": ["2024-01-01 10:00", "2024-01-01 10:05", "2024-01-01 10:12"],
})

# Option 1: Sequential integers (simplest)
df["id"] = range(len(df))

# Option 2: UUIDs (globally unique)
df["id"] = [str(uuid.uuid4()) for _ in range(len(df))]

# Option 3: Hash of content (deterministic)
df["id"] = df.apply(lambda r: f"{r['event']}_{r['timestamp']}", axis=1)

context = ContextBuilder().add_entity("Event", df, id_col="id").build()
```

Type Coercion

ID types must match between entity and relationship tables. If your entity has integer IDs but your relationship CSV loads them as strings, the join will produce zero results.

```
# Entity: integer IDs
people = pd.DataFrame({"person_id": [1, 2, 3], "name": ["A", "B", "C"]})

# Relationship: string IDs (common after CSV loading)
knows = pd.DataFrame({"from": ["1", "2"], "to": ["2", "3"]})

# Fix: convert to matching types before loading
knows["from"] = knows["from"].astype(int)
knows["to"] = knows["to"].astype(int)
```

5.1.5 Troubleshooting Common Data Issues

Empty Query Results

Symptom: `MATCH (a)-[:REL]->(b)` returns an empty DataFrame.

Most likely cause: ID type mismatch between entity and relationship.

```
# Diagnose: check the dtypes
print(entity_df["customer_id"].dtype)    # int64
print(relationship_df["customer_id"].dtype) # object (string!)

# Fix: align the types
relationship_df["customer_id"] = relationship_df["customer_id"].astype(int)
```

Other causes:

- Relationship `source_col`/target_col point to wrong columns

- IDs in the relationship don't exist in the entity table (dangling references)

KeyError: `__ID__`

Symptom: KeyError: '`__ID__`' when loading data.

Cause: Using `from_dict()` or `from_dataframe()` without providing an `__ID__` column or `id_col` parameter.

```
# This fails — no __ID__ and no id_col
df = pd.DataFrame({"name": ["Alice", "Bob"]})
table = EntityTable.from_dataframe("Person", df) # KeyError!

# Fix option 1: add __ID__ column
df["__ID__"] = [1, 2]

# Fix option 2: use ContextBuilder with id_col
context = ContextBuilder().add_entity("Person", df, id_col="name").build()

# Fix option 3: let ContextBuilder auto-generate IDs (omit id_col)
context = ContextBuilder().add_entity("Person", df).build()
```

Duplicate ID Warning

Symptom: MATCH returns unexpected extra rows (cross-join behavior).

Cause: Multiple rows share the same `__ID__` within an entity type. Each duplicate multiplies the result rows.

```
# Diagnose: check for duplicates
dups = df[df["customer_id"].duplicated(keep=False)]
print(f"Found {len(dups)} duplicate IDs")

# Fix: deduplicate before loading
df = df.drop_duplicates(subset=["customer_id"], keep="first")
```

Missing Relationship Columns

Symptom: ValueError or KeyError when adding a relationship.

Cause: The `source_col` or `target_col` name doesn't match any column in the data.

```
# Check actual column names
print(df.columns.tolist())
# ['cust_id', 'prod_id', 'qty'] — not "customer_id"!

# Fix: use the actual column names
context = ContextBuilder().add_relationship(
    "BOUGHT", df,
    source_col="cust_id",    # matches the actual column
    target_col="prod_id",
)
```


5.1.6 Quick Reference

Internal column	Your parameter	Purpose
__ID__	id_col="your_column"	Unique identifier for each entity row
__SOURCE__	source_col="your_column"	Which entity a relationship starts from
__TARGET__	target_col="your_column"	Which entity a relationship points to

Rules:

- Entity IDs must be unique within each entity type
- Relationship source/target values must match entity IDs (same type)
- If id_col is omitted, sequential integers are auto-generated
- source_col and target_col are required for relationships

5.1.7 Next Steps

- [Graph Modeling](#) — graph design patterns and best practices
- [Data ETL Pipeline](#) — building full ETL pipelines
- [Error Handling Patterns](#) — understanding PyCypher error messages

5.2 Variables in PyCypher

5.2.1 Understanding Variable Representation

As of the latest version, PyCypher uses a consistent Variable class throughout the AST to represent all variable references. This ensures type safety and makes variable tracking more robust.

5.2.2 The Variable Class

```
from pycypher.ast_models import Variable

# Create a variable
var = Variable(name="person")

# Access the name
print(var.name) # "person"
```

5.2.3 Where Variables Are Used

Variables appear in many AST node types:

Node Patterns

```
from pycypher.ast_models import NodePattern, Variable

# Node pattern with variable
node = NodePattern(
    variable=Variable(name="n"),
```

(continues on next page)

(continued from previous page)

```

labels=["Person"],
properties=None
)

```

Relationship Patterns

```

from pycypher.ast_models import RelationshipPattern, Variable

# Relationship pattern with variable
rel = RelationshipPattern(
    variable=Variable(name="r"),
    types=["KNOWS"],
    properties=None,
    direction="outgoing"
)

```

Property Lookups

```

from pycypher.ast_models import PropertyLookup, Variable

# Property lookup on a variable
prop = PropertyLookup(
    expression=Variable(name="person"),
    property_name="age"
)

```

List Comprehensions

```

from pycypher.ast_models import ListComprehension, Variable

comp = ListComprehension(
    variable=Variable(name="x"),
    list_expression=some_list,
    filter_expression=some_filter,
    map_expression=some_map
)

```

5.2.4 Migration from String Variables

If you have code that previously used string variables, you need to update it:

Old code:

```

node = NodePattern(
    variable="n", # String
    labels=["Person"]
)

```

New code:

```
from pycypher.ast_models import Variable

node = NodePattern(
    variable=Variable(name="n"), # Variable instance
    labels=["Person"]
)
```

5.2.5 Benefits

Using Variable instances provides several advantages:

1. Type Safety: Pydantic validates that variables are proper Variable instances
2. Consistency: Same representation everywhere in the AST
3. Extensibility: Can add metadata to variables (scope, type, etc.)
4. Better Validation: Can track variable definitions and references more easily

5.2.6 See Also

- [PyCypher API](#) - Complete API reference for Variable class
- [Basic Query Execution](#) - Tutorial on working with variables

5.3 AST Nodes

Complete reference for all AST node types in PyCypher.

5.3.1 Overview

PyCypher uses strongly-typed AST nodes based on Pydantic models. All AST nodes inherit from the `ASTNode` base class and provide validation, serialization, and tree traversal capabilities.

5.3.2 Node Hierarchy

The AST follows the openCypher grammar structure:

- Query - Top-level query container
- Clause - MATCH, CREATE, RETURN, WITH, WHERE, etc.
- Expression - All expressions (arithmetic, comparison, literals, etc.)
- Pattern - Graph patterns for matching and creation
- Literal - Literal values (integers, strings, booleans, lists, maps)

5.3.3 Core Node Types

Query

The root node for all Cypher queries:

```
from pycypher.ast_models import Query, Match, Return

# A Query contains clauses
query = Query(clauses=[
```

(continues on next page)

(continued from previous page)

```

    Match(...),
    Return(...)
])

```

Clauses

Match Clause

```

from pycypher.ast_models import Match, Pattern, NodePattern, Variable

# MATCH (n:Person)
match = Match(
    pattern=Pattern(
        pattern_paths=[
            PatternPath(
                node_pattern=NodePattern(
                    variable=Variable(name="n"),
                    labels=["Person"]
                )
            )
        ]
    )
)

```

Return Clause

```

from pycypher.ast_models import Return, ReturnBody, ReturnItem, Variable

# RETURN n.name
return_clause = Return(
    return_body=ReturnBody(
        return_items=[
            ReturnItem(
                expression=PropertyLookup(
                    expression=Variable(name="n"),
                    property_name="name"
                )
            )
        ]
    )
)

```

Expressions

Comparison

```

from pycypher.ast_models import Comparison, Variable, IntegerLiteral

# n.age > 30
comparison = Comparison(
    operator=">",
    left=PropertyLookup(

```

(continues on next page)

(continued from previous page)

```

        expression=Variable(name="n"),
        property_name="age"
    ),
    right=IntegerLiteral(value=30)
)

```

Arithmetic

```

from pycypher.ast_models import Arithmetic

# a + b
addition = Arithmetic(
    operator="+",
    left=Variable(name="a"),
    right=Variable(name="b")
)

```

List Comprehension

```

from pycypher.ast_models import ListComprehension

# [x IN items WHERE x > 5 | x * 2]
list_comp = ListComprehension(
    variable=Variable(name="x"),
    list_expr=Variable(name="items"),
    where=Comparison(
        operator=">",
        left=Variable(name="x"),
        right=IntegerLiteral(value=5)
    ),
    map_expr=Arithmetic(
        operator="*",
        left=Variable(name="x"),
        right=IntegerLiteral(value=2)
    )
)

```

Patterns

Node Pattern

```

from pycypher.ast_models import NodePattern, Variable, MapLiteral

# (person:Person {name: 'Alice'})
node = NodePattern(
    variable=Variable(name="person"),
    labels=["Person"],
    properties=MapLiteral(
        entries={"name": StringLiteral(value="Alice")}
    )
)

```

Relationship Pattern

```

from pycypher.ast_models import RelationshipPattern

# -[:KNOWS {since: 2020}]->
rel = RelationshipPattern(
    variable=Variable(name="r"),
    types=["KNOWS"],
    properties=MapLiteral(
        entries={"since": IntegerLiteral(value=2020)}
    ),
    direction="OUTGOING"
)

```

Literals

All literal types:

```

from pycypher.ast_models import (
    IntegerLiteral,
    FloatLiteral,
    StringLiteral,
    BooleanLiteral,
    NullLiteral,
    ListLiteral,
    MapLiteral
)

# Integer
int_lit = IntegerLiteral(value=42)

# String
str_lit = StringLiteral(value="Hello")

# Boolean
bool_lit = BooleanLiteral(value=True)

# Null
null_lit = NullLiteral()

# List
list_lit = ListLiteral(
    elements=[
        IntegerLiteral(value=1),
        IntegerLiteral(value=2)
    ]
)

# Map
map_lit = MapLiteral(
    entries={
        "key1": StringLiteral(value="value1"),
        "key2": IntegerLiteral(value=123)
    }
)

```

Variables

All variable references use the Variable class:

```
from pycypher.ast_models import Variable

# Create a variable
var = Variable(name="person")

# Variables are used throughout the AST
# - In patterns
# - In expressions
# - In comprehensions
# - In return items
```

5.3.4 Validation

All AST nodes support validation:

```
from pycypher.grammar_parser import GrammarParser
from pycypher.ast_models import ASTConverter

parser = GrammarParser()
converter = ASTConverter()

query = "MATCH (n:Person) WHERE n.age > 30 RETURN n.name"
raw_ast = parser.parse_to_ast(query)
typed_ast = converter.convert(raw_ast)

# Validate the AST
result = typed_ast.validate()

if result.is_valid:
    print("Query is valid!")
else:
    for issue in result.issues:
        print(f"Validation issue: {issue}")
```

5.3.5 Tree Traversal

AST nodes inherit from TreeMixin providing tree traversal methods:

```
# Find all variables in the AST
variables = typed_ast.find_all(Variable)

# Find all node patterns
node_patterns = typed_ast.find_all(NodePattern)

# Get all children of a node
children = node.children()
```

5.3.6 Pydantic Integration

All AST nodes are Pydantic models:

```
# Serialize to dict
node_dict = node.model_dump()

# Serialize to JSON
node_json = node.model_dump_json()

# Load from dict
node = NodePattern(**node_dict)

# Validation is automatic
try:
    invalid_node = NodePattern(variable="not a variable")
except ValidationError as e:
    print(f"Validation error: {e}")
```

5.3.7 For More Information

- See [PyCypher API](#) for complete API reference
- See [AST Manipulation](#) for examples
- See [Variables in PyCypher](#) for details on variable handling

5.4 Query Processing

Understanding query parsing, validation, and execution in PyCypher.

5.4.1 Overview

PyCypher processes Cypher queries through a multi-stage pipeline:

1. Parsing: Convert Cypher text to a raw AST using the Lark grammar
2. Conversion: Transform the raw AST to typed Pydantic models via `ASTConverter`
3. Validation: Check query semantics and structure (via `SemanticValidator`)
4. Execution: Translate and execute against a `BindingFrame` IR

5.4.2 Parsing Pipeline

Raw Parsing

The `GrammarParser` uses Lark to parse Cypher queries:

```
from pycypher.grammar_parser import GrammarParser

parser = GrammarParser()

query = """
MATCH (person:Person)-[:KNOWS]->(friend)
WHERE person.age > 30
```

(continues on next page)

(continued from previous page)

```

RETURN person.name, friend.name
"""

# Parse to raw AST (nested dicts and lists)
raw_ast = parser.parse_to_ast(query)
print(type(raw_ast)) # dict

```

The raw AST is a nested structure of dictionaries matching the Lark grammar.

AST Conversion

The ASTConverter transforms raw AST to typed nodes:

```

from pycypher.ast_models import ASTConverter

converter = ASTConverter()
typed_ast = converter.convert(raw_ast)

# Now we have strongly-typed Pydantic models
print(type(typed_ast)) # Query
print(type(typed_ast.clauses[0])) # Match

```

Benefits of typed AST:

- Type safety with mypy/ty
- Automatic validation
- IDE autocomplete
- Serialization support
- Tree traversal methods

5.4.3 Validation

Semantic Validation

The SemanticValidator checks query correctness:

```

from pycypher.semantic_validator import SemanticValidator

validator = SemanticValidator()
result = validator.validate(typed_ast)

if result.is_valid:
    print("Query is valid!")
else:
    for issue in result.issues:
        print(f"Issue: {issue.message}")
        print(f"Location: {issue.location}")
        print(f"Severity: {issue.severity}")

```

What gets validated:

- Variable scoping and references

- Type compatibility
- Required vs optional clauses
- Pattern structure
- Function signatures
- Aggregation rules

For the simplest use case, the top-level `validate_query()` convenience function parses and validates in a single call:

```
from pycypher import validate_query

errors = validate_query("MATCH (n:Person) RETURN m")
for error in errors:
    print(error)
```

See the [Query Validation](#) tutorial for a detailed walkthrough.

5.4.4 Query Execution

The `Star` class parses, translates, and executes a Cypher query against a populated `Context` in a single call:

```
from pycypher.relational_models import Context
from pycypher.star import Star

star = Star(context=context)

# Execute a complete query and get a pandas DataFrame
result = star.execute_query("MATCH (p:Person) RETURN p.name AS name")
```

BindingFrame IR

The execution engine uses a `BindingFrame` as its internal representation. A `BindingFrame` is a `pd.DataFrame` whose columns are Cypher variable names (e.g. `"p"`, `"r"`) and whose values are entity IDs. Attributes are never stored in the frame — they are fetched on demand from the `Context`.

```
from pycypher.binding_frame import BindingFrame, EntityScan

# EntityScan produces a frame with one column per variable
scan = EntityScan(entity_type="Person", var_name="p")
frame = scan.scan(context)

# Fetch an attribute — ID-keyed lookup against the entity table
names = frame.get_property("p", "name")
```

This design eliminates three sources of fragility in earlier approaches:

1. Opaque 32-hex `HASH_ID` column names
2. `EntityType__propertyname` column prefixes bleeding into operator logic
3. `variable_map` metadata that had to be threaded through every operator

5.4.5 Expression Evaluation

The `BindingExpressionEvaluator` computes expression values vectorially against a `BindingFrame`. It returns a `pd.Series` aligned with the frame:

```
from pycypher.binding_evaluator import BindingExpressionEvaluator
from pycypher.ast_models import Arithmetic, IntegerLiteral

evaluator = BindingExpressionEvaluator(frame)

expr = Arithmetic(
    operator="+",
    left=IntegerLiteral(value=2),
    right=IntegerLiteral(value=3),
)
result_series = evaluator.evaluate(expr)
print(result_series.iloc[0]) # 5
```

Supported expression types:

- Arithmetic: +, -, *, /, %, ^
- Comparison: =, <>, <, <=, >, >=
- Logical: AND, OR, NOT, XOR (Kleene three-valued logic, fully vectorised)
- Null checks: IS NULL, IS NOT NULL
- Label predicates: `n:Label`, `n:Label1:Label2` — test whether a node variable belongs to a specific entity type. The compound form (colon-separated labels) applies AND semantics: `n:Person:Employee` is true only if the node has both labels simultaneously. Fully composable with AND/OR/NOT: `WHERE n:Person AND NOT n:Manager`.
- String predicates: `STARTS WITH`, `ENDS WITH`, `CONTAINS`, `=~` (regex), `IN`. When used with a non-string left-hand operand (integer, float, boolean), PyCypher raises a `TypeError` identifying the operator, the actual type, and a suggested remedy:

```
# p.age is integer — raises TypeError:
# "Operator 'STARTS WITH' requires a string left-hand operand,
# but got 'int64'. Use toString() to convert if needed."
WHERE p.age STARTS WITH '3'

# Fix:
WHERE toString(p.age) STARTS WITH '3'
```

All-null columns pass through without error (null IS NOT a non-string value).

- CASE expressions (both searched and simple, vectorised)
- List index access: `list[i]`
- List slicing: `list[start..end]`
- List comprehensions: `[x IN list WHERE cond | expr]` — filter and/or transform a list per row. The `WHERE` and `map` expression are each evaluated once over all (row, element) pairs (vectorised).
- Pattern comprehensions: `[(p)-[:REL]->(q) WHERE cond | expr]` — collect graph neighbours into a per-row list. Evaluated via a single `pd.merge` across all anchor rows (vectorised).

- Quantifier predicates: `any(x IN list WHERE cond)`, `all(x IN list WHERE cond)`, `none(x IN list WHERE cond)`, `single(x IN list WHERE cond)` — return a Boolean per row, vectorised via the same explode-evaluate-regroup pipeline as list comprehensions.
- EXISTS subqueries: `EXISTS { MATCH (p)-[:REL]->(q) WHERE cond }` — returns true if the inner pattern matches at least one row. The subquery is executed once for the entire outer frame using a sentinel-column batch technique (not once per row).
- REDUCE expressions: `reduce(acc = init, x IN list | step)` — fold a list into a scalar using an accumulator. The step expression is evaluated once per step position, batched across all active rows (rows whose list has not yet been exhausted). The accumulator dependency within a single row remains sequential, but rows at the same step position are evaluated together in one `BindingFrame`, reducing allocations from $O(\text{rows} \times \text{max_items})$ to $O(\text{max_items})$.
- Scalar functions: via `ScalarFunctionRegistry` — 131 functions across string (`toUpper`, `toLower`, `trim`, `substring`, `startsWith`, `endsWith`, `contains`, `byteSize`, ...), math (`abs`, `ceil`, `floor`, `sqrt`, `log`, `log10`, `pow`, `hypot`, `fmod`, `gcd`, `lcm`, ...), bitwise (`bitAnd`, `bitOr`, `bitXor`, `bitNot`, `bitShiftLeft`, `bitShiftRight`), trigonometric (`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `sinh`, `cosh`, `tanh`, ...), angle conversion (`degrees`, `radians`), constants (`pi`, `e`), list (`head`, `last`, `tail`, `range`, `toList`, ...), type predicates (`isString`, `isInteger`, `isFloat`, ...), temporal (`date`, `datetime`, `localdatetime`, `duration`, `date.truncate`, `datetime.truncate`, `localdatetime.truncate`, ...), hash & encoding (`md5`, `sha256`, `encodeBase64`, ...), and utility (`coalesce`, `toString`, `toInteger`, `randomUUID`, ...). Math, trig, bitwise, and string predicate functions are fully vectorised using numpy or pandas str operations — no per-row Python overhead for `abs(n.score)`, `sin(n.angle)`, `bitAnd(n.flags, 0xFF)`, or `startsWith(n.name, 'A')`. `round()` supports an optional third argument to select any of the seven Neo4j 5.x rounding modes: `HALF_UP` (default, ties away from zero), `HALF_DOWN` (ties toward zero), `HALF_EVEN` (banker's rounding), `CEILING`, `FLOOR`, `UP` (always away from zero), `DOWN` (truncation toward zero).

5.4.6 Cross-Product MATCH

When two `MATCH` patterns share no common variables, pycypher produces the Cartesian product (SQL `CROSS JOIN`) of both scans:

```
# All 3×2=6 (person, product) pairs
result = star.execute_query(
    "MATCH (p:Person), (pr:Product) "
    "RETURN p.name AS person, pr.title AS product"
)

# Filter: who can afford what?
result = star.execute_query(
    "MATCH (p:Person), (pr:Product) "
    "WHERE p.budget >= pr.price "
    "RETURN p.name AS person, pr.title AS product"
)

# Self cross-join: all ordered (A, B) person pairs where A < B
result = star.execute_query(
    "MATCH (p:Person), (q:Person) "
    "WHERE p.name <> q.name "
    "RETURN p.name AS pname, q.name AS qname"
)
```

Aggregation

Aggregation functions (count, sum, avg, min, max, collect) are evaluated by `evaluate_aggregation()`, which returns a scalar suitable for use in a `WITH` or `RETURN` clause:

```
# Full-table aggregation (no GROUP BY)
result = star.execute_query(
    "MATCH (p:Person) RETURN count(*) AS n"
)

# Grouped aggregation
result = star.execute_query(
    "MATCH (p:Person) RETURN p.dept AS dept, count(p) AS n"
)
```

The `DISTINCT` modifier is supported for all aggregation functions:

```
# Count unique departments (not total people)
result = star.execute_query(
    "MATCH (p:Person) RETURN count(DISTINCT p.dept) AS dept_count"
)

# Collect unique scores only
result = star.execute_query(
    "MATCH (p:Person) RETURN collect(DISTINCT p.score) AS unique_scores"
)

# Sum of unique scores
result = star.execute_query(
    "MATCH (p:Person) RETURN sum(DISTINCT p.score) AS total"
)
```

5.4.7 Projection Modifiers: ORDER BY, LIMIT, SKIP, DISTINCT

`RETURN` and `WITH` both support the standard Cypher projection modifiers. They are applied after aggregation in the following order: `DISTINCT` → `ORDER BY` → `SKIP` → `LIMIT`.

```
# Sort by age descending, return the top 3
result = star.execute_query(
    "MATCH (p:Person) RETURN p.name, p.age ORDER BY p.age DESC LIMIT 3"
)

# Remove duplicate departments
result = star.execute_query(
    "MATCH (p:Person) RETURN DISTINCT p.dept"
)

# Paginate: skip the first 10 rows, return the next 5
result = star.execute_query(
    "MATCH (p:Person) RETURN p.name SKIP 10 LIMIT 5"
)

# WITH pipeline: sort and cap rows before the next stage
result = star.execute_query(
```

(continues on next page)

(continued from previous page)

```

"MATCH (p:Person) "
"WITH p.name AS name, p.age AS age ORDER BY age ASC LIMIT 1 "
"RETURN name, age"
)

```

ORDER BY expressions may reference any property of the matched nodes, including properties that are not in the RETURN item list. If an ORDER BY expression cannot be evaluated against the projected columns it is automatically evaluated against the pre-projection binding frame.

Null placement — By default PyCypher matches Neo4j 5.x semantics: nulls sort last for both ASC and DESC. Use the optional NULLS FIRST / NULLS LAST suffix to override this per sort key:

```

# Put rows with null scores first, then ascending non-nulls
result = star.execute_query(
    "MATCH (n:Person) RETURN n.name, n.score "
    "ORDER BY n.score ASC NULLS FIRST"
)

# Descending by score; explicit NULLS LAST (same as default)
result = star.execute_query(
    "MATCH (n:Person) RETURN n.name, n.score "
    "ORDER BY n.score DESC NULLS LAST"
)

# Multi-column sort with mixed null placement
result = star.execute_query(
    "MATCH (n:Item) RETURN n.val, n.tag "
    "ORDER BY n.val ASC NULLS FIRST, n.tag ASC NULLS LAST"
)

```

Each sort key carries its own NULLS directive independently. Without a NULLS keyword the default is NULLS LAST (Neo4j 5.x default).

Auto-generated Column Names

When a RETURN item has no explicit AS alias, PyCypher infers a display name from the expression (matching Neo4j's behaviour):

- RETURN p.name → column name (unqualified property name)
- RETURN p → column p (variable name)
- RETURN toUpper(p.name) → column toUpper(name)
- RETURN p.age + 1 → column age + 1
- RETURN 42 → column 42
- RETURN true → column true
- RETURN p.age > 25 → column age > 25 (comparison)
- RETURN p.name STARTS WITH 'A' → column name STARTS WITH A
- RETURN p.name IS NULL → column name IS NULL
- RETURN NOT p.active → column NOT active
- RETURN p.age > 18 AND p.active → column age > 18 AND active

- RETURN -p.age → column -age (unary negation)
- RETURN \$limit → column \$limit (query parameter)
- RETURN count(*) → column count(*)
- RETURN xs[0] → column xs[0] (index lookup)
- RETURN xs[1..3] → column xs[1..3] (slicing)
- RETURN [x IN xs | x * 2] → column [x IN xs | x * 2] (list comprehension)
- RETURN CASE WHEN ... END → column case
- RETURN reduce(s = 0, x IN ns | s + x) → column reduce(s, ns)

Two or more aliasless items always receive distinct column names:

```
result = star.execute_query(
    "MATCH (p:Person) RETURN toUpper(p.name), p.age + 1"
)
# Columns: ['toUpper(name)', 'age + 1']

result = star.execute_query(
    "MATCH (p:Person) RETURN p.age > 30, p.name IS NOT NULL"
)
# Columns: ['age > 30', 'name IS NOT NULL']
```

An explicit AS alias always overrides the auto-generated name.

5.4.8 OPTIONAL MATCH — Left-Join Semantics

OPTIONAL MATCH is the Cypher analogue of a SQL LEFT JOIN. Rows from the preceding MATCH are always preserved; when the optional pattern finds no match, the unbound variables take null values.

```
# Everyone returned; fname is null for those with no KNOWS edge
result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:KNOWS]->(f:Person)
    WITH p.name AS pname, f.name AS fname
    RETURN pname, fname
    """
)

# Filter to those who have no outgoing KNOWS relationship
result = star.execute_query(
    """
    MATCH (p:Person)
    OPTIONAL MATCH (p)-[:KNOWS]->(f:Person)
    WITH p.name AS pname, f.name AS fname
    WHERE fname IS NULL
    RETURN pname
    """
)

# Combine with coalesce for null-safe display
result = star.execute_query(
```

(continues on next page)

(continued from previous page)

```

"""
MATCH (p:Person)
OPTIONAL MATCH (p)-[:KNOWS]->(f:Person)
WITH p.name AS pname, coalesce(f.name, 'nobody') AS friend
RETURN pname, friend
"""
)

```

When `OPTIONAL MATCH` appears as the first clause of a query (no preceding `MATCH`), it behaves like a regular `MATCH` except that an unknown entity type produces 0 rows instead of raising an error.

5.4.9 UNWIND — List Explosion

`UNWIND` takes a list expression and produces one row per list element, binding each element to an alias variable. It is commonly used after `collect()` to re-expand aggregated lists:

```

# Collect names then unwind into individual rows
result = star.execute_query(
    """
    MATCH (p:Person)
    WITH collect(p.name) AS names
    UNWIND names AS name
    RETURN name
    """
)

# Explode a list property (one row per tag per person)
result = star.execute_query(
    """
    MATCH (p:Person)
    WITH p.name AS name, p.tags AS tags
    UNWIND tags AS tag
    RETURN name, tag
    """
)

# Filter after UNWIND using WITH ... WHERE
result = star.execute_query(
    """
    MATCH (p:Person)
    WITH p.name AS name, p.tags AS tags
    UNWIND tags AS tag
    WITH name, tag WHERE tag = 'python'
    RETURN name, tag
    """
)

```

5.4.10 WITH * — Pass-Through Projection

`WITH *` passes all variables from the preceding `MATCH` (or earlier pipeline stage) into subsequent clauses unchanged. No explicit alias list is needed:


```
# Access any matched property after WITH *
result = star.execute_query(
    "MATCH (p:Person) WITH * RETURN p.name AS name, p.age AS age"
)

# Filter using WITH * WHERE
result = star.execute_query(
    "MATCH (p:Person) WITH * WHERE p.age > 30 RETURN p.name AS name"
)

# Sort and cap rows without enumerating aliases
result = star.execute_query(
    "MATCH (p:Person) WITH * ORDER BY p.age ASC LIMIT 3 RETURN p.name AS name"
)

# Chain: use WITH * to pass variables, then project explicitly in the next WITH
result = star.execute_query(
    "MATCH (p:Person) WITH * WITH p.name AS name, p.dept AS dept RETURN name, dept"
)
```

5.4.11 Variable-Length Paths

Relationship patterns support `*min..max` hop-depth syntax for reachability queries. The BFS traversal honours relationship direction:

```
# All paths of exactly 1–3 KNOWS hops
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*1..3]->(b:Person) RETURN a.name, b.name"
)

# Unbounded — any positive number of hops
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*]->(b:Person) RETURN a.name, b.name"
)

# Exactly N hops
result = star.execute_query(
    "MATCH (a:Person)-[:KNOWS*2..2]->(b:Person) RETURN a.name, b.name"
)
```

The `length()` function returns the number of relationships (hops) in a named path variable:

```
result = star.execute_query(
    "MATCH p = (a:Person)-[:KNOWS*1..4]->(b:Person) "
    "RETURN a.name, b.name, length(p) AS hops"
)
```

5.4.12 `shortestPath` and `allShortestPaths`

`shortestPath(pattern)` returns one row per (start, end) pair at the minimum hop count. `allShortestPaths(pattern)` returns every path that ties for the minimum — there may be more than one:

```
# One row per (start, end) pair: minimum hops
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})
    MATCH p = shortestPath((a)-[:KNOWS*]->(b))
    RETURN a.name AS start, b.name AS end, length(p) AS hops
    """
)

# All minimum-hop paths between the same pair
result = star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Dave'})
    MATCH p = allShortestPaths((a)-[:KNOWS*]->(b))
    RETURN a.name AS start, b.name AS end, length(p) AS hops
    """
)
```

Both functions return an empty result (not an error) when no path exists.

5.4.13 SET Clause and Atomicity

The SET clause writes property updates back to entity and relationship tables via `BindingFrame.mutate()`. All mutations within a query are buffered in a shadow layer and only promoted to the canonical tables on successful completion. If the query raises an exception, all pending mutations are discarded:

```
# Successful SET — changes persist for subsequent queries
star.execute_query("MATCH (p:Person) SET p.score = 99 RETURN p.name")

# Failed SET — context remains unchanged
try:
    star.execute_query(
        "MATCH (p:Person) SET p.name = 'Changed' RETURN nonexistent.x"
    )
except Exception:
    pass # Person.name is still the original value

# Set a relationship property
star.execute_query(
    "MATCH (a:Person)-[r:KNOWS]->(b:Person) SET r.since = 2020"
)
```

5.4.14 REMOVE Clause

REMOVE deletes a property from matching entities. Property deletion is also buffered and only committed on success:

```
# Remove a property from all matching nodes
star.execute_query(
    "MATCH (p:Person) WHERE p.temp IS NOT NULL REMOVE p.temp"
)

# Remove from a relationship
```

(continues on next page)

(continued from previous page)

```
star.execute_query(
    "MATCH (a:Person)-[r:KNOWS]->(b:Person) REMOVE r.since"
)
```

5.4.15 CREATE and MERGE Clauses

CREATE inserts new nodes and relationships. MERGE is an upsert: it first tries to match the pattern; if no match is found, it creates the pattern.

```
# Create a node
star.execute_query(
    "CREATE (p:Person {name: 'Eve', age: 27})"
)

# Create a relationship between matched nodes
star.execute_query(
    """
    MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Eve'})
    CREATE (a)-[:KNOWS]->(b)
    """
)

# MERGE with ON CREATE / ON MATCH actions
star.execute_query(
    """
    MERGE (p:Person {name: 'Alice'})
    ON CREATE SET p.created_at = timestamp()
    ON MATCH SET p.updated_at = timestamp()
    """
)
```

MERGE error semantics — when the entity type in a MERGE pattern is not registered in the context, MERGE interprets the absence as “no match” and creates the pattern. This is the expected Cypher upsert behaviour for bootstrapping new entity types. Internally, the engine catches only `GraphTypeNotFoundError` during the match phase; any other exception propagates normally so genuine bugs are never silently swallowed. `GraphTypeNotFoundError` is exported from the top-level `pycypher` package and can be caught directly:

```
from pycypher import GraphTypeNotFoundError

try:
    scan = EntityScan(entity_type="Ghost", var_name="g")
    scan.scan(context)
except GraphTypeNotFoundError as e:
    print(f"Entity type {e.type_name!r} is not in the context")
```

5.4.16 DELETE Clause

DELETE removes matched nodes from the entity table:

```
# Delete a specific node
star.execute_query(
```

(continues on next page)

(continued from previous page)

```
"MATCH (p:Person {name: 'Eve'}) DELETE p"
)
```

5.4.17 FOREACH Clause

FOREACH applies a write clause to every element of a list. The loop variable is scoped to the FOREACH body; variables in the outer query are visible but cannot be assigned:

```
# Activate every person in a collected list
star.execute_query(
    """
    MATCH (p:Person)
    WITH collect(p) AS people
    FOREACH (person IN people | SET person.active = true)
    """
)

# Create nodes from a literal list
star.execute_query(
    """
    FOREACH (name IN ['Alice', 'Bob', 'Carol'] |
        MERGE (p:Person {name: name})
    )
    """
)
```

5.4.18 UNION — Combining Results

UNION merges two or more sub-query result sets and deduplicates the combined rows. UNION ALL skips deduplication:

```
# Deduplicated union
result = star.execute_query(
    """
    MATCH (p:Person) WHERE p.dept = 'Eng' RETURN p.name AS name
    UNION
    MATCH (p:Person) WHERE p.age < 30 RETURN p.name AS name
    """
)

# Keep all rows
result = star.execute_query(
    """
    MATCH (p:Person) RETURN p.name AS name
    UNION ALL
    MATCH (p:Person) WHERE p.active = true RETURN p.name AS name
    """
)
```

All sub-queries in a UNION must project the same column names.

End-to-End Processing

Complete query processing example:

```
from pycypher.relational_models import Context
from pycypher.star import Star

context = Context() # populate with EntityTable / RelationshipTable objects

# Parse, translate, and execute in one call
star = Star(context=context)
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 30 RETURN p.name AS name"
)
# result is a pandas DataFrame with a 'name' column
```

5.4.19 Error Handling

Handling Parse Errors

Lark parse errors are raised as `lark.exceptions.UnexpectedInput` subclasses (`UnexpectedToken`, `UnexpectedCharacters`, `UnexpectedEOF`).

```
from lark.exceptions import UnexpectedInput

try:
    raw_ast = parser.parse_to_ast(invalid_query)
except UnexpectedInput as e:
    print(f"Parse error: {e}")
```

To check syntax without raising, use `validate()`:

```
parser = GrammarParser()

# Returns True for valid Cypher, False for genuine syntax errors.
# NOTE: internal transformer bugs (VisitError) are NOT caught and will
# propagate — this is intentional so bugs surface rather than being masked.
if parser.validate("MATCH (n:Person) RETURN n.name"):
    print("Valid query")
else:
    print("Syntax error in query")
```

5.4.20 Performance Considerations

Parse caching (automatic)

The Earley-based grammar parser is compiled once per process and cached as a singleton. AST parse results are additionally cached by query string using an LRU cache (capacity 512 entries). Identical query strings hit an $O(1)$ cache lookup on all calls after the first — no Cypher parsing overhead for repeated executions of the same query.

For an ETL pipeline executing 5 queries across 1 000 data batches this means roughly 5 cold parses (~56 ms each) rather than 5 000 (~280 s total). No manual caching is required.

Vectorised execution

- BindingFrame operations execute against pandas DataFrames using vectorised pandas operations. The following paths are fully vectorised (no per-row Python loops):
 - Boolean logic (AND, OR, NOT, XOR) — Kleene three-valued numpy operations.
 - Property lookups — two-level ID-keyed index cache:
 1. Arrow→pandas conversion cache — for ContextBuilder-loaded data (Arrow-backed tables), the `pyarrow.Table` → `pd.DataFrame` conversion is performed once per entity type and cached in `Context._property_lookup_cache`. Subsequent property lookups on the same entity type skip the Arrow conversion entirely.
 2. `set_index` cache — the `raw_df.set_index(ID_COLUMN)` indexed DataFrame is reused across all property accesses in the same query. `EntityScan.scan()` pre-warms the cache so all subsequent lookups in the same query are guaranteed hits.
 3. Cross-query persistence — the cache is retained across read-only queries; it is only cleared when a mutation (SET/CREATE/DELETE) is committed.

Combined effect: 8.4× speedup on repeated read-only queries vs. the uncached baseline (0.805s → 0.096s for 20 warm queries on a 2 000-row Arrow-backed context).
 - List comprehensions — all (row, element) pairs exploded into one flat DataFrame; WHERE and map expressions evaluated once each.
 - Pattern comprehensions — single `pd.merge` across all anchor rows; WHERE and map evaluated once over all pairs.
 - Quantifier predicates (any/all/none/single) — same explode pipeline as list comprehensions.
 - EXISTS subqueries — sentinel-column batch: the inner query executes once with all outer rows rather than once per row.
 - Grouped aggregations (count, sum, avg, min, max, stdev, stdevp) — pandas native Cython aggregation.
 - Scalar math functions (abs, ceil, floor, sign, sqrt, cbrt, log, log2, log10, exp, pow) — numpy C-level array operations via `np.abs`, `np.log`, `np.power`, etc.
 - Trigonometric functions (sin, cos, tan, asin, acos, atan, atan2, sinh, cosh, tanh, degrees, radians, cot, haversin) — numpy C-level.
 - String predicate functions (startsWith, endsWith, contains) — pandas `.str` accessor (Cython-level); pattern is always treated as a literal string, not a regex.
- `reduce(acc = init, x IN list | step)` uses a batch-per-step approach: all rows that are at the same step position are evaluated together in a single BindingFrame. Accumulator state within a row is sequential (step *i* uses the value from step *i*−1), but the step expression itself is only called $O(\text{max_items})$ times regardless of row count — 50 calls for 200 rows × 50-element lists instead of 10 000.

Scaling

- Performance scales linearly with the number of matching rows in a MATCH clause. For very large tables (>1 M rows), push selective filters as early as possible using WHERE clauses immediately after MATCH.
- Variable-length paths (`[[:REL*]]`) perform a BFS over the relationship table; execution time is proportional to the number of reachable edges. Bound hop limits (e.g. `*1..3`) significantly reduce the search space compared to unbounded `*`.

5.4.21 For More Information

- See [PyCypher API](#) for complete API reference
- See [Query Validation](#) for validation examples

5.5 Error Handling Patterns

PyCypher uses a structured exception hierarchy that lets you catch errors at the granularity you need. All exceptions are importable from the top-level `pycypher` package.

5.5.1 Exception Hierarchy

SyntaxError	
CypherSyntaxError	— invalid Cypher syntax
ValueError	
ASTConversionError	— grammar parsed but AST build failed
GrammarTransformerSyncError	— grammar/AST model mismatch
GraphTypeNotFoundError	— unknown entity label or relationship type
VariableNotFoundError	— variable not in scope
VariableTypeMismatchError	— variable exists but wrong type
UnsupportedFunctionError	— unknown function name
FunctionArgumentError	— wrong argument count
MissingParameterError	— query parameter not provided
InvalidCastError	— failed type cast (e.g. <code>toInteger("abc")</code>)
TypeError	
WrongCypherTypeError	— unexpected expression type
IncompatibleOperatorError	— operator/type mismatch
TemporalArithmeticError	— date/time arithmetic error
TimeoutError	
QueryTimeoutError	— query exceeded wall-clock budget
MemoryError	
QueryMemoryBudgetError	— estimated memory exceeds budget
RuntimeError	
RateLimitError	— query rate limit exceeded

Every custom exception inherits from a Python built-in, so existing except `ValueError` or except `TypeError` handlers continue to work without modification.

5.5.2 Catching Parse Errors

Use `CypherSyntaxError` for user-provided queries that might have typos:

```
from pycypher import Star, CypherSyntaxError

star = Star(context)
try:
    result = star.execute_query(user_query)
except CypherSyntaxError as e:
```

(continues on next page)

(continued from previous page)

```
print(f"Syntax error at line {e.line}, column {e.column}")
# e.query contains the original query string
```

5.5.3 Catching Runtime Errors

For dynamic queries, catch the specific exception types:

```
from pycypher import (
    Star,
    VariableNotFoundError,
    UnsupportedFunctionError,
    GraphTypeNotFoundError,
)

try:
    result = star.execute_query(query)
except VariableNotFoundError as e:
    # e.variable_name, e.available_variables, e.hint
    print(f"Unknown variable '{e.variable_name}'")
    print(f"Available: {e.available_variables}")
except UnsupportedFunctionError as e:
    # e.function_name, e.supported_functions, e.category
    print(f"No such function '{e.function_name}'")
except GraphTypeNotFoundError as e:
    # e.type_name
    print(f"No entity type '{e.type_name}' registered")
```

5.5.4 Resource Limit Errors

Protect against runaway queries in production:

```
from pycypher import Star, QueryTimeoutError, QueryMemoryBudgetError

try:
    result = star.execute_query(
        query,
        timeout_seconds=10.0,
        memory_budget_bytes=500 * 1024 * 1024, # 500 MB
    )
except QueryTimeoutError as e:
    print(f"Query timed out after {e.elapsed_seconds:.1f}s "
          f"(budget: {e.timeout_seconds}s)")
except QueryMemoryBudgetError as e:
    print(f"Estimated {e.estimated_bytes / 1e6:.0f}MB exceeds "
          f"budget {e.budget_bytes / 1e6:.0f}MB")
```

5.5.5 Rate Limit Errors

When rate limiting is enabled via `PYCYPHER_RATE_LIMIT_QPS`, queries that exceed the sustained rate raise `RateLimitError`:


```

from pycypher import RateLimitError

try:
    result = star.execute_query(query)
except RateLimitError:
    # Back off and retry, or return a 429 to the caller
    print("Rate limit exceeded — try again shortly")

```

5.5.6 Pre-execution Validation

Use `validate_query()` to catch errors before executing the query:

```

from pycypher import validate_query

errors = validate_query("MATCH (n:Person) RETURN m.name")
for error in errors:
    print(f"{error.severity.value}: {error.message}")
    # "error: Variable 'm' is not defined..."

```

5.5.7 Security and Pipeline Errors

`SecurityError` is raised when SQL injection, path traversal, or SSRF attempts are detected in data source URIs or DuckDB queries.

`CyclicDependencyError` is raised when a multi-query pipeline contains circular dependencies that prevent topological ordering.

5.5.8 Best Practices

1. Catch specific exceptions first, then broad ones:

```

try:
    result = star.execute_query(query)
except CypherSyntaxError:
    ... # user typo
except (VariableNotFoundError, GraphTypeNotFoundError):
    ... # schema mismatch
except (QueryTimeoutError, QueryMemoryBudgetError):
    ... # resource limits
except (TypeError, ValueError):
    ... # catch-all for remaining pycypher errors

```

2. Use structured attributes rather than parsing error messages. Every exception exposes the relevant data as named attributes.
3. Use pre-execution validation for user-submitted queries to give faster, friendlier feedback without the cost of execution.
4. Set timeouts in production — either via `timeout_seconds` parameter or the `PY-CYPHER_QUERY_TIMEOUT_S` environment variable.

5.5.9 Troubleshooting Quick Reference

Table 1: Common Errors and Fixes

Error	Likely Cause	Fix
CypherSyntaxError	Typo in Cypher keyword or missing clause	Check the line/column pointer in the error. Look for the “Did you mean ...?” suggestion. Ensure MATCH has a RETURN clause and all quotes/brackets are closed.
GraphTypeNotFoundError	Entity label or relationship type not loaded	Check e.available_types for registered types. Verify your --entity / --rel flags match the labels in your query.
VariableNotFoundError	Variable used in RETURN/WHERE not bound in MATCH	Check e.available_variables for what’s in scope. Ensure variables are defined in a MATCH or WITH clause before use. Look for the “Did you mean ...?” hint.
UnsupportedFunctionError	Function name not recognised	Check e.supported_functions for available functions. Function names are case-insensitive (count, COUNT, Count all work).
FunctionArgumentError	Wrong number of arguments to a function	Check e.expected_args vs e.actual_args. Refer to e.argument_description for the correct signature.
IncompatibleOperatorError	Operator applied to incompatible types (e.g. string + integer)	The error message includes a type-specific suggestion. Common fixes: use coalesce() for NULLs, toString()/toInteger() for type conversion.
QueryTimeoutError	Query exceeded wall-clock time budget	Add LIMIT to reduce result set. Add WHERE filters to reduce scan scope. Increase timeout_seconds if the query is genuinely large.
QueryMemoryBudgetError	Estimated memory exceeds configured budget	Add LIMIT/WHERE to reduce working set. Process in batches with SKIP/LIMIT. Increase memory_budget_bytes if resources allow.
QueryComplexityError	Query complexity score exceeds limit	Check e.breakdown for top contributors. Reduce MATCH clauses, shorten variable-length paths, add join conditions. Increase PYCYPHER_MAX_COMPLEXITY_SCORE if needed.
InvalidCastError	Type cast failed (e.g. toInteger("abc"))	Verify the source value is castable. Use CASE WHEN to handle non-castable values gracefully.
MissingParameterError	Parameterised query missing a \$param value	Pass the parameter: execute_query(query, parameters={'param': value}). The error message shows the exact usage.
CyclicDependencyError	Circular dependency in multi-query pipeline	Check e.remaining_nodes for the cycle. Break the cycle by splitting a query into separate read/write steps.
SecurityError	SQL injection, path traversal, or SSRF attempt detected	Review the data source URI or query for suspicious patterns. Use parameterised queries instead of string interpolation.
RateLimitError	Query rate limit exceeded	Back off and retry. Check e.qps and e.burst for current limits. Adjust PYCYPHER_RATE_LIMIT_QPS if needed.

Debugging Commands

When troubleshooting, these patterns help identify the root cause:

```
# Check available entity types and relationship types
print(context.entity_types())
print(context.relationship_types())

# Inspect DataFrame columns for property errors
for name, df in context.entities.items():
    print(f"{name}: {list(df.columns)}")

# Validate a query before executing it
from pycypher import validate_query
errors = validate_query(query_string)
for e in errors:
    print(f"{e.severity.value}: {e.message}")

# Profile a slow query
star = Star(context)
result = star.execute_query(query, profile=True)
print(star.explain_query(query))
```

5.6 Configuration Reference

PyCypher is configured entirely through environment variables and constructor parameters. All environment variables are read once at import time from `pycypher.config`.

5.6.1 Query Execution

Environment Variable	Default	Description
PYCYPHER_QUERY_TIMEOUT	None	Wall-clock timeout (seconds) for a single query. None means no limit. Can also be set per-query via <code>execute_query(timeout_seconds=...)</code> .
PYCYPHER_MAX_CROSS_JOIN	10,000,000	Hard ceiling on cross-join result size. Prevents accidental Cartesian explosions when MATCH patterns share no variables.
PYCYPHER_MAX_UNBOUNDED	20	Maximum BFS hops for unbounded variable-length paths ([*]). Higher values risk exponential memory growth in dense graphs.
PYCYPHER_MAX_COMPLEXITY	None	Hard ceiling on query complexity score. Queries exceeding this are rejected before execution with <code>QueryComplexityError</code> . None (or 0) disables the gate. Typical production values: 50–200. Can also be set per-query via <code>execute_query(max_complexity_score=...)</code> .
PYCYPHER_COMPLEXITY_WARN	None	Soft threshold for query complexity warnings. Queries scoring above this emit a warning but still execute. None (or 0) disables warnings. Set lower than <code>PYCYPHER_MAX_COMPLEXITY_SCORE</code> for early alerts.

5.6.2 Caching

Environment Variable	Default	Description
PYCYPHER_RESULT_CACHE_SIZE	100	Maximum memory (MB) for the LRU query result cache. Set to 0 to disable result caching.
PYCYPHER_RESULT_CACHE_TTL	0	Time-to-live (seconds) for cached results. 0 means entries never expire — only evicted by size pressure.
PYCYPHER_AST_CACHE_MAX	1024	Maximum parsed ASTs cached per GrammarParser instance. LRU eviction when full. 0 disables caching.

5.6.3 Logging and Metrics

Environment Variable	Default	Description
PYCYPHER_LOG_LEVEL	WARNING	Log level: DEBUG, INFO, WARNING, ERROR, CRITICAL.
PYCYPHER_LOG_FORMAT	rich	Log format: rich (human-readable) or json (machine-readable, suitable for ELK/Datadog/Splunk).
PYCYPHER_METRICS_ENABLED	1	Enable query metrics collection. Set to 0 or false to disable.
PYCYPHER_SLOW_QUERY_THRESHOLD_MS	1000	Threshold (milliseconds) for slow-query warnings in the metrics collector.

5.6.4 Telemetry

Environment Variable	Default	Description
PYROSCOPE_ENABLED	1	Enable Pyroscope continuous profiling. Set to 0 or false to disable. Requires pyroscope package.
PYROSCOPE_SERVER	http://localhost	Pyroscope server endpoint.
PYROSCOPE_APP_NAME	nmetl	Application name tag in Pyroscope.
PYROSCOPE_SAMPLE_RATE	100	Samples per second.

5.6.5 Per-Query Overrides

Many configuration options can be overridden per-query via `execute_query()` parameters:

```
result = star.execute_query(
    "MATCH (a)-[:KNOWS*]->(b) RETURN a, b",
    timeout_seconds=5.0,           # overrides PYCYPHER_QUERY_TIMEOUT_S
    memory_budget_bytes=256 * 1024 * 1024, # 256 MB hard limit
    max_complexity_score=100,      # reject overly complex queries
)
```

5.6.6 Production Configuration Example

```
# .env for production deployment
export PYCYPHER_QUERY_TIMEOUT_S=30
export PYCYPHER_MAX_CROSS_JOIN_ROWS=1000000
export PYCYPHER_MAX_UNBOUNDED_PATH_HOPS=10
export PYCYPHER_RESULT_CACHE_MAX_MB=500
export PYCYPHER_RESULT_CACHE_TTL_S=300
export PYCYPHER_LOG_LEVEL=INFO
export PYCYPHER_LOG_FORMAT=json
export PYCYPHER_SLOW_QUERY_MS=500
```

5.7 Performance Tuning

Practical strategies for optimising PyCypher query execution in production workloads — covering timeouts, memory budgets, caching, cross-join limits, and query structure best practices.

In this guide

- Environment Variables at a Glance
- Query Timeouts
- Memory Budget
- Cross-Join Limits
- Result Caching
 - Parse Caching
- Graph-Native Indexes
- LeapfrogTriejoin (Multi-Way Joins)
- Cardinality Feedback Loops
- Rate Limiting
- Audit Logging
- Query Structure Best Practices
 - Push Filters Early
 - Bound Variable-Length Paths
 - Use WITH to Pipeline Results
 - Prefer Relationship Patterns over Cross-Joins
- Monitoring and Diagnostics
 - Query Metrics
 - Query Execution Logging
- Common Bottlenecks
- Production Deployment Checklist

- For More Information

5.7.1 Environment Variables at a Glance

All runtime tuning knobs are read from environment variables at import time and centralised in `pycypher.config`.

5.7.2 Query Timeouts

Prevent runaway queries from consuming resources indefinitely.

Per-query timeout (programmatic):

```
result = star.execute_query(
    "MATCH (a)-[*]->(b) RETURN a, b",
    timeout_seconds=5.0,
)
```

Default timeout (environment):

```
export PYCYPHER_QUERY_TIMEOUT_S=10
```

When a query exceeds its timeout, PyCypher raises `QueryTimeoutError` with the elapsed time and a query fragment for diagnostics:

```
from pycypher import QueryTimeoutError

try:
    result = star.execute_query(slow_query, timeout_seconds=2.0)
except QueryTimeoutError as e:
    print(f"Timed out after {e.elapsed_seconds:.1f}s")
```

The timeout is enforced cooperatively between clauses and via `SIGALRM` on Unix (main thread only) so that queries stuck inside C extensions (e.g. a large pandas merge) are still interrupted.

5.7.3 Memory Budget

Cap the peak memory a single query is allowed to consume. The query planner estimates memory requirements before execution begins and rejects queries that exceed the budget:

```
result = star.execute_query(
    "MATCH (p:Person), (q:Person) RETURN p.name, q.name",
    memory_budget_bytes=500 * 1024 * 1024, # 500 MB
)
```

If the planner's estimate exceeds the budget, a `QueryMemoryBudgetError` is raised before any execution begins, avoiding wasted work:

```
from pycypher import QueryMemoryBudgetError

try:
    result = star.execute_query(big_query, memory_budget_bytes=100_000_000)
except QueryMemoryBudgetError as e:
    print(f"Estimated {e.estimated_bytes / 1e6:.0f} MB exceeds budget")
```

5.7.4 Cross-Join Limits

Cartesian products (`MATCH (a:X), (b:Y)`) can produce explosive row counts. PyCypher enforces a configurable hard ceiling:

```
# Reject cross-joins exceeding 1 million rows
export PYCYPHER_MAX_CROSS_JOIN_ROWS=1_000_000
```

The default is 10 million rows. When exceeded, the query fails immediately with a clear error rather than consuming all available memory.

Best practice: avoid unfiltered cross-joins. Push WHERE predicates as early as possible:

```
# BAD: cross-join first, filter later (produces N×M rows)
star.execute_query(
    "MATCH (p:Person), (c:Company) "
    "WHERE p.company_id = c.id "
    "RETURN p.name, c.name"
)

# BETTER: use a relationship to join directly
star.execute_query(
    "MATCH (p:Person)-[:WORKS_AT]->(c:Company) "
    "RETURN p.name, c.name"
)
```

5.7.5 Result Caching

PyCypher caches query results in an LRU cache keyed by query string + parameters. Repeated identical read-only queries skip parsing and execution entirely.

Configuration:

```
# Programmatic: set cache size and TTL per Star instance
star = Star(
    context=context,
    result_cache_max_mb=200,      # 200 MB cache
    result_cache_ttl_seconds=300.0, # 5-minute TTL
)

# Or via environment variables (affects all Star instances)
# export PYCYPHER_RESULT_CACHE_MAX_MB=200
# export PYCYPHER_RESULT_CACHE_TTL_S=300
```

Disable caching (useful for mutation-heavy workloads):

```
star = Star(context=context, result_cache_max_mb=0)
```

Smart invalidation: the cache is automatically invalidated whenever a mutation query (`SET`, `CREATE`, `DELETE`, `MERGE`, `REMOVE`, `FOREACH`) commits. Mutation queries are never cached.

Monitoring cache effectiveness:

```
from pycypher import get_cache_stats

stats = get_cache_stats(star=star)
```

(continues on next page)

(continued from previous page)

```
print(f"Hit rate: {stats['result_cache_hit_rate']:.0%}")
print(f"Entries: {stats['result_cache_entries']}")
print(f"Size: {stats['result_cache_size_mb']:.1f} MB")
print(f"Evictions: {stats['result_cache_evictions']}")
```

A hit rate below 50% on analytical workloads may indicate:

- Queries with unique literal values that prevent cache matches
- Frequent mutations invalidating the cache
- Cache too small for the working set — increase `result_cache_max_mb`

Parse Caching

Independent of the result cache, PyCypher maintains an LRU cache (capacity 512) for parsed AST trees. Identical query strings hit an $O(1)$ lookup on all calls after the first. No configuration is needed — this is automatic.

For an ETL pipeline executing 5 queries across 1 000 batches, this means roughly 5 cold parses (~56 ms each) rather than 5 000.

5.7.6 Graph-Native Indexes

PyCypher builds graph-native indexes lazily on first access, accelerating pattern matching from $O(E)$ full table scans to $O(\text{degree})$ neighbor lookups. Indexes are managed through the Context and invalidated automatically when mutations commit.

Index types:

- `AdjacencyIndex` — per-relationship-type adjacency lists for fast neighbor lookups
- `PropertyValueIndex` — per-(entity_type, property) hash index for value-to-ID resolution in $O(1)$
- `EntityLabelIndex` — per-label sorted ID arrays for $O(\log n)$ membership tests

Usage (indexes are automatic, but you can inspect them):

```
ctx = ContextBuilder().add_entity("Person", people).build()
manager = ctx.index_manager

# Neighbor lookup — O(degree) instead of O(E)
neighbors = manager.get_neighbors("KNOWS", source_id=42, direction="outgoing")

# Property index — O(1) instead of O(N)
ids = manager.lookup_property("Person", "name", "Alice")
```

When indexes help most: queries with selective relationship traversals or property equality filters. Indexes are rebuilt automatically after mutations, so there is no manual maintenance.

5.7.7 LeapfrogTriejoin (Multi-Way Joins)

For queries joining 3+ patterns on a shared variable, PyCypher uses the LeapfrogTriejoin algorithm (Veldhuizen 2014) which achieves worst-case optimal time complexity $O(N^{\lceil w/2 \rceil})$ — exponentially better than iterated binary joins at $O(N^{w-1})$.

This is particularly effective for cyclic query patterns such as triangle queries:

```
# Triangle query — LeapfrogTriejoin automatically used
star.execute_query("""
    MATCH (a:Person)-[:KNOWS]->(b:Person),
          (b)-[:KNOWS]->(c:Person),
          (c)-[:KNOWS]->(a)
    RETURN a.name, b.name, c.name
""")
```

The algorithm activates automatically when 3+ frames share a common join variable. For 2-frame joins, standard hash or sort-merge joins are used instead.

5.7.8 Cardinality Feedback Loops

The query planner maintains a `CardinalityFeedbackStore` that learns from execution history to improve future query plans. After each query, actual vs. estimated cardinalities are recorded.

How it works:

- Stores rolling window of (estimated, actual) cardinality pairs per entity type (max 32 observations)
- Computes geometric mean correction factors, clamped to [0.01, 100]
- Applies corrections to future cardinality estimates for better join ordering and algorithm selection

This system is automatic — no configuration needed. Over time, query plans improve as the planner accumulates execution statistics.

5.7.9 Rate Limiting

Protect shared deployments from query abuse with the built-in rate limiter.

```
from pycypher.rate_limiter import QueryRateLimiter, get_global_limiter

# Global limiter (configured via environment variables)
limiter = get_global_limiter()
limiter.acquire() # raises RateLimitError if over limit

# Custom per-caller limiter
limiter = QueryRateLimiter(qps=5.0, burst=20)
limiter.acquire(caller_id="user-123")
```

When the rate limit is exceeded, a `RateLimitError` is raised immediately (non-blocking fail-fast).

5.7.10 Audit Logging

Enable structured query audit logging for compliance and debugging:

```
export PYCYPHER_AUDIT_LOG=1
```

Each query execution emits a JSON record to the `pycypher.audit` logger containing: `query_id`, `timestamp`, `query` (truncated), `status`, `elapsed_ms`, `rows`, and `parameter_keys`.

Security: parameter values and result data are never logged.

```
# Programmatic activation
from pycypher.audit import enable_audit_log
enable_audit_log()
```

For production, configure the pycypher.audit logger via [logging.config](#) to route records to a file, syslog, or log aggregator.

5.7.11 Query Structure Best Practices

Push Filters Early

The most impactful optimisation is to filter rows as early as possible. WHERE clauses immediately after MATCH reduce the working set before subsequent joins and aggregations:

```
# GOOD: filter before aggregation
star.execute_query(
    "MATCH (p:Person) WHERE p.active = true "
    "RETURN p.dept AS dept, count(p) AS n"
)

# LESS EFFICIENT: aggregate all rows, then filter
star.execute_query(
    "MATCH (p:Person) "
    "WITH p.dept AS dept, count(p) AS n "
    "WHERE n > 5 RETURN dept, n"
)
```

Bound Variable-Length Paths

Unbounded paths ([*]) are capped at 20 hops by default, but even this can produce large intermediate results on dense graphs. Always specify explicit bounds when possible:

```
# GOOD: bounded path — predictable performance
star.execute_query(
    "MATCH (a:Person)-[:KNOWS*1..3]->(b:Person) RETURN a.name, b.name"
)

# RISKY: unbounded path — BFS explores up to 20 hops
star.execute_query(
    "MATCH (a:Person)-[:KNOWS*]->(b:Person) RETURN a.name, b.name"
)
```

Reduce the hop cap globally if your graph has long chains:

```
export PYCYPHER_MAX_UNBOUNDED_PATH_HOPS=5
```

Use WITH to Pipeline Results

Use WITH to narrow the working set between query stages. Only project the columns you need for subsequent clauses:

```
star.execute_query(
    """
    MATCH (p:Person)-[:WORKS_AT]->(c:Company)
    WITH c.name AS company, count(p) AS headcount
    WHERE headcount > 10
    RETURN company, headcount ORDER BY headcount DESC
    """
)
```

Prefer Relationship Patterns over Cross-Joins

Relationship patterns (`-[:REL]->`) scan only related rows. Cross-joins (`MATCH (a), (b)`) produce the full Cartesian product. The difference is orders of magnitude on large tables.

5.7.12 Monitoring and Diagnostics

Query Metrics

PyCypher collects execution metrics automatically:

```
from shared.metrics import QUERY_METRICS

stats = QUERY_METRICS.snapshot()
print(f"Total queries:      {stats.total_queries}")
print(f"Timing p50:          {stats.timing_p50_ms:.1f} ms")
print(f"Timing p99:           {stats.timing_p99_ms:.1f} ms")
print(f"Slow queries:         {stats.slow_queries}")
print(f"Error rate:           {stats.error_rate:.1%}")
print(f"Memory delta p50:     {stats.memory_delta_p50_mb:.1f} MB")
```

The slow-query threshold is configurable:

```
export PYCYPHER_SLOW_QUERY_MS=500 # 500ms threshold
```

Query Execution Logging

PyCypher logs every query execution at the INFO level with row count, elapsed time, and memory delta. Enable debug logging for parse timing and clause-by-clause breakdown:

```
import logging
logging.getLogger("pycypher").setLevel(logging.DEBUG)
```

5.7.13 Common Bottlenecks

5.7.14 Production Deployment Checklist

```
# 1. Set a default timeout to prevent runaway queries
export PYCYPHER_QUERY_TIMEOUT_S=30

# 2. Limit cross-join size for safety
export PYCYPHER_MAX_CROSS_JOIN_ROWS=1_000_000

# 3. Size the result cache for your working set
export PYCYPHER_RESULT_CACHE_MAX_MB=200

# 4. Set a TTL if data changes externally
export PYCYPHER_RESULT_CACHE_TTL_S=60

# 5. Bound variable-length paths for dense graphs
export PYCYPHER_MAX_UNBOUNDED_PATH_HOPS=10

# 6. Set slow-query threshold for alerting
export PYCYPHER_SLOW_QUERY_MS=500
```

(continues on next page)

(continued from previous page)

```
# 7. Enable audit logging for compliance
export PYCYPHER_AUDIT_LOG=1

# 8. Rate limit for multi-tenant deployments
export PYCYPHER_RATE_LIMIT_QPS=50
export PYCYPHER_RATE_LIMIT_BURST=100
```

5.7.15 For More Information

- [Query Processing](#) — detailed query lifecycle and expression evaluation
- [PyCypher API](#) — complete API reference
- [Data ETL Pipeline](#) — ETL pipeline patterns

5.8 Constants and Magic Values Reference

This document explains every configurable constant and internal magic value in PyCypher, including the rationale for each default.

On this page

- [Environment-Configurable Constants](#)
 - [Query Execution Limits](#)
 - [Security Limits](#)
 - [Caching](#)
- [Internal Constants \(Not Configurable\)](#)
 - [Cardinality Estimator](#)
 - [Query Planner](#)
 - [Path Expansion \(BFS\)](#)
 - [Query Learning](#)
 - [Other Internal Limits](#)
 - [Display Defaults](#)

5.8.1 Environment-Configurable Constants

These constants are read from environment variables at import time via `pycypher.config`. All can be overridden without code changes.

Query Execution Limits

Env Variable	Default	Module	Rationale
PY-CYPHER_QUERY	None	config	Wall-clock budget for a single query (seconds). None means no limit. Uses signal.SIGALRM on Unix. Set this in production to prevent runaway queries from holding resources indefinitely.
PY-CYPHER_MAX_C	1,000,000	config	Hard ceiling on cross-join (Cartesian product) result rows. Prevents accidental memory exhaustion from MATCH (a), (b) patterns. 1M rows is ~8 MB for ID-only frames; well within typical memory budgets.
PY-CYPHER_MAX_U	20	config	Maximum BFS depth for unbounded variable-length paths ([*]). 20 hops prevents runaway expansion on cyclic graphs while allowing traversal of most practical graph structures. Social networks (diameter ~6) and supply chains (depth ~10) are well within this limit.
PY-CYPHER_MAX_C (0)	None	config	Hard gate on query complexity score. Queries exceeding this are rejected before execution. None disables the gate. Typical production values: 50-200.
PY-CYPHER_COMPL (0)	None	config	Soft threshold for complexity warnings. Queries scoring above this emit a warning but still execute. Set lower than MAX_COMPLEXITY_SCORE for early alerts.

Security Limits

Env Variable	Default	Module	Rationale
PY-CYPHER_MAX_C (1 MiB)	1,048,576	config	Rejects queries exceeding this size before parsing. Prevents DoS via oversized query strings. 1 MiB accommodates even very large Cypher queries while protecting against adversarial payloads.
PY-CYPHER_MAX_C	200	config	Maximum AST traversal depth. Prevents stack exhaustion on deeply nested WHERE clauses or subqueries. 200 is generous for legitimate queries (typical nesting: 5-20 levels).
PY-CYPHER_MAX_C	1,000,000	config	Ceiling on generated collection/string sizes from range(), repeat(), lpad(), rpad(), and UNWIND. Prevents memory exhaustion from adversarial inputs like range(0, 999999999).
PY-CYPHER_RATE_ (disabled)	0	config	Maximum sustained queries per second. When enabled, excess queries receive a RateLimitError.
PY-CYPHER_RATE_	10	config	Burst allowance for rate limiting. Allows short bursts above the sustained QPS rate. Only meaningful when rate limiting is enabled.

Caching

Env Variable	Default	Module	Rationale
PY-CYPHER_RESULT_CACHE_SIZE	100	config	Maximum in-memory size for the query result cache. 100 MB balances cache hit rates against memory pressure. LRU eviction removes least-recently-used entries when the limit is reached.
PY-CYPHER_RESULT_CACHE_TTL	0 (no expiry)	config	Time-to-live for cached results. 0 means entries are only evicted by size pressure. Set a TTL when the underlying data changes between queries.
PY-CYPHER_AST_CACHE_SIZE	1,024	config	LRU cache size for parsed ASTs. 1,024 entries accommodate typical application workloads with repeated query patterns. Set to 0 to disable caching (useful during grammar development).

5.8.2 Internal Constants (Not Configurable)

These constants are defined in source code and require code changes to modify.

Cardinality Estimator

Constants defined in `cardinality_estimator.py` (extracted from `query_planner.py` in 2026-03, see [ADR-007: Extract Cardinality Estimator from Query Planner](#)).

Constant	Value	Rationale
STATS_SAMPLE_SIZE	10,000	Maximum rows sampled when computing column statistics. 10K rows provides statistically meaningful NDV and null-fraction estimates while keeping computation fast on large tables.
HISTOGRAM_BINS	64	Number of equi-width bins for histogram-based range selectivity. 64 bins balances granularity against memory overhead (~512 bytes per histogram).
HISTOGRAM_MIN_ROWS	10	Minimum non-null rows required to build a histogram. Below this count, histograms are unreliable and the estimator falls back to uniform distribution assumptions.
DEFAULT_FILTER_SELECTIVITY	0.33	Default fraction of rows passing a WHERE filter when no statistics are available. Same value as <code>DEFAULT_FILTER_SELECTIVITY</code> in the query planner (see below).
AVG_BYTES_PER_CELL	64	Conservative per-cell memory estimate for mixed-type columns when the actual DataFrame is unavailable. Used for memory budgeting in query planning.

Query Planner

Constant	Value	Rationale
<code>_DE-FAULT_FILTER_SELECTIVITY</code>	0.33	Default fraction of rows that pass a WHERE filter when no statistics are available. 0.33 (one-third) is a standard database heuristic: conservative enough to avoid over-optimistic plan choices, while not so pessimistic that it prevents useful optimisations. Used for equality checks, comparisons, string predicates, and IS NULL tests. Override by recording actual selectivity via the query learning system.
<code>_BROADCAST_THRESHOLD</code>	10,000	Row count below which the smaller side of a join is broadcast to all partitions. 10K rows fit comfortably in L2 cache (~256 KB for 8-byte IDs), making broadcast join faster than hash partitioning for small-vs-large joins.
<code>_MERGE_SORTED_THRESHOLD</code>	0.8	Fraction of data that must be pre-sorted to trigger a merge-sort join instead of hash join. 0.8 means at least 80% sorted — merge join only wins when data is mostly ordered.
<code>_STREAMING_AGG_THRESHOLD</code>	10,000,000	Row count above which aggregation switches to streaming (chunked) mode. 10M rows is approximately when a single-pass aggregation starts competing with available memory.
<code>_CROSS_JOIN_WARNING</code>	100,000	Output row count above which the planner emits a warning for cross-join operations. Separate from the hard ceiling (<code>MAX_CROSS_JOIN_ROWS</code>) — this is an advisory threshold.

Path Expansion (BFS)

Constant	Value	Rationale
<code>_MAX_UNBOUNDED_ROWS</code>	20	Module-level default for the configurable env var. See above.
<code>_MAX_FRONTIER_ROWS</code>	1,000,000	Maximum BFS frontier size (rows) at any single hop level. Prevents memory explosion during breadth-first path expansion. 1M rows is ~8 MB of IDs, keeping BFS within reasonable memory bounds.
<code>_MAX_BFS_TOTAL_ROWS</code>	5,000,000	Maximum total accumulated result rows across all BFS hops. 5M rows (~40 MB of IDs) is the overall safety valve for path expansion.

Query Learning

Constant	Value	Rationale
<code>_MAX_SELECTIVITY_HISTORY</code>	64	Rolling window size for selectivity observations per (entity_type, property, operator) triple. 64 provides enough samples for stable exponential moving averages without unbounded memory growth.
<code>_MAX_JOIN_HISTORY</code>	64	Rolling window size for join performance observations per (strategy, size_bucket) pair. Same rationale as selectivity history.
<code>_MAX_PLAN_CACHE</code>	256	Maximum entries in the adaptive plan cache (keyed by query fingerprint). 256 covers typical application workloads with ~50-100 distinct query shapes.
<code>_CONFIDENCE_THRESHOLD</code>	0.5	Minimum confidence score before learned selectivity overrides the default heuristic. 0.5 ensures at least moderate statistical evidence before changing plan decisions.
<code>_MAX_HISTORY</code>	32	Cardinality estimator rolling window per entity type. 32 observations balance recency with stability.

Other Internal Limits

Constant	Value	Module	Rationale
<code>__MAX_REGEX_PATT</code>	1,000	<code>binding_evaluator</code> , <code>string_predica</code>	Maximum regex pattern length for <code>=~</code> operator. Prevents ReDoS (Regular Expression Denial of Service) attacks via adversarial patterns.
<code>__MAX_QUERIES</code>	1,000	<code>multi_query_</code>	Maximum queries in a multi-query dependency analysis batch. Bounds the $O(n^2)$ dependency inference cost.
<code>__MAX_NAME_GENE</code>	10,000	<code>variable_manager</code>	Safety limit for generating unique variable names. 10K attempts ensures name generation succeeds even in heavily aliased queries.
<code>__MAX_CONTENT_LE</code>	10 MiB	<code>cypher_lsp</code>	Maximum LSP JSON-RPC payload size. Rejects oversized payloads before parsing to prevent memory exhaustion.
<code>__MAX_DOCUMENTS</code>	128	<code>cypher_lsp</code>	Maximum concurrent open documents in the LSP server. Evicts oldest documents beyond this limit.
<code>__MAX_QUERY_LOG</code>	200	<code>grammar_parser</code>	Truncation limit for query strings in log messages. Keeps logs readable without losing diagnostic value.
<code>__MAX_HISTORY</code>	1,000	<code>repl</code>	Maximum readline history entries in the REPL.
<code>__MAX_QUERY LENG</code>	2,048	<code>audit</code>	Truncation limit for query strings in audit log records. Longer than log truncation to preserve audit trail fidelity.
<code>CROSS_JOIN_WARN</code>	(100K, 1M)	<code>config</code>	Progressive warning thresholds for cross-join cardinality. Warnings at 100K and 1M rows give users early notice before hitting the hard ceiling.

Display Defaults

Constant	Value	Module	Rationale
<code>PY-CYPHER_REPL_MAX</code>	50	<code>repl</code> (env var)	Maximum rows displayed in REPL output. Prevents terminal flooding.
<code>__DE-FAULT_LOCK_TIMEO</code>	5.0s	<code>star</code> (Result-Cache)	Timeout for acquiring cache locks. 5 seconds is long enough for normal contention, short enough to detect deadlocks.

5.9 Overview

The user guide provides in-depth reference material for PyCypher’s core abstractions:

- [Data Model Requirements](#) — Why PyCypher needs ID columns, how to use your existing column names, real-world data loading patterns, and troubleshooting common data issues.
- [Variables in PyCypher](#) — How the Variable class represents named references throughout the AST and execution engine.
- [AST Nodes](#) — Complete catalogue of all AST node types with field descriptions and construction examples.
- [Query Processing](#) — The full query lifecycle: parsing, AST conversion, semantic validation, BindingFrame execution, expression evaluation, aggregation, mutation, and performance characteristics.

- [Error Handling Patterns](#) — Exception hierarchy, catching patterns, and best practices for robust query error handling.
- [Configuration Reference](#) — Complete reference for all environment variables and per-query configuration options.
- [Performance Tuning](#) — Practical strategies for optimising query execution: timeouts, memory budgets, cross-join limits, result caching, parse caching, query structure best practices, and a production deployment checklist.
- [Constants and Magic Values Reference](#) — Complete reference for all configurable constants and internal magic values, with rationale for each default.

This guide covers production deployment, container orchestration, scaling strategies, monitoring, and operational best practices for PyCypher.

6.1 Security Hardening

This guide covers security hardening for production PyCypher deployments. It is based on the internal security audit that identified and mitigated vulnerabilities in the data ingestion pipeline, query engine, and external integrations.

On this page

- [Production Security Checklist](#)
- [Input Validation](#)
 - [SQL Query Validation](#)
 - [Path Validation](#)
 - [SSRF Protection](#)
 - [Regex Safety](#)
- [Environment Hardening](#)
 - [Secrets](#)
 - [Security-Sensitive Environment Variables](#)
- [Network Security](#)
 - [Egress Restrictions](#)
 - [Neo4j Connection Security](#)
- [Resource Limits](#)
 - [Query Timeout](#)
 - [BFS Path Expansion](#)

- AST Cache
- Monitoring and Alerting
 - Security Events to Monitor
 - Structured Logging
- Docker Security
- Function Registration Security
- Incident Response

6.1.1 Production Security Checklist

Complete every item before going live. Items marked CRITICAL must not be deferred.

#	Item	Severity	Status
1	Environment secrets are generated with openssl rand -hex 32	CRITICAL	
2	.env file is excluded from version control and has mode 0600	CRITICAL	
3	Neo4j TLS is enabled (bolt+s:// or encrypted=True)	CRITICAL	
4	Containers run as non-root (UID 1000)	HIGH	
5	Docker socket is not mounted into containers	HIGH	
6	PYCYIPHER_AST_CACHE_MAX is tuned for expected query diversity	MEDIUM	
7	PYROSCOPE_SERVER (if set) points to an internal monitoring host	MEDIUM	
8	Log output is directed to a centralised log aggregator (not stdout)	MEDIUM	
9	Network policies restrict egress from PyCypher containers	MEDIUM	
10	Query timeout is configured via --timeout or PYCYPHER_QUERY_TIMEOUT_MS	MEDIUM	

6.1.2 Input Validation

PyCypher applies defence-in-depth to every user-controlled input. Understanding the validation layers helps operators diagnose rejected queries and tune security policy.

SQL Query Validation

User-supplied SQL queries (for data source ingestion) pass through four validation phases:

1. Comment stripping – removes --, #, and /* */ comments to prevent comment-based injection.
2. Statement multiplicity – rejects queries with more than one semicolon (multi-statement injection).
3. Keyword allowlist – only SELECT and WITH are permitted as the first keyword.
4. Table function blocklist – dangerous DuckDB table functions are rejected even inside valid SELECT queries.

The following DuckDB functions are blocked:

```
read_csv, read_csv_auto, read_parquet, read_json, read_json_auto,
read_json_objects, read_json_objects_auto, read_blob, read_text,
```

(continues on next page)

(continued from previous page)

```
read_ndjson, read_ndjson_auto, read_ndjson_objects, glob,
parquet_scan, parquet_metadata, parquet_schema,
parquet_file_metadata, parquet_kv_metadata, sniff_csv,
query_table, query
```

Functions with these prefixes are also blocked: duckdb_, pg_, pragma_, information_schema.

Why this matters: Without the table function blocklist, a query such as `SELECT * FROM read_csv('/etc/passwd')` would pass the keyword allowlist (it starts with `SELECT`) and read arbitrary files from the filesystem.

Path Validation

File paths are validated against:

- Path traversal – `..` components are rejected.
- Sensitive prefixes – `/etc/`, `/proc/`, `/sys/`, `/dev/`, `/root/`, `/var/run/`, `/var/lib/`, `/boot/`, `/sbin/` are blocked.
- SQL string literal safety – single quotes, NUL bytes, URL-encoded attacks (`%27`), and Unicode normalisation attacks are rejected when paths are interpolated into DuckDB SQL.

SSRF Protection

HTTP/HTTPS URIs are checked for Server-Side Request Forgery (SSRF):

- Blocked hostnames: `localhost`, `ip6-localhost`, etc.
- Blocked IP ranges: RFC 1918 private, loopback, link-local, reserved.
- DNS resolution: hostnames are resolved and the resulting IP is checked against the same blocklist.

Warning

DNS rebinding attacks (TOCTOU) are a known limitation. The hostname is resolved once during validation, but the downstream HTTP library may resolve it again. For maximum protection in untrusted environments, use network-level egress controls (see [Network Security](#) below).

Regex Safety

The Cypher `=~` regex operator validates patterns before execution:

- Length limit – patterns longer than 1,000 characters are rejected.
- Syntax check – invalid regex is rejected before reaching the engine.
- ReDoS detection – patterns with nested quantifiers `((a+)+)`, overlapping alternation `((a|a)*)`, or chained quantifiers `(a{n}{m})` are rejected to prevent catastrophic backtracking.

6.1.3 Environment Hardening

Secrets

Generate strong random values for all credentials:

```
# Generate secrets
export NEO4J_PASSWORD=$(openssl rand -hex 32)
export SPARK_RPC_SECRET=$(openssl rand -hex 32)
export NOMINATIM_PASSWORD=$(openssl rand -hex 32)

# Write to .env (never commit this file)
cat > .env << EOF
NEO4J_USER=neo4j
NEO4J_PASSWORD=${NEO4J_PASSWORD}
SPARK_RPC_SECRET=${SPARK_RPC_SECRET}
NOMINATIM_PASSWORD=${NOMINATIM_PASSWORD}
EOF
chmod 600 .env
```

Security-Sensitive Environment Variables

These variables control security-relevant behaviour:

Variable	Default	Security Notes
PY-CYPHER_AST_CACHE_MAX_ENTRIES	1024	Maximum cached AST entries. Set lower in memory-constrained environments. Set higher for ETL pipelines with many unique queries.
PY-CYPHER_QUERY_TIMEOUT	(none)	Per-query timeout in milliseconds. Strongly recommended for production to prevent runaway queries from consuming resources.
PYCYPHER_LOG_LEVEL	WARNING	Set to INFO for audit trails. Avoid DEBUG in production (logs query text which may contain sensitive data).
PY-CYPHER_LOG_FORMAT	text	Set to json for structured logging with log aggregators.
PYROSCOPE_SERVER	(disabled)	Profiling data endpoint. Must point to a trusted internal host. An attacker who controls this variable can exfiltrate performance data to an external server.
PYROSCOPE_ENABLED	false	Disable in production unless actively profiling. Profiling adds CPU overhead and exposes timing information.

6.1.4 Network Security

Egress Restrictions

PyCypher containers should have restricted egress to prevent data exfiltration via cloud read functions or SSRF bypasses:

```
# docker-compose.override.yml — production egress restrictions
services:
  pycypher:
    networks:
      - internal
    # No access to external networks
  neo4j:
    networks:
      - internal
  spark-master:
```

(continues on next page)

(continued from previous page)

```

networks:
  - internal

networks:
  internal:
    internal: true # No external connectivity

```

If the pipeline must read from cloud storage (S3, GCS, Azure), use a proxy or IAM role with minimum required permissions rather than granting broad network access.

Neo4j Connection Security

Always enable TLS for Neo4j connections in production:

```

import os
from pycypher.sinks.neo4j import Neo4jSink

# Use bolt+s:// scheme for TLS
with Neo4jSink(
    "bolt+s://neo4j-host:7687",
    "neo4j",
    os.environ["NEO4J_PASSWORD"],
) as sink:
    sink.write_nodes(result, mapping)

```

The Neo4j sink validates all identifiers (labels, property names, relationship types) against Cypher injection, including:

- Backtick escaping attacks
- NUL byte injection
- Unicode normalisation confusables (e.g., fullwidth grave accent)

6.1.5 Resource Limits

Query Timeout

Set a per-query timeout to prevent runaway queries:

```

# Via CLI
nmetl query --timeout 30000 config.yaml # 30 second limit

# Via environment
export PYCYPHER_QUERY_TIMEOUT_MS=30000

```

BFS Path Expansion

Variable-length path patterns (`[*1..10]`) use BFS expansion with two safety limits:

- Hop limit: unbounded paths (`[*]`) are capped at 20 hops.
- Frontier limit: the BFS frontier is truncated at 1,000,000 rows to prevent memory exhaustion in highly-connected graphs.

If your graph has high average degree (>100 edges per node), consider adding explicit hop bounds to variable-length patterns.

AST Cache

The query parser caches AST results with LRU eviction:

- Default size: 1,024 entries.
- Configuration: `PYCYPHER_AST_CACHE_MAX` environment variable.
- Monitoring: cache hit rate, eviction count, and current size are available via `parser.cache_stats`.

For ETL pipelines with a fixed set of queries, the default is sufficient. For applications generating many unique queries (e.g., parameterised by user input), monitor eviction rates and increase the limit if hit rate drops below 80%.

6.1.6 Monitoring and Alerting

Security Events to Monitor

Configure alerts for these log patterns:

Log Pattern	Indicates	Action
SecurityError	Blocked SQL injection, path traversal, or SSRF attempt	Investigate source IP / user
Dangerous DuckDB table function	Attempted file system access via SQL query	Block source, review query logs
SSRF protection	Attempted access to private/internal network	Review URI source, tighten egress
BFS frontier.*exceeds safety limit	Possible resource exhaustion attack via complex queries	Review query patterns, add hop bounds
catastrophic backtracking	ReDoS attack via regex <code>=~</code> operator	Block pattern, review query source
Query.*timed out	Runaway query (accidental or malicious)	Review query, tune timeout

Structured Logging

Enable JSON logging for machine-parseable security audit trails:

```
export PYCYPHER_LOG_FORMAT=json
export PYCYPHER_LOG_LEVEL=INFO
```

6.1.7 Docker Security

The production Dockerfile follows container security best practices:

- Non-root user: runs as UID 1000 (not root).
- Minimal base image: Python slim variant.
- No shell access needed: consider using `--read-only filesystem`.
- Health checks: Docker health checks verify the engine is responsive.

Additional hardening for production:

```
services:
  pycypher:
```

(continues on next page)

(continued from previous page)

```

read_only: true
tmpfs:
  - /tmp
security_opt:
  - no-new-privileges:true
cap_drop:
  - ALL
mem_limit: 4g      # Prevent OOM from taking down the host
memswap_limit: 4g  # Disable swap to avoid performance collapse

```

6.1.8 Function Registration Security

The YAML configuration system supports registering custom Python functions as Cypher scalar functions. This is protected by:

- Module blacklist: imports from os, subprocess, sys, socket, pickle, ctypes, importlib, and 13 other dangerous modules are rejected.
- Import path validation: only dotted Python identifiers (package.module.function) are accepted.
- Top-level check: the first component of the import path is checked against the blacklist.

Warning

Custom function registration executes arbitrary Python code from the imported module. Only register functions from trusted, audited packages. Never allow end users to specify import paths directly.

6.1.9 Incident Response

If a security event is detected:

1. Contain – stop the affected container immediately:

```
docker compose stop pycypher
```

2. Preserve evidence – save logs before they rotate:

```
docker compose logs pycypher > incident_$(date +%s).log
```

3. Investigate – search logs for the triggering event:

```
grep -i "SecurityError\|SSRF\|injection\|backtracking" incident_*.log
```

4. Remediate – apply the appropriate fix:

- SQL injection attempt → review and tighten query validation
- SSRF attempt → add network egress restrictions
- Resource exhaustion → lower timeout, add hop bounds
- ReDoS → the pattern is already blocked; check for bypass attempts

5. Post-incident – update monitoring rules and review access controls.

6.2 Docker Containerisation

PyCypher ships two Dockerfiles and a Docker Compose configuration that orchestrates the full development and integration stack.

6.2.1 Images

Production Image (Dockerfile)

The production image uses a multi-stage build to minimize image size. Stage 1 installs dependencies with build tools (compilers, git). Stage 2 copies only the virtual environment into a python:3.14-slim runtime image, excluding all build-only tooling.

```
docker build -t pycypher:latest .
```

Key characteristics:

- Multi-stage build – ~400 MB final image (vs ~1.2 GB single-stage)
- Non-root user (appuser, UID 1000) for security hardening
- Built-in health check – HEALTHCHECK directive runs nmetl health
- Health server default – CMD starts the HTTP health endpoint on port 8079
- Layer-cache-optimized – dependency manifests copied before source code

Running the production image:

```
# Start with health server (default CMD)
docker run -p 8079:8079 pycypher:latest

# Run a one-off query instead
docker run pycypher:latest nmetl query --inline "MATCH (n) RETURN n"

# Start health server on all interfaces
docker run -p 8079:8079 pycypher:latest nmetl health-server --bind 0.0.0.0
```

Development Image (Dockerfile.dev)

The development image adds interactive tooling on top of the same Python base.

```
docker build -f Dockerfile.dev -t pycypher-dev:latest .
```

Additional tools included:

- ipython, ipdb for interactive debugging
- pytest, pytest-cov for testing
- sudo access for the developer user
- VS Code Dev Container compatible

6.2.2 Docker Compose

The docker-compose.yml defines the full multi-service stack. All services share the pycypher-dev-network bridge network.

Quick Start

```
# 1. Copy and populate the environment file
cp .env.example .env
# Edit .env with real credentials (see Environment Configuration)

# 2. Start the development stack
make dev-up

# 3. Access the dev container
make dev-shell

# 4. Run the test suite inside the container
cd /workspace && uv run pytest
```

Services

Service	Container	Ports	Notes
pycypher-dev	pycypher-dev	–	Main dev container; uv sync on start
spark-master	pycypher-spark-master	7077, 8090	Bitnami Spark 3.5; RPC auth + encryption
spark-worker	pycypher-spark-worker	8091	2 GB RAM, 2 cores per worker
neo4j	pycypher-neo4j	7474, 7687	Neo4j 5 Community + APOC plugin
nominatim	pycypher-nominatim	8092	US OSM geocoder; 300s start period
fastopendata	pycypher-fastopendata	–	ETL pipeline container
fastopendata-api	pycypher-fastopendata-api	8093	REST API (Swagger at /docs)
code-server	pycypher-code-server	8080	VS Code in browser (profile: code-server)

Optional Profiles

Some services are behind Docker Compose profiles to avoid starting them by default:

```
# Start with Jupyter Lab
make dev-jupyter
# → http://localhost:8888

# Start with browser-based VS Code
make dev-vscode
# → http://localhost:8080
```

Volumes

Persistent volumes ensure data survives container restarts:

Volume	Purpose
uv-cache	Python package cache (speeds up rebuilds)
pytest-cache	Test result cache
neo4j-data	Neo4j graph database files
neo4j-logs	Neo4j server logs
neo4j-import	Neo4j CSV import directory
spark-events	Spark event history for the History Server UI
fastopendata-raw	Downloaded raw datasets
nominatim-data	PostgreSQL database for Nominatim

Building Custom Images

When adding new Python dependencies:

```
# Rebuild just the dev container
make dev-rebuild

# Rebuild all images
docker compose build

# Rebuild with no cache (after Dockerfile changes)
docker compose build --no-cache pycypher-dev
```

Health Checks

All key containers include health checks:

- pycypher-dev: nmetl health every 30s (60s start grace period)
- Neo4j: wget http://localhost:7474 every 10s (10 retries)
- Nominatim: geocode query every 30s (20 retries, 300s start grace period)

The nmetl health command returns exit code 0 (healthy), 1 (degraded), or 2 (unhealthy), making it compatible with Docker HEALTHCHECK and Kubernetes liveness/readiness probes.

Check health status:

```
docker compose ps
# Shows health status for each service

# Detailed health report from the dev container
docker compose exec pycypher-dev uv run nmetl health --json
```

Kubernetes Deployment

The production image works directly with Kubernetes. The nmetl health-server command provides standard HTTP endpoints for liveness and readiness probes:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pycypher
spec:
```

(continues on next page)

(continued from previous page)

```
replicas: 1
selector:
  matchLabels:
    app: pycypher
template:
  metadata:
    labels:
      app: pycypher
spec:
  containers:
    - name: pycypher
      image: pycypher:latest
      ports:
        - containerPort: 8079
      livenessProbe:
        httpGet:
          path: /health
          port: 8079
        initialDelaySeconds: 10
        periodSeconds: 30
      readinessProbe:
        httpGet:
          path: /ready
          port: 8079
        initialDelaySeconds: 5
        periodSeconds: 10
```

The `/metrics` endpoint serves Prometheus text format, ready for scraping by Prometheus or any OpenMetrics-compatible collector.

6.3 Environment Configuration

PyCypher uses environment variables for service credentials and connection strings. All required variables must be set before running docker compose.

6.3.1 Setup

```
# Copy the template
cp .env.example .env

# Edit with real values
$EDITOR .env
```

6.3.2 Required Variables

Variable	Example	Description
NEO4J_USER	neo4j	Neo4j database username
NEO4J_PASSWORD	<strong-password>	Neo4j database password
SPARK_RPC_SECRET	<random-secret>	Spark inter-node RPC authentication secret
NOMINATIM-TIM_PASSWORD	<strong-password>	Nominatim PostgreSQL password

6.3.3 Optional Variables

Variable	Default	Description
GITHUB_TOKEN	(empty)	GitHub token for private dependencies
CODE_SERVER_PASS	(none)	Password for browser-based VS Code
NOMINATIM_URL	http://nominatim:8080	Nominatim endpoint (inside Docker network)
NEO4J_URI	bolt://neo4j:7687	Neo4j Bolt endpoint

Connection URLs

Services are accessible at different URLs depending on whether you are inside the Docker network or on the host:

Service	Inside Docker	From Host
Spark Master	spark://spark-master:7077	spark://localhost:7077
Neo4j Bolt	bolt://neo4j:7687	bolt://localhost:7687
Neo4j Browser	–	http://localhost:7474
Spark UI	–	http://localhost:8090
Nominatim	http://nominatim:8080	http://localhost:8092
FastOpenData API	http://fastopendata-api:8000	http://localhost:8093

6.3.4 Monitoring & Observability

Variable	Default	Description
PY-CYPHER_METRICS_ENAB	1	Enable in-process metrics collection (0 to disable)
PY-CYPHER_SLOW_QUERY_M	1000	Threshold (ms) for flagging slow queries
PY-CYPHER_METRICS_EXPO	(empty)	Comma-separated exporters: prometheus, statsd, json
PY-CYPHER_METRICS_PREFI	pycypher	Metric name prefix for exporters
PY-CYPHER_METRICS_EXPO	60	Push interval for metrics export (seconds)
PY-CYPHER_STATSD_HOST	127.0.0.1	StatsD/Datadog agent host
PY-CYPHER_STATSD_PORT	8125	StatsD/Datadog agent port
PY-CYPHER_METRICS_JSON	metrics.jsonl	Output path for JSON file exporter
PYCYPHER_LOG_LEVEL	WARNING	Logging level (DEBUG, INFO, WARNING, ERROR)
PY-CYPHER_LOG_FORMAT	rich	Log format: rich (console) or json (structured)
PY-CYPHER_OTEL_ENABLED	0	Enable OpenTelemetry distributed tracing
OTEL_SERVICE_NAME	pycypher	Service name for OTel spans
PYROSCOPE_ENABLED	0	Enable Pyroscope continuous profiling
PYROSCOPE_SERVER	http://localhost:4040	Pyroscope server endpoint
PYROSCOPE_APP_NAME	pycypher	Application name for Pyroscope

6.3.5 Python Environment

PyCypher requires Python 3.14+ and uses uv for dependency management.

```
# Install uv
curl -Lsf https://astral.sh/uv/install.sh | sh

# Sync all workspace dependencies
uv sync

# Verify installation
uv run python -c "from pycypher import Star; print('OK')"
```

The workspace is structured as a monorepo with cross-package dependencies:

- packages/shared – common utilities (no dependencies on other packages)
- packages/pycypher – core engine (depends on shared)
- packages/fastopendata – data pipeline (depends on pycypher)

After editing any pyproject.toml, always re-sync:

```
uv sync
```

6.3.6 Security Notes

- Never commit .env files to version control (already in .gitignore)
- Generate strong random secrets for SPARK_RPC_SECRET and passwords
- The production Dockerfile runs as a non-root user (UID 1000)
- Spark RPC authentication and encryption are enabled by default
- Neo4j heap and page cache are capped to prevent OOM (1 GB heap, 512 MB page cache)

6.4 Scaling & Performance

PyCypher supports several strategies for scaling query execution from single-machine development to multi-node production workloads.

6.4.1 Backend Selection

PyCypher's query engine uses pandas DataFrames as the default execution backend. The `backend_engine` module provides an abstraction layer for alternative backends:

```
from pycypher.backend_engine import select_backend

# Select a backend programmatically
backend = select_backend("pandas")
```

Backend	Status	Best for	Notes
Pandas	Default, fully integrated	All workloads, widest compatibility	Used by <code>Star.execute_query()</code>
DuckDB	Integrated for data ingestion	Large CSV/Parquet scans, SQL sources	Used by <code>ContextBuilder</code> for loading
Polars	Optional (included in <code>uv sync --group dev</code>)	CPU-bound transforms	Backend engine support, not default path

Note

`ContextBuilder.from_dict()` does not accept a backend parameter. Backend selection is handled by the `backend_engine` module, which is used internally by the query planner.

Health Checks and Circuit Breaker

The backend engine includes a health check probe that validates scan, filter, join, and materialise operations before committing to a backend:

```
from pycypher.backend_engine import check_backend_health, PandasBackend

backend = PandasBackend()
healthy = check_backend_health(backend)
```


The circuit breaker uses a three-state model:

- CLOSED (normal) – requests flow through; failures increment a counter
- OPEN (tripped) – backend is skipped; checked again after a recovery timeout
- HALF_OPEN (probing) – one test request; success resets, failure re-opens

Default thresholds: 3 consecutive failures to open, 60-second recovery timeout.

6.4.2 Spark Cluster (Docker Infrastructure)

The Docker Compose setup includes an optional Spark cluster for external data processing. This is part of the deployment infrastructure, not the PyCypher query engine. To scale:

```
# Start with default 1 worker
make spark-up

# Scale to 5 workers
make spark-scale WORKERS=5

# View the Spark UI
make spark-ui
# → http://localhost:8090
```

Worker Configuration

Each Spark worker is configured with:

- 2 GB memory (SPARK_WORKER_MEMORY=2G)
- 2 CPU cores (SPARK_WORKER_CORES=2)
- RPC authentication enabled with shared secret
- RPC encryption and local storage encryption enabled

To change worker resources, modify docker-compose.yml:

```
spark-worker:
  environment:
    - SPARK_WORKER_MEMORY=4G # increase per-worker memory
    - SPARK_WORKER_CORES=4   # increase per-worker cores
```

Neo4j Tuning (Docker Infrastructure)

Default Neo4j memory settings are conservative:

```
neo4j:
  environment:
    - NEO4J_dbms_memory_heap_max_size=1G
    - NEO4J_dbms_memory_pagecache_size=512m
```

For production workloads with larger graphs, increase these values. A common guideline: allocate 50% of available RAM to page cache and 25% to heap.

6.4.3 Query-Level Performance

Memory Budget

The query planner estimates memory usage and enforces a configurable budget:

```
from pycypher import Star

star = Star(context=context)
result = star.execute_query(
    "MATCH (a)-[:KNOWS]->(b) RETURN a.name, b.name",
    memory_budget_mb=512, # cap at 512 MB
)
```

Cross-Join Limits

Unbounded cross-products (e.g., MATCH (a), (b)) are capped by default to prevent resource exhaustion. The limit is configurable:

```
result = star.execute_query(
    "MATCH (a:Person), (b:Person) RETURN count(*)",
    max_cross_join_rows=1_000_000,
)
```

Query Timeout

Set a timeout to abort runaway queries:

```
result = star.execute_query(
    "MATCH (a)-[:KNOWS*1..10]->(b) RETURN a, b",
    timeout_seconds=30,
)
```

Lazy Evaluation

Variable-length path queries and large scans benefit from lazy evaluation, which defers computation until results are materialised:

```
# Lazy evaluation is automatic for variable-length paths
result = star.execute_query(
    "MATCH (a)-[:KNOWS*1..5]->(b) RETURN a.name, b.name LIMIT 100"
)
# Only paths that survive the LIMIT are fully materialised
```

6.4.4 Performance Testing

Run the built-in performance benchmarks:

```
# Large-dataset integration tests (120s timeout)
make test-large-dataset

# Backend compatibility tests
make test-backends
```

(continues on next page)

(continued from previous page)

```
# Full test suite in parallel
make test
```

6.4.5 Resource Planning

Dataset size	Recommended backend	RAM per worker	Workers
< 100K rows	Pandas	2 GB	1
100K – 10M rows	DuckDB for loading, Pandas for execution	4 GB	1
10M – 100M rows	DuckDB for loading, tune memory budget	8 GB	1
> 100M rows	Partition data, batch processing	16 GB+	1+

These are guidelines. Actual requirements depend on query complexity (number of joins, variable-length paths, aggregation cardinality). Use the query planner’s memory estimation to calibrate for your workload.

6.5 Monitoring & Operations

PyCypher includes built-in metrics collection, structured logging, and diagnostic tools for production observability.

6.5.1 Query Metrics

The `shared.metrics` module collects per-query execution metrics automatically:

```
from shared.metrics import QUERY_METRICS

# Execute some queries...
star.execute_query("MATCH (p:Person) RETURN p.name")

# Get a point-in-time snapshot
snapshot = QUERY_METRICS.snapshot()
print(f"Queries executed: {snapshot.total_queries}")
print(f"Total time: {snapshot.total_time_ms:.1f} ms")
print(f"Error count: {snapshot.error_count}")
```

Metrics Snapshot

The `MetricsSnapshot` includes:

- Query counts – total, successful, failed
- Timing – total, mean, p50, p95, p99 latencies
- Throughput – queries per second
- Error rates – error count and error rate trend
- Per-clause breakdown – time spent in `MATCH`, `WHERE`, `RETURN`, etc.

Diagnostic Report

For operator dashboards, use the diagnostic report on a metrics snapshot:

```
snapshot = QUERY_METRICS.snapshot()
report = snapshot.diagnostic_report()
# Returns a human-readable multi-section report with hotspot
# identification, error analysis, and cache efficiency
```

Cache Statistics

Monitor AST parse cache and result cache pressure:

```
from pycypher import get_cache_stats

stats = get_cache_stats()
print(f"AST cache hits: {stats['ast_hits']}")
print(f"AST cache misses: {stats['ast_misses']}")
print(f"AST cache size: {stats['ast_size']}")
print(f"Result cache hit rate: {stats.get('result_cache_hit_rate', 0):.1%}")
```

6.5.2 Metrics Export

PyCypher includes pluggable metrics exporters for Prometheus, StatsD, and JSON file output. All exporters use stdlib only — no external dependencies.

Configure via the PYCYPHER_METRICS_EXPORT environment variable:

```
# Enable one or more exporters (comma-separated)
export PYCYPHER_METRICS_EXPORT=prometheus,statsd,json
```

Prometheus

Generates text/plain; version=0.0.4 output compatible with Prometheus, VictoriaMetrics, or any OpenMetrics scraper. The health server at /metrics uses this exporter automatically.

```
from shared.exporters import PrometheusExporter
from shared.metrics import QUERY_METRICS

exporter = PrometheusExporter(prefix="pycypher")
text = exporter.render(QUERY_METRICS.snapshot())
# Returns multi-line Prometheus text exposition format
```

Exported metrics include pycypher_queries_total, pycypher_query_duration_p50_ms, pycypher_cache_hit_rate, pycypher_health_status, per-clause timings, and per-error-type counters.

StatsD / Datadog

Pushes metrics via UDP to any StatsD-compatible daemon (StatsD, Datadog Agent, Telegraf).

```
export PYCYPHER_METRICS_EXPORT=statsd
export PYCYPHER_STATSD_HOST=127.0.0.1 # default
export PYCYPHER_STATSD_PORT=8125     # default
```

JSON File

Appends JSON-lines snapshots for ingestion by ELK, Datadog Logs, or Loki.

```
export PYCYPHER_METRICS_EXPORT=json
export PYCYPHER_METRICS_JSON_PATH=metrics.jsonl # default
```

Programmatic Export

Push metrics to all configured exporters in one call:

```
from shared.exporters import export_once
from shared.metrics import QUERY_METRICS

export_once(QUERY_METRICS.snapshot())
```

Additional configuration:

- PYCYPHER_METRICS_PREFIX — metric name prefix (default: pycypher)
- PYCYPHER_METRICS_EXPORT_INTERVAL_S — push interval in seconds (default: 60)

6.5.3 OpenTelemetry Distributed Tracing

PyCypher supports OpenTelemetry distributed tracing for end-to-end query visibility across services. If opentelemetry-api is not installed, all tracing calls are silent no-ops with zero runtime cost.

```
# Enable tracing
export PYCYPHER_OTEL_ENABLED=1

# Standard OTEL configuration
export OTEL_SERVICE_NAME=pycypher
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
```

Wrap query execution in traced spans:

```
from shared.otel import trace_query, trace_phase

# Trace an entire query
with trace_query("MATCH (p:Person) RETURN p.name", query_id="q-123") as span:
    result = star.execute_query(query)
    span.set_attribute("result.rows", len(result))

# Trace individual phases (parse, plan, execute)
with trace_phase("parse", query_id="q-123") as span:
    ast = parser.parse(query)
```

Span attributes set automatically:

- db.system = "pycypher"
- db.statement = query text (truncated to 500 chars)
- db.operation = first Cypher keyword (MATCH, CREATE, etc.)
- pycypher.query_id = correlation ID

Exceptions are recorded on the span and the span status is set to ERROR.

6.5.4 Structured Logging

PyCypher uses the `shared.logger` module for structured output:

```
from shared.logger import LOGGER

LOGGER.info("Query executed", query="MATCH (n) RETURN n", rows=42)
LOGGER.debug("Pattern match", entity_type="Person", matches=1000)
```

Query execution is automatically logged with:

- Query text (truncated for security)
- Execution time
- Row count
- Error details (on failure)
- Query ID for correlation

Query ID Correlation

Each query execution is assigned a unique ID for end-to-end tracing:

```
result = star.execute_query("MATCH (p:Person) RETURN p.name")
# Log output: {"query_id": "abc-123", "time_ms": 12.3, "rows": 100}
```

6.5.5 Container Logs

View logs for any service:

```
# Development container
make dev-logs

# Spark cluster
make spark-logs

# Neo4j
make neo4j-logs

# Nominatim
make nominatim-logs

# FastOpenData
make fod-logs
make fod-api-logs
```

6.5.6 Service Health

Check container status and health:

```
# All services
docker compose ps

# Nominatim import/service status
```

(continues on next page)

(continued from previous page)

```
make nominatim-status

# Neo4j health (via browser endpoint)
curl -s http://localhost:7474

# Spark cluster status
# → open http://localhost:8090
```

Backend Health Monitoring

Monitor backend health programmatically:

```
from pycypher.backend_engine import (
    check_backend_health,
    get_circuit_breaker,
    PandasBackend,
)

# Probe a specific backend
backend = PandasBackend()
healthy = check_backend_health(backend)
print(f"Pandas backend healthy: {healthy}")

# Check circuit breaker state
cb = get_circuit_breaker()
for name in ["pandas", "duckdb", "polars"]:
    state = cb.state(name)
    available = cb.is_available(name)
    print(f"{name}: state={state.value}, available={available}")
```

6.5.7 Operational Runbook

Restart a Stuck Container

```
# Restart just the dev container
docker compose restart pycypher-dev

# Full rebuild if state is corrupted
make dev-rebuild
```

Clear Neo4j Data

```
# WARNING: deletes all graph data (3s abort window)
make neo4j-reset
```

Reset Spark State

```
make spark-down
docker volume rm pycypher-nmetl_spark-events
make spark-up
```

Check Nominatim Import Progress

The first Nominatim start imports US OSM data, which takes several hours:

```
# Watch import progress
make nominatim-logs

# Check status endpoint
make nominatim-status

# Test geocoding
make nominatim-search
```

6.6 Troubleshooting

Common issues and their solutions when deploying and operating PyCypher.

6.6.1 Docker Issues

docker compose up fails with “variable not set”

Cause: Required environment variables are missing.

Fix: Copy and populate the .env file:

```
cp .env.example .env
# Set NEO4J_USER, NEO4J_PASSWORD, SPARK_RPC_SECRET, NOMINATIM_PASSWORD
```

Container starts but uv sync fails

Cause: uv cache is stale or corrupted.

Fix:

```
# Clear the uv cache volume
docker compose down
docker volume rm pycypher-nmetl_uv-cache
make dev-up
```

Port conflicts

Cause: Another service is using the same port.

Fix: Check which ports are in use:

```
# macOS / Linux
lsof -i :7474 # Neo4j
lsof -i :7077 # Spark
lsof -i :8090 # Spark UI
lsof -i :8092 # Nominatim
lsof -i :8093 # FastOpenData API
```

Stop the conflicting service or change the port mapping in docker-compose.yml.

6.6.2 Python / Dependency Issues

ModuleNotFoundError: No module named 'pycypher'

Cause: Workspace not synced after changes.

Fix:

```
uv sync
uv run python -c "from pycypher import Star; print('OK')"
```

Python >= 3.14 required

Cause: System Python is too old.

Fix: Install Python 3.14+ via uv:

```
uv python install 3.14
```

Backend failures

Cause: Optional backend dependency not installed.

Fix:

```
# Install DuckDB backend
uv pip install duckdb

# Install Polars backend
uv pip install polars

# Verify
uv run python -c "from pycypher.backend_engine import select_backend; print(select_backend(hint=
↳ 'pandas').name)"
```

6.6.3 Spark Issues

Spark worker cannot connect to master

Cause: RPC secret mismatch or network issue.

Fix: Verify the secret is consistent:

```
# Both master and worker must use the same SPARK_RPC_SECRET
docker compose logs spark-master | grep -i "auth"
docker compose logs spark-worker | grep -i "auth"
```

Spark job fails with OOM

Cause: Worker memory is insufficient.

Fix: Increase worker memory in docker-compose.yml:

```
spark-worker:
  environment:
    - SPARK_WORKER_MEMORY=4G
```

Or scale out with more workers:

```
make spark-scale WORKERS=4
```

6.6.4 Neo4j Issues

Neo4j authentication failure

Cause: Credentials in .env do not match the Neo4j volume state.

Fix: If you changed the password after first setup, reset the volume:

```
make neo4j-down
docker volume rm pycypher-nmetl_neo4j-data
make neo4j-up
```

Neo4j APOC procedures not available

Cause: APOC plugin failed to install.

Fix: Check logs and ensure the plugin config is correct:

```
make neo4j-logs
# Look for APOC-related messages
```

6.6.5 Nominatim Issues

Nominatim import takes hours

Expected: The first-start import of the US OSM extract takes several hours and requires approximately 32 GB of RAM. Subsequent starts skip the import.

```
# Watch progress
make nominatim-logs

# Check status
make nominatim-status
```

Nominatim returns empty results

Cause: Import is incomplete or PBF file is missing.

Fix: Ensure the PBF exists before starting Nominatim:

```
ls packages/fastopendata/raw_data/us-latest.osm.pbf
```

If missing, download it via the fastopendata container:

```
make fod-shell
# Inside container: follow DATASETS.md instructions to download the PBF
```

6.6.6 Query Execution Issues

Query hangs or runs too long

Fix: Set a timeout:

```
result = star.execute_query("...", timeout_seconds=30)
```

Cross-product produces too many rows

Fix: The cross-join limit prevents runaway cartesian products:

```
# Default limit applies; increase if needed
result = star.execute_query("...", max_cross_join_rows=500_000)
```

GraphTypeNotFoundError

Cause: Query references an entity or relationship type not in the context.

Fix: Check available types:

```
print(repr(context))
# Context(backend='pandas', entities={'Person': 4}, relationships={'KNOWS': 3})
```

6.6.7 Getting Help

- Issue tracker: Report bugs at the GitHub repository
- Test suite: Run `uv run pytest -x` to identify failures
- Logs: Check `make dev-logs` for container output
- Metrics: Use `QUERY_METRICS.snapshot().diagnostic_report()` for execution stats

6.7 Architecture Overview

PyCypher deployments consist of these components:

- PyCypher core – the Cypher query engine (always required)
- Spark cluster – optional distributed compute for large datasets
- Neo4j – optional graph database sink for writing query results
- Nominatim – optional geocoding service for geospatial ETL
- FastOpenData API – optional REST API for data ingestion

All components are containerised and orchestrated via Docker Compose. The Makefile provides convenience targets for every operational task.

6.7.1 Deployment Modes

Mode	Command	What it starts
Development	<code>make dev-up</code>	Dev container + Spark + Neo4j
Infrastructure only	<code>make infra-up</code>	Spark master/worker + Neo4j
Spark cluster	<code>make spark-up</code>	Spark master + worker(s)
Neo4j only	<code>make neo4j-up</code>	Neo4j 5 Community with APOC
FastOpenData	<code>make fod-up</code>	ETL pipeline container
FastOpenData API	<code>make fod-api-up</code>	REST API on port 8093
Geocoding	<code>make nominatim-up</code>	Nominatim (first start imports OSM data)

6.7.2 Prerequisites

- Docker (via Docker Desktop or Rancher Desktop)
- Python 3.14+ (for local development outside containers)
- uv package manager (`curl -Lsf https://astral.sh/uv/install.sh | sh`)
- make (standard on macOS/Linux)

Information for developers contributing to PyCypher.

7.1 Architecture

Overview of the PyCypher architecture and design principles.

7.1.1 Project Structure

This is a monorepo workspace containing three interdependent packages:

```
pycypher-nmetl/  
  packages/  
    shared/      # Common utilities and logging  
    pycypher/    # Cypher parser, AST, and BindingFrame execution engine  
    fastopendata/ # ETL pipeline for open data (Census, TIGER, OSM)  
  tests/        # Test suite  
  docs/         # Documentation  
  pyproject.toml # Workspace configuration
```

Dependency Order

Packages depend on each other in this order:

```
shared → pycypher → fastopendata
```

- shared has no dependencies on other packages
- pycypher depends on shared
- fastopendata depends on shared and pycypher

7.1.2 PyCypher Architecture

Core Components

Grammar Parser

Uses Lark with the Earley algorithm for maximum grammar expressiveness. The compiled parser is cached as a process-level singleton via `get_default_parser()`; do not construct `GrammarParser()` directly in application code:

```
from pycypher.grammar_parser import get_default_parser

# Returns the cached singleton — safe to call repeatedly
parser = get_default_parser()
raw_ast = parser.parse_to_ast(query)
```

Grammar Rule Mixins (modular transformer architecture)

Grammar transformation rules are organized into five focused mixin modules under `pycypher.grammar_rule_mixins/`:

```
grammar_rule_mixins/
  __init__.py    # Re-exports all 5 mixins
  literals.py    # LiteralRulesMixin — numbers, strings, booleans, null, lists, maps
  expressions.py # ExpressionRulesMixin — boolean, comparison, arithmetic, string ops
  patterns.py    # PatternRulesMixin — node/relationship patterns, labels, properties
  functions.py   # FunctionRulesMixin — function calls, CASE, comprehensions, REDUCE
  clauses.py     # ClauseRulesMixin — MATCH, RETURN, WITH, SET, DELETE, CREATE, ↪ MERGE
```

`CompositeTransformer` (in `grammar_transformers.py`) orchestrates these mixins with method-resolution caching for thread-safe concurrent AST warmup.

AST Models

Pydantic-based strongly-typed AST nodes:

```
from pycypher.ast_models import (
    Query, Match, Return,
    NodePattern, Variable,
    ASTConverter
)

# All nodes are Pydantic models
# - Automatic validation
# - Type safety
# - Serialization support
```

BindingFrame IR

The core intermediate representation for query execution:

```
from pycypher.binding_frame import BindingFrame, EntityScan, BindingFilter
from pycypher.binding_evaluator import BindingExpressionEvaluator

# BindingFrame: DataFrame whose columns ARE Cypher variable names
# No column prefixing, no hash IDs, no variable_map threading
scan = EntityScan(entity_type="Person", var_name="p")
frame = scan.scan(context)

# Properties fetched on demand — never stored in the frame
names = frame.get_property("p", "name")
```

Relational Models

Data containers for entity and relationship tables:

```
from pycypher.relational_models import (
    EntityTable,      # Holds entity IDs + attributes
    RelationshipTable, # Holds relationship IDs + source/target
    Context,          # Holds all tables + registered functions
)
```

7.1.3 Design Patterns

Visitor Pattern

AST traversal uses visitor pattern:

```
class MyVisitor:
    def visit_Match(self, node):
        # Process Match nodes
        pass

    def visit_Return(self, node):
        # Process Return nodes
        pass
```

Used in:

- AST conversion (raw → typed)
- Validation
- Translation to BindingFrame operations

BindingFrame Execution Model

Variables are column names. Each row is one possible assignment of entity IDs to those variables. Attributes are fetched on demand by ID-keyed lookup against the entity table in the Context:

```
# EntityScan: produces single-column frame
# Person IDs in column "p"

# RelationshipScan: produces three-column frame
# rel IDs in "r", source in "_src_r", target in "_tgt_r"

# join(left_col, right_col): inner join on the shared structural key
# filter(mask): boolean mask filter
```

Benefits over legacy relational algebra:

- No opaque 32-hex HASH_ID column names
- No EntityType__propertyname column names bleeding into operator logic
- No variable_map metadata threading through every operator
- Variables as column names: the column is the variable

Query-Scoped Atomicity

SET clause mutations are buffered in a shadow layer on Context and only committed on successful query completion:

```
context.begin_query() # initialise shadow layer
try:
    result = execute_inner(query)
    context.commit_query() # promote shadows to canonical tables
    return result
except Exception:
    context.rollback_query() # discard all pending mutations
    raise
```

This ensures a failed query never leaves the context in a partially-mutated state.

7.1.4 Extension Points

Custom Cypher Functions

Register user-defined functions via the `@context.cypher_function` decorator:

```
from pycypher.relational_models import Context

context = Context()

@context.cypher_function
def my_function(x):
    return x * 2

# Now callable as my_function(n.value) in Cypher queries
```

Custom Scalar Functions

Register additional scalar functions in the `ScalarFunctionRegistry`:

```
from pycypher.scalar_functions import ScalarFunctionRegistry

registry = ScalarFunctionRegistry.get_instance()
registry.register_function(
    "myFunc", lambda s: s.apply(str.upper), min_args=1, max_args=1,
)
```

7.1.5 Testing Architecture

Test Organization

Tests are co-located in the `tests/` directory:

```
tests/
  test_ast_models.py           # AST node tests
  test_ast_models_coverage_gaps.py # Coverage improvements
  test_grammar_parser.py       # Parser tests
  test_data_correctness_*.py    # Data correctness tests
  test_semantic_validator.py    # Validation tests
```

(continues on next page)

(continued from previous page)

```
test_binding_frame.py      # BindingFrame IR tests
test_star_gaps.py         # Star execution tests
```

Conventions:

- Use pytest fixtures for common test data
- Separate unit tests from integration tests
- Aim for >90% coverage on new code

Test Execution

```
# Run all tests
uv run pytest

# Run with coverage
uv run pytest --cov=pycypher --cov-report=html

# Run specific tests
uv run pytest tests/test_ast_models.py

# Parallel execution
uv run pytest -n 4
```

7.1.6 Configuration Management

Environment Management

Use uv for all Python operations:

```
uv sync

# Run Python scripts
uv run python script.py

# Install packages
uv pip install <package>
```

Never use pip directly - always use uv pip to maintain workspace consistency.

Type Checking

Use ty (not mypy) for type checking:

```
uv run ty check
```

Requirements:

- All functions MUST have type annotations
- Python 3.14.0a6+freethreaded required
- Strict type checking enabled

Code Formatting

```
# Format code
make format # Runs isort + ruff format

# Check without modifying
ruff check
```

Configuration: `pyproject.toml` ([`tool.ruff`] section) with `select = ["ALL"]` (highly strict)

7.1.7 Build System

Makefile Targets

```
make test      # Run test suite
make coverage  # Generate coverage report
make format    # Format code
make docs      # Build documentation
make clean     # Remove build artifacts
```

Build ordering: The Makefile handles workspace dependencies automatically.

Package Management

Workspace packages reference each other via `tool.uv.sources`:

```
[tool.uv.sources]
pycypher = { workspace = true }
shared = { workspace = true }
```

After editing “`pyproject.toml`”: Always run `uv sync`

7.1.8 Performance Characteristics

Parser

- Grammar: Earley parser (handles ambiguous grammars; trades speed for expressiveness vs LALR)
- Caching: Grammar compiled once per process (`get_default_parser()` singleton). Parse result (AST) also cached by query string with LRU capacity 512 — repeated queries are $O(1)$ lookups.
- Cold parse: ~56 ms per unique query; warm (cached): < 0.1 ms.

Query Execution

- Translation: $O(n)$ in AST size
- Property lookups: ID-keyed, vectorised via pandas `set_index` + `map`; never fetched unless referenced in a clause.
- Aggregation: Grouped aggregation uses pandas native Cython paths (“count”, “mean”, “min”, “max”, `grouped.sum()`, `grouped.std()`) rather than per-group Python callbacks.
- Scalar functions: Math functions (`abs`, `ceil`, `floor`, `sign`, `sqrt`, `cbrt`, `log`, `log2`, `log10`, `exp`, `pow`, and all trig functions) use numpy C-level array operations via a `_make_math1_np` / `_make_trig1_np` factory pattern — one numpy call per Series regardless of row count. String predicate functions (`startsWith`, `endsWith`, `contains`) use the pandas `.str` Cython accessor. These replace the previous `pd.Series.apply()` paths (one Python call per row) and give ~3–5× speedup on large frames.

- Performance tips: Push WHERE predicates early; use inline property filters {prop: val} on node patterns to reduce scan sizes; avoid unbounded variable-length paths (*) on large graphs.

7.1.9 For More Information

- See [Testing](#) for test guidelines
- See [Contributing](#) for development workflow
- See [Release Process](#) for release process

7.2 Architecture Decision Records

This directory records significant architectural decisions made in PyCypher. Each ADR describes the context, decision, alternatives considered, and consequences.

Format: ADRs follow the [Michael Nygard template](#) — Context, Decision, Alternatives, Consequences.

Status values: Accepted (active), Superseded (replaced by another ADR), Deprecated (no longer relevant).

7.2.1 ADR-001: BindingFrame as Core Intermediate Representation

Status

Accepted

Date

2025-01

Relates to

packages/pycypher/src/pycypher/binding_frame.py

Context

The original query execution engine used opaque relational algebra operators with 32-character hex HASH_ID column names and EntityType__propertyname prefixed columns. Every operator had to thread a variable_map dictionary to translate between Cypher variable names and internal column names. This was fragile, error-prone, and made debugging nearly impossible — a DataFrame dump showed hex IDs rather than recognizable variable names.

Decision

Replace the legacy relational algebra with BindingFrame, a DataFrame-native IR where column names are Cypher variable names. Each row represents one possible assignment of entity/relationship IDs to those variables. Properties are fetched on demand via get_property(var, prop) rather than being stored in the frame.

Key operations:

- EntityScan(entity_type, var_name) — produces a single-column frame
- RelationshipScan(rel_type, var_name) — produces a three-column frame (rel ID, source, target)
- join(left_col, right_col) — inner join on shared structural key
- filter(mask) — boolean mask filter

Alternatives Considered

1. Keep legacy relational algebra — Too fragile; `variable_map` threading was the root cause of multiple production bugs.
2. Prefixed column names (`Person__name`) — Still bleeds internal naming conventions into operator logic and forces every operator to understand the prefix format.
3. Hash-ID columns with external symbol table — Marginally cleaner but still requires lookup-table indirection for every property access.

Consequences

- Eliminates three fragility sources: opaque `HASH_IDs`, prefixed columns, `variable_map` threading.
- Self-documenting: a `DataFrame` dump shows `p, r, q` instead of `a7f3...`
- On-demand property fetch via ID-keyed vectorized lookup avoids fetching unused properties.
- Requires `GraphIndexManager` for efficient property/adjacency lookups.
- All evaluators, the pattern matcher, and the mutation engine operate directly on `BindingFrame` columns.

7.2.2 ADR-002: Lark Parser with Earley Algorithm

Status

Accepted

Date

2024

Relates to

`packages/pycypher/src/pycypher/grammar_parser.py`

Context

PyCypher needs a complete Cypher grammar parser. Cypher has complex operator precedence, optional clauses, and syntactic ambiguities (e.g., `MATCH (n)` could be a function call or a node pattern depending on context). The parser must handle the full grammar without requiring manual disambiguation.

Decision

Use Lark with the Earley parsing algorithm. Earley handles ambiguous and context-sensitive grammars without requiring the grammar author to restructure rules for lookahead constraints.

The compiled parser is cached as a process-level singleton via `get_default_parser()`; application code must not construct `GrammarParser()` directly. Parse results (ASTs) are also cached with LRU capacity of 512 entries.

Alternatives Considered

1. LALR parser (Lark also supports LALR) — Faster, but requires the grammar to be unambiguous. Cypher's syntax makes this difficult without significant grammar restructuring.
2. ANTLR — Powerful but requires a Java runtime for grammar compilation and generates code that is harder to integrate into a pure-Python project.
3. Hand-written recursive descent — Maximum flexibility but extremely labor-intensive for a grammar as large as Cypher, and difficult to maintain as the grammar evolves.

Consequences

- Cold parse: ~56 ms per unique query; warm (cached): < 0.1 ms.
- Grammar changes are expressed declaratively in .lark files rather than imperative parser code.
- Earley's $O(n^3)$ worst case is acceptable for query strings (typically < 1 KB).
- CompositeTransformer converts raw parse trees to typed Pydantic AST nodes via Lark's visitor/transformer mechanism.

7.2.3 ADR-003: Pydantic-Based Strongly-Typed AST Models

Status

Accepted

Date

2024

Relates to

packages/pycypher/src/pycypher/ast_models.py

Context

After Lark parses a Cypher query, the result is a raw parse tree of dictionaries and lists. Downstream consumers (semantic validation, query planning, evaluators) need to safely navigate and pattern-match on AST nodes. Dict-based trees are error-prone — a typo in a key name produces None rather than an error, and there is no IDE support for available fields.

Decision

Define all AST node types as Pydantic BaseModel classes with explicit field declarations and type annotations. ASTConverter translates the raw Lark parse tree into typed Pydantic nodes.

Examples: Query, Match, Return, NodePattern, Variable, PropertyAccess, FunctionCall, BinaryOp.

Alternatives Considered

1. Dict-based AST — Standard in many parser projects but loses type safety, IDE support, and validation.
2. Dataclasses — Lighter weight but lacks built-in validation, JSON serialization, and schema generation that Pydantic provides.
3. TypedDict — Provides type hints but no runtime validation; still dict-based so no method dispatch.

Consequences

- AST node shapes are validated at construction time — malformed trees fail fast with clear error messages.
- IDE autocompletion and type checking work across the entire AST traversal pipeline.
- Variables are uniformly represented as Variable instances (not strings), eliminating a class of type confusion bugs.
- Serialization support enables AST caching and inter-process communication.
- Slight construction overhead vs raw dicts, negligible relative to parse time.

7.2.4 ADR-004: Query-Scoped Shadow Write Atomicity

Status
Accepted

Date
2025

Relates to
packages/pycypher/src/pycypher/mutation_engine.py

Context

Cypher queries with SET, CREATE, or DELETE clauses mutate the in-memory graph. If a query fails mid-execution (e.g., a type error in a later clause), the context could be left in a partially-mutated state — some entities updated, others not. This violates the principle of least surprise and makes error recovery impossible.

Decision

Buffer all mutations in a per-query shadow layer on Context. The mutation lifecycle is:

```
context.begin_query() # initialise shadow layer
try:
    result = execute_inner(query)
    context.commit_query() # promote shadows to canonical tables
    return result
except Exception:
    context.rollback_query() # discard all pending mutations
    raise
```

MutationEngine coordinates CREATE, SET, DELETE, REMOVE, and MERGE operations, writing exclusively to the shadow layer until commit.

Alternatives Considered

1. Direct mutation — Simplest implementation but breaks atomicity; no recovery from partial failures.
2. Per-clause checkpoints — More granular but significantly more complex; unclear what “partial success” means for a multi-clause query.
3. Undo log with replay — Possible but adds complexity for a scenario where discard-and-retry is simpler and sufficient.

Consequences

- Failed queries never leave the context in a partially-mutated state.
- Shadow layer memory is proportional to the number of mutated entities (not total graph size).
- Query semantics are predictable and testable.
- MERGE operations can check existing state while buffering new writes.

7.2.5 ADR-005: Composable Evaluator Pattern

Status
Accepted

Date

2025

Relates to

`packages/pycypher/src/pycypher/evaluator_protocol.py`

Context

Expression evaluation in Cypher spans arithmetic, boolean logic, string predicates, comparisons, aggregation, collection operations, and function calls. A single monolithic evaluator class would be thousands of lines long and create circular import dependencies between the expression evaluator and the aggregation system.

Decision

Decompose expression evaluation into focused, single-responsibility evaluator classes that compose via delegation through an `ExpressionEvaluatorProtocol`:

- `BindingExpressionEvaluator` — main coordinator
- `ArithmeticEvaluator` — `+`, `-`, `*`, `/`, `%`, `^`
- `BooleanEvaluator` — `AND`, `OR`, `NOT`, `XOR`
- `ComparisonEvaluator` — `=`, `<>`, `<`, `>`, `<=`, `>=`
- `ScalarFunctionEvaluator` — built-in and user-defined functions
- `StringPredicateEvaluator` — `STARTS WITH`, `ENDS WITH`, `CONTAINS`
- `AggregationEvaluator` — `COUNT`, `SUM`, `AVG`, `COLLECT`, etc.
- `CollectionEvaluator` — list comprehensions, `UNWIND`, `range()`
- `ExistsEvaluator` — `EXISTS { ... }` subqueries

The `ExpressionEvaluatorProtocol` breaks circular imports by defining the interface that evaluators depend on rather than concrete implementations.

Alternatives Considered

1. Monolithic evaluator — Simpler dispatch but creates a multi-thousand-line file with circular import issues.
2. Visitor pattern with separate visitors per expression type — More traditional but harder to compose and share evaluation context.
3. Expression compilation to bytecode — Maximum performance but significantly more complex; premature optimization for current workloads.

Consequences

- Each evaluator can be developed, tested, and optimized independently.
- Vectorization is applied per-evaluator (numpy for math, pandas .str for strings).
- Circular import chains eliminated via protocol-based dependency inversion.
- Slightly higher dispatch overhead, negligible vs actual computation cost.

7.2.6 ADR-006: Pluggable Backend Engine Protocol

Status
Accepted

Date
2025

Relates to
packages/pycypher/src/pycypher/backend_engine.py

Context

PyCypher’s execution engine was tightly coupled to pandas. For large datasets, pandas becomes a bottleneck — it cannot push predicates into native C code and struggles with datasets that exceed memory. Users need the option to use DuckDB (analytical SQL), Polars (Arrow-native lazy evaluation), or Dask (distributed) without changing their Cypher queries.

Decision

Define a BackendEngine protocol that captures ~15 primitive DataFrame operations (filter, join, project, aggregate, sort, limit, etc.). Three implementations:

- PandasBackend — default, zero-cost wrapper around existing pandas code
- DuckDBBackend — pushes operations to DuckDB’s SQL engine for analytical workloads
- PolarsBackend — uses Polars lazy evaluation for Arrow-native execution

Selection via Context(backend=“duckdb”) or PYCYPHER_BACKEND environment variable. The “auto” mode selects the backend based on dataset size and available libraries.

Alternatives Considered

1. Hard-code pandas throughout — No migration path for large datasets.
2. Abstract all 918 pandas API calls — Discovered during audit; impractical to abstract every call. The ~15 primitive operations capture the essential semantics.
3. Use an existing dataframe abstraction library (e.g., Ibis, Narwhals) — None were mature enough at decision time, and the abstraction surface was small enough to own.

Consequences

- Users opt into DuckDB/Polars transparently — no query changes needed.
- User-facing API always returns pd.DataFrame via to_pandas() escape hatch for compatibility.
- All existing pandas-based code unaffected; backend adoption is incremental.
- ~15 operations is a manageable abstraction surface to maintain.

7.2.7 ADR-007: Graph-Native Index Structures

Status
Accepted

Date
2025

Relates to
packages/pycypher/src/pycypher/graph_index.py

Context

Pattern matching in Cypher requires finding neighbors, looking up entities by property values, and filtering by labels. Without indexes, every pattern match performs a full table scan — $O(E)$ for each relationship traversal and $O(N)$ for each property filter. For graphs with millions of edges, this makes interactive queries impractical.

Decision

Implement three graph-native index types managed by `GraphIndexManager`:

- `AdjacencyIndex` — maps (`entity_id`, `direction`) to neighbor lists. Reduces neighbor lookups from $O(E)$ to $O(\text{degree})$.
- `PropertyValueIndex` — maps (`entity_type`, `property`, `value`) to ID sets via hash table. Reduces equality filters from $O(N)$ to $O(1)$.
- `EntityLabelIndex` — sorted ID arrays per label. Reduces label membership tests to $O(\log n)$ via binary search.

Indexes are built lazily on first access and invalidated on mutations (`CREATE`, `SET`, `DELETE`). Index data uses frozen tuples for thread-safe concurrent reads.

Alternatives Considered

1. No indexing — Unacceptable for any non-trivial graph size.
2. B-tree indexes — More overhead for construction and maintenance; similar lookup performance for equality queries, slightly better for range queries which are uncommon in pattern matching.
3. Hash-only indexes — Would work for equality but not ordered access; the sorted label index enables efficient range scans and set intersection.

Consequences

- Pattern matching goes from $O(E)$ to $O(\text{degree})$ for each relationship traversal.
- Enables the query planner to choose between hash join, broadcast join, merge join, or nested-loop join based on index availability and estimated cardinalities.
- Memory cost proportional to data size (index storage mirrors data storage).
- Lazy construction avoids upfront cost for indexes that may never be needed.
- Invalidation on mutation ensures index consistency without explicit management.

7.2.8 ADR-008: Monorepo Workspace Structure

Status

Accepted

Date

2024

Relates to

pyproject.toml (root workspace configuration)

Context

The project contains three logical components: shared utilities, the Cypher query engine, and an ETL pipeline for open data. These share common logging, configuration, and helper code. They need to be developed, tested, and (in some cases) released independently.

Decision

Use a uv workspace monorepo with three packages:

```
packages/
  shared/      # Common utilities, logging (no internal deps)
  pycypher/    # Cypher parser and query engine (depends on shared)
  fastopendata/ # ETL pipeline (depends on shared + pycypher)
```

Cross-package dependencies are declared in `tool.uv.sources` as workspace references. Dependency groups (`dev-core`, `dev`, `dev-full`) provide layered developer environments from lightweight to comprehensive.

Alternatives Considered

1. Separate git repositories with pip/uv dependencies — Adds friction to cross-package changes (separate PRs, version pinning, publish cycles).
2. Single monolithic package — Simpler packaging but forces all users to install ETL dependencies even if they only need the query engine.
3. Git submodules — Historically fragile; adds clone complexity and makes atomic cross-package commits difficult.

Consequences

- Cross-package changes are atomic (single commit, single PR).
- Dependency order (`shared` → `pycypher` → `fastopendata`) prevents circular imports at the package level.
- `uv sync` must be run after `pyproject.toml` changes — CI enforces this with a lock-check job.
- Users can install `pycypher` alone without `fastopendata` dependencies.
- All tests live in a single `tests/` directory for unified test execution.

7.2.9 ADR-009: Star as Central Query Coordinator

Status

Accepted (with known limitations)

Date

2024

Relates to

`packages/pycypher/src/pycypher/star.py`

Context

Users need a single entry point to execute Cypher queries against an in-memory graph. The execution pipeline involves parsing, AST conversion, semantic validation, query planning, clause-by-clause execution, mutation coordination, and result formatting. These steps must be orchestrated in the correct order with proper error handling and resource management.

Decision

Implement Star as the main user-facing coordinator. The primary API is:

```
star = Star(context=context)
result = star.execute_query("MATCH (p:Person) RETURN p.name")
```

Star delegates to focused components:

- `get_default_parser()` for parsing
- `ASTConverter` for AST conversion
- `SemanticValidator` for pre-execution validation
- `PatternMatcher` for `MATCH` clause execution
- `BindingExpressionEvaluator` for expression evaluation
- `MutationEngine` for `CREATE/SET/DELETE`
- `QueryProfiler` for timing (optional)

Alternatives Considered

1. Separate parser, planner, executor classes with factory — More modular but adds ceremony for the common case. Users would need to assemble a pipeline before executing a query.
2. Functional pipeline (compose functions) — Clean but makes shared state (context, profiler, cache) awkward to thread through.
3. Builder pattern — `QueryBuilder().parse(q).validate().plan().execute()` — Flexible but verbose for the 90% case of “just run this query.”

Consequences

- Simple, discoverable API: `Star.execute_query()` does everything.
- Star is becoming a god object — it coordinates too many concerns (see Task #6: “Decompose Star class”). Future work should extract a `QueryExecutor` that owns the pipeline while Star remains a thin user-facing facade.
- Adding new pipeline stages (e.g., cost-based planning) requires modifying Star, which is not ideal for extensibility.

7.2.10 ADR-010: Vectorized Scalar Function Evaluation

Status

Accepted

Date

2025

Relates to

`packages/pycypher/src/pycypher/scalar_function_evaluator.py`

Context

Scalar functions (math, string, type predicates) were initially implemented using `pd.Series.apply(func)` which calls a Python function once per row. For large DataFrames (100K+ rows), this is 10–100x slower than vectorized alternatives because of Python function-call overhead per row.

Decision

Replace `pd.Series.apply()` with vectorized operations wherever possible:

- Math functions (`abs`, `ceil`, `floor`, `sqrt`, `log`, trig functions, etc.) use numpy C-level array operations via `_make_math1_np()` / `_make_trig1_np()` factory functions — one numpy call per Series regardless of row count.
- String predicates (`STARTS WITH`, `ENDS WITH`, `CONTAINS`) use the pandas `.str` Cython accessor (`.str.startswith()`, `.str.endswith()`, `.str.contains()`).
- Aggregations (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) use pandas native grouped Cython paths rather than per-group Python callbacks.

Alternatives Considered

1. Keep “`pd.Series.apply()`” — Simplest but unacceptable performance on large datasets.
2. Numba JIT compilation — Maximum performance for numeric operations but adds a heavy dependency and compilation latency on first call.
3. Compile expressions to SQL for DuckDB execution — Attractive for the DuckDB backend but does not help the pandas-only path which is the default.

Consequences

- 3–5x speedup on large frames for math and string operations.
- Null handling requires explicit pre-allocation and masking (numpy operations do not propagate pandas NA natively).
- Math functions must be stateless to be vectorizable.
- Tight coupling to numpy/pandas internals — tested against pinned versions to prevent breakage on library updates.

7.2.11 ADR-011: Grammar Rule Mixins Package Split

Status

Accepted

Date

2026-03

Relates to

`packages/pycypher/src/pycypher/grammar_rule_mixins/`

Supersedes

Original monolithic `grammar_rule_mixins.py`

Context

The original `grammar_rule_mixins.py` was a single file containing all Lark transformer methods for converting raw parse trees into Pydantic AST nodes. As the Cypher grammar grew, this file exceeded 3,000 lines covering five unrelated domains (literals, expressions, patterns, functions, clauses). This made the file difficult to navigate, review, and test in isolation.

Additionally, during concurrent AST cache warmup, per-call Rich logging in the monolithic transformer caused `OSError: bad file descriptor errors` due to file descriptor contention.

Decision

Split the monolithic file into a Python package with five focused mixin modules:

```
grammar_rule_mixins/
__init__.py      # Re-exports all 5 mixins
literals.py      # LiteralRulesMixin (278 lines)
expressions.py   # ExpressionRulesMixin (683 lines)
patterns.py      # PatternRulesMixin (520 lines)
functions.py     # FunctionRulesMixin (594 lines)
clauses.py       # ClauseRulesMixin (1,165 lines)
```

CompositeTransformer in grammar_transformers.py delegates to specialized transformer classes (LiteralTransformer, ExpressionTransformer, PatternTransformer, StatementTransformer) that inherit from these mixins. Method resolution is cached on the instance after first lookup to avoid repeated MRO iteration during concurrent warmup.

Alternatives Considered

1. Keep monolithic file — Too large for effective review and navigation; concurrent Rich logging bug made this untenable.
2. Separate transformer classes per rule group — Would break Lark’s method-name-based dispatch, which expects all visitor methods on a single transformer instance.
3. Decorator-based organization — Methods stay in one file but are tagged by domain. Doesn’t reduce file size or enable independent testing.

Consequences

- Each mixin can be understood, reviewed, and tested independently.
- Lark’s visitor mechanism is preserved via Python MRO (multiple inheritance).
- `__init__.py` re-exports all mixins, so from `pycypher.grammar_rule_mixins` import `ClauseRulesMixin` continues to work.
- Method-resolution caching eliminates the Rich `OSError` during concurrent AST cache warmup.
- Import path changed from module to package — existing from `pycypher.grammar_rule_mixins` import ... imports are unaffected.

7.2.12 ADR-012: Automatic Test Markers and Isolation Fixtures

Status

Accepted

Date

2026-03

Relates to

tests/conftest.py

Context

With ~300 test files across multiple directories, manually applying pytest markers (`@pytest.mark.integration`, `@pytest.mark.performance`, etc.) was inconsistent — many tests lacked markers, making it difficult to run targeted subsets (e.g., `-m "not slow"`). Additionally, two classes of cross-test contamination caused flaky failures:

1. SIGALRM leakage — Tests using `timeout_seconds` set a POSIX alarm that could fire during a subsequent, unrelated test, causing spurious `QueryTimeoutError`.
2. Logger level contamination — Tests using `caplog.at_level(DEBUG)` left `shared.logger.LOGGER` in `DEBUG` mode, causing subsequent tests to emit unexpected log output.

Decision

Add automatic marker inference and isolation fixtures to `conftest.py`:

Auto-markers (applied when no explicit marker is present):

- `tests/benchmarks/` → performance
- `tests/large_dataset/` → integration + slow
- `tests/load_testing/` → performance + slow
- `tests/property_based/` → unit
- Filenames containing `_e2e_` or `_end_to_end` → integration
- Filenames containing `_performance_` → performance
- Filenames containing `_security_` → security

Per-marker timeouts:

- integration: 120s (vs 30s default)
- slow: 300s (vs 30s default)

Isolation fixtures (auto-use, session-wide):

- `_clear_pending_sigalarm()` — Resets SIGALRM after each test.
- `_restore_shared_logger_level()` — Saves and restores logger level.

Alternatives Considered

1. Manual markers everywhere — Reliable but requires discipline; 300+ files means markers would drift out of date.
2. pytest plugin with custom collection hook — More powerful but adds external dependency or maintenance burden for a custom plugin.
3. Directory-based `conftest.py` files — Each subdirectory gets its own `conftest.py` that applies markers. Works but scatters configuration across many files.

Consequences

- Running `pytest -m "not slow"` now reliably excludes all slow tests, even those that were never manually marked.
- Explicit `@pytest.mark.X` always takes precedence — auto-markers never override developer intent.
- SIGALRM and logger contamination eliminated, reducing flaky test rate.
- `perf_threshold()` helper scales timing assertions for CI environments (configurable via `PY-CYPHER_PERF_MULTIPLIER`).
- New test files automatically get appropriate markers based on their location — no action needed from the developer.

7.3 Contributing

How to set up a development environment and contribute to PyCypher.

In this guide

- Development Environment
 - Requirements
 - Initial Setup
 - Project Structure
- Code Style
 - Formatting
 - Type Annotations
 - Docstrings
- Running Tests
- Making Changes
 - Branch Naming
 - Commit Messages
- Pull Request Process
 - Code Review
- Adding Dependencies
- Documentation
- Editor Integration (LSP)
 - Features
 - Starting the Server
 - VS Code
 - Neovim (nvim-lspconfig)
 - Emacs (lsp-mode)
 - File Type Detection

7.3.1 Development Environment

Requirements

- Python 3.14 or higher (free-threaded build recommended: 3.14t)
- `uv` for dependency management
- Docker (via Rancher Desktop or Docker Desktop) for containerised testing
- Git

Initial Setup

```
# Clone the repository
git clone <repository-url>
cd pycypher-nmetl

# Install dependencies (uv manages the workspace virtualenv)
uv sync

# Verify the setup
uv run pytest --co -q  # list collected tests without running
```

All Python operations go through uv — it manages the workspace virtual environment automatically.

Project Structure

This is a monorepo workspace with two interdependent packages:

```
packages/
  pycypher/  # Cypher parser, AST, relational algebra, ETL
  shared/    # Common utilities, logging, telemetry

tests/       # All tests live at the repo root
docs/        # Sphinx documentation
examples/    # Runnable example scripts
```

Dependency order: shared → pycypher. The Makefile handles build ordering automatically.

7.3.2 Code Style

Formatting

```
make format  # Runs isort + ruff format
```

Ruff is configured in the root pyproject.toml with select = ["ALL"] (highly strict linting). Fix any issues before committing.

Type Annotations

All functions and methods must have type annotations. Use ty (not mypy) for type checking:

```
uv run ty check
```

Docstrings

Use [Google-style docstrings](#). Sphinx autodoc extracts documentation from source, so always update docstrings when modifying public APIs.

```
def example_function(name: str, count: int = 1) -> list[str]:
    """Short summary of what this function does.

    Args:
        name: Description of name parameter.
        count: Description of count parameter.
```

(continues on next page)

(continued from previous page)

Returns:
Description of return value.

Raises:
ValueError: When name is empty.

"""

7.3.3 Running Tests

```
# Quick iteration during development (parallel, minimal output, skips slow tests)
make test-quick

# Run all tests (parallel, 8 threads)
make test

# Run a specific test file
make test-file FILE=tests/test_ast_models.py

# Stop on first failure
make test-fast

# Re-run only previously failed tests
make test-failed

# Run with coverage report
make coverage
```

See [Testing](#) for the full testing guide.

7.3.4 Making Changes

Branch Naming

Use descriptive branch names that indicate the type of change:

- feature/add-temporal-functions
- fix/undefined-variable-error
- docs/update-contributing-guide

Commit Messages

Write clear, concise commit messages:

- Start with an imperative verb (Add, Fix, Update, Remove, Refactor)
- Keep the first line under 72 characters
- Reference issues or context in the body when relevant

7.3.5 Pull Request Process

1. Create a branch from main
2. Make your changes — keep PRs focused on a single concern
3. Run the pre-PR check — a single command that formats, lints, type-checks, and runs the test suite:

```
make check # runs: lock-check → format → lint → typecheck → test-fast
```

4. Push and open a PR against main

PR descriptions should include:

- A summary of what changed and why
- How to test the changes
- Any breaking changes or migration notes

Code Review

- All PRs require review before merging
- Reviewers check for correctness, test coverage, and adherence to project conventions
- Address review feedback promptly; use the conversation to discuss trade-offs

7.3.6 Adding Dependencies

```
# Edit the relevant package's pyproject.toml, then:
uv sync
```

Never use `pip install` directly — the workspace relies on `uv` for consistent dependency resolution.

7.3.7 Documentation

```
# Build docs
make docs

# Or directly:
uv run sphinx-build -b html docs docs/_build/html
```

Update documentation alongside code changes. See [Tutorials](#) for tutorial conventions.

7.3.8 Editor Integration (LSP)

PyCypher includes a built-in Language Server Protocol (LSP) server that provides real-time Cypher query assistance in any LSP-capable editor. No external extensions are required — the server uses only the standard library.

Features

- Diagnostics — parse errors and semantic validation as you type
- Completion — keywords, functions, and entity labels
- Hover — function documentation and signatures
- Signature Help — parameter hints inside function calls

- Go to Definition — jump to variable definitions
- Formatting — auto-format Cypher queries

Starting the Server

```
python -m pycypher.cypher_lsp
```

The server communicates via JSON-RPC over stdin/stdout per the LSP specification.

VS Code

Add to your `.vscode/settings.json`:

```
{
  "pycypher.lspServer.path": "python -m pycypher.cypher_lsp"
}
```

For workspace-specific configuration, create `.vscode/settings.json` at the repository root.

Neovim (nvim-lspconfig)

Add to your Neovim configuration (`init.lua` or equivalent):

```
local lspconfig = require('lspconfig')
local configs = require('lspconfig.configs')

if not configs.pycypher then
  configs.pycypher = {
    default_config = {
      cmd = { 'python', '-m', 'pycypher.cypher_lsp' },
      filetypes = { 'cypher' },
      root_dir = lspconfig.util.find_git_ancestor,
      settings = {},
    },
  }
end

lspconfig.pycypher.setup({})
```

Emacs (lsp-mode)

Add to your Emacs configuration:

```
(with-eval-after-load 'lsp-mode
  (add-to-list 'lsp-language-id-configuration '(cypher-mode . "cypher"))
  (lsp-register-client
    (make-lsp-client
      :new-connection (lsp-stdio-connection '("python" "-m" "pycypher.cypher_lsp"))
      :activation-fn (lsp-activate-on "cypher")
      :server-id 'pycypher-lsp)))
```

File Type Detection

Most editors need to associate .cypher files with the Cypher file type. For Neovim, add:

```
vim.filetype.add({ extension = { cypher = 'cypher' } })
```

For VS Code, add to settings.json:

```
{
  "files.associations": {
    "*.cypher": "cypher"
  }
}
```

7.4 Testing

Comprehensive guide to running, writing, and maintaining tests for PyCypher.

On this page

- Running Tests
 - Quick Reference
 - Targeting Specific Tests
 - Coverage
- Test Architecture
 - Directory Layout
 - Automatic Test Markers
 - Test Isolation
 - Shared Fixtures
- Writing Tests
 - Test Structure
 - Testing Query Execution (End-to-End)
 - Testing AST Construction
 - Testing Error Paths
 - Testing Mutations (CREATE, SET, DELETE)
 - Testing Shadow State Atomicity
- Performance Testing
 - Writing Robust Performance Tests
 - Benchmark Tests
- Integration Tests
 - Spark Integration

- Neo4j Integration
- Large Dataset Tests
- Configuration for Testing
 - Environment Variables
 - Timeout Configuration
- Continuous Integration
- Type Checking

7.4.1 Running Tests

Quick Reference

All commands use `uv run` to execute within the workspace virtual environment.

```
# Run all tests (parallel)
make test

# Run all tests (serial, useful for debugging)
make test-serial

# Fast: stop on first failure, parallel
make test-fast

# Quick: minimal output, no coverage
make test-quick

# Re-run only previously failed tests
make test-failed

# Re-run failed tests first, then all
make test-changed
```

Targeting Specific Tests

```
# Single file
uv run pytest tests/test_ast_models.py

# Single test class
uv run pytest tests/test_ast_models.py::TestPatterns

# Single test method
uv run pytest tests/test_ast_models.py::TestPatterns::test_node_pattern_basic

# By keyword pattern (matches test names)
uv run pytest -k "aggregation and not percentile"

# By marker
uv run pytest -m "not integration"
```

Coverage

```
# HTML report in coverage_report/
make coverage

# Detailed terminal + HTML report
make coverage-detailed

# Inline coverage for a specific file
uv run pytest tests/test_star_gaps.py --cov=pycypher.star --cov-report=term-missing
```

7.4.2 Test Architecture

Directory Layout

All tests live in `tests/` at the repository root. The project uses a flat directory structure — each test file targets a specific module or feature area:

```
tests/
  conftest.py           # Shared fixtures, auto-markers, isolation
  fixtures/data/        # Static test data files (Parquet, CSV)
  benchmarks/          # Performance benchmark tests (auto: performance)
  large_dataset/        # Large-dataset integration tests (auto: integration+slow)
  load_testing/         # Load and stress tests (auto: performance+slow)
  property_based/       # Property-based tests (auto: unit)
  test_ast_models.py    # AST node construction and validation
  test_grammar_parser.py # Lark grammar → raw AST parsing
  test_semantic_validator.py # Static semantic analysis
  test_binding_frame.py  # BindingFrame IR operations
  ...                   # ~300 test files covering all features
```

Automatic Test Markers

Tests are automatically marked based on their file location and name, so you rarely need to add `@pytest.mark` decorators manually.

Directory-based auto-markers:

- `tests/benchmarks/` → performance
- `tests/large_dataset/` → integration + slow
- `tests/load_testing/` → performance + slow
- `tests/property_based/` → unit

Filename-based auto-markers:

- Files containing `_e2e_` or `_end_to_end` → integration
- Files containing `_performance_` → performance
- Files containing `_security_` → security

Explicit `@pytest.mark.X` decorators always take precedence over auto-markers.

Per-marker timeouts are also applied automatically:

- integration tests: 120 seconds (vs 30s default)
- slow tests: 300 seconds (vs 30s default)

Test Isolation

Two auto-use fixtures in `conftest.py` prevent cross-test contamination:

- **SIGALRM cleanup** — Clears pending alarm signals after each test, preventing spurious `QueryTimeoutError` in unrelated tests.
- **Logger level restoration** — Restores `shared.logger.LOGGER` level after each test, preventing `caplog.at_level()` from leaving loggers in `DEBUG` mode.

Shared Fixtures

`conftest.py` provides reusable fixtures for the most common test patterns. Always prefer these over creating one-off test data:

```
def test_basic_query(person_star: Star) -> None:
    """person_star provides a Star with 4 Person entities."""
    result = person_star.execute_query(
        "MATCH (p:Person) RETURN p.name AS name"
    )
    assert len(result) == 4
```

Available fixtures:

`people_df`
4-row Person DataFrame (Alice, Bob, Carol, Dave) with name, age, dept, salary.

`knows_df`
3-row KNOWS relationship DataFrame with since attribute.

`person_star`
Star with Person entities only.

`social_star`
Star with Person entities and KNOWS relationships.

`empty_star`
Star with empty context (no entities or relationships).

`scalar_registry`
Shared `ScalarFunctionRegistry` singleton.

7.4.3 Writing Tests

Test Structure

All test functions and methods must have type annotations. Use `pytest`-style classes to group related tests:

```
from __future__ import annotations

import pandas as pd
import pytest
from pycypher import Star
from pycypher.relational_models import Context, EntityMapping, EntityTable

ID_COLUMN = "__ID__"

@pytest.fixture
```

(continues on next page)

(continued from previous page)

```
def product_star() -> Star:
    """Star with a Product entity table."""
    df = pd.DataFrame({
        ID_COLUMN: [1, 2, 3],
        "name": ["Widget", "Gadget", "Doohickey"],
        "price": [9.99, 19.99, 4.99],
    })
    table = EntityTable.from_dataframe("Product", df)
    ctx = Context(entity_mapping=EntityMapping(mapping={"Product": table}))
    return Star(context=ctx)

class TestProductQueries:
    """Tests for product catalog queries."""

    def test_return_all_products(self, product_star: Star) -> None:
        result = product_star.execute_query(
            "MATCH (p:Product) RETURN p.name AS name"
        )
        assert len(result) == 3
        assert set(result["name"]) == {"Widget", "Gadget", "Doohickey"}

    def test_filter_by_price(self, product_star: Star) -> None:
        result = product_star.execute_query(
            "MATCH (p:Product) WHERE p.price > 10 RETURN p.name AS name"
        )
        assert set(result["name"]) == {"Gadget"}
```

Testing Query Execution (End-to-End)

The most common test pattern exercises the full pipeline — parsing, planning, and execution — via `Star.execute_query()`:

```
def test_aggregation_with_grouping(social_star: Star) -> None:
    """Grouped COUNT aggregation returns correct counts per department."""
    result = social_star.execute_query(
        "MATCH (p:Person) RETURN p.dept AS dept, count(p) AS cnt"
    )
    # Convert to dict for order-independent assertion
    dept_counts = dict(zip(result["dept"], result["cnt"], strict=False))
    assert dept_counts["eng"] == 2
    assert dept_counts["mktg"] == 1
```

Testing AST Construction

For parser and AST tests, validate structure without executing queries:

```
from pycypher.ast_models import ASTConverter, Query, Match, Return

def test_parse_match_return() -> None:
    query = ASTConverter.from_cypher("MATCH (n:Person) RETURN n.name")
```

(continues on next page)

(continued from previous page)

```

assert isinstance(query, Query)
assert len(query.clauses) == 2
assert isinstance(query.clauses[0], Match)
assert isinstance(query.clauses[1], Return)

```

Testing Error Paths

Always verify that errors produce clear, actionable messages:

```

import pytest

def test_empty_query_raises(person_star: Star) -> None:
    with pytest.raises(ValueError, match="must not be empty"):
        person_star.execute_query("")

def test_unknown_function_suggests_correction(person_star: Star) -> None:
    with pytest.raises(Exception, match="(?i)did you mean"):
        person_star.execute_query(
            "MATCH (p:Person) RETURN tUpperr(p.name)"
        )

```

Testing Mutations (CREATE, SET, DELETE)

Mutation tests should verify both the returned result and the context state:

```

def test_set_updates_context(person_star: Star) -> None:
    """SET clause persists changes to the underlying context."""
    person_star.execute_query(
        "MATCH (p:Person) WHERE p.name = 'Alice' SET p.age = 31 RETURN p"
    )
    # Verify the mutation persisted in the context
    person_df = person_star.context.entity_mapping["Person"].source_obj
    alice_row = person_df[person_df["name"] == "Alice"].iloc[0]
    assert alice_row["age"] == 31

```

Testing Shadow State Atomicity

Mutations use a shadow-write pattern (begin → execute → commit/rollback). Verify that failed queries leave the context unchanged:

```

def test_failed_mutation_rolls_back(person_star: Star) -> None:
    """A query that fails mid-execution must not leave partial mutations."""
    original_names = set(
        person_star.context.entity_mapping["Person"].source_obj["name"]
    )
    with pytest.raises(Exception):
        person_star.execute_query("MATCH (p:Person) SET p.INVALID RETURN p")
    current_names = set(
        person_star.context.entity_mapping["Person"].source_obj["name"]
    )
    assert current_names == original_names
    assert person_star.context._shadow == {}

```

7.4.4 Performance Testing

Writing Robust Performance Tests

Performance assertions are inherently noisy. Use statistical approaches to avoid flaky tests:

```
import statistics
import time

def test_view_faster_than_copy() -> None:
    """Verify that DataFrame view is faster than copy for read-only access."""
    n_trials = 7
    copy_times: list[float] = []
    view_times: list[float] = []

    df = pd.DataFrame({"a": range(10_000), "b": range(10_000)})

    for _ in range(n_trials):
        t0 = time.perf_counter()
        for _ in range(50):
            _ = df[["a"]].copy()
        copy_times.append(time.perf_counter() - t0)

        t0 = time.perf_counter()
        for _ in range(50):
            _ = df[["a"]]
        view_times.append(time.perf_counter() - t0)

    # Use median and generous tolerance (3x) for CI stability
    assert statistics.median(view_times) <= statistics.median(copy_times) * 3.0
```

Rules for performance tests:

1. Use median of multiple trials (5) instead of single measurements
2. Apply generous tolerance (2-3x) — test for regressions, not exact speedups
3. Include a descriptive assertion message with actual values for debugging
4. Never assert exact timings — only relative comparisons or order-of-magnitude

Benchmark Tests

The tests/benchmarks/ directory contains pytest-benchmark tests:

```
uv run pytest tests/benchmarks/ --benchmark-only
```

7.4.5 Integration Tests

Spark Integration

Spark tests require a running Spark cluster or use local[*] mode:

```
# Run Spark tests (requires Docker)
make test-spark
```

(continues on next page)

(continued from previous page)

```
# Or directly with local mode
uv run pytest tests/ -k "spark" -v
```

Neo4j Integration

Neo4j tests require a running Neo4j instance:

```
# Start Neo4j via Docker
docker compose up neo4j -d

# Run Neo4j tests
make test-neo4j

# Or with custom connection
NEO4J_URI=bolt://localhost:7687 NEO4J_PASSWORD=secret uv run pytest -k "neo4j"
```

Large Dataset Tests

Tests that exercise distributed backends (Dask, DuckDB, Polars):

```
make test-large-dataset
```

7.4.6 Configuration for Testing

Environment Variables

Tests respect the same PYCYPHER_* environment variables as production (see `pycypher.config`):

PYCYPHER_QUERY_TIMEOUT_S (default: None)

Default query timeout in seconds.

PYCYPHER_MAX_CROSS_JOIN_ROWS (default: 10,000,000)

Hard ceiling on cross-join result size (rows).

PYCYPHER_RESULT_CACHE_MAX_MB (default: 100)

Maximum result cache size in megabytes.

PYCYPHER_RESULT_CACHE_TTL_S (default: 0)

Result cache TTL in seconds (0 = no expiry).

PYCYPHER_MAX_UNBOUNDED_PATH_HOPS (default: 20)

BFS hop limit for unbounded variable-length paths ([*]).

Timeout Configuration

All tests have a 30-second timeout enforced by `pytest-timeout` (configured in `pyproject.toml`). Tests that need longer (e.g. Spark integration) should use:

```
@pytest.mark.timeout(120)
def test_large_spark_query(spark_session):
    ...
```

7.4.7 Continuous Integration

The CI pipeline (.github/workflows/ci.yml) runs three jobs:

1. Tests: Full test suite with `pytest --timeout=30` on Python 3.14t
2. Dependency Compatibility: Verifies all backend imports (Dask, DuckDB, etc.)
3. Documentation: Builds Sphinx docs to catch broken references

Tests must pass before merging to main.

7.4.8 Type Checking

Run type checks with `ty` (not `mypy`):

```
uv run ty check
```

All functions and methods must have type annotations. The project uses Python 3.14.0 (free-threaded build).

7.5 Release Process

How PyCypher versions are numbered, built, and published.

In this guide

- [Version Numbering](#)
- [Pre-Release Checklist](#)
 - [Bump Version](#)
- [Building Packages](#)
- [Publishing](#)
- [Post-Release](#)
- [CI/CD](#)

7.5.1 Version Numbering

PyCypher follows [Semantic Versioning](#) (MAJOR.MINOR.PATCH):

- MAJOR — incompatible API changes
- MINOR — new functionality, backwards-compatible
- PATCH — backwards-compatible bug fixes

The current version is declared in:

- `pyproject.toml` (root workspace)
- `packages/pycypher/pyproject.toml`
- `packages/shared/pyproject.toml`

All three must be updated in sync for a release.

7.5.2 Pre-Release Checklist

Before cutting a release, verify:

1. All tests pass

```
uv run pytest
```

2. Type checking passes

```
uv run ty check
```

3. Formatting is clean

```
make format
```

4. Documentation builds without errors

```
make docs
```

5. CHANGELOG or commit history documents all notable changes since the last release

Bump Version

The Makefile provides a BUMP variable (default: micro):

```
# Patch bump (0.0.1 → 0.0.2)
make bump BUMP=micro

# Minor bump (0.0.1 → 0.1.0)
make bump BUMP=minor

# Major bump (0.0.1 → 1.0.0)
make bump BUMP=major
```

If the bump target is not available, update the version strings in the three pyproject.toml files manually.

7.5.3 Building Packages

```
# Build wheel and sdist for each package
uv build packages/shared
uv build packages/pycypher
```

Built artefacts are placed in dist/.

7.5.4 Publishing

```
# Publish to PyPI (requires credentials)
uv publish dist/*

# Or publish to a test index first
uv publish --index testpypi dist/*
```

Ensure your PyPI credentials are configured (e.g. via ~/.pypirc or environment variables UV_PUBLISH_TOKEN).

7.5.5 Post-Release

After a successful publish:

1. Tag the release in git: `git tag v0.1.0 && git push origin v0.1.0`
2. Create a GitHub Release from the tag with release notes
3. Bump the version in `pyproject.toml` files to the next development version (e.g. `0.1.1.dev0`) if desired

7.5.6 CI/CD

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and PR:

- Tests on Python 3.14 (free-threaded build)
- Linting and formatting checks
- Type checking with `ty`

Releases can be automated by configuring a publish workflow triggered on tagged commits.

7.6 Security Hardening & Compliance

This guide documents PyCypher's security architecture, threat mitigation strategies, and best practices for secure deployment.

On this page

- Threat Model
- Input Validation
 - Cypher query parameters
 - SQL data-source queries
 - File path sanitisation
 - URI scheme validation
- Resource Limits
 - Cross-join row limit
 - Query timeout
 - Memory budget
- Internal Column Protection
- Semantic Validation
- Error Handling
- Logging and Audit
- Deployment Checklist
- Security Testing

7.6.1 Threat Model

➡ See also

[Threat Model & Trust Boundaries](#) for the comprehensive trust boundary mapping, data flow analysis, and coordination requirements.

PyCypher processes user-supplied Cypher queries against in-process data. The primary threat vectors are:

1. Query injection — malicious Cypher or SQL embedded in user input.
2. Resource exhaustion — unbounded cross-joins, runaway queries, or memory bombs that crash the host process.
3. Path traversal — data-source URIs or file paths that escape the intended directory.
4. Data leakage — returning internal columns (`__ID__`, `__SOURCE__`, `__TARGET__`) or properties from unintended entity types.

7.6.2 Input Validation

Cypher query parameters

Always use parameterised queries instead of string interpolation:

```
# GOOD — parameters are never interpolated into the query string
star.execute_query(
    "MATCH (p:Person) WHERE p.age > $min_age RETURN p.name",
    parameters={"min_age": 18},
)

# BAD — opens the door to injection
star.execute_query(
    f"MATCH (p:Person) WHERE p.age > {user_input} RETURN p.name"
)
```

SQL data-source queries

The ingestion layer uses DuckDB for reading external files. All SQL passed to DuckDB is validated by `validate_sql_query()`, which rejects:

- Multiple statements (semicolon stacking).
- DDL keywords (DROP, ALTER, CREATE, TRUNCATE).
- DML mutation keywords (INSERT, UPDATE, DELETE).
- Comment markers (`--`, `/* */`) that could hide payloads.

Use `sanitize_sql_identifier()` for any user-supplied table or column names, and `parameterize_duckdb_query()` for safe value substitution.

File path sanitisation

`sanitize_file_path()` prevents path traversal by rejecting `..` components and blocking access to sensitive system directories (`/etc/`, `/root/`, `/proc/`).

URI scheme validation

`validate_uri_scheme()` restricts data source URIs to an allow-list of safe schemes: file, http, https, s3, gs, postgresql, mysql, sqlite, duckdb.

7.6.3 Resource Limits

Cross-join row limit

Unbounded MATCH (a), (b), (c) patterns can produce Cartesian explosions. PyCypher enforces a configurable row ceiling:

```
# Default: 10,000,000 rows
# Override via environment variable:
import os
os.environ["PYCYPHER_MAX_CROSS_JOIN_ROWS"] = "1000000"
```

Exceeding the limit raises `MemoryError` with a descriptive message.

Query timeout

Arm a per-query wall-clock budget to prevent runaway execution:

```
result = star.execute_query(
    "MATCH (a)-[*1..10]->(b) RETURN a, b",
    timeout_seconds=5.0,
)
```

Exceeding the timeout raises `QueryTimeoutError`.

Memory budget

The query planner estimates memory usage before execution begins. Set an explicit budget to reject queries that would exceed it:

```
result = star.execute_query(
    "MATCH (a)-[:KNOWS]->(b) RETURN a, b",
    memory_budget_bytes=512 * 1024 * 1024, # 512 MB
)
```

Exceeding the budget raises `QueryMemoryBudgetError`.

7.6.4 Internal Column Protection

PyCypher's data model uses internal columns (`__ID__`, `__SOURCE__`, `__TARGET__`) to track entity identity and relationship endpoints. These are automatically excluded from user-visible results:

- `properties(n)` omits internal columns.
- `keys(n)` omits internal columns.
- `RETURN *` omits internal columns.
- `MapProjection` with `.*` omits internal columns.

If you build custom functions that access raw `DataFrames`, filter out columns whose names start and end with `__`.

7.6.5 Semantic Validation

`SemanticValidator` performs static analysis on parsed queries before execution:

- Detects undefined variables.
- Validates aggregation/non-aggregation mixing.
- Checks function arity and argument types.
- Reports unused variables.

Always validate user-supplied queries before execution in security-sensitive contexts:

```
from pycypher import validate_query

issues = validate_query("MATCH (n) RETURN m")
for issue in issues:
    if issue.severity == "error":
        raise ValueError(f"Query validation failed: {issue.message}")
```

7.6.6 Error Handling

PyCypher raises typed exceptions rather than generic `Exception`:

- `GraphTypeNotFoundError` — unknown entity or relationship type.
- `UndefinedVariableError` — reference to unbound variable.
- `CypherSyntaxError` — malformed Cypher query.
- `QueryTimeoutError` — wall-clock budget exceeded.
- `QueryMemoryBudgetError` — estimated memory exceeds budget.
- `SecurityError` — SQL injection or path traversal detected.

Catch specific exceptions rather than broad except `Exception` to avoid masking security-relevant errors.

7.6.7 Logging and Audit

PyCypher logs query execution through the `shared.logger` module:

- Query start/end: query text, elapsed time, row count, RSS delta.
- Query plan: node count, memory estimate, join presence.
- Per-clause timing: individual clause execution durations.
- Errors: full traceback with query context.

For audit compliance, configure the logger to write to a persistent destination (file, syslog, or structured logging service). Query correlation IDs (`_query_id`) enable tracing individual queries across log entries.

7.6.8 Deployment Checklist

Priority	Action	Status
P0	Set PYCYPHER_MAX_CROSS_JOIN_ROWS to a value appropriate for your hardware.	Required
P0	Always use parameterised queries for user-supplied values.	Required
P0	Set timeout_seconds on all user-facing query endpoints.	Required
P1	Run validate_query() on user-supplied Cypher before execution.	Recommended
P1	Set memory_budget_bytes based on available host memory.	Recommended
P1	Configure persistent audit logging for query execution events.	Recommended
P2	Pin dependency versions with upper bounds (already done in pyproject.toml).	Done
P2	Run uv sync --frozen in CI to prevent supply-chain drift.	Done

7.6.9 Security Testing

PyCypher includes comprehensive security test suites:

- tests/test_security_sql_injection_tdd.py — SQL injection prevention.
- tests/test_security_sql_injection_proof.py — SQL injection proof tests.
- tests/test_backend_engine_security.py — backend engine security.
- tests/test_data_source_security.py — data source URI validation.
- tests/test_duckdb_reader_security.py — DuckDB reader security.
- tests/test_helpers_serialization_security.py — serialisation safety.
- tests/test_neo4j_sink_security.py — Neo4j sink security.
- tests/test_dask_distributed_security.py — distributed security.
- tests/test_distributed_security_contracts.py — distributed contracts.

Run the full security suite:

```
uv run pytest tests/ -m security -q
```

Or run all security-named test files:

```
uv run pytest tests/test_security*.py tests/test_*security*.py -q
```

7.7 Threat Model & Trust Boundaries

This document defines the formal trust boundaries, data flow analysis, and security-sensitive operations in the PyCypher-nmetl system. It serves as the baseline for systematic security review per Amendment Cycle 2.

On this page

- [Overview](#)

- Trust Boundaries
 - TB-1: User Input Boundary
 - TB-2: Configuration Boundary
 - TB-3: Data I/O Boundary
 - TB-4: SQL Execution Boundary
 - TB-5: Output Boundary
- Security-Sensitive Operations
- Audit & Logging Architecture
- Data Flow Summary
- Coordination Requirements
- Security Testing
- Document History

7.7.1 Overview

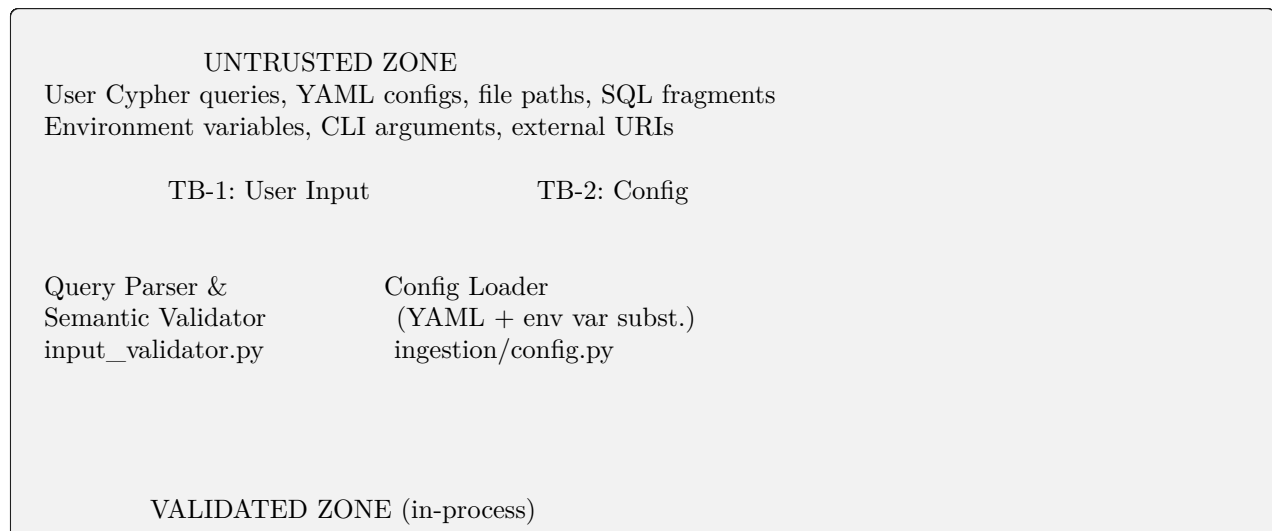
PyCypher-nmetl is an ETL pipeline system that:

1. Parses user-supplied Cypher queries against in-process data.
2. Ingests data from external sources (files, databases, cloud storage).
3. Transforms data through a query execution engine.
4. Outputs results to files or graph databases (Neo4j).

The system runs as a single-process application with no built-in authentication or multi-tenancy. Security controls focus on input validation, resource limits, and preventing unintended data access.

7.7.2 Trust Boundaries

The system has five trust boundaries where data crosses from a less-trusted domain into a more-trusted one.



(continues on next page)

(continued from previous page)

Parsed AST, validated config, sanitized identifiers

TB-3: Data I/O TB-4: SQL Exec TB-5: Output

File System DuckDB Engine Neo4j / File Output
Cloud (S3/GS) (in-process) (external)

TB-1: User Input Boundary

Crossing point: User-supplied Cypher queries enter the system.

Entry points:

- `star.execute_query()` — primary query API
- `star.add_cypher_query()` — pipeline query registration
- CLI `pycypher run` command

Protections:

- Cypher parser rejects malformed syntax (`pycypher/parser.py`)
- Semantic validator detects undefined variables, type mismatches (`pycypher/semantic_validator.py`)
- Input validator enforces query ID uniqueness and non-empty content (`pycypher/input_validator.py:44-141`)
- Resource limits: cross-join row ceiling, query timeout, memory budget

Residual risks:

- Valid but expensive queries (e.g., deep recursive traversals) may still consume significant resources within configured limits.

TB-2: Configuration Boundary

Crossing point: YAML pipeline configs and environment variables.

Entry points:

- `load_pipeline_config()` — YAML config loading (`ingestion/config.py:829-854`)
- Environment variable substitution `${VAR}` in config values (`ingestion/config.py:743-826`)

Protections:

- `yaml.safe_load()` prevents deserialization attacks (`ingestion/config.py:852`)
- Blocked environment variables: `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, `AZURE_CLIENT_SECRET`, `GOOGLE_APPLICATION_CREDENTIALS`, `GCP_SERVICE_ACCOUNT_KEY`, `API_SECRET`, `SECRET_KEY`, `PRIVATE_KEY`, `GITHUB_TOKEN`, `GITLAB_TOKEN`, `NPM_TOKEN`, `PYPI_TOKEN`, `SSH_PRIVATE_KEY`, `SSL_KEY`, `TLS_KEY` (`ingestion/config.py:745-771`)
- Blocked prefixes: `PRIVATE_`, `CREDENTIAL_`
- Blocked module imports in config context: `os`, `subprocess`, `pickle`, `ctypes`, `socket`, `shutil`, etc. (`ingestion/config.py:379-400`)
- Unresolved or blocked vars left as `${VAR}` placeholders (fail-safe)

Residual risks:

- Non-blocked env vars containing secrets (custom naming conventions) will be substituted. Operators should audit env var names used in configs.

TB-3: Data I/O Boundary

Crossing point: Reading external data from files, databases, cloud.

Entry points:

- `FileDataSource.read()` — file-based data ingestion (`ingestion/data_sources.py:432`)
- URI scheme handler: `file://`, `s3://`, `gs://`, `https://`, `postgresql://`, `mysql://`, `sqlite://`, `duckdb://`, bare paths

Protections:

- Path traversal prevention: `sanitize_file_path()` rejects `..` components, blocks sensitive directories (`/etc/`, `/root/`, `/proc/`, `/sys/`, `/dev/`, `/var/run/`, `/boot/`, `/sbin/`), resolves symlinks to prevent TOCTOU attacks (`ingestion/security.py:120-189`)
- URI scheme allowlist: `validate_uri_scheme()` restricts to known-safe schemes (`ingestion/security.py:560-609`)
- SSRF protection: `_check_ssrf_hostname()` blocks RFC 1918 private ranges, loopback, link-local, reserved addresses, and internal hostnames (`ingestion/security.py:612-697`). DNS resolution failures are rejected (fail-closed).
- Credential detection: `mask_uri_credentials()` masks passwords in URIs before logging (`ingestion/security.py:770`). CLI `security_check` command warns on embedded credentials (`cli/security.py:120-143`).

Residual risks:

- Cloud storage access relies on ambient credentials (IAM roles, env vars). Misconfigured IAM policies could expose unintended data.
- TOCTOU window between `resolve()` and actual open is minimal but non-zero on adversarial filesystems.

TB-4: SQL Execution Boundary

Crossing point: SQL fragments passed to DuckDB for data reading.

Entry points:

- `validate_sql_query()` — SQL validation gate (`ingestion/security.py:402-481`)
- `sanitize_sql_identifier()` — table/column name validation (`ingestion/security.py:484`)
- `validate_identifier()` — DuckDB backend identifier validation (`backends/_helpers.py:10-39`)

Protections:

- Statement stacking prevention: rejects multiple semicolons (`ingestion/security.py:441`)
- DDL/DML blocking: rejects `DROP`, `ALTER`, `CREATE`, `TRUNCATE`, `INSERT`, `UPDATE`, `DELETE`
- Dangerous function blocking: `read_csv`, `read_parquet`, `read_json`, `glob`, and DuckDB system functions blocked (`ingestion/security.py:317-341`)
- Dangerous prefix blocking: `duckdb_`, `pg_`, `pragma_`, `information_schema` (`ingestion/security.py:344-349`)
- Comment stripping: removes `--` and `/* */` before validation (`ingestion/security.py:192`)

- NUL byte rejection: prevents C-level string truncation attacks (ingestion/security.py:503, 715)
- Identifier pattern: `^[A-Za-z_][A-Za-z0-9_]*$` enforced

Residual risks:

- DuckDB runs in-process with full host permissions. A bypass of SQL validation could access arbitrary host resources.
- New DuckDB versions may introduce functions not yet on the blocklist.

TB-5: Output Boundary

Crossing point: Query results written to files or external databases.

Entry points:

- `write_dataframe_to_uri()` — file output (CSV, Parquet, JSON) (ingestion/output_writer.py:37-119)
- `Neo4jSink` — graph database output (sinks/neo4j.py:1-349)

Protections:

- File output: path validated via `sanitize_file_path()` (output_writer.py:86)
- Neo4j Cypher injection prevention: `_validate_cypher_identifier()` rejects backticks, NUL bytes, curly braces, square brackets, backslashes (sinks/neo4j.py:241-298). Unicode normalization check for look-alike character attacks (neo4j.py:277). All identifiers are backtick-quoted. Parameterized queries used for data values.
- Internal column filtering: `__ID__`, `__SOURCE__`, `__TARGET__` automatically excluded from results via `properties()`, `keys()`, `RETURN *`.

Residual risks:

- Neo4j connection credentials stored in config or env vars. See TB-2 for credential handling.
- Output file permissions inherit from process umask.

7.7.3 Security-Sensitive Operations

The following operations require security review when modified:

Operation	Module	Review Trigger
SQL query validation	ingestion/security.py	Any change to blocklists or validation logic
File path sanitization	ingestion/security.py	Any change to path validation or sensitive dirs
SSRF hostname checking	ingestion/security.py	Any change to blocked IP ranges or hostnames
URI scheme allowlist	ingestion/security.py	Adding new URI schemes
Env var substitution	ingestion/config.py	Changes to blocked var list or substitution logic
Module import blocking	ingestion/config.py	Changes to blocked module list
DuckDB identifier validation	backends/_helpers.py	Changes to identifier pattern
Neo4j identifier validation	sinks/neo4j.py	Changes to Cypher identifier validation
Audit logging	audit.py	Changes to what is/isn't logged
Health server endpoints	health_server.py	Adding endpoints or changing response content
Credential masking	cli/security.py	Changes to URI credential detection
Output path handling	output_writer.py	Changes to file write paths

7.7.4 Audit & Logging Architecture

Security audit log (pycypher.audit):

- Opt-in via PYCYPHER_AUDIT_LOG environment variable
- Structured JSON output for SIEM integration
- Never logs parameter values — only parameter names
- Query text truncated to 2048 characters
- Separate logger from application logs

Security event log (pycypher.security):

- Opt-in via PYCYPHER_SECURITY_LOG environment variable
- Logs: event type, timestamp, truncated input (256 chars), source function, detail
- Intended for incident response correlation

7.7.5 Data Flow Summary

User Query	Parser	Semantic Validator	Query Planner
YAML Config	safe_load	env subst	Pipeline Config
File/DB URI	scheme check	SSRF check	path sanitize
SQL fragment	comment strip	blocklist	DuckDB exec
Results	internal col filter	output path validate	Write

(continues on next page)

(continued from previous page)

```
Neo4j    identifier validate
        parameterized query
```

7.7.6 Coordination Requirements

Per Amendment Cycle 2, changes to the following require security specialist coordination before merge:

1. Data entry/exit points — any new data source type, output format, or URI scheme.
2. File I/O operations — changes to file reading/writing paths or permission handling.
3. Config parsing — changes to YAML loading, env var substitution, or module import blocking.
4. Credential handling — changes to credential detection, masking, or storage.
5. Input validation — changes to SQL validation, path sanitization, SSRF checking, or identifier validation.
6. Network operations — changes to health server, SSRF protections, or any new network endpoints.

7.7.7 Security Testing

Existing security test coverage:

- tests/test_security_sql_injection_tdd.py — SQL injection TDD
- tests/test_security_sql_injection_proof.py — SQL injection proofs
- tests/test_backend_engine_security.py — backend engine security
- tests/test_data_source_security.py — data source URI validation
- tests/test_duckdb_reader_security.py — DuckDB reader security
- tests/test_helpers_serialization_security.py — serialization safety
- tests/test_neo4j_sink_security.py — Neo4j sink security
- tests/test_dask_distributed_security.py — distributed security
- tests/test_distributed_security_contracts.py — distributed contracts

Run all security tests:

```
uv run pytest tests/ -k security -q
```

7.7.8 Document History

- 2026-04-06 — Initial threat model and trust boundary documentation established per Amendment Cycle 2 requirements.

7.8 New Contributors

Start with the [CONTRIBUTING.md](#) file at the repository root — it covers quick start setup, what to work on, debugging tips, and the submission process. The pages below go deeper into each topic.

7.9 Overview

This guide is for developers who want to contribute to PyCypher:

- [Architecture](#) — Monorepo layout, package dependencies, the Lark-to-Pydantic parsing pipeline, and the BindingFrame execution model.
- [Contributing](#) — Development environment setup (uv, Python 3.14t), code style (ruff, ty, Google docstrings), branch naming, and PR process.
- [Testing](#) — Running tests with pytest, writing new tests, parallel execution, and coverage reporting.
- [Release Process](#) — Semantic versioning, pre-release checklist, building and publishing packages, and CI/CD integration.
- [Security Hardening & Compliance](#) — Threat model, input validation, resource limits, audit logging, and deployment checklist.

Architecture Decision Records

This directory documents significant architectural decisions made in PyCypher. Each ADR captures the context, decision, and consequences of a design choice so that future contributors understand why things are the way they are.

Format: Each ADR follows a consistent structure — Status, Context, Decision, Consequences. ADRs are numbered sequentially and never deleted; superseded decisions are marked as such.

8.1 ADR-001: Use Lark with Earley Algorithm for Cypher Parsing

Status

Accepted

Date

2024-01

Affects

`packages/pycypher/src/pycypher/grammar_parser.py`, `packages/pycypher/src/pycypher/grammar_transformers.py`

8.1.1 Context

PyCypher needs a parser that can handle the full openCypher grammar specification. The grammar is complex, with ambiguous constructs (e.g. label predicates in WHERE clauses, list comprehensions vs pattern comprehensions) that require a parser capable of handling ambiguity.

Key requirements:

- Full openCypher BNF coverage
- Good error messages for malformed queries
- Reasonable cold-start performance for interactive use
- Maintainable grammar definition (not hand-written parser tables)

8.1.2 Decision

Use `Lark` with the Earley parsing algorithm. The grammar is defined declaratively in Lark's EBNF notation, derived from the official openCypher BNF specification.

To eliminate cold-start latency (~96ms for grammar compilation), the parser is compiled eagerly in a daemon thread at module import time and cached as a process-level singleton via `get_default_parser()`.

The Lark parse tree is transformed into Pydantic AST models through a CompositeTransformer architecture with specialised transformers for each concern (literals, expressions, patterns, statements), following the Single Responsibility Principle.

8.1.3 Consequences

Benefits:

- Earley handles all context-free grammars including ambiguous ones, so the grammar can match the openCypher spec without workarounds
- Lark's declarative grammar is readable and maintainable
- Eager background compilation means the parser is ready before the first query in interactive sessions
- The transformer architecture keeps grammar-to-AST conversion modular

Trade-offs:

- Earley is slower than LALR for unambiguous grammars (microseconds vs nanoseconds per parse); acceptable because query execution dominates
- Grammar compilation cost (~96ms) is paid once at import time
- Lark is a runtime dependency

Constraints:

- Application code must use `get_default_parser()` (the singleton) rather than constructing `GrammarParser()` directly
- Grammar changes require updating both the Lark grammar and the corresponding transformer logic

8.2 ADR-002: BindingFrame as Intermediate Representation

Status

Accepted

Date

2024-06

Affects

packages/pycypher/src/pycypher/binding_frame.py, packages/pycypher/src/pycypher/star.py

8.2.1 Context

The original query execution pipeline used Relation subclasses (`EntityTable`, `Join`, `FilterRows`, `Projection`) that operated on pandas DataFrames with prefixed column names like `Person__name` and opaque `HASH_ID` columns. This caused three recurring problems:

1. Opaque column names — `Projection` produced `HASH_ID` columns that required a `variable_map` lookup table to trace back to Cypher variables.

2. Prefixed entity columns — Column names like `Person__name` leaked into operator logic, requiring hacks like `_ensure_full_entity_data`.
3. Metadata threading — `variable_map` and `variable_type_map` had to be threaded through every `Relation` subclass, creating tight coupling.

8.2.2 Decision

Introduce `BindingFrame` as the intermediate representation for query execution. A `BindingFrame` is a pandas `DataFrame` where:

- Columns are named after Cypher variables (e.g. `p`, `q`, `r`)
- Each row is one assignment of entity/relationship IDs to those variables
- Attributes are never stored in the frame; they are fetched on-demand from `Context` via `get_property()`

The column is the variable — no lookup table required.

8.2.3 Consequences

Benefits:

- Eliminates the `variable_map/variable_type_map` metadata entirely
- Column names are self-describing — debugging is straightforward
- Clean separation between structure (which IDs match) and content (what properties those IDs have)
- Enables lazy attribute resolution — only fetch what `RETURN` actually needs
- Simplifies operator implementation (join, filter, project all work on variable-named columns directly)

Trade-offs:

- Property access requires a lookup into `Context` rather than reading from the `DataFrame` directly — adds one level of indirection
- Migrating from the `Relation`-based pipeline required touching most of the execution path

Performance notes:

- Module-level debug checks (set once at import) avoid per-call overhead
- Pre-built numpy ufuncs for ndarray normalisation
- `_null_series()` uses `np.empty + fill` instead of list allocation

8.3 ADR-003: ID-Only Column Preservation Strategy

Status

Accepted

Date

2024-06

Affects

`packages/pycypher/src/pycypher/constants.py`, `packages/pycypher/src/pycypher/relational_models.py`, `packages/pycypher/src/pycypher/binding_frame.py`

8.3.1 Context

During query execution, the engine must track which entities and relationships match a pattern (MATCH), filter them (WHERE), and then project attributes (RETURN). The naive approach — carrying all attributes through every intermediate step — wastes memory and complicates joins when different entity types have overlapping column names.

8.3.2 Decision

Intermediate execution state (BindingFrames) stores only identity columns:

- `__ID__` — unique entity identifier
- `__SOURCE__` — relationship source node ID
- `__TARGET__` — relationship target node ID

Attributes (name, age, etc.) are fetched on-demand from the Context layer only when needed — typically during RETURN projection or WHERE property access.

This is enforced by convention: BindingFrame columns correspond to Cypher variables and contain only ID values.

8.3.3 Consequences

Benefits:

- Joins operate on small ID-only frames, reducing memory and improving speed
- No column name collisions between entity types during joins
- Clear separation of concerns: BindingFrame tracks structure (which IDs match), Context provides content (what properties those IDs have)
- Lazy attribute resolution means unused properties are never fetched

Trade-offs:

- Every property access in WHERE or RETURN requires a lookup into Context, adding one indirection step per property reference
- Property caching is needed for repeated access to the same property within a single query (handled by the property lookup cache)

Constants:

```
ID_COLUMN = "__ID__"
RELATIONSHIP_SOURCE_COLUMN = "__SOURCE__"
RELATIONSHIP_TARGET_COLUMN = "__TARGET__"
```

8.4 ADR-004: Shadow-Write Mutation Pattern for Atomicity

Status

Accepted

Date

2024-08

Affects

packages/pycypher/src/pycypher/mutation_engine.py, packages/pycypher/src/pycypher/relational_models.py

8.4.1 Context

Cypher supports write operations (CREATE, SET, DELETE, MERGE, REMOVE) that must be atomic — either all mutations in a query succeed, or none take effect. A query like MATCH (a) SET a.x = 1 SET a.y = a.x + 1 must see consistent state within the same query, and a failing query must leave the context unchanged.

Without atomicity, a crash mid-query could leave entity tables in a partially-mutated state with no way to recover.

8.4.2 Decision

Implement a two-phase shadow-write pattern in Context:

1. `begin_query()` — opens a transaction by initialising an empty shadow layer (`context._shadow = {}`)
2. Mutation methods (SET, CREATE, DELETE, etc.) write to the shadow layer rather than modifying original DataFrames in place
3. `commit_query()` — promotes shadow DataFrames to the canonical `source_obj` on each affected EntityTable/RelationshipTable
4. `rollback_query()` — discards the shadow layer, leaving `source_obj` unchanged

All write-path operations go through MutationEngine, which delegates to Context's shadow layer.

8.4.3 Consequences

Benefits:

- Full rollback semantics — failed queries never corrupt state
- Consistent read-your-writes within a single query (mutations read from shadow when available)
- Simple implementation — shadow is a dict of entity-type to DataFrame
- No external transaction coordinator needed

Trade-offs:

- Memory overhead: shadow layer holds copies of modified DataFrames during query execution
- Not suitable for concurrent multi-query transactions (single-writer model)
- Shadow merge on commit is $O(n)$ in the number of modified entity types

Lifecycle:

```
begin_query()
  SET a.x = 1    → writes to _shadow["Person"]
  SET a.y = a.x + 1 → reads from _shadow, writes to _shadow
  success?
    yes → commit_query() → _shadow → source_obj
    no  → rollback_query() → discard _shadow
```

8.5 ADR-005: Backend Engine Protocol for Pluggable DataFrames

Status

Accepted

Date

2024-10

Affects

packages/pycypher/src/pycypher/backend_engine.py, packages/pycypher/src/pycypher/backends/

8.5.1 Context

PyCypher was originally hard-coded to pandas for all DataFrame operations. As datasets grow, users need alternative backends (DuckDB for analytical queries, Polars for Arrow-native processing, Dask for distributed computation) without rewriting the query engine.

Abstracting all ~918 pandas API calls would be impractical and fragile.

8.5.2 Decision

Define a BackendEngine protocol that captures the ~15 primitive operations that BindingFrame, Star, and MutationEngine actually depend on. Operations are grouped into five categories:

1. Scan — load entity IDs from source data
2. Transform — filter, join, rename, concat, distinct, assign_column, drop_columns
3. Aggregate — grouped or full-table aggregation
4. Order — sort, limit, skip
5. Materialise — to_pandas, row_count, is_empty, memory_estimate_bytes

Concrete implementations:

- PandasBackend — default, zero-cost wrapper around existing pandas ops
- DuckDBBackend — SQL-based analytical backend with lazy evaluation
- PolarsBackend — Arrow-native backend

Backend selection is controlled via PYCYPHER_BACKEND environment variable (auto, pandas, duckdb, polars) or programmatically through Context(backend=...).

8.5.3 Consequences

Benefits:

- Users can switch backends without changing query code
- Protocol surface is small (~15 methods) and stable
- Each backend can optimise for its strengths (DuckDB for aggregations, Polars for lazy evaluation)
- auto mode selects the best available backend

Trade-offs:

- Not all pandas features are available through the protocol; some advanced operations may fall back to pandas
- Backend implementations must be kept in sync when new primitive operations are added
- Testing must cover all backends to prevent drift

Integration points:

Operations that route through the backend:

- BindingFrame.join, left_join, cross_join
- BindingFrame.filter, rename

- PatternMatcher — concat, distinct
- MutationEngine — CREATE via concat, DELETE via filter

8.6 ADR-006: Star Class Decomposition into Specialised Modules

Status

Accepted

Date

2024-09

Affects

packages/pycypher/src/pycypher/star.py, packages/pycypher/src/pycypher/
pattern_matcher.py, packages/pycypher/src/pycypher/mutation_engine.py, packages/
pycypher/src/pycypher/path_expander.py

8.6.1 Context

The Star class was originally a monolithic god object responsible for parsing, pattern matching, path expansion, mutation handling, result caching, and query orchestration. This made the class difficult to test in isolation, hard to navigate (thousands of lines), and tightly coupled across concerns.

8.6.2 Decision

Extract specialised modules from Star while keeping it as the single entry point for query execution:

- PatternMatcher — translates MATCH patterns into BindingFrames via node scanning, relationship joins, and multi-path merging
- PathExpander — handles variable-length BFS expansion for [*min..max] relationship patterns
- MutationEngine — all write-path operations (CREATE, SET, DELETE, MERGE, FOREACH, REMOVE) with shadow-write atomicity

Star.execute_query() remains the top-level orchestrator. Each Cypher clause type dispatches to the appropriate module. Star retains:

- Query parsing and clause sequencing
- Result caching (LRU with TTL and thread-safe locking)
- Context management
- RETURN/WITH/ORDER BY/LIMIT projection

8.6.3 Consequences

Benefits:

- Each module can be tested independently with mock Contexts
- Clear ownership: mutation bugs go to MutationEngine, pattern bugs go to PatternMatcher
- Smaller files are easier to navigate and review
- Reduced risk of unintended coupling between read and write paths

Trade-offs:

- Star still has substantial coordination logic and remains the largest single file (~2,581 lines)

- Modules need access to Context and sometimes to each other (e.g. MutationEngine may need PatternMatcher for MERGE)

2026-03 Assessment: No Further Decomposition Recommended

A review (Task #6) confirmed that Star now delegates to 7 specialised engines: MutationEngine, PatternMatcher, FrameJoiner, PathExpander, ProjectionPlanner, AggregationPlanner, and ExpressionRenderer. The remaining ~2,581 lines are orchestration logic (execute_query entry point, clause dispatch, query planning coordination, timeout/cache management). This follows the Mediator pattern appropriately — further splitting would scatter tightly-coupled execution flow across files without improving testability, since each delegate is already independently testable.

Current delegates:

- pattern_matcher.py — MATCH execution
- mutation_engine.py — write path (CREATE, SET, DELETE, MERGE, FOREACH, REMOVE)
- path_expander.py — BFS expansion for variable-length paths
- frame_joiner.py — BindingFrame join operations
- projection_planner.py — RETURN/WITH projection
- aggregation_planner.py — aggregation dispatch
- expression_renderer.py — expression evaluation for display

8.7 ADR-007: Extract Cardinality Estimator from Query Planner

Status

Accepted

Date

2026-03

Affects

packages/pycypher/src/pycypher/cardinality_estimator.py, packages/pycypher/src/
pycypher/query_planner.py

8.7.1 Context

The query_planner.py module (1,360 lines) contained both query planning logic and cardinality estimation primitives (ColumnStatistics, TableStatistics, CardinalityFeedbackStore). While there was no actual bidirectional dependency with relational_models.py (as initially suspected), the cardinality estimation code was tightly embedded in the planner, making it difficult to reuse in other contexts (e.g. the query plan analyser) or test independently.

8.7.2 Decision

Extract cardinality estimation into a dedicated cardinality_estimator.py module (347 lines) containing:

- ColumnStatistics — per-column NDV, null fraction, histograms
- TableStatistics — lazily computed column statistics for a DataFrame
- CardinalityFeedbackStore — accumulates actual-vs-estimated ratios for self-correcting estimates over time

All three classes are re-exported from query_planner.py for backward compatibility. query_planner.py reduced from 1,360 to 1,061 lines.

8.7.3 Consequences

Benefits:

- Cardinality estimation is independently importable and testable
- Other components (query plan analyser, query learning) can import statistics primitives without pulling in the full planner
- Clearer separation of concerns: planner decides what to do, estimator provides data for those decisions
- Reduced cognitive load when working on either module

Trade-offs:

- One additional module in the package
- Re-exports in `query_planner.py` add a small indirection for existing callers (no code changes required)

The project ships two packages:

- PyCypher — openCypher query parser, AST models, and BindingFrame execution engine
- Shared — Common utilities, logging, and telemetry

PyCypher also includes nmetl, a command-line ETL tool that executes Cypher queries defined in a YAML pipeline config.

10

Key Features

- Complete openCypher grammar support (Earley parser via Lark)
- Strongly-typed AST models with Pydantic
- Vectorised query execution against pandas DataFrames
- 130+ built-in scalar functions
- Mutation support (CREATE, SET, DELETE, MERGE) with atomic rollback
- Query parameter injection for safe value binding

```
import pandas as pd
from pycypher import Star
from pycypher.ingestion import ContextBuilder

# Build a context from plain DataFrames
ctx = ContextBuilder.from_dict({
    "Person": pd.DataFrame({
        "__ID__": ["p1", "p2", "p3"],
        "name":   ["Alice", "Bob", "Carol"],
        "age":    [30, 25, 35],
    }),
    "KNOWS": pd.DataFrame({
        "__SOURCE__": ["p1", "p2"],
        "__TARGET__": ["p2", "p3"],
        "since":      [2020, 2021],
    }),
})

star = Star(context=ctx)

# Execute Cypher — returns a pandas DataFrame
result = star.execute_query(
    "MATCH (p:Person) WHERE p.age > 25 RETURN p.name AS name ORDER BY p.age"
)
print(result)
```

See [Getting Started](#) for the full walkthrough including relationships, aggregation, mutation, and the ContextBuilder API.

The `examples/` directory contains runnable scripts that demonstrate PyCypher features end-to-end:

- “`ast_conversion_example.py`” — Parse a Cypher query, convert to typed AST, traverse and pretty-print the tree.
- “`advanced_grammar_examples.py`” — Exercise the full breadth of the openCypher grammar (OPTIONAL MATCH, UNWIND, CASE, list comprehensions, variable-length paths, etc.).
- “`functions_in_where.py`” — Use scalar functions (`toUpper`, `toLower`, `trim`, `abs`, `toInteger`) inside WHERE predicates against a sample dataset.
- “`scalar_functions_in_with.py`” — Demonstrate scalar function usage in WITH clause projections.

Run any example with:

```
uv run python examples/ast_conversion_example.py
```

13

Indices and tables

- `genindex`
- `modindex`
- `search`

p

pycypher.aggregation_evaluator, 196
pycypher.aggregation_planner, 198
pycypher.arithmetic_evaluator, 188
pycypher.ast_converter, 154
pycypher.ast_models, 82
pycypher.ast_rewriter, 154
pycypher.audit, 332
pycypher.backend_engine, 211
pycypher.binding_evaluator, 185
pycypher.binding_frame, 178
pycypher.boolean_evaluator, 190
pycypher.cardinality_estimator, 235
pycypher.clause_executor, 170
pycypher.cluster, 325
pycypher.collection_evaluator, 199
pycypher.comparison_evaluator, 192
pycypher.config, 209
pycypher.constants, 165
pycypher.cyper_types, 336
pycypher.evaluator_protocol, 184
pycypher.exceptions, 345
pycypher.exists_evaluator, 195
pycypher.expression_renderer, 177
pycypher.frame_joiner, 255
pycypher.grammar_parser, 115
pycypher.grammar_rule_mixins, 120
pycypher.grammar_transformers, 118
pycypher.graph_index, 260
pycypher.ingestion.arrow_utils, 317
pycypher.ingestion.config, 304
pycypher.ingestion.context_builder, 295
pycypher.ingestion.data_sources, 297
pycypher.ingestion.duckdb_reader, 315
pycypher.ingestion.output_writer, 318
pycypher.ingestion.security, 319
pycypher.input_validator, 254
pycypher.lazy_eval, 278
pycypher.leapfrog_triejoin, 258
pycypher.multi_query_analyzer, 289
pycypher.multi_query_executor, 291
pycypher.mutation_engine, 286
pycypher.nmetl_cli, 332
pycypher.path_expander, 175
pycypher.pattern_matcher, 172
pycypher.pipeline, 271
pycypher.projection_planner, 267
pycypher.query_analyzer, 234
pycypher.query_complexity, 241
pycypher.query_formatter, 249
pycypher.query_learning, 337
pycypher.query_optimizer, 242
pycypher.query_planner, 228
pycypher.query_profiler, 251
pycypher.query_validator, 248
pycypher.rate_limiter, 334
pycypher.relational_models, 156
pycypher.result_cache, 282
pycypher.scalar_function_evaluator, 207
pycypher.scalar_functions, 202
pycypher.scan_operators, 238
pycypher.semantic_validator, 293
pycypher.sinks.neo4j, 321
pycypher.star, 166
pycypher.string_predicate_evaluator, 194
pycypher.timeout_handler, 284
pycypher.variable_manager, 269

S

shared, 358
shared.compat, 373
shared.deprecation, 377
shared.exporters, 370
shared.helpers, 366

[shared.logger](#), 359
[shared.metrics](#), 360
[shared.otel](#), 368
[shared.telemetry](#), 368

Non-alphabetical

`__bool__()` (pycypher.ast_models.ValidationResult method), 86
`__call__()` (pycypher.evaluator_protocol.ExpressionEvaluatorFactory method), 185
`__call__()` (pycypher.relational_models.RegisteredFunction method), 162
`__del__()` (pycypher.backend_engine.DuckDBBackend method), 219
`__enter__()` (pycypher.backend_engine.DuckDBBackend method), 219
`__enter__()` (pycypher.sinks.neo4j.Neo4jSink method), 323
`__enter__()` (pycypher.timeout_handler.TimeoutHandler method), 285
`__exit__()` (pycypher.backend_engine.DuckDBBackend method), 219
`__exit__()` (pycypher.sinks.neo4j.Neo4jSink method), 323
`__exit__()` (pycypher.timeout_handler.TimeoutHandler method), 285
`__getattr__()` (pycypher.grammar_transformers.CompositeTransformer method), 119
`__getitem__()` (pycypher.relational_models.EntityMapping method), 159
`__getitem__()` (pycypher.relational_models.RelationshipMapping method), 162
`__hash__()` (pycypher.ast_models.Variable method), 112
`__init__()` (pycypher.aggregation_evaluator.AggregationExpressionEvaluator method), 197
`__init__()` (pycypher.arithmetic_evaluator.ArithmeticExpressionEvaluator method), 188
`__init__()` (pycypher.ast_models.ValidationIssue method), 85
`__init__()` (pycypher.backend_engine.BackendEngine method), 217
`__init__()` (pycypher.backend_engine.CircuitBreaker method), 217
`__init__()` (pycypher.backend_engine.DuckDBBackend method), 219
`__init__()` (pycypher.backend_engine.InstrumentedBackend method), 221
`__init__()` (pycypher.backend_engine.PolarsBackend method), 225
`__init__()` (pycypher.binding_evaluator.BindingExpressionEvaluator method), 186
`__init__()` (pycypher.binding_frame.BindingFrame method), 184
`__init__()` (pycypher.boolean_evaluator.BooleanExpressionEvaluator method), 190
`__init__()` (pycypher.cardinality_estimator.CardinalityFeedbackStore method), 236
`__init__()` (pycypher.cardinality_estimator.ColumnStatistics method), 238
`__init__()` (pycypher.cardinality_estimator.TableStatistics method), 238
`__init__()` (pycypher.clause_executor.ClauseExecutor method), 170
`__init__()` (pycypher.cluster.ClusterCoordinator method), 332
`__init__()` (pycypher.cluster.ClusterHealth method), 329
`__init__()` (pycypher.cluster.LocalWorker method), 330
`__init__()` (pycypher.cluster.QueryRouter method), 329
`__init__()` (pycypher.cluster.RoundRobinRouter method), 329
`__init__()` (pycypher.cluster.Worker method), 326
`__init__()` (pycypher.cluster.WorkerHealth method), 327
`__init__()` (pycypher.collection_evaluator.CollectionExpressionEvaluator method), 199

<code>__init__()</code> (pycypher.comparison_evaluator.ComparisonEvaluator method), 192	<code>__init__()</code> (pycypher.exceptions.WrongCypherTypeError method), 357
<code>__init__()</code> (pycypher.constants.NullMaskResult method), 166	<code>__init__()</code> (pycypher.exists_evaluator.ExistsEvaluator method), 195
<code>__init__()</code> (pycypher.evaluator_protocol.ExpressionEvaluatorFact method), 185	<code>__init__()</code> (pycypher.expression_renderer.ExpressionRenderer method), 178
<code>__init__()</code> (pycypher.evaluator_protocol.ExpressionEvaluatorProtocol method), 185	<code>__init__()</code> (pycypher.frame_joiner.FrameJoiner method), 256
<code>__init__()</code> (pycypher.exceptions.ASTConversionError method), 346	<code>__init__()</code> (pycypher.grammar_parser.GrammarParser method), 115
<code>__init__()</code> (pycypher.exceptions.CacheLockTimeoutError method), 348	<code>__init__()</code> (pycypher.grammar_transformers.CompositeTransformer method), 119
<code>__init__()</code> (pycypher.exceptions.CyclicDependencyError method), 348	<code>__init__()</code> (pycypher.graph_index.AdjacencyIndex method), 262
<code>__init__()</code> (pycypher.exceptions.CypherSyntaxError method), 347	<code>__init__()</code> (pycypher.graph_index.EntityLabelIndex method), 264
<code>__init__()</code> (pycypher.exceptions.DocsLink method), 345	<code>__init__()</code> (pycypher.graph_index.GraphIndexManager method), 266
<code>__init__()</code> (pycypher.exceptions.FunctionArgumentError method), 348	<code>__init__()</code> (pycypher.graph_index.PropertyValueIndex method), 263
<code>__init__()</code> (pycypher.exceptions.GrammarTransformerSymbolError method), 347	<code>__init__()</code> (pycypher.graph_index.VectorizedPropertyStore method), 265
<code>__init__()</code> (pycypher.exceptions.GraphTypeNotFoundError method), 349	<code>__init__()</code> (pycypher.ingestion.context_builder.ContextBuilder method), 296
<code>__init__()</code> (pycypher.exceptions.IncompatibleOperatorError method), 350	<code>__init__()</code> (pycypher.ingestion.data_sources.ArrowDataSource method), 303
<code>__init__()</code> (pycypher.exceptions.InvalidCastError method), 350	<code>__init__()</code> (pycypher.ingestion.data_sources.CsvFormat method), 299
<code>__init__()</code> (pycypher.exceptions.MissingParameterError method), 351	<code>__init__()</code> (pycypher.ingestion.data_sources.DataFrameDataSource method), 302
<code>__init__()</code> (pycypher.exceptions.PatternComprehensionError method), 351	<code>__init__()</code> (pycypher.ingestion.data_sources.FileDataSource method), 301
<code>__init__()</code> (pycypher.exceptions.QueryComplexityError method), 352	<code>__init__()</code> (pycypher.ingestion.data_sources.JsonFormat method), 300
<code>__init__()</code> (pycypher.exceptions.QueryMemoryBudgetError method), 352	<code>__init__()</code> (pycypher.ingestion.data_sources.ParquetFormat method), 300
<code>__init__()</code> (pycypher.exceptions.QueryTimeoutError method), 353	<code>__init__()</code> (pycypher.ingestion.data_sources.SqlDataSource method), 302
<code>__init__()</code> (pycypher.exceptions.RateLimitError method), 353	<code>__init__()</code> (pycypher.input_validator.InputValidationResult method), 255
<code>__init__()</code> (pycypher.exceptions.SecurityError method), 353	<code>__init__()</code> (pycypher.lazy_eval.ComputationGraph method), 281
<code>__init__()</code> (pycypher.exceptions.TemporalArithmeticError method), 354	<code>__init__()</code> (pycypher.lazy_eval.OpNode method), 279
<code>__init__()</code> (pycypher.exceptions.UnsupportedFunctionError method), 354	<code>__init__()</code> (pycypher.leapfrog_triejoin.LeapfrogIterator method), 259
<code>__init__()</code> (pycypher.exceptions.UnsupportedOperatorError method), 355	<code>__init__()</code> (pycypher.multi_query_analyzer.DependencyGraph method), 290
<code>__init__()</code> (pycypher.exceptions.VariableNotFoundError method), 355	<code>__init__()</code> (pycypher.multi_query_analyzer.QueryNode method), 290
<code>__init__()</code> (pycypher.exceptions.VariableTypeMismatchError method), 356	<code>__init__()</code> (pycypher.multi_query_executor.MultiQueryExecutor method), 292
<code>__init__()</code> (pycypher.exceptions.WorkerExecutionError method), 357	<code>__init__()</code> (pycypher.mutation_engine.MutationEngine method), 286

<code>__init__()</code> (pycypher.path_expander.PathExpander method), 176	<code>__init__()</code> (pycypher.query_profiler.ProfileReport method), 253
<code>__init__()</code> (pycypher.pattern_matcher.PatternMatcher method), 174	<code>__init__()</code> (pycypher.query_profiler.QueryProfiler method), 254
<code>__init__()</code> (pycypher.pattern_matcher.VariableLengthHintsParam method), 173	<code>__init__()</code> (pycypher.query_validator.ValidationResult method), 249
<code>__init__()</code> (pycypher.pipeline.Pipeline method), 272	<code>__init__()</code> (pycypher.rate_limiter.QueryRateLimiter method), 335
<code>__init__()</code> (pycypher.pipeline.PipelineContext method), 275	<code>__init__()</code> (pycypher.relational_models.Context method), 157
<code>__init__()</code> (pycypher.pipeline.PipelineResult method), 276	<code>__init__()</code> (pycypher.result_cache.ReadWriteLock method), 282
<code>__init__()</code> (pycypher.projection_planner.ProjectionPlan method), 268	<code>__init__()</code> (pycypher.result_cache.ResultCache method), 283
<code>__init__()</code> (pycypher.query_analyzer.QueryAnalyzer method), 234	<code>__init__()</code> (pycypher.scalar_function_evaluator.ScalarFunctionEvaluator method), 208
<code>__init__()</code> (pycypher.query_complexity.ComplexityScore method), 242	<code>__init__()</code> (pycypher.scalar_functions.FunctionMetadata method), 205
<code>__init__()</code> (pycypher.query_formatter.LintIssue method), 251	<code>__init__()</code> (pycypher.scalar_functions.ScalarFunctionRegistry method), 205
<code>__init__()</code> (pycypher.query_learning.AdaptiveEvictionPolicy method), 338	<code>__init__()</code> (pycypher.scan_operators.BindingFilter method), 241
<code>__init__()</code> (pycypher.query_learning.AdaptivePlanCache method), 338	<code>__init__()</code> (pycypher.scan_operators.EntityScan method), 239
<code>__init__()</code> (pycypher.query_learning.DistributedLearningSync method), 339	<code>__init__()</code> (pycypher.scan_operators.RelationshipScan method), 240
<code>__init__()</code> (pycypher.query_learning.JoinPerformanceTracker method), 339	<code>__init__()</code> (pycypher.semantic_validator.SemanticValidator method), 294
<code>__init__()</code> (pycypher.query_learning.PlanVersionTracker method), 340	<code>__init__()</code> (pycypher.semantic_validator.ValidationError method), 294
<code>__init__()</code> (pycypher.query_learning.PredicateSelectivityTracker method), 341	<code>__init__()</code> (pycypher.semantic_validator.VariableScope method), 294
<code>__init__()</code> (pycypher.query_learning.QueryFingerprint method), 342	<code>__init__()</code> (pycypher.sinks.neo4j.Neo4jSink method), 322
<code>__init__()</code> (pycypher.query_learning.QueryLearningStore method), 343	<code>__init__()</code> (pycypher.star.ResultCache method), 167
<code>__init__()</code> (pycypher.query_learning.SemanticResultCache method), 344	<code>__init__()</code> (pycypher.star.Star method), 169
<code>__init__()</code> (pycypher.query_optimizer.OptimizationPlan method), 245	<code>__init__()</code> (pycypher.timeout_handler.TimeoutHandler method), 285
<code>__init__()</code> (pycypher.query_optimizer.OptimizationResult method), 244	<code>__init__()</code> (shared.compat.APISurface method), 375
<code>__init__()</code> (pycypher.query_optimizer.OptimizeStage method), 248	<code>__init__()</code> (shared.compat.SurfaceDiff method), 376
<code>__init__()</code> (pycypher.query_optimizer.QueryOptimizer method), 247	<code>__init__()</code> (shared.compat.SymbolInfo method), 374
<code>__init__()</code> (pycypher.query_planner.JoinPlan method), 230	<code>__init__()</code> (shared.exporters.JSONFileExporter method), 372
<code>__init__()</code> (pycypher.query_planner.QueryPlan method), 231	<code>__init__()</code> (shared.exporters.MetricsExporter method), 371
<code>__init__()</code> (pycypher.query_planner.QueryPlanAnalyzer method), 231	<code>__init__()</code> (shared.exporters.PrometheusExporter method), 371
<code>__init__()</code> (pycypher.query_planner.QueryPlanner method), 232	<code>__init__()</code> (shared.exporters.StatsDExporter method), 371
	<code>__init__()</code> (shared.metrics.MetricsSnapshot method), 371

method), 364	__str__() (pycypher.exceptions.WrongCypherTypeError method), 357
__init__() (shared.metrics.QueryMetrics method), 365	__str__() (pycypher.input_validator.InputValidationResult method), 255
__len__() (pycypher.binding_frame.BindingFrame method), 180	__str__() (pycypher.query_profiler.ProfileReport method), 253
__post_init__() (pycypher.scan_operators.RelationshipScan method), 240	__str__() (pycypher.query_validator.ValidationResult method), 249
__repr__() (pycypher.ast_models.ValidationIssue method), 85	__str__() (pycypher.semantic_validator.ValidationError method), 294
__repr__() (pycypher.exceptions.ASTConversionError method), 346	
__repr__() (pycypher.exceptions.CyclicDependencyError method), 348	accumulator (pycypher.ast_models.Reduce attribute), 109, 110
__repr__() (pycypher.exceptions.CypherSyntaxError method), 347	acquire() (pycypher.rate_limiter.QueryRateLimiter method), 335
__repr__() (pycypher.exceptions.FunctionArgumentError method), 349	acquire_read() (pycypher.result_cache.ReadWriteLock method), 283
__repr__() (pycypher.exceptions.GrammarTransformerSyncError method), 347	acquire_write() (pycypher.result_cache.ReadWriteLock method), 283
__repr__() (pycypher.exceptions.GraphTypeNotFoundError method), 349	ACTIVE (pycypher.cluster.WorkerStatus attribute), 326
__repr__() (pycypher.exceptions.IncompatibleOperatorError method), 350	active_queries (pycypher.cluster.WorkerHealth attribute), 327
__repr__() (pycypher.exceptions.InvalidCastError method), 350	active_workers (pycypher.cluster.ClusterHealth attribute), 328
__repr__() (pycypher.exceptions.MissingParameterError method), 351	actual_args (pycypher.exceptions.FunctionArgumentError attribute), 348
__repr__() (pycypher.exceptions.PatternComprehensionError method), 351	actual_type (pycypher.exceptions.VariableTypeMismatchError attribute), 356
__repr__() (pycypher.exceptions.QueryComplexityError method), 352	AdaptiveEvictionPolicy (class in py-cypher.query_learning), 337
__repr__() (pycypher.exceptions.QueryMemoryBudgetError method), 352	AdaptivePlanCache (class in py-cypher.query_learning), 338
__repr__() (pycypher.exceptions.QueryTimeoutError method), 353	add_all_properties_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 120
__repr__() (pycypher.exceptions.TemporalArithmeticError method), 354	add_entity() (pycypher.ingestion.context_builder.ContextBuilder method), 296
__repr__() (pycypher.exceptions.UnsupportedFunctionError method), 355	add_error() (pycypher.ast_models.ValidationResult method), 86
__repr__() (pycypher.exceptions.UnsupportedOperatorError method), 355	add_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 130
__repr__() (pycypher.exceptions.VariableNotFoundError method), 356	add_flag() (pycypher.ast_models.ValidationResult method), 86
__repr__() (pycypher.exceptions.VariableTypeMismatchError method), 356	add_issue() (pycypher.ast_models.ValidationResult method), 86
__repr__() (pycypher.exceptions.WorkerExecutionError method), 357	add_node() (pycypher.lazy_eval.ComputationGraph method), 280
__repr__() (pycypher.exceptions.WrongCypherTypeError method), 357	add_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 131
__repr__() (pycypher.relational_models.Context method), 158	add_relationship() (pycypher.ingestion.context_builder.ContextBuilder method), 296
__repr__() (pycypher.star.Star method), 169	
__str__() (pycypher.exceptions.InvalidCastError method), 350	

[add_rule\(\)](#) (pycypher.query_optimizer.QueryOptimizer method), 247
[add_warning\(\)](#) (pycypher.ast_models.ValidationResult method), 87
[AddAllPropertiesItem](#) (class in pycypher.ast_models), 88
[added](#) (shared.compat.SurfaceDiff attribute), 376
[AdjacencyIndex](#) (class in pycypher.graph_index), 261
[AGGREGATE](#) (pycypher.lazy_eval.OpType attribute), 278
[aggregate\(\)](#) (pycypher.backend_engine.BackendEngine method), 215
[aggregate\(\)](#) (pycypher.backend_engine.DuckDBBackend method), 219
[aggregate\(\)](#) (pycypher.backend_engine.InstrumentedBackend method), 222
[aggregate\(\)](#) (pycypher.backend_engine.PandasBackend method), 223
[aggregate\(\)](#) (pycypher.backend_engine.PolarsBackend method), 225
[aggregate_items\(\)](#) (pycypher.aggregation_planner.AggregationPlanner method), 198
[aggregation](#) (pycypher.query_planner.QueryPlan attribute), 230, 231
[aggregation_evaluator](#) (pycypher.binding_evaluator.BindingExpressionEvaluator property), 186
[AggregationExpressionEvaluator](#) (class in pycypher.aggregation_evaluator), 196
[AggregationPlanner](#) (class in pycypher.aggregation_planner), 198
[AggStrategy](#) (class in pycypher.query_planner), 228
[Algebraizable](#) (class in pycypher.ast_models), 84
[alias](#) (pycypher.ast_models.ReturnItem attribute), 96
[alias](#) (pycypher.ast_models.Unwind attribute), 98
[alias](#) (pycypher.ast_models.YieldItem attribute), 99
[all_flags](#) (pycypher.ast_models.UnionQuery attribute), 98
[all_null](#) (pycypher.constants.NullMaskResult property), 166
[all_properties](#) (pycypher.ast_models.MapElement attribute), 104
[AllShortestPaths](#) (class in pycypher.ast_models), 99
[analyze\(\)](#) (pycypher.multi_query_analyzer.QueryDependencyAnalyzer method), 291
[analyze\(\)](#) (pycypher.multi_query_executor.MultiQueryExecutor method), 292
[analyze\(\)](#) (pycypher.query_optimizer.FilterPushdownRule method), 246
[analyze\(\)](#) (pycypher.query_optimizer.IndexScanRule method), 247
[analyze\(\)](#) (pycypher.query_optimizer.JoinReorderingRule method), 246
[analyze\(\)](#) (pycypher.query_optimizer.LimitPushdownRule method), 246
[analyze\(\)](#) (pycypher.query_optimizer.OptimizationRule method), 245
[analyze\(\)](#) (pycypher.query_optimizer.PredicateSimplificationRule method), 246
[analyze\(\)](#) (pycypher.query_planner.QueryPlanAnalyzer method), 232
[analyze_and_plan\(\)](#) (pycypher.query_analyzer.QueryAnalyzer method), 235
[analyze_workload\(\)](#) (in module pycypher.query_profiler), 254
[And](#) (class in pycypher.ast_models), 99
[and_expression\(\)](#) (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 131
[anon_counter](#) (pycypher.pattern_matcher.VariableLengthHopParameter attribute), 173
[APISurface](#) (class in shared.compat), 375
[append\(\)](#) (pycypher.pipeline.Pipeline method), 272
[applied](#) (pycypher.query_optimizer.OptimizationResult attribute), 243, 244
[applied_plan](#) (pycypher.query_optimizer.OptimizationPlan property), 245
[apply\(\)](#) (pycypher.scan_operators.BindingFilter method), 241
[apply_expression_evaluator\(\)](#) (pycypher.query_analyzer.QueryAnalyzer method), 235
[apply_preset\(\)](#) (in module pycypher.config), 211
[apply_projection_modifiers\(\)](#) (pycypher.projection_planner.ProjectionPlanner method), 268
[apply_where_filter\(\)](#) (pycypher.clause_executor.ClauseExecutor static method), 170
[argument_description](#) (pycypher.exceptions.FunctionArgumentError attribute), 348
[arguments](#) (pycypher.ast_models.Call attribute), 89
[arguments](#) (pycypher.ast_models.FunctionInvocation attribute), 102
[Arithmetic](#) (class in pycypher.ast_models), 99
[arithmetic_evaluator](#) (pycypher.binding_evaluator.BindingExpressionEvaluator property), 186
[ArithmeticExpressionEvaluator](#) (class in pycypher.arithmetic_evaluator), 188
[arity](#) (pycypher.relational_models.RegisteredFunction attribute), 162
[ArrowData](#) (class in pycypher.ingestion.data_sources), 303
[as_html\(\)](#) (pycypher.exceptions.DocsLink method), 345
[as_plain\(\)](#) (pycypher.exceptions.DocsLink method), 345
[as_terminal\(\)](#) (pycypher.exceptions.DocsLink method), 345

- ascending (pycypher.ast_models.OrderByItem attribute), 92
- assign_column() (pycypher.backend_engine.BackendEngine method), 214
- assign_column() (pycypher.backend_engine.DuckDBBackend method), 219
- assign_column() (pycypher.backend_engine.InstrumentedBackend method), 222
- assign_column() (pycypher.backend_engine.PandasBackend method), 223
- assign_column() (pycypher.backend_engine.PolarsBackend method), 225
- ast (pycypher.multi_query_analyzer.QueryNode attribute), 289, 290
- ast (pycypher.pipeline.PipelineContext attribute), 274, 275
- AST_CACHE_MAX_ENTRIES (in module pycypher.config), 210
- ASTConversionError, 346
- ASTConverter (class in pycypher.ast_converter), 154
- ASTNode (class in pycypher.ast_models), 82
- ASTRewriter (class in pycypher.ast_rewriter), 155
- at_end (pycypher.leapfrog_triejoin.LeapfrogIterator property), 259
- attribute_map (pycypher.relational_models.EntityTable attribute), 160
- attribute_map (pycypher.relational_models.RelationshipTable attribute), 163
- AUDIT_LOGGER (in module pycypher.audit), 333
- audit_mutation() (in module pycypher.audit), 334
- audit_query_error() (in module pycypher.audit), 333
- audit_query_success() (in module pycypher.audit), 333
- available_functions() (pycypher.star.Star method), 169
- available_variables (pycypher.exceptions.VariableNotFoundError attribute), 355
- avg_latency_ms (pycypher.cluster.ClusterHealth attribute), 328
- avg_latency_ms (pycypher.cluster.WorkerHealth attribute), 327
- ## B
- backend (pycypher.query_profiler.QueryProfiler attribute), 253
- backend (pycypher.relational_models.Context property), 157
- backend_name (pycypher.relational_models.Context property), 157
- backend_timings (pycypher.query_profiler.ProfileReport attribute), 253
- BackendEngine (class in pycypher.backend_engine), 213
- BackendFrame (in module pycypher.cypher_types), 336
- BackendMask (in module pycypher.cypher_types), 337
- begin_query() (pycypher.relational_models.Context method), 158
- begin_strategy() (pycypher.query_learning.JoinPerformanceTracker method), 339
- best_version() (pycypher.query_learning.PlanVersionTracker method), 340
- BinaryExpression (class in pycypher.ast_models), 99
- BindingExpressionEvaluator (class in pycypher.binding_evaluator), 185
- BindingFilter (class in pycypher.scan_operators), 240
- BindingFrame (class in pycypher.binding_frame), 179
- bindings (pycypher.binding_frame.BindingFrame attribute), 180
- boolean_evaluator (pycypher.binding_evaluator.BindingExpressionEvaluator property), 186
- BooleanExpressionEvaluator (class in pycypher.boolean_evaluator), 190
- BooleanLiteral (class in pycypher.ast_models), 100
- breakdown (pycypher.exceptions.QueryComplexityError attribute), 352
- breakdown (pycypher.query_complexity.ComplexityScore attribute), 242
- BROADCAST (pycypher.query_planner.JoinStrategy attribute), 230
- budget_bytes (pycypher.exceptions.QueryMemoryBudgetError attribute), 352
- build() (pycypher.graph_index.AdjacencyIndex class method), 261
- build() (pycypher.graph_index.EntityLabelIndex class method), 264
- build() (pycypher.graph_index.PropertyValueIndex class method), 263
- build() (pycypher.graph_index.VectorizedPropertyStore class method), 264
- build() (pycypher.ingestion.context_builder.ContextBuilder method), 297
- build_computation_graph() (in module pycypher.lazy_eval), 281
- burst (pycypher.exceptions.RateLimitError attribute), 353
- burst (pycypher.rate_limiter.QueryRateLimiter property), 335
- ## C
- cache_plan() (pycypher.query_learning.QueryLearningStore method), 343
- cache_stats (pycypher.grammar_parser.GrammarParser property), 116

- CacheLockTimeoutError, 347
- Call (class in pycypher.ast_models), 88
- call_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 120
- call_statement() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 120
- callable (pycypher.ingestion.config.FunctionConfig attribute), 310, 311
- callable (pycypher.scalar_functions.FunctionMetadata attribute), 204
- caller_id (pycypher.exceptions.RateLimitError attribute), 353
- can_use_leapfrog() (in module pycypher.leapfrog_triejoin), 260
- CardinalityFeedbackStore (class in pycypher.cardinality_estimator), 236
- case_expression() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 138
- CaseExpression (class in pycypher.ast_models), 100
- category (pycypher.exceptions.UnsupportedFunctionError attribute), 354
- category (pycypher.exceptions.UnsupportedOperatorError attribute), 355
- changed (shared.compat.SurfaceDiff attribute), 376
- check_backend_health() (in module pycypher.backend_engine), 226
- check_exclusive_forms() (pycypher.ingestion.config.FunctionConfig method), 311
- check_neo4j_compat() (in module shared.compat), 376
- check_query_source() (pycypher.ingestion.config.QueryConfig method), 312
- check_timeout() (pycypher.relational_models.Context method), 158
- check_unique_source_ids() (pycypher.ingestion.config.SourcesConfig method), 310
- check_uri() (pycypher.ingestion.config.EntitySourceConfig method), 307
- check_uri() (pycypher.ingestion.config.OutputConfig method), 312
- check_uri() (pycypher.ingestion.config.RelationshipSourceConfig method), 309
- check_version() (pycypher.ingestion.config.PipelineConfig class method), 314
- CircuitBreaker (class in pycypher.backend_engine), 217
- CircuitState (class in pycypher.backend_engine), 218
- Clause (class in pycypher.ast_models), 89
- clause_counts (shared.metrics.MetricsSnapshot attribute), 361, 362
- clause_signature (pycypher.query_learning.QueryFingerprint attribute), 342, 343
- clause_timing_max_ms (shared.metrics.MetricsSnapshot attribute), 363
- clause_timing_p50_ms (shared.metrics.MetricsSnapshot attribute), 363
- clause_timing_p90_ms (shared.metrics.MetricsSnapshot attribute), 363
- clause_timings (pycypher.query_profiler.ProfileReport attribute), 252, 253
- ClauseExecutor (class in pycypher.clause_executor), 170
- ClauseRulesMixin (class in pycypher.grammar_rule_mixins), 120
- clauses (pycypher.ast_models.Foreach attribute), 90
- clauses (pycypher.ast_models.Query attribute), 93
- clear() (pycypher.cardinality_estimator.CardinalityFeedbackStore method), 236
- clear() (pycypher.query_learning.AdaptiveEvictionPolicy method), 338
- clear() (pycypher.query_learning.AdaptivePlanCache method), 338
- clear() (pycypher.query_learning.JoinPerformanceTracker method), 340
- clear() (pycypher.query_learning.PlanVersionTracker method), 340
- clear() (pycypher.query_learning.PredicateSelectivityTracker method), 341
- clear() (pycypher.query_learning.QueryLearningStore method), 344
- clear() (pycypher.result_cache.ResultCache method), 284
- clear() (pycypher.star.ResultCache method), 167
- clear_deadline() (pycypher.relational_models.Context method), 158
- clear_history() (pycypher.query_profiler.QueryProfiler method), 254
- clone() (pycypher.ast_models.ASTNode method), 83
- close() (pycypher.backend_engine.DuckDBBackend method), 219
- close() (pycypher.sinks.neo4j.Neo4jSink method), 223
- CLOSED (pycypher.backend_engine.CircuitState attribute), 218
- cluster_error_rate (pycypher.cluster.ClusterHealth attribute), 328
- cluster_health() (pycypher.cluster.ClusterCoordinator method), 332
- ClusterCoordinator (class in pycypher.cluster), 331
- ClusterHealth (class in pycypher.cluster), 327
- code (pycypher.ast_models.ValidationIssue attribute), 86
- coerce_join() (pycypher.frame_joiner.FrameJoiner

- method), 256
- collect_variables() (pycypher.variable_manager.VariableManager method), 270
- collection_evaluator (pycypher.binding_evaluator.BindingEvaluator property), 186
- CollectionExpressionEvaluator (class in pycypher.collection_evaluator), 199
- column (pycypher.exceptions.CypherSyntaxError attribute), 347
- column (pycypher.query_formatter.LintIssue attribute), 250, 251
- column (pycypher.semantic_validator.ValidationError attribute), 293
- column_stats() (pycypher.cardinality_estimator.TableStatistics method), 238
- ColumnStatistics (class in pycypher.cardinality_estimator), 236
- ColumnValues (in module pycypher.cypher_types), 337
- combine() (pycypher.multi_query_executor.MultiQueryExecutor method), 292
- CombinedQueryValidator (class in pycypher.query_validator), 249
- commit_query() (pycypher.relational_models.Context method), 158
- Comparison (class in pycypher.ast_models), 100
- comparison_evaluator (pycypher.binding_evaluator.BindingExpressionEvaluator property), 186
- comparison_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 131
- ComparisonEvaluator (class in pycypher.comparison_evaluator), 192
- COMPLEXITY_WARN_THRESHOLD (in module pycypher.config), 210
- ComplexityScore (class in pycypher.query_complexity), 241
- CompositeTransformer (class in pycypher.grammar_transformers), 119
- ComputationGraph (class in pycypher.lazy_eval), 280
- compute_live_columns() (in module pycypher.lazy_eval), 282
- concat() (pycypher.backend_engine.BackendEngine method), 214
- concat() (pycypher.backend_engine.DuckDBBackend method), 219
- concat() (pycypher.backend_engine.InstrumentedBackend method), 222
- concat() (pycypher.backend_engine.PandasBackend method), 223
- concat() (pycypher.backend_engine.PolarsBackend method), 225
- condition (pycypher.ast_models.WhenClause attribute), 112
- ContextBuilder (class in pycypher.ingestion.context_builder), 296
- convert() (pycypher.ast_converter.ASTConverter method), 154
- convert_ast() (in module pycypher.ast_models), 113
- correction_factor() (pycypher.cardinality_estimator.CardinalityFeeder method), 236
- correction_factor() (pycypher.query_learning.PredicateSelectivityTransformer method), 341
- count() (pycypher.graph_index.EntityLabelIndex method), 211
- count_star() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 131
- CountStar (class in pycypher.ast_models), 100
- Create (class in pycypher.ast_models), 89
- create_child_scope() (pycypher.semantic_validator.VariableScope method), 294
- create_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
- create_with_star() (pycypher.ast_rewriter.ASTRewriter method), 155
- cross_join() (pycypher.binding_frame.BindingFrame method), 182
- CROSS_JOIN_WARN_THRESHOLDS (in module pycypher.config), 210
- CSV (pycypher.ingestion.config.OutputFormat attribute), 306
- CsvFormat (class in pycypher.ingestion.data_sources), 298
- CURRENT_CONFIG_VERSION (in module pycypher.ingestion.config), 313
- CyclicDependencyError, 348
- cypher_function() (pycypher.relational_models.Context method), 159
- cypher_functions (pycypher.relational_models.Context attribute), 157
- cypher_query (pycypher.multi_query_analyzer.QueryNode attribute), 289, 290

`cypher_query()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`cypher_validate()` (pycypher.ast_models.ASTNode method), 83
`CypherSyntaxError`, 346
D
`data_source_from_uri()` (in module `pycypher.ingestion.data_sources`), 303
`dataframe` (pycypher.ingestion.data_sources.DataFrameDataSource property), 303
`DataFrameDataSource` (class in `pycypher.ingestion.data_sources`), 302
`DataSource` (class in `pycypher.ingestion.data_sources`), 300
`decode()` (in module `shared.helpers`), 367
`default()` (pycypher.pipeline.Pipeline class method), 274
`default()` (pycypher.query_optimizer.QueryOptimizer class method), 247
`DEFAULT_WEIGHTS` (in module `pycypher.query_complexity`), 241
`define()` (pycypher.semantic_validator.VariableScope method), 294
`Delete` (class in `pycypher.ast_models`), 89
`delete_clause()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`delete_items()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`delimiter` (pycypher.ingestion.data_sources.CsvFormat attribute), 298
`dependencies` (pycypher.multi_query_analyzer.QueryNode attribute), 290
`DependencyGraph` (class in `pycypher.multi_query_analyzer`), 290
`deprecated()` (in module `shared.deprecation`), 377
`deregister_worker()` (pycypher.cluster.ClusterCoordinator method), 331
`description` (pycypher.ingestion.config.ProjectConfig attribute), 306
`description` (pycypher.ingestion.config.QueryConfig attribute), 311, 312
`description` (pycypher.query_optimizer.OptimizationResult attribute), 243, 244
`description` (pycypher.scalar_functions.FunctionMetadata attribute), 204, 205
`detach` (pycypher.ast_models.Delete attribute), 89
`detach_keyword()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`detail` (pycypher.exceptions.PatternComprehensionError attribute), 351
`detect_conflicts()` (pycypher.variable_manager.VariableManager method), 270
`delete_clause()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`delete_items()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`delimiter` (pycypher.ingestion.data_sources.CsvFormat attribute), 298
`dependencies` (pycypher.multi_query_analyzer.QueryNode attribute), 290
`DependencyGraph` (class in `pycypher.multi_query_analyzer`), 290
`deprecated()` (in module `shared.deprecation`), 377
`deregister_worker()` (pycypher.cluster.ClusterCoordinator method), 331
`description` (pycypher.ingestion.config.ProjectConfig attribute), 306
`description` (pycypher.ingestion.config.QueryConfig attribute), 311, 312
`description` (pycypher.query_optimizer.OptimizationResult attribute), 243, 244
`description` (pycypher.scalar_functions.FunctionMetadata attribute), 204, 205
`detach` (pycypher.ast_models.Delete attribute), 89
`detach_keyword()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`detail` (pycypher.exceptions.PatternComprehensionError attribute), 351
`detect_conflicts()` (pycypher.variable_manager.VariableManager method), 270
`diagnostic_report()` (shared.metrics.MetricsSnapshot method), 363
`diagnostics()` (pycypher.query_learning.QueryLearningStore method), 344
`diff_surfaces()` (in module `shared.compat`), 376
`digest` (pycypher.query_learning.QueryFingerprint attribute), 342
`direction` (pycypher.ast_models.RelationshipPattern attribute), 94
`direction` (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
`dispatch_clause()` (pycypher.clause_executor.ClauseExecutor method), 172
`distinct` (pycypher.ast_models.FunctionInvocation attribute), 102
`distinct` (pycypher.ast_models.Return attribute), 95
`distinct` (pycypher.ast_models.With attribute), 98
`distinct()` (pycypher.backend_engine.BackendEngine method), 214
`distinct()` (pycypher.backend_engine.DuckDBBackend method), 219
`distinct()` (pycypher.backend_engine.InstrumentedBackend method), 222
`distinct()` (pycypher.backend_engine.PandasBackend method), 223
`distinct()` (pycypher.backend_engine.PolarsBackend method), 225
`distinct_keyword()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 121
`distinct_values` (pycypher.graph_index.PropertyValueIndex property), 263
`DistributedLearningSync` (class in `pycypher.query_learning`), 339
`DocsLink` (class in `pycypher.exceptions`), 345
`DRAINING` (pycypher.cluster.WorkerStatus attribute), 326
`drop_columns()` (pycypher.backend_engine.BackendEngine method), 215
`drop_columns()` (pycypher.backend_engine.DuckDBBackend method), 219
`drop_columns()` (pycypher.backend_engine.InstrumentedBackend method), 222
`drop_columns()` (pycypher.backend_engine.PandasBackend method), 223
`drop_columns()` (pycypher.backend_engine.PolarsBackend method), 225
`DuckDBBackend` (class in `pycypher.backend_engine`), 218
`DuckDBReader` (class in `pycypher.ingestion.duckdb_reader`), 315

E

- eager_build_vectorized_stores() (pycypher.graph_index.GraphIndexManager method), 267
- elapsed_ms (pycypher.exceptions.WorkerExecutionError attribute), 357
- elapsed_ms (pycypher.query_optimizer.OptimizationPlan attribute), 244, 245
- elapsed_seconds (pycypher.exceptions.QueryTimeoutError attribute), 353
- elements (pycypher.ast_models.ListLiteral attribute), 104
- elements (pycypher.ast_models.MapProjection attribute), 105
- elements (pycypher.ast_models.PatternPath attribute), 93
- else_clause() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 138
- else_expr (pycypher.ast_models.CaseExpression attribute), 100
- emit_deprecation() (in module shared.deprecation), 377
- enable_audit_log() (in module pycypher.audit), 334
- enable_security_log() (in module pycypher.ingestion.security), 319
- enabled (pycypher.rate_limiter.QueryRateLimiter property), 335
- enabled (pycypher.result_cache.ResultCache property), 284
- enabled (pycypher.star.ResultCache property), 167
- encode() (in module shared.helpers), 367
- end (pycypher.ast_models.Slicing attribute), 111
- ends_with_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 132
- ensure_bytes() (in module shared.helpers), 368
- ensure_uri() (in module shared.helpers), 367
- entities (pycypher.ingestion.config.SourcesConfig attribute), 309
- entity_mapping (pycypher.relational_models.Context attribute), 157
- entity_row_count() (pycypher.query_planner.QueryPlanAnalyzer method), 231
- entity_type (pycypher.graph_index.EntityLabelIndex attribute), 263, 264
- entity_type (pycypher.graph_index.PropertyValueIndex attribute), 262, 263
- entity_type (pycypher.graph_index.VectorizedPropertyStore attribute), 264
- entity_type (pycypher.ingestion.config.EntitySourceConfig attribute), 307
- entity_type (pycypher.relational_models.EntityTable attribute), 160
- entity_type (pycypher.scan_operators.EntityScan attribute), 239
- entity_type() (pycypher.binding_frame.BindingFrame method), 180
- entity_types (pycypher.query_learning.QueryFingerprint attribute), 342, 343
- entity_types_tracked (pycypher.cardinality_estimator.CardinalityFeeder property), 236
- EntityLabelIndex (class in pycypher.graph_index), 263
- EntityMapping (class in pycypher.relational_models), 159
- EntityScan (class in pycypher.scan_operators), 238
- EntitySourceConfig (class in pycypher.ingestion.config), 306
- EntityTable (class in pycypher.relational_models), 159
- entries (pycypher.ast_models.MapLiteral attribute), 105
- equality_selectivity() (pycypher.cardinality_estimator.ColumnStatistics method), 238
- ERROR (pycypher.ast_models.ValidationSeverity attribute), 88
- error (pycypher.ingestion.config.SkippedSource attribute), 305
- ERROR (pycypher.semantic_validator.ErrorSeverity attribute), 293
- error_counts (shared.metrics.MetricsSnapshot attribute), 361, 362
- error_rate (shared.metrics.MetricsSnapshot attribute), 363
- ErrorHandlingPolicy (class in pycypher.ingestion.config), 305
- errors (pycypher.ast_models.ValidationResult property), 87
- errors (pycypher.cluster.WorkerHealth attribute), 327
- errors (pycypher.input_validator.InputValidationResult attribute), 255
- errors (pycypher.query_validator.ValidationResult attribute), 248, 249
- ErrorSeverity (class in pycypher.semantic_validator), 293
- escape_sql_string_literal() (in module pycypher.ingestion.security), 320
- estimate_compound_selectivity() (pycypher.query_learning.PredicateSelectivityTracker method), 341
- estimate_compound_selectivity() (pycypher.query_learning.QueryLearningStore method), 344
- estimate_memory() (in module pycypher.lazy_eval), 281
- estimate_memory() (pycypher.query_planner.QueryPlanner method), 233
- estimate_predicate_selectivity()

(pycypher.query_planner.QueryPlanAnalyzer.evaluate_aggregation_grouped()
 method), 231
 estimated_bytes (pycypher.exceptions.QueryMemoryBudgetError.method), 187
 attribute), 352
 estimated_memory_bytes
 (pycypher.lazy_eval.OpNode attribute), 279
 estimated_memory_bytes
 (pycypher.query_planner.JoinPlan attribute), 229, 230
 estimated_rows (pycypher.lazy_eval.OpNode attribute), 279
 estimated_rows (pycypher.query_planner.JoinPlan attribute), 229
 estimated_speedup (pycypher.query_optimizer.OptimizationResult.method), 189
 attribute), 244
 eval_list_comprehension()
 (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 200
 eval_map_literal() (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 201
 eval_map_projection()
 (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 202
 eval_pattern_comprehension()
 (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 201
 eval_property_lookup()
 (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 199
 eval_quantifier() (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 200
 eval_quantifier_vectorized()
 (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 201
 eval_reduce() (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 201
 eval_slicing() (pycypher.collection_evaluator.CollectionExpressionEvaluator.method), 200
 evaluate() (pycypher.ast_models.Literal method), 104
 evaluate() (pycypher.binding_evaluator.BindingExpressionEvaluator.method), 187
 evaluate() (pycypher.evaluator_protocol.ExpressionEvaluator.method), 184
 evaluate_aggregation()
 (pycypher.aggregation_evaluator.AggregationExpressionEvaluator.method), 197
 evaluate_aggregation()
 (pycypher.binding_evaluator.BindingExpressionEvaluator.method), 187
 evaluate_aggregation_grouped()
 (pycypher.aggregation_evaluator.AggregationExpressionEvaluator.method), 197
 evaluate_and() (pycypher.boolean_evaluator.BooleanExpressionEvaluator.method), 190
 evaluate_arithmetic() (pycypher.arithmetic_evaluator.ArithmeticExpressionEvaluator.method), 188
 evaluate_bool_chain()
 (pycypher.boolean_evaluator.BooleanExpressionEvaluator.method), 192
 evaluate_case() (pycypher.comparison_evaluator.ComparisonEvaluator.method), 194
 evaluate_comparison()
 (pycypher.arithmetic_evaluator.ArithmeticExpressionEvaluator.method), 189
 evaluate_comparison()
 (pycypher.comparison_evaluator.ComparisonEvaluator.method), 192
 evaluate_exists() (pycypher.exists_evaluator.ExistsEvaluator.method), 191
 evaluate_not() (pycypher.boolean_evaluator.BooleanExpressionEvaluator.method), 191
 evaluate_or() (pycypher.comparison_evaluator.ComparisonEvaluator.method), 193
 evaluate_or() (pycypher.boolean_evaluator.BooleanExpressionEvaluator.method), 191
 evaluate_pattern_comprehension()
 (pycypher.exists_evaluator.ExistsEvaluator.method), 196
 evaluate_scalar_function()
 (pycypher.scalar_function_evaluator.ScalarFunctionEvaluator.method), 208
 evaluate_string_predicate()
 (pycypher.string_predicate_evaluator.StringPredicateEvaluator.method), 194
 evaluate_string_predicate()
 (pycypher.arithmetic_evaluator.ArithmeticExpressionEvaluator.method), 189
 evaluate_xor() (pycypher.comparison_evaluator.ComparisonEvaluator.method), 193
 evaluate_xor() (pycypher.boolean_evaluator.BooleanExpressionEvaluator.method), 191
 eviction_store() (pycypher.query_learning.AdaptiveEvictionPolicy.method), 338
 example() (pycypher.exceptions.TemporalArithmeticError attribute), 354
 example (pycypher.scalar_functions.FunctionMetadata attribute), 204, 205
 example_usage (pycypher.exceptions.MissingParameterError attribute), 351
 extend_pathname (pycypher.exceptions.DocsLink attribute), 346
 execute() (pycypher.pipeline.ExecuteStage method), 347
 execute() (pycypher.pipeline.ParseStage method), 347

- 277
- `execute()` (pycypher.pipeline.PlanStage method), 277
- `execute()` (pycypher.pipeline.Stage method), 276
- `execute()` (pycypher.pipeline.ValidateStage method), 277
- `execute()` (pycypher.query_optimizer.OptimizeStage method), 248
- `execute()` (pycypher.scalar_functions.ScalarFunctionRegistry method), 207
- `execute_multi_query()` (pycypher.multi_query_executor.MultiQueryExecutor method), 292
- `execute_query()` (pycypher.cluster.ClusterCoordinator method), 331
- `execute_query()` (pycypher.cluster.LocalWorker method), 330
- `execute_query()` (pycypher.cluster.Worker method), 326
- `execute_query()` (pycypher.star.Star method), 169
- `execute_query_async()` (pycypher.star.Star method), 169
- `execute_query_binding_frame()` (pycypher.clause_executor.ClauseExecutor method), 172
- `execute_query_inner()` (pycypher.clause_executor.ClauseExecutor method), 172
- `execute_union_query()` (pycypher.clause_executor.ClauseExecutor method), 172
- `ExecuteStage` (class in pycypher.pipeline), 276
- `Exists` (class in pycypher.ast_models), 101
- `exists_content()` (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 132
- `exists_evaluator` (pycypher.binding_evaluator.BindingEvaluator property), 187
- `exists_expression()` (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 132
- `ExistsEvaluator` (class in pycypher.exists_evaluator), 195
- `expand_variable_length_path()` (pycypher.path_expander.PathExpander method), 176
- `expected_args` (pycypher.exceptions.FunctionArgumentError attribute), 348
- `expected_type` (pycypher.exceptions.VariableTypeMismatchError attribute), 356
- `explain()` (pycypher.query_optimizer.OptimizationPlan method), 245
- `explain_query()` (pycypher.star.Star method), 169
- `explicit_args()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 122
- `export()` (shared.exporters.JSONFileExporter method), 372
- `export()` (shared.exporters.MetricsExporter method), 370
- `export()` (shared.exporters.PrometheusExporter method), 371
- `export()` (shared.exporters.StatsDExporter method), 372
- `export_once()` (in module shared.exporters), 372
- `export_state()` (pycypher.query_learning.DistributedLearningSync method), 339
- `Expression` (class in pycypher.ast_models), 101
- `Expression` (pycypher.ast_models.CaseExpression attribute), 100
- `expression` (pycypher.ast_models.IndexLookup attribute), 102
- `expression` (pycypher.ast_models.MapElement attribute), 104
- `expression` (pycypher.ast_models.OrderByItem attribute), 92
- `expression` (pycypher.ast_models.PropertyLookup attribute), 108
- `expression` (pycypher.ast_models.ReturnItem attribute), 96
- `expression` (pycypher.ast_models.SetItem attribute), 97
- `expression` (pycypher.ast_models.Slicing attribute), 111
- `expression` (pycypher.ast_models.Unwind attribute), 98
- `ExpressionEvaluatorFactory` (class in pycypher.evaluator_protocol), 185
- `ExpressionEvaluatorProtocol` (class in pycypher.evaluator_protocol), 184
- `ExpressionRenderMixin` (class in pycypher.expression_renderer), 178
- `ExpressionRulesMixin` (class in pycypher.grammar_rule_mixins), 130
- `ExpressionRulesMixin` (pycypher.ast_models.Delete attribute), 89
- `ExpressionTransformer` (class in pycypher.grammar_transformers), 118
- `extract_limit_hint()` (pycypher.query_analyzer.QueryAnalyzer method), 234
- `extract_referenced_variables()` (in module pycypher.ast_models), 113
- `FetchError`
- `FAIL` (pycypher.ingestion.config.ErrorHandlingPolicy attribute), 305
- `failure_threshold` (pycypher.backend_engine.CircuitBreaker property), 217
- `raise()` (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 144
- `fetch()` (pycypher.graph_index.VectorizedPropertyStore method), 265

fetch_multi() (pycypher.graph_index.VectorizedPropertyStore cypher.cypher_types), 336
 method), 265
 field_name() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 122
 FileDataSource (class in pycypher.ingestion.data_sources), 301
 FILTER (pycypher.lazy_eval.OpType attribute), 278
 filter() (pycypher.backend_engine.BackendEngine method), 213
 filter() (pycypher.backend_engine.DuckDBBackend method), 219
 filter() (pycypher.backend_engine.InstrumentedBackend method), 221
 filter() (pycypher.backend_engine.PandasBackend method), 223
 filter() (pycypher.backend_engine.PolarsBackend method), 225
 filter() (pycypher.binding_frame.BindingFrame method), 181
 FilterPushdownRule (class in pycypher.query_optimizer), 245
 find_all() (pycypher.ast_models.ASTNode method), 83
 find_first() (pycypher.ast_models.ASTNode method), 83
 find_nodes() (in module pycypher.ast_models), 113
 fingerprint() (pycypher.query_learning.QueryFingerprint method), 343
 fingerprint() (pycypher.query_learning.QueryLearningStore method), 343
 FloatLiteral (class in pycypher.ast_models), 101
 Foreach (class in pycypher.ast_models), 89
 foreach_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 122
 Format (class in pycypher.ingestion.data_sources), 298
 format (pycypher.ingestion.config.OutputConfig attribute), 312
 format (pycypher.ingestion.data_sources.FileDataSource property), 301
 format_query() (in module pycypher.query_formatter), 250
 frame (pycypher.binding_evaluator.BindingExpressionEvaluator attribute), 186
 frame (pycypher.comparison_evaluator.ComparisonEvaluator method), 139
 attribute), 192
 frame (pycypher.evaluator_protocol.ExpressionEvaluatorProtocol property), 184
 frame (pycypher.pattern_matcher.VariableLengthHopParams attribute), 348
 attribute), 173
 frame_size() (pycypher.clause_executor.ClauseExecutor static method), 171
 FrameDataFrame (in module pycypher.frame_joiner), 255
 FrameSeries (in module pycypher.cypher_types), 336
 from_arrow() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 317
 from_arrow() (pycypher.relational_models.EntityTable class method), 161
 from_arrow() (pycypher.relational_models.RelationshipTable class method), 164
 from_csv() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 315
 from_cypher() (pycypher.ast_converter.ASTConverter class method), 154
 from_dataframe() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 317
 from_dataframe() (pycypher.relational_models.EntityTable class method), 160
 from_dataframe() (pycypher.relational_models.RelationshipTable class method), 163
 from_dict() (pycypher.ingestion.context_builder.ContextBuilder class method), 297
 from_dict() (shared.compat.APISurface class method), 375
 from_dict() (shared.compat.SymbolInfo class method), 374
 from_json() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 316
 from_parquet() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 315
 from_sql() (pycypher.ingestion.duckdb_reader.DuckDBReader static method), 316
 FULL (pycypher.ast_models.JoinType attribute), 85
 full_rel_any() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 147
 full_rel_both() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 147
 full_rel_left() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 148
 full_rel_right() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 148
 function_arg_list() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 138
 function_args() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 139
 function_invocation() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 139
 function_name (pycypher.ast_models.FunctionInvocation property), 101
 function_name (pycypher.exceptions.FunctionArgumentError attribute), 354
 function_name (pycypher.exceptions.UnsupportedFunctionError attribute), 354
 function_name() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 139

function_simple_name() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 139
 FunctionArgumentError, 348
 FunctionConfig (class in pycypher.ingestion.config), 310
 FunctionInvocation (class in pycypher.ast_models), 101
 FunctionMetadata (class in pycypher.scalar_functions), 204
 FunctionRulesMixin (class in pycypher.grammar_rule_mixins), 138
 functions (pycypher.ingestion.config.PipelineConfig attribute), 313, 314
 FunctionTransformer (class in pycypher.grammar_transformers), 118
 fuse_filters() (in module pycypher.lazy_eval), 281

G

generate_unique_name() (pycypher.variable_manager.VariableManager method), 270
 get() (pycypher.query_learning.AdaptivePlanCache method), 338
 get() (pycypher.query_learning.SemanticResultCache method), 344
 get() (pycypher.result_cache.ResultCache method), 284
 get() (pycypher.star.ResultCache method), 167
 get_adjacency_index() (pycypher.graph_index.GraphIndexManager method), 266
 get_best_join_strategy() (pycypher.query_learning.QueryLearningStore method), 344
 get_cache_stats() (in module pycypher.star), 170
 get_cached_plan() (pycypher.query_learning.QueryLearningStore method), 343
 get_circuit_breaker() (in module pycypher.backend_engine), 226
 get_correlation_factor() (pycypher.query_learning.PredicateSelectivityTracker method), 341
 get_default_parser() (in module pycypher.grammar_parser), 117
 get_exporters() (in module shared.exporters), 372
 get_global_limiter() (in module pycypher.rate_limiter), 336
 get_history() (pycypher.query_learning.PlanVersionTracker method), 340
 get_inputs() (pycypher.lazy_eval.ComputationGraph method), 280
 get_instance() (pycypher.scalar_functions.ScalarFunctionRegistry class method), 206
 get_label_index() (pycypher.graph_index.GraphIndexManager method), 266
 get_learned_selectivity() (pycypher.query_learning.PredicateSelectivityTracker method), 341
 get_learned_selectivity() (pycypher.query_learning.QueryLearningStore method), 343
 get_learning_store() (in module pycypher.query_learning), 345
 get_plan_metrics() (pycypher.query_learning.QueryLearningStore method), 344
 get_properties_batch() (pycypher.binding_frame.BindingFrame method), 181
 get_property() (pycypher.binding_frame.BindingFrame method), 180
 get_property_index() (pycypher.graph_index.GraphIndexManager method), 266
 get_query_id() (in module shared.logger), 359
 get_rows_for_key() (pycypher.leapfrog_triejoin.LeapfrogIterator method), 259
 get_rss_mb() (in module shared.metrics), 360
 get_source_by_id() (pycypher.ingestion.config.PipelineConfig method), 314
 get_source_by_id() (pycypher.ingestion.config.SourcesConfig method), 309
 get_state() (pycypher.backend_engine.CircuitBreaker method), 218
 get_tracer() (in module shared.otel), 369
 get_undefined_vars() (pycypher.semantic_validator.VariableScope method), 294
 get_vectorized_store() (pycypher.graph_index.GraphIndexManager method), 266
 get_version_metrics() (pycypher.query_learning.PlanVersionTracker method), 340
 GrammarParser (class in pycypher.grammar_parser), 115
 GrammarTransformerSyncError, 347
 GraphIndexManager (class in pycypher.graph_index), 265
 GraphTypeNotFoundError, 349

H

HALF_OPEN (pycypher.backend_engine.CircuitState attribute), 218
 handle_match_clause() (pycypher.clause_executor.ClauseExecutor method), 171
 has_breaking_changes (shared.compat.SurfaceDiff property), 376
 has_reporters (pycypher.ast_models.ValidationResult property), 87

[has_function\(\)](#) (pycypher.scalar_functions.ScalarFunctionRegistry method), 206
[has_warnings](#) (pycypher.ast_models.ValidationResult property), 87
[HASH](#) (pycypher.query_planner.JoinStrategy attribute), 230
[HASH_AGG](#) (pycypher.query_planner.AggStrategy attribute), 228
[header](#) (pycypher.ingestion.data_sources.CsvFormat attribute), 298, 299
[health_check\(\)](#) (pycypher.cluster.LocalWorker method), 331
[health_check\(\)](#) (pycypher.cluster.Worker method), 326
[health_status\(\)](#) (shared.metrics.MetricsSnapshot method), 363
[heartbeat_timeout_s](#) (pycypher.cluster.ClusterCoordinator attribute), 331
[hints](#) (pycypher.query_optimizer.OptimizationPlan attribute), 244, 245
[hints](#) (pycypher.query_optimizer.OptimizationResult attribute), 244
[histogram_counts](#) (pycypher.cardinality_estimator.ColumnStatistics attribute), 237
[histogram_edges](#) (pycypher.cardinality_estimator.ColumnStatistics attribute), 237
[history](#) (pycypher.query_profiler.QueryProfiler attribute), 253
[hotspot](#) (pycypher.query_profiler.ProfileReport attribute), 252, 253
I
[id](#) (pycypher.ingestion.config.EntitySourceConfig attribute), 306, 307
[id](#) (pycypher.ingestion.config.QueryConfig attribute), 311, 312
[id](#) (pycypher.ingestion.config.RelationshipSourceConfig attribute), 308, 309
[id_col](#) (pycypher.ingestion.config.EntitySourceConfig attribute), 307
[id_col](#) (pycypher.ingestion.config.RelationshipSourceConfig attribute), 308, 309
[ID_COLUMN](#) (in module pycypher.constants), 165
[id_column](#) (pycypher.sinks.neo4j.NodeMapping attribute), 324
[id_property](#) (pycypher.sinks.neo4j.NodeMapping attribute), 324
[ids](#) (pycypher.graph_index.EntityLabelIndex attribute), 263, 264
[implementation](#) (pycypher.relational_models.RegisteredFunction attribute), 162
[import_state\(\)](#) (pycypher.query_learning.DistributedLearningSystem method), 339
[include_all](#) (pycypher.ast_models.MapProjection attribute), 105, 106
[incoming](#) (pycypher.graph_index.AdjacencyIndex attribute), 261
[IncompatibleOperatorError](#), 349
[index](#) (pycypher.ast_models.IndexLookup attribute), 102
[index_lookup\(\)](#) (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 132
[index_manager](#) (pycypher.relational_models.Context property), 157
[indexed_property_lookup\(\)](#) (pycypher.graph_index.GraphIndexManager method), 266
[indexed_relationship_scan\(\)](#) (pycypher.graph_index.GraphIndexManager method), 266
[IndexLookup](#) (class in pycypher.ast_models), 102
[IndexScanRule](#) (class in pycypher.query_optimizer), 246
[infer_statistics\(\)](#) (pycypher.projection_planner.ProjectionPlanner method), 268
[info_attribute_map\(\)](#) (in module pycypher.ingestion.arrow_utils), 318
[INFO](#) (pycypher.ast_models.ValidationSeverity attribute), 88
[INFO](#) (pycypher.semantic_validator.ErrorSeverity attribute), 293
[infos](#) (pycypher.ast_models.ValidationResult property), 87
[initial](#) (pycypher.ast_models.Reduce attribute), 109, 110
[inline](#) (pycypher.ingestion.config.QueryConfig attribute), 311, 312
[inline_pattern_predicate\(\)](#) (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 132
[INNER](#) (pycypher.ast_models.JoinType attribute), 84, 85
[inputs](#) (pycypher.lazy_eval.OpNode attribute), 279
[InputValidationResult](#) (class in pycypher.input_validator), 254
[InputValidator](#) (class in pycypher.input_validator), 255
[insert_after\(\)](#) (pycypher.pipeline.Pipeline method), 273
[insert_before\(\)](#) (pycypher.pipeline.Pipeline method), 272
[InstrumentedBackend](#) (class in pycypher.backend_engine), 221
[IntegerLiteral](#) (class in pycypher.ast_models), 102
[invalidate\(\)](#) (pycypher.graph_index.GraphIndexManager

- method), 267
- invalidate() (pycypher.query_learning.AdaptivePlanCache method), 338
- invalidate() (pycypher.query_learning.SemanticResultCache method), 345
- invalidate() (pycypher.result_cache.ResultCache method), 284
- invalidate() (pycypher.star.ResultCache method), 167
- invalidate_on_mutation() (pycypher.query_learning.QueryLearningStore method), 344
- InvalidCastError, 350
- is_audit_enabled() (in module pycypher.audit), 334
- is_available() (pycypher.backend_engine.CircuitBreaker method), 218
- is_defined() (pycypher.semantic_validator.VariableScope method), 294
- is_empty() (pycypher.backend_engine.BackendEngine method), 216
- is_empty() (pycypher.backend_engine.DuckDBBackend method), 220
- is_empty() (pycypher.backend_engine.InstrumentedBackend method), 222
- is_empty() (pycypher.backend_engine.PandasBackend method), 223
- is_empty() (pycypher.backend_engine.PolarsBackend method), 225
- is_not_null() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 133
- is_null() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 133
- is_null_raw_list() (in module shared.helpers), 366
- is_null_value() (in module shared.helpers), 366
- is_security_log_enabled() (in module pycypher.ingestion.security), 319
- is_valid (pycypher.ast_models.ValidationResult property), 87
- is_valid (pycypher.input_validator.InputValidationResult attribute), 254, 255
- is_valid (pycypher.query_validator.ValidationResult attribute), 248, 249
- issues (pycypher.ast_models.ValidationResult attribute), 87
- items (pycypher.ast_models.Remove attribute), 95
- items (pycypher.ast_models.Return attribute), 95
- items (pycypher.ast_models.Set attribute), 96
- items (pycypher.ast_models.With attribute), 98
- ## J
- JOIN (pycypher.lazy_eval.OpType attribute), 278
- join() (pycypher.backend_engine.BackendEngine method), 213
- join() (pycypher.backend_engine.DuckDBBackend method), 220
- join() (pycypher.backend_engine.InstrumentedBackend method), 221
- join() (pycypher.backend_engine.PandasBackend method), 223
- join() (pycypher.backend_engine.PolarsBackend method), 225
- join() (pycypher.binding_frame.BindingFrame method), 181
- join_key (pycypher.query_planner.JoinPlan attribute), 229
- JoinPerformanceTracker (class in pycypher.query_learning), 339
- JoinPlan (class in pycypher.query_planner), 229
- JoinReorderingRule (class in pycypher.query_optimizer), 246
- joins (pycypher.query_planner.QueryPlan attribute), 230, 231
- JoinStrategy (class in pycypher.query_planner), 230
- JoinType (class in pycypher.ast_models), 84
- JSON (pycypher.ingestion.config.OutputFormat attribute), 306
- JSONFileExporter (class in shared.exporters), 372
- JsonFormat (class in pycypher.ingestion.data_sources), 300
- ## K
- key (pycypher.leapfrog_triejoin.LeapfrogIterator property), 259
- keyword_suggestion (pycypher.exceptions.CypherSyntaxError attribute), 347
- kind (shared.compat.SymbolInfo attribute), 374
- kleene_and() (in module pycypher.boolean_evaluator), 190
- kleene_not() (in module pycypher.boolean_evaluator), 190
- kleene_or() (in module pycypher.boolean_evaluator), 190
- kleene_xor() (in module pycypher.boolean_evaluator), 190
- KNOWN_AGGREGATIONS (in module pycypher.aggregation_evaluator), 196
- ## L
- label (pycypher.sinks.neo4j.NodeMapping attribute), 324
- label_expression() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 148
- label_factor() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 148
- label_name() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 148

label_predicate_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 133
 label_predicate_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 224
 label_primary() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 149
 label_primary() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 225
 label_term() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 149
 label_term() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 225
 LabelPredicate (class in pycypher.ast_models), 102
 labels (pycypher.ast_models.LabelPredicate attribute), 102, 103
 labels (pycypher.ast_models.NodePattern attribute), 91
 labels (pycypher.ast_models.RelationshipPattern attribute), 94
 labels (pycypher.ast_models.RemoveItem attribute), 95
 labels (pycypher.ast_models.SetItem attribute), 97
 labels (pycypher.ast_models.SetLabelsItem attribute), 97
 last_heartbeat (pycypher.cluster.WorkerHealth attribute), 327
 LEAPFROG (pycypher.query_planner.JoinStrategy attribute), 230
 leapfrog_triejoin() (in module pycypher.leapfrog_triejoin), 259
 LeapfrogIterator (class in pycypher.leapfrog_triejoin), 259
 LeastLoadedRouter (class in pycypher.cluster), 329
 left (pycypher.ast_models.BinaryExpression attribute), 100
 LEFT (pycypher.ast_models.JoinType attribute), 84, 85
 LEFT (pycypher.ast_models.RelationshipDirection attribute), 85
 left_join() (pycypher.binding_frame.BindingFrame method), 182
 left_name (pycypher.query_planner.JoinPlan attribute), 229
 left_type (pycypher.exceptions.IncompatibleOperatorError attribute), 350
 length (pycypher.ast_models.RelationshipPattern attribute), 94
 limit (pycypher.ast_models.Return attribute), 95
 limit (pycypher.ast_models.With attribute), 98
 limit (pycypher.exceptions.QueryComplexityError attribute), 351
 LIMIT (pycypher.lazy_eval.OpType attribute), 279
 limit() (pycypher.backend_engine.BackendEngine method), 215
 limit() (pycypher.backend_engine.DuckDBBackend method), 220
 limit() (pycypher.backend_engine.InstrumentedBackend method), 222
 limit() (pycypher.backend_engine.PandasBackend method), 224
 limit() (pycypher.backend_engine.PolarsBackend method), 225
 limit() (pycypher.backend_engine.PolarsBackend method), 225
 line (pycypher.exceptions.CypherSyntaxError attribute), 346
 line (pycypher.query_formatter.LintIssue attribute), 250, 251
 line (pycypher.semantic_validator.ValidationError attribute), 293
 lint_query() (in module pycypher.query_formatter), 251
 LintIssue (class in pycypher.query_formatter), 250
 list_comprehension() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 139
 list_elements() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 144
 list_expr (pycypher.ast_models.ListComprehension attribute), 103, 104
 list_expr (pycypher.ast_models.Quantifier attribute), 108, 109
 list_expr (pycypher.ast_models.Reduce attribute), 109, 110
 list_expression (pycypher.ast_models.Foreach attribute), 90
 list_filter() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 140
 list_functions() (pycypher.scalar_functions.ScalarFunctionRegistry method), 206
 list_literal() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 145
 list_projection() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 140
 list_variable() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 140
 ListComprehension (class in pycypher.ast_models), 103
 ListLiteral (class in pycypher.ast_models), 104
 Literal (class in pycypher.ast_models), 104
 LiteralRulesMixin (class in pycypher.grammar_rule_mixins), 144
 LiteralTransformer (class in pycypher.grammar_transformers), 118
 load_pipeline_config() (in module pycypher.ingestion.config), 314
 load_snapshot() (in module shared.compat), 375
 load_with_sources() (pycypher.ingestion.config.PipelineConfig class method), 314
 LocalWorker (class in pycypher.cluster), 330
 log_cardinality_feedback() (pycypher.query_planner.QueryPlanAnalyzer method), 224

- method), 232
- LOGGER (in module `shared.logger`), 359
- LOGGING_LEVEL (in module `shared.logger`), 359
- lookup() (`pycypher.graph_index.PropertyValueIndex` method), 263
- ## M
- main() (in module `pycypher.nmetl_cli`), 332
- make_seed_frame() (`pycypher.frame_joiner.FrameJoiner` method), 258
- map_element() (`pycypher.grammar_rule_mixins.FunctionRulesMixin` method), 140
- map_elements() (`pycypher.grammar_rule_mixins.FunctionRulesMixin` method), 140
- map_entries() (`pycypher.grammar_rule_mixins.LiteralRulesMixin` method), 145
- map_entry() (`pycypher.grammar_rule_mixins.LiteralRulesMixin` method), 145
- map_expr (`pycypher.ast_models.ListComprehension` attribute), 103, 104
- map_expr (`pycypher.ast_models.PatternComprehension` attribute), 107, 108
- map_expr (`pycypher.ast_models.Reduce` attribute), 110
- map_literal() (`pycypher.grammar_rule_mixins.LiteralRulesMixin` method), 145
- map_projection() (`pycypher.grammar_rule_mixins.FunctionRulesMixin` method), 141
- MapElement (class in `pycypher.ast_models`), 104
- MapLiteral (class in `pycypher.ast_models`), 104
- mapping (`pycypher.relational_models.EntityMapping` attribute), 159
- mapping (`pycypher.relational_models.RelationshipMapping` attribute), 162
- MapProjection (class in `pycypher.ast_models`), 105
- mask_uri_credentials() (in module `pycypher.ingestion.security`), 321
- Match (class in `pycypher.ast_models`), 90
- match_clause() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 122
- match_to_binding_frame() (`pycypher.pattern_matcher.PatternMatcher` method), 175
- max (`pycypher.ast_models.PathLength` attribute), 92
- max_args (`pycypher.scalar_functions.FunctionMetadata` attribute), 204
- MAX_COLLECTION_SIZE (in module `pycypher.config`), 210
- MAX_COMPLEXITY_SCORE (in module `pycypher.config`), 210
- max_complexity_score (`pycypher.pipeline.PipelineContext` attribute), 275
- MAX_CROSS_JOIN_ROWS (in module `pycypher.config`), 210
- MAX_QUERY_NESTING_DEPTH (in module `pycypher.config`), 210
- MAX_QUERY_SIZE_BYTES (in module `pycypher.config`), 210
- MAX_UNBOUNDED_PATH_HOPS (in module `pycypher.config`), 210
- max_value (`pycypher.cardinality_estimator.ColumnStatistics` attribute), 237
- memory_budget_bytes (`pycypher.pipeline.PipelineContext` attribute), 275
- memory_delta_max_mb (`shared.metrics.MetricsSnapshot` attribute), 363
- memory_delta_mb (`pycypher.query_profiler.ProfileReport` attribute), 252, 253
- memory_delta_p50_mb (`shared.metrics.MetricsSnapshot` attribute), 362
- memory_delta_p90_mb (`shared.metrics.MetricsSnapshot` attribute), 362
- memory_estimate_bytes() (`pycypher.backend_engine.BackendEngine` method), 217
- memory_estimate_bytes() (`pycypher.backend_engine.DuckDBBackend` method), 220
- memory_estimate_bytes() (`pycypher.backend_engine.InstrumentedBackend` method), 222
- memory_estimate_bytes() (`pycypher.backend_engine.PandasBackend` method), 224
- memory_estimate_bytes() (`pycypher.backend_engine.PolarsBackend` method), 225
- Merge (class in `pycypher.ast_models`), 90
- MERGE (`pycypher.query_planner.JoinStrategy` attribute), 230
- merge_action() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 123
- merge_action_create() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 123
- merge_action_match() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 123
- merge_action_type() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 123
- merge_clause() (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 123

<code>merge_frames_for_match()</code> (<code>pycypher.frame_joiner.FrameJoiner</code> method), 257	<code>model_config</code> (<code>pycypher.ast_models.Comparison</code> attribute), 100
<code>merge_queries()</code> (<code>pycypher.ast_rewriter.ASTRewriter</code> method), 155	<code>model_config</code> (<code>pycypher.ast_models.CountStar</code> attribute), 100
<code>message</code> (<code>pycypher.ast_models.ValidationIssue</code> attribute), 86	<code>model_config</code> (<code>pycypher.ast_models.Create</code> attribute), 89
<code>message</code> (<code>pycypher.exceptions.InvalidCastError</code> attribute), 350	<code>model_config</code> (<code>pycypher.ast_models.Delete</code> attribute), 89
<code>message</code> (<code>pycypher.exceptions.WrongCypherTypeError</code> attribute), 357	<code>model_config</code> (<code>pycypher.ast_models.Exists</code> attribute), 101
<code>message</code> (<code>pycypher.query_formatter.LintIssue</code> attribute), 250, 251	<code>model_config</code> (<code>pycypher.ast_models.Expression</code> attribute), 101
<code>message</code> (<code>pycypher.semantic_validator.ValidationError</code> attribute), 293	<code>model_config</code> (<code>pycypher.ast_models.FloatLiteral</code> attribute), 101
<code>metadata</code> (<code>pycypher.pipeline.PipelineContext</code> attribute), 274, 275	<code>model_config</code> (<code>pycypher.ast_models.Foreach</code> attribute), 90
<code>metadata</code> (<code>pycypher.pipeline.PipelineResult</code> attribute), 276	<code>model_config</code> (<code>pycypher.ast_models.FunctionInvocation</code> attribute), 102
<code>metrics_summary()</code> (<code>pycypher.query_profiler.QueryProfiler</code> method), 253	<code>model_config</code> (<code>pycypher.ast_models.IndexLookup</code> attribute), 102
<code>MetricsExporter</code> (class in <code>shared.exporters</code>), 370	<code>model_config</code> (<code>pycypher.ast_models.IntegerLiteral</code> attribute), 102
<code>MetricsSnapshot</code> (class in <code>shared.metrics</code>), 360	<code>model_config</code> (<code>pycypher.ast_models.LabelPredicate</code> attribute), 103
<code>min</code> (<code>pycypher.ast_models.PathLength</code> attribute), 92	<code>model_config</code> (<code>pycypher.ast_models.ListComprehension</code> attribute), 103
<code>min_args</code> (<code>pycypher.scalar_functions.FunctionMetadata</code> attribute), 204	<code>model_config</code> (<code>pycypher.ast_models.ListLiteral</code> attribute), 104
<code>min_value</code> (<code>pycypher.cardinality_estimator.ColumnStatistics</code> attribute), 237	<code>model_config</code> (<code>pycypher.ast_models.Literal</code> attribute), 104
<code>missing_node_type</code> (<code>pycypher.exceptions.GrammarTransformerSyntaxError</code> attribute), 347	<code>model_config</code> (<code>pycypher.ast_models.MapElement</code> attribute), 104
<code>MissingParameterError</code> , 350	<code>model_config</code> (<code>pycypher.ast_models.MapLiteral</code> attribute), 105
<code>model_config</code> (<code>pycypher.ast_models.AddAllPropertiesLiteral</code> attribute), 88	<code>model_config</code> (<code>pycypher.ast_models.MapProjection</code> attribute), 105
<code>model_config</code> (<code>pycypher.ast_models.Algebraizable</code> attribute), 84	<code>model_config</code> (<code>pycypher.ast_models.Match</code> attribute), 90
<code>model_config</code> (<code>pycypher.ast_models.AllShortestPaths</code> attribute), 99	<code>model_config</code> (<code>pycypher.ast_models.Merge</code> attribute), 90
<code>model_config</code> (<code>pycypher.ast_models.And</code> attribute), 99	<code>model_config</code> (<code>pycypher.ast_models.NodePattern</code> attribute), 91
<code>model_config</code> (<code>pycypher.ast_models.Arithmetic</code> attribute), 99	<code>model_config</code> (<code>pycypher.ast_models.Not</code> attribute), 106
<code>model_config</code> (<code>pycypher.ast_models.ASTNode</code> attribute), 83	<code>model_config</code> (<code>pycypher.ast_models.NullCheck</code> attribute), 106
<code>model_config</code> (<code>pycypher.ast_models.BinaryExpression</code> attribute), 99	<code>model_config</code> (<code>pycypher.ast_models.NullLiteral</code> attribute), 106
<code>model_config</code> (<code>pycypher.ast_models.BooleanLiteral</code> attribute), 100	<code>model_config</code> (<code>pycypher.ast_models.Or</code> attribute), 106
<code>model_config</code> (<code>pycypher.ast_models.Call</code> attribute), 88	<code>model_config</code> (<code>pycypher.ast_models.OrderByItem</code> attribute), 91
<code>model_config</code> (<code>pycypher.ast_models.CaseExpression</code> attribute), 100	<code>model_config</code> (<code>pycypher.ast_models.Parameter</code> attribute), 107
<code>model_config</code> (<code>pycypher.ast_models.Clause</code> attribute), 89	

<code>model_config</code> (<code>pycypher.ast_models.PathLength</code> attribute), 92	<code>model_config</code> (<code>pycypher.ast_models.ValidationIssue</code> attribute), 85
<code>model_config</code> (<code>pycypher.ast_models.Pattern</code> attribute), 92	<code>model_config</code> (<code>pycypher.ast_models.ValidationResult</code> attribute), 87
<code>model_config</code> (<code>pycypher.ast_models.PatternComprehension</code> attribute), 107	<code>model_config</code> (<code>pycypher.ast_models.Variable</code> attribute), 112
<code>model_config</code> (<code>pycypher.ast_models.PatternIntersection</code> attribute), 92	<code>model_config</code> (<code>pycypher.ast_models.WhenClause</code> attribute), 112
<code>model_config</code> (<code>pycypher.ast_models.PatternPath</code> attribute), 93	<code>model_config</code> (<code>pycypher.ast_models.With</code> attribute), 98
<code>model_config</code> (<code>pycypher.ast_models.PropertyLookup</code> attribute), 108	<code>model_config</code> (<code>pycypher.ast_models.Xor</code> attribute), 112
<code>model_config</code> (<code>pycypher.ast_models.Quantifier</code> attribute), 109	<code>model_config</code> (<code>pycypher.ast_models.YieldItem</code> attribute), 99
<code>model_config</code> (<code>pycypher.ast_models.Query</code> attribute), 93	<code>model_config</code> (<code>pycypher.ingestion.config.EntitySourceConfig</code> attribute), 307
<code>model_config</code> (<code>pycypher.ast_models.Reduce</code> attribute), 110	<code>model_config</code> (<code>pycypher.ingestion.config.FunctionConfig</code> attribute), 311
<code>model_config</code> (<code>pycypher.ast_models.RelationshipPattern</code> attribute), 94	<code>model_config</code> (<code>pycypher.ingestion.config.OutputConfig</code> attribute), 312
<code>model_config</code> (<code>pycypher.ast_models.Remove</code> attribute), 95	<code>model_config</code> (<code>pycypher.ingestion.config.PipelineConfig</code> attribute), 314
<code>model_config</code> (<code>pycypher.ast_models.RemoveItem</code> attribute), 95	<code>model_config</code> (<code>pycypher.ingestion.config.ProjectConfig</code> attribute), 306
<code>model_config</code> (<code>pycypher.ast_models.Return</code> attribute), 95	<code>model_config</code> (<code>pycypher.ingestion.config.QueryConfig</code> attribute), 312
<code>model_config</code> (<code>pycypher.ast_models.ReturnAll</code> attribute), 96	<code>model_config</code> (<code>pycypher.ingestion.config.RelationshipSourceConfig</code> attribute), 309
<code>model_config</code> (<code>pycypher.ast_models.ReturnItem</code> attribute), 96	<code>model_config</code> (<code>pycypher.ingestion.config.SourcesConfig</code> attribute), 310
<code>model_config</code> (<code>pycypher.ast_models.Set</code> attribute), 96	<code>model_config</code> (<code>pycypher.relational_models.Context</code> attribute), 159
<code>model_config</code> (<code>pycypher.ast_models.SetAllPropertiesItem</code> attribute), 96	<code>model_config</code> (<code>pycypher.relational_models.EntityMapping</code> attribute), 159
<code>model_config</code> (<code>pycypher.ast_models.SetItem</code> attribute), 97	<code>model_config</code> (<code>pycypher.relational_models.EntityTable</code> attribute), 162
<code>model_config</code> (<code>pycypher.ast_models.SetLabelsItem</code> attribute), 97	<code>model_config</code> (<code>pycypher.relational_models.RegisteredFunction</code> attribute), 162
<code>model_config</code> (<code>pycypher.ast_models.SetPropertyItem</code> attribute), 97	<code>model_config</code> (<code>pycypher.relational_models.RelationshipMapping</code> attribute), 162
<code>model_config</code> (<code>pycypher.ast_models.ShortestPath</code> attribute), 110	<code>model_config</code> (<code>pycypher.relational_models.RelationshipTable</code> attribute), 163
<code>model_config</code> (<code>pycypher.ast_models.Slicing</code> attribute), 110	<code>model_config</code> (<code>pycypher.sinks.neo4j.NodeMapping</code> attribute), 324
<code>model_config</code> (<code>pycypher.ast_models.StringLiteral</code> attribute), 111	<code>model_config</code> (<code>pycypher.sinks.neo4j.RelationshipMapping</code> attribute), 325
<code>model_config</code> (<code>pycypher.ast_models.StringPredicate</code> attribute), 111	<code>model_post_init()</code> (<code>pycypher.ast_models.PropertyLookup</code> method), 108
<code>model_config</code> (<code>pycypher.ast_models.Unary</code> attribute), 111	<code>model_post_init()</code> (<code>pycypher.relational_models.Context</code> method), 157
<code>model_config</code> (<code>pycypher.ast_models.UnionQuery</code> attribute), 98	module
<code>model_config</code> (<code>pycypher.ast_models.Unwind</code> attribute), 98	<code>pycypher.aggregation_evaluator</code> , 196
	<code>pycypher.aggregation_planner</code> , 198
	<code>pycypher.arithmetic_evaluator</code> , 188

pycypher.ast_converter, 154
 pycypher.ast_models, 82
 pycypher.ast_rewriter, 154
 pycypher.audit, 332
 pycypher.backend_engine, 211
 pycypher.binding_evaluator, 185
 pycypher.binding_frame, 178
 pycypher.boolean_evaluator, 190
 pycypher.cardinality_estimator, 235
 pycypher.clause_executor, 170
 pycypher.cluster, 325
 pycypher.collection_evaluator, 199
 pycypher.comparison_evaluator, 192
 pycypher.config, 209
 pycypher.constants, 165
 pycypher.cypher_types, 336
 pycypher.evaluator_protocol, 184
 pycypher.exceptions, 345
 pycypher.exists_evaluator, 195
 pycypher.expression_renderer, 177
 pycypher.frame_joiner, 255
 pycypher.grammar_parser, 115
 pycypher.grammar_rule_mixins, 120
 pycypher.grammar_transformers, 118
 pycypher.graph_index, 260
 pycypher.ingestion.arrow_utils, 317
 pycypher.ingestion.config, 304
 pycypher.ingestion.context_builder, 295
 pycypher.ingestion.data_sources, 297
 pycypher.ingestion.duckdb_reader, 315
 pycypher.ingestion.output_writer, 318
 pycypher.ingestion.security, 319
 pycypher.input_validator, 254
 pycypher.lazy_eval, 278
 pycypher.leapfrog_triejoin, 258
 pycypher.multi_query_analyzer, 289
 pycypher.multi_query_executor, 291
 pycypher.mutation_engine, 286
 pycypher.nmetl_cli, 332
 pycypher.path_expander, 175
 pycypher.pattern_matcher, 172
 pycypher.pipeline, 271
 pycypher.projection_planner, 267
 pycypher.query_analyzer, 234
 pycypher.query_complexity, 241
 pycypher.query_formatter, 249
 pycypher.query_learning, 337
 pycypher.query_optimizer, 242
 pycypher.query_planner, 228
 pycypher.query_profiler, 251
 pycypher.query_validator, 248
 pycypher.rate_limiter, 334
 pycypher.relational_models, 156
 pycypher.result_cache, 282

pycypher.scalar_function_evaluator, 207
 pycypher.scalar_functions, 202
 pycypher.scan_operators, 238
 pycypher.semantic_validator, 293
 pycypher.sinks.neo4j, 321
 pycypher.star, 166
 pycypher.string_predicate_evaluator, 194
 pycypher.timeout_handler, 284
 pycypher.variable_manager, 269
 shared, 358
 shared.compat, 373
 shared.deprecation, 377
 shared.exporters, 370
 shared.helpers, 366
 shared.logger, 359
 shared.metrics, 360
 shared.otel, 368
 shared.telemetry, 368
 module (pycypher.ingestion.config.FunctionConfig attribute), 310, 311
 module (shared.compat.SymbolInfo attribute), 374
 module_name (shared.compat.APISurface attribute), 375
 mult_expression() (pycypher.grammar_rule_mixins.ExpressionRules method), 134
 mult_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 134
 multi_way_join() (pycypher.frame_joiner.FrameJoiner method), 256
 MultiQueryExecutor (class in pycypher.multi_query_executor), 291
 mutate() (pycypher.binding_frame.BindingFrame method), 183
 mutate_batch() (pycypher.binding_frame.BindingFrame method), 183
 MutationEngine (class in pycypher.mutation_engine), 286

N

name (pycypher.ast_models.FunctionInvocation attribute), 102
 name (pycypher.ast_models.Parameter attribute), 107
 name (pycypher.ast_models.Variable attribute), 111, 112
 name (pycypher.backend_engine.BackendEngine property), 213
 name (pycypher.backend_engine.DuckDBBackend property), 220
 name (pycypher.backend_engine.InstrumentedBackend property), 221
 name (pycypher.backend_engine.PandasBackend property), 224

name (pycypher.backend_engine.PolarsBackend property), 226
 name (pycypher.ingestion.config.ProjectConfig attribute), 306
 name (pycypher.ingestion.data_sources.CsvFormat property), 299
 name (pycypher.ingestion.data_sources.Format property), 298
 name (pycypher.ingestion.data_sources.JsonFormat property), 300
 name (pycypher.ingestion.data_sources.ParquetFormat property), 299
 name (pycypher.pipeline.ExecuteStage attribute), 277
 name (pycypher.pipeline.ParseStage attribute), 277
 name (pycypher.pipeline.PlanStage attribute), 277
 name (pycypher.pipeline.Stage attribute), 276
 name (pycypher.pipeline.ValidateStage attribute), 277
 name (pycypher.query_optimizer.FilterPushdownRule attribute), 246
 name (pycypher.query_optimizer.IndexScanRule attribute), 247
 name (pycypher.query_optimizer.JoinReorderingRule attribute), 246
 name (pycypher.query_optimizer.LimitPushdownRule attribute), 246
 name (pycypher.query_optimizer.OptimizationRule attribute), 245
 name (pycypher.query_optimizer.OptimizeStage attribute), 248
 name (pycypher.query_optimizer.PredicateSimplificationRule attribute), 246
 name (pycypher.relational_models.RegisteredFunction attribute), 162
 name (pycypher.scalar_functions.FunctionMetadata attribute), 204
 name (shared.compat.SymbolInfo attribute), 374
 name (shared.exporters.JSONFileExporter property), 372
 name (shared.exporters.MetricsExporter property), 370
 name (shared.exporters.PrometheusExporter property), 371
 name (shared.exporters.StatsDExporter property), 372
 names (pycypher.ingestion.config.FunctionConfig attribute), 310, 311
 namespace_name() (pycypher.grammar_rule_mixins.FunctionRuleMixin method), 141
 ndv (pycypher.cardinality_estimator.ColumnStatistics attribute), 237
 neighbors_incoming() (pycypher.graph_index.AdjacencyIndex method), 262
 neighbors_incoming_batch() (pycypher.graph_index.AdjacencyIndex method), 262
 neighbors_outgoing() (pycypher.graph_index.AdjacencyIndex method), 262
 neighbors_outgoing_batch() (pycypher.graph_index.AdjacencyIndex method), 262
 NEO4J_COMPAT_NOTES (in module shared.compat), 373
 Neo4jSink (class in pycypher.sinks.neo4j), 322
 NESTED_LOOP (pycypher.query_planner.JoinStrategy attribute), 230
 new_version (shared.compat.SurfaceDiff attribute), 376
 next() (pycypher.leapfrog_triejoin.LeapfrogIterator method), 259
 next_entity_ids() (pycypher.mutation_engine.MutationEngine method), 287
 next_relationship_ids() (pycypher.mutation_engine.MutationEngine method), 287
 next_type (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
 next_var (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
 node (pycypher.ast_models.ValidationIssue attribute), 86
 node_id (pycypher.lazy_eval.OpNode attribute), 279
 node_labels() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 149
 node_pattern() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 149
 node_pattern_filler() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 149
 node_pattern_to_binding_frame() (pycypher.pattern_matcher.PatternMatcher method), 174
 node_properties() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 150
 node_type (pycypher.ast_models.ValidationIssue attribute), 86
 node_type (pycypher.exceptions.ASTConversionError attribute), 346
 node_type (pycypher.semantic_validator.ValidationError attribute), 293
 node_where() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 150
 NodeMapping (class in pycypher.sinks.neo4j), 323
 NodePattern (class in pycypher.ast_models), 91
 nodes (pycypher.lazy_eval.ComputationGraph attribute), 280
 nodes (pycypher.multi_query_analyzer.DependencyGraph

attribute), 290
 non_null_mask (pycypher.constants.NullMaskResult attribute), 166
 non_null_vals (pycypher.constants.NullMaskResult attribute), 166
 normalize_entity_table() (in module pycypher.ingestion.arrow_utils), 317
 normalize_relationship_table() (in module pycypher.ingestion.arrow_utils), 318
 Not (class in pycypher.ast_models), 106
 not_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 134
 not_in_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 134
 notes (pycypher.query_planner.JoinPlan attribute), 229, 230
 null_check_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 134
 null_fraction (pycypher.cardinality_estimator.ColumnStatistics attribute), 237
 null_literal() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 145
 null_padding (pycypher.ingestion.data_sources.CsvFormat attribute), 298, 299
 null_predicate_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 134
 NullCheck (class in pycypher.ast_models), 106
 NullLiteral (class in pycypher.ast_models), 106
 NullMaskResult (class in pycypher.constants), 165
 nulls_placement (pycypher.ast_models.OrderByItem attribute), 92
 nulls_placement() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 123
 number_literal() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 145

O
 old_version (shared.compat.SurfaceDiff attribute), 376
 on_create (pycypher.ast_models.Merge attribute), 90
 on_error (pycypher.ingestion.config.EntitySourceConfig attribute), 307
 on_error (pycypher.ingestion.config.RelationshipSourceConfig attribute), 309
 on_match (pycypher.ast_models.Merge attribute), 90
 op_type (pycypher.lazy_eval.OpNode attribute), 279
 OPEN (pycypher.backend_engine.CircuitState attribute), 218
 operand (pycypher.ast_models.LabelPredicate attribute), 102, 103
 operand (pycypher.ast_models.Not attribute), 106
 operand (pycypher.ast_models.NullCheck attribute), 106
 operand (pycypher.ast_models.Unary attribute), 111
 operands (pycypher.ast_models.And attribute), 99
 operands (pycypher.ast_models.Or attribute), 106
 operands (pycypher.ast_models.Xor attribute), 113
 operation (pycypher.exceptions.CacheLockTimeoutError attribute), 348
 operation_rules_mixin (pycypher.backend_engine.InstrumentedBackend attribute), 221
 operation_rules_mixin (pycypher.backend_engine.InstrumentedBackend attribute), 221
 operator (pycypher.ast_models.And attribute), 99
 operator (pycypher.ast_models.BinaryExpression attribute), 100
 operator (pycypher.ast_models.NullCheck attribute), 106
 operator (pycypher.ast_models.Or attribute), 106
 operator (pycypher.ast_models.Unary attribute), 111
 operator (pycypher.ast_models.Xor attribute), 113
 operator (pycypher.exceptions.IncompatibleOperatorError attribute), 349
 operator (pycypher.exceptions.UnsupportedOperatorError attribute), 355
 OpNode (class in pycypher.lazy_eval), 279
 OptimizationPlan (class in pycypher.query_optimizer), 244
 OptimizationResult (class in pycypher.query_optimizer), 243
 OptimizationRule (class in pycypher.query_optimizer), 245
 OptimizeStage (pycypher.query_optimizer.QueryOptimizer method), 247
 OptimizeStage (class in pycypher.query_optimizer), 247
 optional (pycypher.ast_models.Match attribute), 90
 OpType (class in pycypher.lazy_eval), 278
 Or (class in pycypher.ast_models), 106
 or_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 135
 order_by (pycypher.ast_models.Return attribute), 95
 order_by (pycypher.ast_models.With attribute), 98
 order_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 123
 order_direction() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124
 order_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124
 order_items() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124

-
- OrderByItem (class in pycypher.ast_models), 91
 - OTEL_ENABLED (in module shared.otel), 368
 - outgoing (pycypher.graph_index.AdjacencyIndex attribute), 261
 - output (pycypher.ingestion.config.PipelineConfig attribute), 313, 314
 - output_node (pycypher.lazy_eval.ComputationGraph attribute), 280
 - OutputConfig (class in pycypher.ingestion.config), 312
 - OutputFormat (class in pycypher.ingestion.config), 305
 - P**
 - PandasBackend (class in pycypher.backend_engine), 223
 - Parameter (class in pycypher.ast_models), 106
 - parameter() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 146
 - parameter_name (pycypher.exceptions.MissingParameterError attribute), 351
 - parameter_name() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 146
 - parameterize_duckdb_query() (in module pycypher.ingestion.security), 321
 - parameters (pycypher.pipeline.PipelineContext attribute), 274, 275
 - params (pycypher.lazy_eval.OpNode attribute), 279
 - PARQUET (pycypher.ingestion.config.OutputFormat attribute), 306
 - ParquetFormat (class in pycypher.ingestion.data_sources), 299
 - parse() (pycypher.grammar_parser.GrammarParser method), 116
 - parse_file() (pycypher.grammar_parser.GrammarParser method), 116
 - parse_file_to_ast() (pycypher.grammar_parser.GrammarParser method), 117
 - parse_time_max_ms (shared.metrics.MetricsSnapshot attribute), 363
 - parse_time_ms (pycypher.query_profiler.ProfileReport attribute), 252, 253
 - parse_time_p50_ms (shared.metrics.MetricsSnapshot attribute), 363
 - parse_time_p90_ms (shared.metrics.MetricsSnapshot attribute), 363
 - parse_to_ast() (pycypher.grammar_parser.GrammarParser method), 116
 - parser (pycypher.grammar_parser.GrammarParser attribute), 115, 116
 - ParseStage (class in pycypher.pipeline), 277
 - PATH_HOP_COLUMN_PREFIX (in module pycypher.binding_frame), 179
 - path_length() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 150
 - path_length_range() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 150
 - path_pattern() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 150
 - path_var_name (pycypher.pattern_matcher.VariableLengthHopParameter attribute), 173
 - PathExpander (class in pycypher.path_expander), 176
 - PathLength (class in pycypher.ast_models), 92
 - paths (pycypher.ast_models.Pattern attribute), 92
 - Pattern (class in pycypher.ast_models), 92
 - pattern (pycypher.ast_models.AllShortestPaths attribute), 99
 - pattern (pycypher.ast_models.Create attribute), 89
 - pattern (pycypher.ast_models.Match attribute), 90
 - pattern (pycypher.ast_models.Merge attribute), 90
 - pattern (pycypher.ast_models.PatternComprehension attribute), 107, 108
 - pattern (pycypher.ast_models.ShortestPath attribute), 110
 - pattern() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 151
 - pattern_comp_variable() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 141
 - pattern_comprehension() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 141
 - pattern_element() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 151
 - pattern_filter() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 141
 - pattern_list (pycypher.ast_models.PatternIntersection attribute), 92
 - pattern_path_to_binding_frame() (pycypher.pattern_matcher.PatternMatcher method), 175
 - pattern_projection() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 142
 - PatternComprehension (class in pycypher.ast_models), 107
 - PatternComprehensionError, 351
 - PatternIntersection (class in pycypher.ast_models), 92
 - PatternMatcher (class in pycypher.pattern_matcher), 173
 - PatternPath (class in pycypher.ast_models), 92
 - PatternRulesMixin (class in pycypher.grammar_rule_mixins), 147
 - PatternTransformer (class in pycypher.grammar_transformers), 118
 - Pipeline (class in pycypher.pipeline), 272

PipelineConfig (class in pycypher.ingestion.config), 313
 PipelineContext (class in pycypher.pipeline), 274
 PipelineResult (class in pycypher.pipeline), 275
 plan_aggregation() (pycypher.query_planner.QueryPlanner method), 233
 plan_analysis (pycypher.pipeline.PipelineContext attribute), 275
 plan_cross_join() (pycypher.query_planner.QueryPlanner method), 233
 plan_join() (pycypher.query_planner.QueryPlanner method), 232
 plan_query() (pycypher.query_analyzer.QueryAnalyzer method), 234
 plan_time_max_ms (shared.metrics.MetricsSnapshot attribute), 363
 plan_time_ms (pycypher.query_profiler.ProfileReport attribute), 252, 253
 plan_time_p50_ms (shared.metrics.MetricsSnapshot attribute), 363
 plan_time_p90_ms (shared.metrics.MetricsSnapshot attribute), 363
 planner_accuracy_ratio_p50 (shared.metrics.MetricsSnapshot attribute), 363
 planner_mae_mb (shared.metrics.MetricsSnapshot attribute), 363
 PlanStage (class in pycypher.pipeline), 277
 PlanVersionTracker (class in pycypher.query_learning), 340
 PolarsBackend (class in pycypher.backend_engine), 224
 policy (pycypher.ingestion.config.SkippedSource attribute), 305
 postfix_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 135
 postfix_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 135
 pow_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
 power_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
 predicate (pycypher.scan_operators.BindingFilter attribute), 241
 PredicateSelectivityTracker (class in pycypher.query_learning), 341
 PredicateSimplificationRule (class in pycypher.query_optimizer), 246
 pretty() (pycypher.ast_models.ASTNode method), 83
 prev_var (pycypher.pattern_matcher.VariableLengthHopPattern attribute), 173
 print_ast() (in module pycypher.ast_models), 114
 procedure_name (pycypher.ast_models.Call attribute), 89
 procedure_reference() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124
 process_call() (pycypher.mutation_engine.MutationEngine method), 289
 process_create() (pycypher.mutation_engine.MutationEngine method), 287
 process_delete() (pycypher.mutation_engine.MutationEngine method), 288
 process_foreach() (pycypher.mutation_engine.MutationEngine method), 288
 process_merge() (pycypher.mutation_engine.MutationEngine method), 288
 process_optional_match() (pycypher.frame_joiner.FrameJoiner method), 257
 process_optional_match_failure() (pycypher.frame_joiner.FrameJoiner method), 257
 process_unwind_clause() (pycypher.clause_executor.ClauseExecutor method), 171
 produces (pycypher.multi_query_analyzer.QueryNode attribute), 290
 profile() (pycypher.query_profiler.QueryProfiler method), 253
 ProfileReport (class in pycypher.query_profiler), 251
 project (pycypher.ingestion.config.PipelineConfig attribute), 313
 PROJECT (pycypher.lazy_eval.OpType attribute), 278
 project() (pycypher.binding_frame.BindingFrame method), 183
 ProjectConfig (class in pycypher.ingestion.config), 313
 ProjectionPlanner (class in pycypher.projection_planner), 267
 PrometheusExporter (class in shared.exporters), 371
 PromoteProperty (pycypher.ast_models.AddAllPropertiesItem attribute), 88
 PropagateProperty (pycypher.ast_models.NodePattern attribute), 91
 properties (pycypher.ast_models.RelationshipPattern attribute), 94
 properties (pycypher.ast_models.SetAllPropertiesItem attribute), 96
 properties (pycypher.graph_index.VectorizedPropertyStore property), 265
 properties() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 151
 HopPattern (pycypher.ast_models.MapElement attribute), 104
 property (pycypher.ast_models.PropertyLookup attribute), 108

property (pycypher.ast_models.RemoveItem attribute), 95
 property (pycypher.ast_models.SetItem attribute), 97
 property (pycypher.ast_models.SetPropertyItem attribute), 97
 property_arrays (pycypher.graph_index.VectorizedPropertyStar attribute), 264
 property_columns (pycypher.sinks.neo4j.NodeMapping property attribute), 324
 property_columns (pycypher.sinks.neo4j.RelationshipMapping property attribute), 325
 property_key_value() (pycypher.grammar_rule_mixins.PatternRuleMixin method), 151
 property_list() (pycypher.grammar_rule_mixins.PatternRuleMixins method), 151
 property_lookup() (pycypher.grammar_rule_mixins.ExpressionRuleMixins method), 136
 property_name (pycypher.graph_index.PropertyValueIndex attribute), 262, 263
 property_name() (pycypher.grammar_rule_mixins.PatternRuleMixins method), 152
 PropertyLookup (class in pycypher.ast_models), 108
 PropertyValueIndex (class in pycypher.graph_index), 262
 push_filters_down() (in module pycypher.lazy_eval), 281
 put() (pycypher.query_learning.AdaptivePlanCache method), 339
 put() (pycypher.query_learning.SemanticResultCache method), 345
 put() (pycypher.result_cache.ResultCache method), 284
 put() (pycypher.star.ResultCache method), 167
 pycypher.aggregation_evaluator module, 196
 pycypher.aggregation_planner module, 198
 pycypher.arithmetic_evaluator module, 188
 pycypher.ast_converter module, 154
 pycypher.ast_models module, 82
 pycypher.ast_rewriter module, 154
 pycypher.audit module, 332
 pycypher.backend_engine module, 211
 pycypher.binding_evaluator module, 185
 pycypher.binding_frame module, 178
 pycypher.boolean_evaluator module, 190
 pycypher.cardinality_estimator module, 235
 pycypher.clause_executor module, 170
 pycypher.collection_evaluator module, 199
 pycypher.comparison_evaluator module, 192
 pycypher.config module, 209
 pycypher.constants module, 165
 pycypher.exceptions module, 336
 pycypher.evaluator_protocol module, 184
 pycypher.expressions module, 345
 pycypher.exists_evaluator module, 195
 pycypher.expression_renderer module, 177
 pycypher.frame_joiner module, 255
 pycypher.grammar_parser module, 115
 pycypher.grammar_rule_mixins module, 120
 pycypher.grammar_transformers module, 118
 pycypher.graph_index module, 260
 pycypher.ingestion.arrow_utils module, 317
 pycypher.ingestion.config module, 304
 pycypher.ingestion.context_builder module, 295
 pycypher.ingestion.data_sources module, 297
 pycypher.ingestion.duckdb_reader module, 315
 pycypher.ingestion.output_writer module, 318
 pycypher.ingestion.security module, 319
 pycypher.input_validator module, 254
 pycypher.lazy_eval module, 278

pycypher.leapfrog_triejoin
 module, 258
 pycypher.multi_query_analyzer
 module, 289
 pycypher.multi_query_executor
 module, 291
 pycypher.mutation_engine
 module, 286
 pycypher.nmetl_cli
 module, 332
 pycypher.path_expander
 module, 175
 pycypher.pattern_matcher
 module, 172
 pycypher.pipeline
 module, 271
 pycypher.projection_planner
 module, 267
 pycypher.query_analyzer
 module, 234
 pycypher.query_complexity
 module, 241
 pycypher.query_formatter
 module, 249
 pycypher.query_learning
 module, 337
 pycypher.query_optimizer
 module, 242
 pycypher.query_planner
 module, 228
 pycypher.query_profiler
 module, 251
 pycypher.query_validator
 module, 248
 pycypher.rate_limiter
 module, 334
 pycypher.relational_models
 module, 156
 pycypher.result_cache
 module, 282
 pycypher.scalar_function_evaluator
 module, 207
 pycypher.scalar_functions
 module, 202
 pycypher.scan_operators
 module, 238
 pycypher.semantic_validator
 module, 293
 pycypher.sinks.neo4j
 module, 321
 pycypher.star
 module, 166
 pycypher.string_predicate_evaluator
 module, 194

pycypher.timeout_handler
 module, 284
 pycypher.variable_manager
 module, 269

Q

qps (pycypher.exceptions.RateLimitError attribute),
 353
 qps (pycypher.rate_limiter.QueryRateLimiter prop-
 erty), 335
 qualify_alias() (pycypher.projection_planner.ProjectionPlanner
 method), 268
 Quantifier (class in pycypher.ast_models), 108
 quantifier (pycypher.ast_models.Quantifier at-
 tribute), 108, 109
 quantifier() (pycypher.grammar_rule_mixins.FunctionRulesMixin
 method), 142
 quantifier_expression()
 (pycypher.grammar_rule_mixins.FunctionRulesMixin
 method), 142
 quantifier_variable() (pycypher.grammar_rule_mixins.FunctionRules
 method), 142
 queries (pycypher.ingestion.config.PipelineConfig at-
 tribute), 313, 314
 queries_executed (pycypher.cluster.WorkerHealth
 attribute), 327
 queries_per_second (shared.metrics.MetricsSnapshot
 attribute), 362
 Query (class in pycypher.ast_models), 93
 query (pycypher.exceptions.CypherSyntaxError at-
 tribute), 346
 query (pycypher.ingestion.config.EntitySourceConfig
 attribute), 307
 query (pycypher.ingestion.config.RelationshipSourceConfig
 attribute), 308, 309
 query (pycypher.ingestion.data_sources.FileDataSource
 property), 301
 query (pycypher.ingestion.data_sources.SqlDataSource
 property), 302
 query (pycypher.query_profiler.ProfileReport at-
 tribute), 252
 query_fragment (pycypher.exceptions.ASTConversionError
 attribute), 346
 query_fragment (pycypher.exceptions.QueryTimeoutError
 attribute), 353
 query_id (pycypher.ingestion.config.OutputConfig
 attribute), 312
 query_id (pycypher.multi_query_analyzer.QueryNode
 attribute), 289, 290
 query_input (pycypher.pipeline.PipelineContext at-
 tribute), 275
 QUERY_METRICS (in module shared.metrics), 360
 query_snippet (pycypher.exceptions.WorkerExecutionError
 attribute), 357

- `query_statement()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124
`query_string` (pycypher.pipeline.PipelineContext attribute), 274, 275
`QUERY_TIMEOUT_S` (in module pycypher.config), 210
`QueryAnalyzer` (class in pycypher.query_analyzer), 234
`QueryComplexityError`, 351
`QueryConfig` (class in pycypher.ingestion.config), 311
`QueryDependencyAnalyzer` (class in pycypher.multi_query_analyzer), 291
`QueryFingerprint` (class in pycypher.query_learning), 342
`QueryFingerprinter` (class in pycypher.query_learning), 343
`QueryLearningStore` (class in pycypher.query_learning), 343
`QueryMemoryBudgetError`, 352
`QueryMetrics` (class in shared.metrics), 364
`QueryNode` (class in pycypher.multi_query_analyzer), 289
`QueryOptimizer` (class in pycypher.query_optimizer), 247
`QueryPlan` (class in pycypher.query_planner), 230
`QueryPlanAnalyzer` (class in pycypher.query_planner), 231
`QueryPlanner` (class in pycypher.query_planner), 232
`QueryProfiler` (class in pycypher.query_profiler), 253
`QueryRateLimiter` (class in pycypher.rate_limiter), 335
`QueryRouter` (class in pycypher.cluster), 329
`QueryTimeoutError`, 352
- ## R
- `random_hash()` (in module pycypher.ast_models), 88
`range_selectivity()` (pycypher.cardinality_estimator.ColumnStatistics method), 238
`RATE_LIMIT_BURST` (in module pycypher.config), 210
`RATE_LIMIT_QPS` (in module pycypher.config), 211
`RateLimitError`, 353
`read()` (pycypher.ingestion.data_sources.ArrowDataSource method), 303
`read()` (pycypher.ingestion.data_sources.DataFrameDataSource method), 303
`read()` (pycypher.ingestion.data_sources.DataSource method), 301
`read()` (pycypher.ingestion.data_sources.FileDataSource method), 301
`read_clause()` (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 124
`ReadWriteLock` (class in pycypher.result_cache), 282
`recent_error_rate` (shared.metrics.MetricsSnapshot attribute), 363
`recent_queries_per_second` (shared.metrics.MetricsSnapshot attribute), 362
`recommendations` (pycypher.query_profiler.ProfileReport attribute), 252, 253
`record()` (pycypher.cardinality_estimator.CardinalityFeedbackStore method), 236
`record()` (pycypher.query_learning.JoinPerformanceTracker method), 340
`record()` (pycypher.query_learning.PredicateSelectivityTracker method), 342
`record_access()` (pycypher.query_learning.AdaptiveEvictionPolicy method), 338
`record_cardinality_feedback()` (pycypher.query_analyzer.QueryAnalyzer method), 235
`record_compound()` (pycypher.query_learning.PredicateSelectivityTracker method), 342
`record_error()` (shared.metrics.QueryMetrics method), 365
`record_execution()` (pycypher.query_learning.PlanVersionTracker method), 340
`record_failure()` (pycypher.backend_engine.CircuitBreaker method), 217
`record_join_performance()` (pycypher.query_learning.QueryLearningStore method), 343
`record_metrics_to_span()` (in module shared.otel), 369
`record_plan()` (pycypher.query_learning.PlanVersionTracker method), 341
`record_statistics_execution()` (pycypher.query_learning.QueryLearningStore method), 344
`record_plan_version()` (pycypher.query_learning.QueryLearningStore method), 344
`record_query()` (shared.metrics.QueryMetrics method), 364
`record_selectivity()` (pycypher.query_learning.QueryLearningStore method), 343
`record_success()` (pycypher.backend_engine.CircuitBreaker method), 217
`records` (pycypher.ingestion.data_sources.JsonFormat attribute), 300
`recovery_timeout` (pycypher.backend_engine.CircuitBreaker property), 217

Reduce (class in pycypher.ast_models), 109
 reduce_accumulator() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 142
 reduce_expression() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 143
 reduce_variable() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 143
 regex_match_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
 register_function() (pycypher.scalar_functions.ScalarFunctionRegistry method), 206
 register_worker() (pycypher.cluster.ClusterCoordinator attribute), 309
 RegisteredFunction (class in pycypher.relational_models), 162
 rel_ast (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
 rel_detail() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 152
 rel_filler() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 152
 rel_properties() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 152
 rel_type (pycypher.graph_index.AdjacencyIndex attribute), 261
 rel_type (pycypher.scan_operators.RelationshipScan attribute), 240
 rel_type (pycypher.sinks.neo4j.RelationshipMapping attribute), 324
 rel_type() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 152
 rel_types() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 153
 rel_var (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
 rel_var (pycypher.scan_operators.RelationshipScan attribute), 240
 rel_where() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 153
 relationship_mapping (pycypher.relational_models.Context attribute), 157
 relationship_pattern() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 153
 relationship_row_count() (pycypher.query_planner.QueryPlanAnalyzer method), 232
 RELATIONSHIP_SOURCE_COLUMN (in module pycypher.constants), 165
 RELATIONSHIP_TARGET_COLUMN (in module pycypher.constants), 165
 relationship_type (pycypher.ingestion.config.RelationshipSourceConfig attribute), 308, 309
 relationship_type (pycypher.relational_models.RelationshipTable attribute), 163
 relationship_types (pycypher.query_learning.QueryFingerprint attribute), 342, 343
 RelationshipDirection (class in pycypher.ast_models), 85
 RelationshipMapping (class in pycypher.relational_models), 162
 RelationshipMapping (class in pycypher.sinks.neo4j), 324
 RelationshipPattern (class in pycypher.ast_models), 163
 Registry (pycypher.ingestion.config.SourcesConfig attribute), 309
 RelationshipScan (class in pycypher.scan_operators), 239
 RelationshipSourceConfig (class in pycypher.ingestion.config), 308
 RelationshipTable (class in pycypher.relational_models), 162
 release_read() (pycypher.result_cache.ReadWriteLock method), 283
 release_write() (pycypher.result_cache.ReadWriteLock method), 283
 remaining_nodes (pycypher.exceptions.CyclicDependencyError attribute), 348
 Remove (class in pycypher.ast_models), 95
 remove() (pycypher.pipeline.Pipeline method), 273
 remove() (pycypher.query_learning.AdaptiveEvictionPolicy method), 338
 remove_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 125
 remove_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 125
 remove_items() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 125
 remove_labels_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 125
 remove_properties() (pycypher.mutation_engine.MutationEngine method), 288
 remove_property_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 125
 rs_patch (pycypher.backends.compat.SurfaceDiff attribute), 376
 RemoveItem (class in pycypher.ast_models), 95
 rename() (pycypher.backend_engine.BackendEngine method), 214
 rename() (pycypher.backend_engine.DuckDBBackend method), 220
 rename() (pycypher.backend_engine.InstrumentedBackend method), 221
 rename() (pycypher.backend_engine.PandasBackend method), 224
 rename() (pycypher.backend_engine.PolarsBackend method), 226
 rename() (pycypher.binding_frame.BindingFrame method), 226

method), 182
 rename_variables() (pycypher.variable_manager.VariableManager method), 126
 method), 271
 render() (pycypher.expression_renderer.ExpressionRenderer method), 178
 render() (shared.exporters.PrometheusExporter method), 371
 require_bound_frame()
 (pycypher.clause_executor.ClauseExecutor static method), 171
 reset() (pycypher.backend_engine.CircuitBreaker method), 218
 reset() (pycypher.rate_limiter.QueryRateLimiter method), 335
 reset() (shared.metrics.QueryMetrics method), 365
 reset_global_limiter() (in module pycypher.rate_limiter), 336
 reset_query_id() (in module shared.logger), 359
 restore_savepoint() (pycypher.relational_models.Context method), 159
 result (pycypher.ast_models.WhenClause attribute), 112
 result (pycypher.constants.NullMaskResult attribute), 166
 result (pycypher.pipeline.PipelineContext attribute), 274, 275
 result (pycypher.pipeline.PipelineResult attribute), 275, 276
 result_cache_entries (shared.metrics.MetricsSnapshot attribute), 363
 result_cache_evictions
 (shared.metrics.MetricsSnapshot attribute), 363
 result_cache_hit_rate
 (shared.metrics.MetricsSnapshot attribute), 363
 result_cache_hits (shared.metrics.MetricsSnapshot attribute), 363
 RESULT_CACHE_MAX_MB (in module pycypher.config), 211
 result_cache_misses (shared.metrics.MetricsSnapshot attribute), 363
 result_cache_size_mb
 (shared.metrics.MetricsSnapshot attribute), 363
 RESULT_CACHE_TTL_S (in module pycypher.config), 211
 ResultCache (class in pycypher.result_cache), 283
 ResultCache (class in pycypher.star), 166
 results (pycypher.query_optimizer.OptimizationPlan attribute), 244, 245
 Return (class in pycypher.ast_models), 95
 return_alias() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 126
 return_body() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 126
 return_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 126
 return_from_frame() (pycypher.projection_planner.ProjectionPlanner method), 269
 return_item() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 126
 return_items() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 126
 ReturnAll (class in pycypher.ast_models), 96
 ReturnItem (class in pycypher.ast_models), 96
 right (pycypher.ast_models.BinaryExpression attribute), 100
 RIGHT (pycypher.ast_models.JoinType attribute), 85
 RIGHT (pycypher.ast_models.RelationshipDirection attribute), 85
 right_name (pycypher.query_planner.JoinPlan attribute), 229
 right_type (pycypher.exceptions.IncompatibleOperatorError attribute), 350
 rollback_query() (pycypher.relational_models.Context method), 158
 RoundRobinRouter (class in pycypher.cluster), 329
 router (pycypher.cluster.ClusterCoordinator attribute), 331
 row_count (pycypher.cardinality_estimator.ColumnStatistics attribute), 237
 row_count (pycypher.cardinality_estimator.TableStatistics property), 238
 row_count (pycypher.query_profiler.ProfileReport attribute), 252, 253
 row_count() (pycypher.backend_engine.BackendEngine method), 216
 row_count() (pycypher.backend_engine.DuckDBBackend method), 220
 row_count() (pycypher.backend_engine.InstrumentedBackend method), 222
 row_count() (pycypher.backend_engine.PandasBackend method), 224
 row_count() (pycypher.backend_engine.PolarsBackend method), 226
 row_limit (pycypher.pattern_matcher.VariableLengthHopParams attribute), 173
 rows_per_second (shared.metrics.MetricsSnapshot attribute), 362
 rule_name (pycypher.query_optimizer.OptimizationResult attribute), 243, 244
 rule_names (pycypher.query_optimizer.QueryOptimizer property), 247
 run() (pycypher.pipeline.Pipeline method), 273

S

- `sanitize_error_message()` (in module `py-cypher.exceptions`), 346
- `sanitize_file_path()` (in module `py-cypher.ingestion.security`), 319
- `sanitize_sql_identifer()` (in module `py-cypher.ingestion.security`), 320
- `save_snapshot()` (in module `shared.compat`), 375
- `savepoint()` (`pycypher.relational_models.Context` method), 159
- `scalar_function_evaluator` (`pycypher.binding_evaluator.BindingExpressionEvaluator` property), 186
- `scalar_registry` (`pycypher.binding_evaluator.BindingExpressionEvaluator` attribute), 186
- `ScalarFunctionEvaluator` (class in `py-cypher.scalar_function_evaluator`), 207
- `ScalarFunctionRegistry` (class in `py-cypher.scalar_functions`), 205
- `SCAN` (`pycypher.lazy_eval.OpType` attribute), 278
- `scan()` (`pycypher.scan_operators.EntityScan` method), 239
- `scan()` (`pycypher.scan_operators.RelationshipScan` method), 240
- `scan_entity()` (`pycypher.backend_engine.BackendEngine` method), 213
- `scan_entity()` (`pycypher.backend_engine.DuckDBBackend` method), 220
- `scan_entity()` (`pycypher.backend_engine.InstrumentedBackend` method), 221
- `scan_entity()` (`pycypher.backend_engine.PandasBackend` method), 224
- `scan_entity()` (`pycypher.backend_engine.PolarsBackend` method), 226
- `scan_rel_types` (`pycypher.pattern_matcher.VariableLengthHopParameters` attribute), 173
- `schema_hints` (`pycypher.ingestion.config.EntitySourceConfig` attribute), 307
- `schema_hints` (`pycypher.ingestion.config.RelationshipSourceConfig` attribute), 308, 309
- `score` (`pycypher.exceptions.QueryComplexityError` attribute), 351
- `score_query()` (in module `py-cypher.query_complexity`), 242
- `searched_case()` (`pycypher.grammar_rule_mixins.FunctionRulesMixin` method), 143
- `searched_when()` (`pycypher.grammar_rule_mixins.FunctionRulesMixin` method), 143
- `security_event_log()` (in module `py-cypher.ingestion.security`), 319
- `SECURITY_LOGGER` (in module `py-cypher.ingestion.security`), 319
- `SecurityError`, 353
- `seek()` (`pycypher.leapfrog_triejoin.LeapfrogIterator` method), 259
- `select_backend()` (in module `py-cypher.backend_engine`), 226
- `select_backend_for_query()` (in module `py-cypher.backend_engine`), 227
- `select_eviction_candidate()` (`pycypher.query_learning.AdaptiveEvictionPolicy` method), 338
- `select_worker()` (`pycypher.cluster.LeastLoadedRouter` method), 330
- `select_worker()` (`pycypher.cluster.QueryRouter` method), 329
- `select_worker()` (`pycypher.cluster.RoundRobinRouter` method), 329
- `SemanticResultCache` (class in `py-cypher.query_learning`), 344
- `SemanticValidator` (class in `py-cypher.semantic_validator`), 294
- `Set` (class in `pycypher.ast_models`), 96
- `set_all_properties_item()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 127
- `set_clause()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 127
- `set_deadline()` (`pycypher.relational_models.Context` method), 158
- `set_item()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 127
- `set_items()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 127
- `set_join_plans()` (`pycypher.frame_joiner.FrameJoiner` method), 256
- `set_labels_item()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 127
- `set_properties()` (`pycypher.mutation_engine.MutationEngine` method), 287
- `set_property_item()` (`pycypher.grammar_rule_mixins.ClauseRulesMixin` method), 128
- `set_query_id()` (in module `shared.logger`), 359
- `SetAllPropertiesItem` (class in `py-cypher.ast_models`), 96
- `SetItem` (class in `pycypher.ast_models`), 96
- `SetLabelsItem` (class in `pycypher.ast_models`), 97
- `SetPropertyItem` (class in `pycypher.ast_models`), 97
- `severity` (`pycypher.ast_models.ValidationIssue` attribute), 85
- `severity` (`pycypher.query_formatter.LintIssue` attribute), 250, 251
- `severity` (`pycypher.semantic_validator.ValidationError` attribute), 293
- `shadow_create_entity()` (`pycypher.mutation_engine.MutationEngine` method), 287
- `shadow_create_relationship()`

- (pycypher.mutation_engine.MutationEngine method), 287
- shared
 - module, 358
- shared.compat
 - module, 373
- shared.deprecation
 - module, 377
- shared.exporters
 - module, 370
- shared.helpers
 - module, 366
- shared.logger
 - module, 359
- shared.metrics
 - module, 360
- shared.otel
 - module, 368
- shared.telemetry
 - module, 368
- short_circuit_reason (pycypher.pipeline.PipelineContext attribute), 275
- short_circuited (pycypher.pipeline.PipelineContext attribute), 275
- shortest_path() (pycypher.grammar_rule_mixins.PatternRulesMixin method), 153
- shortest_path_mode (pycypher.ast_models.PatternPath attribute), 93
- shortest_path_to_binding_frame()
 - (pycypher.path_expander.PathExpander method), 177
- ShortestPath (class in pycypher.ast_models), 110
- show_config() (in module pycypher.config), 211
- signature (shared.compat.SymbolInfo attribute), 374
- signed_number() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 146
- simple_case() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 144
- simple_when() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 144
- size (pycypher.graph_index.AdjacencyIndex attribute), 261
- size (pycypher.graph_index.PropertyValueIndex attribute), 263
- size (pycypher.graph_index.VectorizedPropertyStore property), 265
- skip (pycypher.ast_models.Return attribute), 95
- skip (pycypher.ast_models.With attribute), 98
- SKIP (pycypher.ingestion.config.ErrorHandlingPolicy attribute), 305
- skip()
 - (pycypher.backend_engine.BackendEngine method), 216
 - (pycypher.backend_engine.DuckDBBackend method), 220
- skip() (pycypher.backend_engine.InstrumentedBackend method), 222
- skip() (pycypher.backend_engine.PandasBackend method), 224
- skip() (pycypher.backend_engine.PolarsBackend method), 226
- skip_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 128
- skipped_rules (pycypher.query_optimizer.OptimizationPlan property), 245
- SkippedSource (class in pycypher.ingestion.config), 305
- slice_end() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
- slice_start() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
- Slicing (class in pycypher.ast_models), 110
- slicing() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
- slow_queries (shared.metrics.MetricsSnapshot attribute), 361, 362
- SLOW_QUERY_THRESHOLD_S (in module shared.metrics), 360
- snapshot() (shared.metrics.QueryMetrics method), 361
- snapshot_api_surface() (in module shared.compat), 375
- SORT (pycypher.lazy_eval.OpType attribute), 278
- sort()
 - (pycypher.backend_engine.BackendEngine method), 215
 - (pycypher.backend_engine.DuckDBBackend method), 220
 - (pycypher.backend_engine.InstrumentedBackend method), 222
 - (pycypher.backend_engine.PandasBackend method), 224
 - (pycypher.backend_engine.PolarsBackend method), 226
- sort() (pycypher.query_planner.AggrStrategy attribute), 228
- sorted_ids (pycypher.graph_index.VectorizedPropertyStore attribute), 264
- source (pycypher.ingestion.config.QueryConfig attribute), 311, 312
- source_col (pycypher.ingestion.config.RelationshipSourceConfig attribute), 308, 309
- source_id (pycypher.ingestion.config.SkippedSource attribute), 305
- source_id_column (pycypher.sinks.neo4j.RelationshipMapping attribute), 325
- source_id_property (pycypher.sinks.neo4j.RelationshipMapping attribute), 325
- source_label (pycypher.sinks.neo4j.RelationshipMapping attribute), 324

source_obj (pycypher.relational_models.EntityTable attribute), 160
 source_obj (pycypher.relational_models.RelationshipTable attribute), 163
 source_obj_attribute_map (pycypher.relational_models.EntityTable attribute), 160
 source_obj_attribute_map (pycypher.relational_models.RelationshipTable attribute), 163
 SourceObject (in module pycypher.cypher_types), 337
 sources (pycypher.ingestion.config.PipelineConfig attribute), 313
 SourcesConfig (class in pycypher.ingestion.config), 309
 SqlDataSource (class in pycypher.ingestion.data_sources), 302
 src_col (pycypher.scan_operators.RelationshipScan attribute), 240
 Stage (class in pycypher.pipeline), 276
 stage_names (pycypher.pipeline.Pipeline property), 273
 stage_timings (pycypher.pipeline.PipelineContext attribute), 274, 275
 stage_timings (pycypher.pipeline.PipelineResult attribute), 275, 276
 Star (class in pycypher.star), 168
 star (pycypher.pipeline.PipelineContext attribute), 275
 star (pycypher.query_profiler.QueryProfiler attribute), 253
 start (pycypher.ast_models.Slicing attribute), 111
 starts_with_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 136
 state() (pycypher.backend_engine.CircuitBreaker method), 218
 statement() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 128
 statement_list() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 128
 statements (pycypher.ast_models.UnionQuery attribute), 98
 StatementTransformer (class in pycypher.grammar_transformers), 119
 stats (pycypher.query_learning.AdaptivePlanCache property), 339
 stats (pycypher.query_learning.SemanticResultCache property), 345
 stats() (pycypher.graph_index.GraphIndexManager method), 267
 stats() (pycypher.result_cache.ResultCache method), 284
 stats() (pycypher.star.ResultCache method), 168
 StatsDExporter (class in shared.exporters), 371
 status (pycypher.cluster.LocalWorker property), 330
 status (pycypher.cluster.Worker property), 326
 status (pycypher.cluster.WorkerHealth attribute), 326, 327
 strategy (pycypher.query_planner.JoinPlan attribute), 229
 strategy_stats() (pycypher.query_learning.JoinPerformanceTracker method), 340
 STREAMING_AGG (pycypher.query_planner.AggregStrategy attribute), 228
 string_literal() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 146
 string_predicate_evaluator (pycypher.binding_evaluator.BindingExpressionEvaluator property), 187
 string_predicate_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 137
 string_predicate_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 137
 StringLiteral (class in pycypher.ast_models), 111
 StringPredicate (class in pycypher.ast_models), 111
 StringPredicateEvaluator (class in pycypher.string_predicate_evaluator), 194
 strip_return() (pycypher.ast_rewriter.ASTRewriter method), 155
 suggest_close_match() (in module shared.helpers), 366
 suggestion (pycypher.ast_models.ValidationIssue attribute), 86
 suggestion (pycypher.exceptions.IncompatibleOperatorError attribute), 350
 suggestion (pycypher.exceptions.QueryMemoryBudgetError attribute), 352
 suggestion (pycypher.exceptions.VariableTypeMismatchError attribute), 356
 suggestion (pycypher.exceptions.UnsupportedFunctionError attribute), 354
 suggestion (pycypher.exceptions.UnsupportedOperatorError attribute), 355
 summary() (shared.compat.SurfaceDiff method), 376
 summary() (shared.metrics.MetricsSnapshot method), 363
 SUPPORTED_CONFIG_VERSIONS (in module pycypher.ingestion.config), 313
 supported_functions (pycypher.exceptions.UnsupportedFunctionError attribute), 354
 supported_operators (pycypher.exceptions.UnsupportedOperatorError attribute), 355
 SurfaceDiff (class in shared.compat), 375
 SymbolInfo (class in shared.compat), 374
 symbols (shared.compat.APISurface attribute), 375

T

table (pycypher.ingestion.data_sources.ArrowDataSource property), 303

TableStatistics (class in py-cypher.cardinality_estimator), 238
 target_col (pycypher.ingestion.config.RelationshipSourceConfig method), 165
 attribute), 308, 309
 target_id_column (pycypher.sinks.neo4j.RelationshipMapping method), 161
 attribute), 325
 target_id_property (pycypher.sinks.neo4j.RelationshipMapping method), 165
 attribute), 325
 target_label (pycypher.sinks.neo4j.RelationshipMapping method), 280
 attribute), 324
 TemporalArithmeticError, 354
 tgt_col (pycypher.scan_operators.RelationshipScan attribute), 240
 timeout_seconds (pycypher.exceptions.CacheLockTimeoutError attribute), 347
 timeout_seconds (pycypher.exceptions.QueryTimeoutError attribute), 353
 timeout_seconds (pycypher.pipeline.PipelineContext attribute), 275
 timeout_seconds (pycypher.timeout_handler.TimeoutHandler attribute), 285
 property), 285
 TimeoutHandler (class in py-cypher.timeout_handler), 285
 timing_max_ms (shared.metrics.MetricsSnapshot attribute), 362
 timing_min_ms (shared.metrics.MetricsSnapshot attribute), 362
 timing_p50_ms (shared.metrics.MetricsSnapshot attribute), 361, 362
 timing_p90_ms (shared.metrics.MetricsSnapshot attribute), 361, 362
 timing_p99_ms (shared.metrics.MetricsSnapshot attribute), 361, 362
 timing_summary() (pycypher.backend_engine.InstrumentedBackend method), 221
 to_cypher() (pycypher.ast_rewriter.ASTRewriter method), 155
 to_dict() (pycypher.ast_models.ASTNode method), 84
 to_dict() (shared.compat.APISurface method), 375
 to_dict() (shared.compat.SymbolInfo method), 374
 to_dict() (shared.metrics.MetricsSnapshot method), 364
 to_pandas() (pycypher.backend_engine.BackendEngine method), 216
 to_pandas() (pycypher.backend_engine.DuckDBBackend method), 221
 to_pandas() (pycypher.backend_engine.InstrumentedBackend method), 222
 to_pandas() (pycypher.backend_engine.PandasBackend method), 224
 to_pandas() (pycypher.backend_engine.PolarsBackend method), 226
 to_pandas() (pycypher.relational_models.EntityTable method), 161
 to_pandas() (pycypher.relational_models.EntityTable method), 165
 to_spark() (pycypher.relational_models.EntityTable method), 161
 to_spark() (pycypher.relational_models.RelationshipTable method), 165
 topological_order() (pycypher.lazy_eval.ComputationGraph method), 290
 topological_sort() (pycypher.multi_query_analyzer.DependencyGraph method), 290
 total (pycypher.query_complexity.ComplexityScore attribute), 242
 total_errors (pycypher.cluster.ClusterHealth attribute), 328
 total_errors (shared.metrics.MetricsSnapshot attribute), 361, 362
 total_estimated_memory_bytes (pycypher.query_planner.QueryPlan attribute), 230, 231
 total_estimated_speedup (pycypher.query_optimizer.OptimizationPlan attribute), 244, 245
 total_queries (pycypher.cluster.ClusterHealth attribute), 328
 total_queries (shared.metrics.MetricsSnapshot attribute), 361, 362
 total_rows_returned (shared.metrics.MetricsSnapshot attribute), 362
 total_time_ms (pycypher.query_profiler.ProfileReport attribute), 252, 253
 total_workers (pycypher.cluster.ClusterHealth attribute), 327, 328
 trace_backend() (in module shared.otel), 369
 trace_query() (in module shared.otel), 369
 tracked_patterns (pycypher.query_learning.PredicateSelectivityTracker property), 342
 transform() (pycypher.grammar_transformers.CompositeTransformer method), 119
 transformer (pycypher.grammar_parser.GrammarParser attribute), 115, 116
 traverse() (pycypher.ast_models.ASTNode method), 84
 traverse_ast() (in module pycypher.ast_models), 114
 rule() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 147
 backacquire() (pycypher.rate_limiter.QueryRateLimiter method), 335
 type_name (pycypher.exceptions.GraphTypeNotFoundError attribute), 349
 type_registry (pycypher.binding_frame.BindingFrame attribute), 180

U

Unary (class in pycypher.ast_models), 111

unary_expression() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 137

unary_op() (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), 137

UNAVAILABLE (pycypher.cluster.WorkerStatus attribute), 326

unavailable_workers (pycypher.cluster.ClusterHealth attribute), 328

unbounded (pycypher.ast_models.PathLength attribute), 92

UNDIRECTED (pycypher.ast_models.RelationshipDirection attribute), 85

UNION (pycypher.lazy_eval.OpType attribute), 279

union_all_marker() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 128

union_op() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 129

UnionQuery (class in pycypher.ast_models), 97

unsigned_number() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 147

UnsupportedFunctionError, 354

UnsupportedOperatorError, 355

Unwind (class in pycypher.ast_models), 98

unwind_binding_frame() (pycypher.clause_executor.ClauseExecutor method), 171

unwind_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 129

update_cache_stats() (shared.metrics.QueryMetrics method), 365

update_clause() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 129

update_statement() (pycypher.grammar_rule_mixins.ClauseRulesMixin method), 129

uptime_s (shared.metrics.MetricsSnapshot attribute), 362

uri (pycypher.ingestion.config.EntitySourceConfig attribute), 306, 307

uri (pycypher.ingestion.config.OutputConfig attribute), 312

uri (pycypher.ingestion.config.RelationshipSourceConfig attribute), 308, 309

uri (pycypher.ingestion.data_sources.ArrowDataSource value property), 303

uri (pycypher.ingestion.data_sources.DataFrameDataSource value property), 302

uri (pycypher.ingestion.data_sources.DataSource value property), 301

uri (pycypher.ingestion.data_sources.FileDataSource value property), 301

uri (pycypher.ingestion.data_sources.SqlDataSource value property), 302

url (pycypher.exceptions.DocsLink attribute), 345

use() (pycypher.semantic_validator.VariableScope method), 294

V

validate() (pycypher.grammar_parser.GrammarParser method), 117

validate() (pycypher.input_validator.InputValidator method), 255

validate() (pycypher.multi_query_executor.MultiQueryExecutor method), 292

validate() (pycypher.query_validator.CombinedQueryValidator method), 249

validate() (pycypher.semantic_validator.SemanticValidator method), 295

validate_callable_path() (pycypher.ast_models.ASTNode method), 84

validate_callable_path() (pycypher.ingestion.config.FunctionConfig class method), 311

validate_identifier() (in module pycypher.backend_engine), 227

validate_module_path() (pycypher.ingestion.config.FunctionConfig class method), 311

validate_query() (in module pycypher.semantic_validator), 295

validate_sql_query() (in module pycypher.ingestion.security), 320

validate_uri_scheme() (in module pycypher.ingestion.security), 320

ValidateStage (class in pycypher.pipeline), 277

ValidateRuleMixin (class in pycypher.semantic_validator), 293

ValidationIssue (class in pycypher.ast_models), 85

ValidationResult (class in pycypher.ast_models), 86

ValidationResult (class in pycypher.query_validator), 248

ValidationSeverity (class in pycypher.ast_models), 88

value (pycypher.ast_models.BooleanLiteral attribute), 100

value (pycypher.ast_models.FloatLiteral attribute), 101

value (pycypher.ast_models.IntegerLiteral attribute), 102

value (pycypher.ast_models.ListLiteral attribute), 104

value (pycypher.ast_models.Literal attribute), 104

value (pycypher.ast_models.MapLiteral attribute), 105

- value (pycypher.ast_models.NullLiteral attribute), 106
 - value (pycypher.ast_models.SetPropertyItem attribute), 97
 - value (pycypher.ast_models.StringLiteral attribute), 111
 - value_to_ids (pycypher.graph_index.PropertyValueIndex attribute), 262, 263
 - var_name (pycypher.scan_operators.EntityScan attribute), 239
 - var_names (pycypher.binding_frame.BindingFrame property), 180
 - Variable (class in pycypher.ast_models), 111
 - variable (pycypher.ast_models.AddAllPropertiesItem attribute), 88
 - variable (pycypher.ast_models.Foreach attribute), 90
 - variable (pycypher.ast_models.ListComprehension attribute), 103, 104
 - variable (pycypher.ast_models.MapProjection attribute), 105
 - variable (pycypher.ast_models.NodePattern attribute), 91
 - variable (pycypher.ast_models.PatternComprehension attribute), 107
 - variable (pycypher.ast_models.PatternPath attribute), 93
 - variable (pycypher.ast_models.PropertyLookup attribute), 108
 - variable (pycypher.ast_models.Quantifier attribute), 108, 109
 - variable (pycypher.ast_models.Reduce attribute), 109, 110
 - variable (pycypher.ast_models.RelationshipPattern attribute), 94
 - variable (pycypher.ast_models.RemoveItem attribute), 95
 - variable (pycypher.ast_models.SetAllPropertiesItem attribute), 96
 - variable (pycypher.ast_models.SetItem attribute), 97
 - variable (pycypher.ast_models.SetLabelsItem attribute), 97
 - variable (pycypher.ast_models.SetPropertyItem attribute), 97
 - variable (pycypher.ast_models.YieldItem attribute), 99
 - variable_name (pycypher.exceptions.VariableNotFoundError attribute), 355
 - variable_name (pycypher.exceptions.VariableTypeMismatchError attribute), 356
 - variable_name (pycypher.semantic_validator.ValidationError attribute), 294
 - variable_name() (pycypher.grammar_rule_mixins.LiteralRulesMixin method), 147
 - VariableLengthHopParams (class in pycypher.pattern_matcher), 173
 - VariableManager (class in pycypher.variable_manager), 270
 - VariableNotFoundError, 355
 - VariableScope (class in pycypher.semantic_validator), 294
 - VariableTypeMismatchError, 356
 - VectorizedPropertyStore (class in pycypher.graph_index), 264
 - version (pycypher.ingestion.config.PipelineConfig attribute), 313
 - version (shared.compat.APISurface attribute), 375
 - view_sql() (pycypher.ingestion.data_sources.CsvFormat method), 299
 - view_sql() (pycypher.ingestion.data_sources.Format method), 298
 - view_sql() (pycypher.ingestion.data_sources.JsonFormat method), 300
 - view_sql() (pycypher.ingestion.data_sources.ParquetFormat method), 299
 - violation_type (pycypher.exceptions.SecurityError attribute), 353
- ## W
- WARN (pycypher.ingestion.config.ErrorHandlingPolicy attribute), 305
 - WARNING (pycypher.ast_models.ValidationSeverity attribute), 88
 - WARNING (pycypher.semantic_validator.ErrorSeverity attribute), 293
 - warnings (pycypher.ast_models.ValidationResult property), 87
 - warnings (pycypher.query_complexity.ComplexityScore attribute), 242
 - warnings (pycypher.query_planner.QueryPlan attribute), 230, 231
 - when_clauses (pycypher.ast_models.CaseExpression attribute), 100
 - when_operands() (pycypher.grammar_rule_mixins.FunctionRulesMixin method), 144
 - WhenClause (class in pycypher.ast_models), 112
 - where (pycypher.ast_models.Call attribute), 89
 - where (pycypher.ast_models.ListComprehension attribute), 103, 104
 - where (pycypher.ast_models.Match attribute), 90
 - where (pycypher.ast_models.PatternComprehension attribute), 107, 108
 - where (pycypher.ast_models.Quantifier attribute), 108, 109
 - where (pycypher.ast_models.RelationshipPattern attribute), 94, 95
 - where (pycypher.ast_models.With attribute), 98

[where_clause\(\)](#) (pycypher.grammar_rule_mixins.ClauseRulesMixin method), [129](#)
[With](#) (class in pycypher.ast_models), [98](#)
[with_clause\(\)](#) (pycypher.grammar_rule_mixins.ClauseRulesMixin method), [130](#)
[with_to_binding_frame\(\)](#)
 (pycypher.projection_planner.ProjectionPlanner method), [269](#)
[Worker](#) (class in pycypher.cluster), [326](#)
[worker_count](#) (pycypher.cluster.ClusterCoordinator property), [332](#)
[worker_health](#) (pycypher.cluster.ClusterHealth attribute), [328](#)
[worker_id](#) (pycypher.cluster.LocalWorker property), [330](#)
[worker_id](#) (pycypher.cluster.Worker property), [326](#)
[worker_id](#) (pycypher.cluster.WorkerHealth attribute), [326](#), [327](#)
[worker_id](#) (pycypher.exceptions.WorkerExecutionError attribute), [357](#)
[WorkerExecutionError](#), [356](#)
[WorkerHealth](#) (class in pycypher.cluster), [326](#)
[WorkerStatus](#) (class in pycypher.cluster), [325](#)
[write_dataframe_to_uri\(\)](#) (in module pycypher.ingestion.output_writer), [318](#)
[write_nodes\(\)](#) (pycypher.sinks.neo4j.Neo4jSink method), [322](#)
[write_owner](#) (pycypher.result_cache.ReadWriteLock property), [283](#)
[write_relationships\(\)](#) (pycypher.sinks.neo4j.Neo4jSink method), [323](#)
[WrongCypherTypeError](#), [357](#)

X

[Xor](#) (class in pycypher.ast_models), [112](#)
[xor_expression\(\)](#) (pycypher.grammar_rule_mixins.ExpressionRulesMixin method), [138](#)

Y

[yield_clause\(\)](#) (pycypher.grammar_rule_mixins.ClauseRulesMixin method), [130](#)
[yield_item\(\)](#) (pycypher.grammar_rule_mixins.ClauseRulesMixin method), [130](#)
[yield_items](#) (pycypher.ast_models.Call attribute), [89](#)
[yield_items\(\)](#) (pycypher.grammar_rule_mixins.ClauseRulesMixin method), [130](#)
[YieldItem](#) (class in pycypher.ast_models), [98](#)